CAPÍTULO 1

Introducción

Un **sistema gestor de bases de datos** (SGBD) consiste en una colección de datos interrelacionados y un conjunto de programas para acceder a dichos datos. La colección de datos, normalmente denominada **base de datos**, contiene información relevante para una empresa. El objetivo principal de un SGBD es proporcionar una forma de almacenar y recuperar la información de una base de datos de manera que sea tanto *práctica* como *eficiente*.

Los sistemas de bases de datos se diseñan para gestionar grandes cantidades de información. La gestión de los datos implica tanto la definición de estructuras para almacenar la información como la provisión de mecanismos para la manipulación de la información. Además, los sistemas de bases de datos deben garantizar la fiabilidad de la información almacenada, a pesar de las caídas del sistema o de los intentos de acceso no autorizados. Si los datos van a ser compartidos entre diferentes usuarios, el sistema debe evitar posibles resultados anómalos.

Dado que la información es tan importante en la mayoría de las organizaciones, los científicos informáticos han desarrollado una gran cuerpo de conceptos y técnicas para la gestión de los datos. Estos conceptos y técnicas constituyen el objetivo central de este libro. En este capítulo se presenta una breve introducción a los principios de los sistemas de bases de datos.

1.1 Aplicaciones de los sistemas de bases de datos

Las bases de datos se usan ampliamente. Algunas de sus aplicaciones representativas son:

- *Banca*: para información de los clientes, cuentas, préstamos y transacciones bancarias.
- *Líneas aéreas*: para reservas e información de horarios. Las líneas aéreas fueron de las primeras en usar las bases de datos de forma distribuida geográficamente.
- *Universidades*: para información de los estudiantes, matrículas en las asignaturas y cursos.
- *Transacciones de tarjetas de crédito*: para compras con tarjeta de crédito y la generación de los extractos mensuales.
- *Telecomunicaciones*: para guardar un registro de las llamadas realizadas, generar las facturas mensuales, mantener el saldo de las tarjetas telefónicas de prepago y para almacenar información sobre las redes de comunicaciones.
- *Finanzas*: para almacenar información sobre compañías tenedoras, ventas y compras de productos financieros, como acciones y bonos; también para almacenar datos del mercado en tiempo real para permitir a los clientes la compraventa en línea y a la compañía la compraventa automática.
- *Ventas*: para información de clientes, productos y compras.

www.detodoprogramacion.com

- *Comercio en línea*: para los datos de ventas ya mencionados y para el seguimiento de los pedidos Web, generación de listas de recomendaciones y mantenimiento de evaluaciones de productos en línea.
- *Producción*: para la gestión de la cadena de proveedores y para el seguimiento de la producción de artículos en las factorías, inventarios en los almacenes y pedidos.
- *Recursos humanos*: para información sobre los empleados, salarios, impuestos sobre los sueldos y prestaciones sociales, y para la generación de las nóminas.

Como muestra esta lista, las bases de datos forman una parte esencial de casi todas las empresas actuales.

Durante las últimas cuatro décadas del siglo veinte, el uso de las bases de datos creció en todas las empresas. En los primeros días, muy pocas personas interactuaban directamente con los sistemas de bases de datos, aunque sin darse cuenta interactuaban indirectamente con bases de datos—con informes impresos como los extractos de las tarjetas de crédito, o mediante agentes como los cajeros de los bancos y los agentes de reservas de las líneas aéreas. Después vinieron los cajeros automáticos y permitieron a los usuarios interactuar directamente con las bases de datos. Las interfaces telefónicas con las computadoras (sistemas de respuesta vocal interactiva) también permitieron a los usuarios tratar directamente con las bases de datos—la persona que llamaba podía marcar un número y pulsar las teclas del teléfono para introducir información o para seleccionar opciones alternativas, para conocer las horas de llegada o salida de los vuelos, por ejemplo, o para matricularse de asignaturas en una universidad.

La revolución de Internet a finales de los años noventa aumentó significativamente el acceso directo del usuario a las bases de datos. Las organizaciones convirtieron muchas de sus interfaces telefónicas a las bases de datos en interfaces Web, y dejaron disponibles en línea muchos servicios. Por ejemplo, cuando se accede a una librería en línea y se busca en una colección de libros o de música, se está accediendo a datos almacenados en una base de datos. Cuando se realiza un pedido en línea, el pedido se almacena en una base de datos. Cuando se accede al sitio Web de un banco y se consultan el estado de la cuenta y los movimientos, la información se recupera del sistema de bases de datos del banco. Cuando se accede a un sitio Web, puede que se recupere información personal de una base de datos para seleccionar los anuncios que se deben mostrar. Más aún, los datos sobre los accesos Web pueden almacenarse en una base de datos

Así, aunque las interfaces de usuario ocultan los detalles del acceso a las bases de datos, y la mayoría de la gente ni siquiera es consciente de que están interactuando con una base de datos, el acceso a las bases de datos forma actualmente una parte esencial de la vida de casi todas las personas.

La importancia de los sistemas de bases de datos se puede juzgar de otra forma—actualmente, los fabricantes de sistemas de bases de datos como Oracle están entre las mayores compañías de software del mundo, y los sistemas de bases de datos forman una parte importante de la línea de productos de compañías más diversificadas como Microsoft e IBM.

1.2 Propósito de los sistemas de bases de datos

Los sistemas de bases de datos surgieron en respuesta a los primeros métodos de gestión informatizada de los datos comerciales. A modo de ejemplo de dichos métodos, típicos de los años sesenta, considérese parte de una entidad bancaria que, entre otros datos, guarda información sobre todos los clientes y todas las cuentas de ahorro. Una manera de guardar la información en la computadora es almacenarla en archivos del sistema operativo. Para permitir que los usuarios manipulen la información, el sistema tiene varios programas de aplicación que gestionan los archivos, incluyendo programas para:

- Efectuar cargos o abonos en las cuentas.
- Añadir cuentas nuevas.
- Calcular el saldo de las cuentas.
- Generar los extractos mensuales.

www.detodoprogramacion.com

Estos programas de aplicación los han escrito programadores de sistemas en respuesta a las necesidades del banco.

Se añaden nuevos programas de aplicación al sistema según surgen las necesidades. Por ejemplo, supóngase que una caja de ahorros decide ofrecer cuentas corrientes. En consecuencia, se crean nuevos archivos permanentes que contienen información acerca de todas las cuentas corrientes abiertas en el banco y puede que haya que escribir nuevos programas de aplicación para afrontar situaciones que no se dan en las cuentas de ahorro, como los descubiertos. Así, con el paso del tiempo, se añaden más archivos y programas de aplicación al sistema.

Los sistemas operativos convencionales soportan este **sistema de procesamiento de archivos** típico. El sistema almacena los registros permanentes en varios archivos y necesita diferentes programas de aplicación para extraer y añadir a los archivos correspondientes. Antes de la aparición de los sistemas gestores de bases de datos (SGBDs), las organizaciones normalmente almacenaban la información en sistemas de este tipo.

Guardar la información de la organización en un sistema de procesamiento de archivos tiene una serie de inconvenientes importantes:

- **Redundancia e inconsistencia de los datos**. Debido a que los archivos y programas de aplicación los crean diferentes programadores en el transcurso de un largo período de tiempo, es probable que los diversos archivos tengan estructuras diferentes y que los programas estén escritos en varios lenguajes de programación diferentes. Además, puede que la información esté duplicada en varios lugares (archivos). Por ejemplo, la dirección y el número de teléfono de un cliente dado pueden aparecer en un archivo que contenga registros de cuentas de ahorros y en un archivo que contenga registros de cuentas corrientes. Esta redundancia conduce a costes de almacenamiento y de acceso más elevados. Además, puede dar lugar a la **inconsistencia de los datos**; es decir, puede que las diferentes copias de los mismos datos no coincidan. Por ejemplo, puede que el cambio en la dirección de un cliente esté reflejado en los registros de las cuentas de ahorro pero no en el resto del sistema.

- **Dificultad en el acceso a los datos**. Supóngase que uno de los empleados del banco necesita averiguar los nombres de todos los clientes que viven en un código postal dado. El empleado pide al departamento de procesamiento de datos que genere esa lista. Debido a que esta petición no fue prevista por los diseñadores del sistema original, no hay un programa de aplicación a mano para satisfacerla. Hay, sin embargo, un programa de aplicación que genera la lista de todos los clientes. El empleado del banco tiene ahora dos opciones: bien obtener la lista de *todos* los clientes y extraer manualmente la información que necesita, o bien pedir a un programador de sistemas que escriba el programa de aplicación necesario. Ambas alternativas son obviamente insatisfactorias. Supóngase que se escribe el programa y que, varios días más tarde, el mismo empleado necesita reducir esa lista para que incluya únicamente a aquellos clientes que tengan una cuenta con saldo igual o superior a 10.000 €. Como se puede esperar, no existe ningún programa que genere tal lista. De nuevo, el empleado tiene que elegir entre dos opciones, ninguna de las cuales es satisfactoria.

  La cuestión aquí es que los entornos de procesamiento de archivos convencionales no permiten recuperar los datos necesarios de una forma práctica y eficiente. Hacen falta sistemas de recuperación de datos más adecuados para el uso general.

- **Aislamiento de datos.** Como los datos están dispersos en varios archivos, y los archivos pueden estar en diferentes formatos, es difícil escribir nuevos programas de aplicación para recuperar los datos correspondientes.

- **Problemas de integridad.** Los valores de los datos almacenados en la base de datos deben satisfacer ciertos tipos de **restricciones de consistencia**. Por ejemplo, el saldo de ciertos tipos de cuentas bancarias no puede nunca ser inferior a una cantidad predeterminada (por ejemplo, 25 €). Los desarrolladores hacen cumplir esas restricciones en el sistema añadiendo el código correspondiente en los diversos programas de aplicación. Sin embargo, cuando se añaden nuevas restricciones, es difícil cambiar los programas para hacer que se cumplan. El problema se complica cuando las restricciones implican diferentes elementos de datos de diferentes archivos.

www.detodoprogramacion.com

- **Problemas de atomicidad.** Los sistemas informáticos, como cualquier otro dispositivo mecánico o eléctrico, está sujeto a fallos. En muchas aplicaciones es crucial asegurar que, si se produce algún fallo, los datos se restauren al estado consistente que existía antes del fallo. Considérese un programa para transferir 50 € desde la cuenta *A* a la *B*. Si se produce un fallo del sistema durante la ejecución del programa, es posible que los 50 € fueran retirados de la cuenta *A* pero no abonados en la cuenta *B*, dando lugar a un estado inconsistente de la base de datos. Evidentemente, resulta esencial para la consistencia de la base de datos que tengan lugar tanto el abono como el cargo, o que no tenga lugar ninguno. Es decir, la transferencia de fondos debe ser *atómica*—debe ocurrir en su totalidad o no ocurrir en absoluto. Resulta difícil asegurar la atomicidad en los sistemas convencionales de procesamiento de archivos.

- **Anomalías en el acceso concurrente**. Para aumentar el rendimiento global del sistema y obtener una respuesta más rápida, muchos sistemas permiten que varios usuarios actualicen los datos simultáneamente. En realidad, hoy en día, los principales sitios de comercio electrónico de Internet pueden tener millones de accesos diarios de compradores a sus datos. En tales entornos es posible la interacción de actualizaciones concurrentes y puede dar lugar a datos inconsistentes. Considérese una cuenta bancaria *A*, que contenga 500 €. Si dos clientes retiran fondos (por ejemplo, 50 € y 100 €, respectivamente) de la cuenta *A* aproximadamente al mismo tiempo, el resultado de las ejecuciones concurrentes puede dejar la cuenta en un estado incorrecto (o inconsistente). Supóngase que los programas que se ejecutan para cada retirada leen el saldo anterior, reducen su valor en el importe que se retira y luego escriben el resultado. Si los dos programas se ejecutan concurrentemente, pueden leer el valor 500 €, y escribir después 450 € y 400 €, respectivamente. Dependiendo de cuál escriba el valor en último lugar, la cuenta puede contener 450 € o 400 €, en lugar del valor correcto, 350 €. Para protegerse contra esta posibilidad, el sistema debe mantener alguna forma de supervisión. Pero es difícil ofrecer supervisión, ya que muchos programas de aplicación diferentes que no se han coordinado con anterioridad pueden tener acceso a los datos.

- **Problemas de seguridad**. No todos los usuarios de un sistema de bases de datos deben poder acceder a todos los datos. Por ejemplo, en un sistema bancario, el personal de nóminas sólo necesita ver la parte de la base de datos que contiene información acerca de los diferentes empleados del banco. No necesitan tener acceso a la información acerca de las cuentas de clientes. Pero, como los programas de aplicación se añaden al sistema de procesamiento de datos de una forma ad hoc, es difícil hacer cumplir tales restricciones de seguridad.

Estas dificultades, entre otras, motivaron el desarrollo de los sistemas de bases de datos. En el resto del libro se examinarán los conceptos y los algoritmos que permiten que los sistemas de bases de datos resuelvan los problemas de los sistemas de procesamiento de archivos. En general, en este libro se usa una entidad bancaria como ejemplo de aplicación típica de procesamiento de datos que puede encontrarse en una empresa.

1.3 Visión de los datos

Un sistema de bases de datos es una colección de datos interrelacionados y un conjunto de programas que permiten a los usuarios tener acceso a esos datos y modificarlos. Una de las principales finalidades de los sistemas de bases de datos es ofrecer a los usuarios una visión *abstracta* de los datos. Es decir, el sistema oculta ciertos detalles del modo en que se almacenan y mantienen los datos.

1.3.1 Abstracción de datos

Para que el sistema sea útil debe recuperar los datos eficientemente. La necesidad de eficiencia ha llevado a los diseñadores a usar estructuras de datos complejas para la representación de los datos en la base de datos. Dado que muchos de los usuarios de sistemas de bases de datos no tienen formación en informática, los desarrolladores ocultan esa complejidad a los usuarios mediante varios niveles de abstracción para simplificar la interacción de los usuarios con el sistema:

www.detodoprogramacion.com

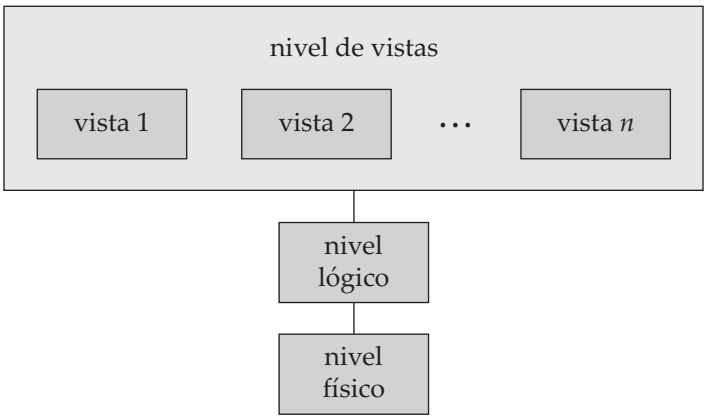


Figura 1.1 Los tres niveles de abstracción de datos.

- **Nivel físico**. El nivel más bajo de abstracción describe *cómo* se almacenan realmente los datos. El nivel físico describe en detalle las estructuras de datos complejas de bajo nivel.
- **Nivel lógico**. El nivel inmediatamente superior de abstracción describe *qué* datos se almacenan en la base de datos y qué relaciones existen entre esos datos. El nivel lógico, por tanto, describe toda la base de datos en términos de un número pequeño de estructuras relativamente simples. Aunque la implementación de esas estructuras simples en el nivel lógico puede involucrar estructuras complejas del nivel físico, los usuarios del nivel lógico no necesitan preocuparse de esta complejidad. Los administradores de bases de datos, que deben decidir la información que se guarda en la base de datos, usan el nivel de abstracción lógico.
- **Nivel de vistas**. El nivel más elevado de abstracción sólo describe parte de la base de datos. Aunque el nivel lógico usa estructuras más simples, queda algo de complejidad debido a la variedad de información almacenada en las grandes bases de datos. Muchos usuarios del sistema de bases de datos no necesitan toda esta información; en su lugar sólo necesitan tener acceso a una parte de la base de datos. El nivel de abstracción de vistas existe para simplificar su interacción con el sistema. El sistema puede proporcionar muchas vistas para la misma base de datos.

La Figura 1.1 muestra la relación entre los tres niveles de abstracción.

Una analogía con el concepto de tipos de datos en lenguajes de programación puede clarificar la diferencia entre los niveles de abstracción. La mayoría de los lenguajes de programación de alto nivel soportan el concepto de tipo estructurado. Por ejemplo, en los lenguajes tipo Pascal se pueden declarar registros de la manera siguiente:

```
type cliente = record
                 id_cliente : string;
                 nombre_cliente : string;
                 calle_cliente : string;
                 ciudad_cliente : string;
               end;
```

Este código define un nuevo tipo de registro denominado *cliente* con cuatro campos. Cada campo tiene un nombre y un tipo asociados. Una entidad bancaria puede tener varios tipos de estos registros, incluidos:

- *cuenta*, con los campos *número_cuenta* y *saldo*.
- *empleado*, con los campos *nombre_empleado* y *sueldo*.

En el nivel físico, los registros *cliente*, *cuenta* o *empleado* se pueden describir como bloques de posiciones consecutivas de almacenamiento (por ejemplo, palabras o bytes). El compilador oculta este nivel

www.detodoprogramacion.com

de detalle a los programadores. De manera parecida, el sistema de base de datos oculta muchos de los detalles de almacenamiento de los niveles inferiores a los programadores de bases de datos. Los administradores de bases de datos, por otro lado, pueden ser conscientes de ciertos detalles de la organización física de los datos.

En el nivel lógico cada registro de este tipo se describe mediante una definición de tipo, como en el fragmento de código anterior, y también se define la relación entre estos tipos de registros. Los programadores que usan un lenguaje de programación trabajan en este nivel de abstracción. De manera parecida, los administradores de bases de datos suelen trabajar en este nivel de abstracción.

Finalmente, en el nivel de vistas, los usuarios de computadoras ven un conjunto de programas de aplicación que ocultan los detalles de los tipos de datos. De manera parecida, en el nivel de vistas se definen varias vistas de la base de datos y los usuarios de la base de datos pueden verlas. Además de ocultar los detalles del nivel lógico de la base de datos, las vistas también proporcionan un mecanismo de seguridad para evitar que los usuarios tengan acceso a ciertas partes de la base de datos. Por ejemplo, los cajeros de un banco sólo ven la parte de la base de datos que contiene información de las cuentas de los clientes; no pueden tener acceso a la información referente a los sueldos de los empleados.

1.3.2 Ejemplares y esquemas

Las bases de datos van cambiando a lo largo del tiempo conforme la información se inserta y se elimina. La colección de información almacenada en la base de datos en un momento dado se denomina **ejemplar** de la base de datos. El diseño general de la base de datos se denomina **esquema** de la base de datos. Los esquemas se modifican rara vez, si es que se modifican.

El concepto de esquemas y ejemplares de las bases de datos se puede comprender por analogía con los programas escritos en un lenguaje de programación. El esquema de la base de datos se corresponde con las declaraciones de las variables (junto con las definiciones de tipos asociadas) de los programas. Cada variable tiene un valor concreto en un instante dado. Los valores de las variables de un programa en un instante dado se corresponden con un *ejemplar* del esquema de la base de datos.

Los sistemas de bases de datos tiene varios esquemas divididos según los niveles de abstracción. El **esquema físico** describe el diseño de la base de datos en el nivel físico, mientras que el **esquema lógico** describe su diseño en el nivel lógico. Las bases de datos también pueden tener varios esquemas en el nivel de vistas, a veces denominados **subesquemas**, que describen diferentes vistas de la base de datos.

De éstos, el esquema lógico es con mucho el más importante en términos de su efecto sobre los programas de aplicación, ya que los programadores crean las aplicaciones usando el esquema lógico. El esquema físico está oculto bajo el esquema lógico, y generalmente puede modificarse fácilmente sin afectar a los programas de aplicación. Se dice que los programas de aplicación muestran **independencia!física respecto de los datos** si no dependen del esquema físico y, por tanto, no hace falta volver a escribirlos si se modifica el esquema físico.

Se estudiarán los lenguajes para la descripción de los esquemas, después de introducir el concepto de modelos de datos en el apartado siguiente.

1.3.3 Modelos de datos

Bajo la estructura de las bases de datos se encuentra el **modelo de datos**: una colección de herramientas conceptuales para describir los datos, sus relaciones, su semántica y las restricciones de consistencia. Los modelos de datos ofrecen un modo de describir el diseño de las bases de datos en los niveles físico, lógico y de vistas.

En este texto se van a tratar varios modelos de datos diferentes. Los modelos de datos pueden clasificarse en cuatro categorías diferentes:

- **Modelo relacional**. El modelo relacional usa una colección de tablas para representar tanto los datos como sus relaciones. Cada tabla tiene varias columnas, y cada columna tiene un nombre único. El modelo relacional es un ejemplo de un modelo basado en registros. Los modelos basados en registros se denominan así porque la base de datos se estructura en registros de formato fijo de varios tipos. Cada tabla contiene registros de un tipo dado. Cada tipo de registro define un número fijo de campos, o atributos. Las columnas de la tabla se corresponden con los atributos

www.detodoprogramacion.com

del tipo de registro. El modelo de datos relacional es el modelo de datos más ampliamente usado, y una gran mayoría de sistemas de bases de datos actuales se basan en el modelo relacional. Los Capítulos 2 al 7 tratan el modelo relacional en detalle.

- **El modelo entidad-relación**. El modelo de datos entidad-relación (E-R) se basa en una percepción del mundo real que consiste en una colección de objetos básicos, denominados *entidades*, y de las *relaciones* entre ellos. Una entidad es una "cosa" u "objeto" del mundo real que es distinguible de otros objetos. El modelo entidad-relación se usa mucho en el diseño de bases de datos y en el Capítulo 6 se examina detalladamente.
- **Modelo de datos orientado a objetos**. El modelo de datos orientado a objetos es otro modelo de datos que está recibiendo una atención creciente. El modelo orientado a objetos se puede considerar como una extensión del modelo E-R con los conceptos de la encapsulación, los métodos (funciones) y la identidad de los objetos. En el Capítulo 9 se examina este modelo de datos.
- **Modelo de datos semiestructurados**. El modelo de datos semiestructurados permite la especificación de datos donde los elementos de datos individuales del mismo tipo pueden tener diferentes conjuntos de atributos. Esto lo diferencia de los modelos de datos mencionados anteriormente, en los que cada elemento de datos de un tipo particular debe tener el mismo conjunto de atributos. El **lenguaje de marcas extensible** (XML, eXtensible Markup Language) se emplea mucho para representar datos semiestructurados. Se estudia en el Capítulo 10.

El **modelo de datos de red** y el **modelo de datos jerárquico** precedieron cronológicamente al relacional. Estos modelos estuvieron íntimamente ligados a la implementación subyacente y complicaban la tarea del modelado de datos. En consecuencia, se usan muy poco hoy en día, excepto en el código de bases de datos antiguas que sigue estando en servicio en algunos lugares. Se describen brevemente en los Apéndices A y B para los lectores interesados.

1.4 Lenguajes de bases de datos

Los sistemas de bases de datos proporcionan un **lenguaje de definición de datos** para especificar el esquema de la base de datos y un **lenguaje de manipulación de datos** para expresar las consultas y las modificaciones de la base de datos. En la práctica, los lenguajes de definición y manipulación de datos no son dos lenguajes diferentes; en cambio, simplemente forman parte de un único lenguaje de bases de datos, como puede ser el muy usado SQL.

1.4.1 Lenguaje de manipulación de datos

Un **lenguaje de manipulación de datos** (LMD) es un lenguaje que permite a los usuarios tener acceso a los datos organizados mediante el modelo de datos correspondiente o manipularlos. Los tipos de acceso son:

- La recuperación de la información almacenada en la base de datos.
- La inserción de información nueva en la base de datos.
- El borrado de la información de la base de datos.
- La modificación de la información almacenada en la base de datos.

Hay fundamentalmente dos tipos:

- Los **LMDs procedimentales** necesitan que el usuario especifique *qué* datos se necesitan y *cómo* obtener esos datos.
- Los **LMDs declarativos** (también conocidos como **LMDs no procedimentales**) necesitan que el usuario especifique *qué* datos se necesitan *sin* que haga falta que especifique cómo obtener esos datos.

www.detodoprogramacion.com

Los LMDs declarativos suelen resultar más fáciles de aprender y de usar que los procedimentales. Sin embargo, como el usuario no tiene que especificar cómo conseguir los datos, el sistema de bases de datos tiene que determinar un medio eficiente de acceso a los datos.

Una **consulta** es una instrucción que solicita que se recupere información. La parte de los LMDs implicada en la recuperación de información se denomina **lenguaje de consultas**. Aunque técnicamente sea incorrecto, resulta habitual usar las expresiones *lenguaje de consultas* y *lenguaje de manipulación de datos* como sinónimas.

Existen varios lenguajes de consultas de bases de datos en uso, tanto comercial como experimentalmente. El lenguaje de consultas más ampliamente usado, SQL, se estudiará en los Capítulos 3 y 4. También se estudiarán otros lenguajes de consultas en el Capítulo 5.

Los niveles de abstracción que se trataron en el Apartado 1.3 no sólo se aplican a la definición o estructuración de datos, sino también a su manipulación. En el nivel físico se deben definir los algoritmos que permitan un acceso eficiente a los datos. En los niveles superiores de abstracción se pone el énfasis en la facilidad de uso. El objetivo es permitir que los seres humanos interactúen de manera eficiente con el sistema. El componente procesador de consultas del sistema de bases de datos (que se estudia en los Capítulos 13 y 14) traduce las consultas LMD en secuencias de acciones en el nivel físico del sistema de bases de datos.

1.4.2 Lenguaje de definición de datos

Los esquemas de las bases de datos se especifican mediante un conjunto de definiciones expresadas mediante un lenguaje especial denominado **lenguaje de definición de datos (LDD)**. El LDD también se usa para especificar más propiedades de los datos.

La estructura de almacenamiento y los métodos de acceso usados por el sistema de bases de datos se especifican mediante un conjunto de instrucciones en un tipo especial de LDD denominado **lenguaje de almacenamiento y definición de datos**. Estas instrucciones definen los detalles de implementación de los esquemas de las bases de datos, que suelen ocultarse a los usuarios.

Los valores de los datos almacenados en la base de datos deben satisfacer ciertas **restricciones de consistencia**. Por ejemplo, supóngase que el saldo de una cuenta no debe caer por debajo de 100 €. El LDD proporciona facilidades para especificar tales restricciones. Los sistemas de bases de datos las comprueban cada vez que se modifica la base de datos. En general, las restricciones pueden ser predicados arbitrarios relativos a la base de datos. No obstante, los predicados arbitrarios pueden resultar costosos de comprobar. Por tanto, los sistemas de bases de datos se concentran en las restricciones de integridad que pueden comprobarse con una sobrecarga mínima:

- **Restricciones de dominio.** Se debe asociar un dominio de valores posibles a cada atributo (por ejemplo, tipos enteros, tipos de carácter, tipos fecha/hora). La declaración de un atributo como parte de un dominio concreto actúa como restricción de los valores que puede adoptar. Las restricciones de dominio son la forma más elemental de restricción de integridad. El sistema las comprueba fácilmente siempre que se introduce un nuevo elemento de datos en la base de datos.
- **Integridad referencial.** Hay casos en los que se desea asegurar que un valor que aparece en una relación para un conjunto de atributos dado aparece también para un determinado conjunto de atributos en otra relación (integridad referencial). Las modificaciones de la base de datos pueden causar violaciones de la integridad referencial. Cuando se viola una restricción de integridad, el procedimiento normal es rechazar la acción que ha causado esa violación.
- **Asertos.** Un aserto es cualquier condición que la base de datos debe satisfacer siempre. Las restricciones de dominio y las restricciones de integridad referencial son formas especiales de asertos. No obstante, hay muchas restricciones que no pueden expresarse empleando únicamente esas formas especiales. Por ejemplo: "Cada préstamo tiene como mínimo un cliente tenedor de una cuenta con un saldo mínimo de 1.000,00 €" debe expresarse en forma de aserto. Cuando se crea un aserto, el sistema comprueba su validez. Si el aserto es válido, cualquier modificación futura de la base de datos se permite únicamente si no hace que se viole ese aserto.
- **Autorización.** Puede que se desee diferenciar entre los usuarios en cuanto al tipo de acceso que se les permite a diferentes valores de los datos de la base de datos. Estas diferenciaciones se

www.detodoprogramacion.com

expresan en términos de **autorización**, cuyas modalidades más frecuentes son: **autorización de lectura**, que permite la lectura pero no la modificación de los datos; **autorización de inserción**, que permite la inserción de datos nuevos, pero no la modificación de los datos ya existentes; **autorización de actualización**, que permite la modificación, pero no la eliminación, de los datos; y la **autorización de eliminación**, que permite la eliminación de datos. A cada usuario se le pueden asignar todos, ninguno o una combinación de estos tipos de autorización.

El LDD, al igual que cualquier otro lenguaje de programación, obtiene como entrada algunas instrucciones y genera una salida. La salida del LDD se coloca en el **diccionario de datos**, que contiene **metadatos**—es decir, datos sobre datos. El diccionario de datos se considera un tipo especial de tabla, a la que sólo puede tener acceso y actualizar el propio sistema de bases de datos (no los usuarios normales). El sistema de bases de datos consulta el diccionario de datos antes de leer o modificar los datos reales.

1.5 Bases de datos relacionales

Las bases de datos relacionales se basan en el modelo relacional y usan un conjunto de tablas para representar tanto los datos como las relaciones entre ellos. También incluyen un LMD y un LDD. La mayor parte de los sistemas de bases de datos relacionales comerciales emplean el lenguaje SQL, que se trata en este apartado y que se tratará con gran detalle en los Capítulos 3 y 4. En el Capítulo 5 se estudiarán otros lenguajes influyentes.

1.5.1 Tablas

Cada tabla tiene varias columnas, y cada columna tiene un nombre único. En la Figura 1.2 se presenta un ejemplo de base de datos relacional consistente en tres tablas: una muestra detalles de los clientes de un banco, la segunda muestra las cuentas y la tercera muestra las cuentas que pertenecen a cada cliente.

La primera tabla, la tabla *cliente*, muestra, por ejemplo, que el cliente identificado por *id_cliente* 19.283.746 se llama González y vive en la calle Arenal en La Granja. La segunda tabla, *cuenta*, muestra, por ejemplo, que la cuenta C-101 tiene un saldo de 500 € y la C-201 un saldo de 900 €.

La tercera tabla muestra las cuentas que pertenecen a cada cliente. Por ejemplo, la cuenta C-101 pertenece al cliente cuyo *id_cliente* es 19.283.746 (González), y los clientes 19.283.746 (González) y 01.928.374 (Gómez) comparten el número de cuenta C-201 (pueden compartir un negocio).

El modelo relacional es un ejemplo de modelo basado en registros. Los modelos basados en registros se denominan así porque la base de datos se estructura en registros de formato fijo de varios tipos. Cada tabla contiene registros de un tipo dado. Cada tipo de registro define un número fijo de campos, o atributos. Las columnas de la tabla se corresponden con los atributos del tipo de registro.

No es difícil ver cómo se pueden almacenar las tablas en archivos. Por ejemplo, se puede usar un carácter especial (como la coma) para delimitar los diferentes atributos de un registro, y otro carácter especial (como el carácter de nueva línea) para delimitar los registros. El modelo relacional oculta esos detalles de implementación de bajo nivel a los desarrolladores de bases de datos y a los usuarios.

El modelo de datos relacional es el modelo de datos más ampliamente usado, y una gran mayoría de los sistemas de bases de datos actuales se basan en el modelo relacional. Los Capítulos 2 al 7 tratan el modelo relacional con detalle.

Obsérvese también que en el modelo relacional es posible crear esquemas que tengan problemas tales como información duplicada innecesariamente. Por ejemplo, supóngase que se almacena *número_cuenta* como atributo de un registro *cliente*. Entonces, para representar el hecho de que tanto la cuentas C-101 como la cuenta C-201 pertenece al cliente González (con *id_cliente* 19.283.746) sería necesario almacenar dos filas en la tabla *cliente*. Los valores de *nombre_cliente*, *calle_cliente* y *ciudad_cliente* de González estarían innecesariamente duplicados en las dos filas. En el Capítulo 7 se estudiará el modo de distinguir los buenos diseños de esquema de los malos.

www.detodoprogramacion.com

id_cliente	*nombre_cliente*	*calle_cliente*	*ciudad_cliente*
19.283.746	González	Arenal, 12	La Granja
67.789.901	López	Mayor, 3	Peguerinos
18.273.609	Abril	Preciados, 123	Valsaín
32.112.312	Santos	Mayor, 100	Peguerinos
33.666.999	Rupérez	Ramblas, 175	León
01.928.374	Gómez	Carretas, 72	Cerceda

(a) La tabla *cliente*

número_cuenta	*saldo*
C-101	500
C-215	700
C-102	400
C-305	350
C-201	900
C-217	750
C-222	700

(b) La tabla *cuenta*

id_cliente	*número_cuenta*
19.283.746	C-101
19.283.746	C-201
01.928.374	C-215
67.789.901	C-102
18.273.609	C-305
32.112.312	C-217
33.666.999	C-222
01.928.374	C-201

(c) La tabla *impositor*

Figura 1.2 Un ejemplo de base de datos relacional.

1.5.2 Lenguaje de manipulación de datos

El lenguaje de consultas de SQL no es procedimental. Usa como entrada varias tablas (posiblemente sólo una) y devuelve siempre una sola tabla. A continuación se ofrece un ejemplo de consulta SQL que halla el nombre de todos los clientes que residen en Peguerinos:

select *cliente.nombre_cliente*
from *cliente*
where *cliente.ciudad_cliente* = 'Peguerinos'

La consulta especifica que hay que recuperar (*select*) las filas de (*from*) la tabla *cliente* en las que (*where*) la *ciudad_cliente* es Peguerinos, y que sólo debe mostrarse el atributo *nombre_cliente* de esas filas. Más concretamente, el resultado de la ejecución de esta consulta es una tabla con una sola columna denominada *nombre_cliente* y un conjunto de filas, cada una de las cuales contiene el nombre de un cliente cuya *ciudad_cliente* es Peguerinos. Si la consulta se ejecuta sobre la tabla de la Figura 1.2, el resultado constará de dos filas, una con el nombre López y otra con el nombre Santos.

Las consultas pueden involucrar información de más de una tabla. Por ejemplo, la siguiente consulta busca todos los números de cuenta y sus saldos del cliente con *id_cliente* 19.283.746.

www.detodoprogramacion.com

```
select cuenta.número_cuenta, cuenta.saldo
from impositor, cuenta
where impositor.id_cliente = '19.283.746' and
      impositor.número_cuenta = cuenta.número_cuenta
```

Si la consulta anterior se ejecutase sobre las tablas de la Figura 1.2, el sistema encontraría que las dos cuentas denominadas C-101 y C-201 pertenecen al cliente 19.283.746 y el resultado consistiría en una tabla con dos columnas (*número_cuenta, saldo*) y dos filas (C-101, 500) y (C-201,900).

1.5.3 Lenguaje de definición de datos

SQL ofrece un LDD elaborado que permite definir tablas, restricciones de integridad, asertos, etc.

Por ejemplo, la siguiente instrucción del lenguaje SQL define la tabla *cuenta*:

```
create table cuenta
      (número_cuenta char(10),
       saldo integer)
```

La ejecución de esta instrucción LDD crea la tabla *cuenta.* Además, actualiza el diccionario de datos, que contiene metadatos (1.4.2). Los esquemas de las tablas son ejemplos de metadatos.

1.5.4 Acceso a las bases de datos desde los programas de aplicación

SQL no es tan potente como la máquina universal de Turing; es decir, hay algunos cálculos que no pueden obtenerse mediante ninguna consulta SQL. Esos cálculos deben escribirse en un lenguaje *anfitrión*, como Cobol, C, C++ o Java. Los **programas de aplicación** son programas que se usan para interactuar de esta manera con las bases de datos. Algunos de los ejemplos de un sistema bancario serían los programas que generan las nóminas, realizan cargos en las cuentas, realizan abonos en las cuentas o transfieren fondos entre las cuentas.

Para tener acceso a la base de datos, las instrucciones LMD deben ejecutarse desde el lenguaje anfitrión. Hay dos maneras de conseguirlo:

- Proporcionando una interfaz de programas de aplicación (conjunto de procedimientos) que se pueda usar para enviar instrucciones LMD y LDD a la base de datos y recuperar los resultados.

  El estándar de conectividad abierta de bases de datos (ODBC, Open Data Base Connectivity) definido por Microsoft para su empleo con el lenguaje C es un estándar de interfaz de programas de aplicación usado habitualmente. El estándar de conectividad de Java con bases de datos (JDBC, Java Data Base Connectivity) ofrece las características correspondientes para el lenguaje Java.

- Extendiendo la sintaxis del lenguaje anfitrión para que incorpore las llamadas LMD dentro del programa del lenguaje anfitrión. Generalmente, un carácter especial precede a las llamadas LMD y un preprocesador, denominado **precompilador LMD**, convierte las instrucciones LMD en llamadas normales a procedimientos en el lenguaje anfitrión.

1.6 Diseño de bases de datos

Los sistemas de bases de datos se diseñan para gestionar grandes cantidades de información. Esas grandes cantidades de información no existen aisladas. Forman parte del funcionamiento de alguna empresa, cuyo producto final puede que sea la información obtenida de la base de datos o algún dispositivo o servicio para el que la base de datos sólo desempeña un papel secundario.

El diseño de bases de datos implica principalmente el diseño del esquema de las bases de datos. El diseño de un entorno completo de aplicaciones para la base de datos que satisfaga las necesidades de la empresa que se está modelando exige prestar atención a un conjunto de aspectos más amplio. Este texto se centrará inicialmente en la escritura de las consultas a la base de datos y en el diseño de los esquemas de las bases de datos. En el Capítulo 8 se estudia el proceso general de diseño de las aplicaciones.

www.detodoprogramacion.com

1.6.1 Proceso de diseño

Los modelos de datos de alto nivel resultan útiles a los diseñadores de bases de datos al ofrecerles un marco conceptual en el que especificar, de manera sistemática, los requisitos de datos de los usuarios de las bases de datos y la manera en que se estructurará la base de datos para satisfacer esos requisitos. La fase inicial del diseño de las bases de datos, por tanto, es caracterizar completamente los requisitos de datos de los hipotéticos usuarios de la base de datos. Los diseñadores de bases de datos deben interactuar ampliamente con los expertos y usuarios del dominio para llevar a cabo esta tarea. El resultado de esta fase es la especificación de los requisitos de los usuarios.

A continuación, el diseñador escoge un modelo de datos y, mediante la aplicación de los conceptos del modelo de datos elegido, traduce esos requisitos en un esquema conceptual de la base de datos. El esquema desarrollado en esta fase de **diseño conceptual** ofrece una visión general detallada de la empresa. El diseñador revisa el esquema para confirmar que todos los requisitos de datos se satisfacen realmente y no entran en conflicto entre sí. El diseñador también puede examinar el diseño para eliminar cualquier característica redundante. En este punto, la atención se centra en describir los datos y sus relaciones, más que en especificar los detalles del almacenamiento físico.

En términos del modelo relacional, el proceso de diseño conceptual implica decisiones sobre *qué* atributos se desea capturar en la base de datos y *cómo agruparlos* para formar las diferentes tablas. La parte "qué" es, esencialmente, una decisión conceptual, y no se seguirá estudiando en este texto. La parte del "cómo" es, esencialmente, un problema informático. Hay dos vías principales para afrontar el problema. La primera supone usar el modelo entidad-relación (Apartado 1.6.3); la otra es emplear un conjunto de algoritmos (denominados colectivamente como *normalización*) que toma como entrada el conjunto de todos los atributos y genera un conjunto de tablas (Apartado 1.6.4).

Un esquema conceptual completamente desarrollado también indica los requisitos funcionales de la empresa. En la **especificación de requisitos funcionales** los usuarios describen el tipo de operaciones (o transacciones) que se llevarán a cabo con los datos. Un ejemplo de estas operaciones es modificar o actualizar los datos, buscar y recuperar datos concretos y eliminar datos. En esta etapa del diseño conceptual el diseñador puede revisar el esquema para asegurarse de que satisface los requisitos funcionales.

El proceso de pasar de un modelo de datos abstracto a la implementación de la base de datos continúa con dos fases de diseño finales. En la **fase de diseño lógico** el diseñador relaciona el esquema conceptual de alto nivel con el modelo de implementación de datos del sistema de bases de datos que se va a usar. El diseñador usa el esquema de bases de datos específico para el sistema resultante en la **fase de diseño físico** posterior, en la que se especifican las características físicas de la base de datos. Entre esas características están la forma de organización de los archivos y las estructuras de almacenamiento interno; se estudian en el Capítulo 11.

1.6.2 Diseño de la base de datos para una entidad bancaria

Para ilustrar el proceso de diseño, examinemos el modo en que puede diseñarse una base de datos para una entidad bancaria. La especificación inicial de los requisitos de los usuarios puede basarse en entrevistas con los usuarios de la base de datos y en el análisis de la entidad realizado por el propio diseñador. La descripción que surge de esta fase de diseño sirve como base para especificar la estructura conceptual de la base de datos. Éstas son las principales características de la entidad bancaria:

- El banco está organizado en sucursales. Cada sucursal se ubica en una ciudad determinada y queda identificada por un nombre único. El banco supervisa los activos de cada sucursal.
- Los clientes quedan identificados por el valor de su *id_cliente.* El banco almacena el nombre de cada cliente y la calle y la ciudad en la que vive. Los clientes pueden abrir cuentas y solicitar préstamos. Cada cliente puede asociarse con un empleado del banco en concreto, que puede actuar como prestamista o como asesor personal para ese cliente.
- El banco ofrece dos tipos de cuenta: de ahorro y corriente. Las cuentas pueden tener como titular a más de un cliente, y cada cliente puede abrir más de una cuenta. A cada cuenta se le asigna un número de cuenta único. El banco guarda un registro del saldo de cada cuenta y de la fecha

www.detodoprogramacion.com

más reciente en que cada cliente titular de esa cuenta tuvo acceso a ella. Además, cada cuenta de ahorro tiene una tasa de interés y se registran los descubiertos de las cuentas corrientes.

- El banco ofrece préstamos a sus clientes. Cada préstamo se origina en una sucursal concreta y puede tener como titulares a uno o más clientes. Cada préstamo queda identificado por un número de préstamo único. De cada préstamo el banco realiza un seguimiento del importe del préstamo y de sus pagos. Aunque el número de pago del préstamo no identifica de manera única un pago concreto entre todos los del banco, el número de pago identifica cada pago concreto de un préstamo dado. Se registran la fecha y el importe de cada pago.
- Los empleados del banco quedan identificados por el valor de su *id_empleado*. La administración del banco almacena el nombre y número de teléfono de cada empleado, el nombre de las personas dependientes de cada empleado y el número de *id_empleado* del jefe de cada empleado. El banco también realiza un seguimiento de la fecha de contratación y, por tanto, de la antigüedad de cada empleado.

En las entidades bancarias reales, el banco realizaría un seguimiento de los depósitos y de los reintegros de las cuentas de ahorro y corrientes, igual que realiza un seguimiento de los pagos de las cuentas de crédito. Dado que los requisitos de modelado para ese seguimiento son parecidas y nos gustaría que la aplicación de ejemplo no tuviera un tamaño excesivo, en nuestro modelo no se realiza el seguimiento de esos depósitos y reintegros.

1.6.3 El modelo entidad-relación

El modelo de datos entidad-relación (E-R) está basado en una percepción del mundo real que consiste en un conjunto de objetos básicos, denominados *entidades*, y de las *relaciones* entre esos objetos. Una entidad es una "cosa" u "objeto" del mundo real que es distinguible de otros objetos. Por ejemplo, cada persona es una entidad, y las cuentas bancarias pueden considerarse entidades.

Las entidades se describen en las bases de datos mediante un conjunto de **atributos**. Por ejemplo, los atributos *número_cuenta* y *saldo* pueden describir una cuenta concreta de un banco y constituyen atributos del conjunto de entidades *cuenta*. Análogamente, los atributos *nombre_cliente*, *calle_cliente* y *ciudad_cliente* pueden describir una entidad *cliente*.

Se usa un atributo extra, *id_cliente*, para identificar unívocamente a los clientes (dado que es posible que haya dos clientes con el mismo nombre, calle y ciudad). Se debe asignar un identificador de cliente único a cada cliente. En Estados Unidos, muchas empresas usan el número de la seguridad social de cada persona (un número único que el Gobierno de Estados Unidos asigna a cada persona) como identificador de cliente.

Una **relación** es una asociación entre varias entidades. Por ejemplo, la relación *impositor* asocia un cliente con cada cuenta que tiene. El conjunto de todas las entidades del mismo tipo, y el conjunto de todas las relaciones del mismo tipo se denominan, respectivamente, **conjunto de entidades** y **conjunto de relaciones**.

La estructura lógica general (esquema) de la base de datos se puede expresar gráficamente mediante un *diagrama E-R*, que está constituido por los siguientes componentes:

- **Rectángulos**, que representan conjuntos de entidades.
- **Elipses**, que representan atributos.
- **Rombos**, que representan conjuntos de relaciones entre miembros de varios conjuntos de entidades.
- **Líneas**, que unen los atributos con los conjuntos de entidades entre sí, y también los conjuntos de entidades con las relaciones.

Cada componente se etiqueta con la entidad o relación que representa.

Como ilustración, considérese parte de un sistema bancario de bases de datos consistente en los clientes y las cuentas que tienen esos clientes. La Figura 1.3 muestra el diagrama E-R correspondiente. El

www.detodoprogramacion.com

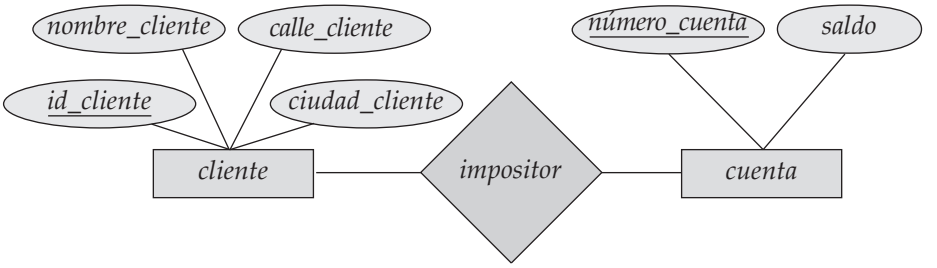


Figura 1.3 Un ejemplo de diagrama E-R.

id_cliente	*número_cuenta*	*saldo*
19.283.746	C-101	500
19.283.746	C-201	900
01.928.374	C-215	700
67.789.901	C-102	400
18.273.609	C-305	350
32.112.312	C-217	750
33.666.999	C-222	700
01.928.374	C-201	900

Figura 1.4 La tabla *impositor'*.

diagrama E-R indica que hay dos conjuntos de entidades, *cliente* y *cuenta*, con los atributos descritos anteriormente. El diagrama también muestra la relación *impositor* entre cliente y cuenta.

Además de entidades y relaciones, el modelo E-R representa ciertas restricciones que los contenidos de la base de datos deben cumplir. Una restricción importante es la **correspondencia de cardinalidades**, que expresa el número de entidades con las que otra entidad se puede asociar a través de un conjunto de relaciones. Por ejemplo, si cada cuenta sólo debe pertenecer a un cliente, el modelo puede expresar esa restricción.

El modelo entidad-relación se usa ampliamente en el diseño de bases de datos, y en el Capítulo 6 se explora en detalle.

1.6.4 Normalización

Otro método de diseño de bases de datos es usar un proceso que suele denominarse normalización. El objetivo es generar un conjunto de esquemas de relaciones que permita almacenar información sin redundancias innecesarias, pero que también permita recuperar la información con facilidad. El enfoque es diseñar esquemas que se hallen en la *forma normal* adecuada. Para determinar si un esquema de relación se halla en una de las formas normales deseadas, hace falta información adicional sobre la empresa real que se está modelando con la base de datos. El enfoque más frecuente es usar **dependencias funcionales**, que se tratan en el Apartado 7.4.

Para comprender la necesidad de la normalización, obsérvese lo que puede fallar en un mal diseño de base de datos. Algunas de las propiedades no deseables de un mal son:

- Repetición de la información.
- Imposibilidad de representar determinada información.

Se examinarán estos problemas con la ayuda de un diseño de bases de datos modificado para el ejemplo bancario.

Supóngase que, en lugar de tener las dos tablas separadas *cuenta* e *impositor*, se tiene una sola tabla, *impositor'*, que combina la información de las dos tablas (como puede verse en la Figura 1.4). Obsérvese que hay dos filas de *impositor'* que contienen información sobre la cuenta C-201. La repetición de información en este diseño alternativo no es deseable. La repetición de información malgasta espacio.

www.detodoprogramacion.com

id_cliente	*nombre_cliente*	*calle_cliente*	*ciudad_cliente*	*número_cuenta*
19.283.746	González	Arenal, 12	La Granja	C-101
19.283.746	González	Arenal, 12	La Granja	C-201
67.789.901	López	Mayor, 3	Peguerinos	C-102
18.273.609	Abril	Preciados, 123	Valsaín	C-305
32.112.312	Santos	Mayor, 100	Peguerinos	C-217
33.666.999	Rupérez	Ramblas, 175	León	C-222
01.928.374	Gómez	Carretas, 72	Cerceda	C-201

Figura 1.5 La tabla *cliente'*.

Además, complica las actualizaciones de la base de datos. Supóngase que se desea modificar el saldo de la cuenta C-201 de 900 a 950 €. Esta modificación debe reflejarse en las dos filas; compárese con el diseño original, en el que esto daría lugar a la actualización de una sola fila. Por tanto, las actualizaciones resultan más costosas bajo el diseño alternativo que bajo el diseño original. Cuando se lleva a cabo la actualización de la base de datos alternativa, hay que asegurarse de que *todas* las tuplas que afectan a la cuenta C-201 se actualicen, o la base de datos mostrará dos valores de saldo diferentes para la cuenta C-201.

Examínese ahora el problema de la "imposibilidad de representar determinada información". Supóngase que, en vez de tener las dos tablas separadas *cliente* e *impositor*, se tuviera una sola tabla, *cliente'*, que combinara la información de esas dos tablas (como puede verse en la Figura 1.5). No se puede representar directamente la información relativa a los clientes (*id_cliente*, *nombre_cliente*, *calle_cliente*, *ciudad_cliente*) a menos que el cliente tenga, como mínimo, una cuenta en el banco. Esto se debe a que las filas de *cliente'* necesitan valores de *número_cuenta*.

Una solución de este problema es introducir valores **nulos**. Los valores *nulos* indican que el valor no existe (o es desconocido). Los valores desconocidos pueden ser valores *ausentes* (el valor existe, pero no se tiene la información) o valores *desconocidos* (no se sabe si el valor existe realmente o no). Como se verá más adelante, los valores nulos resultan difíciles de tratar, y es preferible no recurrir a ellos. Si no se desea tratar con valores nulos, se puede crear un elemento concreto de información del cliente sólo si el cliente tiene cuenta en el banco (obsérvese que los clientes pueden tener un préstamo pero no tener ninguna cuenta). Además, habría que eliminar esa información cuando el cliente cerrara la cuenta. Claramente, esta situación no es deseable ya que, bajo el diseño original de la base de datos, la información de los clientes estaría disponible independientemente de si el cliente tiene cuenta en el banco o no, y sin necesidad de recurrir a los valores nulos.

1.7 Bases de datos basadas en objetos y semiestructuradas

Varias áreas de aplicaciones de los sistemas de bases de datos están limitadas por las restricciones del modelo de datos relacional. En consecuencia, los investigadores han desarrollado varios modelos de datos para tratar con estos dominios de aplicación. Los modelos de datos que se tratarán en este texto son el orientado a objetos y el relacional orientado a objetos, representativos de los modelos de datos basados en objetos, y XML, representativo de los modelos de datos semiestructurados.

1.7.1 Modelos de datos basados en objetos

El **modelo de datos orientado a objetos** se basa en el paradigma de los lenguajes de programación orientados a objetos, que actualmente se usa en gran medida. La herencia, la identidad de los objetos y la encapsulación (ocultación de la información), con métodos para ofrecer una interfaz para los objetos, están entre los conceptos principales de la programación orientada a objetos que han encontrado aplicación en el modelado de datos. El modelo de datos orientado a objetos también soporta un sistema elaborado de tipos, incluidos los tipos estructurados y las colecciones. El modelo orientado a objetos puede considerarse una extensión del modelo E-R con los conceptos de encapsulación, métodos (funciones) e identidad de los objetos.

www.detodoprogramacion.com

El **modelo de datos relacional orientado a objetos** extiende el modelo relacional tradicional con gran variedad de características como los tipos estructurados y las colecciones, así como la orientación a objetos.

En el Capítulo 9 se examinan las bases de datos relacionales orientadas a objetos (es decir, las bases de datos construidas según el modelo relacional orientado a objetos), así como las bases de datos orientadas a objetos (es decir, las bases de datos construidas según el modelo de datos orientado a objetos).

1.7.2 Modelos de datos semiestructurados

Los modelos de datos semiestructurados permiten la especificación de los datos en los que cada elemento de datos del mismo tipo puede tener conjuntos de atributos diferentes. Esto los diferencia de los modelos de datos mencionados anteriormente, en los que todos los elementos de datos de un tipo dado deben tener el mismo conjunto de atributos.

El lenguaje XML se diseñó inicialmente como un modo de añadir información de marcas a los documentos de texto, pero se ha vuelto importante debido a sus aplicaciones en el intercambio de datos. XML ofrece un modo de representar los datos que tienen una estructura anidada y, además, permite una gran flexibilidad en la estructuración de los datos, lo cual es importante para ciertas clases de datos no tradicionales. En el Capítulo 10 se describe el lenguaje XML, diferentes maneras de expresar las consultas sobre datos representados en XML y la transformación de los datos XML de una forma a otra.

1.8 Almacenamiento de datos y consultas

Los sistemas de bases de datos están divididos en módulos que tratan con cada una de las responsabilidades del sistema general. Los componentes funcionales de los sistemas de bases de datos pueden dividirse grosso modo en los componentes gestor de almacenamiento y procesador de consultas.

El gestor de almacenamiento es importante porque las bases de datos suelen necesitar una gran cantidad de espacio de almacenamiento. Las bases de datos corporativas tienen un tamaño que va de los centenares de gigabytes hasta, para las bases de datos de mayor tamaño, los terabytes de datos. Un gigabyte son aproximadamente 1.000 megabytes (1.000 millones de bytes), y un terabyte es aproximadamente un millón de megabytes (1 billón de bytes). Debido a que la memoria principal de las computadoras no puede almacenar toda esta información, la información se almacena en discos. Los datos se intercambian entre los discos de almacenamiento y la memoria principal cuando sea necesario. Como el intercambio de datos con el disco es lento comparado con la velocidad de la unidad central de procesamiento, es fundamental que el sistema de base de datos estructure los datos para minimizar la necesidad de intercambio de datos entre los discos y la memoria principal.

El procesador de consultas es importante porque ayuda al sistema de bases de datos a simplificar y facilitar el acceso a los datos. Las vistas de alto nivel ayudan a conseguir este objetivo; con ellas, los usuarios del sistema no se ven molestados innecesariamente con los detalles físicos de la implementación del sistema. Sin embargo, el rápido procesamiento de las actualizaciones y de las consultas es importante. Es función del sistema de bases de datos traducir las actualizaciones y las consultas escritas en lenguajes no procedimentales, en el nivel lógico, en una secuencia eficiente de operaciones en el nivel físico.

1.8.1 Gestor de almacenamiento

Un *gestor de almacenamiento* es un módulo de programa que proporciona la interfaz entre los datos de bajo nivel almacenados en la base de datos y los programas de aplicación y las consultas remitidas al sistema. El gestor de almacenamiento es responsable de la interacción con el gestor de archivos. Los datos en bruto se almacenan en el disco mediante el sistema de archivos que suele proporcionar un sistema operativo convencional. El gestor de almacenamiento traduce las diferentes instrucciones LMD a comandos de bajo nivel del sistema de archivos. Así, el gestor de almacenamiento es responsable del almacenamiento, la recuperación y la actualización de los datos de la base de datos.

Entre los componentes del gestor de almacenamiento se encuentran:

- **Gestor de autorizaciones e integridad**, que comprueba que se satisfagan las restricciones de integridad y la autorización de los usuarios para tener acceso a los datos.

www.detodoprogramacion.com

- **Gestor de transacciones**, que garantiza que la base de datos quede en un estado consistente (correcto) a pesar de los fallos del sistema, y que la ejecución concurrente de transacciones transcurra si conflictos.
- **Gestor de archivos**, que gestiona la asignación de espacio de almacenamiento de disco y las estructuras de datos usadas para representar la información almacenada en el disco.
- **Gestor de la memoria intermedia**, que es responsable de traer los datos desde el disco de almacenamiento a la memoria principal y decidir los datos a guardar en la memoria caché. El gestor de la memoria intermedia es una parte fundamental de los sistemas de bases de datos, ya que permite que la base de datos maneje tamaños de datos que son mucho mayores que el tamaño de la memoria principal.

El gestor de almacenamiento implementa varias estructuras de datos como parte de la implementación física del sistema:

- **Archivos de datos**, que almacenan la base de datos en sí misma.
- **Diccionario de datos**, que almacena metadatos acerca de la estructura de la base de datos; en particular, su esquema.
- **Índices**, que pueden proporcionar un acceso rápido a los elementos de datos. Como el índice de este libro de texto, los índices de las bases de datos facilitan punteros a los elementos de datos que tienen un valor concreto. Por ejemplo, se puede usar un índice para buscar todos los registros *cuenta* con un *número_cuenta* determinado. La asociación es una alternativa a la indexación que es más rápida en algunos casos, pero no en todos.

Se estudiarán los medios de almacenamiento, las estructuras de archivos y la gestión de la memoria intermedia en el Capítulo 11. Los métodos de acceso eficiente a los datos mediante indexación o asociación se explican en el Capítulo 12.

1.8.2 El procesador de consultas

Entre los componentes del procesador de consultas se encuentran:

- **Intérprete del LDD**, que interpreta las instrucciones del LDD y registra las definiciones en el diccionario de datos.
- **Compilador del LMD**, que traduce las instrucciones del LMD en un lenguaje de consultas a un plan de evaluación que consiste en instrucciones de bajo nivel que entienda el motor de evaluación de consultas.

  Las consultas se suelen poder traducir en varios planes de ejecución alternativos, todos los cuales proporcionan el mismo resultado. El compilador del LMD también realiza **optimización de consultas**, es decir, elige el plan de evaluación de menor coste de entre todas las opciones posibles.
- **Motor de evaluación de consultas**, que ejecuta las instrucciones de bajo nivel generadas por el compilador del LMD.

La evaluación de las consultas se trata en el Capítulo 13, mientras que los métodos por los que el optimizador de consultas elige entre las estrategias de evaluación posibles se tratan en el Capítulo 14.

1.9 Gestión de transacciones

A menudo, varias operaciones sobre la base de datos forman una única unidad lógica de trabajo. Un ejemplo son las transferencia de fondos, como se vio en el Apartado 1.2, en las que se realiza un cargo en una cuenta (llámese *A*) y un abono en otra cuenta (llámese *B*). Evidentemente, resulta fundamental que, o bien tengan lugar tanto el cargo como el abono, o bien que no se produzca ninguno. Es decir, la transferencia de fondos debe tener lugar por completo o no producirse en absoluto. Este requisito

www.detodoprogramacion.com

de todo o nada se denomina **atomicidad**. Además, resulta esencial que la ejecución de la transferencia de fondos preserve la consistencia de la base de datos. Es decir, el valor de la suma $A + B$ se debe preservar. Este requisito de corrección se denomina **consistencia**. Finalmente, tras la ejecución correcta de la transferencia de fondos, los nuevos valores de las cuentas A y B deben persistir, a pesar de la posibilidad de fallo del sistema. Este requisito de persistencia se denomina **durabilidad**.

Una **transacción** es un conjunto de operaciones que lleva a cabo una única función lógica en una aplicación de bases de datos. Cada transacción es una unidad de atomicidad y consistencia. Por tanto, se exige que las transacciones no violen ninguna restricción de consistencia de la base de datos. Es decir, si la base de datos era consistente cuando la transacción comenzó, debe ser consistente cuando la transacción termine con éxito. Sin embargo, durante la ejecución de una transacción, puede ser necesario permitir inconsistencias temporalmente, ya que el cargo a A o el abono a B se debe realizar en primer lugar. Esta inconsistencia temporal, aunque necesaria, puede conducir a dificultades si ocurre un fallo.

Es responsabilidad del programador definir adecuadamente las diferentes transacciones, de tal manera que cada una preserve la consistencia de la base de datos. Por ejemplo, la transacción para transferir fondos de la cuenta A a la cuenta B puede definirse como si estuviera compuesta de dos programas diferentes: uno que realiza el cargo en la cuenta A y otro que realiza el abono en la cuenta B. La ejecución de estos dos programas uno después del otro preservará realmente la consistencia. Sin embargo, cada programa en sí mismo no transforma la base de datos de un estado consistente a otro nuevo. Por tanto, estos programas no son transacciones.

Garantizar las propiedades de atomicidad y de durabilidad es responsabilidad del propio sistema de bases de datos —concretamente del **componente de gestión de transacciones**. A falta de fallos, todas las transacciones se completan con éxito y la atomicidad se consigue fácilmente. Sin embargo, debido a diversos tipos de fallos, puede que las transacciones no siempre completen su ejecución con éxito. Si se va a asegurar la propiedad de atomicidad, las transacciones fallidas no deben tener ningún efecto sobre el estado de la base de datos. Por tanto, la base de datos debe restaurarse al estado en que estaba antes de que la transacción en cuestión comience a ejecutarse. El sistema de bases de datos, por tanto, debe realizar la **recuperación de fallos**, es decir, detectar los fallos del sistema y restaurar la base de datos al estado que tenía antes de que ocurriera el fallo.

Finalmente, cuando varias transacciones actualizan la base de datos de manera concurrente, puede que no se preserve la consistencia de los datos, aunque cada una de las transacciones sea correcta. Es responsabilidad del **gestor de control de concurrencia** controlar la interacción entre las transacciones concurrentes para garantizar la consistencia de la base de datos.

Los conceptos básicos del procesamiento de transacciones se tratan en el Capítulo 15. La gestión de las transacciones concurrentes se trata en el Capítulo 16. En el Capítulo 17 se trata con detalle la recuperación de fallos.

Puede que los sistemas de bases de datos diseñados para su empleo en computadoras personales pequeños no tengan todas estas características. Por ejemplo, muchos sistemas pequeños sólo permiten que un usuario tenga acceso a la base de datos en cada momento. Otros no ofrecen copias de seguridad ni recuperación, y dejan esas tareas a los usuarios. Estas restricciones permiten un gestor de datos de menor tamaño, con menos requisitos de recursos físicos —especialmente de memoria principal. Aunque tales enfoques de bajo coste y bajas prestaciones son adecuados para bases de datos personales pequeñas, resultan inadecuados para empresas medianas y grandes.

El concepto de transacción se ha aplicado ampliamente en los sistemas y en las aplicaciones de bases de datos. Aunque el empleo inicial de las transacciones se produjo en las aplicaciones financieras, el concepto se usa ahora en aplicaciones de tiempo real de telecomunicaciones, así como en la gestión de las actividades de larga duración como el diseño de productos o los flujos de trabajo administrativos. Estas aplicaciones más amplias del concepto de transacción se estudian en el Capítulo 25.

1.10 Minería y análisis de datos

El término **minería de datos** se refiere en líneas generales al proceso de análisis semiautomático de grandes bases de datos para descubrir patrones útiles. Al igual que el descubrimiento de conocimiento en inteligencia artificial (también denominado **aprendizaje de la máquina**) o el análisis estadístico, la minería de datos intenta descubrir reglas y patrones en los datos. Sin embargo, la minería de datos

www.detodoprogramacion.com

se diferencia del aprendizaje de la máquina y de la estadística en que maneja grandes volúmenes de datos, almacenados principalmente en disco. Es decir, la minería de datos trata del "descubrimiento de conocimiento en las bases de datos".

Algunos tipos de conocimiento descubiertos en las bases de datos pueden representarse mediante un conjunto de **reglas**. Lo que sigue es un ejemplo de regla, definida informalmente: "las mujeres jóvenes con ingresos anuales superiores a 50.000 € son las personas con más probabilidades de comprar coches deportivos pequeños". Por supuesto, esas reglas no son universalmente ciertas, sino que tienen grados de "apoyo" y de "confianza". Otros tipos de conocimiento se representan mediante ecuaciones que relacionan diferentes variables, o mediante otros mecanismos para la predicción de los resultados cuando se conocen los valores de algunas variables.

Hay gran variedad de tipos posibles de patrones que pueden resultar útiles y se emplean diferentes técnicas para descubrir tipos de patrones diferentes. En el Capítulo 18 se estudian unos cuantos ejemplos de patrones y se ve la manera en que pueden obtenerse de las bases de datos de forma automática.

Generalmente hay un componente manual en la minería de datos, que consiste en el preprocesamiento de los datos de una manera aceptable para los algoritmos, y el postprocesamiento de los patrones descubiertos para descubrir otros nuevos que puedan resultar útiles. Puede haber también más de un tipo de patrón que pueda descubrirse en una base de datos dada, y puede ser necesaria la interacción manual para escoger los tipos de patrones útiles. Por este motivo, la minería de datos es, en realidad, un proceso semiautomático en la vida real. No obstante, en nuestra descripción se concentra la atención en el aspecto automático de la minería.

Las empresas han comenzado a explotar la creciente cantidad de datos en línea para tomar mejores decisiones sobre sus actividades, como los artículos de los que hay que tener existencias y la mejor manera de llegar a los clientes para incrementar las ventas. Muchas de sus consultas son bastante complicadas, sin embargo, y ciertos tipos de información no pueden extraerse ni siquiera usando SQL.

Se dispone de varias técnicas y herramientas para ayudar a la toma de decisiones. Varias herramientas para el análisis de datos permiten a los analistas ver los datos de diferentes maneras. Otras herramientas de análisis realizan cálculos previos de resúmenes de grandes cantidades de datos, con objeto de dar respuestas rápidas a las preguntas. El estándar SQL:1999 contiene actualmente constructores adicionales para soportar el análisis de datos.

Los datos textuales también han crecido de manera explosiva. Estos datos carecen de estructura, a diferencia de los datos rígidamente estructurados de las bases de datos relacionales. La consulta de datos textuales no estructurados se denomina *recuperación de la información*. Los sistemas de recuperación de la información tienen mucho en común con los sistemas de bases de datos —en especial, el almacenamiento y recuperación de datos en medios de almacenamiento secundarios. Sin embargo, el énfasis en el campo de los sistemas de información es diferente del de los sistemas de bases de datos, y se concentra en aspectos como las consultas basadas en palabras clave; la relevancia de los documentos para la consulta, y el análisis, clasificación e indexación de los documentos. En los Capítulos 18 y 19 se trata la ayuda a la toma de decisiones, incluyendo el procesamiento analítico en línea, la minería de datos y la recuperación de la información.

1.11 Arquitectura de las bases de datos

Ahora es posible ofrecer una visión única (Figura 1.6) de los diversos componentes de los sistemas de bases de datos y de las conexiones existentes entre ellos.

La arquitectura de los sistemas de bases de datos se ve muy influida por el sistema informático subyacente sobre el que se ejecuta el sistema de bases de datos. Los sistemas de bases de datos pueden estar centralizados o ser del tipo cliente-servidor, en los que una máquina servidora ejecuta el trabajo en nombre de multitud de máquinas clientes. Los sistemas de bases de datos pueden diseñarse también para aprovechar las arquitecturas de computadoras paralelas. Las bases de datos distribuidas se extienden por varias máquinas geográficamente separadas.

En el Capítulo 20 se trata la arquitectura general de los sistemas informáticos modernos. El Capítulo 21 describe el modo en que diversas acciones de las bases de datos, en especial el procesamiento de las consultas, pueden implementarse para aprovechar el procesamiento paralelo. El Capítulo 22 presenta varios problemas que surgen en las bases de datos distribuidas y describe el modo de afrontarlos. Entre

www.detodoprogramacion.com

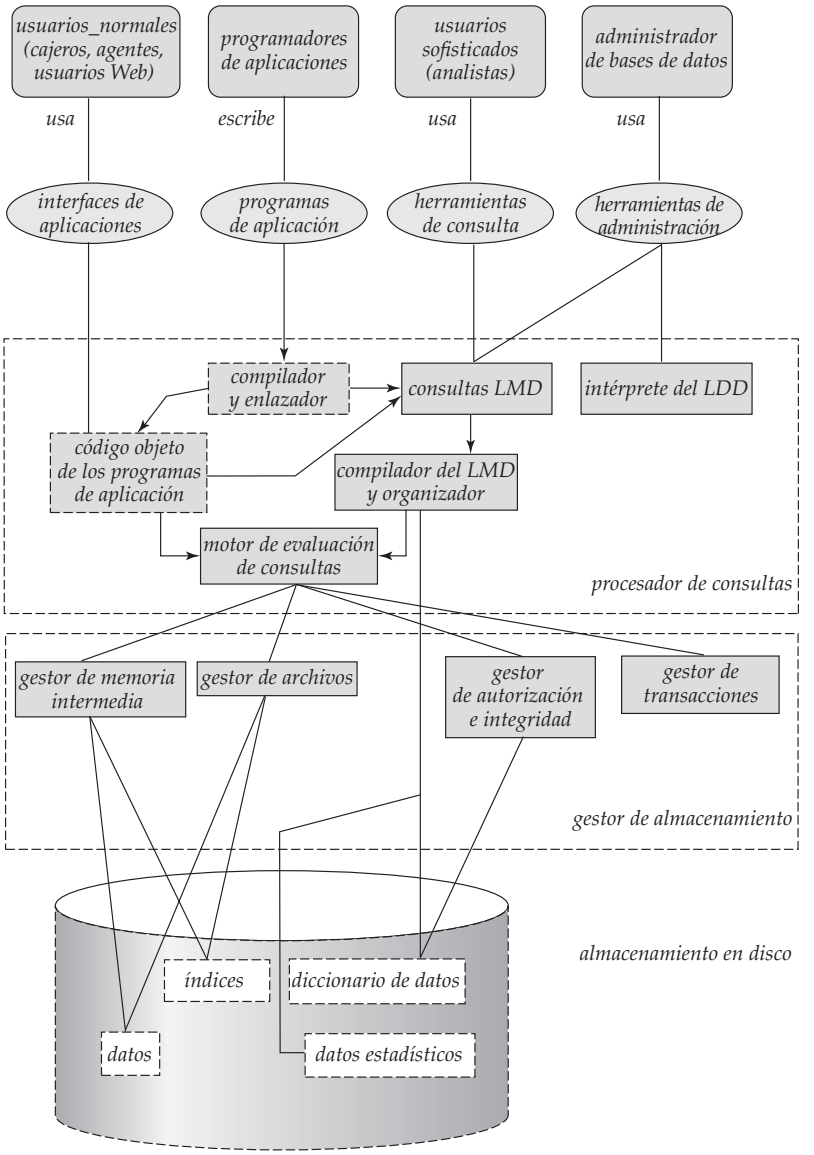


Figura 1.6 Arquitectura del sistema.

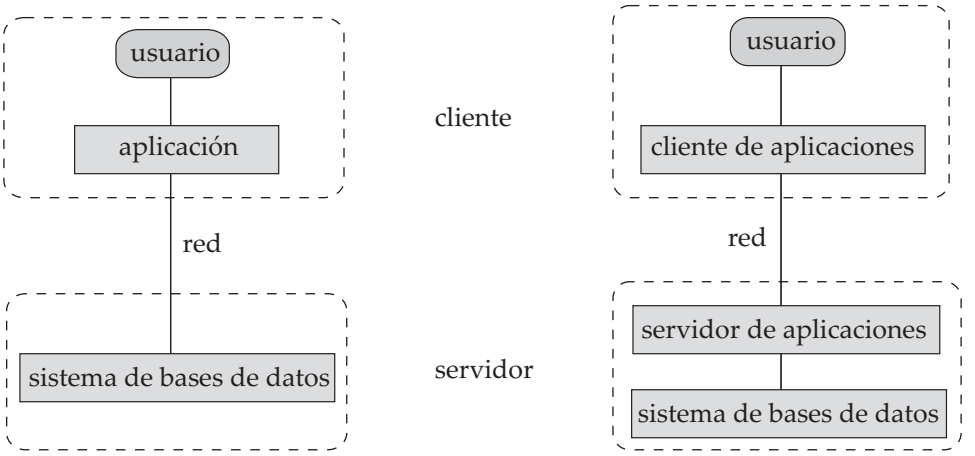
los problemas se encuentran el modo de almacenar los datos, la manera de asegurar la atomicidad de las transacciones que se ejecutan en varios sitios, cómo llevar a cabo controles de concurrencia y el modo de ofrecer alta disponibilidad en presencia de fallos. El procesamiento distribuido de las consultas y los sistemas de directorio también se describen en ese capítulo.

Hoy en día la mayor parte de los usuarios de los sistemas de bases de datos no está presente en el lugar físico en que se encuentra el sistema de bases de datos, sino que se conectan a él a través de una red. Por tanto, se puede diferenciar entre los sistemas **clientes**, en los que trabajan los usuarios remotos de la base de datos, y los sistemas **servidores**, en los que se ejecutan los sistemas de bases de datos.

Las aplicaciones de bases de datos suelen dividirse en dos o tres partes, como puede verse en la Figura 1.7. En una **arquitectura de dos capas**, la aplicación se divide en un componente que reside en la máquina cliente, que llama a la funcionalidad del sistema de bases de datos en la máquina servidora mediante instrucciones del lenguaje de consultas. Los estándares de interfaces de programas de aplicación como ODBC y JDBC se usan para la interacción entre el cliente y el servidor.

En cambio, en una **arquitectura de tres capas**, la máquina cliente actúa simplemente como una parte visible al usuario y no contiene ninguna llamada directa a la base de datos. En vez de eso, el extremo

www.detodoprogramacion.com



(a) Arquitectura de dos capas (b) Arquitectura de tres capas

Figura 1.7 Arquitecturas de dos y tres capas.

cliente se comunica con un **servidor de aplicaciones**, generalmente mediante una interfaz de formularios. El servidor de aplicaciones, a su vez, se comunica con el sistema de bases de datos para tener acceso a los datos. La **lógica de negocio** de la aplicación, que establece las acciones que se deben realizar según las condiciones reinantes, se incorpora en el servidor de aplicaciones, en lugar de estar distribuida entre múltiples clientes. Las aplicaciones de tres capas resultan más adecuadas para aplicaciones de gran tamaño y para las aplicaciones que se ejecutan en World Wide Web.

1.12 Usuarios y administradores de bases de datos

Uno de los objetivos principales de los sistemas de bases de datos es recuperar información de la base de datos y almacenar en ella información nueva. Las personas que trabajan con una base de datos se pueden clasificar como usuarios o administradores de bases de datos.

1.12.1 Usuarios de bases de datos e interfaces de usuario

Hay cuatro tipos diferentes de usuarios de los sistemas de bases de datos, diferenciados por la forma en que esperan interactuar con el sistema. Se han diseñado diferentes tipos de interfaces de usuario para los diferentes tipos de usuarios.

- Los **usuarios normales** son usuarios no sofisticados que interactúan con el sistema invocando alguno de los programas de aplicación que se han escrito previamente. Por ejemplo, un cajero bancario que necesita transferir 50 € de la cuenta *A* a la cuenta *B* invoca un programa llamado *transferencia*. Ese programa le pide al cajero el importe de dinero que se va a transferir, la cuenta desde la que se va a transferir el dinero y la cuenta a la que se va a transferir el dinero.

  Como ejemplo adicional, considérese un usuario que desea averiguar el saldo de su cuenta en World Wide Web. Ese usuario puede acceder a un formulario en el que introduce su número de cuenta. Un programa de aplicación en el servidor Web recupera entonces el saldo de la cuenta, usando el número de cuenta proporcionado, y devuelve la información al usuario.

  La interfaz de usuario habitual para los usuarios normales es una interfaz de formularios, donde el usuario puede rellenar los campos correspondientes del formulario. Los usuarios normales también pueden limitarse a leer *informes* generados por la base de datos.

- Los **programadores de aplicaciones** son profesionales informáticos que escriben programas de aplicación. Los programadores de aplicaciones pueden elegir entre muchas herramientas para desarrollar las interfaces de usuario. Las herramientas de **desarrollo rápido de aplicaciones**

www.detodoprogramacion.com

(DRA) son herramientas que permiten al programador de aplicaciones crear formularios e informes con un mínimo esfuerzo de programación.

- Los **usuarios sofisticados** interactúan con el sistema sin escribir programas. En su lugar, formulan sus consultas en un lenguaje de consultas de bases de datos. Remiten cada una de las consultas al **procesador de consultas**, cuya función es dividir las instrucciones LMD en instrucciones que el gestor de almacenamiento entienda. Los analistas que remiten las consultas para explorar los datos de la base de datos entran en esta categoría.
- Los **usuarios especializados** son usuarios sofisticados que escriben aplicaciones de bases de datos especializadas que no encajan en el marco tradicional del procesamiento de datos. Entre estas aplicaciones están los sistemas de diseño asistido por computadora, los sistemas de bases de conocimientos y los sistemas expertos, los sistemas que almacenan datos con tipos de datos complejos (por ejemplo, los datos gráficos y los datos de sonido) y los sistemas de modelado del entorno. En el Capítulo 9 se estudian varias de estas aplicaciones.

1.12.2 Administrador de bases de datos

Una de las principales razones de usar SGBDs es tener un control centralizado tanto de los datos como de los programas que tienen acceso a esos datos. La persona que tiene ese control central sobre el sistema se denomina **administrador de bases de datos** (**ABD**). Las funciones del ABD incluyen:

- La **definición del esquema**. El ABD crea el esquema original de la base de datos mediante la ejecución de un conjunto de instrucciones de definición de datos en el LDD.
- La **definición de la estructura y del método de acceso**.
- La **modificación del esquema y de la organización física**. El ABD realiza modificaciones en el esquema y en la organización física para reflejar las necesidades cambiantes de la organización, o para alterar la organización física a fin de mejorar el rendimiento.
- La **concesión de autorización para el acceso a los datos**. Mediante la concesión de diferentes tipos de autorización, el administrador de bases de datos puede regular las partes de la base de datos a las que puede tener acceso cada usuario. La información de autorización se guarda en una estructura especial del sistema que el SGBD consulta siempre que alguien intenta tener acceso a los datos del sistema.
- El **mantenimiento rutinario**. Algunos ejemplos de las actividades de mantenimiento rutinario del administrador de la base de datos son:
  - Copia de seguridad periódica de la base de datos, bien sobre cinta o sobre servidores remotos, para impedir la pérdida de datos en caso de desastres como las inundaciones.
  - Asegurarse de que se dispone de suficiente espacio libre en disco para las operaciones normales y aumentar el espacio en disco según sea necesario.
  - Supervisar los trabajos que se ejecuten en la base de datos y asegurarse de que el rendimiento no se degrade debido a que algún usuario haya remitido tareas muy costosas.

1.13 Historia de los sistemas de bases de datos

El procesamiento de datos impulsa el crecimiento de las computadoras, como lo ha hecho desde los primeros días de las computadoras comerciales. De hecho, la automatización de las tareas de procesamiento de datos precede a las computadoras. Las tarjetas perforadas, inventadas por Herman Hollerith, se emplearon a principios del siglo XX para registrar los datos del censo de Estados Unidos, y se usaron sistemas mecánicos para procesar las tarjetas y para tabular los resultados. Las tarjetas perforadas se usaron posteriormente con profusión como medio para introducir datos en las computadoras.

Las técnicas de almacenamiento y de procesamiento de datos han evolucionado a lo largo de los años:

- **Años cincuenta y primeros años sesenta**: se desarrollaron las cintas magnéticas para el almacenamiento de datos. Las tareas de procesamiento de datos como la elaboración de nóminas se

www.detodoprogramacion.com

automatizaron, con los datos almacenados en cintas. El procesamiento de datos consistía en leer datos de una o varias cintas y escribir datos en una nueva cinta. Los datos también se podían introducir desde paquetes de tarjetas perforadas e imprimirse en impresoras. Por ejemplo, los aumentos de sueldo se procesaban introduciendo los aumentos en las tarjetas perforadas y leyendo el paquete de cintas perforadas de manera sincronizada con una cinta que contenía los detalles principales de los salarios. Los registros debían estar en el mismo orden. Los aumentos de sueldo se añadían a los sueldos leídos de la cinta maestra y se escribían en una nueva cinta; esa nueva cinta se convertía en la nueva cinta maestra.

Las cintas (y los paquetes de tarjetas perforadas) sólo se podían leer secuencialmente, y el tamaño de datos era mucho mayor que la memoria principal; por tanto, los programas de procesamiento de datos se veían obligados a procesar los datos en un orden determinado, leyendo y mezclando datos de las cintas y de los paquetes de tarjetas perforadas.

- **Finales de los años sesenta y años setenta**: el empleo generalizado de los discos duros a finales de los años sesenta modificó en gran medida la situación del procesamiento de datos, ya que permitieron el acceso directo a los datos. La ubicación de los datos en disco no era importante, ya que se podía tener acceso a cualquier posición del disco en sólo unas decenas de milisegundos. Los datos se liberaron así de la tiranía de la secuencialidad. Con los discos pudieron crearse las bases de datos de red y las bases de datos jerárquicas, que permitieron que las estructuras de datos como las listas y los árboles pudieran almacenarse en disco. Los programadores pudieron crear y manipular estas estructuras de datos.

  El artículo histórico de Codd [1970] definió el modelo relacional y las formas no procedimentales de consultar los datos en el modelo relacional, y así nacieron las bases de datos relacionales. La simplicidad del modelo relacional y la posibilidad de ocultar completamente los detalles de implementación a los programadores resultaron realmente atractivas. Codd obtuvo posteriormente el prestigioso premio Turing de la ACM (Association of Computing Machinery, asociación de maquinaria informática) por su trabajo.

- **Años ochenta**: aunque académicamente interesante, el modelo relacional no se usó inicialmente en la práctica debido a sus inconvenientes en cuanto a rendimiento; las bases de datos relacionales no podían igualar el rendimiento de las bases de datos de red y jerárquicas existentes. Esta situación cambió con System R, un proyecto innovador del centro de investigación IBM Research que desarrolló técnicas para la construcción de un sistema de bases de datos relacionales eficiente. En Astrahan et al. [1976] y Chamberlin et al. [1981] se pueden encontrar excelentes visiones generales de System R. El prototipo de System R completamente funcional condujo al primer producto de bases de datos relacionales de IBM: SQL/DS. Los primeros sistemas comerciales de bases de datos relacionales, como DB2 de IBM, Oracle, Ingres y Rdb de DEC, desempeñaron un importante papel en el desarrollo de técnicas para el procesamiento eficiente de las consultas declarativas. En los primeros años ochenta las bases de datos relacionales habían llegado a ser competitivas frente a los sistemas de bases de datos jerárquicas y de red incluso en cuanto a rendimiento. Las bases de datos relacionales eran tan sencillas de usar que finalmente reemplazaron a las bases de datos jerárquicas y de red; los programadores que usaban esas bases de datos se veían obligados a tratar muchos detalles de implementación de bajo nivel y tenían que codificar sus consultas de forma procedimental. Lo que era aún más importante, tenían que tener presente el rendimiento durante el diseño de los programas, lo que suponía un gran esfuerzo. En cambio, en las bases de datos relacionales, casi todas estas tareas de bajo nivel las realiza de manera automática el sistema de bases de datos, lo que libera al programador para que se centre en el nivel lógico. Desde su obtención de liderazgo en los años ochenta, el modelo relacional ha reinado sin discusión entre todos los modelos de datos.

  Los años ochenta también fueron testigos de una gran investigación en las bases de datos paralelas y distribuidas, así como del trabajo inicial en las bases de datos orientadas a objetos.

- **Primeros años noventa**: el lenguaje SQL se diseñó fundamentalmente para las aplicaciones de ayuda a la toma de decisiones, que son intensivas en consultas, mientras que el objetivo principal de las bases de datos en los años ochenta eran las aplicaciones de procesamiento de tran-

www.detodoprogramacion.com

sacciones, que son intensivas en actualizaciones. La ayuda a la toma de decisiones y las consultas volvieron a emerger como una importante área de aplicación para las bases de datos. El uso de las herramientas para analizar grandes cantidades de datos experimentó un gran crecimiento.

En esta época muchas marcas de bases de datos introdujeron productos de bases de datos paralelas. Las diferentes marcas de bases de datos también comenzaron a añadir soporte relacional orientado a objetos a sus bases de datos.

- **Finales de los años noventa**: el principal acontecimiento fue el crecimiento explosivo de World Wide Web. Las bases de datos se implantaron mucho más ampliamente que nunca. Los sistemas de bases de datos tenían que soportar tasas de procesamiento de transacciones muy elevadas, así como una fiabilidad muy alta y tener disponibilidad 24 $\times$ 7 (disponibilidad 24 horas al día y 7 días a la semana, lo que significa que no hay momentos de inactividad debidos a actividades de mantenimiento planificadas). Los sistemas de bases de datos también tenían que soportar interfaces Web para los datos.
- **Principios del siglo XXI**: los principios del siglo XXI han sido testigos de la emergencia de XML y de su lenguaje de consultas asociado, XQuery, como nueva tecnología de las bases de datos. Todavía es pronto para decir el papel que XML desempeñará en las bases de datos futuras. En este periodo también se ha podido presenciar el crecimiento de las técnicas de "informática autónoma/administración automática" para la minimización del esfuerzo de administración.

1.14 Resumen

- Un **sistema gestor de bases de datos (SGBD)** consiste en un conjunto de datos interrelacionados y en un conjunto de programas para tener acceso a esos datos. Los datos describen una empresa concreta.
- El objetivo principal de un SGBD es proporcionar un entorno que sea tanto conveniente como eficiente para las personas que lo usan para la recuperación y almacenamiento de información.
- Los sistemas de bases de datos resultan ubicuos hoy en día, y la mayor parte de la gente interactúa, directa o indirectamente, con bases de datos muchas veces al día.
- Los sistemas de bases de datos se diseñan para almacenar grandes cantidades de información. La gestión de los datos implica tanto la definición de estructuras para el almacenamiento de la información como la provisión de mecanismos para la manipulación de la información. Además, los sistemas de bases de datos deben preocuparse de la seguridad de la información almacenada, en caso de caídas del sistema o de intentos de acceso sin autorización. Si los datos deben compartirse entre varios usuarios, el sistema debe evitar posibles resultados anómalos.
- Uno de los propósitos principales de los sistemas de bases de datos es ofrecer a los usuarios una visión abstracta de los datos. Es decir, el sistema oculta ciertos detalles de la manera en que los datos se almacenan y mantienen.
- Por debajo de la estructura de la base de datos se halla el **modelo de datos**: un conjunto de herramientas conceptuales para describir los datos, las relaciones entre los datos, la semántica de los datos y las restricciones de los datos.
- Un **lenguaje de manipulación de datos (LMD)** es un lenguaje que permite a los usuarios tener acceso a los datos o manipularlos. Los LMDs no procedimentales, que sólo necesitan que el usuario especifique los datos que necesita, sin especificar exactamente la manera de obtenerlos, se usan mucho hoy en día.
- Un **lenguaje de definición de datos (LDD)** es un lenguaje para la especificación del esquema de la base de datos y otras propiedades de los datos.
- El modelo de datos relacional es el más implantado para el almacenamiento de datos en las bases de datos. Otros modelos de datos son el modelo de datos orientado a objetos, el modelo relacional orientado a objetos y los modelos de datos semiestructurados.

www.detodoprogramacion.com

PARTE 2

Diseño de bases de datos

Los sistemas de bases de datos están diseñados para gestionar grandes cantidades de información. Estas grandes cantidades de información no se encuentran aisladas. Forman parte del funcionamiento de alguna empresa cuyo producto final puede ser la información obtenida de la base de datos o algún producto o servicio para el que la base de datos sólo desempeña un papel secundario.

Los dos primeros capítulos de esta parte se centran en el diseño de los esquemas de las bases de datos. El modelo entidad-relación (E-R) descrito en el Capítulo 6 es un modelo de datos de alto nivel. En lugar de representar todos los datos en tablas, distingue entre los objetos básicos, denominados *entidades*, y las *relaciones* entre esos objetos. Suele utilizarse como un primer paso en el diseño de los esquemas de las bases de datos.

El diseño de las bases de datos relacionales—el diseño del esquema relacional—se trató informalmente en los capítulos anteriores. No obstante, existen criterios para distinguir los buenos diseños de bases de datos de los malos. Éstos se formalizan mediante varias "formas normales" que ofrecen diferentes compromisos entre la posibilidad de inconsistencias y la eficiencia de determinadas consultas. El Capítulo 7 describe el diseño formal de los esquemas de las relaciones.

El diseño de un entorno completo de aplicaciones para bases de datos, que responda a las necesidades de la empresa que se modela, exige prestar atención a un conjunto de aspectos más amplio, muchos de los cuales se tratan en el Capítulo 8. Este capítulo describe las interfaces para bases de datos basadas en Web y amplía el estudio de capítulos anteriores sobre la integridad de los datos y la seguridad.

www.detodoprogramacion.com

CAPÍTULO 6

Diseño de bases de datos y el modelo E-R

Hasta este punto se ha dado por supuesto un determinado esquema de la base de datos y se ha estudiado la manera de expresar las consultas y las actualizaciones. Ahora se va a considerar en primer lugar la manera de diseñar el esquema de la base de datos. En este capítulo nos centraremos en el modelo de datos entidad-relación (E-R), que ofrece una manera de identificar las entidades que se van a representar en la base de datos y el modo en que se relacionan entre sí. Finalmente, el diseño de la base de datos se expresará en términos del diseño de bases de datos relacionales y del conjunto de restricciones asociado. En este capítulo se mostrará la manera en que el diseño E-R puede transformarse en un conjunto de esquemas de relación y el modo en que se pueden incluir algunas de las restricciones en ese diseño. Luego, en el Capítulo 7, se considerará en detalle si el conjunto de esquemas de relación representa un diseño de la base de datos bueno o malo y se estudiará el proceso de creación de buenos diseños usando un conjunto de restricciones más amplio. Estos dos capítulos tratan los conceptos fundamentales del diseño de bases de datos.

6.1 Visión general del proceso de diseño

La tarea de creación de aplicaciones de bases de datos es una labor compleja, que implica varias fases, como el diseño del esquema de la base de datos, el diseño de los programas que tienen acceso a los datos y los actualizan y el diseño del esquema de seguridad para controlar el acceso a los datos. Las necesidades de los usuarios desempeñan un papel central en el proceso de diseño. En este capítulo nos centraremos en el diseño del esquema de la base de datos, aunque se esbozarán brevemente algunas de las otras tareas más adelante en este capítulo.

El diseño de un entorno completo de aplicaciones de bases de datos que responda a las necesidades de la empresa que se está modelando exige prestar atención a un amplio conjunto de consideraciones. Estos aspectos adicionales del uso esperado de la base de datos influyen en gran variedad de opciones de diseño en los niveles físico, lógico y de vistas.

6.1.1 Fases del diseño

Para aplicaciones pequeñas puede resultar factible para un diseñador de bases de datos que comprenda los requisitos de la aplicación decidir directamente sobre las relaciones que hay que crear, sus atributos y las restricciones sobre las relaciones. Sin embargo, un proceso de diseño tan directo resulta difícil para las aplicaciones reales, ya que a menudo son muy complejas. Frecuentemente no existe una sola persona que comprenda todas las necesidades de datos de la aplicación. El diseñador de la base de datos debe interactuar con los usuarios para comprender las necesidades de la aplicación, realizar una representación de alto nivel de esas necesidades que pueda ser comprendida por los usuarios y luego traducir esos requisitos a niveles inferiores del diseño. Los modelos de datos de alto nivel sirven a los diseñadores de bases de datos ofreciéndoles un marco conceptual en el que especificar de forma

www.detodoprogramacion.com

sistemática los requisitos de datos de los usuarios de la base de datos y una estructura para la base de datos que satisfaga esos requisitos.

- La fase inicial del diseño de las bases de datos es la caracterización completa de las necesidades de datos de los posibles usuarios de la base de datos. El diseñador de la base de datos debe interactuar intensamente con los expertos y los usuarios del dominio para realizar esta tarea. El resultado de esta fase es una especificación de requisitos del usuario. Aunque existen técnicas para representar en diagramas los requisitos de los usuarios, en este capítulo sólo se describirán textualmente esos requisitos, que se ilustrarán posteriormente en el Apartado 6.8.2.
- A continuación, el diseñador elige el modelo de datos y, aplicando los conceptos del modelo de datos elegido, traduce estos requisitos en un esquema conceptual de la base de datos. El esquema desarrollado en esta fase de **diseño conceptual** proporciona una visión detallada de la empresa. Se suele emplear el modelo entidad-relación, que se estudiará en el resto de este capítulo, para representar el diseño conceptual. En términos del modelo entidad-relación, el esquema conceptual especifica las entidades que se representan en la base de datos, sus atributos, las relaciones entre ellas y las restricciones que las afectan. Generalmente, la fase de diseño conceptual da lugar a la creación de un diagrama entidad-relación que ofrece una representación gráfica del esquema.

  El diseñador revisa el esquema para confirmar que realmente se satisfacen todos los requisitos y que no entran en conflicto entre sí. También puede examinar el diseño para eliminar características redundantes. Su atención en este momento se centra en describir los datos y sus relaciones, más que en especificar los detalles del almacenamiento físico.
- Un esquema conceptual completamente desarrollado indica también los requisitos funcionales de la empresa. En la **especificación de requisitos funcionales** los usuarios describen los tipos de operaciones (o transacciones) que se llevarán a cabo sobre los datos. Algunos ejemplos de operaciones son la modificación o actualización de datos, la búsqueda y recuperación de datos concretos y el borrado de datos. En esta fase de diseño conceptual el diseñador puede revisar el esquema para asegurarse de que satisface los requisitos funcionales.
- El proceso de paso desde el modelo abstracto de datos a la implementación de la base de datos se divide en de dos fases de diseño finales.
  - En la **fase de diseño lógico** el diseñador traduce el esquema conceptual de alto nivel al modelo de datos de la implementación del sistema de bases de datos que se va a usar. El modelo de implementación de los datos suele ser el modelo relacional, y este paso suele consistir en la traducción del esquema conceptual definido mediante el modelo entidad-relación en un esquema de relación.
  - Finalmente, el diseñador usa el esquema de base de datos resultante propio del sistema en la siguiente **fase de diseño físico**, en la que se especifican las características físicas de la base de datos. Entre estas características están la forma de organización de los archivos y las estructuras de almacenamiento interno; se estudian en el Capítulo 11.

El esquema físico de la base de datos puede modificarse con relativa facilidad una vez creada la aplicación. Sin embargo, las modificaciones del esquema lógico suelen resultar más difíciles de llevar a cabo, ya que pueden afectar a varias consultas y actualizaciones dispersas por todo el código de la aplicación. Por tanto, es importante llevar a cabo con cuidado la fase de diseño de la base de datos antes de crear el resto de la aplicación de bases de datos.

6.1.2 Alternativas de diseño

Una parte importante del proceso de diseño de las bases de datos consiste en decidir la manera de representar en el diseño los diferentes tipos de "cosas", como personas, lugares, productos y similares. Se usa el término *entidad* para hacer referencia a cualquiera de esos elementos claramente identificables. Las diferentes entidades tienen algunos aspectos en común y algunas diferencias. Se desea aprovechar los aspectos en común para tener un diseño conciso, fácilmente comprensible, pero hay que conservar la suficiente flexibilidad como para representar las diferencias entre las diferentes entidades existentes en

www.detodoprogramacion.com

el momento del diseño o que se puedan materializar en el futuro. Las diferentes entidades se relacionan entre sí de diversas maneras, todas las cuales deben incluirse en el diseño de la base de datos.

Al diseñar el esquema de una base de datos hay que asegurarse de que se evitan dos peligros importantes:

1. **Redundancia**. Un mal diseño puede repetir información. En el ejemplo bancario que se ha venido usando, se tiene una relación con la información sobre los clientes y una relación separada con la información sobre las cuentas. Supóngase que, en lugar de eso, se repitiera toda la información sobre los clientes (nombre, dirección, etc.) una vez por cada cuenta o préstamo que tuviera cada cliente. Evidentemente, sería redundante. Lo ideal sería que la información apareciera exactamente en un solo lugar.
2. **Incompletitud**. Un mal diseño puede hacer que determinados aspectos de la empresa resulten difíciles o imposibles de modelar. Por ejemplo, supóngase que se usa un diseño de base de datos para el escenario bancario que almacena la información del nombre y de la dirección del cliente con cada cuenta y con cada préstamo, pero que no tiene una relación separada para los clientes. Resultaría imposible introducir el nombre y la dirección de los clientes nuevos a menos que ya tuvieran abierta una cuenta o concedido un préstamo. Se podría intentar salir del paso con este diseño problemático almacenando valores nulos para la información de las cuentas o de los préstamos, como puede ser el número de cuenta o el importe del préstamo. Este parche no sólo resulta poco atractivo, sino que puede evitarse mediante restricciones de clave primaria.

Evitar los malos diseños no es suficiente. Puede haber gran número de buenos diseños entre los que haya que escoger. Como ejemplo sencillo, considérese un cliente que compra un producto. ¿La venta de este producto es una relación entre el cliente y el producto? Dicho de otra manera, ¿es la propia venta una entidad que está relacionada con el cliente y con el producto? Esta elección, aunque simple, puede suponer una importante diferencia en cuanto a los aspectos de la empresa que se pueden modelar bien. Considerando la necesidad de tomar decisiones como ésta para el gran número de entidades y de relaciones que hay en las empresas reales, no es difícil ver que el diseño de bases de datos puede constituir un problema arduo. En realidad, se verá que exige una combinación de conocimientos y de "buen gusto".

6.2 El modelo entidad-relación

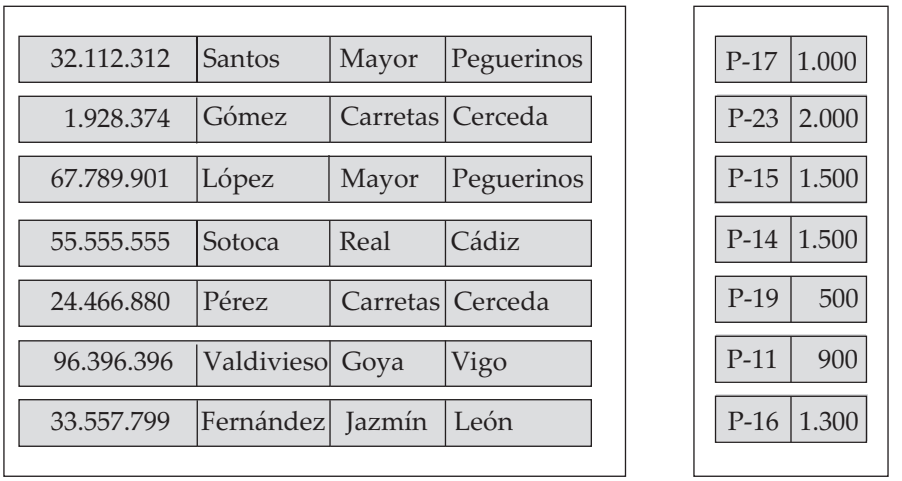
El modelo de datos **entidad–relación (E-R)** se desarrolló para facilitar el diseño de bases de datos permitiendo la especificación de un *esquema de la empresa* que representa la estructura lógica global de la base de datos. El modelo de datos E-R es uno de los diferentes modelos de datos semánticos; el aspecto semántico del modelo radica en la representación del significado de los datos. El modelo E-R resulta muy útil para relacionar los significados e interacciones de las empresas reales con el esquema conceptual. Debido a esta utilidad, muchas herramientas de diseño de bases de datos se basan en los conceptos del modelo E-R. El modelo de datos E-R emplea tres conceptos básicos: los conjuntos de entidades, los conjuntos de relaciones y los atributos.

6.2.1 Conjuntos de entidades

Una **entidad** es una "cosa" u "objeto" del mundo real que es distinguible de todos los demás objetos. Por ejemplo, cada persona de una empresa es una entidad. Una entidad tiene un conjunto de propiedades, y los valores de algún conjunto de propiedades pueden identificar cada entidad de forma unívoca. Por ejemplo, el DNI 67.789.901 identifica unívocamente una persona concreta de la empresa. Análogamente, los préstamos se pueden considerar entidades, y el número de préstamo P-15 de la sucursal de Navacerrada identifica unívocamente una entidad de préstamo. Las entidades pueden ser concretas, como las personas o los libros, o abstractas, como los préstamos, las vacaciones o los conceptos.

Un **conjunto de entidades** es un conjunto de entidades del mismo tipo que comparten las mismas propiedades, o atributos. El conjunto de todas las personas que son clientes en un banco dado, por

www.detodoprogramacion.com



32.112.312	Santos	Mayor	Peguerinos
1.928.374	Gómez	Carretas	Cerceda
67.789.901	López	Mayor	Peguerinos
55.555.555	Sotoca	Real	Cádiz
24.466.880	Pérez	Carretas	Cerceda
96.396.396	Valdivieso	Goya	Vigo
33.557.799	Fernández	Jazmín	León

cliente

P-17	1.000
P-23	2.000
P-15	1.500
P-14	1.500
P-19	500
P-11	900
P-16	1.300

préstamo

Figura 6.1 Conjuntos de entidades *cliente* y *préstamo*.

ejemplo, se puede definir como el conjunto de entidades *cliente*. Análogamente, el conjunto de entidades *préstamo* puede representar el conjunto de todos los préstamos concedidos por un banco concreto. Cada una de las entidades que constituyen un conjunto se denomina *extensión* de ese conjunto de entidades. Por tanto, todos los clientes de un banco son la extensión del conjunto de entidades *cliente*.

Los conjuntos de entidades no son necesariamente disjuntos. Por ejemplo, es posible definir el conjunto de entidades de todos los empleados de un banco (*empleado*) y el conjunto de entidades de todos los clientes del banco (*cliente*). Una entidad *persona* puede ser una entidad *empleado*, una entidad *cliente*, ambas cosas, o ninguna.

Cada entidad se representa mediante un conjunto de **atributos**. Los atributos son propiedades descriptivas que posee cada miembro de un conjunto de entidades. La designación de un atributo para un conjunto de entidades expresa que la base de datos almacena información parecida relativa a cada entidad del conjunto de entidades; sin embargo, cada entidad puede tener su propio valor para cada atributo. Posibles atributos del conjunto de entidades *cliente* son *id_cliente*, *nombre_cliente*, *calle_cliente* y *ciudad_cliente.* En la vida real habría más atributos, como el número de la calle, el número del piso la provincia, el código postal, y el país, pero se omiten para no complicar el ejemplo. Posibles atributos del conjunto de entidades *préstamo* son *número_préstamo* e *importe*.

Cada entidad tiene un **valor** para cada uno de sus atributos. Por ejemplo, una entidad *cliente* concreta puede tener el valor 32.112.312 para *id_cliente*, el valor Santos para *nombre_cliente*, el valor Mayor para *calle_cliente* y el valor Peguerinos para *ciudad_cliente*.

El atributo *id_cliente* se usa para identificar unívocamente a los clientes, dado que puede haber más de un cliente con el mismo nombre, calle y ciudad. En Estados Unidos, muchas empresas consideran adecuado usar el número *seguridad_social* de cada persona[1] como atributo cuyo valor identifica unívocamente a esa persona. En general la empresa tendría que crear y asignar un identificador unívoco a cada cliente.

Por tanto, las bases de datos incluyen una serie de conjuntos de entidades, cada una de las cuales contiene cierto número de entidades del mismo tipo. La Figura 6.1 muestra parte de una base de datos de un banco que consta de dos conjuntos de entidades, *cliente* y *préstamo*.

Las bases de datos para entidades bancarias pueden incluir diferentes conjuntos de entidades. Por ejemplo, además del seguimiento de los clientes y de los préstamos, el banco también ofrece cuentas, que se representan mediante el conjunto de entidades *cuenta* con los atributos *número_cuenta* y *saldo*. Además, si el banco tiene varias sucursales, se puede guardar información acerca de todas las sucursales del

1. En España se asigna a cada ciudadano un número único, denominado número del documento nacional de identidad (DNI) para identificarla unívocamente. Se supone que cada persona tiene un único DNI y que no hay dos personas con el mismo DNI.

www.detodoprogramacion.com

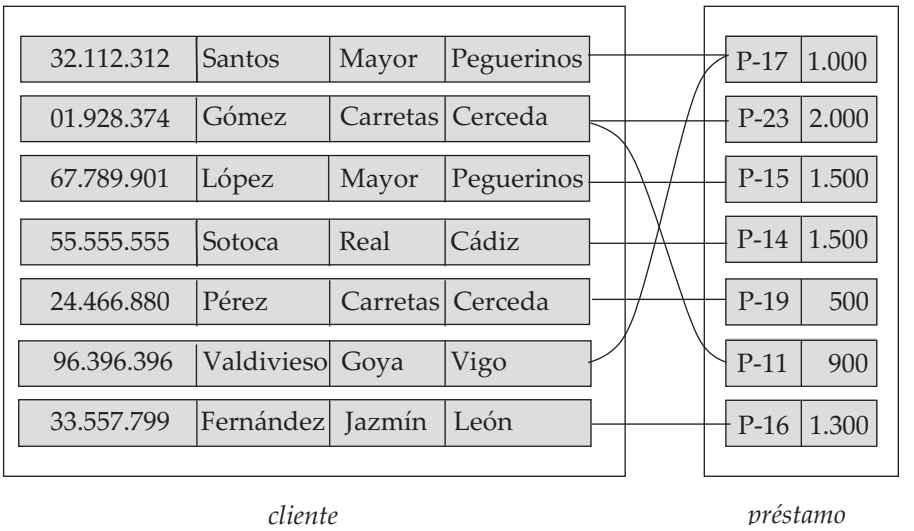


Figura 6.2 Conjunto de relaciones *prestatario*.

banco. Cada conjunto de entidades *sucursal* se puede describir mediante los atributos *nombre_sucursal*, *ciudad_sucursal* y *activos*.

6.2.2 Conjuntos de relaciones

Una **relación** es una asociación entre varias entidades. Por ejemplo, se puede definir una relación que asocie al cliente López con el préstamo P-15. Esta relación especifica que López es un cliente con el préstamo número P-15.

Un **conjunto de relaciones** es un conjunto de relaciones del mismo tipo. Formalmente es una relación matemática con $n \geq 2$ de conjuntos de entidades (posiblemente no distintos). Si $E_1,\ E_2, \ldots, E_n$ son conjuntos de entidades, entonces un conjunto de relaciones R es un subconjunto de:

$$\{(e_1, e_2, \ldots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

donde $(e_1, e_2, \ldots, e_n)$ es una relación.

Considérense las dos entidades *cliente* y *préstamo* de la Figura 6.1. Se define el conjunto de relaciones *prestatario* para denotar la asociación entre los clientes y los préstamos bancarios que tienen concedidos. La Figura 6.2 muestra esta asociación.

Como ejemplo adicional, considérense los dos conjuntos de entidades *préstamo* y *sucursal*. Se puede definir el conjunto de relaciones *sucursal_préstamo* para denotar la asociación entre un préstamo y la sucursal en que se concede ese préstamo.

La asociación entre conjuntos de entidades se conoce como *participación*; es decir, los conjuntos de entidades $E_1,\ E_2, \ldots, E_n$ **participan** en el conjunto de relaciones R. Un **ejemplar de la relación** de un esquema E-R representa una asociación entre las entidades citadas en la empresa real que se está modelando. Como ilustración, la entidad *cliente* López, que tiene el identificador de cliente 67.789.901, y la entidad *préstamo* P-15 participan en un ejemplar de la relación *prestatario*. Este ejemplar de relación representa que, en la empresa real, la persona llamada López que tiene el *id_cliente* 67.789.901 tiene concedido el préstamo que está numerado como P-15.

La función que desempeña una entidad en una relación se denomina **rol** de esa entidad. Como los conjuntos de entidades que participan en un conjunto de relaciones, generalmente, son distintos, los roles están implícitos y no se suelen especificar. Sin embargo, resultan útiles cuando el significado de una relación necesita aclaración. Tal es el caso cuando los conjuntos de entidades de una relación no son distintos; es decir, el mismo conjunto de entidades participa en un conjunto de relaciones más de una vez, con diferentes roles. En este tipo de conjunto de relaciones, que se denomina a veces conjunto de relaciones **recursivo**, son necesarios los nombres explícitos para los roles para especificar la manera

www.detodoprogramacion.com

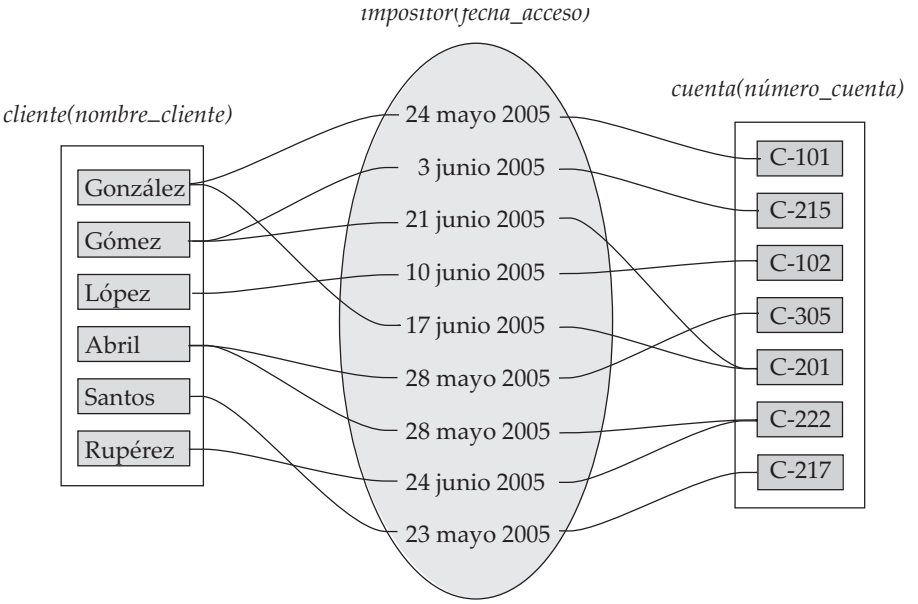


Figura 6.3 *Fecha_acceso* como atributo del conjunto de entidades *impositor*.

en que cada entidad participa en cada ejemplar de la relación. Por ejemplo, considérese un conjunto de entidades *empleado* que almacena información sobre todos los empleados del banco. Se puede tener un conjunto de relaciones *trabaja_para* que se modela mediante pares ordenados de entidades *empleado*. El primer empleado de cada par adopta el rol de *trabajador*, mientras el segundo desempeña el rol de *jefe*. De esta manera, todas las relaciones *trabaja_para* se caracterizan mediante pares (trabajador, jefe); los pares (jefe, trabajador) quedan excluidos.

Una relación puede también tener atributos denominados **atributos descriptivos**. Considérese el conjunto de relaciones *impositor* con los conjuntos de entidades *cliente* y *cuenta*. Se puede asociar el atributo *fecha_acceso* con esta relación para especificar la fecha más reciente de acceso del cliente a la cuenta. La relación *impositor* entre las entidades correspondientes al cliente Santos y la cuenta C-217 tiene el valor "23 mayo 2005" para el atributo *fecha_acceso*, lo que significa que el último día en que Santos accedió a la cuenta C-217 fue el 23 de mayo de 2005.

La Figura 6.3 muestra el conjunto de relaciones *impositor* con el atributo descriptivo *fecha_acceso*; en aras de la sencillez, sólo se muestran algunos atributos de los dos conjuntos de entidades.

Como ejemplo adicional de los atributos descriptivos de las relaciones, supóngase que se tienen los conjuntos de entidades *estudiante* y *asignatura* que participan en el conjunto de relaciones *matriculado*. Puede que se desee almacenar el atributo descriptivo *créditos* con la relación, para registrar si el estudiante se ha matriculado en la asignatura para obtener créditos o sólo como oyente.

Cada ejemplar de una relación de un conjunto de relaciones determinado debe identificarse unívocamente a partir de sus entidades participantes, sin usar los atributos descriptivos. Para comprender esto, supóngase que deseemos representar todas las fechas en las que un cliente ha tenido acceso a una cuenta. El atributo monovalorado *fecha_acceso* sólo puede almacenar una fecha de acceso. No se pueden representar varias fechas de acceso mediante varios ejemplares de la relación entre el mismo cliente y la misma cuenta, ya que los ejemplares de la relación no quedarían identificados unívocamente usando solamente las entidades participantes. La forma correcta de tratar este caso es crear el atributo multivalorado *fechas_acceso*, que puede almacenar todas las fechas de acceso.

Sin embargo, puede haber más de un conjunto de relaciones que implique a los mismos conjuntos de entidades. En nuestro ejemplo, los conjuntos de entidades *cliente* y *préstamo* participan en el conjunto de relaciones *prestatario*. Además, supóngase que cada préstamo deba tener otro cliente que sirva como avalista del préstamo. Entonces los conjuntos de entidades *cliente* y *préstamo* pueden participar en otro conjunto de relaciones: *avalista*.

www.detodoprogramacion.com

Los conjuntos de relaciones *prestatario* y *sucursal_préstamo* proporcionan un ejemplo de conjunto de relaciones **binario**—es decir, uno que implica dos conjuntos de entidades. La mayor parte de los conjuntos de relaciones de los sistemas de bases de datos son binarios. A veces, no obstante, los conjuntos de relaciones implican a más de dos conjuntos de entidades.

Por ejemplo, considérense los conjuntos de entidades *empleado*, *sucursal* y *trabajo*. Ejemplos de entidades *trabajo* pueden ser director, cajero, interventor, etc. Las entidades trabajo pueden tener los atributos *puesto* y *nivel*. El conjunto de relaciones *trabaja_en* entre *empleado*, *sucursal* y *trabajo* es un ejemplo de relación ternaria. Una relación ternaria entre Santos, Navacerrada y director indica que Santos trabaja de director de la sucursal de Navacerrada. Santos también podría actuar como interventor de la sucursal de Centro, lo que estaría representado por otra relación. Podría haber otra relación entre Gómez, Centro y cajero, que indicaría que Gómez trabaja de cajero en la sucursal de Centro.

El número de conjuntos de entidades que participan en un conjunto de relaciones es también el **grado** de ese conjunto de relaciones. Los conjuntos de relaciones binarios tienen grado 2; los conjuntos de relaciones ternarios tienen grado 3.

6.2.3 Atributos

Para cada atributo hay un conjunto de valores permitidos, denominados **dominio** o **conjunto de valores** de ese atributo. El dominio del atributo *nombre_cliente* puede ser el conjunto de todas las cadenas de texto de una cierta longitud. Análogamente, el dominio del atributo *número_préstamo* puede ser el conjunto de todas las cadenas de caracteres de la forma "P-n", donde n es un entero positivo.

Formalmente, cada atributo de un conjunto de entidades es una función que asigna el conjunto de entidades a un dominio. Dado que el conjunto de entidades puede tener varios atributos, cada entidad se puede describir mediante un conjunto de pares (atributo,valor), un par por cada atributo del conjunto de entidades. Por ejemplo, una entidad *cliente* concreta se puede describir mediante el conjunto (*id_cliente*, 67.789.901), (*nombre_cliente*, López), (*calle_cliente*, Mayor), (*ciudad_cliente*, Peguerinos), lo que significa que esa entidad describe a una persona llamada López que tiene el DNI número 67.789.901 y que reside en la calle Mayor de Peguerinos. Ahora se puede ver que existe una integración del esquema abstracto con la empresa real que se está modelando. Los valores de los atributos que describen cada entidad constituyen una parte significativa de los datos almacenados en la base de datos.

Cada atributo, tal y como se usa en el modelo E-R, se puede caracterizar por los siguientes tipos de atributo.

- Atributos **simples** y **compuestos**. En los ejemplos considerados hasta ahora los atributos han sido simples; es decir, no estaban divididos en subpartes. Los atributos **compuestos**, en cambio, se pueden dividir en subpartes (es decir, en otros atributos). Por ejemplo, el atributo *nombre* puede estar estructurado como un atributo compuesto consistente en *nombre*, *primer_apellido* y *segundo_apellido*. Usar atributos compuestos en un esquema de diseño es una buena elección si el usuario desea referirse a un atributo completo en algunas ocasiones y, en otras, solamente a algún componente del atributo. Supóngase que hubiera que sustituir para el conjunto de entidades *cliente* los atributos *calle_cliente* y *ciudad_cliente* por el atributo compuesto *dirección*, con los atributos *calle*, *ciudad*, *provincia*, y *código_postal*[2]. Los atributos compuestos ayudan a agrupar atributos relacionados, lo que hace que los modelos sean más claros.

  Obsérvese también que los atributos compuestos pueden aparecer como una jerarquía. En el atributo compuesto dirección, el componente *calle* puede dividirse a su vez en *número_calle*, *nombre_calle* y *piso*. La Figura 6.4 muestra estos ejemplos de atributos compuestos para el conjunto de entidades *cliente*.

- Atributos **monovalorados** y **multivalorados**. Todos los atributos que se han especificado en los ejemplos anteriores tienen un único valor para cada entidad concreta. Por ejemplo, el atributo *número_préstamo* para una entidad préstamo específica hace referencia a un único número de préstamo. Se dice que estos atributos son **monovalorados**. Puede haber ocasiones en las que

2. Se asume el formato de *calle_cliente* y *dirección* usado en España, que incluye un código postal numérico llamado "código postal".

www.detodoprogramacion.com

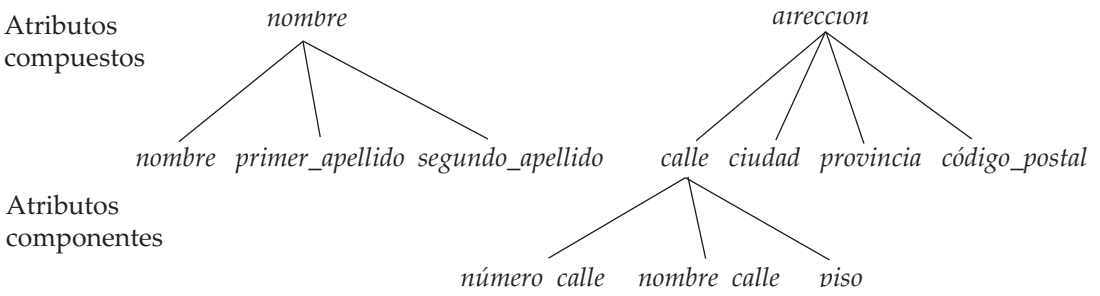


Figura 6.4 Atributos compuestos *nombre* y *dirección*.

un atributo tenga un conjunto de valores para una entidad concreta. Considérese un conjunto de entidades *empleado* con el atributo *número_teléfono*. Cada empleado puede tener cero, uno o varios números de teléfono, y empleados diferentes pueden tener diferente cantidad de teléfonos. Se dice que este tipo de atributo es **multivalorado**. Como ejemplo adicional, el atributo *nombre_subordinado* del conjunto de entidades *empleado* es multivalorado, ya que cada empleado podría tener cero, uno o más subordinados.

Si resulta necesario, se pueden establecer apropiadamente límites inferior y superior al número de valores en el atributo **multivalorado**. Por ejemplo, el banco puede limitar a dos el número de números de teléfono que se guardan por cliente. El establecimiento de límites en este caso expresa que el atributo *número_teléfono* del conjunto de entidades *cliente* puede tener entre cero y dos valores.

- Atributos **derivados**. El valor de este tipo de atributo se puede obtener a partir del valor de otros atributos o entidades relacionados. Por ejemplo, supóngase que el conjunto de entidades *cliente* que tiene un atributo *préstamos* que representa el número de préstamos que cada cliente tiene concedidos en el banco. Ese atributo se puede obtener contando el número de entidades *préstamo* asociadas con cada cliente.

  Como ejemplo adicional, supóngase que el conjunto de entidades *cliente* tiene el atributo *edad*, que indica la edad del cliente. Si el conjunto de entidades *cliente* tiene también un atributo *fecha_de_nacimiento*, se puede calcular *edad* a partir de *fecha_de_nacimiento* y de la fecha actual. Por tanto, *edad* es un atributo derivado. En este caso, *fecha_de_nacimiento* puede considerarse un atributo *básico*, o *almacenado*. El valor de los atributos derivados no se almacena, sino que se calcula cada vez que hace falta.

Los atributos toman valores **nulos** cuando las entidades no tienen ningún valor para ese atributo. El valor *nulo* también puede indicar "no aplicable"—es decir, que el valor no existe para esa entidad. Por ejemplo, una persona puede no tener un segundo nombre de pila. *Nulo* puede también designar que el valor del atributo es desconocido. Un valor desconocido puede ser *falta* (el valor existe pero no se tiene esa información) o *desconocido* (no se sabe si ese valor existe realmente o no).

Por ejemplo, si el valor *nombre* de un *cliente* dado es *nulo*, se da por supuesto que el valor es falta, ya que todos los clientes deben tener nombre. Un valor *nulo* para el atributo *piso* puede significar que la dirección no incluye un piso (no aplicable), que existe el valor piso pero no se conoce cuál es (falta), o que no se sabe si el valor piso forma parte o no de la dirección del cliente (desconocido).

6.3 Restricciones

Un esquema de desarrollo E-R puede definir ciertas restricciones a las que el contenido de la base de datos se debe adaptar. En este apartado se examinan la correspondencia de cardinalidades, las restricciones de claves y las restricciones de participación.

www.detodoprogramacion.com

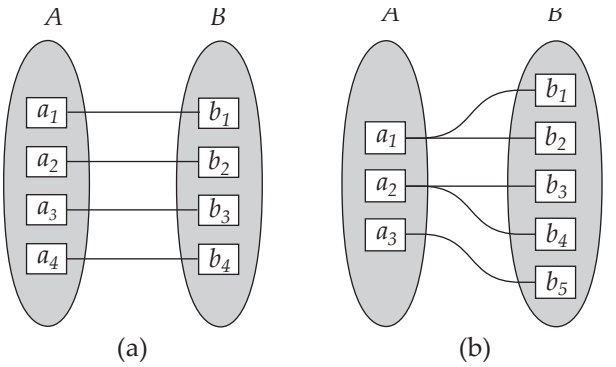


Figura 6.5 Correspondencia de cardinalidades. (a) Uno a uno. (b) Uno a varios.

6.3.1 Correspondencia de cardinalidades

La **correspondencia de cardinalidades**, o razón de cardinalidad, expresa el número de entidades a las que otra entidad se puede asociar mediante un conjunto de relaciones.

La correspondencia de cardinalidades resulta muy útil para describir conjuntos de relaciones binarias, aunque pueda contribuir a la descripción de conjuntos de relaciones que impliquen más de dos conjuntos de entidades. En este apartado se centrará la atención únicamente en los conjuntos de relaciones binarias.

Para un conjunto de relaciones binarias R entre los conjuntos de entidades A y B, la correspondencia de cardinalidades debe ser una de las siguientes:

- **Uno a uno** Cada entidad de A se asocia, *a lo sumo*, con una entidad de B, y cada entidad en B se asocia, *a lo sumo*, con una entidad de A (véase la Figura 6.5a).
- **Uno a varios** Cada entidad de A se asocia con cualquier número (cero o más) de entidades de B. Cada entidad de B, sin embargo, se puede asociar, *a lo sumo*, con una entidad de A (véase la Figura 6.5b).
- **Varios a uno** Cada entidad de A se asocia, *a lo sumo*, con una entidad de B. Cada entidad de B, sin embargo, se puede asociar con cualquier número (cero o más) de entidades de A (véase la Figura 6.6a).
- **Varios a varios** Cada entidad de A se asocia con cualquier número (cero o más) de entidades de B, y cada entidad de B se asocia con cualquier número (cero o más) de entidades de A (véase la Figura 6.6b).

La correspondencia de cardinalidades adecuada para un conjunto de relaciones dado depende, obviamente, de la situación del mundo real que el conjunto de relaciones modele.

Como ilustración, considérese el conjunto de relaciones *prestatario*. Si, en un banco dado, cada préstamo sólo puede pertenecer a un cliente y cada cliente puede tener varios préstamos, entonces el conjunto de relaciones de *cliente* a *préstamo* es uno a varios. Si cada préstamo puede pertenecer a varios clientes (como los préstamos solicitados conjuntamente por varios socios de un negocio) el conjunto de relaciones es varios a varios. La Figura 6.2 muestra este tipo de relación.

6.3.2 Claves

Es necesario tener una forma de especificar la manera de distinguir las entidades pertenecientes a un conjunto de entidades dado. Conceptualmente cada entidad es distinta; desde el punto de vista de las bases de datos, sin embargo, la diferencia entre ellas se debe expresar en términos de sus atributos.

Por lo tanto, los valores de los atributos de cada entidad deben ser tales que permitan *identificar unívocamente* a esa entidad. En otras palabras, no se permite que ningún par de entidades de un conjunto de entidades tenga exactamente el mismo valor en todos sus atributos.

www.detodoprogramacion.com

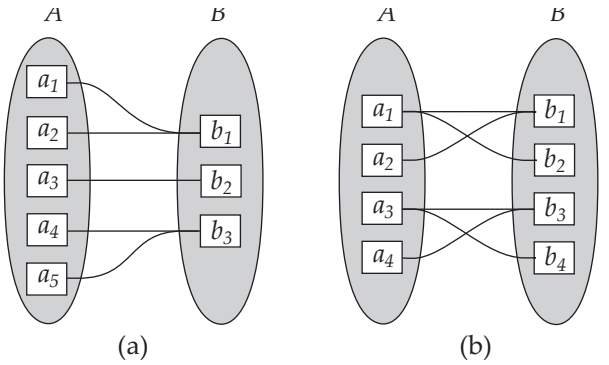


Figura 6.6 Correspondencia de cardinalidades. (a) Varios a uno. (b) Varios a varios.

Las *claves* permiten identificar un conjunto de atributos que resulta suficiente para distinguir las entidades entre sí. Las claves también ayudan a identificar unívocamente las relaciones y, por tanto, a distinguir las relaciones entre sí.

6.3.2.1 Conjuntos de entidades

Una **superclave** es un conjunto de uno o más atributos que, tomados conjuntamente, permiten identificar de forma unívoca una entidad del conjunto de entidades. Por ejemplo, el atributo *id_cliente* del conjunto de entidades *cliente* es suficiente para distinguir una entidad *cliente* de las demá. Por tanto, *id_cliente* es una superclave. Análogamente, la combinación de *nombre_cliente* e *id_cliente* es una superclave del conjunto de entidades *cliente*. El atributo *nombre_cliente* de *cliente* no es una superclave, ya que varias personas pueden tener el mismo nombre.

El concepto de superclave no es suficiente para lo que aquí se propone ya que, como se ha visto, las superclaves pueden contener atributos innecesarios. Si *C* es una superclave, entonces también lo es cualquier superconjunto de *C*. A menudo interesan las superclaves para las que ningún subconjunto propio es superclave. Esas superclaves mínimas se denominan **claves candidatas**.

Es posible que conjuntos distintos de atributos puedan servir como clave candidata. Supóngase que una combinación de *nombre_cliente* y *calle_cliente* sea suficiente para distinguir entre los miembros del conjunto de entidades *cliente*. Entonces, tanto *id_cliente* como *nombre_cliente*, *calle_cliente* son claves candidatas. Aunque los atributos *id_cliente* y *nombre_cliente* juntos puedan diferenciar las entidades *cliente*, su combinación no forma una clave candidata, ya que el atributo *id_cliente* por sí solo es una clave candidata.

Se usa el término **clave primaria** para denotar la clave candidata elegida por el diseñador de la base de datos como elemento principal de identificación de las entidades pertenecientes a un conjunto de entidades. Las claves (primarias, candidatas y superclaves) son propiedades del conjunto de entidades, más que de cada una de las entidades. Dos entidades cualesquiera del conjunto no pueden tener el mismo valor de los atributos de su clave al mismo tiempo. La designación de una clave representa una restricción de la empresa real que se está modelando.

Las claves candidatas se deben escoger con cuidado. Como se ha indicado, el nombre de cada persona es obviamente insuficiente, ya que puede haber muchas personas con el mismo nombre. En España, el atributo DNI de cada persona puede ser una clave candidata. Como los no ciudadanos de en España no tienen DNI, las empresas con trabajadores extranjeros deben generar sus propios identificadores únicos. Una alternativa es usar como clave alguna combinación única de otros atributos.

La clave primaria se debe escoger de manera que sus atributos nunca, o muy raramente, cambien. Por ejemplo, el campo dirección de cada persona no debe formar parte de la clave primaria, ya que es probable que cambie. El número de DNI, por otra parte, es seguro que no cambiará. Los identificadores únicos generados por las empresas generalmente no cambian, excepto si se fusionan dos empresas; en tal caso, las dos empresas pueden haber emitido el mismo identificador y será necesario reasignar los identificadores para asegurarse de que sean únicos.

www.detodoprogramacion.com

6.3.2.2 Conjuntos de relaciones

La clave primaria de cada conjunto de entidades permite distinguir entre las diferentes entidades del conjunto. Se necesita un mecanismo parecido para distinguir entre las diferentes relaciones de cada conjunto de relaciones.

Sea R un conjunto de relaciones que involucra los conjuntos de entidades $E_1,\ E_2, \ldots, E_n$. Sea *clave_primaria* (E_i) el conjunto de atributos que forman la clave primaria del conjunto de entidades E_i. Por ahora se dará por supuesto que los nombres de los atributos de todas las claves primarias son únicos y que cada conjunto de entidades participa sólo una vez en la relación. La composición de la clave primaria de un conjunto de relaciones depende del conjunto de atributos asociado con el conjunto de relaciones R.

Si el conjunto de relaciones R no tiene atributos asociados, entonces el conjunto de atributos

$$\textit{clave_primaria}(E_1) \cup \textit{clave_primaria}(E_2) \cup \cdots \cup \textit{clave_primaria}(E_n)$$

describe una relación concreta del conjunto R.

Si el conjunto de relaciones R tiene asociados los atributos $a_1, a_2, \ldots, a_m$, entonces el conjunto de atributos

$$\textit{clave_primaria}(E_1) \cup \textit{clave_primaria}(E_2) \cup \cdots \cup \textit{clave_primaria}(E_n)\ \cup \{a_1, a_2, \ldots, a_m\}$$

describe una relación concreta del conjunto R.

En ambos casos, el conjunto de atributos

$$\textit{clave_primaria}(E_1) \cup \textit{clave_primaria}(E_2) \cup \cdots \cup \textit{clave_primaria}(E_n)$$

forma una superclave del conjunto de relaciones.

En el caso de que los nombres de los atributos de las claves primarias no sean únicos en todos los conjuntos de entidades, hay que renombrar los atributos para distinguirlos; el nombre del conjunto de entidades combinado con el del atributo formará un nombre único. En el caso de que un conjunto de entidades participe más de una vez en el conjunto de relaciones (como en la relación *trabaja_para* del Apartado 6.2.2), se usa el nombre del rol en lugar del nombre del conjunto de entidades para formar un nombre de atributo único.

La estructura de la clave primaria para el conjunto de relaciones depende de la correspondencia de cardinalidades del conjunto de relaciones. Como ejemplo, considérense los conjuntos de entidades *cliente* y *cuenta* y el conjunto de relaciones *impositor*, con el atributo *fecha_acceso* del Apartado 6.2.2. Supóngase que el conjunto de relaciones es varios a varios. Entonces, la clave primaria de *impositor* consiste en la unión de las claves primarias de *cliente* y de *cuenta*. Sin embargo, si un cliente sólo puede tener una cuenta—es decir, si la relación *impositor* es varios a uno de *cliente* a *cuenta*—entonces la clave primaria de *impositor* es simplemente la clave primaria de *cliente*. Análogamente, si la relación es varios a uno de *cuenta* a *cliente*—es decir, cada cuenta pertenece, a lo sumo, a un cliente— entonces la clave primaria de *impositor* es simplemente la clave primaria de *cuenta*. Para relaciones uno a uno se puede usar cualquiera de las claves primarias.

Para las relaciones no binarias, si no hay restricciones de cardinalidad, la superclave formada como se ha descrito anteriormente en este apartado es la única clave candidata, y se elige como clave primaria. La elección de la clave primaria resulta más complicada si hay restricciones de cardinalidad. Dado que no se ha estudiado la manera de especificar las restricciones de cardinalidad en relaciones no binarias, no se estudiará más este aspecto en este capítulo. Se considerará este aspecto con más detalle en el Apartado 7.4.

6.3.3 Restricciones de participación

Se dice que la participación de un conjunto de entidades E en un conjunto de relaciones R es **total** si cada entidad de E participa, al menos, en una relación de R. Si sólo algunas entidades de E participan en relaciones de R, se dice que la participación del conjunto de entidades E en la relación R es **parcial**. Por ejemplo, se puede esperar que cada entidad *préstamo* esté relacionada, al menos, con un cliente mediante

www.detodoprogramacion.com

la relación *prestatario*. Por tanto, la participación de *préstamo* en el conjunto de relaciones *prestatario* es total. En cambio, un individuo puede ser cliente del banco tenga o no tenga concedido algún préstamo en el banco. Por tanto, es posible que sólo algunas de las entidades *cliente* estén relacionadas con el conjunto de entidades *préstamo* mediante la relación *prestatario*, y la participación de *cliente* en la relación *prestatario* es, por tanto, parcial.

6.4 Diagramas entidad-relación

Como se vio brevemente en el Apartado 1.3.3, los **diagramas E-R** pueden expresar gráficamente la estructura lógica general de las bases de datos. Los diagramas E-R son sencillos y claros—cualidades que pueden ser responsables en gran parte de la popularidad del modelo E-R. Estos diagramas constan de los siguientes componentes principales:

- **Rectángulos**, que representan conjuntos de entidades.
- **Elipses**, que representan atributos.
- **Rombos**, que representan conjuntos de relaciones.
- **Líneas**, que unen los atributos con los conjuntos de entidades y los conjuntos de entidades con los conjuntos de relaciones.
- **Elipses dobles**, que representan atributos multivalorados.
- **Elipses discontinuas**, que denotan atributos derivados.
- **Líneas dobles**, que indican participación total de una entidad en un conjunto de relaciones.
- **Rectángulos dobles**, que representan conjuntos de entidades débiles (se describirán posteriormente en el Apartado 6.6).

Considérese el diagrama entidad-relación de la Figura 6.7, que consiste en dos conjuntos de entidades, *cliente* y *préstamo*, relacionados mediante el conjunto de relaciones binarias *prestatario*. Los atributos asociados con *cliente* son *id_cliente*, *nombre_cliente*, *calle_cliente*, y *ciudad_cliente*. Los atributos asociados con *préstamo* son *número_préstamo* e *importe*. En la Figura 6.7 los atributos del conjunto de entidades que son miembros de la clave primaria están subrayados.

El conjunto de relaciones *prestatario* puede ser varios a varios, uno a varios, varios a uno o uno a uno. Para distinguir entre estos tipos, se dibuja o una línea dirigida (→) o una línea no dirigida (—) entre el conjunto de relaciones y el conjunto de entidades en cuestión.

- Una línea dirigida desde el conjunto de relaciones *prestatario* al conjunto de entidades *préstamo* especifica que *prestatario* es un conjunto de relaciones uno a uno o varios a uno desde *cliente* a *préstamo*; *prestatario* no puede ser un conjunto de relaciones varios a varios ni uno a varios desde *cliente* a *préstamo*.

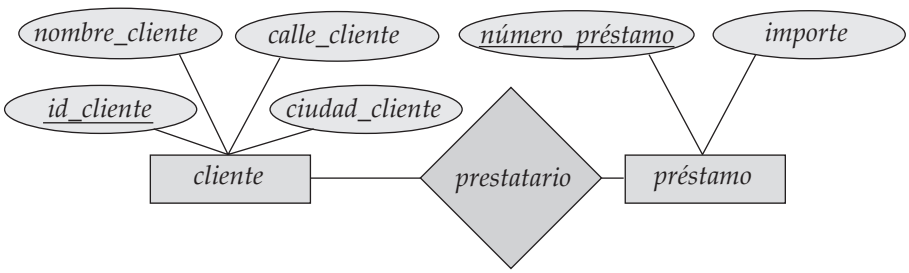


Figura 6.7 Diagrama E-R correspondiente a clientes y préstamos.

www.detodoprogramacion.com

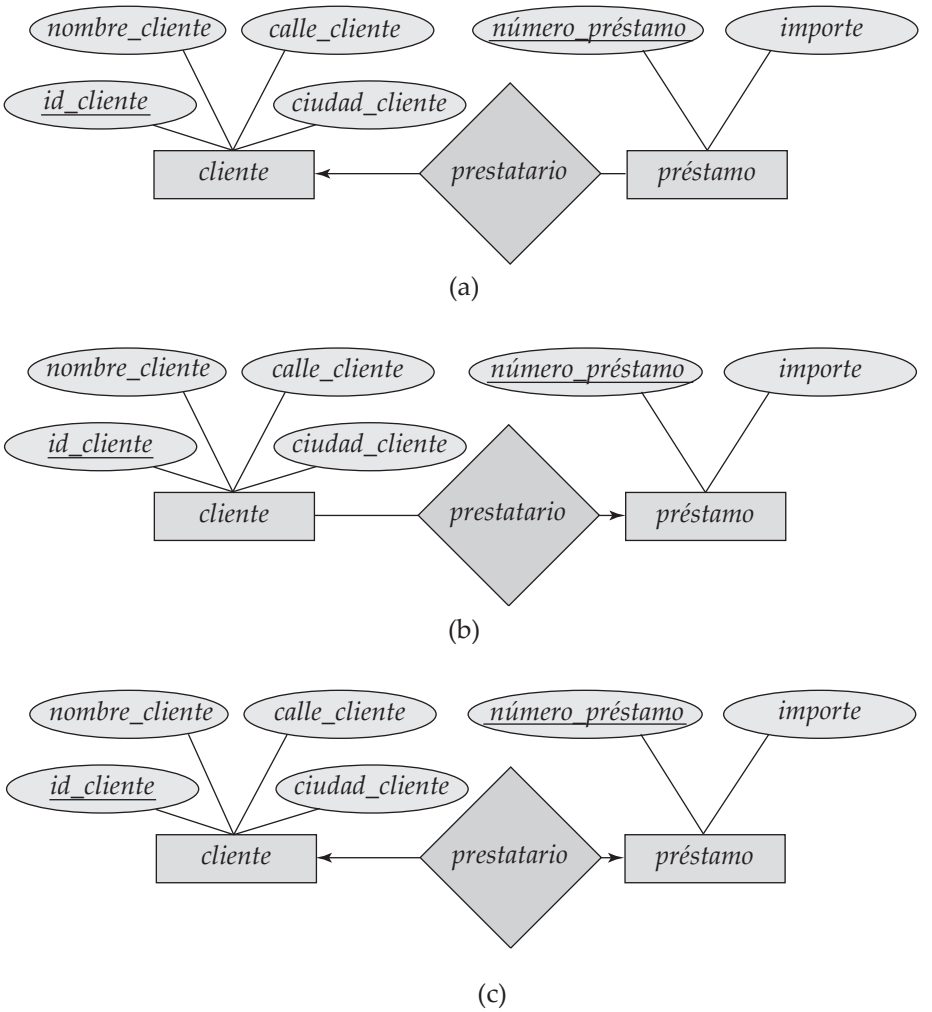


Figura 6.8 Relaciones. (a) Uno a varios. (b) Varios a uno. (c) Uno a uno.

- Una línea no dirigida desde el conjunto de relaciones *prestatario* al conjunto de relaciones *préstamo* especifica que *prestatario* es un conjunto de relaciones varios a varios o uno a varios desde *cliente* a *préstamo*.

Volviendo al diagrama E-R de la Figura 6.7, puede verse que el conjunto de relaciones *prestatario* es varios a varios. Si el conjunto de relaciones *prestatario* fuera uno a varios, desde *cliente* a *préstamo*, entonces la línea desde *prestatario* a *cliente* sería dirigida, con una flecha que apuntaría al conjunto de

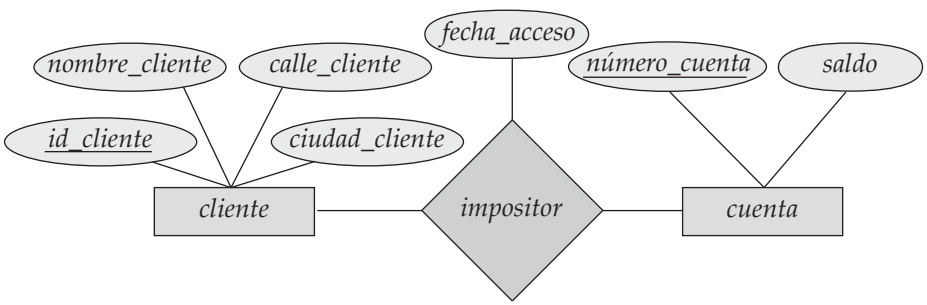


Figura 6.9 Diagrama E-R con un atributo unido a un conjunto de relaciones.

www.detodoprogramacion.com

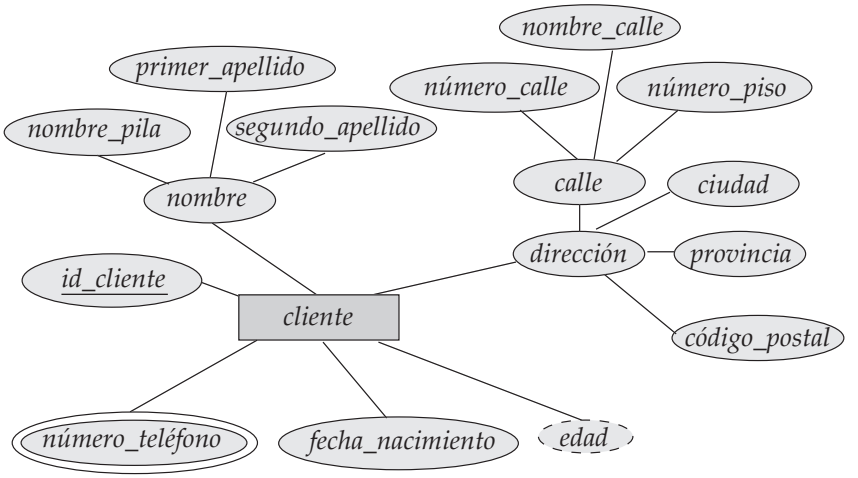


Figura 6.10 Diagrama E-R con atributos compuestos, multivalorados y derivados.

entidades *cliente* (Figura 6.8a). Análogamente, si el conjunto de relaciones *prestatario* fuera varios a uno desde *cliente* a *préstamo*, entonces la línea desde *prestatario* a *préstamo* tendría una flecha que apuntaría al conjunto de entidades *préstamo* (Figura 6.8b). Finalmente, si el conjunto de relaciones *prestatario* fuera uno a uno, entonces las dos líneas que salen de *prestatario* tendrían flecha: una que apuntaría al conjunto de entidades *préstamo* y otra que apuntaría al conjunto de entidades *cliente* (Figura 6.8c).

Si un conjunto de relaciones también tiene asociados algunos atributos, entonces esos atributos se unen con el conjunto de relaciones. Por ejemplo, en la Figura 6.9, se tiene el atributo descriptivo *fecha_acceso* unido al conjunto de relaciones *impositor* para especificar la fecha del último acceso del cliente a esa cuenta.

La Figura 6.10 muestra cómo se pueden representar los atributos compuestos en la notación E-R. En este caso, el atributo compuesto *nombre*, con los atributos componentes *nombre_pila*, *primer_apellido* y *segundo_apellido* sustituye al atributo simple *nombre_cliente* de *cliente*. Además, el atributo compuesto *dirección*, cuyos atributos componentes son *calle*, *ciudad*, *provincia* y *código_postal*, sustituye a los atributos *calle_cliente* y *ciudad_cliente* de *cliente*. El atributo *calle* es por sí mismo un atributo compuesto cuyos atributos componentes son *número_calle*, *nombre_calle* y *número_piso*.

La Figura 6.10 también muestra un atributo multivalorado, *número_teléfono*, indicado por una elipse doble, y un atributo derivado, *edad*, indicado por una elipse discontinua.

En los diagramas E-R los roles se indican mediante etiquetas en las líneas que unen los rombos con los rectángulos. La Figura 6.11 muestra los indicadores de roles *director* y *trabajador* entre el conjunto de entidades *empleado* y el conjunto de relaciones *trabaja_para*.

Los conjuntos de relaciones no binarias se pueden especificar fácilmente en los diagramas E-R. La Figura 6.12 consta de tres conjuntos de entidades *empleado*, *trabajo* y *sucursal*, relacionados mediante el conjunto de relaciones *trabaja_en*.

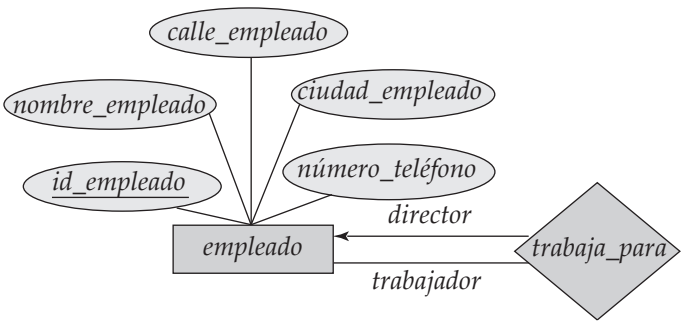


Figura 6.11 Diagrama E-R con indicadores de roles.

www.detodoprogramacion.com

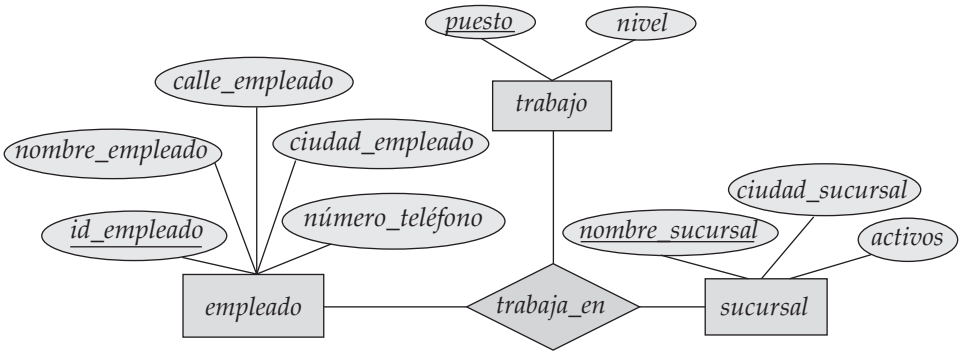


Figura 6.12 Diagrama E-R con una relación ternaria.

En el caso de conjuntos de relaciones no binarias se pueden especificar algunos tipos de relaciones varios a uno. Supóngase que un empleado puede tener, a lo sumo, un trabajo en cada sucursal (por ejemplo, Santos no puede ser director e interventor en la misma sucursal). Esta restricción se puede especificar mediante una flecha que apunte a *trabajo* en el arco de *trabaja_en*.

Como máximo se permite una flecha desde cada conjunto de relaciones, ya que los diagramas E-R con dos o más flechas salientes de cada conjunto de relaciones no binarias se pueden interpretar de dos formas. Supónganse que hay un conjunto de relaciones R entre los conjuntos de entidades $A_1, A_2, \ldots, A_n$ y que las únicas flechas están en los arcos de los conjuntos de entidades $A_{i+1}, A_{i+2}, \ldots, A_n$. Entonces, las dos interpretaciones posibles son:

1. Una combinación concreta de entidades de $A_1, A_2, \ldots, A_i$ se puede asociar, a lo sumo, con una combinación de entidades de $A_{i+1}, A_{i+2}, \ldots, A_n$. Por tanto, la clave primaria de la relación R se puede crear mediante la unión de las claves primarias de $A_1, A_2, \ldots, A_i$.
2. Para cada conjunto de entidades A_k, $i < k \leq n$, cada combinación de las entidades de los otros conjuntos de entidades se puede asociar, a lo sumo, con una entidad de A_k. Cada conjunto $\{A_1, A_2, \ldots, A_{k-1}, A_{k+1}, \ldots, A_n\}$, para $i < k \leq n$, forma, entonces, una clave candidata.

Cada una de estas interpretaciones se ha usado en diferentes libros y sistemas. Para evitar confusiones, sólo se permite una flecha saliente de cada conjunto de relaciones, en cuyo caso las dos interpretaciones son equivalentes. En el Capítulo 7 (Apartado 7.4) se estudia el concepto de *dependencia funcional*, que permite especificar cualquiera de estas dos interpretaciones sin ambigüedad.

En los diagramas E-R se usan líneas dobles para indicar que la participación de un conjunto de entidades en un conjunto de relaciones es total; es decir, cada entidad del conjunto de entidades aparece, al menos, en una relación de ese conjunto de relaciones. Por ejemplo, considérese la relación *prestatario* entre clientes y préstamos. Una línea doble de *préstamo* a *prestatario*, como en la Figura 6.13, indica que cada préstamo debe tener, al menos, un cliente asociado.

Los diagramas E-R también ofrecen una manera de indicar restricciones más complejas sobre el número de veces que cada entidad participa en las relaciones de un conjunto de relaciones. Un segmento entre un conjunto de entidades y un conjunto de relaciones binarias puede tener unas cardinalidades

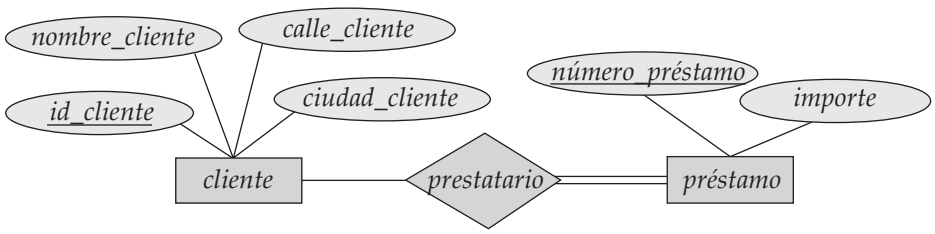


Figura 6.13 Participación total de un conjunto de entidades en un conjunto de relaciones.

www.detodoprogramacion.com

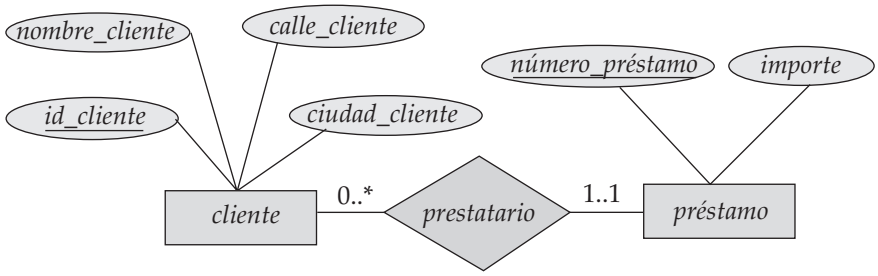


Figura 6.14 Límites de cardinalidad en los conjuntos de relaciones.

mínima y máxima asociadas , que se muestran de la forma $min..max$, donde min es la cardinalidad mínima y max es la máxima. Un valor mínimo de 1 indica una participación total del conjunto de entidades en el conjunto de relaciones. Un valor máximo de 1 indica que la entidad participa, a lo sumo, en una relación, mientras que un valor máximo de $*$ indica que no hay límite. Téngase en cuenta que una etiqueta $1..*$ en un segmento es equivalente a una línea doble.

Por ejemplo, considérese la Figura 6.14. El segmento entre *préstamo* y *prestatario* tiene una restricción de cardinalidad de $1..1$, que significa que tanto la cardinalidad mínima como la máxima son 1. Es decir, cada préstamo debe tener exactamente un cliente asociado. El límite $0..*$ en el segmento de *cliente* a *prestatario* indica que cada cliente puede no tener ningún préstamo o tener varios. Por tanto, la relación *prestatario* es uno a varios de *cliente* a *préstamo* y, además, la participación de *préstamo* en *prestatario* es total.

Es fácil malinterpretar $0..*$ en el segmento entre *cliente* y *prestatario* y pensar que la relación *prestatario* es varios a uno de *cliente* a *préstamo*—esto es exactamente lo contrario a la interpretación correcta.

Si los dos segmentos de una relación binaria tienen un valor máximo de 1, la relación es uno a uno. Si se hubiese especificado un límite de cardinalidad de $1..*$ en el segmento entre *cliente* y *prestatario*, se estaría diciendo que cada cliente debe tener, al menos, un préstamo.

6.5 Aspectos del diseño entidad-relación

Los conceptos de conjunto de entidades y de conjunto de relaciones no son precisos, y es posible definir el conjunto de entidades y las relaciones entre ellas de diferentes formas. En este apartado se examinan aspectos básicos del diseño de esquemas de bases de datos E-R. El Apartado 6.7.4 trata el proceso de diseño con más detalle.

6.5.1 Uso de conjuntos de entidades y de atributos

Considérese el conjunto de entidades *empleado* con los atributos *nombre_empleado* y *número_teléfono*. Se puede argumentar que un *teléfono* es una entidad en sí misma con los atributos *número_teléfono* y *ubicación*; la ubicación puede ser la sucursal o el domicilio donde el teléfono está instalado, y se pueden representar los teléfonos móviles (celulares) mediante el valor "móvil". Si se adopta este punto de vista, hay que redefinir el conjunto de entidades *empleado* como:

- El conjunto de entidades *empleado* con los atributos *id_empleado* y *nombre_empleado*.
- El conjunto de entidades *teléfono* con los atributos *número_teléfono* y *ubicación*.
- La relación *teléfono_empleado*, que denota la asociación entre los empleados y sus teléfonos.

Estas alternativas se muestran en la Figura 6.15.

¿Cuál es, entonces, la diferencia principal entre estas dos definiciones de empleado? Tratar el teléfono como el atributo *número_teléfono* implica que cada empleado tiene exactamente un número de teléfono. Tratar el teléfono como la entidad *teléfono* permite que los empleados tengan varios números de telé-

www.detodoprogramacion.com

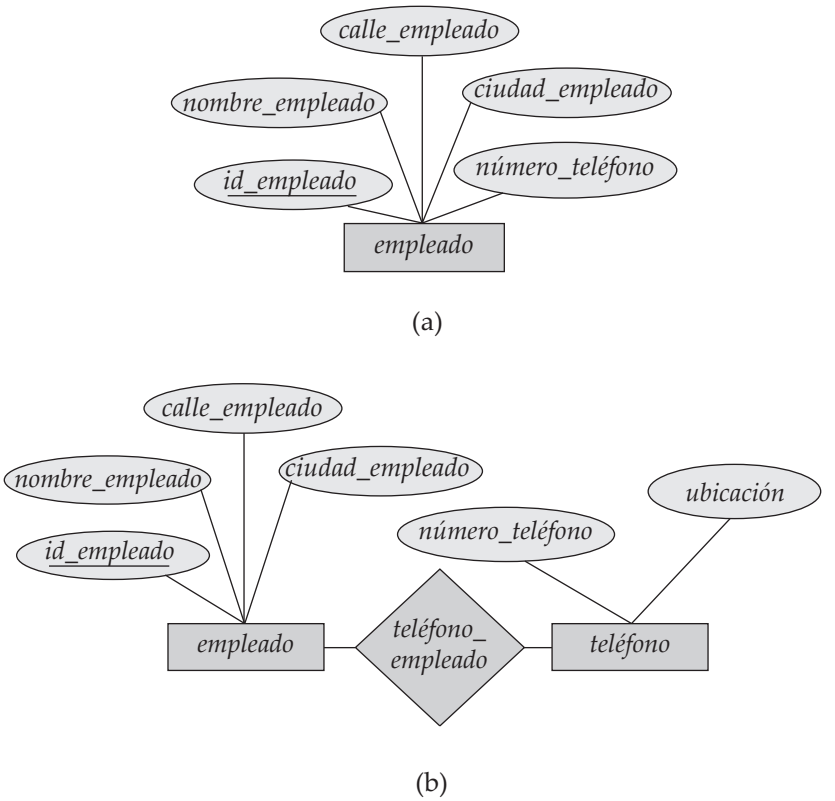


Figura 6.15 Alternativas para *empleado* y *teléfono*.

fono (o ninguno) asociados. Sin embargo, se puede definir en su lugar *número_teléfono* como atributo multivalorado para permitir varios teléfonos por empleado.

La diferencia principal es que tratar el teléfono como entidad modela mejor una situación en la que se puede querer guardar información extra sobre el teléfono, como su ubicación, su tipo (móvil, videoteléfono o fijo) o las personas que lo comparten. Por tanto, tratar el teléfono como entidad es más general que tratarlo como atributo y resulta adecuado cuando la generalidad pueda ser útil.

En cambio, no sería apropiado tratar el atributo *nombre_empleado* como entidad; es difícil argumentar que *nombre_empleado* sea una entidad en sí misma (a diferencia de lo que ocurre con *teléfono*). Así pues, resulta adecuado tener *nombre_empleado* como atributo del conjunto de entidades *empleado*.

Por tanto, se suscitan naturalmente dos preguntas: ¿qué constituye un atributo? y ¿qué constituye un conjunto de entidades? Desafortunadamente no hay respuestas sencillas. Las distinciones dependen principalmente de la estructura de la empresa real que se esté modelando y de la semántica asociada con el atributo en cuestión.

Un error común es usar la clave primaria de un conjunto de entidades como atributo de otro conjunto de entidades en lugar de usar una relación. Por ejemplo, es incorrecto modelar *id_cliente* como atributo de *préstamo*, aunque cada préstamo tenga sólo un cliente. La relación *prestatario* es la forma correcta de representar la conexión entre los préstamos y los clientes, ya que hace explícita su conexión, en lugar de dejarla implícita mediante un atributo.

Otro error relacionado con éste que se comete a veces es escoger los atributos de clave primaria de los conjuntos de entidades relacionados como atributos del conjunto de relaciones. Por ejemplo, *número_préstamo* (el atributo de clave primaria de *préstamo*) e *id_cliente* (la clave primaria de *cliente*) no deben aparecer como atributos de la relación *prestatario*. Esto no es adecuado, ya que los atributos de clave primaria están implícitos en el conjunto de relaciones[3].

3. Cuando se crea un esquema de relación a partir del esquema E-R los atributos pueden aparecer en una tabla creada a partir del conjunto de relaciones *prestatario*, como se verá más adelante; no obstante, no deben aparecer en el conjunto de relaciones *prestatario*.

www.detodoprogramacion.com

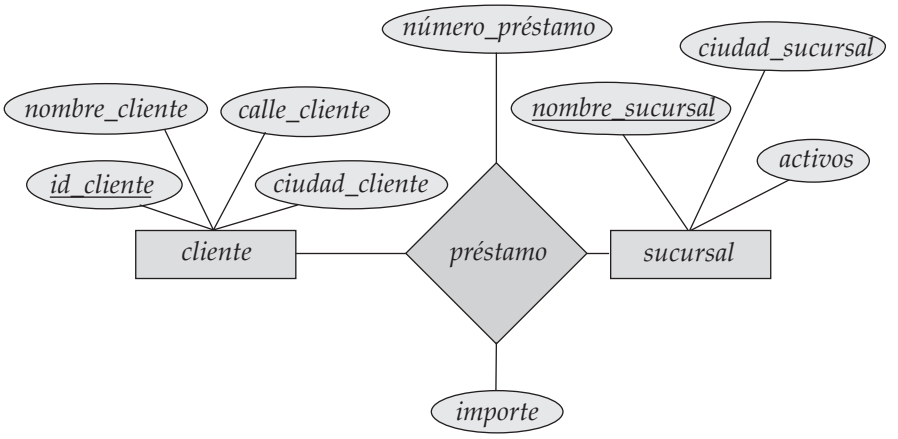


Figura 6.16 *préstamo* como conjunto de relaciones.

6.5.2 Uso de los conjuntos de entidades y de los conjuntos de relaciones

No siempre está claro si es mejor expresar un objeto mediante un conjunto de entidades o mediante un conjunto de relaciones. En el Apartado 6.2.1 se dio por supuesto que los préstamos bancarios se modelaban como entidades. Una alternativa es modelar los préstamos no como entidades, sino como relaciones entre los clientes y las sucursales, con *número_préstamo* e *importe* como atributos descriptivos, como puede verse en la Figura 6.16. Cada préstamo se representa mediante una relación entre un cliente y una sucursal.

Si cada préstamo se concede exactamente a un cliente y se asocia exactamente con una sucursal, se puede encontrar satisfactorio el diseño en el que cada préstamo se representa como una relación. Sin embargo, con este diseño, no se puede representar convenientemente una situación en la que un préstamo se conceda a varios clientes conjuntamente. Para tratar esta situación se debe definir otra relación para cada prestatario de ese préstamo conjunto. Entonces habrá que replicar los valores de los atributos descriptivos *número_préstamo* e *importe* en cada una de esas relaciones. Cada una de esas relaciones debe, por supuesto, tener el mismo valor para los atributos descriptivos *número_préstamo* e *importe*.

Se plantean dos problemas como resultado de esta réplica: (1) los datos se almacenan varias veces, lo que hace que se desperdicie espacio de almacenamiento, y (2) las actualizaciones pueden dejar los datos en un estado inconsistente, en el que los valores de atributos que se supone que tienen el mismo valor difieran. El problema de cómo evitar esta réplica se trata formalmente mediante la *teoría de la normalización*, que se aborda en el Capítulo 7.

El problema de la réplica de los atributos *número_préstamo* e *importe* no aparece en el diseño original del Apartado 6.4, ya que allí *préstamo* es un conjunto de entidades.

Un criterio para determinar si se debe usar un conjunto de entidades o un conjunto de relaciones puede ser escoger un conjunto de relaciones para describir las acciones que ocurran entre entidades. Este enfoque también puede ser útil para decidir si ciertos atributos se pueden expresar mejor como relaciones.

6.5.3 Conjuntos de relaciones binarias y n-arias

Las relaciones en las bases de datos suelen ser binarias. Puede que algunas relaciones que no parecen ser binarias se puedan representar mejor mediante varias relaciones binarias. Por ejemplo, se puede crear la relación ternaria *padres*, que relaciona a cada hijo con su padre y con su madre. Sin embargo, esa relación se puede representar mediante dos relaciones binarias, *padre* y *madre*, que relacionan a cada hijo con su padre y con su madre por separado. El uso de las dos relaciones *padre* y *madre* permite el registro de la madre del niño aunque no se conozca la identidad del padre; si se usara en la relación ternaria *padres* se necesitaría un valor nulo. En este caso es preferible usar conjuntos de relaciones binarias.

www.detodoprogramacion.com

De hecho, siempre es posible sustituir los conjuntos de relaciones no binarias ($n$aria, para $n > 2$) por varios conjuntos de relaciones binarias. Por simplificar, considérense el conjunto de relaciones abstracto ternario ($n = 3$) R y los conjuntos de entidades A, B, y C. Se sustituye el conjunto de relaciones R por un conjunto de entidades E y se crean tres conjuntos de relaciones como se muestra en la Figura 6.17:

- P_A, que relaciona E y A
- P_B, que relaciona E y B
- P_C, que relaciona E y C

Si el conjunto de relaciones R tiene atributos, éstos se asignan al conjunto de entidades E; además, se crea un atributo de identificación especial para E (ya que se debe poder distinguir las diferentes entidades de cada conjunto de entidades con base en los valores de sus atributos). Para cada relación (a_i, b_i, c_i) del conjunto de relaciones R se crea una nueva entidad e_i del conjunto de entidades E. Luego, en cada uno de los tres nuevos conjuntos de relaciones, se inserta una relación del modo siguiente:

- (e_i, a_i) en R_A
- (e_i, b_i) en R_B
- (e_i, c_i) en R_C

Este proceso se puede generalizar de forma directa a los conjuntos de relaciones n-arias. Por tanto, conceptualmente, se puede restringir el modelo E-R para que sólo incluya conjuntos de relaciones binarias. Sin embargo, esta restricción no siempre es deseable.

- Es posible que sea necesario crear un atributo de identificación para que el conjunto de entidades represente el conjunto de relaciones. Este atributo, junto con los conjuntos de relaciones adicionales necesarios, incrementa la complejidad del diseño y (como se verá en el Apartado 6.9) los requisitos globales de almacenamiento.
- Un conjunto de relaciones n-arias muestra más claramente que varias entidades participan en una sola relación.
- Puede que no haya forma de traducir las restricciones a la relación ternaria en restricciones a las relaciones binarias. Por ejemplo, considérese una restricción que dice que R es varios a uno de A, B a C; es decir, cada par de entidades de A y de B se asocia, a lo sumo, con una entidad de C. Esta restricción no se puede expresar mediante restricciones de cardinalidad sobre los conjuntos de relaciones R_A, R_B y R_C.

Considérese el conjunto de relaciones *trabaja_en* del Apartado 6.2.2, que relaciona *empleado*, *sucursal* y *trabajo*. No se puede dividir directamente *trabaja_en* en relaciones binarias entre *empleado* y *sucursal* y entre *empleado* y *trabajo*. Si se hiciese, se podría registrar que Santos es director e interventor y que Santos trabaja en Navacerrada y en Centro; sin embargo, no se podría registrar que Santos es director de Navacerrada e interventor de Centro, pero que no es interventor de Navacerrada ni director de Centro.

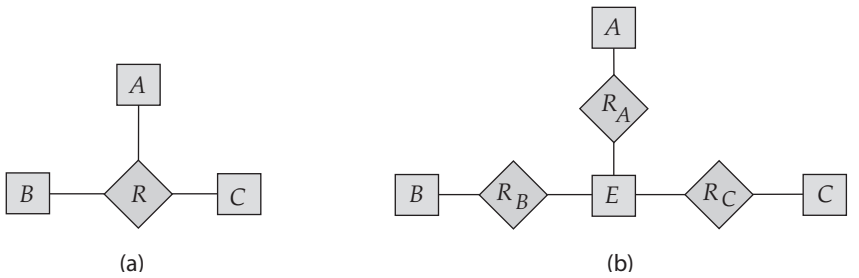


Figura 6.17 Una relación ternaria y tres relaciones binarias.

www.detodoprogramacion.com

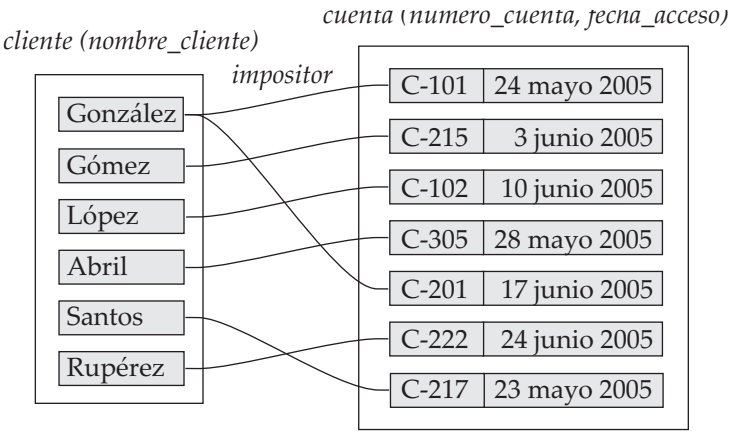


Figura 6.18 *fecha_acceso* como atributo del conjunto de entidades *cuenta*.

El conjunto de relaciones *trabaja_en* se puede dividir en relaciones binarias mediante la creación de nuevos conjuntos de entidades como se ha descrito anteriormente. Sin embargo, no sería muy natural.

6.5.4 Ubicación de los atributos de las relaciones

La razón de cardinalidad de una relación puede afectar a la ubicación de sus atributos. Por tanto, los atributos de los conjuntos de relaciones uno a uno o uno a varios pueden estar asociados con uno de los conjuntos de entidades participantes, en lugar de con el conjunto de relaciones. Por ejemplo, especifiquemos que *impositor* es un conjunto de relaciones uno a varios tal que cada cliente puede tener varias cuentas, pero cada cuenta sólo puede tener un cliente como titular. En este caso, el atributo *fecha_acceso*, que especifica la fecha en que el cliente tuvo acceso a la cuenta por última vez, podría estar asociado con el conjunto de entidades *cuenta*, como se describe en la Figura 6.18; para simplificar la figura sólo se muestran algunos de los atributos de los dos conjuntos de entidades. Dado que cada entidad *cuenta* participa en una relación con un ejemplar de *cliente*, como máximo, hacer esta designación de atributos tendría el mismo significado que colocar *fecha_acceso* en el conjunto de relaciones *impositor*. Los atributos de un conjunto de relaciones uno a varios sólo se pueden recolocar en el conjunto de entidades de la parte "varios" de la relación. Por otra parte, para los conjuntos de entidades uno a uno, los atributos de la relación se pueden asociar con cualquiera de las entidades participantes.

La decisión de diseño sobre la ubicación de los atributos descriptivos en estos casos—como atributo de la relación o de la entidad—debe reflejar las características de la empresa que se modela. El diseñador puede elegir mantener *fecha_acceso* como atributo de *impositor* para expresar explícitamente que se produce un acceso en el punto de interacción entre los conjuntos de entidades *cliente* y *cuenta*.

La elección de la ubicación del atributo es más sencilla para los conjuntos de relaciones varios a varios. Volviendo al ejemplo, especifiquemos el caso, quizá más realista, de que *impositor* sea un conjunto de relaciones varios a varios que expresa que cada cliente puede tener una o más cuentas, y que cada cuenta puede tener a varios clientes como titulares. Si hay que expresar la fecha en que un cliente dado tuvo acceso por última vez a una cuenta concreta, *fecha_acceso* debe ser atributo del conjunto de relaciones *impositor*, en lugar de serlo de cualquiera de las entidades participantes. Si *fecha_acceso* fuese un atributo de *cuenta*, por ejemplo, no se podría determinar qué cliente llevó a cabo el acceso más reciente a una cuenta conjunta. Cuando un atributo se determina mediante la combinación de los conjuntos de entidades participantes, en lugar de por cada entidad por separado, ese atributo debe estar asociado con el conjunto de relaciones varios a varios. La Figura 6.3 muestra la ubicación de *fecha_acceso* como atributo de la relación; de nuevo, para simplificar la figura, sólo se muestran algunos de los atributos de los dos conjuntos de entidades.

www.detodoprogramacion.com

CAPÍTULO 7

El Mundo de la Programacion en tus Manos...!
DETODOPROGRAMACION.COM

Diseño de bases de datos relacionales

En este capítulo se considera el problema de diseñar el esquema de una base de datos relacional. Muchos de los problemas que conlleva son parecidos a los de diseño que se han considerado en el Capítulo 6 en relación con el modelo E-R.

En general, el objetivo del diseño de una base de datos relacional es la generación de un conjunto de esquemas de relación que permita almacenar la información sin redundancias innecesarias, pero que también permita recuperarla fácilmente. Esto se consigue mediante el diseño de esquemas que se hallen en la *forma normal* adecuada. Para determinar si el esquema de una relación se halla en una de las formas normales deseables es necesario obtener información sobre la empresa real que se está modelando con la base de datos. Parte de esa información se halla en un diagrama E-R bien diseñado, pero puede ser necesaria información adicional sobre la empresa.

En este capítulo se introduce un enfoque formal al diseño de bases de datos relacionales basado en el concepto de dependencia funcional. Posteriormente se definen las formas normales en términos de las dependencias funcionales y de otros tipos de dependencias de datos. En primer lugar, sin embargo, se examina el problema del diseño relacional desde el punto de vista de los esquemas derivados de un diseño entidad-relación dado.

7.1 Características de los buenos diseños relacionales

El estudio del diseño entidad-relación llevado a cabo en el Capítulo 6 ofrece un excelente punto de partida para el diseño de bases de datos relacionales. Ya se vio en el Apartado 6.9 que es posible generar directamente un conjunto de esquemas de relación a partir del diseño E-R. Evidentemente, la adecuación (o no) del conjunto de esquemas resultante depende, en primer lugar, de la calidad del diseño E-R. Más adelante en este capítulo se estudiarán maneras precisas de evaluar la adecuación de los conjuntos de esquemas de relación. No obstante, es posible llegar a un buen diseño empleando conceptos que ya se han estudiado.

Para facilitar las referencias, en la Figura 7.1 se repiten los esquemas del Apartado 6.9.7.

7.1.1 Alternativa de diseño: esquemas grandes

A continuación se examinan las características de este diseño de base de datos relacional y algunas alternativas. Supóngase que, en lugar de los esquemas *prestatario* y *préstamo*, se considerase el esquema:

$$prestatario_préstamo = (id_cliente, número_préstamo, importe)$$

Esto representa el resultado de la reunión natural de las relaciones correspondientes a *prestatario* y a *préstamo*. Parece una buena idea, ya que es posible expresar algunas consultas empleando menos reuniones, hasta que se considera detenidamente la entidad bancaria que condujo al diseño E-R. Obsérvese

www.detodoprogramacion.com

sucursal = (*nombre_sucursal*, *ciudad_sucursal*, *activos*)
cliente = (*id_cliente*, *nombre_cliente*, *calle_cliente*, *ciudad_cliente*)
préstamo = (*número_préstamo*, *importe*)
cuenta = (*número_cuenta*, *saldo*)
empleado = (*id_empleado*, *nombre_empleado*, *número_teléfono*, *fecha_contratación*)
nombre_subordinado = (*id_empleado*, *nombre_subordinado*)
sucursal_cuenta = (*número_cuenta*, *nombre_sucursal*)
sucursal_préstamo = (*número_préstamo*, *nombre_sucursal*)
prestatario = (*id_cliente*, *número_préstamo*)
impositor = (*id_cliente*, *número_cuenta*)
asesor = (*id_cliente*, *id_empleado*, *tipo*)
trabaja_para = (*id_empleado_trabajador*, *id_empleado_jefe*)
pago = (*número_préstamo*, *número_pago*, *fecha_pago*, *importe_pago*)
cuenta_ahorro = (*número_cuenta*, *tasa_interés*)
cuenta_corriente = (*número_cuenta*, *importe_descubierto*)

Figura 7.1 Los esquemas bancarios del Apartado 6.9.7.

que el conjunto de relaciones *prestatario* es varios a varios. Esto permite que cada cliente tenga varios préstamos y, también, que se puedan conceder préstamos conjuntamente a varios clientes. Se tomó esta decisión para que fuese posible representar los préstamos realizados conjuntamente a matrimonios o a consorcios de personas (que pueden participar en una aventura empresarial conjunta). Ése es el motivo de que la clave primaria del esquema *prestatario* consista en *id_cliente* y *número_préstamo* en lugar de estar formada sólo por *número_préstamo*.

Considérese un préstamo que se concede a uno de esos consorcios y considérense las tuplas que deben hallarse en la relación del esquema *prestatario_préstamo*. Supóngase que el préstamo P-100 se concede al consorcio formado por los clientes Santiago (con identificador de cliente 23652), Antonio (con identificador de cliente 15202) y Jorge (con identificador de cliente 23521) por un importe de 10.000 €.

La Figura 7.2 muestra la forma en que es posible representar esta situación empleando *préstamo* y *prestatario* y con el diseño alternativo usando *prestatario_préstamo*. La tupla (P-100, 10.000) de la relación del esquema *préstamo* se reúne con tres tuplas de la relación del esquema *prestatario*, lo que genera tres tuplas de la relación del esquema *prestatario_préstamo*. Obsérvese que en *prestatario_préstamo* es necesario repetir una vez el importe del préstamo por cada cliente integrante del consorcio que solicita el préstamo. Es importante que todas estas tuplas concuerden en lo relativo al importe del préstamo P-100, ya que, en caso contrario, la base de datos quedaría en un estado inconsistente. En el diseño original que utiliza *préstamo* y *prestatario* se almacenaba el importe de cada préstamo exactamente una vez. Esto sugiere que el empleo de *prestatario_préstamo* no es buena idea, ya que se almacena el importe de cada préstamo de manera redundante y se corre el riesgo de que algún usuario actualice el importe del préstamo en una de las tuplas pero no en todas, y, por tanto, genere inconsistencia.

Considérese ahora otra alternativa. Sea *préstamo_importe_sucursal* = (*número_préstamo*, *importe*, *nombre_sucursal*), creada a partir de *préstamo* y de *sucursal_préstamo* (mediante una reunión de las relaciones correspondientes). Esto se parece al ejemplo que se acaba de considerar, pero con una importante diferencia. En este caso, *número_préstamo* es la clave primaria de los dos esquemas, *sucursal_préstamo* y *préstamo* y, por tanto, también lo es de *préstamo_importe_sucursal*. Esto se debe a que el conjunto de relaciones *sucursal_préstamo* está definido como varios a uno, a diferencia del conjunto de relaciones *prestatario* del ejemplo anterior. Para cada préstamo sólo hay una sucursal asociada. Por tanto, cada número de préstamo concreto sólo aparece una vez en *sucursal_préstamo*. Supóngase que el préstamo P-100 está asociado con la sucursal de Soria. La Figura 7.3 muestra la manera en que esto se representa empleando *préstamo_importe_sucursal*. La tupla (P-100, 10.000) de la relación del esquema *préstamo* se reúne con una sola tupla de la relación del esquema *sucursal_préstamo*, lo que sólo genera una tupla de la relación del esquema *préstamo_importe_sucursal*. No hay repetición de información en *préstamo_importe_sucursal* y, por tanto, se evitan los problemas que se encontraron en el ejemplo anterior.

www.detodoprogramacion.com

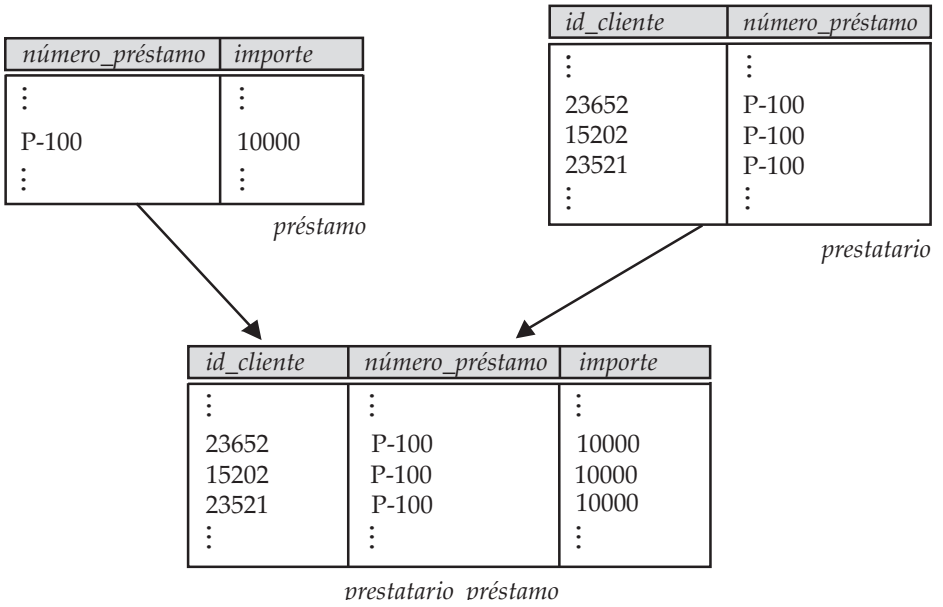


Figura 7.2 Lista parcial de las tuplas de las relaciones *préstamo*, *prestatario* y *prestatario_préstamo*.

Antes de acceder finalmente al empleo de *préstamo_importe_sucursal* en lugar de *sucursal_préstamo* y de *préstamo*, hay otro aspecto que se debe considerar. Puede que se desee registrar en la base de datos un préstamo y su sucursal asociada antes de que se haya determinado su importe. En el diseño antiguo, el esquema *sucursal_préstamo* puede hacerlo pero, con el diseño revisado *préstamo_importe_sucursal*, habría que crear una tupla con un valor nulo para *importe*. En algunos casos los valores nulos pueden crear problemas, como se vio en el estudio de SQL. No obstante, si se decide que en este caso no suponen ningún problema, se puede aceptar el empleo del diseño revisado.

Los dos ejemplos que se acaban de examinar muestran la importancia de la naturaleza de las claves primarias al decidir si las combinaciones de esquemas tienen sentido. Se han presentado problemas —concretamente, la repetición de información—cuando el atributo de reunión (*número_préstamo*) no era la clave primaria de *los dos* esquemas que se combinaban.

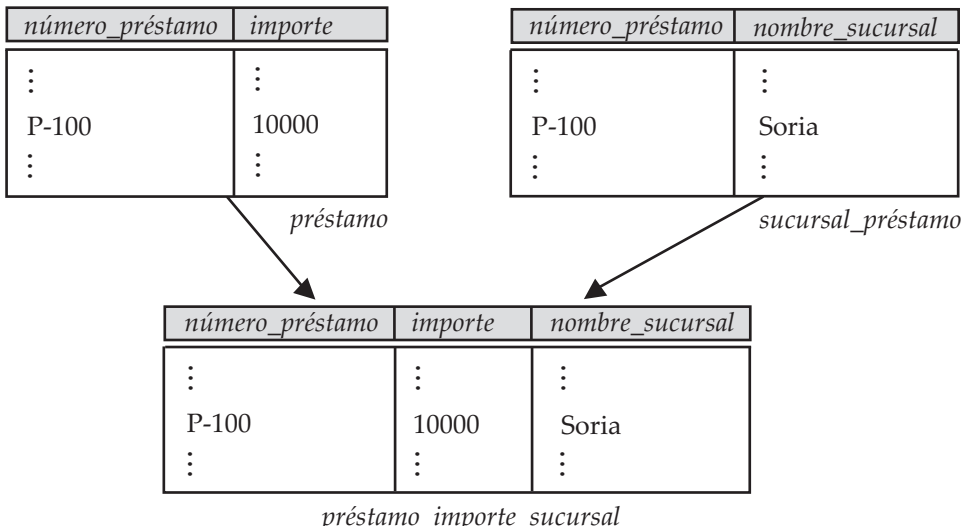


Figura 7.3 Lista parcial de las tuplas de las relaciones *préstamo*, *sucursal_préstamo* y *préstamo_importe_sucursal*.

www.detodoprogramacion.com

7.1.2 Alternativa de diseño: esquemas pequeños

Supóngase nuevamente que, de algún modo, se ha comenzado a trabajar con el esquema *prestatario_préstamo*. ¿Cómo se reconoce que necesita que haya repetición de la información y que hay que dividirlo en dos esquemas, *prestatario* y *préstamo*? Como no se dispone de los esquemas *prestatario* y *préstamo*, se carece de la información sobre la clave primaria que se empleó para describir el problema de *prestatario_préstamo*.

Mediante la observación del contenido de las relaciones reales del esquema *prestatario_préstamo* se puede percibir la repetición de información resultante de tener que relacionar el importe del préstamo una vez por cada prestatario asociado con cada uno de ellos. Sin embargo, se trata de un proceso en el que no se puede confiar. Las bases de datos reales tienen gran número de esquemas y un número todavía mayor de atributos. El número de tuplas puede ser del orden de millones. Descubrir la repetición puede resultar costoso. Hay un problema más fundamental todavía en este enfoque. No permite determinar si la carencia de repeticiones es meramente un caso especial "afortunado" o la manifestación de una regla general. En el ejemplo anterior, ¿cómo se *sabe* que en la entidad bancaria cada préstamo (identificado por su número correspondiente) sólo *debe* tener un importe? ¿El hecho de que el préstamo P-100 aparezca tres veces con el mismo importe es sólo una coincidencia? Estas preguntas no se pueden responder sin volver a la propia empresa y comprender sus reglas de funcionamiento. En concreto, hay que averiguar si el banco exige que cada préstamo (identificado por su número correspondiente) *sólo* tenga un importe.

En el caso de *prestatario_préstamo*, el proceso de creación del diseño E-R logró evitar la creación de ese esquema. Sin embargo, esta situación fortuita no se produce siempre. Por tanto, hay que permitir que el diseñador de la base de datos especifique normas como "cada valor de *número_préstamo* corresponde, como máximo, a un *importe*", incluso en los casos en los que *número_préstamo* no sea la clave primaria del esquema en cuestión. En otras palabras, hay que escribir una norma que diga "si hay un esquema (*número_préstamo, importe*), entonces *número_préstamo* puede hacer de clave primaria". Esta regla se especifica como la **dependencia funcional** *número_préstamo* $\rightarrow$ *importe*. Dada esa regla, ya se tiene suficiente información para reconocer el problema del esquema *prestatario_préstamo*. Como *número_préstamo* no puede ser la clave primaria de *prestatario_préstamo* (porque puede que cada préstamo necesite varias tuplas de la relación del esquema *prestatario_préstamo*), tal vez haya que repetir el importe del préstamo.

Observaciones como éstas y las reglas (especialmente las dependencias funcionales) que generan permiten al diseñador de la base de datos reconocer las situaciones en que hay que dividir, o descomponer, un esquema en dos o más. No es difícil comprender que la manera correcta de descomponer *prestatario_préstamo* es dividirlo en los esquemas *prestatario* y *préstamo*, como en el diseño original. Hallar la descomposición correcta es mucho más difícil para esquemas con gran número de atributos y varias dependencias funcionales. Para trabajar con ellos hay que confiar en una metodología formal que se desarrolla más avanzado este capítulo.

No todas las descomposiciones de los esquemas resultan útiles. Considérese el caso extremo en que todo lo que tengamos sean esquemas con un solo atributo. No se puede expresar ninguna relación interesante de ningún tipo. Considérese ahora un caso menos extremo en el que se decida descomponer el esquema *empleado* en

$$empleado1 = (\underline{id_empleado}, nombre_empleado)$$
$$empleado2 = (nombre_empleado, número_teléfono, fecha_contratación)$$

El problema con esta descomposición surge de la posibilidad de que la empresa tenga dos empleados que se llamen igual. Esto no es improbable en la práctica, ya que muchas culturas tienen algunos nombres muy populares y, además, puede que los niños se llamen como sus padres. Por supuesto, cada persona tiene un identificador de empleado único, que es el motivo de que se pueda utilizar *id_empleado* como clave primaria. A modo de ejemplo, supóngase que dos empleados llamados Koldo trabajan para el mismo banco y tienen las tuplas siguientes de la relación del esquema *empleado* del diseño original:

(1234567890, Koldo, 918820000, 29/03/1984)
(9876543210, Koldo, 918699999, 16/01/1981)

www.detodoprogramacion.com

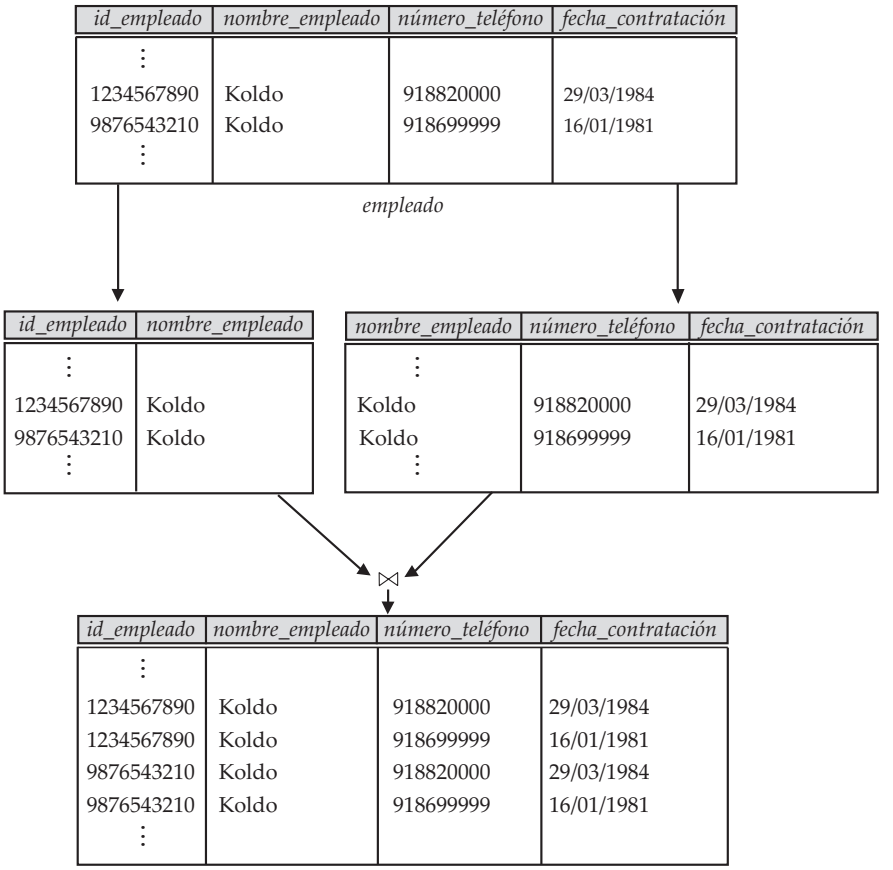


Figura 7.4 Pérdida de información debida a una mala descomposición.

La Figura 7.4 muestra estas tuplas, las tuplas resultantes al utilizar los esquemas procedentes de la descomposición y el resultado que se obtendría si se intentara volver a generar las tuplas originales mediante una reunión natural. Como se ve en la figura, las dos tuplas originales aparecen en el resultado junto con dos tuplas nuevas que mezclan de manera incorrecta valores de fechas correspondientes a los dos empleados llamados Koldo. Aunque se dispone de más tuplas, se tiene en realidad menos información en el sentido siguiente: se puede indicar que un determinado número de teléfono y una fecha de contratación dada corresponden a alguien llamado Koldo, pero no se puede distinguir a cuál de ellos. Por tanto, la descomposición propuesta es incapaz de representar algunos datos importantes de la entidad bancaria. Evidentemente, es preferible evitar este tipo de descomposiciones. Estas descomposiciones se denominan **descomposiciones con pérdidas**, y, a la inversa, aquéllas que no tienen pérdidas se denominan **descomposiciones sin pérdidas**.

7.2 Dominios atómicos y la primera forma normal

El modelo E-R permite que los conjuntos de entidades y los de relaciones tengan atributos con algún tipo de subestructura. Concretamente, permite atributos multivalorados, como *nombre_subordinado* de la Figura 6.25, y atributos compuestos (como el atributo *dirección* con los atributos componentes *calle* y *ciudad*). Cuando se crean tablas a partir de los diseños E-R que contienen ese tipo de atributos, se elimina esa subestructura. Para los atributos compuestos, se deja que cada atributo componente sea un atributo de pleno derecho. Para los atributos multivalorados, se crea una tupla por cada elemento del conjunto multivalorado.

En el modelo relacional se formaliza esta idea de que los atributos no tienen subestructuras. Un dominio es **atómico** si se considera que los elementos de ese dominio son unidades indivisibles. Se dice que

www.detodoprogramacion.com

el esquema de la relación R está en la **primera forma normal** (1FN) si los dominios de todos los atributos de R son atómicos.

Los conjuntos de nombres son ejemplos de valores no atómicos. Por ejemplo, si el esquema de la relación *empleado* incluyera el atributo *hijos*, los elementos de cuyo dominio son conjuntos de nombres, el esquema no estaría en la primera forma normal.

Los atributos compuestos, como el atributo *dirección* con los atributos componentes *calle* y *ciudad*, tienen también dominios no atómicos.

Se da por supuesto que los números enteros son atómicos, por lo que el conjunto de los números enteros es un dominio atómico; el conjunto de todos los conjuntos de números enteros es un dominio no atómico. La diferencia estriba en que normalmente no se considera que los enteros tengan subpartes, pero sí se considera que las tienen los conjuntos de enteros—es decir, los enteros que componen el conjunto. Pero lo importante no es lo que sea el propio dominio, sino el modo en que se utilizan los elementos de ese dominio en la base de datos. El dominio de todos los enteros sería no atómico si se considerara que cada entero es una lista ordenada de cifras.

Como ilustración práctica del punto anterior, considérese una organización que asigna a los empleados números de identificación de la manera siguiente: las dos primeras letras especifican el departamento y las cuatro cifras restantes son un número único para cada empleado dentro de ese departamento. Ejemplos de estos números pueden ser $CS0012$ y $EE1127$. Estos números de identificación pueden dividirse en unidades menores y, por tanto, no son atómicos. Si el esquema de la relación tuviera un atributo cuyo dominio consistiera en números de identificación así codificados, el esquema no se hallaría en la primera forma normal.

Cuando se utilizan números de identificación de este tipo, se puede averiguar el departamento de cada empleado escribiendo código que analice la estructura de los números de identificación. Ello exige programación adicional, y la información queda codificada en el programa de aplicación en vez de en la base de datos. Surgen nuevos problemas si se utilizan esos números de identificación como claves primarias: cada vez que un empleado cambie de departamento, habrá que modificar su número de identificación en todos los lugares en que aparezca, lo que puede constituir una tarea difícil; en caso contrario, el código que interpreta ese número dará un resultado erróneo.

El empleo de atributos con el valor definido por un conjunto puede llevar a diseños con almacenamiento de datos redundante; lo que, a su vez, puede dar lugar a inconsistencias. Por ejemplo, en lugar de representar la relación entre las cuentas y los clientes como la relación independiente *impositor*, puede que el diseñador de bases de datos esté tentado a almacenar un conjunto de *titulares* con cada cuenta, y un conjunto de *cuentas* con cada cliente. Siempre que se cree una cuenta, o se actualice el conjunto de titulares de una cuenta, habrá que llevar a cabo la actualización en dos lugares; no llevar a cabo esas dos actualizaciones puede dejar la base de datos en un estado inconsistente. Guardar sólo uno de esos conjuntos evitaría la información repetida, pero complicaría algunas consultas.

Algunos tipos de valores no atómicos pueden resultar útiles, aunque deben utilizarse con cuidado. Por ejemplo, los atributos con valores compuestos suelen resultar útiles, y los atributos con el valor determinado por un conjunto también resultan útiles en muchos casos, que es el motivo por el que el modelo E-R los soporta. En muchos dominios en los que las entidades tienen una estructura compleja, la imposición de la representación en la primera forma normal supone una carga innecesaria para el programador de las aplicaciones, que tiene que escribir código para convertir los datos a su forma atómica. También hay sobrecarga en tiempo de ejecución por la conversión de los datos entre su forma habitual y su forma atómica. Por tanto, el soporte de los valores no atómicos puede resultar muy útil en ese tipo de dominios. De hecho, los sistemas modernos de bases de datos soportan muchos tipos de valores no atómicos, como se verá en el Capítulo 9. Sin embargo, en este capítulo nos limitamos a las relaciones en la primera forma normal y, por tanto, todos los dominios son atómicos.

7.3 Descomposición mediante dependencias funcionales

En el Apartado 7.1 se indicó que hay una metodología formal para evaluar si un esquema relacional debe descomponerse. Esta metodología se basa en los conceptos de clave y de dependencia funcional.

www.detodoprogramacion.com

7.3.1 Claves y dependencias funcionales

Las claves y, más en general, las dependencias funcionales son restricciones de la base de datos que exigen que las relaciones cumplan determinadas propiedades. Las relaciones que cumplen todas esas restricciones son **legales**.

En el Capítulo 6 se definió el concepto de *superclave* de la manera siguiente. Sea R el esquema de una relación. El subconjunto C de R es una **superclave** de R si, en cualquier relación legal $r(R)$, para todos los pares t_1 y t_2 de tuplas de r tales que $t_1 \neq t_2$, $t_1[C] \neq t_2[C]$. Es decir, ningún par de tuplas de una relación legal $r(R)$ puede tener el mismo valor del conjunto de atributos C.

Mientras que una clave es un conjunto de atributos que identifica de manera unívoca toda una tupla, una dependencia funcional permite expresar restricciones que identifican de manera unívoca el valor de determinados atributos. Considérese el esquema de una relación R y sean $\alpha \subseteq R$ y $\beta \subseteq R$. La **dependencia funcional** $\alpha \rightarrow \beta$ se cumple para el esquema R si, en cualquier relación legal $r(R)$, para todos los pares de tuplas t_1 y t_2 de r tales que $t_1[\alpha] = t_2[\alpha]$, también se cumple que $t_1[\beta] = t_2[\beta]$.

Empleando la notación para la dependencia funcional, se dice que C es superclave de R si $C \rightarrow R$. Es decir, C es superclave si, siempre que $t_1[C] = t_2[C]$, también se cumple que $t_1[R] = t_2[R]$ (es decir, $t_1 = t_2$).

Las dependencias funcionales permiten expresar las restricciones que no se pueden expresar con superclaves. En el Apartado 7.1.2 se consideró el esquema

$$\textit{prestatario_préstamo} = (\underline{\textit{id_cliente, número_préstamo}}, \textit{importe})$$

en el que se cumple la dependencia funcional *número_préstamo* → *importe*, ya que por cada préstamo (identificado por *número_préstamo*) hay un solo importe. El hecho de que el par de atributos (*id_cliente, número_préstamo*) forma una superclave para *prestatario_préstamo* se denota escribiendo

$$\textit{id_cliente, número_préstamo} \rightarrow \textit{id_cliente, número_préstamo, importe}$$

o, de manera equivalente,

$$\textit{id_cliente, número_préstamo} \rightarrow \textit{prestatario_préstamo}$$

Las dependencias funcionales se emplean de dos maneras:

1. Para probar las relaciones y ver si son legales de acuerdo con un conjunto dado de dependencias funcionales. Si una relación r es legal según el conjunto F de dependencias funcionales, se dice que r **satisface** F.
2. Para especificar las restricciones del conjunto de relaciones legales. Así, *sólo* habrá que preocuparse de las relaciones que satisfagan un conjunto dado de dependencias funcionales. Si uno desea restringirse a las relaciones del esquema R que satisfacen el conjunto F de dependencias funcionales, se dice que F se **cumple** en R.

Considérese la relación r de la Figura 7.5, para ver las dependencias funcionales que se satisfacen. Obsérvese que se satisface $A \rightarrow C$. Hay dos tuplas que tienen un valor para A de a_1. Estas tuplas tienen el mismo valor de C—por ejemplo, c_1. De manera parecida, las dos tuplas con un valor para A de a_2

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4

Figura 7.5 Relación de ejemplo r.

www.detodoprogramacion.com

tienen el mismo valor de *C*, c_2. No hay más pares de tuplas diferentes que tengan el mismo valor de *A*. Sin embargo, la dependencia funcional $C \rightarrow A$ no se satisface. Para verlo, considérense las tuplas $t_1 = (a_2, b_3, c_2, d_3)$ y $t_2 = (a_3, b_3, c_2, d_4)$. Estas dos tuplas tienen el mismo valor de *C*, c_2, pero tienen valores diferentes para *A*, a_2 y a_3, respectivamente. Por tanto, se ha hallado un par de tuplas t_1 y t_2 tales que $t_1[C] = t_2[C]$, but $t_1[A] \neq t_2[A]$.

Se dice que algunas dependencias funcionales son **triviales** porque las satisfacen todas las relaciones. Por ejemplo, $A \rightarrow A$ la satisfacen todas las relaciones que implican al atributo *A*. La lectura literal de la definición de dependencia funcional deja ver que, para todas las tuplas t_1 y t_2 tales que $t_1[A] = t_2[A]$, se cumple que $t_1[A] = t_2[A]$. De manera análoga, $AB \rightarrow A$ la satisfacen todas las relaciones que implican al atributo *A*. En general, las dependencias funcionales de la forma $\alpha \rightarrow \beta$ son **triviales** si se cumple la condición $\beta \subseteq \alpha$.

Es importante darse cuenta de que una relación dada puede, en cualquier momento, satisfacer algunas dependencias funcionales cuyo cumplimiento no sea necesario en el esquema de la relación. En la relación *cliente* de la Figura 2.4 puede verse que se satisface *calle_cliente* → *ciudad_cliente*. Sin embargo, se sabe que en el mundo real dos ciudades diferentes pueden tener calles que se llamen igual. Por tanto, es posible, en un momento dado, tener un ejemplar de la relación *cliente* en el que no se satisfaga *calle_cliente* → *ciudad_cliente*. Por consiguiente, no se incluirá *calle_cliente* → *ciudad_cliente* en el conjunto de dependencias funcionales que se cumplen en la relación *cliente*.

Dado un conjunto de dependencias funcionales *F* que se cumple en una relación *r*, es posible que se pueda inferir que también se deban cumplir en esa misma relación otras dependencias funcionales. Por ejemplo, dado el esquema $r = (A, B, C)$, si se cumplen en *r* las dependencias funcionales $A \rightarrow B$ y $B \rightarrow C$, se puede inferir que también $A \rightarrow C$ se debe cumplir en *r*. Ya que, dado cualquier valor de *A*, sólo puede haber un valor correspondiente de *B* y, para ese valor de *B*, sólo puede haber un valor correspondiente de *C*. Más adelante, en el Apartado 7.4.1, se estudia la manera de realizar esas inferencias.

Se utiliza la notación F^+ para denotar el **cierre** del conjunto *F*, es decir, el conjunto de todas las dependencias funcionales que pueden inferirse dado el conjunto *F*. Evidentemente, F^+ es un superconjunto de *F*.

7.3.2 Forma normal de Boyce–Codd

Una de las formas normales más deseables que se pueden obtener es la **forma normal de Boyce-Codd** (**FNBC**). Elimina todas las redundancias que se pueden descubrir a partir de las dependencias funcionales aunque, como se verá en el Apartado 7.6, puede que queden otros tipos de redundancia. El esquema de relación *R* está en la FNBC respecto al conjunto *F* de dependencias funcionales si, para todas las dependencias funcionales de F^+ de la forma $\alpha \rightarrow \beta$, donde $\alpha \subseteq R$ y $\beta \subseteq R$, se cumple, al menos, una de las siguientes condiciones:

- $\alpha \rightarrow \beta$ es una dependencia funcional trivial (es decir, $\beta \subseteq \alpha$).
- α es superclave del esquema *R*.

Los diseños de bases de datos están en la FNBC si cada miembro del conjunto de esquemas de relación que constituye el diseño se halla en la FNBC.

Ya se ha visto que el esquema *prestatario_préstamo* = (*id_cliente, número_préstamo, importe*) no se halla en FNBC. La dependencia funcional *número_préstamo* → *importe* se cumple en *prestatario_préstamo*, pero *número_préstamo* no es superclave (ya que, como se recordará, los préstamos se pueden conceder a consorcios formados por varios clientes). En el Apartado 7.1.2 se vio que la descomposición de *prestatario_préstamo* en *prestatario* y *préstamo* es mejor diseño. El esquema *prestatario* se halla en la FNBC, ya que no se cumple en él ninguna dependencia funcional que no sea trivial. El esquema *préstamo* tiene una dependencia funcional no trivial que se cumple, *número_préstamo* → *importe*, pero *número_préstamo* es superclave (realmente, en este caso, la clave primaria) de *préstamo*. Por tanto, *préstamo* se halla en la FNBC.

www.detodoprogramacion.com

Ahora se va a definir una regla general para la descomposición de esquemas que no se hallen en la FNBC. Sea R un esquema que no se halla en la FNBC. En ese caso, queda, al menos, una dependencia funcional no trivial $\alpha \rightarrow \beta$ tal que α no es superclave de R. En el diseño se sustituye R por dos esquemas:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

En el caso anterior de *prestatario_préstamo*, $\alpha =$ *número_préstamo*, $\beta =$ *importe* y *prestatario_préstamo* se sustituye por

- $(\alpha \cup \beta)$ = (*número_préstamo, importe*)
- $(R - (\beta - \alpha))$ = (*id_cliente, número_préstamo*)

En este caso resulta que $\beta - \alpha = \beta$. Hay que definir la regla tal y como se ha hecho para que trate correctamente las dependencias funcionales que tienen atributos que aparecen a los dos lados de la flecha. Las razones técnicas para esto se tratarán más adelante, en el Apartado 7.5.1.

Cuando se descomponen esquemas que no se hallan en la FNBC, puede que uno o varios de los esquemas resultantes no se hallen en la FNBC. En ese caso, hacen falta más descomposiciones, cuyo resultado final es un conjunto de esquemas FNBC.

7.3.3 FNBC y la conservación de las dependencias

Se han visto varias maneras de expresar las restricciones de consistencia de las bases de datos: restricciones de clave primaria, dependencias funcionales, restricciones **check**, asertos y disparadores. La comprobación de estas restricciones cada vez que se actualiza la base de datos puede ser costosa y, por tanto, resulta útil diseñar la base de datos de manera que las restricciones se puedan comprobar de manera eficiente. En concreto, si la comprobación de las dependencias funcionales puede realizarse considerando sólo una relación, el coste de comprobación de esa restricción será bajo. Se verá que la descomposición en la FNBC puede impedir la comprobación eficiente de determinadas dependencias funcionales.

Para ilustrar esto, supóngase que se lleva a cabo una modificación aparentemente pequeña en el modo en que opera la entidad bancaria. En el diseño de la Figura 6.25 cada cliente sólo puede tener un empleado como "asesor personal". Esto se deduce de que el conjunto de relaciones *asesor* es varios a uno de *cliente* a *empleado*. La "pequeña" modificación que se va a llevar a cabo es que cada cliente pueda tener más de un asesor personal pero, como máximo, uno por sucursal.

Esto se puede implementar en el diagrama E-R haciendo que el conjunto de relaciones *asesor* sea varios a varios (ya que ahora cada cliente puede tener más de un asesor personal) y añadiendo un nuevo conjunto de relaciones, *trabaja_en*, entre *empleado* y *sucursal*, que indique los pares empleado-sucursal, en los que el empleado trabaja en la sucursal. Hay que hacer *trabaja_en* varios a uno de *empleado* a *sucursal*, ya que cada sucursal puede tener varios empleados, pero cada empleado sólo puede trabajar en una sucursal. La Figura 7.6 muestra un subconjunto de la Figura 6.25 con estas adiciones.

No obstante, hay un fallo en este diseño. Permite que cada cliente tenga dos (o más) asesores personales que trabajen en la misma sucursal, algo que el banco no permite. Lo ideal sería que hubiera un solo conjunto de relaciones al que se pudiera hacer referencia para hacer que se cumpla esta restricción. Esto exige que se considere una manera diferente de modificar el diseño E-R. En lugar de añadir el conjunto de relaciones *trabaja_en*, se sustituye el conjunto de relaciones *asesor* por la relación ternaria *sucursal_asesor*, que implica a los conjuntos de entidades *cliente*, *empleado* y *sucursal* y es varios a uno del par *cliente*, *empleado* a *sucursal*, como puede verse en la Figura 7.7. Como este diseño permite que un solo conjunto de relaciones represente la restricción, supone una ventaja significativa respecto del primer enfoque considerado.

Sin embargo, la diferencia entre estos dos enfoques no es tan clara. El esquema derivado de *sucursal_asesor* es

sucursal_asesor = (*id_cliente, id_empleado, nombre_sucursal, tipo*)

www.detodoprogramacion.com

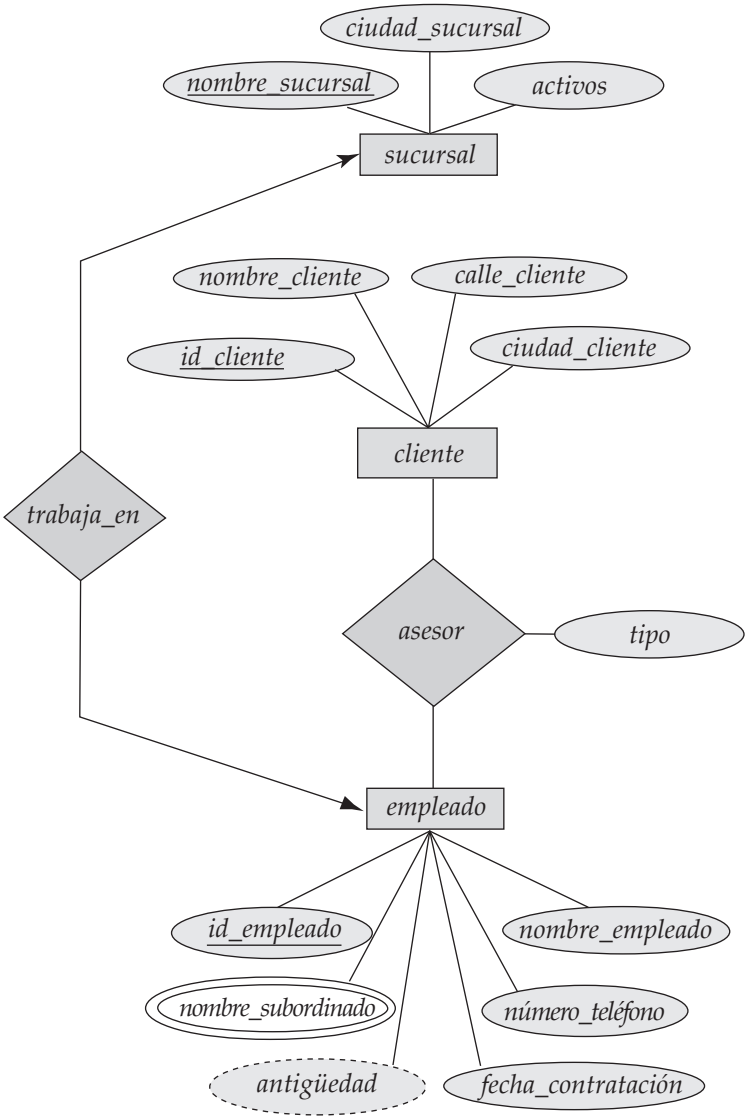


Figura 7.6 Los conjuntos de relaciones *asesor* y *trabaja_en*.

Como cada empleado sólo puede trabajar en una sucursal, en la relación del esquema *sucursal_asesor* sólo puede haber un valor de *nombre_sucursal* asociado con cada valor de *id_empleado*; es decir:

$$id_empleado \rightarrow nombre_sucursal$$

No obstante, no queda más remedio que repetir el nombre de la sucursal cada vez que un empleado participa en la relación *sucursal_asesor*. Es evidente que *sucursal_asesor* no se halla en la FNBC, ya que *id_empleado* no es superclave. De acuerdo con la regla para la descomposición en la FNBC, se obtiene:

(*id_cliente*, *id_empleado*, *tipo*)
(*id_empleado*, *nombre_sucursal*)

Este diseño, que es exactamente igual que el primer enfoque que utilizaba el conjunto de relaciones *trabaja_en*, dificulta que se haga cumplir la restricción de que cada cliente pueda tener, como máximo, un asesor en cada sucursal. Esta restricción se puede expresar mediante la dependencia funcional

$$id_cliente, nombre_sucursal \rightarrow id_empleado$$

www.detodoprogramacion.com

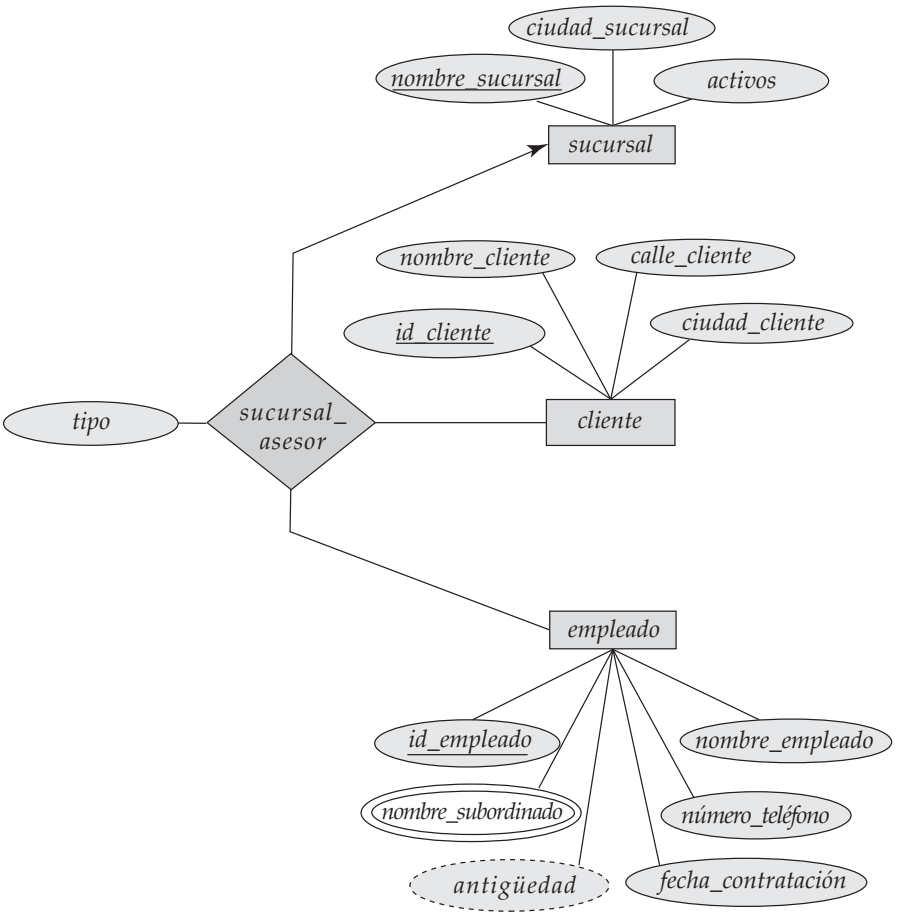


Figura 7.7 El conjunto de relaciones *sucursal_asesor*.

y hacer notar que en el diseño FNBC escogido no hay ningún esquema que incluya todos los atributos que aparecen en esta dependencia funcional. Como el diseño hace computacionalmente difícil que se haga cumplir esta dependencia funcional, se dice que el diseño no **conserva las dependencias**[1]. Como la conservación de las dependencias suele considerarse deseable, se considera otra forma normal, más débil que la FNBC, que permita que se conserven las dependencias. Esa forma normal se denomina tercera forma normal[2].

7.3.4 Tercera forma normal

La FNBC exige que todas las dependencias no triviales sean de la forma $\alpha \rightarrow \beta$, donde α es una superclave. La tercera forma normal (3NF) relaja ligeramente esta restricción al permitir dependencias funcionales no triviales cuya parte izquierda no sea una superclave. Antes de definir la 3NF hay que recordar que las claves candidatas son superclaves mínimas—es decir, superclaves de las que ningún subconjunto propio sea también superclave.

El esquema de relación R está en **tercera forma normal** respecto a un conjunto F de dependencias funcionales si, para todas las dependencias funcionales de F^+ de la forma $\alpha \rightarrow \beta$, donde $\alpha \subseteq R$ y $\beta \subseteq R$, se cumple, al menos, una de las siguientes condiciones:

1. Técnicamente es posible que una dependencia cuyos atributos no aparezcan en su totalidad en ningún esquema se siga haciendo cumplir de manera implícita, debido a la presencia de otras dependencias que la impliquen lógicamente. Este caso se abordará más adelante, en el Apartado 7.4.5.
2. Puede que se haya observado que se ha omitido la segunda forma normal. Sólo tiene significación histórica y, en la práctica, no se utiliza.

www.detodoprogramacion.com

- $\alpha \rightarrow \beta$ es una dependencia funcional trivial.
- α es superclave de R.
- Cada atributo A de $\beta - \alpha$ está contenido en alguna clave candidata de R.

Obsérvese que la tercera condición no dice que una sola clave candidata deba contener todos los atributos de $\beta - \alpha$; cada atributo A de $\beta - \alpha$ puede estar contenido en una clave candidata *diferente*.

Las dos primeras alternativas son iguales que las dos alternativas de la definición de la FNBC. La tercera alternativa de la definición de la 3FN parece bastante poco intuitiva, y no resulta evidente el motivo de su utilidad. Representa, en cierto sentido, una relajación mínima de las condiciones de la FNBC que ayuda a garantizar que cada esquema tenga una descomposición que conserve las dependencias en la 3FN. Su finalidad se aclarará más adelante, cuando se estudie la descomposición en la 3FN.

Obsérvese que cualquier esquema que satisfaga la FNBC satisface también la 3FN, ya que cada una de sus dependencias funcionales satisfará una de las dos primeras alternativas. Por tanto, la FNBC es una forma normal más restrictiva que la 3FN.

La definición de la 3FN permite ciertas dependencias funcionales que no se permiten en la FNBC. Las dependencias $\alpha \rightarrow \beta$ que sólo satisfacen la tercera alternativa de la definición de la 3FN no se permiten en la FNBC, pero sí en la 3FN[3].

Considérese nuevamente el ejemplo *sucursal_asesor* y la dependencia funcional

$$id_empleado \rightarrow nombre_sucursal$$

que hacía que el esquema no se hallara en FNBC. Obsérvese que, en este caso, $\alpha = id_empleado$, $\beta = nombre_sucursal$ y $\beta - \alpha = nombre_sucursal$. Resulta que *nombre_sucursal* está contenido en una clave candidata y que, por tanto, *sucursal_asesor* se halla en la 3FN. Probar esto, no obstante, exige un pequeño esfuerzo.

Se sabe que, además de las dependencias funcionales

$$id_empleado \rightarrow nombre_sucursal$$
$$id_cliente, nombre_sucursal \rightarrow id_empleado$$

que se cumplen, la dependencia funcional

$$id_cliente, id_empleado \rightarrow sucursal_asesor$$

se cumple como consecuencia de que (*id_cliente, id_empleado*) es la clave primaria. Esto hace de (*id_cliente, id_empleado*) clave candidata. Por supuesto, no contiene *nombre_sucursal*, por lo que hay que ver si hay otras claves candidatas. Se concluye que el conjunto de atributos (*id_cliente, nombre_sucursal*) es clave candidata. Veamos el motivo.

Dados unos valores concretos de *id_cliente* y de *nombre_sucursal*, se sabe que sólo hay un valor asociado de *id_empleado*, ya que

$$id_cliente, nombre_sucursal \rightarrow id_empleado$$

Pero entonces, para esos valores concretos de *id_cliente* y de *id_empleado*, sólo puede haber una tupla asociada de *sucursal_asesor*, ya que

$$id_cliente, id_empleado \rightarrow sucursal_asesor$$

Por tanto, se ha argumentado que (*id_cliente, nombre_sucursal*) es superclave. Como resulta que ni *id_cliente* ni *nombre_sucursal* por separado son superclaves, (*id_cliente, nombre_sucursal*) es clave candida-

3. Estas dependencias son ejemplos de **dependencias transitivas** (véase el Ejercicio práctico 7.14). La definición original de la 3NF se hizo en términos de las dependencias transitivas. La definición que aquí se utiliza es equivalente, pero más fácil de comprender.

www.detodoprogramacion.com

ta. Dado que esta clave candidata contiene *nombre_sucursal*, la dependencia funcional

$$id_empleado \rightarrow nombre_sucursal$$

no viola las reglas de la 3FN.

La demostración de que *sucursal_asesor* se halla en la 3NF ha supuesto cierto esfuerzo. Por esta razón (y por otras), resulta útil adoptar un enfoque estructurado y formal del razonamiento sobre las dependencias funcionales, las formas normales y la descomposición de los esquemas, lo cual se hará en el Apartado 7.4.

Se ha visto el equilibrio que hay que guardar entre la FNBC y la 3NF cuando no hay ningún diseño FNBC que conserve las dependencias. Estos equilibrios se describen con más detalle en el Apartado 7.5.3; en ese apartado también se describe un enfoque de la comprobación de dependencias mediante vistas materializadas que permite obtener las ventajas de la FNBC y de la 3NF.

7.3.5 Formas normales superiores

Puede que en algunos casos el empleo de las dependencias funcionales para la descomposición de los esquemas no sea suficiente para evitar la repetición innecesaria de información. Considérese una ligera variación de la definición del conjunto de entidades *empleado* en la que se permita que los empleados tengan varios números de teléfono, alguno de los cuales puede ser compartido entre varios empleados. Entonces, *número_teléfono* será un atributo multivalorado y, de acuerdo con las reglas para la generación de esquemas a partir de los diseños E-R, habrá dos esquemas, uno por cada uno de los atributos multivalorados *número_teléfono* y *nombre_subordinado*:

(*id_empleado, nombre_subordinado*) (*id_empleado, número_teléfono*)

Si se combinan estos esquemas para obtener

(*id_empleado, nombre_subordinado, número_teléfono*)

se descubre que el resultado se halla en la FNBC, ya que sólo se cumplen dependencias funcionales no triviales. En consecuencia, se puede pensar que ese tipo de combinación es una buena idea. Sin embargo, se trata de una mala idea, como puede verse si se considera el ejemplo de un empleado con dos subordinados y dos números de teléfono. Por ejemplo, sea el empleado con *id_empleado* 999999999 que tiene dos subordinados llamados "David" y "Guillermo" y dos números de teléfono, 512555123 y 512555432. En el esquema combinado hay que repetir los números de teléfono una vez por cada subordinado:

(999999999, David, 512555123)
(999999999, David, 512555432)
(999999999, Guillermo, 512555123)
(999999999, Guillermo, 512555432)

Si no se repitieran los números de teléfono y sólo se almacenaran la primera y la última tupla, se habrían guardado el nombre de los subordinados y los números de teléfono, pero las tuplas resultantes implicarían que David correspondería al 512555123 y que Guillermo correspondería al 512555432. Como sabemos, esto es incorrecto.

Debido a que las formas normales basadas en las dependencias funcionales no son suficientes para tratar con situaciones como ésta, se han definido otras dependencias y formas normales. Se estudian en los Apartados 7.6 y 7.7.

7.4 Teoría de las dependencias funcionales

Se ha visto en los ejemplos que resulta útil poder razonar de manera sistemática sobre las dependencias funcionales como parte del proceso de comprobación de los esquemas para la FNBC o la 3NF.

www.detodoprogramacion.com

7.4.1 Cierre de los conjuntos de dependencias funcionales

No es suficiente considerar el conjunto dado de dependencias funcionales. También hay que considerar *todas* las dependencias funcionales que se cumplen. Se verá que, dado un conjunto *F* de dependencias funcionales, se puede probar que también se cumple alguna otra dependencia funcional. Se dice que *F* "implica lógicamente" esas dependencias funcionales.

De manera más formal, dado un esquema relacional *R*, una dependencia funcional *f* de *R* está **implicada lógicamente** por un conjunto de dependencias funcionales *F* de *R* si cada ejemplar de la relación $r(R)$) que satisface *F* satisface también *f*.

Supóngase que se tiene el esquema de relación $R = (A, B, C, G, H, I)$ y el conjunto de dependencias funcionales

$$A \rightarrow B$$
$$A \rightarrow C$$
$$CG \rightarrow H$$
$$CG \rightarrow I$$
$$B \rightarrow H$$

La dependencia funcional

$$A \rightarrow H$$

está implicada lógicamente. Es decir, se puede demostrar que, siempre que el conjunto dado de dependencias funcionales se cumple en una relación, también se debe cumplir $A \rightarrow H$. Supóngase que t_1 y t_2 son tuplas tales que

$$t_1[A] = t_2[A]$$

Como se tiene que $A \rightarrow B$, se deduce de la definición de dependencia funcional que

$$t_1[B] = t_2[B]$$

Entonces, como se tiene que $B \rightarrow H$, se deduce de la definición de dependencia funcional que

$$t_1[H] = t_2[H]$$

Por tanto, se ha demostrado que, siempre que t_1 y t_2 sean tuplas tales que $t_1[A] = t_2[A]$, debe ocurrir que $t_1[H] = t_2[H]$. Pero ésa es exactamente la definición de $A \rightarrow H$.

Sea *F* un conjunto de dependencias funcionales. El **cierre** de *F*, denotado por F^+, es el conjunto de todas las dependencias funcionales implicadas lógicamente por *F*. Dado *F*, se puede calcular F^+ directamente a partir de la definición formal de dependencia funcional. Si *F* fuera de gran tamaño, ese proceso sería largo y difícil. Este tipo de cálculo de F^+ requiere argumentos del tipo que se acaba de utilizar para demostrar que $A \rightarrow H$ está en el cierre del conjunto de dependencias de ejemplo.

Los **axiomas**, o reglas de inferencia, proporcionan una técnica más sencilla para el razonamiento sobre las dependencias funcionales. En las reglas que se ofrecen a continuación se utilizan las letras griegas (α, β, γ, ...) para los conjuntos de atributos y las letras latinas mayúsculas desde el comienzo del alfabeto para los atributos. Se emplea $\alpha\beta$ para denotar $\alpha \cup \beta$.

Se pueden utilizar las tres reglas siguientes para hallar las dependencias funcionales implicadas lógicamente. Aplicando estas reglas *repetidamente* se puede hallar todo F^+, dado *F*. Este conjunto de reglas se denomina **axiomas de Armstrong** en honor de la persona que las propuso por primera vez.

- **Regla de la reflexividad**. Si α es un conjunto de atributos y $\beta \subseteq \alpha$, entonces se cumple que $\alpha \rightarrow \beta$.
- **Regla de la aumentatividad**. Si se cumple que $\alpha \rightarrow \beta$ y γ es un conjunto de atributos, entonces se cumple que $\gamma\alpha \rightarrow \gamma\beta$.
- **Regla de la transitividad**. Si se cumple que $\alpha \rightarrow \beta$ y que $\beta \rightarrow \gamma$, entonces se cumple que $\alpha \rightarrow \gamma$.

Los axiomas de Armstrong son **correctos**, ya que no generan dependencias funcionales incorrectas. Son **completos**, ya que, para un conjunto de dependencias funcionales *F* dado, permiten generar todo

www.detodoprogramacion.com

F^+. Las notas bibliográficas proporcionan referencias de las pruebas de su corrección y de su completitud.

Aunque los axiomas de Armstrong son completos, resulta pesado utilizarlos directamente para el cálculo de F^+. Para simplificar más las cosas se dan unas reglas adicionales. Se pueden utilizar los axiomas de Armstrong para probar que son correctas (véanse los Ejercicios prácticos 7.4 y 7.5 y el Ejercicio 7.21).

- **Regla de la unión**. Si se cumple que $\alpha \rightarrow \beta$ y que $\alpha \rightarrow \gamma$, entonces se cumple que $\alpha \rightarrow \beta\gamma$.
- **Regla de la descomposición**. Si se cumple que $\alpha \rightarrow \beta\gamma$, entonces se cumple que $\alpha \rightarrow \beta$ y que $\alpha \rightarrow \gamma$.
- **Regla de la pseudotransitividad**. Si se cumple que $\alpha \rightarrow \beta$ y que $\gamma\beta \rightarrow \delta$, entonces se cumple que $\alpha\gamma \rightarrow \delta$.

Se aplicarán ahora las reglas al ejemplo del esquema $R = (A, B, C, G, H, I)$ y del conjunto F de dependencias funcionales $\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$. A continuación se relacionan varios miembros de F^+:

- $A \rightarrow H$. Dado que se cumplen $A \rightarrow B$ y $B \rightarrow H$, se aplica la regla de transitividad. Obsérvese que resultaba mucho más sencillo emplear los axiomas de Armstrong para demostrar que se cumple que $A \rightarrow H$, que deducirlo directamente a partir de las definiciones, como se ha hecho anteriormente en este apartado.
- $CG \rightarrow HI$. Dado que $CG \rightarrow H$ y $CG \rightarrow I$, la regla de la unión implica que $CG \rightarrow HI$.
- $AG \rightarrow I$. Dado que $A \rightarrow C$ y $CG \rightarrow I$, la regla de pseudotransitividad implica que se cumple que $AG \rightarrow I$.

  Otra manera de averiguar que se cumple que $AG \rightarrow I$ es la siguiente. Se utiliza la regla de aumentatividad en $A \rightarrow C$ para inferir que $AG \rightarrow CG$. Aplicando la regla de transitividad a esta dependencia y a $CG \rightarrow I$, se infiere que $AG \rightarrow I$.

La Figura 7.8 muestra un procedimiento que demuestra formalmente el modo de utilizar los axiomas de Armstrong para calcular F^+. En este procedimiento puede que la dependencia funcional ya esté presente en F^+ cuando se le añade y, en ese caso, no hay ninguna modificación en F^+. En el Apartado 7.4.2 se verá una manera alternativa de calcular F^+.

Los términos a la derecha y a la izquierda de las dependencias funcionales son subconjuntos de R. Dado que un conjunto de tamaño n tiene 2^n subconjuntos, hay un total de $2 \times 2^n = 2^{n+1}$ dependencias funcionales posibles, donde n es el número de atributos de R. Cada iteración del bucle repetir del procedimiento, salvo la última, añade como mínimo una dependencia funcional a F^+. Por tanto, está garantizado que el procedimiento termine.

```
F+ = F
repeat
    for each dependencia funcional f de F+
        aplicar las reglas de reflexividad y de aumentatividad a f
        añadir las dependencias funcionales resultantes a F+
    for each par de dependencias funcionales f1 y f2 de F+
        if f1 y f2 se pueden combinar mediante la transitividad
            añadir la dependencia funcional resultante a F+
until F+ deje de cambiar
```

Figura 7.8 Procedimiento para calcular F^+.

www.detodoprogramacion.com

7.4.2 Cierre de los conjuntos de atributos

Se dice que un atributo B está **determinado funcionalmente** por α si $\alpha \rightarrow B$. Para comprobar si un conjunto α es superclave hay que diseñar un algoritmo para el cálculo del conjunto de atributos determinados funcionalmente por α. Una manera de hacerlo es calcular F^+, tomar todas las dependencias funcionales con α como término de la izquierda y tomar la unión de los términos de la derecha de todas esas dependencias. Sin embargo, hacer esto puede resultar costoso, ya que F^+ puede ser de gran tamaño.

Un algoritmo eficiente para el cálculo del conjunto de atributos determinados funcionalmente por α no sólo resulta útil para comprobar si α es superclave, sino también para otras tareas, como se verá más adelante en este apartado.

Sea α un conjunto de atributos. Al conjunto de todos los atributos determinados funcionalmente por α bajo un conjunto F de dependencias funcionales se le denomina **cierre** de α bajo F; se denota mediante α^+. La Figura 7.9 muestra un algoritmo, escrito en pseudocódigo, para calcular α^+. La entrada es un conjunto F de dependencias funcionales y el conjunto α de atributos. La salida se almacena en la variable *resultado*.

Para ilustrar el modo en que trabaja el algoritmo se utilizará para calcular $(AG)^+$ con las dependencias funcionales definidas en el Apartado 7.4.1. Se comienza con $result = AG$. La primera vez que se ejecuta el bucle **while** para comprobar cada dependencia funcional se halla que

- $A \rightarrow B$ hace que se incluya B en *resultado*. Para comprobarlo, obsérvese que $A \rightarrow B$ se halla en F y $A \subseteq$ *resultado* (que es AG), por lo que *resultado* := *resultado* $\cup B$.
- $A \rightarrow C$ hace que *resultado* se transforme en $ABCG$.
- $CG \rightarrow H$ hace que *resultado* se transforme en $ABCGH$.
- $CG \rightarrow I$ hace que *resultado* se transforme en $ABCGHI$.

La segunda vez que se ejecuta el bucle **while** ya no se añade ningún atributo nuevo a *resultado*, y se termina el algoritmo.

Veamos ahora el motivo de que el algoritmo de la Figura 7.9 sea correcto. El primer paso es correcto, ya que $\alpha \rightarrow \alpha$ se cumple siempre (por la regla de reflexividad). Se afirma que, para cualquier subconjunto β de *resultado*, $\alpha \rightarrow \beta$. Dado que el bucle **while** se inicia con $\alpha \rightarrow$ *resultado* como cierto, sólo se puede añadir γ a *resultado* si $\beta \subseteq$ *resultado* y $\beta \rightarrow \gamma$. Pero, entonces, *resultado* $\rightarrow \beta$ por la regla de reflexividad, por lo que $\alpha \rightarrow \beta$ por transitividad. Otra aplicación de la transitividad demuestra que $\alpha \rightarrow \gamma$ (empleando $\alpha \rightarrow \beta$ y $\beta \rightarrow \gamma$). La regla de la unión implica que $\alpha \rightarrow$ *resultado* $\cup \gamma$, por lo que α determina funcionalmente cualquier resultado nuevo generado en el bucle **while**. Por tanto, cualquier atributo devuelto por el algoritmo se halla en α^+.

Resulta sencillo ver que el algoritmo halla todo α^+. Si hay un atributo de α^+ que no se halle todavía en *resultado*, debe haber una dependencia funcional $\beta \rightarrow \gamma$ para la que $\beta \subseteq$ *resultado* y, como mínimo, un atributo de γ no se halla en *resultado*.

En el peor de los casos es posible que este algoritmo tarde un tiempo proporcional al cuadrado del tamaño de F. Hay un algoritmo más rápido (aunque ligeramente más complejo) que se ejecuta en un tiempo proporcional al tamaño de F; ese algoritmo se presenta como parte del Ejercicio práctico 7.8.

Existen varias aplicaciones del algoritmo de cierre de atributos:

```
resultado := α;
while (cambios en resultado) do
        for each dependencia funcional β → γ in F do
            begin
                if β ⊆ resultado then resultado := resultado ∪ γ;
            fin
```

Figura 7.9 Algoritmo para el cálculo de α^+, el cierre de α bajo F.

www.detodoprogramacion.com

- Para comprobar si α es superclave, se calcula α^+ y se comprueba si contiene todos los atributos de *R*.
- Se puede comprobar si se cumple la dependencia funcional $\alpha \rightarrow \beta$ (o, en otras palabras, si se halla en F^+), comprobando si $\beta \subseteq \alpha^+$. Es decir, se calcula α^+ empleando el cierre de los atributos y luego se comprueba si contiene a β. Esta prueba resulta especialmente útil, como se verá más adelante en este mismo capítulo.
- Ofrece una manera alternativa de calcular F^+: para cada $\gamma \subseteq R$ se halla el cierre γ^+ y, para cada $S \subseteq \gamma^+$, se genera la dependencia funcional $\gamma \rightarrow S$.

7.4.3 Recubrimiento canónico

Supóngase que se tiene un conjunto *F* de dependencias funcionales de un esquema de relación. Siempre que un usuario lleve a cabo una actualización de la relación, el sistema de bases de datos debe asegurarse de que la actualización no viole ninguna dependencia funcional, es decir, que se satisfagan todas las dependencias funcionales de *F* en el nuevo estado de la base de datos.

El sistema debe retroceder la actualización si viola alguna dependencia funcional del conjunto *F*.

Se puede reducir el esfuerzo dedicado a la comprobación de las violaciones comprobando un conjunto simplificado de dependencias funcionales que tenga el mismo cierre que el conjunto dado. Cualquier base de datos que satisfaga el conjunto simplificado de dependencias funcionales satisfará también el conjunto original y viceversa, ya que los dos conjuntos tienen el mismo cierre. Sin embargo, el conjunto simplificado resulta más sencillo de comprobar. En breve se verá el modo en que se puede crear ese conjunto simplificado. Antes, hacen falta algunas definiciones.

Se dice que un atributo de una dependencia funcional es **raro** si se puede eliminar sin modificar el cierre del conjunto de dependencias funcionales. La definición formal de los **atributos raros** es la siguiente. Considérese un conjunto *F* de dependencias funcionales y la dependencia funcional $\alpha \rightarrow \beta$ de *F*.

- El atributo *A* es raro en $alpha$ si $A \in \alpha$ y *F* implica lógicamente a $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- El atributo *A* es raro en β si $A \in \beta$ y el conjunto de dependencias funcionales $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ implica lógicamente a *F*.

Por ejemplo, supóngase que se tienen las dependencias funcionales $AB \rightarrow C$ y $A \rightarrow C$ de F. Entonces, B es raro en $AB \rightarrow C$. Como ejemplo adicional, supóngase que se tienen las dependencias funcionales $AB \rightarrow CD$ y $A \rightarrow C$ de F. Entonces, C será raro en el lado derecho de $AB \rightarrow CD$.

Hay que tener cuidado con la dirección de las implicaciones al utilizar la definición de los atributos raros: si se intercambian el lado derecho y el izquierdo, la implicación se cumplirá *siempre*. Es decir, $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ siempre implica lógicamente a F, y F también implica lógicamente siempre a $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$.

A continuación se muestra el modo de comprobar de manera eficiente si un atributo es raro. Sea *R* el esquema de la relación y F el conjunto dado de dependencias funcionales que se cumplen en R. Considérese el atributo A de la dependencia $\alpha \rightarrow \beta$.

- Si $A \in \beta$, para comprobar si A es raro hay que considerar el conjunto
  $$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$$
  y comprobar si $\alpha \rightarrow A$ puede inferirse a partir de F'. Para ello hay que calcular α^+ (el cierre de α) bajo F'; si α^+ incluye a A, entonces A es raro en β.
- Si $A \in \alpha$, para comprobar si A es raro, sea $\gamma = \alpha - \{A\}$, hay que comprobar si se puede inferir que $\gamma \rightarrow \beta$ a partir de F. Para ello hay que calcular γ^+ (el cierre de γ) bajo F; si γ^+ incluye todos los atributos de β, entonces A es raro en α.

Por ejemplo, supóngase que F contiene $AB \rightarrow CD$, $A \rightarrow E$ y $E \rightarrow C$. Para comprobar si C es raro en $AB \rightarrow CD$, hay que calcular el cierre de los atributos de AB bajo $F' = \{AB \rightarrow D, A \rightarrow E$ y $E \rightarrow C\}$. El cierre es $ABCDE$, que incluye a CD, por lo que se infiere que C es raro.

www.detodoprogramacion.com

$F_c = F$
repeat
  Utilizar la regla de unión para sustituir las dependencias de F_c de la forma
    $\alpha_1 \rightarrow \beta_1$ y $\alpha_1 \rightarrow \beta_2$ con $\alpha_1 \rightarrow \beta_1\ \beta_2$.
  Hallar una dependencia funcional $\alpha \rightarrow \beta$ de F_c con un atributo
    raro en α o en β.
    /* Nota: la comprobación de los atributos raros se lleva a cabo empleando F_c, no F */
  Si se halla algún atributo raro, hay que eliminarlo de $\alpha \rightarrow \beta$.
until F_c ya no cambie.

Figura 7.10 Cálculo del recubrimiento canónico.

El **recubrimiento canónico** F_c de F es un conjunto de dependencias tal que F implica lógicamente todas las dependencias de F_c y F_c implica lógicamente todas las dependencias de F. Además, F_c debe tener las propiedades siguientes:

- Ninguna dependencia funcional de F_c contiene atributos raros.
- El lado izquierdo de cada dependencia funcional de F_c es único. Es decir, no hay dos dependencias $\alpha_1 \rightarrow \beta_1$ y $\alpha_2 \rightarrow \beta_2$ de F_c tales que $\alpha_1 = \alpha_2$.

El recubrimiento canónico del conjunto de dependencias funcionales F puede calcularse como se muestra en la Figura 7.10. Es importante destacar que, cuando se comprueba si un atributo es raro, la comprobación utiliza las dependencias del valor actual de F_c, y **no** las dependencias de F. Si una dependencia funcional sólo contiene un atributo en su lado derecho, por ejemplo, $A \rightarrow C$, y se descubre que ese atributo es raro, se obtiene una dependencia funcional con el lado derecho vacío. Hay que eliminar esas dependencias funcionales.

Se puede demostrar que el recubrimiento canónico de F, F_c, tiene el mismo cierre que F; por tanto, comprobar si se satisface F_c es equivalente a comprobar si se satisface F. Sin embargo, F_c es mínimo en un cierto sentido—no contiene atributos raros, y combina las dependencias funcionales con el mismo lado izquierdo. Resulta más económico comprobar F_c que comprobar el propio F.

Considérese el siguiente conjunto F de dependencias funcionales para el esquema (A, B, C):

$$A \rightarrow BC$$
$$B \rightarrow C$$
$$A \rightarrow B$$
$$AB \rightarrow C$$

Calcúlese el recubrimiento canónico de F.

- Hay dos dependencias funcionales con el mismo conjunto de atributos a la izquierda de la flecha:

  $$A \rightarrow BC$$
  $$A \rightarrow B$$

  Estas dependencias funcionales se combinan en $A \rightarrow BC$.

- A es raro en $AB \rightarrow C$, ya que F implica lógicamente a $(F - \{AB \rightarrow C\}) \cup \{B \rightarrow C\}$. Esta aseveración es cierta porque $B \rightarrow C$ ya se halla en el conjunto de dependencias funcionales.
- C es raro en $A \rightarrow BC$, ya que $A \rightarrow BC$ está implicada lógicamente por $A \rightarrow B$ y $B \rightarrow C$.

Por tanto, el recubrimiento canónico es

$$A \rightarrow B$$
$$B \rightarrow C$$

www.detodoprogramacion.com

Dado un conjunto F de dependencias funcionales, puede suceder que toda una dependencia funcional del conjunto sea rara, en el sentido de que eliminarla no modifique el cierre de F. Se puede demostrar que el recubrimiento canónico F_c de F no contiene esa dependencia funcional rara. Supóngase que, por el contrario, esa dependencia rara estuviera en F_c. Los atributos del lado derecho de la dependencia serían raros, lo que no es posible por la definición de recubrimiento canónico.

Puede que el recubrimiento canónico no sea único. Por ejemplo, considérese el conjunto de dependencias funcionales $F = \{A \rightarrow BC, B \rightarrow AC \text{ y } C \rightarrow AB\}$. Si se aplica la prueba de rareza a $A \rightarrow BC$ se descubre que tanto B como C son raros bajo F. Sin embargo, sería incorrecto eliminar los dos. El algoritmo para hallar el recubrimiento canónico selecciona uno de los dos y lo elimina. Entonces:

1. Si se elimina C, se obtiene el conjunto $F' = \{A \rightarrow B, B \rightarrow AC \text{ y } C \rightarrow AB\}$. Ahora B ya no es raro en el lado derecho de $A \rightarrow B$ bajo F'. Siguiendo con el algoritmo se descubre que A y B son raros en el lado derecho de $C \rightarrow AB$, lo que genera dos recubrimientos canónicos:

$$F_c = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$$
$$F_c = \{A \rightarrow B, B \rightarrow AC, C \rightarrow B\}$$

2. Si se elimina B, se obtiene el conjunto $\{A \rightarrow C, B \rightarrow AC \text{ y } C \rightarrow AB\}$. Este caso es simétrico del anterior y genera dos recubrimientos canónicos:

$$F_c = \{A \rightarrow C, C \rightarrow B, B \rightarrow A\}$$
$$F_c = \{A \rightarrow C, B \rightarrow C, C \rightarrow AB\}$$

Como ejercicio, el lector debe intentar hallar otro recubrimiento canónico de F.

7.4.4 Descomposición sin pérdida

Sean R un esquema de relación y F un conjunto de dependencias funcionales de R. Supóngase que R_1 y R_2 forman una descomposición de R. Sea $r(R)$ una relación con el esquema R. Se dice que la descomposición es una **descomposición sin pérdidas** si, para todos los ejemplares legales de la base de datos (es decir, los ejemplares de la base de datos que satisfacen las dependencias funcionales especificadas y otras restricciones),

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

En otros términos, si se proyecta r sobre R_1 y R_2 y se calcula la reunión natural del resultado de la proyección, se vuelve a obtener exactamente r. Las descomposiciones que no son sin pérdidas se denominan **descomposiciones con pérdidas**. Los términos **descomposición de reunión sin pérdidas** y **descomposición de reunión con pérdidas** se utilizan a veces en lugar de descomposición sin pérdidas y descomposición con pérdidas.

Se pueden utilizar las dependencias funcionales para probar si ciertas descomposiciones son sin pérdidas. Sean R, R_1, R_2 y F como se acaban de definir. R_1 y R_2 forman una descomposición sin pérdidas de R si, como mínimo, una de las dependencias funcionales siguientes se halla en F^+:

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

En otros términos, si $R_1 \cap R_2$ forma una superclave de R_1 o de R_2, la descomposición de R es una descomposición sin pérdidas. Se puede utilizar el cierre de los atributos para buscar superclaves de manera eficiente, como ya se ha visto antes.

Para ilustrar esto, considérese el esquema

$$\textit{prestatario_préstamo} = (\underline{\textit{id_cliente, número_préstamo}}, \textit{importe})$$

que se descompuso en el Apartado 7.1.2 en

$$\textit{prestatario} = (\textit{id_cliente}, \underline{\textit{número_préstamo}})$$
$$\textit{préstamo} = (\underline{\textit{número_préstamo}}, \textit{importe})$$

www.detodoprogramacion.com

```
calcular F+;
for each esquema Ri in D do
    begin
        Fi : = la restricción de F+ a Ri;
    end
F' := ∅
for each restricción Fi do
    begin
        F' = F' ∪ Fi
    end
calcular F'+;
if (F'+ = F+) then return (cierto)
              else return (falso);
```

Figura 7.11 Comprobación de la conservación de las dependencias.

En este caso *prestatario* ∩ *préstamo* = *número_préstamo* y *número_préstamo* → *importe*, lo que satisface la regla de la descomposición sin pérdidas.

Para el caso general de la descomposición simultánea de un esquema en varios, la comprobación de la descomposición sin pérdidas es más complicada. Véanse las notas bibliográficas para tener referencias sobre este tema.

Mientras que la comprobación de la descomposición binaria es, claramente, una condición suficiente para la descomposición sin pérdidas, sólo se trata de una condición necesaria si todas las las restricciones son dependencias funcionales. Más adelante se verán otros tipos de restricciones (especialmente, un tipo de restricción denominada dependencia multivalorada, que se estudia en el Apartado 7.6.1), que pueden garantizar que una descomposición sea sin pérdidas aunque no esté presente ninguna dependencia funcional.

7.4.5 Conservación de las dependencias

Resulta más sencillo caracterizar la conservación de las dependencias empleando la teoría de las dependencias funcionales que mediante el enfoque ad hoc que se utilizó en el Apartado 7.3.3.

Sea F un conjunto de dependencias funcionales del esquema R y $R_1, R_2, \ldots, R_n$ una descomposición de R. La **restricción** de F a R_i es el conjunto F_i de todas las dependencias funcionales de F^+ que *sólo* incluyen atributos de R_i. Dado que todas las dependencias funcionales de cada restricción implican a los atributos de un único esquema de relación, es posible comprobar el cumplimiento de esas dependencias verificando sólo una relación.

Obsérvese que la definición de restricción utiliza todas las dependencias de F^+, no sólo las de F. Por ejemplo, supóngase que se tiene $F = \{A \rightarrow B, B \rightarrow C\}$ y que se tiene una descomposición en AC y AB. La restricción de F a AC es, por tanto, $A \rightarrow C$, ya que $A \rightarrow C$ se halla en F^+, aunque no se halle en F.

El conjunto de restricciones $F_1, F_2, \ldots, F_n$ es el conjunto de dependencias que pueden comprobarse de manera eficiente. Ahora cabe preguntarse si es suficiente comprobar sólo las restricciones. Sea $F' = F_1 \cup F_2 \cup \cdots \cup F_n$. F' es un conjunto de dependencias funcionales del esquema R pero, en general $F' \neq F$. Sin embargo, aunque $F' \neq F$, puede ocurrir que $F'^+ = F^+$. Si esto último es cierto, entonces, todas las dependencias de F están implicadas lógicamente por F' y, si se comprueba que se satisface F', se habrá comprobado que se satisface F. Se dice que las descomposiciones que tienen la propiedad $F'^+ = F^+$ son **descomposiciones que conservan las dependencias**.

La Figura 7.11 muestra un algoritmo para la comprobación de la conservación de las dependencias. La entrada es el conjunto $D = \{R_1, R_2, \ldots, R_n\}$ de esquemas de relaciones descompuestas y el conjunto F de dependencias funcionales. Este algoritmo resulta costoso, ya que exige el cálculo de F^+. En lugar de aplicar el algoritmo de la Figura 7.11, se consideran dos alternativas.

En primer lugar, hay que tener en cuenta que si se puede comprobar cada miembro de F en una de las relaciones de la descomposición, la descomposición conserva las dependencias. Se trata de una manera sencilla de demostrar la conservación de las dependencias; no obstante, no funciona siempre. Hay casos en los que, aunque la descomposición conserve las dependencias, hay alguna dependencia de F que no

www.detodoprogramacion.com

se puede comprobar para ninguna relación de la descomposición. Por tanto, esta prueba alternativa sólo se puede utilizar como condición suficiente que es fácil de comprobar; si falla, no se puede concluir que la descomposición no conserve las dependencias; en vez de eso, hay que aplicar la prueba general.

A continuación se da una comprobación alternativa de la conservación de las dependencias, que evita tener que calcular F^+. La idea intuitiva subyacente a esta comprobación se explicará tras presentarla. La comprobación aplica el procedimiento siguiente a cada $\alpha \rightarrow \beta$ de F.

resultado = α
while (cambios en *resultado*) **do**
  for each R_i de la descomposición
    t = (*resultado* $\cap$ R_i)$^+$ $\cap$ R_i
    resultado = *resultado* $\cup$ t

En este caso, el cierre de los atributos se halla bajo el conjunto de dependencias funcionales F. Si *resultado* contiene todos los atributos de β, se conserva la dependencia funcional $\alpha \rightarrow \beta$. La descomposición conserva las dependencias si, y sólo si, el procedimiento prueba que se conservan todas las dependencias de F.

Las dos ideas claves subyacentes a la comprobación anterior son las siguientes.

- La primera idea es comprobar cada dependencia funcional $\alpha \rightarrow \beta$ de F para ver si se conserva en F' (donde F' es tal y como se define en la Figura 7.11). Para ello, se calcula el cierre de α bajo F'; la dependencia se conserva exactamente cuando el cierre incluye a β. La descomposición conserva la dependencia si (y sólo si) se comprueba que se conservan todas las dependencias de F.
- La segunda idea es emplear una forma modificada del algoritmo de cierre de atributos para calcular el cierre bajo F', sin llegar a calcular antes F'. Se desea evitar el cálculo de F', ya que resulta bastante costoso. Téngase en cuenta que F' es la unión de las F_i, donde F_i es la restricción de F sobre R_i. El algoritmo calcula el cierre de los atributos de (*resultado* $\cap$ R_i) con respecto a F, intersecta el cierre con R_i y añade el conjunto de atributos resultante a *resultado*; esta secuencia de pasos es equivalente al cálculo del cierre de *resultado* bajo F_i. La repetición de este paso para cada i del interior del bucle mientras genera el cierre de *resultado* bajo F'.

  Para comprender el motivo de que este enfoque modificado al cierre de atributos funcione correctamente, hay que tener en cuenta que para cualquier $\gamma \subseteq R_i$, $\gamma \rightarrow \gamma^+$ es una dependencia funcional de F^+, y que $\gamma \rightarrow \gamma^+ \cap R_i$ es una dependencia funcional que se halla en F_i, la restricción de F^+ a R_i. A la inversa, si $\gamma \rightarrow \delta$ estuvieran en F_i, δ sería un subconjunto de $\gamma^+ \cap R_i$.

Esta comprobación tarda un tiempo que es polinómico, en lugar del exponencial necesario para calcular F^+.

7.5 Algoritmos de descomposición

Los esquemas de bases de datos del mundo real son mucho mayores que los ejemplos que caben en las páginas de un libro. Por este motivo, hacen falta algoritmos para la generación de diseños que se hallen en la forma normal adecuada. En este apartado se presentan algoritmos para la FNBC y para la 3NF.

7.5.1 Descomposición en la FNBC

Se puede emplear la definición de la FNBC para comprobar directamente si una relación se halla en esa forma normal. Sin embargo, el cálculo de F^+ puede resultar una tarea tediosa. En primer lugar se van a describir pruebas simplificadas para verificar si una relación dada se halla en la FNBC. En caso de que no lo esté, se puede descomponer para crear relaciones que sí estén en la FNBC. Más avanzado este apartado se describirá un algoritmo para crear descomposiciones sin pérdidas de las relaciones, de modo que esas descomposiciones se hallen en la FNBC.

www.detodoprogramacion.com

7.5.1.1 Comprobación de la FNBC

La comprobación de relaciones para ver si satisfacen la FNBC se puede simplificar en algunos casos:

- Para comprobar si la dependencia no trivial $\alpha \rightarrow \beta$ provoca alguna violación de la FNBC hay que calcular α^+ (el cierre de los atributos de α) y comprobar si incluye todos los atributos de R; es decir, si es superclave de R.
- Para comprobar si el esquema de relación R se halla en la FNBC basta con comprobar sólo si las dependencias del conjunto F dado violan la FNBC, en vez de comprobar todas las dependencias de F^+.

  Se puede probar que, si ninguna de las dependencias de F provoca violaciones de la FNBC, ninguna de las dependencias de F^+ lo hace tampoco.

Por desgracia, el último procedimiento no funciona cuando se descomponen relaciones. Es decir, en las descomposiciones *no* basta con emplear F cuando se comprueba si la relación R_i de R viola la FNBC. Por ejemplo, considérese el esquema de relación $R\ (A, B, C, D, E)$, con las dependencias funcionales F que contienen $A \rightarrow B$ and $BC \rightarrow D$. Supóngase que se descompusiera en $R_1(A, B)$ y en $R_2(A, C, D, E)$. Ahora ninguna de las dependencias de F contiene únicamente atributos de (A, C, D, E), por lo que se podría creer erróneamente que R_2 satisface la FNBC. De hecho, hay una dependencia $AC \rightarrow D$ de F^+ (que se puede inferir empleando la regla de la pseudotransitividad con las dos dependencias de F), que prueba que R_2 no se halla en la FNBC. Por tanto, puede que haga falta una dependencia que esté en F^+, pero no en F, para probar que una relación descompuesta no se halla en la FNBC.

Una comprobación alternativa de la FNBC resulta a veces más sencilla que calcular todas las dependencias de F^+. Para comprobar si una relación R_i de una descomposición de R se halla en la FNBC, se aplica esta comprobación:

- Para cada subconjunto α de atributos de R_i se comprueba que α^+ (el cierre de los atributos de α bajo F) no incluye ningún atributo de $R_i - \alpha$ o bien incluye todos los atributos de R_i.

Si algún conjunto de atributos α de R_i viola esta condición, considérese la siguiente dependencia funcional, que se puede probar que se halla en F^+:

$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$.

La dependencia anterior muestra que R_i viola la FNBC.

7.5.1.2 Algoritmo de descomposición de la FNBC

Ahora se puede exponer un método general para descomponer los esquemas de relación de manera que satisfagan la FNBC. La Figura 7.12 muestra un algoritmo para esta tarea. Si R no está en la FNBC, se puede descomponer en un conjunto de esquemas en la FNBC, $R_1, R_2, \ldots, R_n$ utilizando este algoritmo. El algoritmo utiliza las dependencias que demuestran la violación de la FNBC para llevar a cabo la descomposición.

```
resultado := {R};
hecho := falso;
calcular F+;
while (not hecho) do
    if (hay algún esquema Ri de resultado que no se halle en la FNBC)
      then begin
                sea α → β una dependencia funcional no trivial que se cumple
                en Ri tal que α → Ri no se halla en F+, y α ∩ β = ∅;
                resultado := (result − Ri) ∪ (Ri − β) ∪ ( α, β);
           end
      else hecho := cierto;
```

Figura 7.12 Algoritmo de descomposición en la FNBC.

www.detodoprogramacion.com

La descomposición que genera este algoritmo no sólo está en la FNBC, sino que también es una descomposición sin pérdidas. Para ver el motivo de que el algoritmo sólo genere descomposiciones sin pérdidas hay que darse cuenta de que, cuando se sustituye el esquema R_i por (($R_i - \beta$) y (α, β), se cumple que $\alpha \rightarrow \beta$ y que $(R_i - \beta) \cap (\alpha, \beta) = \alpha$.

Si no se exigiera que $\alpha \cap \beta = \emptyset$, los atributos de $\alpha \cap \beta$ no aparecerían en el esquema (($R_i - \beta$), y ya no se cumpliria la dependencia $\alpha \rightarrow \beta$.

Es fácil ver que la descomposición de *prestatario_préstamo* del Apartado 7.3.2 se obtiene de la aplicación del algoritmo. La dependencia funcional *número_préstamo* → *importe* satisface la condición $\alpha \cap \beta = \emptyset$ y, por tanto, se elige para descomponer el esquema.

El algoritmo de descomposición en la FNBC tarda un tiempo exponencial en relación con el tamaño del esquema inicial, ya que el algoritmo para comprobar si las relaciones de la descomposición satisfacen la FNBC puede tardar un tiempo exponencial. Las notas bibliográficas ofrecen referencias de un algoritmo que puede calcular la descomposición en la FNBC en un tiempo polinómico. Sin embargo, puede que ese algoritmo "sobrenormalice", es decir, descomponga relaciones sin que sea necesario.

A modo de ejemplo, más prolijo, del empleo del algoritmo de descomposición en la FNBC, supóngase que se tiene un diseño de base de datos que emplea el siguiente esquema *empréstito*:

empréstito = (*nombre_sucursal*, *ciudad_sucursal*, *activos*, *nombre_cliente*, *número_préstamo*, *importe*)

El conjunto de dependencias funcionales que se exige que se cumplan en *empréstito* es

nombre_sucursal → *activos ciudad_sucursal*
número_préstamo → *importe nombre_sucursal*

Una clave candidata para este esquema es {*número_préstamo*, *nombre_cliente*}.

Se puede aplicar el algoritmo de la Figura 7.12 al ejemplo *empréstito* de la manera siguiente:

- La dependencia funcional

  nombre_sucursal → *activos ciudad_sucursal*

  se cumple, pero *nombre_sucursal* no es superclave. Por tanto, *empréstito* no se halla en la FNBC. Se sustituye *empréstito* por

  sucursal = (*nombre_sucursal*, *ciudad_sucursal*, *activos*)
  info_préstamo = (*nombre_sucursal*, *nombre_cliente*, *número_préstamo*, *importe*)

- Entre las pocas dependencias funcionales no triviales que se cumplen en *sucursal* está *nombre_sucursal*, al lado izquierdo de la flecha. Dado que *nombre_sucursal* es clave de *sucursal*, la relación *sucursal* se halla en la FNBC.

- La dependencia funcional

  número_préstamo → *importe nombre_sucursal*

  se cumple en *info_préstamo*, pero *número_préstamo* no es clave de *info_préstamo*. Se sustituye *info_préstamo* por

  sucursal_préstamo = (*número_préstamo*, *nombre_sucursal*, *importe*)
  prestatario = (*nombre_cliente*, *número_préstamo*)

- *sucursal_préstamo* y *prestatario* están en la FNBC.

Por tanto, la descomposición de *empréstito* da lugar a los tres esquemas de relación *sucursal*, *sucursal_préstamo* y *prestatario*, cada uno de los cuales se halla en la FNBC. Se puede comprobar que la descomposición es sin pérdidas y que conserva las dependencias.

www.detodoprogramacion.com

sea F_c un recubrimiento canónico de F;
$i := 0$;
for each dependencia funcional $\alpha \rightarrow \beta$ de F_c **do**
  if ninguno de los esquemas $R_j, j = 1, 2, \ldots, i$ contiene $\alpha\,\beta$
    then begin
      $i := i + 1$;
      $R_i := \alpha\,\beta$;
    end
if ninguno de los esquemas $R_j, j = 1, 2, \ldots, i$ contiene una clave candidata de R
  then begin
    $i := i + 1$;
    $R_i :=$ cualquier clave candidata de R;
  end
return $(R_1, R_2, \ldots, R_i)$

Figura 7.13 Descomposición en la 3NF sin pérdidas que conserva las dependencias.

Téngase en cuenta que, aunque el esquema *sucursal_préstamo* anterior se halle en la FNBC, se puede decidir descomponerlo aún más empleando la dependencia funcional *número_préstamo* → *importe* para obtener los esquemas

préstamo = (*número_préstamo*, *importe*) *sucursal_préstamo* = (*número_préstamo*, *nombre_sucursal*)

Estos esquemas corresponden a los que se han utilizado en este capítulo.

7.5.2 Descomposición en la 3FN

La Figura 7.13 muestra un algoritmo para la búsqueda de descomposiciones en la 3FN sin pérdidas y que conserven las dependencias. El conjunto de dependencias F_c utilizado en el algoritmo es un recubrimiento canónico de F. Obsérvese que el algoritmo considera el conjunto de esquemas $R_j, j = 1, 2, \ldots, i$; inicialmente $i = 0$ y, en ese caso, el conjunto está vacío.

Se aplicará este algoritmo al ejemplo del Apartado 7.3.3 en el que se demostró que

sucursal_asesor = (*id_cliente*, *id_empleado*, *nombre_sucursal*, *tipo*)

se halla en la 3NF, aunque no se halle en la FNBC. El algoritmo utiliza las dependencias funcionales de F, que, en este caso, también son F_c:

(*id_cliente*, *id_empleado* → *nombre_sucursal*, *tipo*)
id_empleado → *nombre_sucursal*

y considera dos esquemas en el bucle **for**. A partir de la primera dependencia funcional el algoritmo genera como R_1 el esquema (*id_cliente*, *id_empleado*, *nombre_sucursal*, *tipo*). A partir de la segunda, el algoritmo genera el esquema (*id_empleado*, *nombre_sucursal*), pero no lo crea como R_2, ya que se halla incluido en R_1 y, por tanto, falla la condición **if**.

El algoritmo garantiza la conservación de las dependencias mediante la creación explícita de un esquema para cada dependencia del recubrimiento canónico. Asegura que la descomposición sea sin pérdidas al garantizar que, como mínimo, un esquema contenga una clave candidata del esquema que se está descomponiendo. El Ejercicio práctico 7.12 proporciona algunos indicios de la prueba de que esto basta para garantizar una descomposición sin pérdidas.

Este algoritmo también se denomina **algoritmo de síntesis de la 3NF**, ya que toma un conjunto de dependencias y añade los esquemas de uno en uno, en lugar de descomponer el esquema inicial de manera repetida. El resultado no queda definido de manera única, ya que cada conjunto de dependencias funcionales puede tener más de un recubrimiento canónico y, además, en algunos casos, el resultado del algoritmo depende del orden en que considere las dependencias de F_c.

Si una relación R_i está en la descomposición generada por el algoritmo de síntesis, entonces R_i está en la 3FN. Recuérdese que, cuando se busca la 3FN, basta con considerar las dependencias funcionales cuyo lado derecho sea un solo atributo. Por tanto, para ver si R_i está en la 3FN, hay que convencerse de que cualquier dependencia funcional $\gamma \rightarrow B$ que se cumpla en R_i satisface la definición de la 3FN.

www.detodoprogramacion.com

Supóngase que la dependencia que generó R_i en el algoritmo de síntesis es $\alpha \rightarrow \beta$. Ahora bien, B debe estar en α o en β, ya que B está en R_i y $\alpha \rightarrow \beta$ ha generado R_i. Considérense los tres casos posibles:

- B está tanto en α como en β. En ese caso, la dependencia $\alpha \rightarrow \beta$ no habría estado en F_c, ya que B sería rara en β. Por tanto, este caso no puede darse.
- B está en β pero no en α. Considérense dos casos:
  - γ es superclave. Se satisface la segunda condición de la 3FN.
  - γ no es superclave. Entonces, α debe contener algún atributo que no se halle en γ. Ahora bien, como $\gamma \rightarrow B$ se halla en F^+, debe poder obtenerse a partir de F_c mediante el algoritmo del cierre de atributos de γ. La obtención no puede haber empleado $\alpha \rightarrow \beta$—si lo hubiera hecho, α debería estar contenida en el cierre de los atributos de γ, lo que no es posible, ya que se ha dado por supuesto que γ no es superclave. Ahora bien, empleando $\alpha \rightarrow (\beta - \{B\})$ y $\gamma \rightarrow B$, se puede obtener que $\alpha \rightarrow B$ (debido a que $\gamma \subseteq \alpha\beta$ y a que γ no puede contener a B porque $\gamma \rightarrow B$ no es trivial). Esto implicaría que B es raro en el lado derecho de $\alpha \rightarrow \beta$, lo que no es posible, ya que $\alpha \rightarrow \beta$ está en el recubrimiento canónico F_c. Por tanto, si B está en β, γ debe ser superclave, y se debe satisfacer la segunda condición de la 3FN.
- B está en α pero no en β.
  Como α es clave candidata, se satisface la tercera alternativa de la definición de la 3FN.

Es interesante que el algoritmo que se ha descrito para la descomposición en la 3FN pueda implementarse en tiempo polinómico, aunque la comprobación de una relación dada para ver si satisface la 3FN sea NP-dura (lo que significa que es muy improbable que se invente nunca un algoritmo de tiempo polinómico para esta tarea).

7.5.3 Comparación de la FNBC y la 3FN

De las dos formas normales para los esquemas de las bases de datos relacionales, la 3FN y la FNBC, la 3FN es más conveniente, ya que se sabe que siempre es posible obtener un diseño en la 3FN sin sacrificar la ausencia de pérdidas ni la conservación de las dependencias. Sin embargo, la 3FN presenta inconvenientes: puede que haya que emplear valores nulos para representar algunas de las relaciones significativas posibles entre los datos y existe el problema de la repetición de la información.

Los objetivos del diseño de bases de datos con dependencias funcionales son:

1. FNBC
2. Ausencia de pérdidas
3. Conservación de las dependencias

Como no siempre resulta posible satisfacer las tres, puede que nos veamos obligados a escoger entre la FNBC y la conservación de las dependencias con la 3FN.

Merece la pena destacar que el SQL no ofrece una manera de especificar las dependencias funcionales, salvo para el caso especial de la declaración de las superclaves mediante las restricciones **primary key** o **unique**. Es posible, aunque un poco complicado, escribir asertos que hagan que se cumpla una dependencia funcional determinada (véase el Ejercicio práctico 7.9); por desgracia, la comprobación de los asertos resultaría muy costosa en la mayor parte de los sistemas de bases de datos. Por tanto, aunque se tenga una descomposición que conserve las dependencias, si se utiliza el SQL estándar, sólo se podrán comprobar de manera eficiente las dependencias funcionales cuyo lado izquierdo sea una clave.

Aunque puede que la comprobación de las dependencias funcionales implique una reunión si la descomposición no conserva las dependencias, se puede reducir su coste empleando vistas materializadas, que se pueden utilizar en la mayor parte de los sistemas de bases de datos. Dada una descomposición en la FNBC que no conserve las dependencias, se considera cada dependencia de un recubrimiento mínimo F_c que no se conserva en la descomposición. Para cada una de esas dependencias $\alpha \rightarrow \beta$, se define una vista materializada que calcula una reunión de todas las relaciones de la descomposición y proyecta el resultado sobre $\alpha\beta$. La dependencia funcional puede comprobarse fácilmente en la vista materializada

www.detodoprogramacion.com

mediante una restricción **unique**(α). La parte negativa es que hay una sobrecarga espacial y temporal debida a la vista materializada, pero la positiva es que el programador de la aplicación no tiene que preocuparse de escribir código para hacer que los datos redundantes se conserven consistentes en las actualizaciones; es labor del sistema de bases de datos conservar la vista materializada, es decir, mantenerla actualizada cuando se actualice la base de datos (más adelante, en el Apartado 14.5, se describe el modo en que el sistema de bases de datos puede llevar a cabo de manera eficiente el mantenimiento de las vistas materializadas).

Por tanto, en caso de que no se pueda obtener una descomposición en la FNBC que conserve las dependencias, suele resultar preferible optar por la 3NF y emplear técnicas como las vistas materializadas para reducir el coste de la comprobación de las dependencias funcionales.

7.6 Descomposición mediante dependencias multivaloradas

No parece que algunos esquemas de relación, aunque se hallen en la FNBC, estén suficientemente normalizados, en el sentido de que siguen sufriendo el problema de la repetición de información. Considérese nuevamente el ejemplo bancario. Supóngase que, en un diseño alternativo del esquema de la base de datos bancaria, se tiene el esquema

$$\textit{préstamo_cliente} = (\underline{\textit{número_préstamo, id_cliente}}, \textit{nombre_cliente}, \textit{calle_cliente}, \textit{ciudad_cliente})$$

El lector avispado reconocerá este esquema como no correspondiente a la FNBC, debido a la dependencia funcional

$$\textit{id_cliente} \rightarrow \textit{nombre_cliente, calle_cliente ciudad_cliente}$$

y a que *id_cliente* no es clave de *préstamo_cliente*. No obstante, supóngase que el banco está atrayendo clientes ricos que tienen varios domicilios (por ejemplo, una residencia de invierno y otra de verano). Entonces, ya no se desea que se cumpla la dependencia funcional *id_cliente* →*calle_cliente ciudad_cliente*, aunque, por supuesto, se sigue deseando que se cumpla *id_cliente* → *nombre_cliente* (es decir, el banco no trata con clientes que operan con varios alias). A partir del algoritmo de descomposición en la FNBC se obtienen dos esquemas:

$$R_1 = (\underline{\textit{id_cliente}}, \textit{nombre_cliente})$$
$$R_2 = (\underline{\textit{número_préstamo, id_cliente}}, \textit{calle_cliente}, \textit{ciudad_cliente})$$

Los dos se encuentran en la FNBC (recuérdese que no sólo puede cada cliente tener concedido más de un préstamo, sino que también se pueden conceder préstamos a grupos de personas y, por tanto, no se cumplen ni *id_cliente* → *número_préstamo* ni *número_préstamo* → *id_cliente*).

Pese a que R_2 se halla en la FNBC, hay redundancia. Se repite la dirección de cada residencia de cada cliente una vez por cada préstamo que tenga concedido ese cliente. Este problema se puede resolver descomponiendo más aún R_2 en:

$$\textit{id_cliente_préstamo} = (\textit{número_préstamo}, \textit{id_cliente})$$
$$\textit{residencia_cliente} = (\textit{id_cliente}, \textit{calle_cliente}, \textit{ciudad_cliente})$$

pero no hay ninguna restricción que nos lleve a hacerlo.

Para tratar este problema hay que definir una nueva modalidad de restricción, denominada *dependencia multivalorada*. Al igual que se hizo con las dependencias funcionales, se utilizarán las dependencias multivaloradas para definir una forma normal para los esquemas de relación. Esta forma normal, denominada **cuarta forma normal** (4FN), es más restrictiva que la FNBC. Se verá que cada esquema en la 4FN también se halla en la FNBC, pero que hay esquemas en la FNBC que no se hallan en la 4FN.

7.6.1 Dependencias multivaloradas

Las dependencias funcionales impiden que ciertas tuplas estén en una relación dada. Si $A \rightarrow B$, entonces no puede haber dos tuplas con el mismo valor de A y diferentes valores de B. Las dependencias

www.detodoprogramacion.com

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Figura 7.14 Representación tabular de $\alpha \twoheadrightarrow \beta$.

multivaloradas, por otro lado, no impiden la existencia de esas tuplas. Por el contrario, *exigen* que estén presentes en la relación otras tuplas de una forma determinada. Por este motivo, las dependencias funcionales se denominan a veces **dependencias que generan igualdades**; y las dependencias multivaloradas, **dependencias!que generan tuplas**.

Sea R un esquema de relación y sean $\alpha \subseteq R$ y $\beta \subseteq R$. La **dependencia multivalorada**

$$\alpha \twoheadrightarrow \beta$$

se cumple en R si, en cualquier relación legal $r(R)$ para todo par de tuplas t_1 y t_2 de *r* tales que $t_1[\alpha] = t_2[\alpha]$ existen unas tuplas t_3 y t_4 de *r* tales que

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$
$$t_3[\beta] = t_1[\beta]$$
$$t_3[R - \beta] = t_2[R - \beta]$$
$$t_4[\beta] = t_2[\beta]$$
$$t_4[R - \beta] = t_1[R - \beta]$$

Esta definición es menos complicada de lo que parece. La Figura 7.14 muestra una representación tabular de t_1, t_2, t_3 y t_4. De manera intuitiva, la dependencia multivalorada $\alpha \twoheadrightarrow \beta$ indica que la relación entre α y β es independiente de la relación entre α y $R - \beta$. Si todas las relaciones del esquema R satisfacen la dependencia multivalorada $\alpha \twoheadrightarrow \beta$, entonces $\alpha \twoheadrightarrow \beta$ es una dependencia multivalorada *trivial* del esquema R. Por tanto, $\alpha \twoheadrightarrow \beta$ es trivial si $\beta \subseteq \alpha$ o $\beta \cup \alpha = R$.

Para ilustrar la diferencia entre las dependencias funcionales y las multivaloradas, considérense de nuevo el esquema R_2 y la relación de ejemplo de ese esquema mostrada en la Figura 7.15. Hay que repetir el número de préstamo una vez por cada dirección que tenga el cliente, y la dirección en cada préstamo que tenga concedido ese cliente. Esta repetición es innecesaria, ya que la relación entre cada cliente y su dirección es independiente de la relación entre ese cliente y el préstamo. Si un cliente con identificador de cliente 99123 tiene concedido un préstamo (por ejemplo, el préstamo número P-23), se desea que ese préstamo esté asociado con todas las direcciones de ese cliente. Por tanto, la relación de la Figura 7.16 es ilegal. Para hacer que esa relación sea legal hay que añadir a la relación de la Figura 7.16 las tuplas (P-23, 99123, Mayor, Chinchón) y (P-27, 99123, Carretas, Cerceda).

Si se compara el ejemplo anterior con la definición de dependencia multivalorada, se ve que se desea que se cumpla la dependencia multivalorada

id_cliente $\twoheadrightarrow$ *calle_cliente ciudad_cliente*

También se cumplirá la dependencia multivalorada *id_cliente* $\twoheadrightarrow$ *número_préstamo*. Pronto se verá que son equivalentes.

Al igual que las dependencias funcionales, las dependencias multivaloradas se utilizan de dos maneras:

número_préstamo	*id_cliente*	*calle_cliente*	*ciudad_cliente*
P-23	99123	Carretas	Cerceda
P-23	99123	Mayor	Chinchón
P-93	15106	Leganitos	Aluche

Figura 7.15 Ejemplo de redundancia en un esquema en la FNBC.

www.detodoprogramacion.com

número_préstamo	*id_cliente*	*calle_cliente*	*ciudad_cliente*
P-23	99123	Carretas	Cerceda
P-27	99123	Mayor	Chinchón

Figura 7.16 La relación ilegal R_2.

1. Para verificar las relaciones y determinar si son legales bajo un conjunto dado de dependencias funcionales y multivaloradas.
2. Para especificar restricciones del conjunto de relaciones legales; de este modo, *sólo* habrá que preocuparse de las relaciones que satisfagan un conjunto dado de dependencias funcionales y multivaloradas.

Téngase en cuenta que, si una relación *r* no satisface una dependencia multivalorada dada, se puede crear una relación *r′* que *sí* la satisfaga añadiendo tuplas a *r*.

Supóngase que *D* denota un conjunto de dependencias funcionales y multivaloradas. El **cierre** D^+ de *D* es el conjunto de todas las dependencias funcionales y multivaloradas implicadas lógicamente por *D*. Al igual que se hizo con las dependencias funcionales, se puede calcular D^+ a partir de *D*, empleando las definiciones formales de dependencia funcional y multivalorada. Con este razonamiento se puede trabajar con dependencias multivaloradas muy sencillas. Afortunadamente, parece que las dependencias multivaloradas que se dan en la práctica son bastante sencillas. Para dependencias complejas es mejor razonar con conjuntos de dependencias mediante un sistema de reglas de inferencia (el Apartado C.1.1 del apéndice describe un sistema de reglas de inferencia para las dependencias multivaloradas).

A partir de la definición de dependencia multivalorada se puede obtener la regla siguiente:

- Si $\alpha \rightarrow \beta$, entonces $\alpha \twoheadrightarrow \beta$.

En otras palabras, todas las dependencias funcionales son también dependencias multivaloradas.

7.6.2 Cuarta forma normal

Considérese nuevamente el ejemplo del esquema en la FNBC

$$R_2 = (\underline{\textit{número_préstamo, id_cliente}}, \textit{calle_cliente, ciudad_cliente})$$

en el que se cumple la dependencia multivalorada *id_cliente* $\twoheadrightarrow$ *calle_cliente ciudad_cliente*. Se vio en los primeros párrafos del Apartado 7.6 que, aunque este esquema se halla en la FNBC, el diseño no es el ideal, ya que hay que repetir la información sobre la dirección del cliente para cada préstamo. Se verá que se puede utilizar esta dependencia multivalorada para mejorar el diseño de la base de datos, realizando la descomposición del esquema en la **cuarta forma normal**.

Un esquema de relación R está en la **cuarta forma normal** (4FN) con respecto a un conjunto D de dependencias funcionales y multivaloradas si, para todas las dependencias multivaloradas de D^+ de la forma $\alpha \twoheadrightarrow \beta$, donde $\alpha \subseteq R$ y $\beta \subseteq R$, se cumple, como mínimo, una de las condiciones siguientes

- $\alpha \twoheadrightarrow \beta$ es una dependencia multivalorada trivial.
- α es superclave del esquema R.

El diseño de una base de datos está en la 4FN si cada componente del conjunto de esquemas de relación que constituye el diseño se halla en la 4FN.

Téngase en cuenta que la definición de la 4FN sólo se diferencia de la definición de la FNBC en el empleo de las dependencias multivaloradas en lugar de las dependencias funcionales. Todos los esquemas en la 4FN están en la FNBC. Para verlo hay que darse cuenta de que, si un esquema R no se halla en la FNBC, hay una dependencia funcional no trivial $\alpha \rightarrow \beta$ que se cumple en R en la que α no es superclave. Como $\alpha \rightarrow \beta$ implica $\alpha \twoheadrightarrow \beta$, R no puede estar en la 4FN.

www.detodoprogramacion.com

```
resultado := {R};
hecho := falso;
calcular D+; dado el esquema Ri, Di denotará la restricción de D+ a Ri
while (not hecho) do
    if (hay un esquema Ri en resultado que no se halla en la 4FN con respecto a Di)
      then begin
              sea α ↠ β una dependencia multivalorada no trivial que se cumple
              en Ri tal que α → Ri no se halla en Di, y α ∩ β = ∅;
              resultado := (resultado − Ri) ∪ (Ri − β) ∪ (α, β);
           end
      else hecho := cierto;
```

Figura 7.17 Algoritmo de descomposición en la 4FN.

Sea R un esquema de relación y sean $R_1, R_2, \ldots, R_n$ una descomposición de R. Para comprobar si cada esquema de relación R_i de la descomposición se halla en la 4FN hay que averiguar las dependencias multivaloradas que se cumplen en cada R_i. Recuérdese que, para un conjunto F de dependencias funcionales, la restricción F_i de F a R_i son todas las dependencias funcionales de F^+ que *sólo* incluyen los atributos de R_i. Considérese ahora un conjunto D de dependencias funcionales y multivaloradas. La **restricción** de D a R_i es el conjunto D_i consistente en

1. Todas las dependencias funcionales de D^+ que sólo incluyen atributos de R_i
2. Todas las dependencias multivaloradas de la forma

$$\alpha \twoheadrightarrow \beta \cap R_i$$

donde $\alpha \subseteq R_i$ y $\alpha \twoheadrightarrow \beta$ está en D^+.

7.6.3 Descomposición en la 4FN

La analogía entre la 4FN y la FNBC es aplicable al algoritmo para descomponer esquemas en la 4FN. La Figura 7.17 muestra el algoritmo de descomposición en la 4FN. Es idéntico al algoritmo de descomposición en la FNBC de la Figura 7.12, salvo en que emplea dependencias multivaloradas en lugar de funcionales y en que utiliza la restricción de D^+ a R_i.

Si se aplica el algoritmo de la Figura 7.17 a (*número_préstamo, id_cliente, calle_cliente, ciudad_cliente*), se descubre que *id_cliente* ↠ *número_préstamo* es una dependencia multivalorada no trivial, y que *id_cliente* no es superclave del esquema. De acuerdo con el algoritmo, se sustituye el esquema original por estos dos esquemas:

id_cliente_préstamo = (*número_préstamo*, *id_cliente*)
residencia_cliente = (*id_cliente*, *calle_cliente*, *ciudad_cliente*)

Este par de esquemas, que se hallan en la 4FN, elimina la redundancia que se encontró anteriormente.

Como ocurría cuando se trataba solamente con las dependencias funcionales, resultan interesantes las descomposiciones que carecen de pérdidas y que conservan las dependencias. La siguiente circunstancia relativa a las dependencias multivaloradas y a la ausencia de pérdidas muestra que el algoritmo de la Figura 7.17 sólo genera descomposiciones sin pérdidas:

- Sean R un esquema de relación y D un conjunto de dependencias funcionales y multivaloradas de R. Supóngase que R_1 y R_2 forman una descomposición de R. Esa descomposición de R carecerá de pérdidas si, y sólo si, como mínimo, una de las siguientes dependencias multivaloradas se halla en D^+:

$$R_1 \cap R_2 \twoheadrightarrow R_1$$
$$R_1 \cap R_2 \twoheadrightarrow R_2$$

www.detodoprogramacion.com

Recuérdese que se afirmó en el Apartado 7.4.4 que, si $R_1 \cap R_2 \rightarrow R_1$ o $R_1 \cap R_2 \rightarrow R_2$, entonces R_1 y R_2 son descomposiciones de R sin pérdidas. Esta circunstancia relativa a las dependencias multivaloradas es una declaración más general sobre la carencia de pérdidas. Afirma que, para *toda* descomposición de R sin pérdidas en dos esquemas R_1 y R_2, debe cumplirse una de las dos dependencias $R_1 \cap R_2 \twoheadrightarrow R_1$ o $R_1 \cap R_2 \twoheadrightarrow R_2$.

El problema de la conservación de las dependencias al descomponer relaciones se vuelve más complejo en presencia de las dependencias multivaloradas. El Apartado C.1.2 del apéndice aborda este asunto.

7.7 Más formas normales

La cuarta forma normal no es, de ningún modo, la forma normal "definitiva". Como ya se ha visto, las dependencias multivaloradas ayudan a comprender y a abordar algunas formas de repetición de la información que no pueden comprenderse en términos de las dependencias funcionales. Hay restricciones denominadas **dependencias de reunión** que generalizan las dependencias multivaloradas y llevan a otra forma normal denominada **forma normal de reunión por proyección (FNRP)** (la FNRP se denomina en algunos libros **quinta forma normal**). Hay una clase de restricciones todavía más generales, que lleva a una forma normal denominada **forma normal de dominios y claves** (FNDC).

Un problema práctico del empleo de estas restricciones generalizadas es que no sólo es difícil razonar con ellas, sino que tampoco hay un conjunto de reglas de inferencia seguras y completas para razonar sobre las restricciones. Por tanto, FNRP y FNDC se utilizan muy rara vez. El Apéndice C ofrece más detalles sobre estas formas normales.

Destaca por su ausencia en este estudio de las formas normales la **segunda forma normal** (2FN). No se ha estudiado porque sólo es de interés histórico. Simplemente se definirá, y se permitirá al lector experimentar con ella, en el Ejercicio práctico 7.15.

7.8 Proceso de diseño de las bases de datos

Hasta ahora se han examinado aspectos detallados de las formas normales y de la normalización. En este apartado se estudiará el modo de encajar la normalización en el proceso global de diseño de las bases de datos.

Anteriormente, en este capítulo, a partir del Apartado 7.3, se ha dado por supuesto que se tenía un esquema de relación R y que se procedía a normalizarlo. Hay varios modos de obtener ese esquema R:

1. R puede haberse generado al convertir un diagrama E-R en un conjunto de esquemas de relación.
2. R puede haber sido una sola relación que contenía *todos* los atributos que resultaban de interés. El proceso de normalización divide a R en relaciones más pequeñas.
3. R puede haber sido el resultado de algún diseño ad hoc de las relaciones, que hay que comprobar para asegurarse de que satisface la forma normal deseada.

En el resto de este apartado se examinarán las implicaciones de estos enfoques. También se examinarán algunos aspectos prácticos del diseño de las bases de datos, incluida la desnormalización para el rendimiento y ejemplos de mal diseño que no son detectados por la normalización.

7.8.1 El modelo ER y la normalización

Cuando se definen con cuidado los diagramas E-R, identificando correctamente todas las entidades, los esquemas de relación generados a partir de ellos no deben necesitar mucha más normalización. No obstante, puede haber dependencias funcionales entre los atributos de alguna entidad. Por ejemplo, supóngase que la entidad *empleado* tiene los atributos *número_departamento* y *dirección_departamento*, y que hay una dependencia funcional *número_departamento* → *dirección_departamento*. Habrá que normalizar la relación generada a partir de *empleado*.

La mayor parte de los ejemplos de este tipo de dependencias surge de un mal diseño del diagrama E-R. En el ejemplo anterior, si se hubiera diseñado correctamente el diagrama E-R, se habría creado una entidad *departamento* con el atributo *dirección_departamento* y una relación entre *empleado* y *departamento*.

www.detodoprogramacion.com

De manera parecida, puede que una relación que implique a más de dos entidades no se halle en la forma normal deseable. Como la mayor parte de las relaciones son binarias, estos casos resultan relativamente raros (de hecho, algunas variantes de los diagramas E-R hacen realmente difícil o imposible especificar relaciones no binarias).

Las dependencias funcionales pueden ayudar a detectar un mal diseño E-R. Si las relaciones generadas no se hallan en la forma normal deseada, el problema puede solucionarse en el diagrama E-R. Es decir, la normalización puede llevarse a cabo formalmente como parte del modelado de los datos. De manera alternativa, la normalización puede dejarse a la intuición del diseñador durante el modelado E-R, y puede hacerse formalmente sobre las relaciones generadas a partir del modelo E-R.

El lector atento se habrá dado cuenta de que para que se pudiera ilustrar la necesidad de las dependencias multivaloradas y de la cuarta forma normal hubo que comenzar con esquemas que no se obtuvieron a partir del diseño E-R. En realidad, el proceso de creación de diseños E-R tiende a generar diseños 4FN. Si se cumple alguna dependencia multivalorada y no la implica la dependencia funcional correspondiente, suele proceder de alguna de las fuentes siguientes:

- Una relación de varios a varios.
- Un atributo multivalorado de un conjunto de entidades.

En las relaciones de varios a varios cada conjunto de entidades relacionado tiene su propio esquema y hay un esquema adicional para el conjunto de relaciones. Para los atributos multivalorados se crea un esquema diferente que consta de ese atributo y de la clave primaria del conjunto de entidades (como en el caso del atributo *nombre_subordinado* del conjunto de entidades *empleado*).

El enfoque de las relaciones universales para el diseño de bases de datos relacionales parte de la suposición de que sólo hay un esquema de relación que contenga todos los atributos de interés. Este esquema único define la manera en que los usuarios y las aplicaciones interactúan con la base de datos.

7.8.2 Denominación de los atributos y de las relaciones

Una característica deseable del diseño de bases de datos es la **asunción de un rol único**, lo que significa que cada nombre de atributo tiene un significado único en toda la base de datos. Esto evita que se utilice el mismo atributo para indicar cosas diferentes en esquemas diferentes. Por ejemplo, puede que, de otra manera, se considerara el empleo del atributo *número* para el número de préstamo en el esquema *préstamo* y para el número de cuenta en el esquema *cuenta*. La reunión de una relación del esquema *préstamo* con otra de *cuenta* carece de significado ("información de los pares préstamo-cuenta en los que coincide el número de préstamo y el de cuenta"). Aunque los usuarios y los desarrolladores de aplicaciones pueden trabajar con esmero para garantizar el empleo del *número* correcto en cada circunstancia, tener nombres de atributo diferentes para el número de préstamo y para el de cuenta sirve para reducir los errores de los usuarios. De hecho, se ha observado la suposición de un rol único en los diseños de bases de datos de este libro, y su aplicación es una buena práctica general que conviene seguir.

Aunque es una buena idea hacer que los nombres de los atributos incompatibles sean diferentes, si los atributos de relaciones diferentes tienen el mismo significado, puede ser conveniente emplear el mismo nombre de atributo. Por ejemplo, se han utilizado los nombres de atributo *id_clente* e *id_empleado* en los conjuntos de entidades *cliente* y *empleado* (y en sus relaciones). Si deseáramos generalizar esos conjuntos de entidades mediante la creación del conjunto de entidades *persona*, habría que renombrar el atributo. Por tanto, aunque no se tenga actualmente una generalización de *cliente* y de *empleado*, si se prevé esa posibilidad es mejor emplear el mismo nombre en los dos conjuntos de relaciones (y en sus relaciones).

Aunque, técnicamente, el orden de los nombres de los atributos en los esquemas no tiene ninguna importancia, es costumbre relacionar en primer lugar los atributos de la clave primaria. Esto facilita la lectura de los resultados predeterminados (como los generados por **select ***).

En los esquemas de bases de datos de gran tamaño los conjuntos de relaciones (y los esquemas derivados) se suelen denominar mediante la concatenación de los nombres de los conjuntos de entidades a los que hacen referencia, quizás con guiones o caracteres de subrayado intercalados. En este libro se han utilizado algunos nombres de este tipo, por ejemplo, *sucursal_cuenta* y *sucursal_préstamo*. Se han utilizado los nombres *prestatario* o *impositor* en lugar de otros concatenados de mayor longitud como *cliente*

www.detodoprogramacion.com

_préstamo o *cliente_cuenta*. Esto resultaba aceptable, ya que no es difícil recordar las entidades asociadas a unas pocas relaciones. No siempre se pueden crear nombres de relaciones mediante la mera concatenación; por ejemplo, la relación jefe o trabaja-para entre empleados no tendría mucho sentido si se llamara *empleado_empleado*. De manera parecida, si hay varios conjuntos de relaciones posibles entre un par de conjuntos de entidades, los nombres de las relaciones deben incluir componentes adicionales para identificar cada relación.

Las diferentes organizaciones tienen costumbres diferentes para la denominación de las entidades. Por ejemplo, a un conjunto de entidades de clientes se le puede denominar *cliente* o *clientes*. En los diseños de las bases de datos de este libro se ha decidido utilizar la forma singular. Es aceptable tanto el empleo del singular como el del plural, siempre y cuando la convención se utilice de manera consistente en todas las entidades.

A medida que los esquemas aumentan de tamaño, con un número creciente de relaciones, el empleo de una denominación consistente de los atributos, de las relaciones y de las entidades facilita mucho la vida de los diseñadores de bases de datos y de los programadores de aplicaciones.

7.8.3 Desnormalización para el rendimiento

A veces, los diseñadores de bases de datos escogen un esquema que tiene información redundante; es decir, que no está normalizado. Utilizan la redundancia para mejorar el rendimiento de aplicaciones concretas. La penalización sufrida por no emplear un esquema normalizado es el trabajo adicional (en términos de tiempos de codificación y de ejecución) de mantener consistentes los datos redundantes.

Por ejemplo, supóngase que hay que mostrar el nombre del titular junto con el número de cuenta y con el saldo cada vez que se accede a la cuenta. En el esquema normalizado esto exige una reunión de *cuenta* con *impositor*.

Una alternativa al cálculo de la reunión sobre la marcha es almacenar una relación que contenga todos los atributos de *cuenta* y de *impositor*. Esto hace más rápida la visualización de la información de la cuenta. Sin embargo, la información del saldo de la cuenta se repite para cada uno de los titulares, y la aplicación debe actualizar todas las copias cada vez que se actualice el saldo. El proceso de tomar un esquema normalizado y hacer que no esté normalizado se denomina **desnormalización**, y los diseñadores lo utilizan para ajustar el rendimiento de los sistemas para que den soporte a las operaciones críticas en el tiempo.

Una opción mejor, soportada hoy en día por muchos sistemas de bases de datos, es emplear el esquema normalizado y, además, almacenar la reunión de *cuenta* e *impositor* en forma de vista materializada (recuérdese que las vistas materializadas son vistas cuyo resultado se almacena en la base de datos y se actualiza cuando se actualizan las relaciones utilizadas en ellas). Al igual que la desnormalización, el empleo de las vistas materializadas supone sobrecargas de espacio y de tiempo; sin embargo, presenta la ventaja de que conservar actualizadas las vistas es labor del sistema de bases de datos, no del programador de la aplicación.

7.8.4 Otros problemas del diseño

Hay algunos aspectos del diseño de bases de datos que la normalización no aborda y, por tanto, pueden llevar a un mal diseño de la base de datos. Los datos relativos al tiempo o a intervalos temporales presentan varios de esos problemas. A continuación se ofrecen algunos ejemplos; evidentemente, conviene evitar esos diseños.

Considérese una base de datos empresarial, en la que se desea almacenar los beneficios de varias compañías a lo largo de varios años. Se puede utilizar la relación *beneficios* (*id_empresa, año, importe*) para almacenar la información sobre los beneficios. La única dependencia funcional de esta relación es *id_empresa, año* → *importe*, y se halla en la FNBC.

Un diseño alternativo es el empleo de varias relaciones, cada una de las cuales almacena los beneficios de un año diferente. Supóngase que los años que nos interesan son 2000, 2001 y 2002; se tendrán, entonces, relaciones de la forma *beneficios_2000*, *beneficios_2001* y *beneficios_2002*, todas las cuales se hallan en el esquema (*id_empresa, beneficios*). En este caso, la única dependencia funcional de cada relación será *id_empresa* → *beneficios*, por lo que esas relaciones también se hallan en la FNBC.

www.detodoprogramacion.com

No obstante, este diseño alternativo es, claramente, una mala idea—habría que crear una relación nueva cada año, y también habría que escribir consultas nuevas todos los años, para tener en cuenta cada nueva relación. Las consultas también serían más complicadas, ya que probablemente tendrían que hacer referencia a muchas relaciones.

Otra manera de representar esos mismos datos es tener una sola relación *año_empresa* (*id_empresa, beneficios_2000, beneficios_2001, beneficios_2002*). En este caso, las únicas dependencias funcionales van de *id_empresa* hacia los demás atributos y, una vez más, la relación se halla en la FNBC. Este diseño también es una mala idea, ya que tiene problemas parecidos a los del diseño anterior—es decir, habría que modificar el esquema de la relación y escribir consultas nuevas cada año. Las consultas también serían más complicadas, ya que puede que tuvieran que hacer referencia a muchos atributos.

Las representaciones como las de la relación *año_empresa*, con una columna para cada valor de cada atributo, se denominan de **referencias cruzadas**; se emplean mucho en las hojas de cálculo, en los informes y en las herramientas de análisis de datos. Aunque esas representaciones resultan útiles para mostrárselas a los usuarios, por las razones que se acaban de dar, no resultan deseables en el diseño de bases de datos. Se han propuesto extensiones del SQL para pasar los datos de la representación relacional normal a la de referencias cruzadas, para su visualización.

7.9 Modelado de datos temporales

Supóngase que en el banco se conservan datos que no sólo muestran la dirección actual de cada cliente, sino también todas las direcciones anteriores de las que el banco tenga noticia. Se pueden formular consultas como "Averiguar todos los clientes que vivían en Vigo en 1981". En ese caso, puede que se tengan varias direcciones por cliente. Cada dirección tiene asociadas una fecha de comienzo y otra de finalización, que indican el periodo en que el cliente residió en esa dirección. Se puede utilizar un valor especial para la fecha de finalización, por ejemplo, nulo, o un valor que se halle claramente en el futuro, como 31/12/9999, para indicar que el cliente sigue residiendo en esa dirección.

En general, los **datos temporales** son datos que tienen asociado un intervalo de tiempo durante el cual son **válidos**[4]. Se utiliza el término **instantánea** de los datos para indicar su valor en un momento determinado. Por tanto, una instantánea de los datos de los clientes muestra el valor de todos los atributos de los clientes, como la dirección, en un momento concreto.

El modelado de datos temporales es una cuestión interesante por varios motivos. Por ejemplo, supóngase que se tiene una entidad *cliente* con la que se desea asociar una dirección que varía con el tiempo. Para añadir información temporal a una dirección hay que crear un atributo multivalorado, cada uno de cuyos valores es un valor compuesto que contiene una dirección y un intervalo de tiempo. Además de los valores de los atributos que varían con el tiempo, puede que las propias entidades tengan un periodo de validez asociado. Por ejemplo, la entidad cuenta puede tener un periodo de validez desde la fecha de apertura hasta la de cancelación. Las relaciones también pueden tener asociados periodos de validez. Por ejemplo, la relación *impositor* entre un cliente y una cuenta puede registrar el momento en que el cliente pasó a ser titular de la cuenta. Por tanto, habría que añadir intervalos de validez a los valores de los atributos, a las entidades y a las relaciones. La adición de esos detalles a los diagramas E-R hace que sean muy difíciles de crear y de comprender. Ha habido varias propuestas para extender la notación E-R para que especifique de manera sencilla que un atributo o una relación varía con el tiempo, pero no hay ninguna norma aceptada al respecto.

Cuando se realiza un seguimiento del valor de los datos a lo largo del tiempo, puede que dejen de cumplirse dependencias funcionales que se suponía que se cumplían, como

$$id_cliente \rightarrow calle_cliente\ ciudad_cliente$$

En su lugar, se cumpliría la restricción siguiente (expresada en castellano): "*id_cliente* sólo tiene un valor de *calle_cliente* y de *ciudad_cliente* para cada momento t dado".

Las dependencias funcionales que se cumplen en un momento concreto se denominan dependencias funcionales temporales. Formalmente, la **dependencia funcional temporal** $X \xrightarrow{T} Y$ se cumple en el

4. Hay otros modelos de datos temporales que distinguen entre **periodo de validez** y **momento de la transacción**; el último registra el momento en que se registró un hecho en la base de datos. Para simplificar se prescinde de estos detalles.

www.detodoprogramacion.com

esquema de relación R si, para todos los ejemplares legales r de R, todas las instantáneas de r satisfacen la dependencia funcional $X \rightarrow Y$.

Se puede extender la teoría del diseño de bases de datos relacionales para que tenga en cuenta las dependencias funcionales temporales. Sin embargo, el razonamiento con las dependencias funcionales normales ya resulta bastante difícil, y pocos diseñadores están preparados para trabajar con las dependencias funcionales temporales.

En la práctica, los diseñadores de bases de datos recurren a enfoques más sencillos para diseñar las bases de datos temporales. Un enfoque empleado con frecuencia es diseñar toda la base de datos (incluidos los diseños E-R y relacional) ignorando las modificaciones temporales (o, lo que es lo mismo, tomando sólo una instantánea en consideración). Tras esto, el diseñador estudia las diferentes relaciones y decide las que necesitan que se realice un seguimiento de su variación temporal.

El paso siguiente es añadir información sobre los periodos de validez a cada una de esas relaciones, añadiendo como atributos el momento de inicio y el de finalización. Por ejemplo, supóngase que se tiene la relación

asignatura (*id_asignatura*, *denominación_asignatura*)

que asocia la denominación de cada asignatura con la asignatura correspondiente, que queda identificada mediante su identificador de asignatura. La denominación de la asignatura puede variar con el tiempo, lo cual se puede tener en cuenta añadiendo un rango de validez; el esquema resultante sería:

asignatura (*id_asignatura*, *denominación_asignatura*, *comienzo*, *final*)

Un ejemplar de esta relación puede tener dos registros (CS101, "Introducción a la programación", 01/01/1985, 31/12/2000) y (CS101,"Introdución a C", 01/01/2001, 31/12/9999). Si se actualiza la relación para cambiar la denominación de la asignatura a "Introducción a Java", se actualizaría la fecha "31/12/9999" a la correspondiente al momento hasta el que fue válido el valor anterior ("Introducción a C"), y se añadiría una tupla nueva que contendría la nueva denominación ("Introducción a Java"), con la fecha de inicio correspondiente.

Si otra relación tuviera una clave externa que hiciera referencia a una relación temporal, el diseñador de la base de datos tendría que decidir si la referencia se hace a la versión actual de los datos o a los datos de un momento concreto. Por ejemplo, una relación que registre la asignación de aulas actual para cada asignatura puede hacer referencia de manera implícita al valor temporal actual asociado con cada *id_asignatura*. Por otro lado, los registros del expediente de cada estudiante deben hacer referencia a la denominación de la asignatura en el momento en que la cursó ese estudiante. En este último caso, la relación que hace la referencia también debe almacenar la información temporal, para identificar cada registro concreto de la relación *asignatura*.

La clave primaria original de una relación temporal ya no identificaría de manera unívoca a cada tupla. Para solucionar este problema se pueden añadir a la clave primaria los atributos de fecha de inicio y de finalización. Sin embargo, persisten algunos problemas:

- Es posible almacenar datos con intervalos que se solapen, pero la restricción de clave primaria no puede detectarlo. Si el sistema soporta un tipo nativo *periodo de validez*, puede detectar y evitar esos intervalos temporales que se solapan.
- Para especificar una clave externa que haga referencia a una relación así, las tuplas que hacen la referencia tienen que incluir los atributos de momento inicial y final como parte de su clave externa, y los valores deberán coincidir con los de la tupla a la que hacen referencia. Además, si la tupla a la que hacen referencia se actualiza (y el momento final que se hallaba en el futuro se actualiza), esa actualización debe propagarse a todas las tuplas que hacen referencia a ella.

  Si el sistema soporta los datos temporales de alguna manera mejor, se puede permitir que la tupla que hace la referencia especifique un momento, en lugar de un rango temporal, y confiar en que el sistema garantice que hay una tupla en la relación a la que hace referencia cuyo periodo de validez contenga ese momento. Por ejemplo, un registro de un expediente puede especificar *id_asignatura* y un momento (por ejemplo, la fecha de comienzo de un trimestre), lo que basta para identificar el registro correcto de la relación *asignatura*.

www.detodoprogramacion.com

Como caso especial frecuente, si todas las referencias a los datos temporales se hacen exclusivamente a los datos actuales, una solución más sencilla es no añadir información temporal a la relación y, en su lugar, crear la relación *historial* correspondiente que contenga esa información temporal, para los valores del pasado. Por ejemplo, en la base de datos bancaria, se puede utilizar el diseño que se ha creado, ignorando las modificaciones temporales, para almacenar únicamente la información actual. Toda la información histórica se traslada a las relaciones históricas. Por tanto, la relación *cliente* sólo puede almacenar la dirección actual, mientras que la relación *historial_cliente* puede contener todos los atributos de *cliente*, con los atributos adicionales *momento_inicial* y *momento_final*.

Aunque no se ha ofrecido ningún método formal para tratar los datos temporales, los problemas estudiados y los ejemplos ofrecidos deben ayudar al lector a diseñar bases de datos que registren datos temporales. Más adelante, en el Apartado 24.2, se tratan otros aspectos del manejo de datos temporales, incluidas las consultas temporales.

7.10 Resumen

- Se han mostrado algunas dificultades del diseño de bases de datos y el modo de diseñar de manera sistemática esquemas de bases de datos que eviten esas dificultades. Entre esas dificultades están la información repetida y la imposibilidad de representar cierta información.
- Se ha mostrado el desarrollo del diseño de bases de datos relacionales a partir de los diseños E-R, cuándo los esquemas se pueden combinar con seguridad y cuándo se deben descomponer. Todas las descomposiciones válidas deben ser sin pérdidas.
- Se han descrito las suposiciones de dominios atómicos y de primera forma normal.
- Se ha introducido el concepto de las dependencias funcionales y se ha utilizado para presentar dos formas normales, la forma normal de Boyce–Codd (FNBC) y la tercera forma normal (3NF).
- Si la descomposición conserva las dependencias, dada una actualización de la base de datos, todas las dependencias funcionales pueden verificarse a partir de las diferentes relaciones, sin necesidad de calcular la reunión de las relaciones de la descomposición.
- Se ha mostrado la manera de razonar con las dependencias funcionales. Se ha puesto un énfasis especial en señalar las dependencias que están implicadas lógicamente por conjuntos de dependencias. También se ha definido el concepto de recubrimiento canónico, que es un conjunto mínimo de dependencias funcionales equivalente a un conjunto dado de dependencias funcionales.
- Se ha descrito un algoritmo para la descomposición de las relaciones en la FNBC. Hay relaciones para las cuales no hay ninguna descomposición en la FNBC que conserve las dependencias.
- Se han utilizado los recubrimientos canónicos para descomponer las relaciones en la 3NF, que es una pequeña relajación de las condiciones de la FNBC. Las relaciones en la 3NF pueden tener alguna redundancia, pero siempre hay una descomposición en la 3NF que conserva las dependencias.
- Se ha presentado el concepto de dependencias multivaloradas, que especifican las restricciones que no pueden especificarse únicamente con las dependencias funcionales. Se ha definido la cuarta forma normal (4FN) con las dependencias multivaloradas. El Apartado C.1.1 del apéndice da detalles del razonamiento sobre las dependencias multivaloradas.
- Otras formas normales, como la FNRP y la FNDC, eliminan formas más sutiles de redundancia. Sin embargo, es difícil trabajar con ellas y se emplean rara vez. El Apéndice C ofrece detalles de estas formas normales.
- Al revisar los temas de este capítulo hay que tener en cuenta que el motivo de que se hayan podido definir enfoques rigurosos del diseño de bases de datos relacionales es que el modelo relacional de datos descansa sobre una base matemática sólida. Ésa es una de las principales ventajas del modelo relacional en comparación con los otros modelos de datos que se han estudiado.

www.detodoprogramacion.com

PARTE 1

Bases de datos relacionales

Un modelo de datos es un conjunto de herramientas conceptuales para la descripción de los datos, las relaciones entre ellos, su semántica y las restricciones de consistencia. Esta parte centra la atención en el modelo relacional.

El modelo relacional utiliza un conjunto de tablas para representar tanto los datos como las relaciones entre ellos. Su simplicidad conceptual ha conducido a su adopción generalizada; actualmente, una amplia mayoría de los productos de bases de datos se basan en el modelo relacional. En la Parte 9 se expone una visión general de cuatro sistemas de bases de datos relacionales muy utilizados.

Aunque el modelo relacional, que se trata en el Capítulo 2, describe los datos en los niveles lógico y de vistas, es un modelo de datos de nivel inferior comparado con el modelo entidad-relación, que se explica en la Parte 2.

Para poner a disposición de los usuarios los datos de una base de datos relacional hay que abordar varios aspectos. Uno de ellos es determinar uno de los distintos lenguajes de consultas para expresar las peticiones de datos. Los Capítulos 3 y 4 tratan el lenguaje SQL, que es el lenguaje de consultas más utilizado hoy en día. El Capítulo 5 trata en primer lugar dos lenguajes de consultas formales, el cálculo relacional de tuplas y el cálculo relacional de dominios, que son lenguajes declarativos basados en la lógica matemática. Estos dos lenguajes formales constituyen la base para dos lenguajes más amigables para los usuarios: QBE y Datalog, que se estudian en ese capítulo.

Otro aspecto es la integridad y la protección de los datos; las bases de datos deben proteger sus datos del daño que puedan causar las acciones de los usuarios, sean involuntarias o intencionadas. El componente de mantenimiento de la integridad de la base de datos garantiza que las actualizaciones no violen las restricciones de integridad que se hayan especificado para los datos. El componente de protección de la base de datos incluye el control de acceso para restringir las acciones permitidas a cada usuario. El Capítulo 4 trata los problemas de integridad y de protección. Los problemas de protección y de integridad están presentes independientemente del modelo de datos pero, en aras de la concreción, se estudiarán en el contexto del modelo relacional.

www.detodoprogramacion.com

CAPÍTULO 2

El modelo relacional

El modelo relacional es hoy en día el principal modelo de datos para las aplicaciones comerciales de procesamiento de datos. Ha conseguido esa posición destacada debido a su simplicidad, lo cual facilita el trabajo del programador en comparación con modelos anteriores, como el de red y el jerárquico.

En este capítulo se estudian en primer lugar los fundamentos del modelo relacional. A continuación se describe el álgebra relacional, que se usa para especificar las solicitudes de información. El álgebra relacional no es cómoda de usar, pero sirve de base formal para lenguajes de consultas que sí lo son y que se estudiarán más adelante, incluido el ampliamente usado lenguaje de consultas SQL, el cual se trata con detalle en los Capítulos 3 y 4.

Existe una amplia base teórica para las bases de datos relacionales. En este capítulo se estudia la parte de esa base teórica referida a las consultas. En los Capítulos 6 y 7 se examinarán aspectos de la teoría de las bases de datos relacionales que ayudan en el diseño de esquemas de bases de datos relacionales, mientras que en los Capítulos 13 y 14 se estudian aspectos de la teoría que se refieren al procesamiento eficiente de consultas.

2.1 La estructura de las bases de datos relacionales

Una base de datos relacional consiste en un conjunto de **tablas**, a cada una de las cuales se le asigna un nombre exclusivo. Cada fila de la tabla representa una *relación* entre un conjunto de valores. De manera informal, cada tabla es un conjunto de entidades, y cada fila es una entidad, tal y como se estudió en el Capítulo 1. Dado que cada tabla es un conjunto de tales relaciones, hay una fuerte correspondencia entre el concepto de *tabla* y el concepto matemático de *relación*, del que toma su nombre el modelo de datos relacional. A continuación se introduce el concepto de relación.

En este capítulo se usarán varias relaciones diferentes para ilustrar los diversos conceptos subyacentes al modelo de datos relacional. Estas relaciones representan parte de una entidad bancaria. Puede que no se correspondan con el modo en que se pueda estructurar realmente una base de datos bancaria, pero así se simplificará la presentación. Los criterios sobre la adecuación de las estructuras relacionales se estudian con gran detalle en los Capítulos 6 y 7.

2.1.1 Estructura básica

Considérese la tabla *cuenta* de la Figura 2.1. Tiene tres cabeceras de columna: *número_cuenta*, *nombre_sucursal* y *saldo*. Siguiendo la terminología del modelo relacional, se puede hacer referencia a estas cabeceras como **atributos**. Para cada atributo hay un conjunto de valores permitidos, denominado **dominio** de ese atributo. Para el atributo *nombre_sucursal*, por ejemplo, el dominio es el conjunto de todos los nombres de sucursal. Supóngase que D_1 denota el conjunto de todos los números de cuenta, D_2 el conjunto de todos los nombres de sucursal y D_3 el conjunto de todos los saldos. Todas las filas de *cuenta* deben consistir en una tupla $(v_1,\ v_2,\ v_3)$, donde v_1 es un número de cuenta (es decir, v_1 está en

www.detodoprogramacion.com

número_cuenta	*nombre_sucursal*	*saldo*
C-101	Centro	500
C-102	Navacerrada	400
C-201	Galapagar	900
C-215	Becerril	700
C-217	Galapagar	750
C-222	Moralzarzal	700
C-305	Collado Mediano	350

Figura 2.1 La relación *cuenta*.

el dominio D_1), v_2 es un nombre de sucursal (v_2 en D_2) y v_3 es un saldo (v_3 en D_3). En general, *cuenta* sólo contendrá un subconjunto del conjunto de todas las filas posibles. Por tanto, *cuenta* será un subconjunto de

$$D_1 \times D_2 \times D_3$$

En general, una **tabla** de n atributos debe ser un subconjunto de

$$D_1 \times D_2 \times \cdots \times D_{n-1} \times D_n$$

Los matemáticos definen las **relaciones** como subconjuntos del producto cartesiano de la lista de dominios. Esta definición se corresponde de manera casi exacta con la definición de *tabla* dada anteriormente. La única diferencia es que aquí se han asignado nombres a los atributos, mientras que los matemáticos sólo usan "nombres" numéricos, usando el entero 1 para denotar el atributo cuyo dominio aparece en primer lugar de la lista de dominios, 2 para el atributo cuyo dominio aparece en segundo lugar, etc. Como las tablas son, esencialmente, relaciones, se usarán los términos matemáticos **relación** y **tupla** en lugar de los términos **tabla** y **fila**. Una **variable tupla** es una variable que representa una tupla; en otras palabras, una variable tupla es una variable cuyo dominio es el conjunto de todas las tuplas.

En la relación *cuenta* de la Figura 2.1 hay siete tuplas. Supóngase que la variable tupla t hace referencia a la primera tupla de la relación. Se usa la notación t[*número_cuenta*] para denotar el valor de t en el atributo *número_cuenta*. Por tanto, t[*número_cuenta*] = "C-101" y t[*nombre_sucursal*] = "Centro". De manera alternativa, se puede escribir t[1] para denotar el valor de la tupla t en el primer atributo (*número_cuenta*), t[2] para denotar *nombre_sucursal*, etc. Dado que las relaciones son conjuntos de tuplas, se usa la notación matemática $t \in r$ para denotar que la tupla t está en la relación r.

El orden en que aparecen las tuplas en cada relación es irrelevante, dado que una relación es un *conjunto* de tuplas. Por tanto, no importa si las tuplas de una relación aparecen ordenadas, como en la Figura 2.1, o desordenadas, como en la Figura 2.2; las relaciones de las dos figuras son la misma, ya que las dos contienen el mismo conjunto de tuplas.

Se exige que, para todas las relaciones r, los dominios de todos los atributos de r sean atómicos. Un dominio es **atómico** si los elementos del dominio se consideran unidades indivisibles. Por ejemplo, el conjunto de los enteros es un dominio atómico, pero el conjunto de todos los conjuntos de enteros

número_cuenta	*nombre_sucursal*	*saldo*
C-101	Centro	500
C-215	Becerril	700
C-102	Navacerrada	400
C-305	Collado Mediano	350
C-201	Galapagar	900
C-222	Moralzarzal	700
C-217	Galapagar	750

Figura 2.2 La relación *cuenta* con las tuplas desordenadas.

www.detodoprogramacion.com

es un dominio no atómico. La diferencia es que no se suele considerar que los enteros tengan partes constituyentes, pero sí se considera que los conjuntos de enteros las tienen; por ejemplo, los enteros que forman cada conjunto. Lo importante no es lo que sea el propio dominio, sino la manera en que se usan los elementos del dominio en la base de datos. El dominio de todos los enteros sería no atómico si se considerara que cada entero es una lista ordenada de cifras. En todos los ejemplos se supondrá que los dominios son atómicos. En el Capítulo 9 se estudiarán extensiones al modelo de datos relacional para permitir dominios no atómicos.

Es posible que varios atributos tengan el mismo dominio. Por ejemplo, supóngase la relación *cliente* con los tres atributos *nombre_cliente*, *calle_cliente* y *ciudad_cliente* y una relación *empleado* con el atributo *nombre_empleado*. Es posible que los atributos *nombre_cliente* y *nombre_empleado* tengan el mismo dominio, el conjunto de todos los nombres de persona, que en el nivel físico es el conjunto de todas las cadenas de caracteres. Los dominios de *saldo* y *nombre_sucursal*, por otra parte, deberían ser distintos. Quizás sea menos evidente si *nombre_cliente* y *nombre_sucursal* deberían tener el mismo dominio. En el nivel físico, tanto los nombres de los clientes como los nombres de las sucursales son cadenas de caracteres. Sin embargo, en el nivel lógico puede que se desee que *nombre_cliente* y *nombre_sucursal* tengan dominios diferentes.

Un valor de dominio que es miembro de todos los dominios posibles es el valor **nulo**, que indica que el valor es desconocido o no existe. Por ejemplo, supóngase que se incluye el atributo *número_teléfono* en la relación *cliente*. Puede ocurrir que algún cliente no tenga número de teléfono, o que su número de teléfono no figure en la guía. Entonces habrá que recurrir a los valores nulos para indicar que el valor es desconocido o que no existe. Más adelante se verá que los valores nulos crean algunas dificultades cuando se tiene acceso a la base de datos o se la actualiza y que, por tanto, deben eliminarse si es posible. Se supondrá inicialmente que no hay valores nulos y en el Apartado 2.5 se describirá el efecto de los valores nulos en las diferentes operaciones.

2.1.2 Esquema de la base de datos

Cuando se habla de bases de datos se debe diferenciar entre el **esquema de la base de datos**, que es el diseño lógico de la misma, y el **ejemplar de la base de datos**, que es una instantánea de los datos de la misma en un momento dado.

El concepto de relación se corresponde con el concepto de variable de los lenguajes de programación. El concepto de **esquema de la relación** se corresponde con el concepto de definición de tipos de los lenguajes de programación.

Resulta conveniente dar nombre a los esquemas de las relaciones, igual que se dan nombres a las definiciones de los tipos en los lenguajes de programación. Se adopta el convenio de usar nombres en minúsculas para las relaciones y nombres que comiencen por una letra mayúscula para los esquemas de las relaciones. Siguiendo esta notación se usará *Esquema_cuenta* para denotar el esquema de la relación *cuenta*. Por tanto,

$$Esquema_cuenta = (\textit{número_cuenta}, \textit{nombre_sucursal}, \textit{saldo})$$

Se denota el hecho de que *cuenta* es una relación de *Esquema_cuenta* mediante

$$cuenta\big(Esquema_cuenta\big)$$

En general, los esquemas de las relaciones consisten en una lista de los atributos y de sus dominios correspondientes. La definición exacta del dominio de cada atributo no será relevante hasta que se estudie el lenguaje SQL en los Capítulos 3 y 4.

El concepto de **ejemplar de la relación** se corresponde con el concepto de valor de una variable en los lenguajes de programación. El valor de una variable dada puede cambiar con el tiempo; de manera parecida, el contenido del ejemplar de una relación puede cambiar con el tiempo cuando la relación se actualiza. Sin embargo, se suele decir simplemente "relación" cuando realmente se quiere decir "ejemplar de la relación".

Como ejemplo de ejemplar de una relación, considérese la relación *sucursal* de la Figura 2.3. El esquema de esa relación es

www.detodoprogramacion.com

nombre_sucursal	*ciudad_sucursal*	*activos*
Becerril	Aluche	400.000
Centro	Arganzuela	9.000.000
Collado Mediano	Aluche	8.000.000
Galapagar	Arganzuela	7.100.000
Moralzarzal	La Granja	2.100.000
Navacerrada	Aluche	1.700.000
Navas de la Asunción	Alcalá de Henares	300.000
Segovia	Cerceda	3.700.000

Figura 2.3 La relación *sucursal*.

$$Esquema_sucursal = (nombre_sucursal, ciudad_sucursal, activos)$$

Obsérvese que el atributo *nombre_sucursal* aparece tanto en *Esquema_sucursal* como en *Esquema_cuenta*. Esta duplicidad no es una coincidencia. Más bien, usar atributos comunes en los esquemas de las relaciones es una manera de relacionar las tuplas de relaciones diferentes. Por ejemplo, supóngase que se desea obtener información sobre todas las cuentas abiertas en sucursales ubicadas en Arganzuela. Primero se busca en la relación *sucursal* para encontrar los nombres de todas las sucursales situadas en Arganzuela. A continuación y para cada una de ellas, se examina la relación *cuenta* para encontrar la información sobre las cuentas abiertas en esa sucursal.

Siguiendo con el ejemplo bancario, se necesita una relación que describa información sobre los clientes. El esquema de la relación es:

$$Esquema_cliente = (nombre_cliente, calle_cliente, ciudad_cliente)$$

La Figura 2.4 muestra un ejemplo de la relación *cliente* (*Esquema_cliente*). Obsérvese que se ha omitido el atributo *id_cliente*, que se usó en el Capítulo 1, ya que se considerarán esquemas de relación más pequeños en nuestro ejemplo de una base de datos bancaria. Se da por supuesto que el nombre de cliente identifica unívocamente a cada cliente—obviamente, puede que esto no sea cierto en el mundo real, pero la suposición hace los ejemplos más sencillos de entender. En una base de datos del mundo real, *id_cliente* (que podría ser *número_seguridad_social* o un identificador generado por el banco) serviría para identificar unívocamente a los clientes.

También se necesita una relación que describa la asociación entre los clientes y las cuentas. El esquema de la relación que describe esta asociación es:

$$Esquema_impositor = (nombre_cliente, número_cuenta)$$

nombre_cliente	*calle_cliente*	*ciudad_cliente*
Abril	Preciados	Valsaín
Amo	Embajadores	Arganzuela
Badorrey	Delicias	Valsaín
Fernández	Jazmín	León
Gómez	Carretas	Cerceda
González	Arenal	La Granja
López	Mayor	Peguerinos
Pérez	Carretas	Cerceda
Rodríguez	Yeserías	Cádiz
Rupérez	Ramblas	León
Santos	Mayor	Peguerinos
Valdivieso	Goya	Vigo

Figura 2.4 La relación *cliente*.

www.detodoprogramacion.com

nombre_cliente	*número_cuenta*
Abril	C-305
Gómez	C-215
González	C-101
González	C-201
López	C-102
Rupérez	C-222
Santos	C-217

Figura 2.5 La relación *impositor*.

La Figura 2.5 muestra un ejemplo de la relación *impositor* (*Esquema_impositor*).

Puede parecer que para este ejemplo bancario se podría tener sólo un esquema de relación, en vez de tener varios. Es decir, puede resultar más sencillo para el usuario pensar en términos de un único esquema de relación, en lugar de en varios esquemas. Supóngase que sólo se usara una relación para el ejemplo, con el esquema

(*nombre_sucursal*, *ciudad_sucursal*, *activos*, *nombre_cliente*, *calle_cliente*
ciudad_cliente, *número_cuenta*, *saldo*)

Obsérvese que, si un cliente tiene varias cuentas, hay que repetir su dirección una vez por cada cuenta. Es decir, hay que repetir varias veces parte de la información. Esta repetición malgasta espacio, pero se evita mediante el empleo de varias relaciones mostradas anteriormente.

Además, si una sucursal no tiene ninguna cuenta (por ejemplo, una sucursal recién abierta que todavía no tenga clientes), no se puede construir una tupla completa en la relación única anterior, dado que no hay todavía ningún dato disponible referente a *cliente* ni a *cuenta*. Para representar las tuplas incompletas hay que usar valores *nulos* que indiquen que ese valor es desconocido o no existe. Por tanto, en el ejemplo presente, los valores de *nombre_cliente*, *calle_cliente*, etc., deben ser nulos. Al emplear varias relaciones se puede representar la información de las sucursales del banco sin clientes sin necesidad de valores nulos. Se usa simplemente una tupla en *Esquema_sucursal* para representar la información de la sucursal, y sólo se crean tuplas en los otros esquemas cuando esté disponible la información correspondiente.

En el Capítulo 7 se estudiarán los criterios para decidir cuándo un conjunto de esquemas de relaciones es más adecuado que otro en términos de repetición de la información y de la existencia de valores nulos. Por ahora se supondrá que los esquemas de las relaciones vienen dados de antemano.

Se incluyen dos relaciones más para describir los datos de los préstamos concedidos en las diferentes sucursales del banco:

Esquema_préstamo = (*número_préstamo*, *nombre_sucursal*, *importe*)
Esquema_prestatario = (*nombre_cliente, número_préstamo*)

Las Figuras 2.6 y 2.7, respectivamente, muestran las relaciones de ejemplo *préstamo* (*Esquema_préstamo*) y *prestatario* (*Esquema_prestatario*).

Los esquemas de relación se corresponden con el conjunto de tablas que podrían generarse con el método descrito en el Apartado 1.6. Obsérvese que la relación *cliente* puede contener información sobre clientes que no tengan ni cuenta ni préstamo en el banco. La entidad bancaria aquí descrita servirá como ejemplo principal en este capítulo. Cuando sea necesario, habrá que introducir más esquemas de relaciones para ilustrar casos concretos.

2.1.3 Claves

Es necesario disponer de un modo de especificar la manera en que las tuplas de una relación dada se distingan entre sí. Esto se expresa en términos de sus atributos. Es decir, los valores de los valores de los atributos de una tupla deben ser tales que puedan *identificarla unívocamente*. En otras palabras, no

www.detodoprogramacion.com

número_préstamo	*nombre_sucursal*	*importe*
P-11	Collado Mediano	900
P-14	Centro	1.500
P-15	Navacerrada	1.500
P-16	Navacerrada	1.300
P-17	Centro	1.000
P-23	Moralzarzal	2.000
P-93	Becerril	500

Figura 2.6 La relación *préstamo*.

se permite que dos tuplas de una misma relación tengan exactamente los mismos valores en todos sus atributos.

Una **superclave** es un conjunto de uno o varios atributos que, considerados conjuntamente, permiten identificar de manera unívoca una tupla de la relación. Por ejemplo, el atributo *id_cliente* de la relación *cliente* es suficiente para distinguir una tupla *cliente* de otra. Por tanto, *id_cliente* es una superclave. De manera parecida, la combinación de *nombre_cliente* e *id_cliente* constituye una superclave para la relación *cliente*. El atributo *nombre_cliente* de *cliente* no es una superclave, ya que es posible que varias personas se llamen igual.

El concepto de superclave no es suficiente para nuestros propósitos, ya que, como se ha podido ver, las superclaves pueden contener atributos innecesarios. Si C es una superclave, entonces también lo es cualquier superconjunto de C. A menudo resultan interesantes superclaves para las que ninguno de sus subconjuntos constituya una superclave. Esas superclaves mínimas se denominan **claves candidatas**.

Es posible que varios conjuntos diferentes de atributos puedan ejercer como claves candidatas. Supóngase que una combinación de *nombre_cliente* y de *calle_cliente* sea suficiente para distinguir entre los miembros de la relación *cliente*. Entonces, tanto {*id_cliente*} como {*nombre_cliente, calle_cliente*} son claves candidatas. Aunque los atributos *id_cliente* y *nombre_cliente* en conjunto pueden diferenciar las tuplas *cliente*, su combinación no forma una clave candidata, ya que el atributo *id_cliente* por sí solo ya lo es.

Se usará el término **clave primaria** para denotar una clave candidata que ha elegido el diseñador de la base de datos como medio principal para la identificación de las tuplas de una relación. Las claves (sean primarias, candidatas o superclaves) son propiedades de toda la relación, no de cada una de las tuplas. Ninguna pareja de tuplas de la relación puede tener simultáneamente el mismo valor de los atributos de la clave. La selección de una clave representa una restricción de la empresa del mundo real que se está modelando.

Las claves candidatas deben escogerse con cuidado. Como se ha indicado, el nombre de una persona evidentemente no es suficiente, ya que puede haber mucha gente con el mismo nombre. En Estados Unidos el atributo número de la seguridad social de cada persona sería clave candidata. Dado que los residentes extranjeros no suelen tener número de la seguridad social, las empresas internacionales deben generar sus propios identificadores unívocos. Una alternativa es usar como clave alguna combinación exclusiva de otros atributos.

nombre_cliente	*número_préstamo*
Fernández	P-16
Gómez	P-11
Gómez	P-23
López	P-15
Pérez	P-93
Santos	P-17
Sotoca	P-14
Valdivieso	P-17

Figura 2.7 La relación *prestatario*.

www.detodoprogramacion.com

La clave primaria debe escogerse de manera que los valores de sus atributos no se modifiquen nunca, o muy rara vez. Por ejemplo, el campo domicilio de una persona no debe formar parte de la clave primaria, ya que es probable que se modifique. Por otra parte, está garantizado que los números de la seguridad social no cambian nunca. Los identificadores exclusivos generados por las empresas no suelen cambiar, salvo si se produce una fusión entre dos de ellas; en ese caso, puede que el mismo identificador haya sido emitido por ambas empresas, y puede ser necesaria una reasignación de identificadores para garantizar que sean únicos.

Formalmente, sea R el esquema de una relación. Si se dice que un subconjunto C de R es una *superclave* de R, se restringe la consideración a las relaciones $r(R)$ en las que no hay dos tuplas diferentes que tengan los mismos valores en todos los atributos de C. Es decir, si t_1 y t_2 están en r y $t_1 \neq t_2$, entonces $t_1[C] \neq t_2[C]$.

El esquema de una relación, por ejemplo r_1, puede incluir entre sus atributos la clave primaria de otro esquema de relación, por ejemplo r_2. Este atributo se denomina **clave externa** de r_1, que hace referencia a r_2. La relación r_1 también se denomina **relación referenciante** de la dependencia de clave externa, y r_2 se denomina **relación referenciada** de la clave externa. Por ejemplo, el atributo *nombre_sucursal* de *Esquema_cuenta* es una clave externa de *Esquema_cuenta* que hace referencia a *Esquema_sucursal*, ya que *nombre_sucursal* es la clave primaria de *Esquema_sucursal*. En cualquier ejemplar de la base de datos, dada cualquier tupla, por ejemplo t_a, de la relación *cuenta*, debe haber alguna tupla, por ejemplo t_b, en la relación *sucursal* tal que el valor del atributo *nombre_sucursal* de t_a sea el mismo que el valor de la clave primaria de t_b *nombre_sucursal*.

Es costumbre relacionar los atributos de la clave primaria de un esquema de relación antes que el resto de los atributos; por ejemplo, el atributo *nombre_sucursal* de *Esquema_sucursal* se relaciona en primer lugar, ya que es la clave primaria.

El esquema de la base de datos, junto con las dependencias de clave primaria y externa, se puede mostrar gráficamente mediante **diagramas de esquema**. La Figura 2.8 muestra el diagrama de esquema del ejemplo bancario. Cada relación aparece como un cuadro con los atributos relacionados en su interior y el nombre de la relación sobre él. Si hay atributos de clave primaria, una línea horizontal cruza el cuadro con los atributos de clave primaria por encima de ella y sobre fondo gris. Las dependencias de clave externa aparecen como flechas desde los atributos de clave externa de la relación referenciante a la clave primaria de la relación referenciada.

Muchos sistemas de bases de datos proporcionan herramientas de diseño con una interfaz gráfica de usuario para la creación de los diagramas de esquema.

2.1.4 Lenguajes de consultas

Un **lenguaje de consultas** es un lenguaje en el que los usuarios solicitan información de la base de datos. Estos lenguajes suelen ser de un nivel superior que el de los lenguajes de programación habituales. Los lenguajes de consultas pueden clasificarse como procedimentales o no procedimentales. En los **lenguajes procedimentales** el usuario indica al sistema que lleve a cabo una serie de operaciones en la base

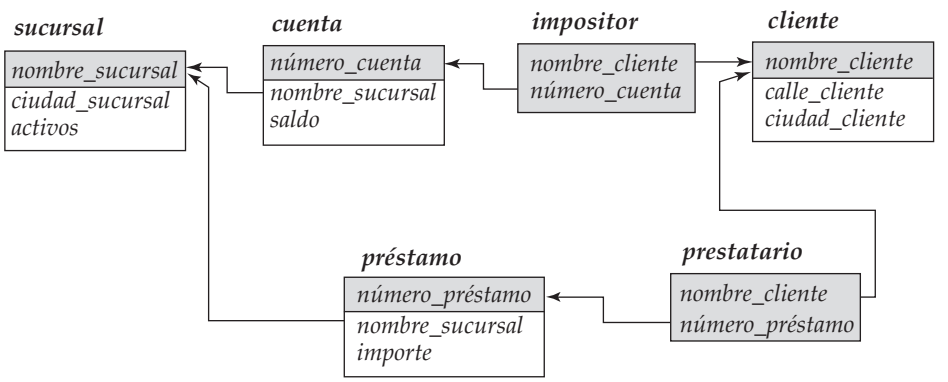


Figura 2.8 Diagrama del esquema de la entidad bancaria.

www.detodoprogramacion.com

de datos para calcular el resultado deseado. En los **lenguajes no procedimentales** el usuario describe la información deseada sin dar un procedimiento concreto para obtener esa información.

La mayor parte de los sistemas comerciales de bases de datos relacionales ofrecen un lenguaje de consultas que incluye elementos de los enfoques procedimental y no procedimental. Se estudiará el muy usado lenguaje de consultas SQL en los Capítulos 3 y 4. El Capítulo 5 trata los lenguajes de consultas QBE y Datalog; este último es un lenguaje de consultas parecido al lenguaje de programación Prolog.

Existen varios lenguajes de consultas "puros": el álgebra relacional es procedimental, mientras que el cálculo relacional de tuplas y el cálculo relacional de dominios no lo son. Estos lenguajes de consultas son rígidos y formales, y carecen del "azúcar sintáctico" de los lenguajes comerciales, pero ilustran las técnicas fundamentales para la extracción de datos de las bases de datos.

En este capítulo se examina con gran detalle el lenguaje del álgebra relacional (en el Capítulo 5 se tratan los lenguajes del cálculo relacional de tuplas y del cálculo relacional de dominios). El álgebra relacional consiste en un conjunto de operaciones que toman una o dos relaciones como entrada y generan otra relación nueva como resultado.

Las operaciones fundamentales del álgebra relacional son *selección*, *proyección*, *unión*, *diferencia de conjuntos*, *producto cartesiano* y *renombramiento*. Además de las operaciones fundamentales hay otras operaciones—por ejemplo, intersección de conjuntos, reunión natural, división y asignación. Estas operaciones se definirán en términos de las operaciones fundamentales.

Inicialmente sólo se estudiarán las consultas. Sin embargo, un lenguaje de manipulación de datos completo no sólo incluye un lenguaje de consultas, sino también un lenguaje para la modificación de las bases de datos. Este tipo de lenguajes incluye comandos para insertar y borrar tuplas, así como para modificar partes de las tuplas existentes. Las modificaciones de las bases de datos se examinarán después de completar la discusión sobre las consultas.

2.2 Operaciones fundamentales del álgebra relacional

Las operaciones selección, proyección y renombramiento se denominan operaciones *unarias* porque operan sobre una sola relación. Las otras tres operaciones operan sobre pares de relaciones y se denominan, por tanto, operaciones *binarias*.

2.2.1 Operación selección

La operación **selección** selecciona tuplas que satisfacen un predicado dado. Se usa la letra griega sigma minúscula (σ) para denotar la selección. El predicado aparece como subíndice de σ. La relación de argumentos se da entre paréntesis a continuación de σ. Por tanto, para seleccionar las tuplas de la relación *préstamo* en que la sucursal es "Navacerrada" se escribe

$$\sigma_{nombre_sucursal = \text{"Navacerrada"}} (préstamo)$$

Si la relación *préstamo* es como se muestra en la Figura 2.6, la relación que resulta de la consulta anterior es como aparece en la Figura 2.9.

número_préstamo	*nombre_sucursal*	*importe*
P-15	Navacerrada	1.500
P-16	Navacerrada	1.300

Figura 2.9 Resultado de $\sigma_{nombre_sucursal = \text{"Navacerrada"}} (préstamo)$.

www.detodoprogramacion.com

Se pueden buscar todas las tuplas en las que el importe prestado sea mayor que 1.200 € escribiendo la siguiente consulta:

$$\sigma_{importe>1200}\ (préstamo)$$

En general, se permiten las comparaciones que usan $=$, $\neq$, $<$, $\leq$, $>$ o $\geq$ en el predicado de selección. Además, se pueden combinar varios predicados en uno mayor usando las conectivas *y* ($\wedge$), *o* ($\vee$) y *no* ($\neg$). Por tanto, para encontrar las tuplas correspondientes a préstamos de más de 1.200 € concedidos por la sucursal de Navacerrada, se escribe

$$\sigma_{nombre_sucursal = \text{“Navacerrada”} \wedge importe>1200}\ (préstamo)$$

El predicado de selección puede incluir comparaciones entre dos atributos. Para ilustrarlo, considérese la relación *responsable_préstamo*, que consta de tres atributos: *nombre_cliente*, *nombre_responsable* y *número_préstamo*, que especifica que un empleado concreto es el responsable del préstamo concedido a un cliente. Para hallar todos los clientes que se llaman igual que su responsable de préstamos se puede escribir

$$\sigma_{nombre_cliente\ =\ nombre_responsable}(responsable_préstamo)$$

2.2.2 Operación proyección

Supóngase que se desea obtener una relación de todos los números e importes de los préstamos, pero sin los nombres de las sucursales. La operación **proyección** permite obtener esa relación. La operación proyección es una operación unaria que devuelve su relación de argumentos, excluyendo algunos argumentos. Dado que las relaciones son conjuntos, se eliminan todas las filas duplicadas. La proyección se denota por la letra griega mayúscula pi (Π). Se crea una lista de los atributos que se desea que aparezcan en el resultado como subíndice de Π. Su único argumento, una relación, se escribe a continuación entre paréntesis. La consulta para crear una lista de todos los números e importes de los préstamos puede escribirse como

$$\Pi_{número_préstamo, importe}(préstamo)$$

La Figura 2.10 muestra la relación que resulta de esta consulta.

2.2.3 Composición de operaciones relacionales

Es importante el hecho de que el resultado de una operación relacional sea también una relación. Considérese la consulta más compleja “Buscar los clientes que viven en Peguerinos”. Hay que escribir:

$$\Pi_{nombre_cliente}\ (\sigma_{ciudad_cliente\ =\ \text{“Peguerinos”}}\ (cliente))$$

Téngase en cuenta que, en vez de dar el nombre de una relación como argumento de la operación proyección, se da una expresión cuya evaluación es una relación.

En general, dado que el resultado de las operaciones del álgebra relacional es del mismo tipo (relación) que los datos de entrada, las operaciones del álgebra relacional pueden componerse para formar

número_préstamo	*importe*
P-11	900
P-14	1.500
P-15	1.500
P-16	1.300
P-17	1.000
P-23	2.000
P-93	500

Figura 2.10 Números e importes de los préstamos.

www.detodoprogramacion.com

una **expresión del álgebra relacional**. Componer operaciones del álgebra relacional para formar expresiones del álgebra relacional es igual que componer operaciones aritméticas (como $+$, $-$, $*$ y $\div$) para formar expresiones aritméticas. La definición formal de las expresiones de álgebra relacional se estudia en el Apartado 2.2.8.

2.2.4 Operación unión

Considérese una consulta para determinar el nombre de todos los clientes del banco que tienen una cuenta, un préstamo o ambas cosas. Obsérvese que la relación *cliente* no contiene esa información, dado que los clientes no necesitan tener ni cuenta ni préstamo en el banco. Para contestar a esta consulta hace falta la información de las relaciones *impositor* (Figura 2.5) y *prestatario* (Figura 2.7). Para determinar los nombres de todos los clientes con préstamos en el banco con las operaciones estudiadas se escribe:

$$\Pi_{nombre_cliente}\ (prestatario)$$

Igualmente, para determinar el nombre de todos los clientes con cuenta en el banco se escribe:

$$\Pi_{nombre_cliente}\ (impositor)$$

Para contestar a la consulta es necesaria la **unión** de estos dos conjuntos; es decir, hacen falta todos los nombres de clientes que aparecen en alguna de las dos relaciones o en ambas. Estos datos se pueden averiguar mediante la operación binaria unión, denotada, como en la teoría de conjuntos, por $\cup$. Por tanto, la expresión buscada es:

$$\Pi_{nombre_cliente}\ (prestatario)\ \cup\ \Pi_{nombre_cliente}\ (impositor)$$

La relación resultante de esta consulta aparece en la Figura 2.11. Téngase en cuenta que en el resultado hay diez tuplas, aunque haya siete prestatarios y seis impositores distintos. Esta discrepancia aparente se debe a que Gómez, López y Santos son a la vez prestatarios e impositores. Dado que las relaciones son conjuntos, se eliminan los valores duplicados.

Obsérvese que en este ejemplo se toma la unión de dos conjuntos, ambos consistentes en valores de *nombre_cliente*. En general, se debe asegurar que las uniones se realicen entre relaciones *compatibles*. Por ejemplo, no tendría sentido realizar la unión de las relaciones *préstamo* y *prestatario*. La primera es una relación con tres atributos, la segunda sólo tiene dos. Más aún, considérese la unión de un conjunto de nombres de clientes y de un conjunto de ciudades. Una unión así no tendría sentido en la mayor parte de los casos. Por tanto, para que la operación unión $r \cup s$ sea válida hay que exigir que se cumplan dos condiciones:

1. Las relaciones r y s deben ser de la misma aridad. Es decir, deben tener el mismo número de atributos.
2. Los dominios de los atributos i-ésimos de r y de s deben ser iguales para todo i.

nombre_cliente
Abril
Fernández
Gómez
González
López
Pérez
Rupérez
Santos
Sotoca
Valdivieso

Figura 2.11 Nombre de todos los clientes que tienen un préstamo o una cuenta.

www.detodoprogramacion.com

nombre_cliente
Abril
González
Rupérez

Figura 2.12 Clientes con cuenta abierta pero sin préstamo concedido.

Téngase en cuenta que *r* y *s* pueden ser, en general, relaciones de la base de datos o relaciones temporales resultado de expresiones del álgebra relacional.

2.2.5 Operación diferencia de conjuntos

La operación **diferencia de conjuntos**, denotada por $-$, permite hallar las tuplas que están en una relación pero no en la otra. La expresión $r - s$ da como resultado una relación que contiene las tuplas que están en *r* pero no en *s*.

Se pueden buscar todos los clientes del banco que tengan abierta una cuenta pero no tengan concedido ningún préstamo escribiendo

$$\Pi_{nombre_cliente}\ (impositor) - \Pi_{nombre_cliente}\ (prestatario)$$

La relación resultante de esta consulta aparece en la Figura 2.12.

Como en el caso de la operación unión, hay que asegurarse de que las diferencias de conjuntos se realicen entre relaciones *compatibles*. Por tanto, para que una operación diferencia de conjuntos $r - s$ sea válida se exige que las relaciones *r* y *s* sean de la misma aridad y que los dominios de los atributos *i*-ésimos de *r* y de *s* sean iguales.

2.2.6 Operación producto cartesiano

La operación **producto cartesiano**, denotada por un aspa ($\times$), permite combinar información de cualesquiera dos relaciones. El producto cartesiano de las relaciones r_1 y r_2 se escribe $r_1 \times r_2$.

Recuérdese que las relaciones se definen como subconjuntos del producto cartesiano de un conjunto de dominios. A partir de esa definición ya se debe tener una intuición sobre la definición de la operación producto cartesiano. Sin embargo, dado que el mismo nombre de atributo puede aparecer tanto en r_1 como en r_2, es necesario crear un convenio de denominación para distinguir unos atributos de otros. En este caso se realiza adjuntando al atributo el nombre de la relación de la que proviene originalmente. Por ejemplo, el esquema de relación de $r = prestatario \times préstamo$ es

(*prestatario.nombre_cliente*, *prestatario.número_préstamo*,
préstamo.número_préstamo, *préstamo.nombre_sucursal*, *préstamo.importe*)

Con este esquema se puede distinguir entre *prestatario.número_préstamo* y *préstamo.número_préstamo*. Para los atributos que sólo aparecen en uno de los dos esquemas se suele omitir el prefijo con el nombre de la relación. Esta simplificación no genera ambigüedad alguna. Por tanto, se puede escribir el esquema de la relación *r* como

(*nombre_cliente*, *prestatario.número_préstamo*,
préstamo.número_préstamo, *nombre_sucursal*, *importe*)

Este convenio de denominaciones *exige* que las relaciones que sean argumentos de la operación producto cartesiano tengan nombres diferentes. Esta exigencia causa problemas en algunos casos, como cuando se desea calcular el producto cartesiano de una relación consigo misma. Se produce un problema parecido si se usa el resultado de una expresión del álgebra relacional en un producto cartesiano, dado que hará falta un nombre de relación para poder hacer referencia a sus atributos. En el Apartado 2.2.7 se verá la manera de evitar estos problemas mediante la operación renombramiento.

Ahora que se conoce el esquema de relación de $r = prestatario \times préstamo$ es necesario hallar las tuplas que aparecerán en *r*. Como es posible imaginar, se crea una tupla de *r* a partir de cada par de tuplas

www.detodoprogramacion.com

nombre_cliente	*prestatario. número_préstamo*	*préstamo. número_préstamo*	*nombre_sucursal*	*importe*
Fernández	P-16	P-11	Collado Mediano	900
Fernández	P-16	P-14	Centro	1.500
Fernández	P-16	P-15	Navacerrada	1.500
Fernández	P-16	P-16	Navacerrada	1.300
Fernández	P-16	P-17	Centro	1.000
Fernández	P-16	P-23	Moralzarzal	2.000
Fernández	P-16	P-93	Becerril	500
Gómez	P-11	P-11	Collado Mediano	900
Gómez	P-11	P-14	Centro	1.500
Gómez	P-11	P-15	Navacerrada	1.500
Gómez	P-11	P-16	Navacerrada	1.300
Gómez	P-11	P-17	Centro	1.000
Gómez	P-11	P-23	Moralzarzal	2.000
Gómez	P-11	P-93	Becerril	500
Gómez	P-23	P-11	Collado Mediano	900
Gómez	P-23	P-14	Centro	1.500
Gómez	P-23	P-15	Navacerrada	1.500
Gómez	P-23	P-16	Navacerrada	1.300
Gómez	P-23	P-17	Centro	1.000
Gómez	P-23	P-23	Moralzarzal	2.000
Gómez	P-23	P-93	Becerril	500
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
Sotoca	P-14	P-11	Collado Mediano	900
Sotoca	P-14	P-14	Centro	1.500
Sotoca	P-14	P-15	Navacerrada	1.500
Sotoca	P-14	P-16	Navacerrada	1.300
Sotoca	P-14	P-17	Centro	1.000
Sotoca	P-14	P-23	Moralzarzal	2.000
Sotoca	P-14	P-93	Becerril	500
Valdivieso	P-17	P-11	Collado Mediano	900
Valdivieso	P-17	P-14	Centro	1.500
Valdivieso	P-17	P-15	Navacerrada	1.500
Valdivieso	P-17	P-16	Navacerrada	1.300
Valdivieso	P-17	P-17	Centro	1.000
Valdivieso	P-17	P-23	Moralzarzal	2.000
Valdivieso	P-17	P-93	Becerril	500

Figura 2.13 Resultado de $prestatario \times préstamo$.

posible: una de la relación *prestatario* y otra de *préstamo*. Por tanto, *r* es una relación de gran tamaño, como se puede ver en la Figura 2.13, donde sólo se ha incluido una parte de las tuplas que constituyen *r*.

Supóngase que se tienen n_1 tuplas de *prestatario* y n_2 tuplas de *préstamo*. Por tanto, hay $n_1 * n_2$ maneras de escoger un par de tuplas—una tupla de cada relación; por lo que hay $n_1 * n_2$ tuplas en *r*. En concreto, obsérvese que para algunas tuplas *t* de *r* puede ocurrir que *t*[*prestatario.número_préstamo*] $\neq$ *t*[*préstamo.número_préstamo*].

En general, si se tienen las relaciones $r_1(R_1)$ y $r_2(R_2)$, $r_1 \times r_2$ es una relación cuyo esquema es la concatenación de R_1 y de R_2. La relación *R* contiene todas las tuplas *t* para las que hay unas tuplas t_1 en r_1 y t_2 en r_2 para las que $t[R_1] = t_1[R_1]$ y $t[R_2] = t_2[R_2]$.

www.detodoprogramacion.com

nombre_cliente	*prestatario. número_préstamo*	*préstamo. número_préstamo*	*nombre_sucursal*	*importe*
Fernández	P-16	P-15	Navacerrada	1.500
Fernández	P-16	P-16	Navacerrada	1.300
Gómez	P-11	P-15	Navacerrada	1.500
Gómez	P-11	P-16	Navacerrada	1.300
Gómez	P-23	P-15	Navacerrada	1.500
Gómez	P-23	P-16	Navacerrada	1.300
López	P-15	P-15	Navacerrada	1.500
López	P-15	P-16	Navacerrada	1.300
Pérez	P-93	P-15	Navacerrada	1.500
Pérez	P-93	P-16	Navacerrada	1.300
Santos	P-17	P-15	Navacerrada	1.500
Santos	P-17	P-16	Navacerrada	1.300
Sotoca	P-14	P-15	Navacerrada	1.500
Sotoca	P-14	P-16	Navacerrada	1.300
Valdivieso	P-17	P-15	Navacerrada	1.500
Valdivieso	P-17	P-16	Navacerrada	1.300

Figura 2.14 Resultado de $\sigma_{nombre_sucursal = \text{“Navacerrada”}} (prestatario \times préstamo)$.

Supóngase que se desea determinar el nombre de todos los clientes que tienen concedido un préstamo en la sucursal de Navacerrada. Se necesita para ello información de las relaciones *préstamo* y *prestatario*. Si se escribe

$$\sigma_{nombre_sucursal = \text{“Navacerrada”}} (prestatario \times préstamo)$$

entonces el resultado es la relación mostrada en la Figura 2.14. Se tiene una relación que sólo atañe a la sucursal de Navacerrada. Sin embargo, la columna *nombre_cliente* puede contener clientes que no tengan concedido ningún préstamo en la sucursal de Navacerrada. (Si no se ve el motivo por el que esto es cierto, recuérdese que el producto cartesiano toma todos los emparejamientos posibles de cada tupla de *prestatario* con cada tupla de *préstamo*).

Dado que la operación producto cartesiano asocia *todas* las tuplas de *préstamo* con todas las tuplas de *prestatario*, se sabe que, si un cliente tiene concedido un préstamo en la sucursal de Navacerrada, hay alguna tupla de *prestatario* $\times$ *préstamo* que contiene su nombre y que *prestatario.número_préstamo* = *préstamo.número_préstamo*. Por tanto, si se escribe

$$\sigma_{prestatario.número_préstamo = préstamo.número_préstamo} (\sigma_{nombre_sucursal = \text{“Navacerrada”}} (prestatario \times préstamo))$$

sólo se obtienen las tuplas de $prestatario \times préstamo$ que corresponden a los clientes que tienen concedido un préstamo en la sucursal de Navacerrada.

Finalmente, dado que sólo se desea obtener *nombre_cliente*, se realiza una proyección:

$$\Pi_{nombre_cliente} (\sigma_{prestatario.número_préstamo = préstamo.número_préstamo} (\sigma_{nombre_sucursal = \text{“Navacerrada”}} (prestatario \times préstamo)))$$

El resultado de esta expresión, mostrada en la Figura 2.15, es la respuesta correcta a la consulta formulada.

2.2.7 Operación renombramiento

A diferencia de las relaciones de la base de datos, los resultados de las expresiones de álgebra relacional no tienen un nombre que se pueda usar para referirse a ellas. Resulta útil poder ponerles nombre; la operación **renombramiento**, denotada por la letra griega ro minúscula (ρ), permite hacerlo. Dada una

www.detodoprogramacion.com

nombre_cliente
Fernández
López

Figura 2.15 Resultado de $\Pi_{nombre_cliente}$ $(\sigma_{prestatario.número_préstamo = préstamo.número_préstamo}$ $(\sigma_{nombre_sucursal = \text{“Navacerrada”}} (prestatario \times préstamo)))$.

expresión E del álgebra relacional, la expresión

$$\rho_x (E)$$

devuelve el resultado de la expresión *E* con el nombre *x*.

Las relaciones r por sí mismas se consideran expresiones (triviales) del álgebra relacional. Por tanto, también se puede aplicar la operación renombramiento a una relación r para obtener la misma relación con un nombre nuevo.

Otra forma de la operación renombramiento es la siguiente. Supóngase que una expresión del álgebra relacional E tiene aridad n. Por tanto, la expresión

$$\rho_{x(A_1,A_2,\ldots,A_n)} (E)$$

devuelve el resultado de la expresión *E* con el nombre *x* y con los atributos con el nombre cambiado a $A_1, A_2, \ldots, A_n$.

Para ilustrar el renombramiento de relaciones, considérese la consulta “Buscar el saldo máximo de las cuentas del banco”. La estrategia empleada para obtener el resultado es (1) calcular en primer lugar una relación temporal consistente en los saldos que *no* son el máximo y (2) realizar la diferencia entre la relación $\Pi_{saldo} (cuenta)$ y la relación temporal recién calculada para obtener el resultado.

Paso 1: para calcular la relación intermedia hay que comparar los valores de los saldos de todas las cuentas. Esta comparación se hará calculando el producto cartesiano $cuenta \times cuenta$ y formando una selección para comparar el valor de cualesquiera dos saldos que aparezcan en una tupla. En primer lugar hay que crear un mecanismo para distinguir entre los dos atributos *saldo*. Se usará la operación renombramiento para cambiar el nombre de una referencia a la relación *cuenta*; de este modo se puede hacer referencia dos veces a la relación sin ambigüedad alguna.

La relación temporal que se compone de los saldos que no son el máximo puede escribirse ahora como:

$$\Pi_{cuenta.saldo} (\sigma_{cuenta.saldo < d.saldo} (cuenta \times \rho_d (cuenta)))$$

Esta expresión proporciona los saldos de la relación *cuenta* para los que aparece un saldo mayor en alguna parte de la relación *cuenta* (cuyo nombre se ha cambiado a *d*). El resultado contiene todos los saldos *salvo* el máximo. La Figura 2.16 muestra esta relación.

Paso 2: la consulta para determinar el saldo máximo de las cuentas del banco puede escribirse de la manera siguiente:

$$\Pi_{saldo} (cuenta) - \Pi_{cuenta.saldo} (\sigma_{cuenta.saldo < d.saldo} (cuenta \times \rho_d (cuenta)))$$

saldo
350
400
500
700
750

Figura 2.16 Resultado de la subexpresión $\Pi_{cuenta.saldo} (\sigma_{cuenta.saldo < d.saldo} (cuenta \times \rho_d (cuenta)))$.

www.detodoprogramacion.com

saldo
900

Figura 2.17 Saldo máximo de las cuentas del banco.

La Figura 2.17 muestra el resultado de esta consulta.

Considérese la consulta "Determinar el nombre de todos los clientes que viven en la misma ciudad y en la misma calle que Gómez" como un nuevo ejemplo de la operación renombramiento. Se puede obtener la calle y la ciudad en las que vive Gómez escribiendo

$$\Pi_{calle_cliente,\ ciudad_cliente}\ (\sigma_{nombre_cliente\ =\ \text{"Gómez"}}\ (cliente))$$

Sin embargo, para hallar a otros clientes que vivan en esa ciudad y en esa calle hay que hacer referencia por segunda vez a la relación *cliente*. En la consulta siguiente se usa la operación renombramiento sobre la expresión anterior para darle al resultado el nombre *dirección_gómez* y para cambiar el nombre de los atributos *calle_cliente* y *ciudad_cliente* a *calle* y *ciudad*, respectivamente:

$$\begin{array}{l}\Pi_{cliente.nombre_cliente} \\ \quad (\sigma_{cliente.calle_cliente=dirección_gómez.calle\ \wedge\ cliente.ciudad_cliente=dirección_gómez.ciudad} \\ \qquad (cliente\ \times\ \rho_{dirección_gómez(calle,ciudad)} \\ \qquad\quad (\Pi_{calle_cliente,\ ciudad_cliente}(\sigma_{nombre_cliente\ =\ \text{"Gómez"}}(cliente)))))\end{array}$$

El resultado de esta consulta, cuando se aplica a la relación *cliente* de la Figura 2.4, se muestra en la Figura 2.18.

La operación renombramiento no es estrictamente necesaria, dado que es posible usar una notación posicional para los atributos. Se pueden nombrar los atributos de una relación de manera implícita donde $\$1, \$2, \ldots$ hagan referencia respectivamente al primer atributo, al segundo, etc. La notación posicional también se aplica a los resultados de las operaciones del álgebra relacional. La siguiente expresión del álgebra relacional ilustra el empleo de esta notación con el operador unario σ:

$$\sigma_{\$2=\$3}(R \times R)$$

Si una operación binaria necesita distinguir entre las dos relaciones que son sus operandos, se puede usar una notación posicional parecida para los nombres de las relaciones. Por ejemplo, $\$R1$ puede hacer referencia al primer operando y $\$R2$, al segundo. Sin embargo, la notación posicional no resulta conveniente para las personas, dado que la posición del atributo es un número en vez de un nombre de atributo fácil de recordar. Por tanto, en este libro no se usa la notación posicional.

2.2.8 Definición formal del álgebra relacional

Las operaciones del Apartado 2.2 permiten dar una definición completa de las expresiones del álgebra relacional. Las expresiones fundamentales del álgebra relacional se componen de alguno de las siguientes elementos:

- Una relación de la base de datos
- Una relación constante

Las relaciones constantes se escriben poniendo una relación de sus tuplas entre llaves ({ }), por ejemplo {(C-101,Centro,500) (C-215, Becerril, 700)}.

nombre_cliente
Gómez
Pérez

Figura 2.18 Clientes que viven en la misma ciudad y en la misma calle que Gómez.

www.detodoprogramacion.com

nombre_cliente
Gómez
López
Santos

Figura 2.19 Clientes con una cuenta abierta y un préstamo en el banco.

Las expresiones generales del álgebra relacional se construyen a partir de subexpresiones más pequeñas. Sean E_1 y E_2 expresiones de álgebra relacional. Todas las expresiones siguientes son también expresiones del álgebra relacional:

- $E_1 \cup E_2$
- $E_1 - E_2$
- $E_1 \times E_2$
- $\sigma_P(E_1)$, donde P es un predicado de atributos de E_1
- $\Pi_S(E_1)$, donde S es una lista que se compone de algunos de los atributos de E_1
- $\rho_x(E_1)$, donde x es el nuevo nombre del resultado de E_1

2.3 Otras operaciones del álgebra relacional

Las operaciones fundamentales del álgebra relacional son suficientes para expresar cualquier consulta del álgebra relacional[1]. Sin embargo, limitándose exclusivamente a las operaciones fundamentales, algunas consultas habituales resultan complicadas de expresar. Por tanto, se definen otras operaciones que no añaden potencia al álgebra, pero que simplifican las consultas habituales. Para cada operación nueva se facilita una expresión equivalente usando sólo las operaciones fundamentales.

2.3.1 Operación intersección de conjuntos

La primera operación adicional del álgebra relacional que se va a definir es la **intersección de conjuntos**($\cap$). Supóngase que se desea conocer todos los clientes con un préstamo concedido y una cuenta abierta. Empleando la intersección de conjuntos se puede escribir

$$\Pi_{nombre_cliente}(prestatario) \cap \Pi_{nombre_cliente}(impositor)$$

La relación resultante de esta consulta aparece en la Figura 2.19.

Obsérvese que se puede volver a escribir cualquier expresión del álgebra relacional que utilice la intersección de conjuntos sustituyendo la operación intersección por un par de operaciones de diferencia de conjuntos, de la manera siguiente:

$$r \cap s = r - (r - s)$$

Por tanto, la intersección de conjuntos no es una operación fundamental y no añade potencia al álgebra relacional. Sencillamente, es más conveniente escribir $r \cap s$ que $r - (r - s)$.

2.3.2 Operación reunión natural

Suele resultar deseable simplificar ciertas consultas que exijan un producto cartesiano. Generalmente, las consultas que implican un producto cartesiano incluyen un operador selección sobre el resultado del producto cartesiano. Considérese la consulta "Hallar los nombres de todos los clientes que tienen concedido un préstamo en el banco y averiguar su número e importe". En primer lugar se calcula el

1. En el Apartado 2.4 se introducen operaciones que extienden la potencia del álgebra relacional al tratamiento de los valores nulos y de los valores de agregación.

www.detodoprogramacion.com

nombre_cliente	*número_préstamo*	*importe*
Fernández	P-16	1.300
Gómez	P-23	2.000
Gómez	P-11	900
López	P-15	1.500
Pérez	P-93	500
Santos	P-17	1.000
Sotoca	P-14	1.500
Valdivieso	P-17	1.000

Figura 2.20 Resultado de $\Pi_{nombre_cliente, número_préstamo, importe}\ (prestatario \bowtie préstamo)$.

producto cartesiano de las relaciones *prestatario* y *préstamo*. Después se seleccionan las tuplas que sólo atañen al mismo *número_préstamo*, seguidas por la proyección de *nombre_cliente*, *número_préstamo* e *importe* resultantes:

$$\Pi_{nombre_cliente, préstamo.número_préstamo, importe} \left(\sigma_{prestatario.número_préstamo = préstamo.número_préstamo}\ (prestatario\ \times\ préstamo)\right)$$

La *reunión natural* es una operación binaria que permite combinar ciertas selecciones y un producto cartesiano en una sola operación. Se denota por el símbolo de **reunión** $\bowtie$. La operación reunión natural forma un producto cartesiano de sus dos argumentos, realiza una selección forzando la igualdad de los atributos que aparecen en ambos esquemas de relación y, finalmente, elimina los atributos duplicados.

Aunque la definición de la reunión natural es compleja, la operación es sencilla de aplicar. A modo de ilustración, considérese nuevamente el ejemplo "Determinar el nombre de todos los clientes que tienen concedido un préstamo en el banco y averiguar su importe". Esta consulta puede expresarse usando la reunión natural de la manera siguiente:

$$\Pi_{nombre_cliente, número_préstamo, importe}\ (prestatario \bowtie préstamo)$$

Dado que los esquemas de *prestatario* y de *préstamo* (es decir, *Esquema_prestatario* y *Esquema_préstamo*) tienen en común el atributo *número_préstamo*, la operación reunión natural sólo considera los pares de tuplas que tienen el mismo valor de *número_préstamo*. Esta operación combina cada uno de estos pares en una sola tupla en la unión de los dos esquemas (es decir, *nombre_cliente, nombre_sucursal, número_préstamo, importe*). Después de realizar la proyección, se obtiene la relación mostrada en la Figura 2.20.

Considérense dos esquemas de relación *R* y *S*—que son, por supuesto, listas de nombres de atributos. Si se consideran los esquemas como *conjuntos*, en vez de como listas, se pueden denotar los nombres de los atributos que aparecen tanto en *R* como en *S* mediante $R \cap S$, y los nombres de los atributos que aparecen en *R*, en *S* o en ambos mediante $R \cup S$. De manera parecida, los nombres de los atributos que aparecen en *R* pero no en *S* se denotan por $R - S$, mientras que $S - R$ denota los nombres de los atributos que aparecen en S pero no en R. Obsérvese que las operaciones unión, intersección y diferencia aquí operan sobre conjuntos de atributos, y no sobre relaciones.

Ahora es posible dar una definición formal de la reunión natural. Considérense dos relaciones $r(R)$ y $s(S)$. La **reunión natural** de *r* y de *s*, denotada mediante $r \bowtie s$ es una relación del esquema $R \cup S$ definida formalmente de la manera siguiente:

$$r \bowtie s\ =\ \Pi_{R \cup S}\ (\sigma_{r.A_1 = s.A_1 \wedge r.A_2 = s.A_2 \wedge \ldots \wedge r.A_n = s.A_n}\ (r\ \times\ s))$$

donde $R \cap S = \{A_1,\ A_2, \ldots, A_n\}$.

Como la reunión natural es fundamental para gran parte de la teoría y de la práctica de las bases de datos relacionales, se ofrecen varios ejemplos de su uso.

- Hallar el nombre de todas las sucursales con clientes que tienen una cuenta abierta en el banco y viven en Peguerinos.

www.detodoprogramacion.com

nombre_sucursal
Galapagar
Navacerrada

Figura 2.21 Resultado de $\Pi_{nombre_sucursal}(\sigma_{ciudad_cliente = \text{“Peguerinos”}} (cliente \bowtie cuenta \bowtie impositor))$.

$$\Pi_{nombre_sucursal} (\sigma_{ciudad_cliente = \text{“Peguerinos”}} (cliente \bowtie cuenta \bowtie impositor))$$

La relación resultante de esta consulta aparece en la Figura 2.21.

Obsérvese que se ha escrito *cliente* $\bowtie$ *cuenta* $\bowtie$ *impositor* sin añadir paréntesis para especificar el orden en que se deben ejecutar las operaciones reunión natural sobre las tres relaciones. En el caso anterior hay dos posibilidades:

$$(cliente \bowtie cuenta) \bowtie impositor$$
$$cliente \bowtie (cuenta \bowtie impositor)$$

No se ha especificado la expresión deseada, ya que las dos son equivalentes. Es decir, la reunión natural es **asociativa**.

- Hallar todos los clientes que tienen una cuenta abierta *y* un préstamo concedido en el banco.

$$\Pi_{nombre_cliente} (prestatario \bowtie impositor)$$

Obsérvese que en el Apartado 2.3.1 se escribió una expresión para esta consulta usando la intersección de conjuntos. A continuación se repite esa expresión.

$$\Pi_{nombre_cliente} (prestatario) \cap \Pi_{nombre_cliente} (impositor)$$

La relación resultante de esta consulta se mostró anteriormente en la Figura 2.19. Este ejemplo ilustra una característica del álgebra relacional: se pueden escribir varias expresiones del álgebra relacional equivalentes que sean bastante diferentes entre sí.

- Sean $r(R)$ y $s(S)$ relaciones sin atributos en común; es decir, $R \cap S = \emptyset$. ($\emptyset$ denota el conjunto vacío). Por tanto, $r \bowtie s = r \times s$.

La operación *reunión zeta* es una extensión de la operación reunión natural que permite combinar una selección y un producto cartesiano en una sola operación. Considérense las relaciones $r(R)$ y $s(S)$, y sea θ un predicado de los atributos del esquema $R \cup S$. La operación **reunión zeta** $r \bowtie_\theta s$ se define de la manera siguiente:

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

2.3.3 Operación división

La operación **división**, denotada por $\div$, resulta adecuada para las consultas que incluyen la expresión “para todos”. Supóngase que se desea hallar a todos los clientes que tengan abierta una cuenta en *todas* las sucursales ubicadas en Arganzuela. Se pueden obtener todas las sucursales de Arganzuela mediante la expresión

$$r_1 = \Pi_{nombre_sucursal} (\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal))$$

La relación resultante de esta expresión aparece en la Figura 2.22.

nombre_sucursal
Centro
Galapagar

Figura 2.22 Resultado de $\Pi_{nombre_sucursal}(\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal))$.

www.detodoprogramacion.com

Se pueden encontrar todos los pares (*nombre_cliente, nombre_sucursal*) para los que el cliente tiene una cuenta en una sucursal escribiendo

$$r_2 = \Pi_{nombre_cliente, nombre_sucursal} (impositor \bowtie cuenta)$$

La Figura 2.23 muestra la relación resultante de esta expresión.

Ahora hay que hallar los clientes que aparecen en r_2 con los nombres de *todas* las sucursales de r_1. La operación que proporciona exactamente esos clientes es la operación división. La consulta se formula escribiendo

$$\Pi_{nombre_cliente, nombre_sucursal} (impositor \bowtie cuenta) \div \Pi_{nombre_sucursal} (\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal))$$

El resultado de esta expresión es una relación que tiene el esquema (*nombre_cliente*) y que contiene la tupla (González).

Formalmente, sean $r(R)$ y $s(S)$ relaciones y $S \subseteq R$; es decir, todos los atributos del esquema S están también en el esquema R. La relación $r \div s$ es una relación del esquema $R - S$ (es decir, del esquema que contiene todos los atributos del esquema R que no están en el esquema S). Una tupla *t* está en $r \div s$ si y sólo si se cumplen estas dos condiciones:

1. t está en $\Pi_{R-S}(r)$
2. Para cada tupla t_s de *s* hay una tupla t_r de *r* que cumple las dos condiciones siguientes:
   a. $t_r[S] = t_s[S]$
   b. $t_r[R - S] = t$

Puede resultar sorprendente descubrir que, dados una operación división y los esquemas de las relaciones, se pueda definir la operación división en términos de las operaciones fundamentales. Sean $r(R)$ y $s(S)$ dadas, con $S \subseteq R$:

$$r \div s = \Pi_{R-S} (r) - \Pi_{R-S} ((\Pi_{R-S} (r) \times s) - \Pi_{R-S,S}(r))$$

Para comprobar que esta expresión es cierta, obsérvese que $\Pi_{R-S} (r)$ proporciona todas las tuplas *t* que cumplen la primera condición de la definición de la división. La expresión del lado derecho del operador diferencia de conjuntos,

$$\Pi_{R-S} ((\Pi_{R-S} (r) \times s) - \Pi_{R-S,S}(r))$$

elimina las tuplas que no cumplen la segunda condición de la definición de la división. Esto se logra de la manera siguiente. Considérese $\Pi_{R-S} (r) \times s$. Esta relación está en el esquema *R* y empareja cada tupla de $\Pi_{R-S} (r)$ con cada tupla de *s*. La expresión $\Pi_{R-S,S}(r)$ sólo reordena los atributos de r.

Por tanto, $(\Pi_{R-S} (r) \times s) - \Pi_{R-S,S}(r)$ genera los pares de tuplas de $\Pi_{R-S} (r)$ y de *s* que no aparecen en *r*. Si una tupla t_j está en

$$\Pi_{R-S} ((\Pi_{R-S} (r) \times s) - \Pi_{R-S,S}(r))$$

nombre_cliente	*nombre_sucursal*
Abril	Collado Mediano
Gómez	Becerril
González	Centro
González	Galapagar
López	Navacerrada
Rupérez	Moralzarzal
Santos	Galapagar

Figura 2.23 Resultado de $\Pi_{nombre_cliente, nombre_sucursal} (impositor \bowtie cuenta)$.

www.detodoprogramacion.com

nombre_cliente	*límite*	*saldo_crédito*
Gómez	2.000	400
López	1.500	1.500
Pérez	2.000	1.750
Santos	6.000	700

Figura 2.24 La relación *información_crédito*.

existe alguna tupla t_s de s que no se combina con la tupla t_j para formar una tupla de r. Por tanto, t_j guarda un valor de los atributos $R - S$ que no aparece en $r \div s$. Estos valores son los que se eliminan de $\Pi_{R-S}(r)$.

2.3.4 Operación asignación

En ocasiones resulta conveniente escribir una expresión del álgebra relacional mediante la asignación de partes de esa expresión a variables de relación temporal. La operación **asignación**, denotada por $\leftarrow$, actúa de manera parecida a la asignación de los lenguajes de programación. Para ilustrar esta operación, considérese la definición de la división dada en el Apartado 2.3.3. Se puede escribir $r \div s$ como

$$\begin{aligned} temp1 &\leftarrow \Pi_{R-S}(r) \\ temp2 &\leftarrow \Pi_{R-S}((temp1 \times s) - \Pi_{R-S,S}(r)) \\ resultado &= temp1 - temp2 \end{aligned}$$

La evaluación de una asignación no hace que se muestre ninguna relación al usuario. Por el contrario, el resultado de la expresión situada a la derecha de $\leftarrow$ se asigna a la variable relación situada a la izquierda de $\leftarrow$. Esta variable relación puede usarse en expresiones posteriores.

Con la operación asignación se pueden escribir las consultas como programas secuenciales que constan de en una serie de asignaciones seguida de una expresión cuyo valor se muestra como resultado de la consulta. En las consultas del álgebra relacional la asignación siempre debe hacerse a una variable de relación temporal. Las asignaciones a relaciones permanentes constituyen una modificación de la base de datos. Este asunto se estudiará en el Apartado 2.6. Obsérvese que la operación asignación no añade potencia alguna al álgebra. Resulta, sin embargo, una manera conveniente de expresar las consultas complejas.

2.4 Operaciones del álgebra relacional extendida

Las operaciones básicas del álgebra relacional se han extendido de varias formas. Una sencilla es permitir operaciones aritméticas como parte de las proyecciones. Una importante es permitir *operaciones de agregación*, como el cálculo de la suma de los elementos de un conjunto, o su media. Otra extensión importante es la operación *reunión externa*, que permite a las expresiones del álgebra relacional trabajar con los valores nulos, que modelan la información ausente.

2.4.1 Proyección generalizada

La operación **proyección generalizada** extiende la proyección permitiendo que se utilicen funciones aritméticas en la lista de proyección. La operación proyección generalizada es de la forma

$$\Pi_{F_1,F_2,\ldots,F_n}(E)$$

donde E es cualquier expresión del álgebra relacional y $F_1, F_2, \ldots, F_n$ son expresiones aritméticas que incluyen constantes y atributos del esquema de E. Como caso especial, la expresión aritmética puede ser simplemente un atributo o una constante.

Por ejemplo, supóngase que se dispone de una relación *información_crédito*, como se muestra en la Figura 2.24, que proporciona el límite de crédito y el importe consumido actualmente (*saldo_crédito*). Si se desea determinar el importe disponible por cada persona, se puede escribir la expresión siguiente:

www.detodoprogramacion.com

nombre_cliente	*crédito_disponible*
Gómez	1.600
López	0
Pérez	250
Santos	5.300

Figura 2.25 Resultado de $\Pi_{nombre_cliente, (límite\ -\ saldo_crédito)\ \textbf{as}\ crédito_disponible}\ (información_crédito)$.

$$\Pi_{nombre_cliente, límite\ -\ saldo_crédito}\ (\textit{información_crédito})$$

El atributo resultante de la expresión $límite\ -\ saldo_crédito$ no tiene nombre asignado. Se puede aplicar la operación renombramiento al resultado de la proyección generalizada para darle nombre. Como conveniencia notacional, el renombramiento de atributos se puede combinar con la proyección generalizada como se ilustra a continuación:

$$\Pi_{nombre_cliente, (límite\ -\ saldo_crédito)\ \textbf{as}\ crédito_disponible}\ (\textit{información_crédito})$$

Al segundo atributo de esta proyección generalizada se le ha asignado el nombre *crédito_disponible*. La Figura 2.25 muestra el resultado de aplicar esta expresión a la relación de la Figura 2.24.

2.4.2 Funciones de agregación

Las **funciones de agregación** toman un conjunto de valores y devuelven como resultado un único valor. Por ejemplo, la función de agregación **sum** toma un conjunto de valores y devuelve su suma. Por tanto, la función **sum** aplicada al conjunto

$$\{1, 1, 3, 4, 4, 11\}$$

devuelve el valor 24. La función de agregación **avg** devuelve la media de los valores. Cuando se aplica al conjunto anterior devuelve el valor 4. La función de agregación **count** devuelve el número de elementos del conjunto, y devolvería 6 en el caso anterior. Otras funciones de agregación habituales son **min** y **max**, que devuelven los valores mínimo y máximo del conjunto; en el ejemplo anterior devuelven 1 y 11, respectivamente.

Los conjuntos sobre los que operan las funciones de agregación pueden contener valores repetidos; el orden en el que aparezcan los valores no tiene importancia. Estos conjuntos se denominan **multiconjuntos**. Los conjuntos son un caso especial de los multiconjuntos, en los que sólo hay una copia de cada elemento.

Para ilustrar el concepto de agregación se usará la relación *trabajo_por_horas* descrita en la Figura 2.26, que muestra los empleados a tiempo parcial. Supóngase que se desea determinar la suma total de los sueldos de los empleados del banco a tiempo parcial. La expresión del álgebra relacional para esta consulta es la siguiente:

$$\mathcal{G}_{\textbf{sum}(sueldo)}(trabajo_por_horas)$$

nombre_empleado	*nombre_sucursal*	*sueldo*
Cana	Leganés	1.500
Cascallar	Navacerrada	5.300
Catalán	Leganés	1.600
Díaz	Centro	1.300
Fernández	Navacerrada	1.500
González	Centro	1.500
Jiménez	Centro	2.500
Ribera	Navacerrada	1.300

Figura 2.26 La relación *trabajo_por_horas*.

www.detodoprogramacion.com

nombre_empleado	*nombre_sucursal*	*sueldo*
Cana	Leganés	1.500
Catalán	Leganés	1.600
Díaz	Centro	1.300
González	Centro	1.500
Jiménez	Centro	2.500
Cascallar	Navacerrada	5.300
Fernández	Navacerrada	1.500
Ribera	Navacerrada	1.300

Figura 2.27 La relación *trabajo_por_horas* después de la agrupación.

El símbolo $\mathcal{G}$ es la letra G en el tipo de letra caligráfico; se lee "G caligráfica". La operación del álgebra relacional $\mathcal{G}$ significa que se debe aplicar la agregación; y su subíndice, la operación de agregación que se aplica. El resultado de la expresión anterior es una relación con un único atributo, que contiene una sola fila con un valor numérico correspondiente a la suma de los sueldos de todos los trabajadores que trabajan en el banco a tiempo parcial.

Supóngase ahora que se desea determinar la suma total de sueldos de los empleados a tiempo parcial de cada sucursal bancaria, en lugar de determinar la suma total. Para ello hay que dividir la relación *trabajo_por_horas* en **grupos** basados en la sucursal y aplicar la función de agregación a cada grupo. La expresión siguiente obtiene el resultado deseado usando el operador de agregación $\mathcal{G}$:

$$_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(sueldo)}(trabajo_por_horas)$$

Existen casos en los que es necesario borrar los valores repetidos antes de calcular una función de agregación. Si se desea eliminar los valores repetidos hay que usar los mismos nombres de funciones que antes, con la cadena de texto "**distinct**" precedida de un guión añadida al final del nombre de la función (por ejemplo, **count-distinct**). Un ejemplo se da en la consulta "Determinar el número de sucursales que aparecen en la relación *trabajo_por_horas*". En este caso, el nombre de cada sucursal sólo se cuenta una vez, independientemente del número de empleados que trabajen en la misma. Esta consulta se escribe de la manera siguiente:

$$\mathcal{G}_{\mathbf{count-distinct}(nombre_sucursal)}(trabajo_por_horas)$$

Al aplicar esta consulta a la relación de la Figura 2.26 el resultado está compuesto por una única fila con valor 3. En esta consulta el atributo *nombre_sucursal* en el subíndice a la izquierda de $\mathcal{G}$ indica que la relación de entrada *trabajo_por_horas* debe dividirse en grupos de acuerdo con el valor de *nombre_sucursal*. La Figura 2.27 muestra los grupos resultantes. La expresión $\mathbf{sum}(sueldo)$ en el subíndice a la derecha de $\mathcal{G}$ indica que, para cada grupo de tuplas (es decir, para cada sucursal) hay que aplicar la función de agregación **sum** al conjunto de valores del atributo *sueldo*. La relación resultante consiste en tuplas con el nombre de la sucursal y la suma de los sueldos de la sucursal, como se muestra en la Figura 2.28.

La forma general de la **operación de agregación** $\mathcal{G}$ es la siguiente:

$$_{G_1,G_2,\ldots,G_n}\mathcal{G}_{F_1(A_1),F_2(A_2),\ldots,F_m(A_m)}(E)$$

nombre_sucursal	*sum(sueldo)*
Centro	5.300
Leganés	3.100
Navacerrada	8.100

Figura 2.28 Resultado de $_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(sueldo)}(trabajo_por_horas)$.

www.detodoprogramacion.com

nombre_sucursal	*suma_sueldo*	*sueldo_máximo*
Centro	5.300	2.500
Leganés	3.100	1.600
Navacerrada	8.100	5.300

Figura 2.29 Resultado de $_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(sueldo)\ \mathbf{as}\ suma_sueldo,\mathbf{max}(sueldo)\ \mathbf{as}\ sueldo_máximo}(trabajo_por_horas)$.

donde E es cualquier expresión del álgebra relacional; $G_1, G_2, \ldots, G_n$ constituye una lista de atributos sobre los que se realiza la agrupación, cada F_i es una función de agregación y cada A_i es el nombre de un atributo. El significado de la operación es el siguiente: las tuplas del resultado de la expresión E se dividen en grupos de manera que

1. Todas las tuplas de cada grupo tienen los mismos valores para $G_1, G_2, \ldots, G_n$.
2. Las tuplas de grupos diferentes tienen valores diferentes para $G_1, G_2, \ldots, G_n$.

Por tanto, los grupos pueden identificarse por el valor de los atributos $G_1, G_2, \ldots, G_n$. Para cada grupo $(g_1, g_2, \ldots, g_n)$ el resultado tiene una tupla $(g_1, g_2, \ldots, g_n, a_1, a_2, \ldots, a_m)$ donde, para cada i, a_i es el resultado de aplicar la función de agregación F_i al multiconjunto de valores del atributo A_i en el grupo.

Como caso especial de la operación de agregación, la lista de atributos $G_1, G_2, \ldots, G_n$ puede estar vacía, en cuyo caso sólo hay un grupo que contiene todas las tuplas de la relación. Esto se corresponde con la agregación sin agrupación.

Volviendo al ejemplo anterior, si se deseara determinar el sueldo máximo de los empleados a tiempo parcial de cada sucursal, además de la suma de los sueldos, habría que escribir la expresión

$$_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(sueldo),\mathbf{max}(sueldo)}(trabajo_por_horas)$$

Como en la proyección generalizada, el resultado de las operaciones de agregación no tiene nombre. Se puede aplicar la operación renombramiento al resultado para darle un nombre. Como conveniencia notacional, los atributos de las operaciones de agregación se pueden renombrar como se indica a continuación:

$$_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(sueldo)\ \mathbf{as}\ suma_sueldo,\mathbf{max}(sueldo)\ \mathbf{as}\ sueldo_máximo}(trabajo_por_horas)$$

La Figura 2.29 muestra el resultado de la expresión.

2.4.3 Reunión externa

La operación **reunión externa** es una ampliación de la operación reunión para trabajar con información ausente. Supóngase que se dispone de relaciones con los siguientes esquemas y que contienen datos de empleados a tiempo completo:

empleado (*nombre_empleado*, *calle*, *ciudad*)
trabajo_a_tiempo_completo (*nombre_empleado*, *nombre_sucursal*, *sueldo*)

Considérense las relaciones *empleado* y *trabajo_a_tiempo_completo* mostradas en la Figura 2.30. Supóngase que se desea generar una única relación con toda la información (calle, ciudad, nombre de la sucursal y sueldo) de los empleados a tiempo completo. Un posible enfoque sería usar la operación reunión natural de la manera siguiente:

$$empleado \bowtie trabajo_a_tiempo_completo$$

El resultado de esta expresión se muestra en la Figura 2.31. Obsérvese que se ha perdido la información sobre la calle y la ciudad de residencia de Gómez, dado que la tupla que describe a Gómez no está presente en la relación *trabajo_a_tiempo_completo*; de manera parecida, se ha perdido la información sobre el nombre de la sucursal y sobre el sueldo de Barea, dado que la tupla que describe a Barea no está presente en la relación *empleado*.

www.detodoprogramacion.com

nombre_empleado	*calle*	*ciudad*
Segura	Tebeo	La Loma
Domínguez	Viaducto	Villaconejos
Gómez	Bailén	Alcorcón
Valdivieso	Fuencarral	Móstoles

nombre_empleado	*nombre_sucursal*	*sueldo*
Segura	Majadahonda	1.500
Domínguez	Majadahonda	1.300
Barea	Fuenlabrada	5.300
Valdivieso	Fuenlabrada	1.500

Figura 2.30 Las relaciones *empleado* y *trabajo_a_tiempo_completo*.

nombre_empleado	*calle*	*ciudad*	*nombre_sucursal*	*sueldo*
Domínguez	Viaducto	Villaconejos	Majadahonda	1.300
Segura	Tebeo	La Loma	Majadahonda	1.500
Valdivieso	Fuencarral	Móstoles	Fuenlabrada	1.500

Figura 2.31 El resultado de $empleado \bowtie trabajo_a_tiempo_completo$.

Se puede usar la operación reunión externa para evitar esta pérdida de información. En realidad, esta operación tiene tres formas diferentes: *reunión externa por la izquierda*, denotada por $\rightouterjoin$; *reunión externa por la derecha*, denotada por $\leftouterjoin$ y *reunión externa completa*, denotada por $\fullouterjoin$. Las tres formas de la reunión externa calculan la reunión y añaden tuplas adicionales al resultado de la misma. El resultado de las expresiones $empleado \rightouterjoin trabajo_a_tiempo_completo$, $empleado \leftouterjoin trabajo_a_tiempo_completo$ y $empleado \fullouterjoin trabajo_a_tiempo_completo$ se muestra en las Figuras 2.32, 2.33 y 2.34, respectivamente.

La **reunión externa por la izquierda** ($\rightouterjoin$) toma todas las tuplas de la relación de la izquierda que no coinciden con ninguna tupla de la relación de la derecha, las rellena con valores nulos en todos los demás atributos de la relación de la derecha y las añade al resultado de la reunión natural. En la Figura 2.32 la tupla (Gómez, Bailén, Alcorcón, $nulo$, $nulo$) es una tupla de este tipo. Toda la información de la relación de la izquierda se halla presente en el resultado de la reunión externa por la izquierda.

La **reunión externa por la derecha** ($\leftouterjoin$) es simétrica de la reunión externa por la izquierda. Rellena con valores nulos las tuplas de la relación de la derecha que no coinciden con ninguna tupla de la relación de la izquierda y las añade al resultado de la reunión natural. En la Figura 2.33 la tupla (Barea, $nulo$,

nombre_empleado	*calle*	*ciudad*	*nombre_sucursal*	*sueldo*
Domínguez	Viaducto	Villaconejos	Majadahonda	1.300
Gómez	Bailén	Alcorcón	*nulo*	*nulo*
Segura	Tebeo	La Loma	Majadahonda	1.500
Valdivieso	Fuencarral	Móstoles	Fuenlabrada	1.500

Figura 2.32 Resultado de $empleado \rightouterjoin trabajo_a_tiempo_completo$.

nombre_empleado	*calle*	*ciudad*	*nombre_sucursal*	*sueldo*
Barea	*nulo*	*nulo*	Fuenlabrada	5.300
Domínguez	Viaducto	Villaconejos	Majadahonda	1.300
Segura	Tebeo	La Loma	Majadahonda	1.500
Valdivieso	Fuencarral	Móstoles	Fuenlabrada	1.500

Figura 2.33 Resultado de $empleado \leftouterjoin trabajo_a_tiempo_completo$.

www.detodoprogramacion.com

nombre_empleado	*calle*	*ciudad*	*nombre_sucursal*	*sueldo*
Barea	*nulo*	*nulo*	Fuenlabrada	5.300
Domínguez	Viaducto	Villaconejos	Majadahonda	1.300
Gómez	Bailén	Alcorcón	*nulo*	*nulo*
Segura	Tebeo	La Loma	Majadahonda	1.500
Valdivieso	Fuencarral	Móstoles	Fuenlabrada	1.500

Figura 2.34 Resultado de $empleado \rightdoublebarjoin trabajo_a_tiempo_parcial$.

$nulo$, Fuenlabrada, 5.300) es una tupla de este tipo. Por tanto, toda la información de la relación de la derecha se halla presente en el resultado de la reunión externa por la derecha.

La **reunión externa completa** ($\rightdoublebarjoin\leftdoublebarjoin$) realiza estas dos operaciones, rellenando las tuplas de la relación de la izquierda que no coinciden con ninguna tupla de la relación de la derecha y las tuplas de la relación de la derecha que no coinciden con ninguna tupla de la relación de la izquierda, y añadiéndolas al resultado de la reunión. La Figura 2.34 muestra el resultado de una reunión externa completa.

Puesto que las operaciones de reunión pueden generar resultados que contengan valores nulos, es necesario especificar la manera en que deben manejar estos valores las diferentes operaciones del álgebra relacional. El Apartado 2.5 aborda este problema.

Es interesante observar que las operaciones de reunión externa pueden expresarse mediante las operaciones básicas del álgebra relacional. Por ejemplo, la operación de reunión externa por la izquierda $r \rightdoublebarjoin s$ se puede expresar como:

$$(r \bowtie s) \cup (r - \Pi_R(r \bowtie s)) \times \{(nulo, \ldots, nulo)\}$$

donde la relación constante $\{(nulo, \ldots, nulo)\}$ se encuentra en el esquema $S - R$.

2.5 Valores nulos

En este apartado se define la forma en que las diferentes operaciones del álgebra relacional tratan los valores nulos y las complicaciones que surgen cuando los valores nulos participan en las operaciones aritméticas o en las comparaciones. Como se verá, a menudo hay varias formas de tratar los valores nulos y, en consecuencia, las siguientes definiciones pueden ser a veces arbitrarias. Las operaciones y las comparaciones con valores nulos se deben evitar siempre que sea posible.

Dado que el valor especial *nulo* indica "valor desconocido o no existente", cualquier operación aritmética (como $+, -, *$ y $/$) que incluya valores nulos debe devolver un valor nulo.

De manera parecida, cualquier comparación (como $<, <=, >, >=$ y $\neq$) que incluya un valor nulo tiene como resultado el valor especial **desconocido**; no se puede decir si el resultado de la comparación es cierto o falso, así que se dice que el resultado es el nuevo valor lógico *desconocido*.

Las comparaciones que incluyen valores nulos pueden aparecer dentro de expresiones booleanas que incluyan las operaciones y (conjunción), o (disyunción) y no (negación). Por tanto, se debe definir la forma en que estas tres operaciones lógicas tratan el valor lógico *desconocido*.

- **Y**: (*cierto* **y** *desconocido*) = *desconocido*; (*falso* **y** *desconocido*) = *falso*; (*desconocido* **y** *desconocido*) = *desconocido*.
- **O**: (*cierto* **o** *desconocido*) = *cierto*; (*falso* **o** *desconocido*) = *desconocido*; (*desconocido* **o** *desconocido*) = *desconocido*.
- **No**: (**no** *desconocido*) = *desconocido*.

Ahora es posible describir la forma en que las diferentes operaciones del álgebra relacional tratan los valores nulos.

- **Selección**. La operación selección evalúa el predicado P en $\sigma_P(E)$ sobre cada tupla t de E. Si el predicado devuelve el valor *cierto*, se añade t al resultado. En caso contrario, si el predicado devuelve *desconocido* o *falso*, t no se añade al resultado.

www.detodoprogramacion.com

- **Reunión**. Las reuniones se pueden expresar como un producto cartesiano seguido de una selección. Por tanto, la definición de la forma en que la selección trata los nulos también define la forma en que lo hacen las operaciones reunión.

  En una reunión natural, por ejemplo, $r \bowtie s$ se puede observar de la definición anterior que si dos tuplas, $t_r \in r$ y $t_s \in s$, tienen un valor nulo en un atributo común, entonces las tuplas no coinciden.

- **Proyección**. La operación proyección trata los valores nulos como cualesquiera otros valores al eliminar valores duplicados. Así, si dos tuplas del resultado de la proyección son exactamente iguales y las dos tienen valores nulos en los mismos campos, se tratan como duplicados.

  Esta decisión es un tanto arbitraria ya que, sin conocer el valor real, no se sabe si los dos valores nulos son duplicados o no.

- **Unión**, **intersección** y **diferencia**. Estas operaciones tratan los valores nulos igual que la operación proyección; tratan las tuplas que tienen los mismos valores en todos los campos como duplicados, aunque algunos de los campos tengan valores nulos en ambas tuplas.

  Este comportamiento es un tanto arbitrario, especialmente en el caso de la intersección y la diferencia, dado que no se sabe si los valores reales (si existen) representados por los nulos son iguales.

- **Proyección generalizada**. Se describió la manera en que se tratan los nulos en las expresiones al comienzo del Apartado 2.5. Las tuplas duplicadas que contienen valores nulos se tratan como en la operación proyección.

- **Funciones de agregación**. Cuando hay valores nulos en los atributos agregados, las operaciones de agregación los tratan igual que en el caso de la proyección: si dos tuplas son iguales en todos los atributos de agregación, la operación las coloca en el mismo grupo, aunque parte de los valores de los atributos sean valores nulos.

  Cuando aparecen valores nulos en los atributos agregados, la operación borra los valores nulos del resultado antes de aplicar la agregación. Si el multiconjunto resultante está vacío, el resultado agregado es nulo.

  Obsérvese que el tratamiento de los valores nulos en este caso es diferente que en las expresiones aritméticas ordinarias; se podría haber definido el resultado de una operación de agregación como nulo si tan sólo uno de los valores agregados fuera nulo. Sin embargo, esto significaría que un único valor desconocido en un gran grupo podría hacer que el resultado agregado sobre el grupo fuese nulo, y se perdería una gran cantidad de información útil.

- **Reunión externa**. Las operaciones de reunión externa se comportan igual que las operaciones de reunión, excepto sobre las tuplas que no aparecen en el resultado de la reunión. Esas tuplas se pueden añadir al resultado (dependiendo de si la operación es $\supset\!\bowtie, \bowtie\!\subset$ o $\supset\!\bowtie\!\subset$) rellenando con valores nulos.

2.6 Modificación de la base de datos

Hasta ahora se ha centrado la atención en la extracción de información de la base de datos. En este apartado se abordará la manera de añadir, eliminar y modificar información de la base de datos.

Las modificaciones de la base de datos se expresan mediante la operación asignación. Las asignaciones a las relaciones reales de la base de datos se realizan empleando la misma notación que se describió para la asignación en el Apartado 2.3.

2.6.1 Borrado

Las solicitudes de borrado se expresan básicamente igual que las consultas. Sin embargo, en lugar de mostrar las tuplas al usuario, se eliminan de la base de datos las tuplas seleccionadas. Sólo se pueden borrar tuplas enteras; no se pueden borrar valores de atributos concretos. En el álgebra relacional los borrados se expresan mediante

$$r \leftarrow r - E$$

www.detodoprogramacion.com

donde *r* es una relación y *E* es una consulta del álgebra relacional.

A continuación se muestran varios ejemplos de borrado:

- Borrar todas las cuentas de Gómez.

$$impositor \leftarrow impositor - \sigma_{nombre_cliente = \text{“Gómez”}}(impositor)$$

- Borrar todos los préstamos con importes entre 0 y 50.

$$préstamo \leftarrow préstamo - \sigma_{importe \geq 0 \wedge importe \leq 50}(préstamo)$$

- Borrar todas las cuentas de las sucursales de Arganzuela

$$\begin{aligned} r_1 &\leftarrow \sigma_{ciudad_sucursal = \text{“Arganzuela”}}(cuenta \bowtie sucursal) \\ r_2 &\leftarrow \Pi_{nombre_sucursal, número_cuenta, saldo}(r_1) \\ cuenta &\leftarrow cuenta - r_2 \end{aligned}$$

  Obsérvese que en el último ejemplo se ha simplificado la expresión mediante la asignación a relaciones temporales (r_1 y r_2).

2.6.2 Inserción

Para insertar datos en una relación hay que especificar la tupla que se va a insertar o escribir una consulta cuyo resultado sea el conjunto de tuplas que se van a insertar. Evidentemente, el valor de los atributos de las tuplas insertadas debe ser miembro del dominio de cada atributo. De manera parecida, las tuplas insertadas deben tener la aridad correcta. El álgebra relacional expresa las inserciones mediante

$$r \leftarrow r \cup E$$

donde *r* es una relación y *E* es una expresión del álgebra relacional. La inserción de una sola tupla se expresa haciendo que *E* sea una relación constante que contiene una tupla.

Supóngase que se desea insertar el hecho de que Gómez tiene 1.200 € en la cuenta C-973 de la sucursal de Navacerrada. Hay que escribir

cuenta ← *cuenta* ∪ {(C-973, “Navacerrada”, 1200)}
impositor ← *impositor* ∪ {(“Gómez”, C-973)}

De forma más general, puede que se desee insertar tuplas según el resultado de una consulta. Supóngase que se desea ofrecer una nueva cuenta de ahorro con 200 € como regalo a todos los clientes con préstamos concedidos en la sucursal de Navacerrada. Se usará el número de préstamo como número de esta cuenta de ahorro. Se escribe:

$$\begin{aligned} r_1 &\leftarrow (\sigma_{nombre_sucursal = \text{“Navacerrada”}}(prestatario \bowtie préstamo)) \\ r_2 &\leftarrow \Pi_{número_préstamo, nombre_sucursal}(r_1) \\ &cuenta \leftarrow cuenta \cup (r_2 \times \{(200)\}) \\ &impositor \leftarrow impositor \cup \Pi_{nombre_cliente, número_préstamo}(r_1) \end{aligned}$$

En lugar de especificar una tupla como se hizo anteriormente, se especifica un conjunto de tuplas que se inserta en las relaciones *cuenta* e *impositor*. Cada tupla de la relación *cuenta* tiene un *número_cuenta* (que es igual que el número de préstamo), un *nombre_sucursal* (Navacerrada) y el saldo inicial de la nueva cuenta (200 €). Cada tupla de la relación *impositor* tiene como *nombre_cliente* el nombre del prestatario al que se le regala la nueva cuenta y el mismo número de cuenta que la correspondiente tupla de *cuenta*.

2.6.3 Actualización

Puede que, en algunas situaciones, se desee modificar un valor de una tupla sin modificar *todos* los valores de esa tupla. Se puede usar el operador proyección generalizada para llevar a cabo esta tarea:

$$r \leftarrow \Pi_{F_1, F_2, \ldots, F_n}(r)$$

www.detodoprogramacion.com

donde cada F_i es, o bien el i-ésimo atributo de r en el caso de que este atributo no se vaya a actualizar o, en caso contrario, una expresión sólo con constantes y atributos de r que proporciona el nuevo valor del atributo. Téngase en cuenta que el esquema de la expresión resultante de la expresión de proyección generalizada debe coincidir con el esquema original de r.

Si se desea seleccionar varias tuplas de r y actualizar sólo esas tuplas, se puede usar la expresión siguiente, donde P denota la condición de selección que escoge las tuplas que hay que actualizar:

$$r \leftarrow \Pi_{F_1, F_2, \ldots, F_n}(\sigma_P(r)) \cup (r - \sigma_P(r))$$

Para ilustrar el uso de la operación actualización supóngase que se va a realizar el pago de los intereses y que hay que aumentar todos los saldos en un cinco por ciento:

$$cuenta \leftarrow \Pi_{número_cuenta, nombre_sucursal,\ saldo\ *\ 1.05}\ (cuenta)$$

Supóngase ahora que las cuentas con saldos superiores a 10.000 € reciben un interés del seis por ciento, mientras que el resto recibe un cinco por ciento:

$$\begin{aligned} cuenta \leftarrow\ & \Pi_{número_cuenta, nombre_sucursal,\ saldo\ *\ 1.06}\ (\sigma_{saldo > 10000}\ (cuenta)) \\ \cup\ & \Pi_{número_cuenta, nombre_sucursal,\ saldo\ *\ 1.05}\ (\sigma_{saldo \leq 10000}\ (cuenta)) \end{aligned}$$

2.7 Resumen

- El **modelo de datos relacional** se basa en un conjunto de tablas. El usuario del sistema de bases de datos puede consultar esas tablas, insertar tuplas nuevas, borrar tuplas y actualizar (modificar) las tuplas. Hay varios lenguajes para expresar estas operaciones.
- El **álgebra relacional** define un conjunto de operaciones algebraicas que operan sobre las tablas y devuelven tablas como resultado. Estas operaciones se pueden combinar para obtener expresiones que definan las consultas deseadas. El álgebra define las operaciones básicas usadas en los lenguajes de consultas relacionales.
- Las operaciones del álgebra relacional se pueden dividir en:
  - Operaciones básicas.
  - Operaciones adicionales que se pueden expresar en términos de las operaciones básicas.
  - Operaciones extendidas, algunas de las cuales añaden mayor poder expresivo al álgebra relacional.
- Las bases de datos se pueden modificar con la **inserción**, el **borrado** y la **actualización** de tuplas. Se ha usado el álgebra relacional con el **operador asignación** para expresar estas modificaciones.
- El álgebra relacional es un lenguaje rígido y formal que no resulta adecuado para los usuarios ocasionales de los sistemas de bases de datos. Los sistemas comerciales de bases de datos, por tanto, usan lenguajes con más "azúcar sintáctico". En los Capítulos 3 y 4 se tomará en consideración el lenguaje más influyente, **SQL**, que está basado en el álgebra relacional.

Términos de repaso

- Tabla.
- Relación.
- Variable tupla.
- Dominio atómico.
- Valor nulo.
- Esquema de la base de datos.
- Ejemplar de la base de datos.
- Esquema de la relación.
- Ejemplar de la relación.
- Claves.
- Clave externa.
  - Relación referenciante.
  - Relación referenciada.
- Diagrama de esquema.
- Lenguaje de consultas.
- Lenguaje procedimental.

www.detodoprogramacion.com

CAPÍTULO 3

SQL

El álgebra relacional descrita en el Capítulo 2 proporciona una notación concisa y formal para la representación de las consultas. Sin embargo, los sistemas de bases de datos comerciales necesitan un lenguaje de consultas más cómodo para el usuario. En este capítulo y en el Capítulo 4 se estudia SQL, el lenguaje de consultas distribuido comercialmente de más influencia. SQL usa una combinación de constructores del álgebra relacional (Capítulo 2) y del cálculo relacional (Capítulo 5).

Aunque se haga referencia al lenguaje SQL como "lenguaje de consultas", puede hacer mucho más que consultar las bases de datos. Usando SQL es posible además definir la estructura de los datos, modificar los datos de la base de datos y especificar restricciones de seguridad.

No se pretende proporcionar un manual de usuario completo de SQL. En cambio, se presentan los constructores y conceptos fundamentales de SQL. Las distintas implementaciones de SQL pueden diferenciarse en detalles o admitir sólo un subconjunto del lenguaje completo.

3.1 Introducción

IBM desarrolló la versión original de SQL, originalmente denominado Sequel, como parte del proyecto System R a principios de 1970. El lenguaje Sequel ha evolucionado desde entonces y su nombre ha pasado a ser SQL (Structured Query Language, lenguaje estructurado de consultas). Hoy en día, numerosos productos son compatibles con el lenguaje SQL y se ha establecido como *el* lenguaje estándar para las bases de datos relacionales.

En 1986, ANSI (American National Standards Institute, Instituto nacional americano de normalización) e ISO (International Standards Organization, Organización internacional de normalización) publicaron una norma SQL, denominada SQL-86. En 1989 ANSI publicó una extensión de la norma para SQL denominada SQL-89. La siguiente versión de la norma fue SQL-92 seguida de SQL:1999; la versión más reciente es SQL:2003. Las notas bibliográficas proporcionan referencias a esas normas.

El lenguaje SQL tiene varios componentes:

- **Lenguaje de definición de datos** (LDD). El LDD de SQL proporciona comandos para la definición de esquemas de relación, borrado de relaciones y modificación de los esquemas de relación.
- **Lenguaje interactivo de manipulación de datos** (LMD). El LMD de SQL incluye un lenguaje de consultas basado tanto en el álgebra relacional (Capítulo 2) como en el cálculo relacional de tuplas (Capítulo 5). También contiene comandos para insertar, borrar y modificar tuplas.
- **Integridad**. El LDD de SQL incluye comandos para especificar las restricciones de integridad que deben cumplir los datos almacenados en la base de datos. Las actualizaciones que violan las restricciones de integridad se rechazan.
- **Definición de vistas**. El LDD de SQL incluye comandos para la definición de vistas.

www.detodoprogramacion.com

- **Control de transacciones**. SQL incluye comandos para especificar el comienzo y el final de las transacciones.
- **SQL incorporado** y **SQL dinámico**. SQL incorporado y SQL dinámico definen cómo se pueden incorporar instrucciones de SQL en lenguajes de programación de propósito general como C, C++, Java, PL/I, Cobol, Pascal y Fortran.
- **Autorización**. El LDD de SQL incluye comandos para especificar los derechos de acceso a las relaciones y a las vistas.

En este capítulo se presenta una visión general del LMD básico y de las características básicas del LDD de SQL. Esta descripción se basa principalmente en la muy extendida norma SQL-92, pero también se tratan algunas extensiones de las normas SQL:1999 y SQL:2003.

En el Capítulo 4 se ofrece un tratamiento más detallado del sistema de tipos de SQL, de las restricciones de integridad y de las autorizaciones. En ese capítulo también se describen brevemente SQL incorporado y SQL dinámico, incluyendo las normas ODBC y JDBC para la interacción con las bases de datos desde programas escritos en los lenguajes C y Java. En el Capítulo 9 se esbozan las extensiones orientadas a objetos de SQL que se introdujeron en SQL:1999.

Muchos sistemas de bases de datos soportan la mayor parte de la norma SQL-92 y parte de los nuevos constructores de SQL:1999 y SQL:2003, aunque actualmente ninguno soporta todos los constructores nuevos. También se debe tener en cuenta que muchos sistemas de bases de datos no soportan algunas características de SQL-92, y que muchas bases de datos presentan algunas no estándar que no se estudian en este libro. En caso de que alguna característica del lenguaje aquí descrita no funcione en el sistema de bases de datos que se esté utilizando, se debe consultar el manual de usuario para determinar exactamente lo que soporta.

La empresa que se utiliza para los ejemplos de este capítulo y de los posteriores es una entidad bancaria. La Figura 3.1 muestra el esquema relacional que se utiliza en los ejemplos con los atributos que son claves primarias subrayados. Recuérdese que en el Capítulo 2 ya se definió el esquema de una relación R mediante una relación de sus atributos, y posteriormente se definió una relación r del esquema mediante la notación $r(R)$. La notación de la Figura 3.1 omite el nombre del esquema y define el esquema de la relación directamente mediante una relación de sus atributos.

3.2 Definición de datos

El conjunto de relaciones de cada base de datos debe especificarse en el sistema en términos de un lenguaje de definición de datos (LDD). El LDD de SQL no sólo permite la especificación de un conjunto de relaciones, sino también de la información relativa a esas relaciones, incluyendo:

- El esquema de cada relación.
- El dominio de valores asociado a cada atributo.
- Las restricciones de integridad.
- El conjunto de índices que se deben mantener para cada relación.
- La información de seguridad y de autorización de cada relación.
- La estructura de almacenamiento físico de cada relación en el disco.

sucursal(<u>*nombre_sucursal*</u>, *ciudad_sucursal*, *activos*)
cliente (<u>*nombre_cliente*</u>, *calle_cliente*, *ciudad_cliente*)
préstamo (<u>*número_préstamo*</u>, *nombre_sucursal*, *importe*)
prestatario (<u>*nombre_cliente*</u>, <u>*número_préstamo*</u>)
cuenta (<u>*número_cuenta*</u>, *nombre_sucursal*, *saldo*)
impositor (<u>*nombre_cliente*</u>, <u>*número_cuenta*</u>)

Figura 3.1 Esquema de la entidad bancaria.

www.detodoprogramacion.com

A continuación se analizará la definición de los esquemas y los valores básicos de los dominios; el análisis de las demás características del LDD de SQL se tratará en el Capítulo 4.

3.2.1 Tipos básicos de dominios

La norma SQL soporta gran variedad de tipos de dominio predefinidos, entre ellos:

- **char**(n). Una cadena de caracteres de longitud fija, con una longitud n especificada por el usuario. También se puede utilizar la palabra completa **character**.
- **varchar**(n). Una cadena de caracteres de longitud variable con una longitud máxima n especificada por el usuario. La forma completa, **character varying**, es equivalente.
- **int**. Un entero (un subconjunto finito de los enteros dependiente de la máquina). La palabra completa, **integer**, es equivalente.
- **smallint**. Un entero pequeño (un subconjunto dependiente de la máquina del tipo de dominio entero).
- **numeric**(p, d). Un número de coma fija, cuya precisión la especifica el usuario. El número está formado por p dígitos (más el signo), y de esos p dígitos, d pertenecen a la parte decimal. Así, **numeric**(3,1) permite que el número 44.5 se almacene exactamente, pero ni 444.5 ni 0.32 se pueden almacenar exactamente en un campo de este tipo.
- **real, double precision**. Números de coma flotante y números de coma flotante de doble precisión, con precisión dependiente de la máquina.
- **float**(n). Un número de coma flotante cuya precisión es, al menos, de n dígitos.

En el Apartado 4.1 se tratan otros tipos de dominio.

3.2.2 Definición básica de esquemas en SQL

Las relaciones se definen mediante el comando **create table**:

$$\begin{aligned}\textbf{create table } r(&A_1\ D_1, A_2\ D_2, \ldots, A_n\ D_n,\\ &\langle\text{restricción-integridad}_1\rangle,\\ &\ldots,\\ &\langle\text{restricción-integridad}_k\rangle)\end{aligned}$$

donde *r* es el nombre de la relación, cada A_i es el nombre de un atributo del esquema de la relación *r* y D_i es el tipo de dominio de los valores del dominio del atributo A_i. Hay varias restricciones de integridad válidas. En este apartado sólo se estudiarán las de clave primaria (primary key), que adopta la forma:

- **primary key** $(A_{j_1}, A_{j_2}, \ldots, A_{j_m})$. La especificación de **clave primaria** determina que los atributos $A_{j_1}, A_{j_2}, \ldots, A_{j_m}$ forman la clave primaria de la relación. Los atributos de la clave primaria tienen que ser *no nulos* y *únicos*; es decir, ninguna tupla puede tener un valor nulo para un atributo de la clave primaria y ningún par de tuplas de la relación puede ser igual en todos los atributos de la clave primaria[1]. Aunque la especificación de clave primaria es opcional, suele ser una buena idea especificar una clave primaria para cada relación.

Otras restricciones de integridad que puede incluir el comando **create table** se tratan más adelante, en el Apartado 4.2.

La Figura 3.2 presenta una definición parcial en el LDD de SQL de la base de datos bancaria. Obsérvese que, al igual que en capítulos anteriores, no se intenta modelar con precisión el mundo real en el ejemplo de la base de datos bancaria. En el mundo real varias personas pueden llamarse igual, por lo que *nombre_cliente* no sería clave primaria de *cliente*; probablemente se utilizaría un *id_cliente* como clave primaria.

1. En SQL-89, los atributos que forman la clave primaria no estaban declarados implícitamente como **not null**; era necesaria una declaración explícita.

www.detodoprogramacion.com

```
create table cliente
    (nombre_cliente     char(20),
     calle_cliente      char(30),
     ciudad_cliente     char(30),
     primary key (nombre_cliente))

create table sucursal
    (nombre_sucursal    char(15),
     ciudad_sucursal    char(30),
     activos            numeric(16,2),
     primary key (nombre_sucursal))

create table cuenta
    (número_cuenta      char(10),
     nombre_sucursal    char(15),
     saldo              numeric(12,2),
     primary key (número_cuenta))

create table impositor
    (nombre_cliente     char(20),
     número_cuenta      char(10),
     primary key (nombre_cliente, número_cuenta))
```

Figura 3.2 Definición de datos en SQL para parte de la base de datos del banco.

Aquí se utiliza *nombre_cliente* como clave primaria para mantener el esquema de la base de datos sencillo y de pequeño tamaño.

Si una tupla recién insertada o recién modificada de una relación contiene valores nulos para cualquiera de los atributos que forman parte de la clave primaria, o si tiene el mismo valor en ellos que otra tupla de la relación, SQL notifica el error e impide la actualización.

Las relaciones recién creadas están inicialmente vacías. Se puede utilizar el comando **insert** para añadir datos a la relación. Por ejemplo, si se desea añadir el hecho de que hay una cuenta C-9732 en la sucursal de Navacerrada con un saldo de 1.200 €, hay que escribir

```
insert into cuenta
        values ('C-9732', 'Navacerrada', 1200)
```

Los valores se especifican en el orden en que se relacionan los atributos correspondientes en el esquema de la relación. El comando **insert** ofrece una serie de características útiles, y se trata con más detalle en el Apartado 3.10.2.

Se puede utilizar el comando **delete** para borrar tuplas de una relación. El comando

```
delete from cuenta
```

borraría todas las tuplas de la relación *cuenta*. Otras formas del comando **delete** permiten borrar sólo tuplas concretas; el comando **delete** se trata con más detalle en el Apartado 3.10.1.

Para eliminar una relación de una base de datos SQL se utiliza el comando **drop table**. Este comando elimina de la base de datos toda la información de la relación. La instrucción

```
drop table r
```

es una acción más drástica que

```
delete from r
```

www.detodoprogramacion.com

La última conserva la relación *r*, pero borra todas sus tuplas. La primera no sólo borra todas las tuplas de la relación *r*, sino que también elimina su esquema. Una vez eliminada *r*, no se puede insertar ninguna tupla en dicha relación, a menos que se vuelva a crear con la instrucción **create table**.

El comando **alter table** se utiliza para añadir atributos a una relación existente. Se asigna a todas las tuplas de la relación un valor *nulo* como valor del atributo nuevo. La forma del comando **alter table** es

alter table *r* **add** $A\ D$

donde *r* es el nombre de una relación ya existente, *A* es el nombre del atributo que se desea añadir y *D* es el dominio del atributo añadido. Se pueden eliminar atributos de una relación utilizando el comando

alter table *r* **drop** A

donde *r* es el nombre de una relación ya existente y *A* es el nombre de un atributo de la relación. Muchos sistemas de bases de datos no permiten el borrado de atributos, aunque sí permiten el borrado de tablas completas.

3.3 Estructura básica de las consultas SQL

Las bases de datos relacionales están formadas por un conjunto de relaciones, a cada una de las cuales se le asigna un nombre único. Cada relación posee una estructura similar a la presentada en el Capítulo 2. SQL permite el uso de valores nulos para indicar que el valor es desconocido o no existe. También permite al usuario especificar los atributos que no pueden contener valores nulos, como se indicó en el Apartado 3.2.

La estructura básica de una expresión SQL consta de tres cláusulas: **select**, **from** y **where**.

- La cláusula **select** se corresponde con la operación proyección del álgebra relacional. Se usa para obtener una relación de los atributos deseados en el resultado de una consulta.
- La cláusula **from** se corresponde con la operación producto cartesiano del álgebra relacional. Genera una lista de las relaciones que deben ser analizadas en la evaluación de la expresión.
- La cláusula **where** se corresponde con el predicado selección del álgebra relacional. Es un predicado que engloba a los atributos de las relaciones que aparecen en la cláusula **from**.

Que el término *select* tenga un significado diferente en SQL que en el álgebra relacional es un hecho histórico desafortunado. En este capítulo se destacan las diferentes interpretaciones para reducir al mínimo las posibles confusiones.

Las consultas habituales de SQL tienen la forma

select $A_1,\ A_2, \ldots, A_n$
from $r_1,\ r_2, \ldots, r_m$
where P

Cada A_i representa un atributo y cada r_i, una relación. *P* es un predicado. La consulta es equivalente a la expresión del álgebra relacional

$$\Pi_{A_1,\ A_2, \ldots,\ A_n}(\sigma_P\,(r_1\ \times\ r_2\ \times\ \cdots\ \times\ r_m))$$

Si se omite la cláusula **where**, el predicado P es **cierto**. Sin embargo, a diferencia de la expresión del álgebra relacional, el resultado de la consulta SQL puede contener varias copias de algunas tuplas; este aspecto se analizará de nuevo en el Apartado 3.3.8.

SQL forma el producto cartesiano de las relaciones incluidas en la cláusula **from**, lleva a cabo la selección del álgebra relacional utilizando el predicado de la cláusula **where** y después proyecta el resultado sobre los atributos de la cláusula **select**. En la práctica, SQL puede convertir la expresión en una forma equivalente que puede ser procesada de manera más eficiente. No obstante, las cuestiones relativas a la eficiencia se posponen hasta los Capítulos 13 y 14.

www.detodoprogramacion.com

3.3.1 La cláusula select

El resultado de las consultas SQL es, por supuesto, una relación. Considérese la consulta simple basada en el ejemplo bancario "Obtener el nombre de todas las sucursales de la relación *préstamo*":

select *nombre_sucursal*
from *préstamo*

El resultado es una relación consistente en un único atributo con el encabezado *nombre_sucursal*.

Los lenguajes formales de consultas están basados en el concepto matemático de que las relaciones son conjuntos. Así, nunca aparecen tuplas duplicadas en las relaciones. En la práctica, la eliminación de duplicados consume tiempo. Por tanto, SQL (como la mayoría de los lenguajes de consultas comerciales) permite duplicados en las relaciones, así como en el resultado de las expresiones SQL. Así, la consulta anterior mostrará el valor de *nombre_sucursal* una vez por cada tupla de la relación *préstamo* en la que aparezca.

En los casos en los que se desea forzar la eliminación de los valores duplicados, se inserta la palabra clave **distinct** después de **select**. Se puede reescribir la consulta anterior como

select distinct *nombre_sucursal*
from *préstamo*

si se desea eliminar los duplicados.

SQL permite utilizar la palabra clave **all** para especificar de manera explícita que no se eliminen los duplicados:

select all *nombre_sucursal*
from *préstamo*

Dado que la retención de duplicados es la opción predeterminada, no se utilizará **all** en los ejemplos. Para garantizar la eliminación de duplicados en el resultado de los ejemplos de consultas, se usará la cláusula **distinct** siempre que sea necesario. En la mayoría de las consultas en que no se utiliza **distinct**, el número exacto de copias duplicadas de cada tupla presentes en el resultado de la consulta no es importante. Sin embargo, el número sí que es importante en ciertas aplicaciones; esto se volverá a tratar en el Apartado 3.3.8.

El símbolo asterisco " * " se puede usar para denotar "todos los atributos". Así, el uso de *préstamo*.* en la cláusula **select** anterior indicaría que se seleccionarían todos los atributos de *préstamo*. Las cláusulas **select** de la forma **select** * indican que se deben seleccionar todos los atributos de todas las relaciones que aparecen en la cláusula **from**.

La cláusula **select** puede contener también expresiones aritméticas que contengan los operadores $+$, $-$, $*$ y $/$ y operen sobre constantes o atributos de la tuplas. Por ejemplo, la consulta

select *número_préstamo*, *nombre_sucursal*, *importe* * 100
from *préstamo*

devolverá una relación que es igual que la relación *préstamo*, salvo que el atributo *importe* está multiplicado por 100.

SQL también proporciona tipos de datos especiales, tales como varias formas del tipo *date* (fecha) y permite que varias funciones aritméticas operen sobre esos tipos.

3.3.2 La cláusula where

A continuación se muestra el uso de la cláusula **where** en SQL. Considérese la consulta "Obtener todos los números de préstamo de los préstamos concedidos en la sucursal de Navacerrada e importe superior a 1.200 €". Esta consulta puede escribirse en SQL como:

www.detodoprogramacion.com

select *número_préstamo*
from *préstamo*
where *nombre_sucursal* = 'Navacerrada' **and** *importe* > 1200

SQL usa las conectivas lógicas **and**, **or** y **not** (en lugar de los símbolos matemáticos $\wedge$, $\vee$ y $\neg$) en la cláusula **where**. Los operandos de las conectivas lógicas pueden ser expresiones que contengan los operadores de comparación <, <=, >, >=, = y <>. SQL permite utilizar los operadores de comparación para comparar cadenas y expresiones aritméticas, así como tipos especiales, como los tipos de fecha.

SQL incluye también un operador de comparación **between** para simplificar las cláusulas **where**, el cual especifica que un valor sea menor o igual que un valor y mayor o igual que otro valor. Si se desea obtener el número de préstamo de aquellos préstamos con importe entre 90.000 € y 100.000 €, se puede usar la comparación **between** para escribir

select *número_préstamo*
from *préstamo*
where *importe* **between** 90000 **and** 100000

en lugar de

select *número_préstamo*
from *préstamo*
where *importe* >= 90000 **and** *importe* <= 100000

De forma análoga, se puede usar el operador de comparación **not between**.

3.3.3 La cláusula from

Finalmente, se estudiará el uso de la cláusula **from**. La cláusula **from** define por sí misma un producto cartesiano de las relaciones que aparecen en la cláusula. Dado que la reunión natural se define en términos de un producto cartesiano, una selección y una proyección, resulta relativamente sencillo escribir una expresión de SQL para la reunión natural. Se escribe la expresión del álgebra relacional

$$\Pi_{nombre_cliente,\, número_préstamo,\, importe} \, (prestatario \bowtie préstamo)$$

para la consulta "Para todos los clientes que tienen un préstamo del banco, obtener el nombre, el número de préstamo y su importe". Esta consulta puede escribirse en SQL como

select *nombre_cliente, prestatario.número_préstamo, importe*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo*

Obsérvese que SQL usa la notación *nombre-relación.nombre-atributo*, como lo hace el álgebra relacional, para evitar ambigüedades en los casos en los que un atributo aparece en más de un esquema de relación. También se podría haber usado *prestatario.nombre_cliente* en lugar de *nombre_cliente* en la cláusula **select**. Sin embargo, como el atributo *nombre_cliente* sólo aparece en una de las relaciones de la cláusula **from**, no existe ambigüedad al escribir *nombre_cliente*.

Se puede extender la consulta anterior y considerar un caso más complicado en el que se también se exija que el préstamo se haya concedido en la sucursal de Navacerrada: "Determinar el nombre, el número de préstamo y su importes de todos los préstamos de la sucursal de Navacerrada". Para escribir esta consulta hay que definir dos restricciones en la cláusula **where**, enlazadas con la conectiva lógica **and**:

select *nombre_cliente, prestatario.número_préstamo, importe*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo* **and**
nombre_sucursal = 'Navacerrada'

www.detodoprogramacion.com

SQL incluye extensiones para llevar a cabo reuniones naturales y reuniones externas en la cláusula **from**. Estas extensiones se estudian en el Apartado 3.11.

3.3.4 La operación renombramiento

SQL proporciona un mecanismo para renombrar tanto relaciones como atributos. Utiliza la cláusula **as** de la forma siguiente:

nombre-antiguo **as** *nombre-nuevo*

La cláusula **as** puede aparecer tanto en la cláusula **select** como en la cláusula **from**.

Considérese de nuevo la consulta anterior:

select *nombre_cliente, prestatario.número_préstamo, importe*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo*

El resultado de esta consulta es una relación con los atributos siguientes:

nombre_cliente, número_préstamo, importe

Los nombres de los atributos en el resultado proceden de los nombres de los atributos de las relaciones que aparecen en la cláusula **from**.

Sin embargo, no se pueden obtener siempre los nombres de este modo por varios motivos. En primer lugar, dos relaciones de la cláusula **from** pueden tener atributos con el mismo nombre, en cuyo caso, un nombre de atributo aparecerá duplicado en el resultado. En segundo lugar, si se ha utilizado una expresión aritmética en la cláusula **select**, los atributos resultantes no tienen nombre. En tercer lugar, incluso si el nombre de un atributo se puede obtener a partir de las relaciones base, como en el ejemplo anterior, puede que se desee cambiar el nombre del atributo en el resultado. Por ello, SQL proporciona una forma de renombrar los atributos de las relaciones resultantes.

Por ejemplo, si se desea cambiar el nombre del atributo *número_préstamo* por *id_préstamo*, se puede volver a escribir la consulta anterior como

select *nombre_cliente, prestatario.número_préstamo* **as** *id_préstamo, importe*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo*

3.3.5 Variables tupla

La cláusula **as** resulta especialmente útil en la definición del concepto de variable tupla. Las variables tupla en SQL se deben asociar con una relación concreta. Las variables tupla se definen en la cláusula **from** mediante la cláusula **as**. Para ilustrarlo, se reescribirá la consulta "Para todos los clientes que tienen concedido un préstamo por el banco, obtener el nombre, el número de préstamo y su importe" como

select *nombre_cliente, T.número_préstamo, S.importe*
from *prestatario* **as** *T*, *préstamo* **as** *S*
where *T.número_préstamo* = *S.número_préstamo*

Obsérvese que la variable tupla se define en la cláusula **from** colocándola después del nombre de la relación a la cual está asociada y detrás de la palabra clave **as** (la palabra clave **as** es opcional). Al escribir expresiones de la forma *nombre_relación.nombre_atributo*, el nombre de la relación es, en efecto, una variable tupla definida implícitamente.

Las variables tupla son de gran utilidad para comparar dos tuplas de la misma relación. Hay que recordar que, en estos casos, se puede usar la operación renombramiento del álgebra relacional. Supóngase que se desea formular la consulta "Determinar el nombre de todas las sucursales que tienen activos mayores que, al menos, una sucursal de Arganzuela". Se puede escribir la expresión SQL siguiente:

www.detodoprogramacion.com

select distinct *T.nombre_sucursal*
from *sucursal* **as** *T*, *sucursal* **as** *S*
where *T.activos* $>$ *S.activos* **and** *S.ciudad_sucursal* $=$ 'Arganzuela'

Obsérvese que no se puede utilizar la notación *sucursal.activos*, puesto que no estaría claro qué referencia a *sucursal* se pretende hacer.

SQL permite usar la notación $(v_1, v_2, \ldots, v_n)$ para designar una tupla de aridad n que contiene los valores $v_1, v_2, \ldots, v_n$. Los operadores de comparación se pueden utilizar sobre las tuplas, y el orden se define lexicográficamente. Por ejemplo, $(a_1, a_2) <= (b_1, b_2)$ es cierto si $a_1 < b_1$, o si se cumple que $(a_1 = b_1) \wedge (a_2 <= b_2)$; de manera parecida, dos tuplas son iguales si lo son todos sus atributos.

3.3.6 Operaciones con cadenas de caracteres

SQL especifica las cadenas de caracteres encerrándolas entre comillas simples, como en 'Navacerrada', como ya se ha visto anteriormente. Los caracteres de comillas que formen parte de una cadena de caracteres se pueden especificar usando dos caracteres de comillas; por ejemplo, la cadena de caracteres "Peter O'Toole es un gran actor" se puede especificar como "Peter O''Toole es un gran actor".

La operación más utilizada sobre las cadenas de caracteres es la comparación de patrones, para la que se usa el operador **like**. Para la descripción de los patrones se utilizan dos caracteres especiales:

- Tanto por ciento (%). El carácter % coincide con cualquier subcadena de caracteres.
- Subrayado (_). El carácter _ coincide con cualquier carácter.

Los patrones distinguen la caja; es decir, los caracteres en mayúsculas se consideran diferentes de los caracteres en minúscula, y viceversa. Para ilustrar la comparación de patrones, se considerarán los siguientes ejemplos:

- 'Nava %' coincide con cualquier cadena de caracteres que empiece con "Nava".
- ' %cer %' coincide con cualquier cadena que contenga "cer" como subcadena, por ejemplo 'Navacerrada', 'Cáceres' y 'Becerril'.
- '___' coincide con cualquier cadena que tenga exactamente tres caracteres.
- '___%' encaja con cualquier cadena que tenga, al menos, tres caracteres.

Los patrones se expresan en SQL utilizando el operador de comparación **like**. Considérese la consulta "Determinar el nombre de todos los clientes cuya dirección contenga la subcadena de caracteres 'Mayor'". Esta consulta se puede escribir como

select *nombre_cliente*
from *cliente*
where *calle_cliente* **like** ' %Mayor %'

Para que los patrones puedan contener los caracteres especiales de patrón (esto es, % y _) SQL permite la especificación de un carácter de escape. El carácter de escape se utiliza inmediatamente antes de los caracteres especiales de patrón para indicar que ese carácter especial se va a tratar como un carácter normal. El carácter de escape para una comparación **like** se define utilizando la palabra clave **escape**. Para ilustrar esto, considérense los siguientes patrones, que utilizan una barra invertida (\) como carácter de escape:

- **like** 'ab\ %cd %' **escape** '\' coincide con todas las cadenas que empiecen por "ab %cd".
- **like** 'ab\\cd %' **escape** '\' coincide con todas las cadenas que empiecen por "ab\cd".

SQL permite buscar discordancias en lugar de concordancias utilizando el operador de comparación **not like**.

www.detodoprogramacion.com

SQL también permite varias funciones que operan sobre cadenas de caracteres, tales como la concatenación (utilizando "||"), la extracción de subcadenas de caracteres, el cálculo de la longitud de las cadenas de caracteres, la conversión a mayúsculas (utilizando **upper**()) y minúsculas (utilizando **lower**()), etc. SQL:1999 también ofrece una operación **similar to**, que proporciona una comparación de patrones más potente que la operación **like**; la sintaxis para especificar patrones es parecida a la usada para las expresiones regulares de Unix.

Existen variaciones en el conjunto concreto de funciones para cadenas de caracteres que soportan los diferentes sistemas de bases de datos. Algunos sistemas de bases de datos no distinguen entre mayúsculas y minúsculas al comparar cadenas de caracteres. Así, 'ABC' **like** 'abc' daría como resultado cierto, igual que 'ABC' = 'abc', en esos sistemas. Otros ofrecen extensiones para especificar que la comparación de cadenas de caracteres debe ignorar si están en mayúsculas o en minúsculas. Conviene leer el manual de cada sistema de bases de datos para conocer más detalles sobre las funciones de cadenas de caracteres que soporta exactamente.

3.3.7 Orden en la presentación de las tuplas

SQL ofrece al usuario cierto control sobre el orden en el cual se presentan las tuplas de una relación. La cláusula **order by** hace que las tuplas del resultado de una consulta se presenten en un cierto orden. Para obtener una relación en orden alfabético de todos los clientes que tienen un préstamo en la sucursal de Navacerrada hay que escribir:

select distinct *nombre_cliente*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo* **and**
nombre_sucursal = 'Navacerrada'
order by *nombre_cliente*

De manera predeterminada, la cláusula **order by** coloca los elementos en orden ascendente. Para especificar el tipo de ordenación se puede especificar **desc** para orden descendente o **asc** para orden ascendente. Además, se puede ordenar con respecto a más de un atributo. Supóngase que se desea colocar toda la relación *préstamo* en orden descendente de *importe*. Si varios préstamos tienen el mismo importe, se ordenan de manera ascendente según sus números de préstamo. Esta consulta en SQL se expresa del modo siguiente:

select *
from *préstamo*
order by *importe* **desc**, *número_préstamo* **asc**

SQL debe llevar a cabo una ordenación para evaluar **order by**. Como ordenar un gran número de tuplas puede resultar costoso, sólo debería hacerse cuando sea estrictamente necesario.

3.3.8 Duplicados

El uso de relaciones con duplicados presenta ventajas en diferentes situaciones. En consecuencia, SQL no sólo define las tuplas que están en el resultado de una consulta, sino también el número de copias de cada una de esas tuplas que aparece en el resultado. La semántica de duplicados de una consulta SQL se define utilizando versiones *multiconjunto* de los operadores relacionales. A continuación se definen las versiones multiconjunto de varios de los operadores del álgebra relacional. Dadas las relaciones multiconjunto r_1 y r_2,

1. Si existen c_1 copias de la tupla t_1 en r_1, y t_1 satisface la selección σ_θ, entonces hay c_1 copias de t_1 en $\sigma_\theta(r_1)$.
2. Para cada copia de la tupla t_1 en r_1, hay una copia de la tupla $\Pi_A(t_1)$ en $\Pi_A(r_1)$, donde $\Pi_A(t_1)$ denota la proyección de la única tupla t_1.

www.detodoprogramacion.com

3. Si existen c_1 copias de la tupla t_1 en r_1 y c_2 copias de la tupla t_2 en r_2, entonces hay $c_1 * c_2$ copias de la tupla $t_1.t_2$ en $r_1 \times r_2$.

Por ejemplo, supóngase que las relaciones r_1 con esquema (A, B) y r_2 con esquema (C) son los multiconjuntos siguientes:

$$r_1 = \{(1, a), (2, a)\} \qquad r_2 = \{(2), (3), (3)\}$$

Entonces, $\Pi_B(r_1)$ sería $\{(a), (a)\}$, mientras que $\Pi_B(r_1) \times r_2$ sería

$$\{(a, 2), (a, 2), (a, 3), (a, 3), (a, 3), (a, 3)\}$$

Ahora se puede definir cuántas copias de cada tupla aparecen en el resultado de una consulta SQL. Una consulta SQL de la forma

select $A_1,\ A_2, \ldots, A_n$
from $r_1,\ r_2, \ldots, r_m$
where P

es equivalente a la expresión del álgebra relacional

$$\Pi_{A_1,\ A_2, \ldots, A_n}(\sigma_P\,(r_1\ \times\ r_2\ \times\ \cdots\ \times\ r_m))$$

que usa las versiones multiconjunto de los operadores relacionales σ, Π y $\times$.

3.4 Operaciones sobre conjuntos

Las operaciones de SQL **union**, **intersect** y **except** operan sobre relaciones y se corresponden con las operaciones del álgebra relacional $\cup$, $\cap$ y $-$. Al igual que la unión, la intersección y la diferencia de conjuntos del álgebra relacional, las relaciones que participan en las operaciones han de ser *compatibles*; esto es, deben tener el mismo conjunto de atributos.

A continuación se estudia el modo de formular en SQL varias de las consultas de ejemplo consideradas en el Capítulo 2. Ahora se crearán consultas que incluyan las operaciones **union**, **intersect** y **except** de dos conjuntos: el conjunto de todos los clientes que tienen cuenta en el banco, que puede obtenerse con

select *nombre_cliente*
from *impositor*

y el conjunto de los clientes que tienen concedido un préstamo por el banco, que puede obtenerse con

select *nombre_cliente*
from *prestatario*

A partir de ahora, se hará referencia a las relaciones obtenidas como resultado de las consultas anteriores como *i* y *p*, respectivamente.

3.4.1 La operación unión

Para determinar todos los clientes del banco que tienen un préstamo, una cuenta o las dos cosas en el banco, hay que escribir:

(**select** *nombre_cliente*
from *impositor*)
union
(**select** *nombre_cliente*
from *prestatario*)

A diferencia de la cláusula **select**, la operación **union** (unión) elimina los valores duplicados automáticamente. Así, en la consulta anterior, si un cliente—por ejemplo, Santos—tiene varias cuentas o préstamos (o ambas cosas) en el banco, sólo aparecerá una vez en el resultado.

www.detodoprogramacion.com

Si se desea conservar todos los duplicados hay que escribir **union all** en lugar de **union**:

(**select** *nombre_cliente*
from *impositor*)
union all
(**select** *nombre_cliente*
from *prestatario*)

El número de tuplas duplicadas en el resultado es igual al número total de valores duplicados que aparecen en *i* y *p*. Así, si Santos tiene tres cuentas y dos préstamos en el banco, en el resultado aparecerán cinco tuplas con el nombre de Santos.

3.4.2 La operación intersección

Para encontrar todos los clientes que tienen tanto un préstamo como una cuenta en el banco, hay que escribir:

(**select distinct** *nombre_cliente*
from *impositor*)
intersect
(**select distinct** *nombre_cliente*
from *prestatario*)

La operacion **intersect** (intersección) elimina los valores duplicados automáticamente. Así, en la consulta anterior, si un cliente—por ejemplo, Santos—tiene varias cuentas o préstamos en el banco, sólo aparecerá una vez en el resultado.

Si se desean conservar todos los valores duplicados hay que escribir **intersect all** en lugar de **intersect**:

(**select** *nombre_cliente*
from *impositor*)
intersect all
(**select** *nombre_cliente*
from *prestatario*)

El número de tuplas duplicadas que aparecen en el resultado es igual al número mínimo de valores duplicados que aparecen en *i* y *p*. Por tanto, si Santos tiene tres cuentas y dos préstamos en el banco, en el resultado de la consulta aparecerán dos tuplas con el nombre de Santos.

3.4.3 La operación excepto

Para determinar todos los clientes que tienen cuenta en el banco pero no tienen ningún préstamo concedido hay que escribir

(**select distinct** *nombre_cliente*
from *impositor*)
except
(**select** *nombre_cliente*
from *prestatario*)

La operación **except** (excepto) elimina los valores duplicados automáticamente. Así, en la consulta anterior, sólo aparecerá en el resultado (exactamente una vez) una tupla con el nombre de Santos si Santos tiene una cuenta en el banco pero no tiene ningún préstamo concedido.

Si se desea conservar todos los valores duplicados hay que escribir **except all** en lugar de **except**:

www.detodoprogramacion.com

(**select** *nombre_cliente*
from *impositor*)
except all
(**select** *nombre_cliente*
from *prestatario*)

El número de copias duplicadas de una tupla en el resultado es igual al número de copias duplicadas de dicha tupla en *impositor* menos el número de copias duplicadas de la misma tupla en *prestatario*, siempre que la diferencia sea positiva. Así, si Santos tiene tres cuentas y un préstamo en el banco, en el resultado aparecerán dos tuplas con el nombre de Santos. Si, por el contrario, ese cliente tiene dos cuentas y tres préstamos en el banco, en el resultado no habrá ninguna tupla con el nombre de Santos.

3.5 Funciones de agregación

Las *funciones de agregación* son funciones que toman una colección (un conjunto o multiconjunto) de valores como entrada y devuelven un solo valor. SQL ofrece cinco funciones de agregación incorporadas:

- Media: **avg**
- Mínimo: **min**
- Máximo: **max**
- Total: **sum**
- Recuento: **count**

Los datos de entrada para **sum** y **avg** deben ser una colección de números, pero los otros operadores también pueden operar sobre colecciones de datos de tipo no numérico como las cadenas de caracteres.

A modo de ejemplo, considérese la consulta "Determinar el saldo medio de las cuentas de la sucursal de Navacerrada". Esta consulta se puede formular del modo siguiente:

select avg (*saldo*)
from *cuenta*
where *nombre_sucursal* = 'Navacerrada'

El resultado de esta consulta es una relación con un único atributo, que contiene una sola tupla con un valor numérico correspondiente al saldo medio de la sucursal de Navacerrada. Opcionalmente se puede dar un nombre al atributo de la relación resultante, utilizando la cláusula **as**.

Existen circunstancias en las cuales sería deseable aplicar las funciones de agregación no sólo a un único conjunto de tuplas sino también a un grupo de conjuntos de tuplas; esto se especifica en SQL mediante la cláusula **group by**. El atributo o atributos especificados en la cláusula **group by** se usan para formar grupos. Las tuplas con el mismo valor en todos los atributos de la cláusula **group by** constituyen un mismo grupo.

Como ejemplo, considérese la consulta "Determinar el saldo medio de las cuentas de cada sucursal". Esta consulta se formula del modo siguiente:

select *nombre_sucursal*, **avg** (*saldo*)
from *cuenta*
group by *nombre_sucursal*

La conservación de los valores duplicados es importante para el cálculo de medias. Supóngase que los saldos de las cuentas en la (pequeña) sucursal de Galapagar son 1.000 €, 3.000 €, 2.000 € y 1.000 €. El saldo medio es 7.000/4 = 1.750,00 €. Si se eliminasen los valores duplicados se obtendría un resultado erróneo (6.000/3 = 2.000 €).

A veces se desea tratar toda la relación como un solo grupo. En estos casos no se utiliza la cláusula **group by**. Considérese la consulta "Determinar el saldo medio de todas las cuentas". Esta consulta se formula del modo siguiente:

www.detodoprogramacion.com

```
select avg (saldo)
from cuenta
```

Con frecuencia se usa la función de agregación **count** para contar el número de tuplas de una relación. La notación de SQL para esta función es **count** (*). Por tanto, para determinar el número de tuplas de la relación *cliente* hay que escribir

```
select count (*)
from cliente
```

Se dan casos en los cuales es necesario eliminar los valores duplicados antes de calcular una función de agregación. Si se desea eliminar los valores duplicados hay que utilizar la palabra clave **distinct** en la expresión de agregación. A modo de ejemplo, considérese la consulta "Determinar el número de impositores de cada sucursal". En este caso cada impositor sólo se debe contar una vez, independientemente del número de cuentas que pueda tener. La consulta se formula del modo siguiente:

```
select nombre_sucursal, count (distinct nombre_cliente)
from impositor, cuenta
where impositor.número_cuenta = cuenta.número_cuenta
group by nombre_sucursal
```

SQL no permite el uso de **distinct** con **count** (*). Sí es legal el uso de **distinct** con **max** y **min**, aunque el resultado no cambia. Se puede utilizar la palabra clave **all** en lugar de **distinct** para especificar la conservación de los valores duplicados pero, como **all** es la opción predeterminada, no hace falta hacerlo.

A veces resulta útil establecer una condición que se aplique a los grupos en vez de a las tuplas. Por ejemplo, puede que sólo estemos interesados en las sucursales en las que el saldo medio de las cuentas sea superior a 1.200 €. Esta condición no se aplica a una sola tupla, sino a cada grupo creado por la cláusula **group by**. Para expresar este tipo de consultas se utiliza la cláusula **having** de SQL. SQL aplica los predicados de la cláusula **having** una vez formados los grupos, de modo que se puedan usar las funciones de agregación. Esta consulta se expresa en SQL del modo siguiente:

```
select nombre_sucursal, avg (saldo)
from cuenta
group by nombre_sucursal
having avg (saldo) > 1200
```

Si en la misma consulta aparecen una cláusula **where** y una cláusula **having**, SQL aplica primero el predicado de la cláusula **where**. Las tuplas que satisfacen el predicado de la cláusula **where** se dividen luego en grupos según la cláusula **group by**. Después se aplica la cláusula **having** (si existe) a cada grupo; SQL elimina los grupos que no satisfacen el predicado de la cláusula **having**. La cláusula **select** utiliza los grupos restantes para generar las tuplas del resultado de la consulta.

Para ilustrar el uso conjunto de la cláusula **where** y la cláusula **having** en la misma consulta, considérese la consulta "Determinar el saldo medio de cada cliente que vive en Peguerinos y tiene, como mínimo, tres cuentas".

```
select impositor.nombre_cliente, avg (saldo)
from impositor, cuenta, cliente
where impositor.número_cuenta = cuenta.número_cuenta and
      impositor.nombre_cliente = cliente.nombre_cliente and
      ciudad_cliente = 'Peguerinos'
group by impositor.nombre_cliente
having count (distinct impositor.número_cuenta) >= 3
```

www.detodoprogramacion.com

3.6 Valores nulos

SQL permite el uso de valores *nulos* para indicar la ausencia de información sobre el valor de un atributo.

Se puede utilizar la palabra clave especial **null** en un predicado para comprobar si un valor es nulo. Así, para determinar todos los números de préstamo que aparecen en la relación *préstamo* con valores nulos para *importe* hay que escribir

select *número_préstamo*
from *préstamo*
where *importe* **is null**

El predicado **is not null** comprueba la ausencia de valores nulos.

El uso de valores *nulos* en las operaciones aritméticas y de comparación causa algunos problemas. En el Apartado 2.5 se vio cómo se manejan los valores nulos en el álgebra relacional. Ahora se va a describir cómo lo hace SQL.

El resultado de las expresiones aritméticas (que incluyan por ejemplo $+$, $-$, $*$ o $/$) es nulo si cualquiera de los valores de entrada es nulo. SQL trata como **unknown** (desconocido) el resultado de cualquier comparación que implique un valor *nulo* (excepto con **is null** y **is not null**).

Dado que el predicado de las cláusulas **where** puede incluir operaciones booleanas como **and**, **or** y **not** sobre los resultados de las comparaciones, se amplían las definiciones de las operaciones booleanas para que manejen el valor **unknown**, como se describe en el Apartado 2.5.

- **and**. El resultado de *cierto* **and** *desconocido* es *desconocido*, *falso* **and** *desconocido* es *falso*, mientras que *desconocido* **and** *desconocido* es *desconocido*.
- **or**. El resultado de *cierto* **or** *desconocido* es *cierto*, *falso* **or** *desconocido* es *desconocido*, mientras que *desconocido* **or** *desconocido* es *desconocido*.
- **not**. El resultado de **not** *desconocido* es *desconocido*.

El resultado de una instrucción SQL de la forma:

$$\textbf{select} \ldots \textbf{from } R_1, \cdots, R_n \textbf{ where } P$$

contiene (proyecciones de) tuplas de $R_1 \times \cdots \times R_n$ para las que el predicado P se evalúa como **cierto**. Si el predicado se evalúa como **falso** o **desconocido** para alguna tupla de $R_1 \times \cdots \times R_n$, (la proyección de) esa tupla no se añade al resultado.

SQL también permite comprobar si el resultado de una comparación es desconocido, en lugar de cierto o falso, mediante las cláusulas **is unknown** (es desconocido) e **is not unknown** (no es desconocido).

La existencia de valores nulos también complica el procesamiento de los operadores de agregación. Por ejemplo, supóngase que algunas tuplas en la relación *préstamo* tienen valor nulo para el atributo *importe*. Considérese la siguiente consulta, que calcula el total de los importes de todos los préstamos:

select sum (*importe*)
from *préstamo*

Entre los valores que se van a sumar en la consulta anterior hay valores nulos, puesto que algunas tuplas tienen valor nulo para *importe*. En lugar de decir que la suma total es *nula*, la norma SQL establece que el operador **sum** debe ignorar los valores nulos de los datos de entrada.

En general, las funciones de agregación tratan los valores nulos según la regla siguiente: todas las funciones de agregación excepto **count (*)** ignoran los valores nulos de la colección de datos de entrada. Como consecuencia de ignorar los valores nulos, la colección de valores de entrada puede estar vacía. El cálculo de **count (*)** de una colección vacía se define como 0 y todas las demás operaciones de agregación devuelven un valor nulo cuando se aplican sobre una colección de datos vacía. El efecto de los valores nulos en algunos de los constructores más complicados de SQL puede ser sutil.

www.detodoprogramacion.com

En SQL:1999 se introdujo un tipo de datos **Boolean**, que puede tomar los valores **true**, **false** y **unknown**. Las funciones de agregación **some** (algún) y **every** (cada), que significan exactamente lo que se espera de ellas, se pueden aplicar a las colecciones de valores booleanos.

3.7 Subconsultas anidadas

SQL proporciona un mecanismo para anidar subconsultas. Las subconsultas son expresiones **select-from-where** que están anidadas dentro de otra consulta. Una finalidad habitual de las subconsultas es llevar a cabo comprobaciones de pertenencia a conjuntos, hacer comparaciones de conjuntos y determinar cardinalidades de conjuntos. Estos usos se estudian en los apartados siguientes.

3.7.1 Pertenencia a conjuntos

SQL permite comprobar la pertenencia de las tuplas a una relación. La conectiva **in** comprueba la pertenencia a un conjunto, donde el conjunto es la colección de valores resultado de una cláusula **select**. La conectiva **not in** comprueba la no pertenencia a un conjunto.

Como ejemplo considérese de nuevo la consulta "Determinar todos los clientes que tienen tanto un préstamo como una cuenta en el banco". Anteriormente se escribió esta consulta como la intersección de dos conjuntos: el conjunto de los impositores del banco y el conjunto de los prestatarios del banco. Se puede adoptar el enfoque alternativo consistente en determinar todos los titulares de cuentas del banco que son miembros del conjunto de prestatarios. Claramente, esta formulación genera el mismo resultado que la anterior, pero obliga a formular la consulta usando la conectiva de SQL **in**. Se comienza determinando todos los titulares de cuentas con:

(**select** *nombre_cliente*
from *impositor*)

A continuación, se determinan los clientes que son prestatarios del banco y que aparecen en la lista de titulares de cuentas obtenida en la subconsulta anterior. Esto se consigue anidando la subconsulta en un **select** más externo. La consulta resultante es:

select distinct *nombre_cliente*
from *prestatario*
where *nombre_cliente* **in** (**select** *nombre_cliente*
from *impositor*)

Este ejemplo muestra que en SQL es posible escribir la misma consulta de diversas formas. Esta flexibilidad es de gran utilidad, puesto que permite a los usuarios pensar en las consultas del modo que les parezca más natural. Más adelante se verá que en SQL hay una gran cantidad de redundancia.

En el ejemplo anterior se comprobaba la pertenencia a un conjunto en una relación con un solo atributo. En SQL también es posible comprobar la pertenencia a un conjunto en una relación cualquiera. Así, es posible formular la consulta "Determinar todos los clientes que tienen tanto una cuenta como un préstamo en la sucursal de Navacerrada" de otra manera distinta:

select distinct *nombre_cliente*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo* **and**
nombre_sucursal = 'Navacerrada' **and**
(*nombre_sucursal, nombre_cliente*) **in**
(**select** *nombre_sucursal, nombre_cliente*
from *impositor, cuenta*
where *impositor.número_cuenta* = *cuenta.número_cuenta*)

El constructor **not in** se utiliza de manera parecida. Por ejemplo, para encontrar todos los clientes que tienen concedido un préstamo en el banco pero no tienen abierta cuenta, se puede escribir:

www.detodoprogramacion.com

```
select distinct nombre_cliente
from prestatario
where nombre_cliente not in (select nombre_cliente
                             from impositor)
```

Los operadores **in** y **not in** también se pueden utilizar sobre conjuntos enumerados. La consulta siguiente selecciona los nombres de los clientes que tienen concedido un préstamo en el banco y cuyos nombres no son ni Santos ni Gómez.

```
select distinct nombre_cliente
from prestatario
where nombre_cliente not in ('Santos', 'Gómez')
```

3.7.2 Comparación de conjuntos

Como ejemplo de la capacidad de comparar conjuntos de las consultas anidadas, considérese la consulta "Determinar el nombre de todas las sucursales que poseen activos mayores que, al menos, una sucursal de Arganzuela". En el Apartado 3.3.5 se formulaba esta consulta del modo siguiente:

```
select distinct T.nombre_sucursal
from sucursal as T, sucursal as S
where T.activos > S.activos and S.ciudad_sucursal = 'Arganzuela'
```

SQL ofrece, sin embargo, un estilo alternativo de formular la consulta anterior. La expresión: "mayor que, al menos, una" se representa en SQL por $>$ **some** Este constructor permite volver a escribir la consulta de forma más parecida a la formulación de la consulta en lenguaje natural.

```
select nombre_sucursal
from sucursal
where activos > some (select activos
                      from sucursal
                      where ciudad_sucursal = 'Arganzuela')
```

La subconsulta

```
(select activos
 from sucursal
 where ciudad_sucursal = 'Arganzuela')
```

genera el conjunto de todos los valores de activos para todas las sucursales situadas en Arganzuela. La comparación $>$ **some** de la cláusula **where** de la cláusula **select** más externa es cierta si el valor del atributo *activos* de la tupla es mayor que, al menos, un miembro del conjunto de todos los valores de activos de las sucursales de Arganzuela.

SQL también permite realizar las comparaciones $<$ **some**, $<=$ **some**, $>=$ **some**, $=$ **some** y $<>$ **some**. Como ejercicio, compruébese que $=$ **some** es idéntico a **in**, mientras que $<>$ **some** *no* es lo mismo que **not in**. En SQL, la palabra clave **any** es sinónimo de **some**. Las primeras versiones de SQL sólo admitían **any**. Versiones posteriores añadieron la alternativa **some** para evitar la ambigüedad lingüística de la palabra inglesa *any* en su idioma original.

Ahora se va a modificar ligeramente la consulta. Se trata de determinar el nombre de todas las sucursales que tienen activos superiores al de todas las sucursales de Arganzuela. El constructor $>$ **all** se corresponde con la expresión "superiores al de todas". Usando este constructor se puede escribir la consulta del modo siguiente:

www.detodoprogramacion.com

```
select nombre_sucursal
from sucursal
where activos > all (select activos
                     from sucursal
                     where ciudad_sucursal = 'Arganzuela')
```

Al igual que con **some**, SQL también permite las comparaciones $<$ **all**, $<=$ **all**, $>=$ **all**, $=$ **all** y $<>$ **all**. Como ejercicio, compruébese que $<>$ **all** es lo mismo que **not in**.

Como ejemplo adicional de comparación de conjuntos, considérese la consulta "Determinar la sucursal que tiene el saldo medio máximo". En SQL las funciones de agregación no se pueden componer. Por tanto, no se puede utilizar **max** (**avg** (...)). En vez de eso, se puede seguir la siguiente estrategia: se comienza por escribir una consulta que determine todos los saldos medios y luego se anida como subconsulta de una consulta mayor que determina las sucursales en las que el saldo medio es mayor o igual que todos los saldos medios:

```
select nombre_sucursal
from cuenta
group by nombre_sucursal
having avg (saldo) >= all (select avg (saldo)
                           from cuenta
                           group by nombre_sucursal)
```

3.7.3 Comprobación de relaciones vacías

SQL incluye la posibilidad de comprobar si las subconsultas tienen alguna tupla en su resultado. El constructor **exists** devuelve el valor **true** si su argumento subconsulta no resulta vacía. Mediante el constructor **exists** se puede formular la consulta "Determinar todos los clientes que tienen tanto una cuenta abierta como un préstamo concedido en el banco" de otra forma más:

```
select nombre_cliente
from prestatario
where exists (select *
              from impositor
              where impositor.nombre_cliente = prestatario.nombre_cliente)
```

Mediante el constructor **not exists** se puede comprobar la inexistencia de tuplas en el resultado de las subconsultas. Se puede utilizar el constructor **not exists** para simular la operación de continencia de conjuntos (es decir, superconjuntos). Se puede escribir "la relación A contiene a la relación B" como "**not exists** (B **except** A)" (aunque no forma parte de las normas SQL-92 ni SQL:1999 el operador **contains** aparecía en algunos de los primeros sistemas relacionales). Para ilustrar el operador **not exists**, considérese otra vez la consulta "Determinar todos los clientes que tienen una cuenta en todas las sucursales de Arganzuela". Hay que comprobar para cada cliente si el conjunto de todas las sucursales en las que dicho cliente tiene cuenta contiene al conjunto de todas las sucursales de Arganzuela. Mediante el constructor **except** se puede formular la consulta del modo siguiente:

```
select distinct S.nombre_cliente
from impositor as S
where not exists ((select nombre_sucursal
                   from sucursal
                   where ciudad_sucursal = 'Arganzuela')
                  except
                   (select R.nombre_sucursal
                    from impositor as T, cuenta as R
                    where T.número_cuenta = R.número_cuenta and
                          S.nombre_cliente = T.nombre_cliente))
```

www.detodoprogramacion.com

En este ejemplo, la subconsulta

```
(select nombre_sucursal
 from sucursal
 where ciudad_sucursal = 'Arganzuela')
```

obtiene todas las sucursales de Arganzuela. La subconsulta

```
(select R.nombre_sucursal
 from impositor as T, cuenta as R
 where T.número_cuenta = R.número_cuenta and
       S.nombre_cliente = T.nombre_cliente)
```

obtiene todas las sucursales en las cuales el cliente *S.nombre_cliente* tiene cuenta. Por tanto, el **select** más externo toma cada cliente y comprueba si el conjunto de todas las sucursales en las que dicho cliente tiene cuenta contiene al conjunto de todas las sucursales de Arganzuela.

En las consultas que contienen subconsultas se aplica una regla de visibilidad para las variables tupla. En las subconsultas, de acuerdo con esta regla, sólo se pueden usar variables tupla definidas en la propia subconsulta o en cualquier consulta que la contenga. Si una variable tupla está definida tanto localmente en una subconsulta como globalmente en una consulta que contiene a la subconsulta, se aplica la definición local. Esta regla es análoga a las reglas de visibilidad utilizadas habitualmente para las variables en los lenguajes de programación.

3.7.4 Comprobación de la ausencia de tuplas duplicadas

SQL incluye la posibilidad de comprobar si las subconsultas tienen tuplas duplicadas en su resultado. El constructor **unique** devuelve el valor **true** si la subconsulta que se le pasa como argumento no contiene tuplas duplicadas. Mediante el constructor **unique** se puede formular la consulta "Determinar todos los clientes que tienen, a lo sumo, una cuenta en la sucursal de Navacerrada" del siguiente modo:

```
select T.nombre_cliente
from impositor as T
where unique (select R.nombre_cliente
              from cuenta, impositor as R
              where T.nombre_cliente = R.nombre_cliente and
                    R.número_cuenta = cuenta.número_cuenta and
                    cuenta.nombre_sucursal = 'Navacerrada')
```

La existencia de tuplas duplicadas en una subconsulta se puede comprobar mediante el constructor **not unique**. Para ilustrar este constructor, considérese la consulta "Determinar todos los clientes que tienen, al menos, dos cuentas en la sucursal de Navacerrada", que se puede formular:

```
select distinct T.nombre_cliente
from impositor T
where not unique (select R.nombre_cliente
                  from cuenta, impositor as R
                  where T.nombre_cliente = R.nombre_cliente and
                        R.número_cuenta = cuenta.número_cuenta and
                        cuenta.nombre_sucursal = 'Navacerrada')
```

Formalmente, la evaluación de **unique** sobre una relación se define como falsa si y sólo si la relación contiene dos tuplas t_1 y t_2 tales que $t_1 = t_2$. Como la comprobación $t_1 = t_2$ es falsa si algún campo de t_1 o de t_2 es nulo, es posible que el resultado de **unique** sea cierto aunque haya varias copias de una misma tupla, siempre que, al menos, uno de los atributos de la tupla sea nulo.

www.detodoprogramacion.com

3.8 Consultas complejas

Las consultas complejas a menudo resultan difíciles o imposibles de escribir como un único bloque de SQL o como unión, intersección o diferencia de bloques de SQL (cada bloque de SQL consta de una sola instrucción **select-from-where**, posiblemente con cláusulas **group by** y **having**). En este apartado se estudian dos formas de componer varios bloques de SQL para expresar consultas complejas: las relaciones derivadas y la cláusula **with**.

3.8.1 Relaciones derivadas

SQL permite el uso de expresiones de subconsulta en las cláusulas **from**. Si se utiliza una expresión de este tipo, hay que proporcionar un nombre a la relación resultado y será posible renombrar los atributos mediante la cláusula **as**. Por ejemplo, considérese la subconsulta:

(**select** *nombre_sucursal*, **avg** (*saldo*)
from *cuenta*
group by *nombre_sucursal*)
as *media_sucursal* (*nombre_sucursal*, *saldo_medio*)

Esta subconsulta genera una relación consistente en los nombres de todas las sucursales y sus correspondientes saldos de cuenta medios. El resultado de la subconsulta recibe el nombre de *media_sucursal*, con los atributos *nombre_sucursal* y *saldo_medio.*

Para ilustrar el uso de una expresión de subconsulta en la cláusula **from**, considérese la consulta "Determinar el saldo medio de las cuentas de las sucursales en las que el saldo medio de las cuentas sea superior a 1.200 €". En el Apartado 3.5 se formulaba esta consulta utilizando la cláusula **having**. Ahora se puede reescribir sin utilizar esa cláusula, de la siguiente forma:

select *nombre_sucursal, avg_saldo*
from (**select** *nombre_sucursal*, **avg** (*saldo*)
from *cuenta*
group by *nombre_sucursal*)
as *media_sucursal* (*nombre_sucursal, saldo_medio*)
where *saldo_medio* $>$ 1200

Obsérvese que no hace falta utilizar la cláusula **having**, ya que la subconsulta de la cláusula **from** calcula el saldo medio y su resultado se denomina *media_sucursal*; los atributos de *media_sucursal* se pueden utilizar directamente en la cláusula **where**.

Como ejemplo adicional supóngase que se desea determinar el saldo total máximo de las sucursales. La cláusula **having** no sirve en este caso, pero esta consulta se puede escribir fácilmente usando una subconsulta en la cláusula **from** como se muestra a continuación:

select max(*saldo_total*)
from (**select** *nombre_sucursal*, **sum**(*saldo*)
from *cuenta*
group by *nombre_sucursal*) **as** *total_sucursal* (*nombre_sucursal*, *saldo_total*)

3.8.2 La cláusula with

Las consultas complicadas son mucho más fáciles de formular y de entender si se estructuran descomponiéndolas en vistas más simples que luego se combinan, igual que se estructuran los programas descomponiendo sus tareas en procedimientos. Sin embargo, a diferencia de la definición de procedimientos, las cláusulas **create view** crean definiciones de vistas en la base de datos y esa definición de vista sigue en la base de datos hasta que se ejecuta una orden **drop view** *nombre_vista*.

La cláusula **with** proporciona una forma de definir vistas temporales cuya definición sólo está disponible para la consulta en la que aparece la cláusula. Considérese la siguiente consulta, que selecciona cuentas con el saldo máximo; si hay muchas cuentas con el mismo saldo máximo, se seleccionan todas.

www.detodoprogramacion.com

```
with saldo_máximo (valor) as
    select max(saldo)
    from cuenta
select número_cuenta
from cuenta, saldo_máximo
where cuenta.saldo = saldo_máximo.valor
```

La cláusula **with**, introducida en SQL:1999 actualmente sólo está soportada en algunas bases de datos.

La consulta anterior se podría haber escrito usando una subconsulta anidada, bien en la cláusula **from**, bien en la **where**. Sin embargo, el uso de subconsultas anidadas hace que la consulta sea más difícil de leer y de entender. La cláusula **with** hace que la lógica de la consulta sea más clara; también permite utilizar definiciones de vistas en varias partes de la consulta.

Por ejemplo, supóngase que se desea determinar todas las sucursales donde el depósito total de las cuentas es mayor que la media de los depósitos totales de las cuentas de todas las sucursales. Se puede escribir la consulta usando la cláusula **with** como se muestra a continuación.

```
with total_sucursales (nombre_sucursal, valor) as
    select nombre_sucursal, sum(saldo)
    from cuenta
    group by nombre_sucursal
with media_total_sucursales(valor) as
    select avg(valor)
    from total_sucursales
select nombre_sucursal
from total_sucursales, media_total_sucursales
where total_sucursales.valor >= media_total_sucursales.valor
```

Por supuesto, se puede crear una consulta equivalente sin la cláusula **with**, pero sería más complicada y difícil de entender. Como ejercicio se puede escribir la consulta equivalente.

3.9 Vistas

Hasta este momento los ejemplos se han limitado a operar en el nivel de los modelos lógicos. Es decir, se ha dado por supuesto que las relaciones facilitadas son las relaciones reales almacenadas en la base de datos.

No resulta deseable que todos los usuarios vean el modelo lógico completo. Las consideraciones de seguridad pueden exigir que se oculten ciertos datos a los usuarios. Considérese una persona que necesita saber el número de préstamo y el nombre de la sucursal de un cliente, pero no necesita ver el importe de ese préstamo. Esa persona debería ver una relación descrita (módulo renombramiento de atributos) en SQL mediante

```
select nombre_cliente, prestatario.número_préstamo, nombre_sucursal
from prestatario, préstamo
where prestatario.número_préstamo = préstamo.número_préstamo
```

Aparte de las consideraciones de seguridad, puede que se desee crear un conjunto personalizado de relaciones que se adapte mejor a la intuición de un usuario determinado que el modelo lógico. Puede que a un usuario del departamento de publicidad, por ejemplo, le guste ver una relación que consista en los clientes que tienen o bien cuenta abierta o bien préstamo concedido en el banco y las sucursales con las que trabajan. La relación que se crearía para ese empleado es la siguiente:

www.detodoprogramacion.com

```
(select nombre_sucursal, nombre_cliente
from impositor, cuenta
where impositor.número_cuenta = cuenta.número_cuenta)
 union
(select nombre_sucursal, nombre_cliente
from prestatario, préstamo
where prestatario.número_préstamo = préstamo.número_préstamo)
```

Las relaciones que no forman parte del modelo lógico pero se hacen visibles a los usuarios como relaciones virtuales se denominan **vistas**. Es posible definir un gran número de vistas de cualquier conjunto dado de relaciones reales.

3.9.1 Definición de vistas

Las vistas en SQL se definen mediante la instrucción **create view**. Para definir una vista hay que darle un nombre e indicar la consulta que la va a calcular. La forma de la instrucción **create view** es

create view v **as** <expresión de consulta>

donde <expresión de consulta> es cualquier expresión legal de consulta. El nombre de la vista se representa mediante v.

A modo de ejemplo, considérese la vista consistente en las sucursales y sus clientes. Supóngase que se desea que esta vista se denomine *todos_los_clientes*. Esta vista se define de la manera siguiente:

```
create view todos_los_clientes as
    (select nombre_sucursal, nombre_cliente
     from impositor, cuenta
     where impositor.número_cuenta = cuenta.número_cuenta)
   union
    (select nombre_sucursal, nombre_cliente
     from prestatario, préstamo
     where prestatario.número_préstamo = préstamo.número_préstamo)
```

Una vez definida la vista se puede utilizar su nombre para hacer referencia a la relación virtual que genera. Utilizando la vista *todos_los_clientes* se puede determinar el nombre de todos los clientes de la sucursal de Navacerrada escribiendo

```
select nombre_cliente
from todos_los_clientes
where nombre_sucursal = 'Navacerrada'
```

Los nombres de las vistas pueden aparecer en cualquier lugar en el que puedan hacerlo los nombres de las relaciones, siempre y cuando no se ejecuten sobre las vistas operaciones de actualización. El asunto de las operaciones de actualización de las vistas se estudia en el Apartado 3.10.4.

Los nombres de los atributos de las vistas pueden especificarse de manera explícita de la manera siguiente:

```
create view total_préstamos_sucursal(nombre_sucursal, total_préstamos) as
select nombre_sucursal, sum(importe)
from préstamo
group by nombre_sucursal
```

La vista anterior da la suma del importe de todos los créditos de cada sucursal. Dado que la expresión **sum**(*importe*) no tiene nombre, el nombre del atributo se especifica de manera explícita en la definición de la vista.

www.detodoprogramacion.com

De manera intuitiva, el conjunto de tuplas de la relación de vistas es el resultado de la evaluación de la expresión de consulta que define en ese momento la vista. Por tanto, si las relaciones de vistas se calculan y se guardan, pueden quedar desfasadas si las relaciones usadas para definirlas se modifican. Para evitarlo, las vistas suelen implementarse de la manera siguiente. Cuando se define una vista, el sistema de la base de datos guarda la definición de la vista, en vez del resultado de la evaluación de la expresión del álgebra relacional que la define. Siempre que aparece una relación de vistas en una consulta, se sustituye por la expresión de consulta almacenada. Por tanto, la relación de vistas se vuelve a calcular siempre que se evalúa la consulta.

Algunos sistemas de bases de datos permiten que se guarden las relaciones de vistas, pero se aseguran de que, si las relaciones reales utilizadas en la definición de la vista cambian, la vista se mantenga actualizada. Estas vistas se denominan **vistas materializadas**. El proceso de mantener actualizada la vista se denomina **mantenimiento de vistas**, y se trata en el Apartado 14.5. Las aplicaciones en las que se utiliza frecuentemente una misma vista aprovechan las vistas materializadas, al igual que las aplicaciones que exigen una respuesta rápida a ciertas consultas basadas en las vistas. Por supuesto, las ventajas para las consultas derivadas de la materialización de las vistas deben sopesarse frente a los costes de almacenamiento y la sobrecarga añadida de las actualizaciones.

3.9.2 Vistas definidas en función de otras

En el Apartado 3.9.1 se mencionó que las relaciones de vistas pueden aparecer en cualquier lugar en que puedan hacerlo los nombres de las relaciones, salvo las restricciones al uso de las vistas en expresiones de actualización. Por tanto, se pueden utilizar vistas en las expresiones que definen otras vistas. Por ejemplo, se puede definir la vista *cliente_navacerrada* de la manera siguiente:

create view *cliente_navacerrada* **as**
select *nombre_cliente*
from *todos_los_clientes*
where *nombre_sucursal* $=$ 'Navacerrada'

donde *todos_los_clientes* es, a su vez, una relación de vistas.

La **expansión de vistas** es una manera de definir el significado de las vistas definidas en términos de otras. El procedimiento asume que las definiciones de vistas no son **recursivas**; es decir, no se utiliza ninguna vista en su propia definición, ni directa ni indirectamente mediante otras definiciones de vistas. Por ejemplo, si $v1$ se utiliza en la definición de $v2$, $v2$ se utiliza en la definición de $v3$ y $v3$ se usa en la definición de $v1$, entonces $v1$, $v2$ y $v3$ son recursivas. Las definiciones recursivas de las vistas resultan útiles en algunos casos, y se volverá a ellas en el contexto del lenguaje Datalog, en el Apartado 5.4.

Supóngase una vista v_1 definida mediante una expresión e_1 que puede contener a su vez relaciones de vistas. Las relaciones de vistas representan a las expresiones que definen las vistas y, por tanto, se pueden sustituir por las expresiones que las definen. Si se modifica una expresión sustituyendo una relación de vistas por su definición, la expresión resultante puede seguir conteniendo otras relaciones de vistas. Por tanto, la expansión de vistas de una expresión repite la etapa de sustitución de la manera siguiente:

repeat
  Buscar todas las relaciones de vistas v_i de e_1
  Sustituir la relación de vistas v_i por la expresión que define v_i
until no queden más relaciones de vistas en e_1

Mientras las definiciones de las vistas no sean recursivas, el bucle concluirá. Por tanto, una expresión e que contenga relaciones de vistas puede entenderse como la expresión resultante de la expansión de vistas de e, que no contiene ninguna relación de vistas.

Como ilustración de la expansión de vistas considérese la expresión siguiente:

select *
from *cliente_navacerrada*
where *nombre_cliente* $=$ 'Martín'

www.detodoprogramacion.com

El procedimiento de expansión de vistas produce inicialmente

select *
from (**select** *nombre_cliente*
from *todos_los_clientes*
where *nombre_sucursal* = 'Navacerrada')
where *nombre_cliente* = 'Martín'

luego produce



select *
from (**select** *nombre_cliente*
from ((**select** *nombre_sucursal*, *nombre_cliente*
from *impositor*, *cuenta*
where *impositor.número_cuenta* = *cuenta.número_cuenta*)
union
(**select** *nombre_sucursal*, *nombre_cliente*
from *prestatario*, *préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo*))
where *nombre_sucursal* = 'Navacerrada')
where *nombre_cliente* = 'Martín'

En este momento no hay más usos de las relaciones de vistas y concluye la expansión de vistas.

3.10 Modificación de la base de datos

Hasta ahora se ha estudiado la extracción de información de las bases de datos. A continuación se mostrará cómo añadir, eliminar y modificar información utilizando SQL.

3.10.1 Borrado

Las solicitudes de borrado se expresan casi igual que las consultas. Sólo se pueden borrar tuplas completas; no se pueden borrar sólo valores de atributos concretos. SQL expresa los borrados mediante:

delete from r
where P

donde P representa un predicado y r representa una relación. La declaración **delete** busca primero todas las tuplas t en r para las que $P(t)$ es cierto y a continuación las borra de r. La cláusula **where** se puede omitir, en cuyo caso se borran todas las tuplas de r.

Obsérvese que cada comando **delete** sólo opera sobre una relación. Si se desea borrar tuplas de varias relaciones hay que utilizar una orden **delete** por cada relación. El predicado de la cláusula **where** puede ser tan complicado como la cláusula **where** de cualquier orden **select**. En el otro extremo, la cláusula **where** puede estar vacía. La consulta

delete from *préstamo*

borra todas las tuplas de la relación *préstamo* (los sistemas bien diseñados exigen una confirmación del usuario antes de ejecutar una petición tan devastadora).

A continuación se muestra una serie de ejemplos de peticiones de borrado en SQL:

- Borrar todas las tuplas de cuentas de la sucursal de Navacerrada.

delete from *cuenta*
where *nombre_sucursal* = 'Navacerrada'

- Borrar todos los préstamos con importe comprendido entre 1.300 € y 1.500 €.

```
delete from préstamo
where importe between 1300 and 1500
```

- Borrar todas las tuplas de cuenta de todas las sucursales de Arganzuela.

```
delete from cuenta
where nombre_sucursal in (select nombre_sucursal
                          from sucursal
                          where ciudad_sucursal = 'Arganzuela')
```

Esta solicitud de borrado busca primero todas las sucursales de Arganzuela y luego borra todas las tuplas *cuenta* correspondientes a esas sucursales.

Obsérvese que, si bien sólo se pueden borrar tuplas de una única relación a la vez, se puede hacer referencia a cualquier número de relaciones en una expresión **select-from-where** anidada en la cláusula **where** de una orden **delete**. La petición **delete** puede contener un **select** anidado que haga referencia a la relación cuyas tuplas se van a borrar. Por ejemplo, supóngase que se desea borrar los registros de todas las cuentas con saldos inferiores a la media del banco. Se puede escribir:

```
delete from cuenta
where saldo < (select avg (saldo)
               from cuenta)
```

El comando **delete** comprueba primero cada tupla de la relación *cuenta* para ver si la cuenta tiene un saldo inferior a la media del banco. Luego se borran todas las tuplas que cumplen la condición; es decir, representan una cuenta con un saldo inferior a la media. Es importante realizar todas las comprobaciones antes de llevar a cabo ningún borrado; si se borra alguna tupla antes de comprobar las demás, el saldo medio puede cambiar ¡y el resultado final del borrado dependería del orden en que se procesaran las tuplas!

3.10.2 Inserción

Para insertar datos en una relación, se especifica la tupla que se desea insertar o se formula una consulta cuyo resultado sea el conjunto de tuplas que se desea insertar. Obviamente, los valores de los atributos de las tuplas que se inserten deben pertenecer al dominio de los atributos. De igual modo, las tuplas insertadas deben ser de la aridad correcta.

La instrucción **insert** más sencilla es una solicitud de inserción de una tupla. Supóngase que se desea insertar el hecho de que hay una cuenta C-9732 en la sucursal de Navacerrada y que tiene un saldo de 1.200 €. Hay que escribir

```
insert into cuenta
    values ('C-9732', 'Navacerrada', 1200)
```

En este ejemplo los valores se especifican en el mismo orden en que aparecen los atributos correspondientes en el esquema de la relación. Para beneficio de los usuarios, que puede que no recuerden el orden de los atributos, SQL permite que los atributos se especifiquen en la cláusula **insert**. Por ejemplo, las siguientes instrucciones **insert** tienen una función idéntica a la anterior:

```
insert into cuenta (número_cuenta, nombre_sucursal, saldo)
    values ('C-9732', 'Navacerrada', 1200)

insert into cuenta (nombre_sucursal, número_cuenta, saldo)
    values ('Navacerrada', 'C-9732', 1200)
```

De forma más general es posible que se desee insertar las tuplas que resultan de una consulta. Supóngase que se les desea ofrecer, como regalo, a todos los clientes titulares de préstamos de la sucursal de Navacerrada una cuenta de ahorro con 200 € por cada préstamo que tengan concedido. Así, se escribe:

www.detodoprogramacion.com

```
insert into cuenta
    select número_préstamo, nombre_sucursal, 200
    from préstamo
    where nombre_sucursal = 'Navacerrada'
```

En lugar de especificar una tupla, como se hizo en los primeros ejemplos de este apartado, se utiliza una instrucción **select** para especificar un conjunto de tuplas. SQL evalúa en primer lugar la instrucción **select**, lo que produce un conjunto de tuplas que se inserta a continuación en la relación *cuenta*. Cada tupla tiene un un *número_préstamo* (que sirve como número de cuenta para la nueva cuenta), un *nombre_sucursal* (Navacerrada) y un saldo inicial de la cuenta (200 €).

También hay que añadir tuplas a la relación *impositor*; para ello hay que escribir

```
insert into impositor
    select nombre_cliente, número_préstamo
    from prestatario, préstamo
    where prestatario.número_préstamo = préstamo.número_préstamo and
        nombre_sucursal = 'Navacerrada'
```

Esta consulta inserta en la relación *impositor* una tupla (*nombre_cliente, número_préstamo*) por cada *nombre_cliente* que tenga concedido un préstamo en la sucursal de Navacerrada, con número de préstamo *número_préstamo*.

Es importante que la evaluación de la instrucción **select** finalice completamente antes de llevar a cabo ninguna inserción. Si se realizase alguna inserción mientras se evalúa la instrucción **select**, una petición del tipo

```
insert into cuenta
    select *
    from cuenta
```

¡podría insertar un número infinito de tuplas! La solicitud insertaría la primera tupla de nuevo en *cuenta*, creando así una segunda copia de la tupla. Como esta segunda copia ya forma parte de *cuenta*, la instrucción **select** puede encontrarla, y se insertaría una tercera copia en *cuenta*. La instrucción **select** puede entonces encontrar esta tercera copia e insertar una cuarta copia, y así indefinidamente. Al evaluar por completo la instrucción **select** antes de realizar ninguna inserción se evita este tipo de problemas.

La discusión de la instrucción **insert** sólo ha considerado ejemplos en los que se especificaba un valor para cada atributo de las tuplas insertadas. Es posible, como se vio en el Capítulo 2, dar valores únicamente a algunos de los atributos del esquema para las tuplas insertadas. A los atributos restantes se les asigna un valor nulo, que se denota por *null*. Considérese la petición

```
insert into cuenta
    values ('C-401', null, 1200)
```

Se sabe que la cuenta C-401 tiene un saldo de 1.200 €, pero no se conoce el nombre de la sucursal. Considérese la consulta

```
select número_cuenta
from cuenta
where nombre_sucursal = 'Navacerrada'
```

Como el nombre de la sucursal en que se ha abierto la cuenta C-401 es desconocido, no se puede determinar si es igual a "Navacerrada".

Se puede prohibir la inserción de valores nulos en atributos concretos utilizando el LDD de SQL, que se estudia en el Apartado 3.2.

La mayor parte de los productos de bases de datos tienen utilidades especiales de "carga masiva" para insertar en las relaciones grandes conjuntos de tuplas. Estas utilidades permiten leer datos de archivos de texto con formato y pueden ejecutarse mucho más rápido que la secuencia equivalente de instrucciones **insert**.

www.detodoprogramacion.com

3.10.3 Actualizaciones

En determinadas situaciones puede ser deseable modificar un valor dentro de una tupla sin cambiar *todos* los valores de la misma. Para este tipo de situaciones se puede utilizar la instrucción **update**. Al igual que ocurre con **insert** y **delete**, se pueden elegir las tuplas que se van a actualizar mediante una consulta.

Supóngase que se va a realizar el pago anual de intereses y que hay que incrementar todos los saldos en un 5 por ciento. Hay que escribir

update *cuenta*
set *saldo* = *saldo* * 1.05

Esta instrucción de actualización se aplica una vez a cada tupla de la relación *cuenta*.

Si sólo se paga el interés a las cuentas con un saldo de 1.000 € o superior, se puede escribir

update *cuenta*
set *saldo* = *saldo* * 1.05
where *saldo* >= 1000

En general, la cláusula **where** de la instrucción **update** puede contener cualquier constructor permitido en la cláusula **where** de la instrucción **select** (incluyendo instrucciones **select** anidadas). Al igual que ocurre con **insert** y con **delete**, un **select** anidado en una instrucción **update** puede hacer referencia a la relación que se esté actualizando. Al igual que antes, SQL primero comprueba todas las tuplas de la relación para determinar si se deben actualizar y después realiza la actualización. Por ejemplo, se puede escribir la solicitud "Pagar un interés del 5 por ciento a las cuentas cuyo saldo sea mayor que la media" como sigue:

update *cuenta*
set *saldo* = *saldo* * 1.05
where *saldo* > (**select avg** (*saldo*)
from *cuenta*)

Supóngase que las cuentas con saldos superiores a 10.000 € reciben un 6 por ciento de interés, mientras que las demás reciben un 5 por ciento. Se pueden escribir dos instrucciones de actualización:

update *cuenta*
set *saldo* = *saldo* * 1.06
where *saldo* > 10000

update *cuenta*
set *saldo* = *saldo* * 1.05
where *saldo* <= 10000

Obsérvese que, como se vio en el Capítulo 2, el orden de las dos instrucciones **update** es importante. Si se cambiara el orden de las dos instrucciones, una cuenta con un saldo igual o muy poco inferior a 10.000 € recibiría un 11,3 por ciento de interés.

SQL ofrece un constructor **case** que se puede utilizar para llevar a cabo las dos instrucciones de actualización anteriores en una única instrucción **update**, evitando el problema del orden de actualización.

update *cuenta*
set *saldo* = **case**
when *saldo* <= 10000 **then** *saldo* * 1.05
else *saldo* * 1.06
end

La forma general de la instrucción **case** es la siguiente:

www.detodoprogramacion.com

case
  when $pred_1$ **then** $result_1$
  when $pred_2$ **then** $result_2$
  ...
  when $pred_n$ **then** $result_n$
  else $result_0$
end

La operación devuelve $result_i$, donde i es el primero de $pred_1, pred_2, \ldots, pred_n$ que se satisface; si ninguno de ellos se satisface, la operación devuelve $result_0$. Las instrucciones **case** se pueden usar en cualquier lugar donde se espere un valor.

3.10.4 Actualización de vistas

Aunque las vistas resultan una herramienta útil para las consultas, presentan serios problemas si se expresa con ellas actualizaciones, inserciones o borrados. La dificultad radica en que las modificaciones de la base de datos expresadas en términos de vistas deben traducirse en modificaciones de las relaciones reales del modelo lógico de la base de datos.

Para ilustrar el problema, considérese un empleado que necesita ver todos los datos de préstamos de la relación *préstamo*, excepto *importe_préstamo*. Sea *sucursal_préstamo* la vista ofrecida al empleado. Esta vista se define como

create view *sucursal_préstamo* **as**
select *número_préstamo, nombre_sucursal*
from *préstamo*

Como se permite que el nombre de la vista aparezca en cualquier lugar en el que pueda aparecer el nombre de una relación, el empleado puede escribir

insert into *sucursal_préstamo*
  values ('P-37', 'Navacerrada')

Esta inserción debe representarse mediante una inserción en la relación *préstamo*, puesto que *préstamo* es la relación real a partir de la cual el sistema de bases de datos construye la vista *sucursal_préstamo*. Sin embargo, para insertar una tupla en *préstamo* hay que tener algún valor para *importe*. Hay dos enfoques razonables para tratar esta inserción:

- Rechazar la inserción y devolver al usuario un mensaje de error.
- Insertar la tupla (P-37, "Navacerrada", *null*) en la relación *préstamo*.

Otro problema con la modificación de la base de datos mediante vistas surge con vistas como

create view *info_préstamo* **as**
select *nombre_cliente, importe*
from *prestatario, préstamo*
where *prestatario.número_préstamo* = *préstamo.número_préstamo*

Esta vista muestra el importe del préstamo para cada préstamo que tenga concedido cualquier cliente del banco. Considérese la siguiente inserción mediante esta vista:

insert into *info_préstamo*
  values ('González', 1900)

El único método posible de inserción de tuplas en las relaciones *prestatario* y *préstamo* es insertar ("González", *null*) en *prestatario* y (*null*, *null*, 1900) en *préstamo*. Así se obtendrían las relaciones que pueden verse en la Figura 3.3. No obstante, esta actualización no tiene el efecto deseado, ya que la relación

www.detodoprogramacion.com

número_préstamo	*nombre_sucursal*	*importe*
P-11	Collado Mediano	900
P-14	Centro	1.500
P-15	Navacerrada	1.500
P-16	Navacerrada	1.300
P-17	Centro	1.000
P-23	Moralzarzal	2.000
P-93	Becerril	500
nulo	*nulo*	1.900

préstamo

nombre_cliente	*número_préstamo*
Fernández	P-16
Gómez	P-11
Gómez	P-23
López	P-15
Pérez	P-93
Santos	P-17
Sotoca	P-14
Valdivieso	P-17
González	*nulo*

prestatario

Figura 3.3 Las tuplas insertadas en *préstamo* y *prestatario*.

de vistas *info_préstamo* sigue *sin* incluir la tupla ("González", 1900). Por tanto, no hay manera de actualizar las relaciones *prestatario* y *préstamo* mediante el uso de valores nulos para obtener la actualización deseada de *info_préstamo*.

Debido a problemas como éstos no se suelen permitir las actualizaciones de las relaciones de vistas, salvo en casos concretos. Los diferentes sistemas de bases de datos especifican condiciones diferentes para permitir las actualizaciones de las relaciones de vistas y es preciso consultar el manual de cada sistema para conocer los detalles. El problema general de la modificación de las bases de datos mediante las vistas ha sido objeto de amplia investigación, y las notas bibliográficas ofrecen indicaciones de parte de esa investigación.

En general se dice que una vista de SQL es **actualizable** (es decir, se le pueden aplicar inserciones, actualizaciones y borrados) si se cumplen todas las condiciones siguientes:

- La cláusula **from** sólo tiene una relación de base de datos.
- La cláusula **select** sólo contiene nombres de atributos de la relación y no tiene ninguna expresión, valor agregado ni especificación **distinct**.
- Cualquier atributo que no aparezca en la cláusula **select** puede definirse como nulo.
- La consulta no tiene cláusulas **group** ni **having**.

Con estas restricciones, las operaciones **update**, **insert** y **delete** estarían prohibidas en la vista de ejemplo *todos_los_clientes* que se había definido anteriormente.

Supóngase que se define la vista *cuenta_centro* de la manera siguiente:

create view *cuenta_centro* **as**
select *número_cuenta*, *nombre_sucursal*, *saldo*
from *cuenta*
where *nombre_sucursal* = 'Centro'

Esta vista es actualizable, ya que cumple las condiciones que se han fijado anteriormente.

Pese a las condiciones para la actualización, sigue existiendo el problema siguiente. Supóngase que un usuario intenta insertar la tupla ('C-999', 'Navacerrada', 1000) en la vista *cuenta_centro*. Esta tupla puede insertarse en la relación *cuenta*, pero no aparecerá en la vista *cuenta_centro*, ya que no cumple la selección impuesta por la vista.

De manera predeterminada, SQL permitiría que la actualización se llevara a cabo. Sin embargo, pueden definirse vistas con una cláusula **with check option** al final de la definición de la vista; por tanto, si una tupla insertada en la vista no cumple la condición de la cláusula **where** de la vista, el sistema de bases de datos rechaza la inserción. De manera parecida se rechazan las actualizaciones si los valores nuevos no cumplen las condiciones de la cláusula **where**.

www.detodoprogramacion.com

SQL:1999 tiene un conjunto de reglas más complejo en cuanto a la posibilidad de ejecutar inserciones, actualizaciones y borrados sobre las vistas, el cual permite las actualizaciones en una clase más amplia de vistas; sin embargo, estas reglas son demasiado complejas para estudiarlas aquí.

3.10.5 Transacciones

Una **transacción** consiste en una secuencia de instrucciones de consulta o de actualización. La norma SQL especifica que una transacción comienza implícitamente cuando se ejecuta una instrucción SQL. Una de las siguientes instrucciones SQL debe finalizar la transacción:

- **Commit work** compromete la transacción actual; es decir, hace que las actualizaciones realizadas por la transacción pasen a ser permanentes en la base de datos. Una vez comprometida la transacción, se inicia de manera automática una nueva transacción.
- **Rollback work** provoca el retroceso de la transacción actual; es decir, deshace todas las actualizaciones realizadas por las instrucciones SQL de la transacción. Por tanto, el estado de la base de datos se restaura al que tenía antes de la ejecución de la primera instrucción de la transacción.

La palabra clave **work** es opcional en ambas instrucciones.

El retroceso de transacciones resulta útil si se detecta alguna condición de error durante la ejecución de la transacción. El compromiso es parecido, en cierto sentido, a guardar los cambios de un documento que se esté editando, mientras que el retroceso es como abandonar la sesión de edición sin guardar los cambios. Una vez que la transacción ha ejecutado **commit work**, sus efectos ya no se pueden deshacer con **rollback work**. El sistema de bases de datos garantiza en caso de fallo (como puede ser un error en una de las instrucciones SQL, una caída de tensión o una caída del sistema) provocar el retroceso de los efectos de la transacción si todavía no se había ejecutado **commit work**. En caso de caída de tensión o caída del sistema, el retroceso ocurre cuando el sistema se reinicia.

Por ejemplo, para transferir dinero de una cuenta a otra hay que actualizar los saldos de dos cuentas. Las dos instrucciones de actualización forman una transacción. Un error mientras una transacción ejecuta alguna de las instrucciones tiene como consecuencia que se deshagan los efectos de las instrucciones anteriores de esa transacción, por lo que la base de datos no se queda en un estado parcialmente actualizado. En el Capítulo 15 se estudian más propiedades de las transacciones.

Si un programa termina sin ejecutar ninguno de estos comandos, las actualizaciones se comprometen o se provoca su retroceso. La norma no especifica cuál de los dos cosas tiene lugar, con lo que la elección depende de la implementación. En muchas implementaciones de SQL cada instrucción de SQL se considera de manera predeterminada como una transacción en sí misma, y se compromete en cuanto se ejecuta. El compromiso automático de cada instrucción de SQL se puede desactivar si hay que ejecutar una transacción que consta de varias instrucciones SQL. La forma de desactivación del compromiso automático depende de cada implementación SQL concreta.

Una alternativa mejor, que forma parte de la norma SQL:1999 (aunque actualmente sólo está soportada por algunas implementaciones de SQL) es permitir que se encierren varias instrucciones SQL entre las palabras clave **begin atomic** ... **end**. Todas las instrucciones entre esas palabras clave forman así una única transacción.

3.11 Reunión de relaciones**

SQL no sólo proporciona el mecanismo básico del producto cartesiano para reunir las tuplas de las relaciones, también ofrece (en SQL-92 y versiones posteriores) otros mecanismos para reunir relaciones, como las reuniones condicionales y las reuniones naturales, así como diversas formas de reuniones externas. Estas operaciones adicionales suelen usarse como expresiones de subconsulta en la cláusula **from**.

www.detodoprogramacion.com

número_préstamo	*nombre_sucursal*	*importe*
P-170	Centro	3.000
P-230	Moralzarzal	4.000
P-260	Navacerrada	1.700

préstamo

nombre_cliente	*número_préstamo*
Santos	P-170
Gómez	P-230
López	P-155

prestatario

Figura 3.4 Las relaciones *préstamo* y *prestatario*.

3.11.1 Ejemplos

Se van a ilustrar las diversas operaciones de reunión mediante las relaciones *préstamo* y *prestatario* en la Figura 3.4. Se comenzará con un ejemplo sencillo de reunión interna. La Figura 3.5 muestra el resultado de la expresión

préstamo **inner join** *prestatario* **on** $préstamo.número_préstamo = prestatario.número_préstamo$

La expresión calcula la reunión zeta de las relaciones *préstamo* y *prestatario* con la condición de reunión *préstamo.número_préstamo* = *prestatario.número_préstamo*. Los atributos del resultado son los atributos de la relación del lado izquierdo seguidos por los de la relación del lado derecho.

Obsérvese que el atributo *número_préstamo* aparece dos veces en la figura—la primera aparición se debe a la relación *préstamo* y la segunda a *prestatario*. La norma SQL no exige que los nombres de atributo en resultados como éstos sean únicos. Se debería usar una cláusula **as** para asignar nombres únicos a los atributos de los resultados de las consultas y subconsultas.

La relación resultado de la reunión y sus atributos se renombra usando una cláusula **as**, como se ilustra a continuación:

préstamo **inner join** *prestatario* **on** *préstamo.número_préstamo* = *prestatario.número_préstamo*
as *pp(número_préstamo, sucursal, importe, cliente, número_préstamo_cliente)*

La segunda aparición de *número_préstamo* se ha renombrado como *número_préstamo_cliente*. El orden de los atributos en el resultado de la reunión es importante a la hora de renombrarlos.

A continuación se toma en consideración un ejemplo de la operación reunión externa por la izquierda (**left outer-join**):

préstamo **left outer join** *prestatario* **on** *préstamo.número_préstamo* = *prestatario.número_préstamo*

Es posible calcular la operación reunión externa por la izquierda del modo siguiente. Primero se calcula el resultado de la reunión interna como antes. Luego, para cada tupla t de la relación del lado izquierdo *préstamo* que no coincide con ninguna tupla de la relación del lado derecho *prestatario* en la reunión interior se añade al resultado de la reunión una tupla r: los atributos de la tupla r que se obtienen a partir de la relación del lado izquierdo se rellenan con los valores de la tupla t y el resto de los atributos de r se rellena con valores nulos. La Figura 3.6 muestra la relación resultante. Las tuplas (P-170, Centro, 3.000) y (P-230, Moralzarzal, 4.000) se reúnen con las tuplas de *prestatario* y aparecen en el resultado de la reunión interna y, por tanto, en el resultado de la reunión externa por la izquierda. Por otra parte, la tupla (P-260, Navacerrada, 1.700) no coincide con ninguna tupla de *prestatario* en la reunión interna y, por eso, en el resultado de la reunión externa por la izquierda aparece la tupla (P-260, Navacerrada, 1.700, nulo, nulo).

número_préstamo	*nombre_sucursal*	*importe*	*nombre_cliente*	*número_préstamo*
P-170	Centro	3.000	Santos	P-170
P-230	Moralzarzal	4.000	Gómez	P-230

Figura 3.5 Resultado de *préstamo* **inner join** *prestatario* **on** $préstamo.número_préstamo = prestatario.número_préstamo$.

www.detodoprogramacion.com

número_préstamo	*nombre_sucursal*	*importe*	*nombre_cliente*	*número_préstamo*
P-170	Centro	3.000	Santos	P-170
P-230	Moralzarzal	4.000	Gómez	P-230
P-260	Navacerrada	1.700	*nulo*	*nulo*

Figura 3.6 Resultado de *préstamo* **left outer join** *prestatario* **on** $préstamo.número_préstamo = prestatario.número_préstamo$.

Finalmente, se considera un ejemplo de la operación reunión natural (**natural join**).

préstamo **natural inner join** *prestatario*

Esta expresión calcula la reunión natural de las dos relaciones. El único nombre de atributo común a *préstamo* y a *prestatario* es *número_préstamo*. La Figura 3.7 muestra el resultado de la expresión. Este resultado es parecido al de la reunión interna con la condición **on** mostrada en la Figura 3.5 ya que, de hecho, tienen la misma condición de reunión. Sin embargo, el atributo *número_préstamo* sólo aparece una vez en el resultado de la reunión natural, mientras que aparece dos veces en el de la reunión con la condición **on**.

3.11.2 Tipos y condiciones de reunión

En el Apartado 3.11.1 se vieron ejemplos de las operaciones de reunión permitidas en SQL. Las operaciones de reunión toman dos relaciones y devuelven como resultado otra relación. Aunque las expresiones de reunión externa se usan normalmente en la cláusula **from**, se pueden utilizar en cualquier lugar en el que se pueda utilizar una relación.

Cada una de las variantes de las operaciones de reunión en SQL consiste en un *tipo de reunión* y una *condición de reunión*. La condición de reunión define las tuplas de las dos relaciones a casar y los atributos que se incluyen en el resultado de la reunión. El tipo de reunión define la manera de tratar las tuplas de cada relación que no casan con ninguna tupla de la otra relación (de acuerdo con la condición de reunión). La Figura 3.8 muestra algunos de los tipos y condiciones de reunión permitidos. El primer tipo de reunión es la reunión interna y los otros tres son reuniones externas. De las tres condiciones de reunión ya se han visto la reunión **natural** y la condición **on**, y se estudiará la condición **using** más adelante en este apartado.

El uso de condiciones de reunión es obligatorio en las reuniones externas, pero es opcional en las internas (si se omite, se obtiene un producto cartesiano). Sintácticamente, la palabra clave **natural** aparece delante del tipo de reunión, como se mostró anteriormente, mientras que las condiciones **on** y **using** aparecen al final de la expresión de reunión. Las palabras clave **inner** y **outer** son opcionales, ya que su forma permite deducir si la reunión es interna o externa.

número_préstamo	*nombre_sucursal*	*importe*	*nombre_cliente*
P-170	Centro	3.000	Santos
P-230	Moralzarzal	4.000	Gómez

Figura 3.7 Resultado de *préstamo* **natural inner join** *prestatario*.

Tipos de reunión	*Condiciones de reunión*
inner join	**natural**
left outer join	**on** <predicado>
right outer join	**using** $(A_1, A_2, \ldots, A_n)$
full outer join	

Figura 3.8 Tipos y condiciones de reunión.

www.detodoprogramacion.com

número_préstamo	*nombre_sucursal*	*importe*	*nombre_cliente*
P-170	Centro	3.000	Santos
P-230	Moralzarzal	4.000	Gómez
P-155	*nulo*	*nulo*	López

Figura 3.9 Resultado de *préstamo* **natural right outer join** *prestatario*.

El significado de la condición de reunión **natural**, en cuanto a las tuplas de las relaciones a casar, es claro. La ordenación de los atributos, dentro del resultado de la reunión natural es el siguiente. Los atributos de reunión (es decir, los atributos comunes a las dos relaciones) aparecen en primer lugar, en el orden en el que aparecen en la relación del lado izquierdo. A continuación se encuentran los demás atributos no comunes de la relación del lado izquierdo y, al final, todos los atributos no comunes de la relación del lado derecho.

La reunión externa por la derecha (**right outer join**) es simétrica a la reunión externa por la izquierda (**left outer join**). Las tuplas de la relación del lado derecho que no casan con ninguna tupla de la relación del lado izquierdo se rellenan con valores nulos y se añaden al resultado de la reunión externa por la derecha.

La siguiente expresión es un ejemplo de combinación de la condición de reunión natural con el tipo de reunión externa por la derecha:

préstamo **natural right outer join** *prestatario*

La Figura 3.9 muestra el resultado de esta expresión. Los atributos del resultado vienen definidos por el tipo de reunión, que es una reunión natural; por tanto, *número_préstamo* sólo aparece una vez. Las dos primeras tuplas del resultado provienen de la reunión natural interna de *préstamo* y *prestatario*. La tupla de la relación del lado derecho (López, P-155) no casa con ninguna tupla de la relación del lado izquierdo *préstamo* en la reunión interna. Así, la tupla (P-155, nulo, nulo, López) aparece en el resultado de la reunión.

La condición de reunión **using**$(A_1, A_2, \ldots, A_n)$ es parecida a la condición de reunión natural, salvo en que los atributos de reunión son los atributos $A_1, A_2, \ldots, A_n$, en lugar de todos los atributos comunes a ambas relaciones. Los atributos $A_1, A_2, \ldots, A_n$ sólo deben ser los comunes a ambas relaciones y sólo aparecen una vez en el resultado de la unión.

La reunión externa completa (**full outer join**) es una combinación de los tipos de reunión externa por la derecha y por la izquierda. Una vez que la operación ha calculado el resultado de la reunión interna, extiende con valores nulos las tuplas de la relación del lado izquierdo que no casan con ninguna tupla de la relación del lado derecho y las añade al resultado. De manera análoga, extiende con valores nulos las tuplas de la relación del lado derecho que no casan con ninguna tupla de la relación del lado izquierdo y las añade al resultado.

Por ejemplo, la Figura 3.10 muestra el resultado de la expresión

préstamo **full outer join** *prestatario* **using** (número_préstamo)

Como ejemplo adicional del uso de la operación reunión externa se puede escribir la consulta: "Determinar todos los clientes que tienen una cuenta abierta pero no tienen ningún préstamo concedido en el banco" como

número_préstamo	*nombre_sucursal*	*importe*	*nombre_cliente*
P-170	Centro	3.000	Santos
P-230	Moralzarzal	4.000	Gómez
P-260	Navacerrada	1.700	*nulo*
P-155	*nulo*	*nulo*	López

Figura 3.10 Resultado de *préstamo* **full outer join** *prestatario* **using** (*número_préstamo*).

www.detodoprogramacion.com

```
select i_NC
from (impositor left outer join prestatario
      on impositor.nombre_cliente = prestatario.nombre_cliente)
      as db1 (i_NC, número_cuenta, p_NC, número_préstamo)
where p_NC is null
```

De forma análoga se puede escribir la consulta "Determinar todos los clientes que tienen una cuenta abierta o un préstamo concedido en el banco (pero no las dos cosas)" con el operador de reunión natural externa completa como:

```
select nombre_cliente
from (impositor natural full outer join prestatario)
where número_cuenta is null or número_préstamo is null
```

SQL-92 proporciona también otros dos tipos de reunión, denominados **cross join** (reunión cruzada) y **union join** (reunión de unión). El primero es equivalente a una reunión interna sin condición de reunión; el segundo es equivalente a una reunión externa completa con condición "false", es decir, donde la reunión interna está vacía.

3.12 Resumen

- Los sistemas comerciales de bases de datos no utilizan la concisa y formal álgebra relacional que se trató en Capítulo 2. El ampliamente usado lenguaje SQL, que se ha estudiado en este capítulo, está basado en el álgebra relacional, pero incluye mucho "azúcar sintáctico".
- El lenguaje de definición de datos de SQL se utiliza para crear relaciones con esquemas especificados. El LDD de SQL soporta diferentes tipos de datos, como **date** y **time**. Se pueden ver más detalles sobre el LDD de SQL, en especial sobre el soporte de las restricciones de integridad, en el Apartado. 3.2.
- SQL incluye varios constructores del lenguaje de consultas a la base de datos. Todas las operaciones del álgebra relacional, incluidas las operaciones del álgebra relacional extendida, se pueden expresar en SQL. SQL también permite la ordenación de los resultados de las consultas según los atributos especificados.
- SQL trata las consultas sobre relaciones que contienen valores nulos añadiendo el valor lógico "desconocido" a los valores lógicos habituales de cierto y falso.
- SQL permite subconsultas anidadas en las cláusulas **where**. La consulta más externa puede llevar a cabo gran variedad de operaciones sobre el resultado de la subconsulta, como la comprobación de relaciones vacías o de pertenencia de valores al resultado de una subconsulta. Las subconsultas de la cláusula **from** se denominan relaciones derivadas.
- Las relaciones se definen como relaciones que contienen el resultado de consultas. Las vistas resultan útiles para ocultar información innecesaria y para recopilar información de más de una relación en una única vista.
- Las vistas temporales definidas con la cláusula **with** también resultan útiles para descomponer consultas complejas en partes más pequeñas y más fáciles de comprender.
- SQL proporciona constructores para actualizar, insertar y borrar información. Las actualizaciones mediante vistas sólo se permiten cuando se cumplen algunas condiciones bastante restrictivas.
- Las transacciones son secuencias de consultas y de actualizaciones que, en su conjunto, desempeñan una tarea. Las transacciones se pueden comprometer o retroceder; cuando se hace retroceder una transacción, se deshacen los efectos de todas las actualizaciones llevadas a cabo por esa transacción.
- SQL soporta varios tipos de reunión externa con diferentes tipos de condiciones de reunión.

www.detodoprogramacion.com

CAPÍTULO 4

SQL avanzado

En el Capítulo 3 se ofreció un tratamiento detallado de la estructura básica de SQL. El lenguaje SQL ha evolucionado a partir de finales de los años setenta desde un lenguaje con pocas funcionalidades a un lenguaje complejo con características para satisfacer a muchos tipos diferentes de usuarios. En este capítulo se tratarán algunas de las características avanzadas de SQL. Se seguirá empleando en los ejemplos el esquema bancario, que se reproduce por comodidad en la Figura 4.1.

4.1 Tipos de datos y esquemas

Ya hemos visto que debe haber asociado un tipo, es decir, un dominio de valores posibles, con cada atributo. En el Capítulo 3 se vieron varios tipos de datos predefinidos soportados por SQL, como los tipos enteros, los reales y los de carácter. Hay otros tipos de datos predefinidos soportados por SQL, que se describirán a continuación. También se describirá la manera de crear en SQL tipos sencillos definidos por los usuarios.

4.1.1 Tipos de datos predefinidos

Además de los tipos de datos básicos que se presentaron en el Apartado 3.2, la norma SQL soporta otros tipos de datos predefinidos, como:

- **date**. Una fecha de calendario que contiene el año (de cuatro cifras), el mes y el día del mes.
- **time**. La hora del día, en horas, minutos y segundos. Se puede usar una variante, **time**(p), para especificar el número de cifras decimales para los segundos (el valor predeterminado es 0). También es posible almacenar la información del huso horario junto a la hora especificando **time with timezone**.
- **timestamp**. Una combinación de **date** y **time**. Se puede usar una variante, **timestamp**(p), para especificar el número de cifras decimales para los segundos (el valor predeterminado es seis). También se almacena información sobre el huso horario si se especifica **with timezone**.

sucursal (*nombre_sucursal*, *ciudad_sucursal*, *activos*)
cliente (*nombre_cliente*, *calle_cliente*, *ciudad_cliente*)
préstamo (*número_préstamo*, *nombre_sucursal*, *importe*)
prestatario (*nombre_cliente*, *número_préstamo*)
cuenta (*número_cuenta*, *nombre_sucursal*, *saldo*)
impositor (*nombre_cliente*, *número_cuenta*)

Figura 4.1 Esquema de la entidad bancaria.

www.detodoprogramacion.com

Los valores de fecha y hora se pueden especificar de esta manera:

date '2001-04-25'
time '09:30:00'
timestamp '2001-04-25 10:29:01.45'

La fecha se debe especificar en el formato de año seguido del mes y del día, tal y como se muestra. El campo segundos de **time** y **timestamp** puede tener parte decimal, como puede verse en el ejemplo anterior.

Se pueden usar expresiones de la forma **cast** *e* **as** *t* para convertir una cadena de caracteres (o una expresión de tipo cadena de caracteres) *e* al tipo *t*, donde *t* es de tipo **date, time** o **timestamp**. La cadena de caracteres debe tener el formato adecuado, como se indicó al comienzo de este párrafo. Si es necesaria, la información sobre el huso horario puede deducirse de la configuración del sistema.

Para extraer campos concretos de un valor *d* de **date** o de **time** se puede utilizar **extract** (*campo* **from** *d*), donde *campo* puede ser **year, month, day, hour, minute** o **second**. La información sobre el huso horario puede obtenerse mediante **timezone_hour** y **timezone_minute**.

SQL también define varias funciones útiles para obtener la fecha y la hora actuales. Por ejemplo, **current_date** devuelve la fecha actual, **current_time** devuelve la hora actual (con su huso horario) y **localtime** devuelve la hora local actual (sin huso horario). Las marcas de tiempo (fecha y hora) se obtienen con **current_timestamp** (con huso horario) y **localtimestamp** (fecha y hora locales sin huso horario).

SQL permite realizar operaciones de comparación sobre todos los tipos de datos que se han mencionado aquí, así como operaciones aritméticas y de comparación sobre los diferentes tipos de datos numéricos. SQL también proporciona un tipo de datos denominado **interval** y permite realizar cálculos basados en fechas, horas e intervalos. Por ejemplo, si x e y son del tipo **date**, entonces $x - y$ es un intervalo cuyo valor es el número de días desde la fecha x hasta la y. De forma análoga, al sumar o restar un intervalo a una fecha o a una hora se obtiene como resultado otra fecha u hora, respectivamente.

A menudo resulta útil comparar valores de diferentes tipos de datos **compatibles**. Por ejemplo, supóngase que el tipo de datos de *nombre_cliente* es una cadena de caracteres de longitud 20 y el tipo de datos de *nombre_sucursal* es una cadena de caracteres de longitud 15. Aunque la longitud de las cadenas sea diferente, la norma SQL considera que los dos tipos de datos son compatibles. Como ejemplo adicional, como cada entero perteneciente al tipo **smallint** es también un entero, las comparaciones $x < y$, donde x es de tipo **smallint** e y es de tipo **int** (o viceversa), son válidas. Este tipo de comparación se lleva a cabo transformando primero el número x al tipo **int**. Las transformaciones de este tipo se denominan **coerción**. La **coerción de tipos** se emplea de manera habitual en los lenguajes de programación comunes, así como en los sistemas de bases de datos.

4.1.2 Tipos definidos por los usuarios

SQL soporta dos formas de tipos de datos definidos por los usuarios. La primera forma, que se tratará a continuación, se denomina **alias de tipos** (**distinct types**). La otra forma, denominada **tipos de datos estructurados**, permite la creación de tipos de datos complejos, que pueden anidar estructuras de registro, arrays y multiconjuntos. En este capítulo no se tratan los tipos de datos complejos, pero se describen más adelante, en el Capítulo 9.

Varios atributos pueden ser del mismo tipo de datos. Por ejemplo, los atributos *nombre_cliente* y *nombre_empleado* pueden tener el mismo dominio: el conjunto de todos los nombres propios. No obstante, los dominios de *saldo* y *nombre_sucursal*, ciertamente, deben ser diferentes. Quizás resulte menos evidente si *nombre_cliente* y *nombre_sucursal* deben tener el mismo dominio. En el nivel de implementación tanto los nombres de los clientes como los de las sucursales son cadenas de caracteres. Sin embargo, normalmente no se considera que la consulta "Determinar todos los clientes que tengan el mismo nombre que alguna sucursal" sea una consulta con sentido. Por tanto, si se considera la base de datos desde el nivel conceptual, en vez de hacerlo desde el nivel físico, *nombre_cliente* y *nombre_sucursal* deben tener dominios distintos.

A nivel práctico resulta más grave asignar el nombre de un cliente a una sucursal; de manera parecida, comparar directamente un valor monetario expresado en euros con otro valor monetario expresado en libras también es, seguramente, un error de programación. Un buen sistema de tipos de datos debe

www.detodoprogramacion.com

poder detectar este tipo de asignaciones o comparaciones. Para soportar estas comprobaciones, SQL aporta el concepto de **alias de tipos**.

La cláusula **create type** puede utilizarse para definir tipos de datos nuevos. Por ejemplo, las instrucciones:

create type *Euros* **as numeric**(12,2) **final**
create type *Libras* **as numeric**(12,2) **final**

declaran los tipos de datos definidos por los usuarios *Euros* y *Libras* como números decimales con un total de doce cifras, dos de las cuales se hallan tras la coma decimal. (La palabra clave **final** no resulta realmente significativa en este contexto, pero la norma de SQL:1999 la exige por motivos que no vienen al caso; algunas implementaciones permiten omitir la palabra clave **final**). Los tipos recién creados pueden utilizarse, por ejemplo, como tipos de los atributos de las relaciones. Por ejemplo, se puede declarar la tabla *cuenta* como:

create table *cuenta*
(*número_cuenta* **char**(10),
nombre_sucursal **char**(15),
saldo Euros)

Cualquier intento de asignar un valor de tipo *Euros* a una variable de tipo *Libras* daría como resultado un error de compilación, aunque ambos sean del mismo tipo numérico. Una asignación de ese tipo probablemente se deba a un error de programación, en el que el programador haya olvidado las diferencias de divisas. La declaración de tipos distintos para divisas diferentes ayuda a detectar esos errores.

Como consecuencia del control riguroso de los tipos de datos, la expresión (*saldo.cuenta* + 20) no se aceptaría, ya que el atributo y la constante entera 20 tienen diferentes tipos de datos. Los valores de un tipo de datos pueden *convertirse* (*cast*) a otro dominio como se muestra a continuación:

cast (*saldo.cuenta* **to** *numeric*(12,2))

Se podría realizar la suma sobre el tipo numeric, pero para volver a guardar el resultado en un atributo del tipo *Euros* habría que emplear otra expresión cast para volver a convertir el tipo de datos a *Euros*.

SQL también ofrece las cláusulas **drop type** y **alter type** para eliminar o modificar los tipos de datos que se han creado anteriormente.

Antes incluso de la incorporación a SQL de los tipos de datos definidos por los usuarios (en SQL:1999), SQL tenía el concepto parecido, pero sutilmente diferente, de **tipo de dominio** (**domain type**) (introducido en SQL-92). Se podría definir el tipo de dominio *EEuros* de la manera siguiente:

create domain *EEuros* **as numeric**(12,2)

El tipo de dominio *EEuros* puede utilizarse como tipo de atributo, igual que se ha utilizado el tipo de datos *Euros*. Sin embargo, hay dos diferencias significativas entre los tipos de datos y los dominios:

1. Sobre los dominios se pueden especificar restricciones, como **not null**, y valores predeterminados para las variables del tipo de dominio, mientras que no se pueden especificar restricciones ni valores predeterminados sobre los tipos de datos definidos por los usuarios. Los tipos de datos definidos por los usuarios están diseñados no sólo para utilizarlos para especificar los tipos de datos de los atributos, sino también en extensiones procedimentales de SQL en las que puede que no sea posible aplicar restricciones. Más adelante se volverá al problema de las restricciones sobre los dominios, en el Apartado 4.2.4.

2. Los dominios no tienen tipos estrictos. En consecuencia, los valores de un tipo de dominio pueden asignarse a los de otro tipo de dominio, siempre y cuando los tipos subyacentes sean compatibles.

www.detodoprogramacion.com

4.1.3 Tipos de datos para objetos grandes

Muchas aplicaciones de bases de datos de última generación necesitan almacenar atributos que pueden ser de gran tamaño (del orden de muchos kilobytes), como pueden ser fotografías de personas, o muy grande (del orden de muchos megabytes o, incluso, gigabytes), como las imágenes médicas de alta resolución o fragmentos de vídeo. SQL, por tanto, ofrece nuevos tipos de datos para objetos de gran tamaño para los datos de caracteres (**clob**) y para los datos binarios (**blob**). Las letras "lob" de estos tipos de datos significan "objeto grande" (Large Object). Por ejemplo, se pueden declarar los atributos

revista_literaria **clob**(10KB)
imagen **blob**(10MB)
película **blob**(2GB)

La ejecución de consultas SQL suele recuperar una o más filas del resultado en la memoria. Los objetos grandes suelen utilizarse en aplicaciones externas, y para objetos de tamaño muy grande (de varios megabytes a gigabytes), resulta poco eficiente o poco práctico recuperar en la memoria objetos de gran tamaño de tipo entero. En vez de eso, las aplicaciones suelen utilizar las consultas SQL para recuperar "localizadores" de los objetos de gran tamaño y luego emplean el localizador para manipular el objeto desde el lenguaje anfitrión. Por ejemplo, la interfaz de programación de aplicaciones JDBC (que se describe en el Apartado 4.5.2) permite recuperar un localizador en lugar de todo el objeto de gran tamaño; luego se puede emplear el localizador para recuperar este objeto en fragmentos más pequeños, en vez de recuperarlo todo de golpe, de manera parecida a como se leen los datos de los archivos del sistema operativo mediante llamadas a funciones de lectura.

4.1.4 Esquemas, catálogos y entornos

Para comprender la razón de que existan esquemas y catálogos, considérese la manera en que se dan nombres a los archivos en los sistemas de archivos. Los primeros sistemas de archivos eran "planos", es decir, todos los archivos se almacenaban en un único directorio. Los sistemas de archivos de última generación, evidentemente, tienen una estructura de directorios y los archivos se almacenan en diferentes subdirectorios. Para dar un nombre único a un archivo hay que especificar su nombre completo, por ejemplo, /usuarios/avi/libro-bd/capítulo4.tex.

Al igual que los primeros sistemas de archivos, los primeros sistemas de bases de datos tenían también un único espacio de nombres para todas las relaciones. Los usuarios tenían que coordinarse para asegurarse de que no intentaban emplear el mismo nombre para relaciones diferentes. Los sistemas de bases de datos contemporáneos ofrecen una jerarquía de tres niveles para los nombres de las relaciones. El nivel superior de la jerarquía consta de **catálogos**, cada uno de los cuales puede contener **esquemas**. Los objetos de SQL, como las relaciones y las vistas, están contenidos en **esquemas**. (Algunas implementaciones de las bases de datos emplean el término "base de datos" en lugar de catálogo).

Para llevar a cabo cualquier acción sobre la base de datos los usuarios (o los programas) primero deben *conectarse* a ella. El usuario debe proporcionar el nombre de usuario y, generalmente, una contraseña secreta para que se compruebe su identidad. Cada usuario tiene un catálogo y un esquema predeterminados, y esa combinación es única para cada usuario. Cuando un usuario se conecta a un sistema de bases de datos, se configuran para la conexión el catálogo y el esquema predeterminados; esto equivale a la conversión del directorio actual en el directorio de inicio del usuario cuando éste inicia sesión en un sistema operativo.

Para identificar de manera única cada relación hay que utilizar un nombre con tres partes. Por ejemplo,

catálogo5.esquema_bancario.cuenta

Se puede omitir el componente del catálogo, en cuyo caso la parte del nombre correspondiente al catálogo se considera que es el catálogo predeterminado de la conexión. Por tanto, si catálogo5 es el catálogo predeterminado, se puede utilizar esquema_bancario.cuenta para identificar de manera única la misma relación. Además, también se puede omitir el nombre del esquema y la parte correspondiente al esquema del nombre vuelve a considerarse el esquema predeterminado de la conexión. Así pues, se puede

www.detodoprogramacion.com

emplear únicamente cuenta si el catálogo predeterminado es catálogo5 y el esquema predeterminado es esquema_bancario.

Cuando se dispone de varios catálogos y de varios esquemas, diferentes aplicaciones y usuarios pueden trabajar de manera independiente sin preocuparse de posibles coincidencias de nombres. Además, pueden ejecutarse varias versiones de la misma aplicación—una versión de producción, otras versiones de prueba—sobre el mismo sistema de bases de datos.

El catálogo y el esquema predeterminados forman parte de un **entorno SQL** que se configura para cada conexión. El entorno contiene, además, el identificador del usuario (también denominado *identificador de autorización*). Todas las instrucciones habituales de SQL, incluidas las instrucciones de LDD y de LMD, operan en el contexto de un esquema. Se pueden crear y descartar esquemas mediante las instrucciones **create schema** y **drop schema**. La creación y la eliminación de catálogos depende de cada implementación y no forma parte de la norma de SQL.

4.2 Restricciones de integridad

Las restricciones de integridad garantizan que las modificaciones realizadas en la base de datos por los usuarios autorizados no den lugar a una pérdida de la consistencia de los datos. Por tanto, las restricciones de integridad protegen contra daños accidentales a las bases de datos.

Algunos ejemplos de restricciones de integridad son:

- El saldo de las cuentas no puede ser nulo.
- No puede haber dos cuentas con el mismo número.
- Todos los números de cuenta de la relación *impositor* deben tener el número de cuenta correspondiente en la relación *cuenta*.
- El salario por hora de los empleados del banco debe ser, como mínimo, de 6,00 € la hora.

En general, las restricciones de integridad pueden ser predicados arbitrarios que hagan referencia a la base de datos. Sin embargo, puede resultar costosa la comprobación de estos predicados arbitrarios. Por tanto, la mayor parte de los sistemas de bases de datos permiten especificar restricciones de integridad que puedan probarse con una sobrecarga mínima. En el Capítulo 7 se estudia otra forma de restricción de integridad, denominada **dependencia funcional**, que se utiliza sobre todo en el proceso de diseño de esquemas.

4.2.1 Restricciones sobre una sola relación

En el Apartado 3.2 se describió la manera de definir tablas mediante el comando **create table**. Este comando también puede incluir instrucciones para restricciones de integridad. Además de la restricción "de clave primaria", hay varias más que pueden incluirse en el comando **create table**. Entre las restricciones de integridad permitidas se encuentran:

- **not null**
- **unique**
- **check**(<predicado>)

Cada uno de estos tipos de restricciones se trata en los apartados siguientes.

4.2.2 Restricción not null

Como se estudió en el Capítulo 2, el valor nulo (*null*) es miembro de todos los dominios y, en consecuencia, de manera predeterminada es un valor legal para todos los atributos de SQL. Para determinados atributos, sin embargo, los valores nulos pueden resultar poco adecuados. Considérese una tupla de la relación *cuenta* en la que *número_cuenta* sea nulo. Este tipo de tupla proporciona información de las cuentas desconocidas; por tanto, no contiene información útil. De manera parecida, no es deseable que

www.detodoprogramacion.com

el saldo de la cuenta sea un valor nulo. En estos casos así se deseará prohibir los valores nulos, lo que se puede hacer restringiendo el dominio de los atributos *número_cuenta* y *saldo* para excluir los valores nulos, declarándolos de la manera siguiente:

número_cuenta **char**(10) **not null**
saldo **numeric**(12,2) **not null**

La especificación **not null** prohíbe la inserción de valores nulos para ese atributo. Cualquier modificación de la base de datos que haga que se inserte un valor nulo en un atributo declarado como **not null** genera un diagnóstico de error.

Existen muchas situaciones en las que se desea evitar los valores nulos. En concreto, SQL prohíbe los valores nulos en la clave primaria de los esquemas de las relaciones. Por tanto, en el ejemplo bancario, en la relación *cuenta*, si el atributo *número_cuenta* se declara clave primaria de *cuenta*, no puede adoptar valores nulos. En consecuencia, no hace falta declararlo **not null** de manera explícita.

La especificación **not null** también puede aplicarse a declaraciones de dominio definidas por los usuarios; en consecuencia, no se permitirá a los atributos de ese tipo de dominio tomar valores nulos. Por ejemplo, si se desea que el dominio *Euros* no tome valores nulos, se puede declarar de la manera siguiente:

create domain *Euros* **numeric**(12,2) **not null**

4.2.3 Restricción unique

SQL también soporta la restricción de integridad

unique $(A_{j_1}, A_{j_2}, \ldots, A_{j_m})$

La especificación **unique** indica que los atributos $A_{j_1}, A_{j_2}, \ldots, A_{j_m}$ forman una clave candidata; es decir, ningún par de tuplas de la relación puede ser igual en todos los atributos de la clave primaria. Sin embargo, se permite que los atributos de la clave candidata tengan valores nulos, a menos que se hayan declarado de manera explícita como **not null**. Recuérdese que los valores nulos no son iguales a ningún otro valor. (El tratamiento de los valores nulos en este caso es el mismo que el del constructor **unique** definido en el Apartado 3.7.4).

4.2.4 La cláusula check

La cláusula **check** de SQL puede aplicarse a declaraciones de relaciones y a declaraciones de dominios. Cuando se aplica a declaraciones de relaciones, la cláusula **check**(P) especifica un predicado P que deben cumplir todas las tuplas de la relación.

Un uso frecuente de la cláusula **check** es garantizar que los valores de los atributos cumplan las condiciones especificadas, lo que permite en realidad construir un potente sistema de tipos. Por ejemplo, la cláusula **check**(*activos*>=0) en el comando **create table** de la relación *sucursal* garantiza que el valor de **activos** no sea negativo.

Considérese ahora el siguiente ejemplo:

```
create table estudiante
   (nombre            char(15)  not null,
    id_estudiante     char(10),
    tipo_titulación   char(15),
    primary key (id_estudiante),
    check (tipo_titulación in ('Diplomatura', 'Licenciatura', 'Doctorado')))
```

En este caso se utiliza la cláusula **check** para simular un tipo enumerado, especificando que *tipo_titulación* debe ser 'Diplomatura', 'Licenciatura' o 'Doctorado'.

www.detodoprogramacion.com

Cuando se aplica a un dominio, la cláusula **check** permite a los diseñadores de esquemas especificar un predicado que debe cumplir cualquier valor asignado a las variables cuyo tipo de datos sea el dominio.

Por ejemplo, una cláusula **check** puede garantizar que el dominio del salario por hora sólo permita valores mayores que un mínimo especificado (como puede ser el salario mínimo):

create domain *SalarioPorHora* **numeric**(5,2)
constraint *prueba_valor_salario* **check**(**value** $>= 6.00$)

Se ha impuesto sobre el dominio *SalarioPorHora* una restricción que garantiza que el salario por hora sea mayor o igual que 6.00. La cláusula **constraint** *prueba_valor_salario* es optativa, y se emplea para dar el nombre *prueba_valor_salario* a la restricción. El nombre lo utiliza el sistema para indicar la restricción que ha violado alguna actualización.

Como ejemplo adicional, se puede restringir un dominio para que sólo contenga un conjunto determinado de valores mediante la cláusula **in**:

create domain *TipoCuenta* **char**(10)
constraint *prueba_tipo_cuenta*
check (**value in** ('Corriente', 'Ahorro'))

Por tanto, la cláusula **check** permite restringir los atributos y los dominios de una forma potente que la mayor parte de los sistemas del tipos de los lenguajes de programación no permiten.

Las anteriores condiciones de **check** pueden comprobarse con relativa facilidad, al insertar o modificar una tupla. Sin embargo, en general, las condiciones de **check** pueden ser más complejas (y más difíciles de comprobar), ya que se permiten subconsultas que hagan referencias a otras relaciones en la condición **check**. Por ejemplo, se podría especificar esta restricción sobre la relación *cuenta*:

check (*nombre_sucursal* **in** (**select** *nombre_sucursal* **from** *sucursal*))

La condición de **check** comprueba que el *nombre_sucursal* de cada tupla de la relación *cuenta* es realmente el nombre de una sucursal de la relación *sucursal*. Por tanto, no sólo hay que comprobar la condición cuando se inserta o modifica una tupla de *cuenta*, sino también cuando se modifica la relación *sucursal* (en ese caso, cuando se borra o modifica alguna tupla de la relación *sucursal*).

La restricción anterior es, en realidad, un ejemplo de una clase de constantes denominadas restricciones de *integridad referencial*. Este tipo de restricciones, junto con una manera más sencilla de especificarlas en SQL, se estudian en el Apartado 4.2.5.

Las condiciones **check** complejas pueden ser útiles cuando se desea garantizar la integridad de los datos, pero deben emplearse con precaución, ya que pueden resultar costosas de comprobar.

4.2.5 Integridad referencial

A menudo se desea garantizar que el valor que aparece en una relación para un conjunto dado de atributos aparezca también para un conjunto determinado de atributos en otra relación. Esta condición se denomina **integridad referencial**.

Las claves externas pueden especificarse como parte de la instrucción de SQL **create table** mediante la cláusula **foreign key**. Las declaraciones de claves externas se ilustran mediante la definición en el LDD de SQL de parte de la base de datos bancaria, como puede verse en la Figura 4.2. La declaración de la tabla *cuenta* tiente una declaración "**foreign key** (*nombre_sucursal*) **references** *sucursal*". Esta declaración de clave externa especifica que para cada tupla de cuenta, el nombre de sucursal especificado en la tupla debe existir en la relación *sucursal*. Sin esta restricción, es posible que alguna cuenta especifique el nombre de una sucursal inexistente.

De manera más general, sean $r_1(R_1)$ y $r_2(R_2)$ relaciones con las claves primarias C_1 y C_2, respectivamente (recuérdese que R_1 y R_2 denotan el conjunto de atributos de r_1 y de r_2, respectivamente). Se dice que un subconjunto α de R_2 es una **clave externa** que hace referencia a C_1 de la relación r_1 si se exige que, para cada tupla t_2 de r_2, deba haber una tupla t_1 de r_1 tal que $t_1[C_1] = t_2[\alpha]$. Las exigencias de este tipo se denominan **restricciones de integridad referencial**, o **dependencias de subconjuntos**.

www.detodoprogramacion.com

```
create table cliente
    (nombre_cliente     char(20),
     calle_cliente      char(30),
     ciudad_cliente     char(30),
     primary key (nombre_cliente))

create table sucursal
    (nombre_sucursal  char(15),
     ciudad_sucursal  char(30),
     activos          numeric(16,2),
     primary key (nombre_sucursal),
     check (activos >= 0))

create table cuenta
    (número_cuenta    char(10),
     nombre_sucursal  char(15),
     saldo            numeric(12,2),
     primary key (número_cuenta),
     foreign key (nombre_sucursal) references sucursal,
     check (saldo >= 0))

create table impositor
    (nombre_cliente   char(20),
     número_cuenta    char(10),
     primary key (nombre_cliente, número_cuenta),
     foreign key (nombre_cliente) references cliente,
     foreign key (número_cuenta) references cuenta)
```

Figura 4.2 Definición de datos con SQL para parte de la base de datos bancaria.

El último término se debe a que la anterior restricción de integrida referencial puede escribirse como $\Pi_\alpha\ (r_2)\ \subseteq\ \Pi_{C_1}\ (r_1)$. Obsérvese que, para que las restricciones de integridad referencial tengan sentido, α y C_1 deben ser conjuntos de atributos compatibles, es decir, o bien α debe ser igual a C_1, o bien deben contener el mismo número de atributos y los tipos de los atributos correspondientes deben ser compatibles (aquí se supone que α y C_1 están ordenados).

De manera predeterminada, en SQL las claves externas hacen referencia a los atributos de la clave primaria de la tabla referenciada. SQL también soporta una versión de la cláusula **references** en la que se puede especificar de manera explícita una lista de atributos de la relación a la que se hace referencia. La lista de atributos especificada, no obstante, debe declararse como clave candidata de la relación a la que hace referencia.

Se puede utilizar la siguiente forma abreviada como parte de la definición de atributos para declarar que el atributo forma una clave externa:

nombre_sucursal **char**(15) **references** *sucursal*

Cuando se viola una restricción de integridad referencial, el procedimiento normal es rechazar la acción que ha causado esa violación (es decir, la transacción que lleva a cabo la acción de actualización se retrocede). Sin embargo, la cláusula **foreign key** puede especificar que si una acción de borrado o de actualización de la relación a la que hace referencia viola la restricción, entonces, en lugar de rechazar la acción, el sistema realice los pasos necesarios para modificar la tupla de la relación que hace la referencia para que se restaure la restricción. Considérese esta definición de una restricción de integridad para la relación *cuenta*:

www.detodoprogramacion.com

```
create table cuenta
    ( . . .
    foreign key (nombre_sucursal) references sucursal
                                  on delete cascade
                                  on update cascade,
    . . . )
```

Debido a la cláusula **on delete cascade** asociada con la declaración de la clave externa, si el borrado de una tupla de *sucursal* da lugar a que se viole la restricción de integridad referencial, el sistema no rechaza el borrado. En vez de eso, el borrado "pasa en cascada" a la relación *cuenta*, borrando la tupla que hace referencia a la sucursal que se ha borrado. De manera parecida, el sistema no rechaza las actualizaciones de los campos a los que hace referencia la restricción aunque la violen; en vez de eso, el sistema actualiza también al nuevo valor el campo *nombre_sucursal* de las tuplas de *cuenta* que hacen la referencia. SQL también permite que la cláusula **foreign key** especifique acciones diferentes de **cascade**, si se viola la restricción. El campo que hace la referencia (en este caso, *nombre_sucursal*) puede definirse como nulo (empleando **set null** en lugar de **cascade**) o con el valor predeterminado para el dominio (utilizando **set default**).

Si hay una cadena de dependencias de clave externa que afecta a varias relaciones, el borrado o la actualización de un extremo de la cadena puede propagarse por toda ella. En el Ejercicio práctico 4.4 aparece un caso interesante en el que la restricción de **clave externa** de una relación hace referencia a la misma relación. Si una actualización o un borrado en cascada provoca una violación de la restricción que no pueda tratarse con otra operación en cascada, el sistema aborta la transacción. En consecuencia, todas las modificaciones provocadas por la transacción y sus acciones en cascada se deshacen.

Los valores **nulos** complican la semántica de las restricciones de integridad referencial en SQL. Se permite que los atributos de las claves externas sean valores nulos, siempre que no hayan sido declarados no nulos previamente. Si todas las columnas de una clave externa son no nulas en una tupla dada, se utiliza para esa tupla la definición habitual de las restricciones de clave externa. Si alguna de las columnas de la clave externa es nula, la tupla se define de manera automática para que satisfaga la restricción.

Puede que esta definición no sea siempre la opción correcta, por lo que SQL ofrece también constructores que permiten modificar el comportamiento cuando hay valores nulos; los constructores no se van a estudiar aquí.

Se pueden añadir restricciones de integridad a relaciones ya existentes utilizando el comando **alter table** *nombre-tabla* **add** *restricción*, donde *restricción* puede ser cualquiera de las restricciones que se han examinado. Cuando se ejecuta un comando de este tipo, el sistema se asegura primero de que la relación satisfaga la restricción especificada. Si lo hace, se añade la restricción a la relación; en caso contrario, se rechaza el comando.

Las transacciones pueden constar de varios pasos, y las restricciones de integridad pueden violarse temporalmente tras uno de los pasos, pero puede que uno posterior subsane la violación. Por ejemplo, supóngase la relación *persona* con la clave primaria *nombre*, y el atributo *cónyuge* y supóngase que *cónyuge* es una clave externa de *persona*. Es decir, la restricción impone que el atributo *cónyuge* debe contener un nombre que aparezca en la tabla *persona*. Supóngase que se desea destacar el hecho de que Martín y María están casados entre sí mediante la inserción de dos tuplas, una para cada uno, en la relación anterior. La inserción de la primera tupla violaría la restricción de clave externa, independientemente de cuál de las dos tuplas se inserte primero. Una vez insertada la segunda tupla, la restricción de clave externa vuelve a cumplirse.

Para tratar estas situaciones la norma de SQL permite que se añada la cláusula **initially deferred** a la especificación de la restricción; la restricción, en este caso, se comprueba al final de la transacción y no en los pasos intermedios[1]. Las restricciones pueden especificarse de manera alternativa como **deferrable**, que significa que, de manera predeterminada, se comprueba inmediatamente, pero puede diferirse si se desea. Para las restricciones declaradas como diferibles, la ejecución de la instrucción **set constraints**

1. Se puede evitar el problema del ejemplo anterior de otra manera, si el atributo *cónyuge* puede definirse como nulo: se definen los atributos cónyuge como nulos al insertar las tuplas de Martín y María y se actualizan posteriormente. Sin embargo, esta técnica es bastante confusa y no funciona si los atributos no pueden definirse como nulos.

www.detodoprogramacion.com

lista-restricciones **deferred** como parte de una transacción hace que se difiera la comprobación de las restricciones especificadas hasta el final de esa transacción.

No obstante, hay que ser consciente de que el comportamiento predeterminado es comprobar las restricciones de manera inmediata, y de que muchas implementaciones no soportan la comprobación diferida de las restricciones.

4.2.6 Asertos

Un **aserto** es un predicado que expresa una condición que la base de datos debe satisfacer siempre. Las restricciones de dominio y las de integridad referencial son formas especiales de los asertos. Se ha prestado una atención especial a estos tipos de asertos porque se pueden verificar con facilidad y pueden afectar a una gran variedad de aplicaciones de bases de datos. Sin embargo, hay muchas restricciones que no se pueden expresar utilizando únicamente estas formas especiales. Ejemplos de estas restricciones son:

- La suma de los importes de todos los préstamos de cada sucursal debe ser menor que la suma de los saldos de todas las cuentas de esa sucursal.
- Cada préstamo tiene al menos un cliente que tiene una cuenta con un saldo mínimo de 1.000 €

En SQL los asertos adoptan la forma

create assertion <nombre-aserto> **check** <predicado>

En la Figura 4.3 se muestra cómo pueden escribirse estos dos ejemplos de restricciones en SQL. Dado que SQL no proporciona ningún mecanismo "para todo X, $P(X)$" (donde P es un predicado), no queda más remedio que implementarlo utilizando su equivalente "no existe X tal que no $P(X)$", que puede escribirse en SQL.

Cuando se crea un aserto, el sistema comprueba su validez. Si el aserto es válido, sólo se permiten las modificaciones posteriores de la base de datos que no hagan que se viole el aserto. Esta comprobación puede introducir una sobrecarga importante si se han realizado asertos complejos. Por tanto, los asertos deben utilizarse con mucha cautela. La elevada sobrecarga debida a la comprobación y al mantenimiento de los asertos ha llevado a algunos desarrolladores de sistemas a soslayar el soporte para los asertos generales, o bien a proporcionar formas especializadas de aserto que resultan más sencillas de comprobar.

```
create assertion restricción_suma check
    (not exists (select * from sucursal
        where (select sum(importe) from préstamo
                where préstamo.nombre_sucursal = sucursal.nombre_sucursal)
            >= (select sum(saldo) from cuenta
                where cuenta.nombre_sucursal = sucursal.nombre_sucursal)))

create assertion restricción_saldo check
    (not exists (select * from préstamo
        where not exists (select *
            from prestatario, impositor, cuenta
            where préstamo.número_préstamo = prestatario.número_préstamo
                and prestatario.nombre_cliente = impositor.nombre_cliente
                and impositor.número_cuenta = cuenta.número_cuenta
                and cuenta.saldo >= 1000)))
```

Figura 4.3 Dos ejemplos de aserto.

www.detodoprogramacion.com

4.3 Autorización

Se pueden asignar a los usuarios varios tipos de autorización para diferentes partes de la base de datos. Por ejemplo:

- La autorización de lectura
- La autorización de inserción
- La autorización de actualización
- La autorización de borrado

Cada uno de estos tipos de autorización se denomina **privilegio**. Se puede conceder a cada usuario todos estos tipos de privilegios, ninguno de ellos o una combinación de los mismos sobre partes concretas de la base de datos, como puede ser una relación o una vista.

La norma de SQL incluye los privilegios **select**, **insert**, **update** y **delete**. El privilegio **select** autoriza al usuario a leer los datos. Además de estas formas de privilegio para el acceso a los datos, SQL soporta otros privilegios, como el de crear, borrar o modificar relaciones y ejecutar procedimientos. Estos privilegios se estudiarán más adelante, en el Apartado 8.7. **all privileges** puede utilizarse como forma abreviada de todos los privilegios que se pueden conceder. El usuario que crea una relación nueva recibe de manera automática todos los privilegios sobre esa relación.

Se puede permitir al usuario al que se le ha concedido alguna forma de autorización que transmita (conceda) esa autorización a otros usuarios o que retire (revoque) una autorización concedida previamente.

El lenguaje de definición de datos de SQL incluye comandos para conceder y retirar privilegios. La instrucción **grant** se utiliza para conceder autorizaciones. La forma básica de esta instrucción es:

grant <lista de privilegios> **on** <nombre de relación o de vista> **to** <lista de usuarios o de roles>

La *lista de privilegios* permite la concesión de varios privilegios con un solo comando. El concepto de rol se trata más adelante, en el Apartado 8.7.

La siguiente instrucción **grant** concede a los usuarios de la base de datos Martín y María la autorización **select** sobre la relación *cuenta*:

grant select on *cuenta* **to** *Martín*, *María*

La autorización **update** puede concederse sobre todos los atributos de la relación o sólo sobre algunos. Si se incluye la autorización **update** en una instrucción **grant**, la lista de atributos sobre los que se concede la autorización update puede aparecer entre paréntesis justo después de la palabra clave **update**. Si se omite la lista de atributos, el privilegio **update** se concede sobre todos los atributos de la relación.

La siguiente instrucción **grant** concede a los usuarios Martín y María la autorización update sobre el atributo *importe* de la relación *préstamo*:

grant update (*importe*) **on** *préstamo* **to** *Martín*, *María*

El privilegio **insert** también puede especificar una lista de atributos; cualquier inserción en la relación debe especificar sólo esos atributos y el sistema asigna al resto de los atributos valores predeterminados (si hay alguno definido para ellos) o los define como nulos.

El nombre de usuario **public** hace referencia a todos los usuarios actuales y futuros del sistema. Por tanto, los privilegios concedidos a **public** se conceden de manera implícita a todos los usuarios actuales y futuros.

De manera predeterminada el usuario o rol al que se le concede un privilegio no está autorizado a concedérselo a otro usuario o rol. SQL permite que la concesión de privilegios especifique que el destinatario puede concedérselo, a su vez, a otro usuario. Esta característica se describe con más detalle en el Apartado 8.7.

Para retirar una autorización se emplea la instrucción **revoke**. Su forma es casi idéntica a la de **grant**:

www.detodoprogramacion.com

revoke <lista de privilegios> **on** <nombre de la relación o nombre de la vista>
from <lista de usuarios o de roles>

Por tanto, para retirar los privilegios que se han concedido anteriormente:

revoke select on *sucursal* **from** *Martín*, *María*
revoke update (*importe*) **on** *préstamo* **from** *Martín*, *María*

La retirada de privilegios resulta más complicada si el usuario al que se le retiran los privilegios se los ha concedido a otros usuarios. Se volverá a este problema en el Apartado 8.7.

4.4 SQL incorporado

SQL proporciona un lenguaje de consultas declarativo muy potente. La formulación de consultas en SQL es normalmente mucho más sencilla que la formulación de las mismas en un lenguaje de programación de propósito general. Sin embargo, los programadores deben tener acceso a la base de datos desde los lenguajes de programación de propósito general, al menos, por dos razones:

1. No todas las consultas pueden expresarse en SQL, ya que SQL no ofrece todo el poder expresivo de los lenguajes de propósito general. Es decir, hay consultas que se pueden expresar en lenguajes como C, Java o Cobol que no se pueden expresar en SQL. Para formular consultas de este tipo, se puede incorporar SQL en un lenguaje más potente.

   SQL está diseñado de tal forma que las consultas formuladas puedan optimizarse automáticamente y se ejecuten de manera eficiente—y proporcionar toda la potencia de los lenguajes de programación hace la optimización automática extremadamente difícil.

2. Las acciones no declarativas—como la impresión de informes, la interacción con los usuarios o el envío de los resultados de las consultas a una interfaz gráfica—no se pueden llevar a cabo desde el propio SQL. Normalmente, las aplicaciones contienen varios componentes y la consulta o actualización de los datos sólo es uno de ellos; los demás componentes se escriben en lenguajes de programación de propósito general. En el caso de las aplicaciones integradas, los programas escritos en el lenguaje de programación deben poder tener acceso a la base de datos.

La norma SQL define la incorporación de SQL en varios lenguajes de programación, tales como C, Cobol, Pascal, Java, PL/I, y Fortran. El lenguaje en el que se incorporan las consultas SQL se denomina lenguaje *anfitrión* y las estructuras de SQL que se admiten en el lenguaje anfitrión constituyen SQL *incorporado*.

Los programas escritos en el lenguaje anfitrión pueden usar la sintaxis de SQL incorporado para tener acceso a los datos almacenados en la base de datos y actualizarlos. Esta forma incorporada de SQL amplía aún más la capacidad de los programadores de manipular las bases de datos. En SQL incorporado toda la ejecución de las consultas la realiza el sistema de bases de datos, que luego pone el resultado de la consulta a disposición del programa tupla (registro) a tupla.

Los programas de SQL incorporado deben procesarlos un preprocesador especial antes de la compilación. El preprocesador sustituye las peticiones de SQL incorporado por declaraciones escritas en el lenguaje anfitrión y por llamadas a procedimientos que permiten la ejecución de los accesos a la base de datos en el momento de la ejecución. Posteriormente, el compilador del lenguaje anfitrión compila el programa resultante. Para identificar las peticiones al preprocesador de SQL incorporado se utiliza la instrucción EXEC SQL; tiene la forma

EXEC SQL <instrucción de SQL incorporado > END-EXEC

La sintaxis exacta de las peticiones de SQL incorporado depende del lenguaje en el que se haya incorporado SQL. Por ejemplo, cuando se incorpora SQL en C, se utiliza un punto y coma en lugar de END-EXEC. La incorporación de SQL en Java (denominada SQLJ) utiliza la sintaxis

SQL { <instrucción de SQL incorporado > };

www.detodoprogramacion.com

En el programa se incluye la instrucción SQL INCLUDE para identificar el lugar donde el preprocesador debe insertar las variables especiales que se emplean para la comunicación entre el programa y el sistema de bases de datos. Las variables del lenguaje anfitrión se pueden utilizar en las instrucciones de SQL incorporado, pero deben ir precedidas de dos puntos (:) para distinguirlas de las variables de SQL.

Antes de ejecutar ninguna instrucción de SQL el programa debe conectarse con la base de datos. Esto se logra mediante

EXEC SQL **connect to** *servidor* **user** *nombre-usuario* END-EXEC

En este caso, *servidor* identifica al servidor con el que hay que establecer la conexión. Algunas implementaciones de las bases de datos pueden exigir que se proporcione una contraseña además del nombre de usuario.

Las instrucciones de SQL incorporado son parecidas en cuanto a la forma a las instrucciones de SQL que se han descrito en este capítulo. Sin embargo, hay varias diferencias que se indican a continuación.

Para formular una consulta relacional se emplea la instrucción **declare cursor**. El resultado de la consulta no se calcula todavía. En vez de eso, el programa debe utilizar los comandos **open** y **fetch** (que se analizarán más adelante en este apartado) para obtener las tuplas resultado.

Considérese el esquema bancario que se ha utilizado en este capítulo. Supóngase la variable del lenguaje anfitrión *importe* y que se desea determinar el nombre y ciudad de residencia de los clientes que tienen más de *importe* euros en alguna de sus cuentas. Se puede escribir esta consulta del modo siguiente:

```
EXEC SQL
        declare c cursor for
        select nombre_cliente, ciudad_cliente
        from impositor, cliente, cuenta
        where impositor.nombre_cliente = cliente.nombre_cliente and
                cuenta.número_cliente = impositor.número_cuenta and
                cuenta.saldo > :importe
END-EXEC
```

La variable *c* de la expresión anterior se denomina *cursor* de la consulta. Se utiliza esta variable para identificar la consulta en la instrucción **open**, que hace que se evalúe la consulta, y en la instrucción **fetch**, que hace que los valores de una tupla se coloquen en las variables del lenguaje anfitrión.

La instrucción **open** para la consulta de anterior es:

EXEC SQL **open** *c* END-EXEC

Esta instrucción hace que el sistema de base de datos ejecute la consulta y guarde el resultado en una relación temporal. La consulta tiene una variable del lenguaje anfitrión (:*importe*); la consulta utiliza el valor de la variable en el momento en que se ejecuta la instrucción **open**.

Si la consulta de SQL genera un error, el sistema de base de datos almacena un diagnóstico de error en las variables del área de comunicación de SQL (SQL communication-area, SQLCA), cuyas declaraciones inserta la instrucción SQL INCLUDE.

El programa de SQL incorporado ejecuta una serie de instrucciones **fetch** para recuperar las tuplas del resultado. La instrucción **fetch** necesita una variable del lenguaje anfitrión por cada atributo de la relación resultado. En la consulta de ejemplo se necesita una variable para almacenar el valor de *nombre_cliente* y otra para el de *ciudad_cliente*. Supóngase que esas variables son *nc* y *cc*, respectivamente. Entonces, la instrucción

EXEC SQL **fetch** *c* **into** :*cn*, :*cc* END-EXEC

genera una tupla de la relación resultado. El programa puede manipular entonces las variables *nc* y *cc* empleando las características del lenguaje de programación anfitrión.

Cada solicitud **fetch** devuelve una sola tupla. Para obtener todas las tuplas del resultado, el programa debe incorporar un bucle para iterar sobre todas las tuplas. SQL incorporado ayuda a los programadores en el tratamiento de esta iteración. Aunque las relaciones sean conceptualmente conjuntos, las tuplas del

www.detodoprogramacion.com

resultado de las consultas se hallan en un orden físico determinado. Cuando el programa ejecuta una instrucción **open** sobre un cursor, ese cursor pasa a apuntar a la primera tupla del resultado. Cada vez que se ejecuta una instrucción **fetch**, el cursor se actualiza para que apunte a la siguiente tupla del resultado. Cuando no quedan por procesar más tuplas, la variable SQLSTATE de SQLCA se define como '02000' (que significa "sin datos"). Por tanto, se puede utilizar un bucle **while** (o su equivalente) para procesar cada tupla del resultado.

La instrucción **close** se debe utilizar para indicar al sistema de bases de datos que borre la relación temporal que guardaba el resultado de la consulta. Para el ejemplo anterior, esta instrucción tiene la forma

EXEC SQL **close** *c* END-EXEC

SQLJ, la incorporación en Java de SQL, ofrece una variación del esquema anterior, en la que se emplean los iteradores de Java en lugar de los cursores. SQLJ asocia los resultados de la consulta con un iterador, y se puede utilizar el método next() del iterador de Java para pasar de una tupla a otra del resultado, igual que los ejemplos anteriores utilizan **fetch** sobre el cursor.

Las expresiones de SQL incorporado para la modificación (**update**, **insert** y **delete**) de las bases de datos no devuelven ningún resultado. Por tanto, son, de algún modo, más sencillas de expresar. Una solicitud de modificación de la base de datos tiene la forma

EXEC SQL < cualquier **update, insert** o **delete** válido > END-EXEC

En la expresión de SQL para la modificación de la base de datos pueden aparecer variables del lenguaje anfitrión precedidas de dos puntos. Si se produce un error en la ejecución de la instrucción, se establece un diagnóstico en SQLCA.

Las relaciones de las bases de datos también se pueden actualizar mediante cursores. Por ejemplo, si se desea sumar 100 euros al atributo *saldo* de cada *cuenta* en la que el nombre de sucursal sea "Navacerrada", se puede declarar un cursor como sigue:

```
declare c cursor for
select *
from cuenta
where nombre_sucursal = 'Navacerrada'
for update
```

Después se itera por las tuplas ejecutando operaciones **fetch** sobre el cursor (como se mostró anteriormente) y, después de obtener cada tupla, se ejecuta el siguiente código:

```
update cuenta
set saldo = saldo + 100
where current of c
```

SQL incorporado permite que los programas en el lenguaje anfitrión tengan acceso a la base de datos, pero no proporciona ninguna ayuda para la presentación de los resultados al usuario ni para la generación de informes. La mayor parte de los productos comerciales de bases de datos incluyen herramientas para ayudar a los programadores de aplicaciones a crear interfaces de usuario e informes con formato. El Capítulo 8 describe la manera de crear aplicaciones de bases de datos con interfaces de usuario, concentrándose en las interfaces de usuario basadas en Web.

4.5 SQL dinámico

El componente *dinámico* de SQL permite que los programas construyan y remitan consultas de SQL en tiempo de ejecución. En cambio, las instrucciones de SQL incorporado deben hallarse presentes completamente en el momento de la compilación; las compila el preprocesador de SQL incorporado. Por medio de SQL dinámico los programas pueden crear consultas de SQL en tiempo de ejecución (quizás basadas en datos introducidos por el usuario) y hacer que se ejecuten inmediatamente o dejarlas *preparadas* para

www.detodoprogramacion.com

su uso posterior. En el proceso de preparación de una instrucción SQL dinámica ésta se compila, y al usarla posteriormente se aprovecha su versión compilada.

SQL define normas para incorporar las llamadas dinámicas a SQL en el lenguaje anfitrión, por ejemplo, C, como se muestra en el siguiente ejemplo.

```
char * prog_SQL = "update cuenta set saldo = saldo *1.05
                   where número_cuenta = ?";
EXEC SQL prepare prog_din from :prog_SQL;
char cuenta[10] = "C-101";
EXEC SQL execute prog_din using :cuenta;
```

El programa SQL dinámico contiene un símbolo de interrogación ?, que representa un valor que se proporciona cuando se ejecuta el programa de SQL.

Sin embargo, esta sintaxis necesita extensiones para el lenguaje o un preprocesador para el lenguaje extendido. Una alternativa que se utiliza con frecuencia es emplear una API para enviar las consultas o actualizaciones de SQL a un sistema de bases de datos, sin realizar cambios en el propio lenguaje de programación.

En el resto de este apartado se examinan dos normas de conexión a bases de datos de SQL y la realización de consultas y de actualizaciones. Una, ODBC, es una interfaz para programas de aplicación desarrollada inicialmente para el lenguaje C, y extendida posteriormente a otros lenguajes como C++, C# y Visual Basic. La otra, JDBC, es una interfaz para programas de aplicación para el lenguaje Java.

Para entender estas normas hay que comprender el concepto de sesión de SQL. El usuario o la aplicación se *conectan* al servidor de SQL y establecen una sesión; ejecutan una serie de instrucciones y, finalmente, se *desconectan* de la sesión. Así, todas las actividades del usuario o de la aplicación están en el contexto de una sesión de SQL. Además de los comandos normales de SQL, las sesiones también pueden incluir comandos para *comprometer* el trabajo realizado en la sesión o para *retrocederlo*.

4.5.1 ODBC

La norma **ODBC** (Open Database Connectivity, conectividad abierta de bases de datos) define el modo de comunicación entre los programas de aplicación y los servidores de bases de datos. ODBC define una **interfaz para programas de aplicación** (API, Application Program Interface) que pueden utilizar las aplicaciones para abrir conexiones con las bases de datos, enviar las consultas y las actualizaciones y obtener los resultados. Las aplicaciones como las interfaces gráficas de usuario, los paquetes estadísticos y las hojas de cálculo pueden emplear la misma API de ODBC para conectarse a cualquier servidor de bases de datos compatible con ODBC.

Cada sistema de bases de datos compatible con ODBC proporciona una biblioteca que se debe enlazar con el programa cliente. Cuando el programa cliente realiza una llamada a la API de ODBC, el código de la biblioteca se comunica con el servidor para llevar a cabo la acción solicitada y obtener los resultados.

La Figura 4.4 muestra un ejemplo de código C que usa la API de ODBC. El primer paso en el uso de ODBC para comunicarse con un servidor es configurar su conexión. Para ello, el programa asigna en primer lugar un entorno de SQL, luego un manejador para la conexión a la base de datos. ODBC define los tipos HENV, HDBCy RETCODE. El programa abre a continuación la conexión a la base de datos empleando SQLConnect. Esta llamada tiene varios parámetros, como son el manejador de la conexión, el servidor al que hay que conectarse, el identificador de usuario y la contraseña para la base de datos. La constante SQL_NTS indica que el argumento anterior es una cadena terminada en un valor nulo.

Una vez configurada la conexión, el programa puede enviar comandos SQL a la base de datos empleando SQLExecDirect. Las variables del lenguaje C se pueden vincular a los atributos del resultado de la consulta, de forma que cuando se obtenga una tupla resultado mediante SQLFetch, los valores de sus atributos se almacenen en las variables de C correspondientes. La función SQLBindCol realiza esta tarea; el segundo argumento identifica la posición del atributo en el resultado de la consulta, y el tercer argumento indica la conversión de tipos de SQL a C requerida. El siguiente argumento proporciona la dirección de la variable. Para los tipos de datos de longitud variable como los arrays de caracteres, los dos últimos argumentos proporcionan la longitud máxima de la variable y una ubicación donde se debe almacenar la longitud real cuando se obtenga una tupla. Un valor negativo devuelto para el argumento

www.detodoprogramacion.com

```
void ODBCexample()
{
    RETCODE error;
    HENV ent; /* entorno */
    HDBC con; /* conexión a la base de datos */

    SQLAllocEnv(&ent);
    SQLAllocConnect(ent, &con);
    SQLConnect(con, "db.yale.edu", SQL_NTS, ”avi”, SQL_NTS,
                ”contraseñaAvi”, SQL_NTS);
    {
        char nombresucursal[80];
        float saldo;
        int lenOut1, lenOut2;
        HSTMT stmt;

        char * consulta = "select nombre_sucursal, sum (saldo)
                           from cuenta
                           group by nombre_sucursal";
        SQLAllocStmt(con, &stmt);
        error = SQLExecDirect(stmt, consulta, SQL_NTS);
        if (error == SQL_SUCCESS) {
            SQLBindCol(stmt, 1, SQL_C_CHAR, nombresucursal , 80, &lenOut1);
            SQLBindCol(stmt, 2, SQL_C_FLOAT, &saldo, 0 , &lenOut2);
            while (SQLFetch(stmt) == SQL_SUCCESS) {
                printf (" %s %g\n", nombresucursal, saldo);
            }
        }
        SQLFreeStmt(stmt, SQL_DROP);
    }
    SQLDisconnect(con);
    SQLFreeConnect(con);
    SQLFreeEnv(ent);
}
```

Figura 4.4 Código de ejemplo ODBC.

de la longitud indica que el valor es **null**. Para los tipos de datos de longitud fija como integer o float, se ignora el argumento que indica la longitud máxima, mientras que la devolución de un valor negativo para el argumento de la longitud indica un valor nulo.

La instrucción SQLFetch se halla en un bucle **while** que se ejecuta hasta que SQLFetch devuelva un valor diferente de SQL_SUCCESS. En cada iteración, el programa almacena los valores en variables de C como se especifica mediante las llamadas a SQLBindCol e imprime esos valores.

Al final de la sesión, el programa libera el controlador de la instrucción, se desconecta de la base de datos y libera la conexión y los manejadores del entorno de SQL. Un buen estilo de programación exige que el resultado de cada llamada a una función se compruebe para asegurarse de que no haya errores; se han omitido la mayoría de estas comprobaciones en aras de la brevedad.

Es posible crear instrucciones SQL con parámetros; por ejemplo, considérese la instrucción insert into cuenta values(?,?,?). Los signos de interrogación representan los valores que se proporcionarán después. Esta instrucción se puede “preparar”, es decir, compilar en la base de datos y ejecutar repetidamente proporcionando los valores reales para los parámetros—en este caso, proporcionando un número de cuenta, un nombre de sucursal y un saldo para la relación *cuenta*.

www.detodoprogramacion.com

ODBC define funciones para gran variedad de tareas, tales como hallar todas las relaciones de la base de datos y los nombres y tipos de las columnas del resultado de una consulta o de una relación de la base de datos.

De forma predeterminada, cada instrucción SQL se trata como una transacción separada que se compromete automáticamente. La llamada SQLSetConnectOption(con, SQL_AUTOCOMMIT, 0) desactiva el compromiso automático en la conexión con, por lo que las transacciones se deben comprometer explícitamente mediante SQLTransact(con, SQL_COMMIT) o retroceder mediante SQLTransact(con, SQL_ROLLBACK).

La norma ODBC define *niveles de conformidad*, que especifican subconjuntos de la funcionalidad definida por la norma. Puede que una implementación de ODBC sólo proporcione las características básicas, o puede proporcionar características más avanzadas (de nivel 1 o de nivel 2). El nivel 1 exige soporte para la captura de información sobre el catálogo, como puede ser la información sobre las relaciones existentes y los tipos de sus atributos. El nivel 2 exige más características, como la capacidad de enviar y obtener arrays de valores de parámetros y la de obtener información del catálogo más detallada.

La norma de SQL define una **interfaz del nivel de llamadas** (CLI, Call Level Interface) que es parecida a la interfaz de ODBC. Las APIs ADO y ADO.NET son alternativas a ODBC, diseñadas para los lenguajes Visual Basic y C#; consúltense las notas bibliográficas si se desea más información.

La norma de **JDBC** define una API que pueden utilizar los programas de Java para conectarse a los servidores de bases de datos (la palabra JDBC era originalmente una abreviatura de **Java Database Connectivity**—conectividad de bases de datos con Java—, pero la forma completa ya no se emplea).

4.5.1.1 Apertura de conexiones y ejecución de consultas

La Figura 4.5 muestra un ejemplo de programa Java que utiliza la interfaz JDBC. El programa, en primer lugar, debe abrir una conexión con la base de datos y después puede ejecutar las instrucciones de SQL, pero antes de abrir la conexión, carga los controladores correspondientes para la base de datos empleando Class.forName. El primer parámetro de la llamada getConnection especifica el nombre de la máquina en la que se ejecuta el servidor (en este caso, db.yale.edu) y el número del puerto que emplea para las comunicaciones (en este caso, 2.000). El parámetro también especifica el esquema de la base de datos que se va a utilizar (en este caso, bdbanco), ya que cada servidor de bases de datos puede dar soporte a varios esquemas. El primer parámetro también especifica el protocolo que se va a utilizar para la comunicación con la base de datos (en este caso, jdbc:oracle:thin:). Obsérvese que JDBC especifica sólo la API, no el protocolo de comunicación. Los controladores de JDBC pueden dar soporte a varios protocolos y se debe especificar alguno compatible tanto con la base de datos como con el controlador. Los otros dos argumentos de **getConnection** son un identificador de usuario y una contraseña.

4.5.2 JDBC

El programa crea a continuación un manejador de instrucciones para la conexión y lo usa para ejecutar una instrucción SQL y obtener los resultados. En nuestro ejemplo, stmt.executeUpdate ejecuta una instrucción de actualización. El constructor try { . . . } catch { . . . } permite capturar cualquier excepción (condición de error) que surja cuando se realicen las llamadas JDBC, e imprime el mensaje correspondiente para el usuario.

El programa puede ejecutar una consulta mediante stmt.executeQuery. Puede recuperar el conjunto de filas del resultado en ResultSet y capturarlas tupla a tupla empleando la función next() en el conjunto de resultados. La Figura 4.5 muestra dos formas de recuperar los valores de los atributos de las tuplas: empleando el nombre del atributo (*nombre_sucursal*) y utilizando la posición del atributo (2, para denotar el segundo atributo).

La conexión se cierra al final del procedimiento. Obsérvese que es importante cerrar la conexión, ya que se ha impuesto un límite al número de conexiones con la base de datos; las conexiones no cerradas pueden hacer que ese límite se supere. Si esto ocurre, la aplicación no podrá abrir más conexiones con la base de datos.

www.detodoprogramacion.com

```
public static void JDBCexample(String dbid, String idusuario, String contraseña)
{
    try
    {
        Class.forName (”oracle.jdbc.driver.OracleDriver”);
        Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@db.yale.edu:2000:bdbanco",
                idusuario, contraseña);
        Statement stmt = con.createStatement();
        try {
            stmt.executeUpdate(
                "insert into cuenta values('C-9732', 'Navacerrada', 1200)");
        } catch (SQLException sqle)
        {
            System.out.println("No se pudo insertar la tupla. "+ sqle);
        }
        ResultSet rset = stmt.executeQuery(
                "select nombre_sucursal, avg (saldo)
                from cuenta
                group by nombre_sucursal");
        while (rset.next()) {
            System.out.println(rset.getString("nombre_sucursal") +  +
                    rset.getFloat(2));
        }
        stmt.close();
        con.close();
    }
    catch (SQLException sqle)
    {
        System.out.println("Excepción de SQL : "+ sqle);
    }
}
```

Figura 4.5 Un ejemplo de código JDBC.

4.5.2.1 Instrucciones preparadas

Se pueden crear instrucciones preparadas en las que algunos valores estén sustituidos por “?”, lo que indica que los valores reales se proporcionarán más tarde. Algunos sistemas de bases de datos compilan las consultas cuando se preparan; cada vez que se ejecuta la consulta (con valores nuevos), la base de datos puede volver a emplear la forma previamente compilada de la consulta. El fragmento de código de la Figura 4.6 muestra la manera de utilizar las instrucciones preparadas. La función setString (y otras funciones parecidas para otros tipos de datos básicos de SQL) permite especificar el valor de los parámetros.

Las instrucciones preparadas son el método preferido de ejecucion de las consultas de SQL, cuando utilizan valores que introducen los usuarios. Supóngase que el valor de las variables número_cuenta, nombre_sucursal, and saldo ha sido introducido por un usuario y hay que insertar la fila correspondiente en la relación *cuenta.* Supóngase que, en lugar de utilizar una instrucción preparada, se crea una consulta mediante la concatenación de las cadenas de caracteres de la manera siguiente:

```
"insert into cuenta values(' "+ número_cuenta + -', ' "+ nombre_sucursal + -', "
        + saldo + ")"
```

y la consulta se ejecuta directamente. Ahora, si el usuario escribiera una sola comilla en el campo del número de cuenta o en el del nombre de la sucursal, la cadena de caracteres de la consulta tendría un error

```
PreparedStatement pStmt = con.prepareStatement(
                "insert into cuenta values(?,?,?)");
pStmt.setString(1, "C-9732");
pStmt.setString(2, "Navacerrada");
pStmt.setInt(3, 1200);
pStmt.executeUpdate();
pStmt.setString(1, "C-9733");
pStmt.executeUpdate();
```

Figura 4.6 Instrucciones preparadas en código JDBC.

de sintaxis. Es bastante probable que alguna sucursal tenga un apóstrofo en su nombre (especialmente si es un nombre irlandés como O'Donnell). Peor todavía, intrusos informáticos maliciosos pueden "inyectar" consultas de SQL propias escribiendo los caracteres adecuados en la cadena de caracteres. Una inyección de SQL de ese tipo puede dar lugar a graves fallos de seguridad.

La adición de caracteres de escape para tratar con los apóstrofos de las cadenas de caracteres es una manera de resolver este problema. El uso de las instrucciones preparadas es una manera más sencilla de hacerlo, ya que el método setString añade caracteres de escape de manera implícita. Además, cuando hay que ejecutar varias veces la misma instrucción con valores diferentes, las instrucciones preparadas suelen ejecutarse mucho más rápido que varias instrucciones de SQL por separado.

JDBC también proporciona una interfaz CallableStatement que permite la llamada a los procedimientos almacenados y a las funciones de SQL (que se describen más adelante, en el Apartado 4.6). Esta interfaz desempeña el mismo rol para las funciones y los procedimientos que prepareStatement para las consultas.

```
CallableStatement cStmt1 = con.prepareCall("{? = call alguna_función(?)}");
CallableStatement cStmt2 = con.prepareCall("{call algún_procedimiento(?,?)}");
```

Los tipos de datos de los valores devueltos por las funciones y de los parámetros externos de los procedimientos deben registrarse empleando el método registerOutParameter(), y pueden recuperarse utilizando métodos get parecidos a los de los conjuntos de resultados.

4.5.2.2 Metadatos

JDBC también proporciona mecanismos para examinar los esquemas de las bases de datos y determinar el tipo de datos de los atributos de los conjuntos de resultados. La interfaz ResultSet tiene un método getMetaData() para obtener objetos ResultSetMetaData y proporcionar los metadatos del conjunto de resultados. La interfaz ResultSetMetaData, a su vez, contiene métodos para determinar la información de los metadatos, como puede ser el número de columnas de un resultado, el nombre de una columna concreta o el tipo de datos de una columna dada. El programa de JDBC que se muestra a continuación ilustra el uso de la interfaz ResultSetMetaData para mostrar el nombre y el tipo de datos de todas las columnas de un conjunto de resultados. Se da por supuesto que la variable cr del siguiente código es un conjunto de resultados obtenido mediante la ejecución de una consulta.

```
ResultSetMetaData rsmd = cr.getMetaData();
for(int i = 1; i <= rsmd.getColumnCount(); i++) {
        System.out.println(rsmd.getColumnName(i));
        System.out.println(rsmd.getColumnTypeName(i));
}
```

El interfaz DatabaseMetaData ofrece una manera de determinar los metadatos de la base de datos. La interfaz Connection contiene un método getMetaData que devuelve el objeto DatabaseMetaData. La interfaz DatabaseMetaData, a su vez, contiene gran número de métodos para obtener los metadatos de la base de datos. El código de la Figura 4.7 ilustra la manera de obtener información sobre las columnas (atributos) de las relaciones de las bases de datos. Se da por supuesto que la variable con almacena una

```
DatabaseMetaData dbmd = con.getMetaData();
ResultSet sr = dbmd.getColumns(null, "bdbanco", "cuenta", "%");
      // Argumentos para getColumns: Catálogo, Patrón-esquema, Patrón-tabla y
      //            Patrón-columna
      // Devuelve: Una fila por columna; cada fila tiene varios atributos, como
      //            COLUMN_NAME (nombre de la columna), TYPE_NAME (nombre del tipo)
while( sr.next()) {
      System.out.println(sr.getString("COLUMN_NAME"), sr.getString("TYPE_NAME");
}
```

Figura 4.7 Búsqueda de información sobre las columnas en JDBC empleando DatabaseMetaData.

conexión con la base de datos ya abierta. El método getColumns toma cuatro argumentos: un nombre de catálogo (null significa que hay que ignorar el nombre del catálogo), un patrón para el nombre de los esquemas, un patrón para el nombre de las tablas y un patrón para el nombre de las columnas. El nombre del esquema, el de la tabla y los patrones del nombre de las columnas pueden utilizarse para especificar un nombre o un patrón. Los patrones pueden emplear los caracteres especiales para encaje de cadenas de caracteres de SQL "%" y "_"; por ejemplo, el patrón "%" encaja con todos los nombres.

Sólo se recuperan las columnas de las tablas de los esquemas que satisfacen el nombre o el patrón especificados. Cada fila del conjunto de resultados contiene información sobre una columna. Las filas tienen varias columnas, como pueden ser el nombre del catálogo, del esquema, de la tabla y de la columna, el tipo de datos de la columna, etc.

Otros métodos proporcionados por DatabaseMetaData permiten la recuperación de metadatos de las relaciones (getTables()), las referencias a claves externas (getCrossReference()), las autorizaciones, los límites de las bases de datos como el número máximo de conexiones, etc.

Las interfaces para metadatos pueden emplearse para gran variedad de tareas. Por ejemplo, se pueden utilizar par escribir un navegador de la base de datos que permita a los usuarios buscar las tablas de las bases de datos, examinar su esquema, examinar las filas de las tablas, aplicar selecciones para ver las filas deseadas, etc. La información sobre los metadatos puede utilizarse para crear código empleado para estas tareas genéricas; por ejemplo, se puede escribir código para mostrar las filas de las relaciones de manera que funcione en todas las relaciones posibles, independientemente de su esquema. De manera parecida, es posible escribir código que tome una cadena de caracteres de consulta, ejecute la consulta e imprima el resultado en forma de tabla con formato; el código puede funcionar independientemente de la consulta concreta que se remita.

4.5.2.3 Otras características

JDBC ofrece varias características más, como los **conjuntos de resultados actualizables**. Se pueden crear conjuntos de resultados actualizables a partir de consultas que lleven a cabo selecciones o proyecciones de las relaciones de las bases de datos. La actualización de una tupla del conjunto de resultados tiene como consecuencia que se actualice la tupla correspondiente de la relación de la base de datos.

De manera predeterminada, cada instrucción de SQL se trata como una transacción independiente que se compromete de manera automática. El método setAutoCommit() de la interfaz Connection de JDBC permite que este comportamiento se active o se desactive. Por tanto, si con es una variable que almacena una conexión abierta, conn.setAutoCommit(false) desactivará el compromiso automático. Las transacciones, entonces, deben comprometerse de manera explícita mediante con.commit() o retroceder-se mediante con.rollback(). El compromiso automático puede activarse mediante con.setAutoCommit(true).

JDBC proporciona interfaces para el tratamiento de objetos de gran tamaño sin necesidad de crearlos completamente en la memoria. Para capturar objetos de gran tamaño la clase ResultSet proporciona los métodos getBlob() y getClob(), que son parecidos al método getString() pero devuelven objetos de los tipos de datos Blob y Clob, respectivamente. Estos objetos no almacenan los objetos de gran tamaño completos, sino que almacenan sus localizadores. Las clases Blob y Clob proporcionan métodos para recuperar objetos de gran tamaño en fragmentos más pequeños. También permiten que se almacenen objetos de gran tamaño en la base de datos; pueden asociarse con corrientes de datos de Java, que se

```
create function recuento_cuentas(nombre_cliente varchar(20))
    returns integer
    begin
    declare recuento_c integer;
        select count(*) into recuento_c
        from impositor
        where impositor.nombre_cliente = nombre_cliente
    return recuento_c;
    end
```

Figura 4.8 Función definida en SQL.

capturan de manera transparente y se envían a la base de datos en fragmentos de pequeño tamaño, por lo que no hace falta crear el objeto completo en la memoria.

JDBC también proporciona la clase RowSet, que ofrece las mismas características de ResultSet y varias características adicionales. Para obtener más información sobre JDBC, consúltese la información bibliográfica del final del capítulo.

4.6 Funciones y procedimientos**

Desde la versión SQL:1999, SQL permite la definición de funciones, procedimientos y métodos. Pueden definirse bien mediante el componente procedimental de SQL:1999 o bien mediante un lenguaje de programación externo como Java, C o C++. En primer lugar se examinarán las definiciones en SQL y luego se verá la manera de utilizar las definiciones en lenguajes externos.

Varios sistemas de bases de datos soportan sus propias extensiones procedimentales de SQL, como PL/SQL en Oracle y TransactSQL en Microsoft SQL Server. Estas extensiones recuerdan la parte procedimental de SQL, pero puede haber diferencias significativas en sintaxis y en semántica; consúltense los manuales de los sistemas respectivos para obtener más detalles.

4.6.1 Funciones y procedimientos de SQL

Supóngase que se desea una función que, dado el nombre de un cliente, devuelva el número de cuentas de las que es titular ese cliente. Se puede definir la función como se muestra en la Figura 4.8.

Esta función puede utilizarse en una consulta que devuelva el nombre y la dirección de todos los clientes con más de una cuenta:

```
select nombre_cliente, calle_cliente, ciudad_cliente
from cliente
where recuento_cuentas(nombre_cliente) > 1
```

Las funciones resultan particularmente útiles con tipos de datos especializados tales como las imágenes y los objetos geométricos. Por ejemplo, un tipo de datos de segmentos de recta empleado en las bases de datos cartográficas puede tener una función asociada que compruebe si se solapan dos segmentos, y un tipo de datos de imagen puede tener asociadas funciones para comparar la semejanza de dos imágenes. Las funciones se pueden escribir en un lenguaje externo como C, como se vio en el Apartado 4.6.3.

Desde la versión SQL:2003, SQL soporta las funciones que pueden devolver tablas como resultado; esas funciones se denominan **funciones de tabla**. Considérese la función definida en la Figura 4.9. Esa función devuelve una tabla que contiene todas las cuentas que tiene una persona dada. Obsérvese que se hace referencia a los parámetros de las funciones anteponiéndoles el nombre de la función (*cuentas_de.nombre_cliente*).

La función puede emplearse en consultas de la manera siguiente:

```
select *
from table(cuentas_de('Gómez'))
```

www.detodoprogramacion.com

```
create function cuentas_de (nombre_cliente char(20))
    returns table (
        número_cuenta char(10),
        nombre_sucursal char(15),
        saldo numeric(12,2))
return table
    (select número_cuenta, nombre_sucursal, saldo
    from cuenta
    where exists (
        select *
        from impositor
        where impositor.nombre_cliente = cuentas_de.nombre_cliente and
            impositor.número_cuenta = cuenta.número_cuenta)
    )
```

Figura 4.9 Función de tabla en SQL.

Esta consulta devuelve todas las cuentas pertenecientes al cliente 'Gómez'. En el sencillo caso anterior, resulta fácil escribir esta consulta sin emplear funciones evaluadas sobre tablas. En general, sin embargo, las funciones evaluadas sobre tablas pueden considerarse **vistas paramétricas** que generalizan el concepto habitual de las vistas permitiendo la introducción de parámetros.

SQL:1999 también permite el uso de procedimientos. La función *recuento_cuentas* se puede escribir en forma de procedimiento:

```
create procedure proc_recuento_cuentas(in nombre_cliente varchar(20),
                                       out recuento_c integer)
begin
    select count(*) into recuento_c
    from impositor
    where impositor.nombre_cliente = proc_recuento_cuentas.nombre_cliente
end
```

Se pueden invocar los procedimientos mediante la instrucción **call** desde otro procedimiento de SQL o desde SQL incorporado:

```
declare recuento_cuentas integer;
call proc_recuento_cuentas('Gómez', recuento_c);
```

Los procedimientos y las funciones pueden invocarse desde SQL dinámico, como se ilustra mediante la sintaxis de JDBC en el Apartado 4.5.2.2.

SQL:1999 permite que haya más de un procedimiento con el mismo nombre, siempre que el número de argumentos de los procedimientos con el mismo nombre sea diferente. El nombre, junto con el número de argumentos, se utiliza para identificar los procedimientos. SQL permite también más de una función con el mismo nombre, siempre que las diferentes funciones con el mismo nombre tengan diferente número de argumentos o, para funciones con el mismo número de argumentos, se diferencien en el tipo de datos de un argumento, como mínimo.

4.6.2 Constructores procedimentales

Desde la versión SQL:1999, SQL soporta varios constructores procedimentales que le proporcionan casi toda la potencia de los lenguajes de programación de propósito general. La parte de la norma SQL que trata de estos constructores se denomina **módulo de almacenamiento persistente** (Persistent Storage Module, PSM).

El objetivo de PSM en SQL no es sustituir a los lenguajes de programación convencionales. Más bien, los constructores procedimentales permiten que se registre la "lógica de negocio" en forma de procedimientos almacenados de la base de datos y se ejecute en propia la base de datos. Por ejemplo, los bancos

www.detodoprogramacion.com

suelen tener muchas reglas sobre la manera y el momento en que se pueden hacer los pagos a los clientes, como los límites máximos de retirada de efectivo, las exigencias de saldo mínimo, las facilidades para los descubiertos que permiten a los clientes retirar más dinero que el saldo disponible mediante la concesión automática de un préstamo, etc.

Aunque esa lógica de negocio puede codificarse en forma de procedimientos de los lenguajes de programación almacenados completamente fuera de las bases de datos, su definición como procedimientos almacenados de la base de datos presenta varias ventajas. Por ejemplo, permite que varias aplicaciones tengan acceso a los procedimientos, así como que baste con modificar un solo punto en caso de cambio de las reglas de negocio, sin que haga falta modificar la aplicación. El código de la aplicación puede llamar a los procedimientos almacenados, en lugar de actualizar directamente las relaciones de la base de datos.

Los constructores procedimentales son necesarios para permitir que se codifiquen como procedimientos almacenados las reglas de negocio complejas y, por tanto, se añadieron a SQL a partir de la versión SQL:1999 (ya las soportaban algunos productos de bases de datos incluso antes).

Las instrucciones compuestas son de la forma **begin** . . . **end**, y pueden contener varias instrucciones de SQL entre **begin** y **end**. En las instrucciones compuestas pueden declararse variables locales, como se ha visto en el Apartado 4.6.1.

SQL:1999 soporta las instrucciones **while** y **repeat** con la sintaxis siguiente:

```
declare n integer default 0;
while n < 10 do
    set n = n + 1;
end while
repeat
    set n = n − 1;
until n = 0
end repeat
```

Este código no lleva a cabo nada útil; simplemente pretende mostrar la sintaxis de los bucles **while** y **repeat**. Posteriormente se verán aplicaciones más significativas.

También hay un bucle **for** que permite la iteración por todos los resultados de una consulta:

```
declare n integer default 0;
for r as
        select saldo from cuenta
        where nombre_sucursal = 'Navacerrada'
do
    set n = n+ r.saldo
end for
```

El programa abre de manera implícita un cursor cuando comienza la ejecución del bucle **for** y lo utiliza para capturar los valores fila a fila en la variable de bucle **for** (*r*, en este ejemplo). Es posible ponerle nombre al cursor, insertando el texto *nc* **cursor for** justo detrás de la palabra clave **as**, donde *nc* es el nombre que se le desea dar al cursor. El nombre del cursor puede emplearse para llevar a cabo operaciones de actualización o borrado sobre la tupla a la que apunte el cursor. La instrucción **leave** se puede emplear para salir del bucle, mientras que **iterate** comienza con la siguiente tupla, a partir del comienzo del bucle, y se salta las instrucciones restantes

Entre las instrucciones condicionales soportadas por SQL están las instrucciones if-then-else con la sintaxis:

www.detodoprogramacion.com

if *r.saldo* < 1000
  then set $l = l +$ *r.saldo*
elseif *r.saldo* < 5000
  then set $m = m +$ *r.saldo*
else set $h = h +$ *r.saldo*
end if

Este código da por supuesto que l, m y h son variables enteras y que r es una variable de fila. Si se sustituye la línea "**set** $n = n +$ *r.saldo*" del bucle **for** del párrafo anterior por el código **if-then-else** (y se proporcionan las declaraciones y los valores iniciales correspondientes de l, m y h), el bucle calculará los saldos totales de las cuentas que se hallan en las categorías de saldos bajo, medio y alto, respectivamente.

SQL también soporta instrucciones **case** parecidas a la instrucción **case** del lenguaje C/C++ (además de las expresiones case, que se vieron en el Capítulo 3).

Finalmente, SQL incluye los conceptos de señalización de las **condiciones de excepción** y de declarar los **manejadores** que pueden tratar esa excepción, como en este código:

declare *agotado* **condition**
declare exit handler for *agotado*
begin
. . .
end

Las instrucciones entre **begin** y **end** pueden provocar una excepción ejecutando **signal** *agotado*. El manejador dice que si se da la condición, la acción que hay que tomar es salir de la instrucción **begin end** circundante. Una acción alternativa sería **continue**, que continúa con la ejecución a partir de la siguiente instrucción que ha generado la excepción. Además de las condiciones definidas de manera explícita, también hay condiciones predefinidas como **sqlexception**, **sqlwarning** y **not found**.

La Figura 4.10 proporciona un ejemplo de mayor tamaño del uso de constructores procedimentales en SQL. El procedimiento *retirar* definido en la figura retira dinero de una cuenta y, si el saldo pasa a ser negativo, comienza el tratamiento del descubierto; no se muestra el código para el tratamiento del descubierto. La función devuelve un código de error; con un valor mayor o igual que cero indica éxito, mientras que un valor negativo indica una condición de error.

Otro ejemplo que ilustra los bucles **while** se presenta más adelante, en el Apartado 4.7.

4.6.3 Rutinas en otros lenguajes

SQL permite definir funciones en lenguajes de programación como Java, C#, C o C++. Las funciones definidas de esta manera pueden ser más eficientes que las definidas en SQL, y cálculos que no pueden llevarse a cabo en SQL pueden ejecutarse mediante estas funciones. Un ejemplo del uso de este tipo de funciones es llevar a cabo un cálculo aritmético complejo sobre los datos de una tupla.

Los procedimientos y funciones externos pueden especificarse de esta manera:

create procedure proc_recuento_cuentas(**in** *nombre_cliente* **varchar**(20),
                                out count **integer**)
language C
external name '/usr/avi/bin/proc_recuento_cuentas'

create function recuento_cuentas (*nombre_cliente* **varchar**(20))
returns integer
language C
external name '/usr/avi/bin/recuento_cuentas'

Los procedimientos del lenguaje externo tienen que tratar con los valores nulos y con las excepciones. Por tanto, deben tener varios parámetros adicionales: un valor **sqlstate** para indicar el estado de éxito o de fracaso, un parámetro para almacenar el valor devuelto por la función y variables indicadoras para el resultado de cada parámetro o función, para indicar si su valor es nulo. Una línea adicional **parameter**

www.detodoprogramacion.com

```
create procedure retirar(
                in número_cuenta varchar(10)
                in importe numeric(12,2))
– – retira dinero de una cuenta
returns integer
begin
    declare nuevosaldo numeric(12,2);
    select saldo into nuevosaldo
    from cuenta
    where cuenta.número_cuenta = retirar.número_cuenta;
    nuevosaldo = nuevosaldo − importe;
    if (nuevosaldo < 0)
        begin
            ... el código para manejar el descubierto se inserta aquí
            ... si el importe es demasiado elevado para que lo maneje descubierto
            ... devuelve el código de error −1
        end
    else begin
    update cuenta
        set saldo = saldo − nuevosaldo
        where cuenta.número_cuenta = retirar.número_cuenta
    end
    return(0);
end
```

Figura 4.10 Procedimiento para la retirada de dinero de las cuentas.

style general añadida a la declaración anterior indica que los procedimientos o funciones externos sólo toman los argumentos mostrados y no tratan con valores nulos ni excepciones.

Las funciones definidas en un lenguaje de programación y compiladas fuera del sistema de base de datos pueden cargarse y ejecutarse con el código del sistema de bases de datos. Sin embargo, ello conlleva el riesgo de que un fallo del programa corrompa las estructuras internas de la base de datos y pueda saltarse la funcionalidad de control de accesos del sistema de bases de datos. Los sistemas de bases de datos que se preocupan más del rendimiento eficiente que de la seguridad pueden ejecutar los procedimientos de este modo. Los sistemas de bases de datos que están preocupados por la seguridad pueden ejecutar ese código como parte de un proceso diferente, comunicarle el valor de los parámetros y recuperar los resultados mediante la comunicación entre procesos. Sin embargo, el tiempo añadido que supone la comunicación entre los procesos es bastante elevado, en las arquitecturas de CPU habituales, decenas a centenares de millares de instrucciones se pueden ejecutar en el tiempo necesario para una comunicación entre los procesos.

Si el código está escrito en un lenguaje "seguro" como Java o C#, cabe otra posibilidad: ejecutar el código en un **cubo** dentro del propio proceso de ejecución de la consulta a la base de datos. El cubo permite que el código de Java o de C# tenga acceso a su propia área de memoria, pero evita que ese código lea o actualice la memoria del proceso de ejecución de la consulta, o tenga acceso a los archivos del sistema de archivos (la creación de cubos no es posible en lenguajes como C, que permite el acceso sin restricciones a la memoria mediante los punteros). El evitar la comunicación entre procesos reduce enormemente la sobrecarga de llamadas a funciones.

Varios sistemas de bases de datos actuales soportan que las rutinas de lenguajes externos se ejecuten en cubos dentro del proceso de ejecución de las consultas. Por ejemplo, Oracle y DB2 de IBM permiten que las funciones de Java se ejecuten como parte del proceso de la base de datos. SQL Server 2005 de Microsoft permite que los procedimientos compilados en CLR (Common Language Runtime) se ejecuten dentro del proceso de la base de datos; esos procedimientos pueden haberse escrito, por ejemplo, en C# o en Visual Basic.

www.detodoprogramacion.com

nombre_empleado	*nombre_jefe*
Alández	Bariego
Bariego	Erice
Corisco	Dalma
Dalma	Santos
Erice	Santos
Santos	Marchamalo
Rienda	Marchamalo

Figura 4.11 La relación *jefe*.

4.7 Consultas recursivas**

Considérese una base de datos que contenga información sobre los empleados de una empresa. Supóngase que se tiene una relación *jefe*(*nombre_empleado, nombre jefe*), que especifica qué empleado es supervisado directamente por cada jefe. La Figura 4.11 muestra un ejemplo de la relación *jefe*.

Supóngase ahora que se desea determinar los empleados que son supervisados, directa o indirectamente por un jefe dado—por ejemplo, Santos. Es decir, se desea determinar los empleados supervisados directamente por Santos o por alguien que esté supervisado por Santos, o por alguien que esté supervisado por alguien que esté supervisado por Santos, y así sucesivamente.

Por tanto, si el jefe de Alández es Bariego y el jefe de Bariego es Erice y el jefe de Erice es Santos, entonces Alández, Bariego y Erice son los empleados supervisados por Santos.

4.7.1 Cierre transitivo mediante iteración

Una manera de escribir la consulta anterior es emplear la iteración: En primer lugar hay que determinar las personas que trabajan bajo Santos, luego las que trabajan bajo el primer conjunto, etc. La Figura 4.12 muestra la función *buscaEmpl*(*jef*) para llevar a cabo esa tarea; la función toma el nombre del jefe como parámetro (*jef*), calcula el conjunto de todos los subordinados directos e indirectos de ese jefe y devuelve el conjunto.

El procedimiento utiliza tres tablas temporales: *empl*, que se utiliza para almacenar el conjunto de tuplas que se va a devolver; *empnuevo*, que almacena los empleados localizados en la iteración anterior; y *temp*, que se utiliza como almacenamiento temporal mientras se manipulan los conjuntos de empleados. El procedimiento inserta todos los empleados que trabajan directamente para *jef* en *empnuevo* antes del bucle **repeat**. El bucle **repeat** añade primero todos los empleados de *empnuevo* a *empl*. Luego calcula los empleados que trabajan para los que están en *empnuevo*, excepto los que ya se sabe que son empleados de *jef* y los almacena en la tabla temporal *temp*. Finalmente, sustituye el contenido de *empnuevo* por el de *temp*. El bucle **repeat** termina cuando no encuentra más subordinados (indirectos) nuevos.

La Figura 4.13 muestra los empleados que se encontrarían en cada iteración si el procedimiento se llamara para el jefe llamado Santos.

Hay que destacar que el uso de la cláusula **except** en la función garantiza que la función trabaje incluso en el caso (anormal) de que haya un ciclo en la jerarquía de jefes. Por ejemplo, si *a* trabaja para *b*, *b* trabaja para *c* y *c* trabaja para *a*, hay un ciclo.

Aunque los ciclos pueden resultar poco realistas en el control de organigramas, son posibles en otras aplicaciones. Por ejemplo, supóngase que se tiene la relación *vuelos*(*a, desde*) que indica las ciudades a las que se puede llegar desde otra ciudad mediante un vuelo directo. Se puede escribir un código parecido al de la función *buscaEmpl* para determinar todas las ciudades a las que se puede llegar mediante una serie de uno o más vuelos desde una ciudad dada. Todo lo que hay que hacer es sustituir *jefe* por *vuelo* y el nombre de los atributos de manera acorde. En esta situación sí que puede haber ciclos de alcanzabilidad, pero la función trabajaría correctamente, ya que eliminaría las ciudades que ya se hubieran tomado en cuenta.

www.detodoprogramacion.com

```
create function buscaEmpl(jef char(10))
    – – Busca todos los empleados que trabajan directa o indirectamente
    – – para jef
returns table (nombre char(10))
    – – La relación jefe(nombre_empleado, nombre_jefe)
    – – especifica quién trabaja directamente para quién.
begin
    create temporary table empl(nombre char(10));
        – – la tabla empl almacena el conjunto de empleados que
        – – se va a devolver
    create temporary table empnuevo(nombre char(10));
        – – la tabla empnuevo contiene los empleados hallados
        – – en la iteración anterior
    create temporary table temp(nombre char(10));
        – – la tabla temp es una tabla temporal utilizada
        – – para almacenar los resultados intermedios
    insert into empnuevo
        select nombre_empleado
        from jefe
        where nombre_jefe = jef
    repeat
        insert into empl
            select nombre
            from empnuevo;

        insert into temp
            (select jefe.nombre_empleado
                from empnuevo, jefe
                where empnuevo.nombre_empleado = jefe.nombre_jefe;
            )
            except (
                select nombre_empleado
                from empl
            );
        delete from empnuevo;
        insert into empnuevo
            select *
            from temp;
        delete from temp;

    until not exists (select * from empnuevo)
    end repeat;
    return table empl
end
```

Figura 4.12 Búsqueda de todos los empleados de cada jefe.

4.7.2 Recursión en SQL

El **cierre transitivo** de la relación *jefe* es una relación que contiene todos los pares (*emp*, *jef*) tales que *emp* es subordinado directo o indirecto de *jef*. Hay numerosas aplicaciones que exigen el cálculo de cierres transitivos parecidos sobre las **jerarquías**. Por ejemplo, las organizaciones suelen constar de varios niveles de unidades organizativas. Las máquinas constan de componentes que, a su vez, tienen subcomponentes, etc.; por ejemplo, una bicicleta puede tener subcomponentes como las ruedas y los pedales

www.detodoprogramacion.com

Número de iteración	*Tuplas de empl*
0	
1	(Dalma), (Erice)
2	(Dalma), (Erice), (Bariego), (Corbacho)
3	(Dalma), (Erice), (Bariego), (Corbacho), (Alández)
4	(Dalma), (Erice), (Bariego), (Corbacho), (Alández)

Figura 4.13 Empleados de Santos en las iteraciones de la función *buscaEmpl.*

que, a su vez, tienen subcomponentes como las cámaras, las llantas y los radios. El cierre transitivo puede utilizarse sobre esas jerarquías para determinar, por ejemplo, todos los componentes de la bicicleta.

Resulta muy poco práctico especificar el cierre transitivo empleando la iteración. Hay un enfoque alternativo, el uso de las definiciones recursivas de las vistas, que resulta más sencillo de utilizar.

Se puede emplear la recursión para definir el conjunto de empleados controlados por un jefe determinado, digamos Santos, del modo siguiente. Las personas supervisadas (directa o indirectamente) por Santos son:

1. Personas cuyo jefe es Santos.
2. Personas cuyo jefe está supervisado (directa o indirectamente) por Santos.

Obsérvese que el caso 2 es recursivo, ya que define el conjunto de personas supervisadas por Santos en términos del conjunto de personas supervisadas por Santos. Otros ejemplos de cierre transitivo, como la búsqueda de todos los subcomponentes (directos o indirectos) de un componente dado también puede definirse de manera parecida, recursivamente.

Desde la versión SQL:1999 la norma de SQL soporta una forma limitada de recursión, que emplea la cláusula **with recursive**, en la que las vistas (o las vistas temporales) se expresan en términos de sí mismas. Se pueden utilizar consultas recursivas, por ejemplo, para expresar de manera concisa el cierre transitivo. Recuérdese que la cláusula **with** se emplea para definir una vista temporal cuya definición sólo está disponible para la consulta en la que se define. La palabra clave adicional **recursive** especifica que la vista es recursiva.

Por ejemplo, se pueden buscar todos los pares (*emp*, *jef*) tales que *emp* esté directa o indirectamente mandado por *jef*, empleando la vista recursiva de SQL que aparece en la Figura 4.14.

Todas las vistas recursivas deben definirse como la unión de dos subconsultas: una **consulta básica** que no es recursiva y una **consulta recursiva** que utiliza la vista recursiva. En el ejemplo de la Figura 4.14 la consulta básica es la selección sobre *jefe*, mientras que la consulta recursiva calcula la reunión de *jefe* y *empl*.

El significado de las vistas recursivas se comprende mejor de la manera siguiente. En primer lugar hay que calcular la consulta básica y añadir todas las tuplas resultantes a la vista (que se halla inicialmente vacía). A continuación hay que calcular la consulta recursiva empleando el contenido actual de la vista y volver a añadir todas las tuplas resultantes a la vista. Hay que seguir repitiendo este paso hasta que no

```
with recursive empl(nombre_empleado, nombre_jefe) as (
        select nombre_empleado, nombre_jefe
        from jefe
    union
        select jefe.nombre_empleado, empl.nombre_jefe
        from jefe, empl
        where jefe.nombre_jefe = empl.nombre_empleado
    )
select ∗
from empl
```

Figura 4.14 Consulta recursiva en SQL.

www.detodoprogramacion.com

se añada ninguna tupla nueva a la vista. La instancia resultante de la vista se denomina **punto fijo** de la definición recursiva de la vista. (El término "fijo" hace referencia al hecho de que ya no se producen más modificaciones). La vista, por tanto, se define para que contenga exactamente las tuplas de la instancia del punto fijo.

Aplicando la lógica anterior a nuestro ejemplo, se hallan primero todos los subordinados directos de cada jefe ejecutando la consulta básica. La consulta recursiva añade un nivel de empleados en cada iteración, hasta alcanzar la profundidad máxima de la relación jefe-empleado. En ese punto ya no se añaden nuevas tuplas a la vista y la iteración ha alcanzado un punto fijo.

Obsérvese que no se exige que el sistema de bases de datos utilice esta técnica iterativa para calcular el resultado de la consulta recursiva; puede obtener el mismo resultado empleando otras técnicas que pueden ser más eficientes.

Hay algunas restricciones para la consulta recursiva de la vista recursiva, concretamente, la consulta debe ser **monótona**, es decir, su resultado sobre la instancia de una relación vista V_1 debe ser un superconjunto de su resultado sobre la instancia de una relación vista V_2 si V_1 es un superconjunto de V_2. De manera intuitiva, si se añaden más tuplas a la relación vista, la consulta recursiva debe devolver, como mínimo, el mismo conjunto de tuplas que antes, y posiblemente devuelva tuplas adicionales.

En concreto, las consultas recursivas no deben emplear ninguno de los constructores siguientes, ya que harían que la consulta no fuera monótona:

- Agregación sobre la vista recursiva.
- **not exists** sobre una subconsulta que utiliza la vista recursiva.
- Diferencia de conjuntos (**except**) cuyo lado derecho utilice la vista recursiva.

Por ejemplo, si la consulta recursiva fuera de la forma $r - v$, donde v es la vista recursiva, si se añade una tupla a v, el resultado de la consulta puede hacerse más pequeño; la consulta, por tanto, no es monótona.

El significado de las vistas recursivas puede definirse mediante el procedimiento recursivo mientras la consulta recursiva sea monótona; si la consulta recursiva no es monótona, el significado de la consulta resulta difícil de definir. SQL, por tanto, exige que las consultas sean monótonas. Las consultas recursivas se estudian con más detalle en el contexto del lenguaje de consultas Datalog, en el Apartado 5.4.6.

SQL también permite la creación de vistas permanentes definidas de manera recursiva mediante el uso de **create recursive view**, en lugar de **with recursive**. Algunas implementaciones soportan consultas recursivas que emplean una sintaxis diferente; consúltese el manual del sistema correspondiente para conocer más detalles.

4.8 Características avanzadas de SQL**

El lenguaje SQL ha crecido durante las dos últimas décadas desde ser un lenguaje sencillo con pocas características a ser un lenguaje bastante complejo con características para satisfacer a muchos tipos diferentes de usuarios. En este capítulo ya se han tratado los fundamentos de SQL. En este apartado se presentan al lector algunas de las características más complejas de SQL. Muchas de estas características se han introducido en versiones de la norma de SQL relativamente recientes y puede que no las soporten todos los sistemas de bases de datos.

4.8.1 Creación de tablas a partir de otras

Las aplicaciones necesitan a menudo la creación de tablas que tengan el mismo esquema que otra tabla ya existente. SQL proporciona la extensión **create table like** para soportar esta tarea:

create table *cuenta_temp* **like** *cuenta*

La instrucción anterior crea una nueva tabla *cuenta_temp* que tiene el mismo esquema que *cuenta*.

Al escribir consultas complejas suele resultar útil almacenar el resultado de la consulta en forma de nueva tabla; esa tabla suele ser temporal. Hacen falta dos instrucciones, una para crear la tabla (con las columnas correspondientes) y otra para insertar el resultado de la consulta en la tabla. SQL:2003

www.detodoprogramacion.com

proporciona una técnica más sencilla para crear una tabla que contenga el resultado de una consulta. Por ejemplo, la instrucción siguiente crea la tabla *t1*, que contiene el resultado de una consulta.

```
create table t1 as
        (select *
        from cuenta
        where nombre_sucursal = 'Navacerrada')
with data
```

De manera predeterminada, el nombre y el tipo de datos de las columnas se deduce del resultado de la consulta. Se puede poner nombre a las columnas de manera explícita escribiendo la lista de los nombres de las columnas tras el nombre de la relación. Si se omite la cláusula **with data**, la tabla se crea, pero no se rellena con datos.

La instrucción **create table ... as** anterior se parece mucho a la instrucción **create view**, y las dos se definen mediante consultas. La principal diferencia es que el contenido de la tabla se define al crearla, mientras que el contenido de una vista siempre refleja el resultado actual de la consulta.

Hay que tener en cuenta que varias implementaciones soportan la funcionalidad de **create table ... like** y **create table ... as** pero emplean sintaxis diferentes; consúltese el manual del sistema respectivo para obtener más detalles.

4.8.2 Otros aspectos sobre las subconsultas

SQL:2003 permite que las subconsultas se produzcan siempre que haga falta un valor, siempre y cuando la consulta sólo devuelva un valor; estas subconsultas se denominan **subconsultas escalares**. Por ejemplo, se puede utilizar una subconsulta en la cláusula **select** como se ilustra en el ejemplo siguiente, que genera una lista de todos los clientes y del número de cuentas que tienen:

```
select nombre_cliente,
           (select count(*)
            from cuenta
            where cuenta.nombre_cliente = cliente.nombre_cliente) as número_cuentas
from cliente
```

Está garantizado que la subconsulta del ejemplo anterior sólo devuelva un único valor, ya que tiene un agregado **count**(*) sin el correspondiente **group by**. Las subconsultas sin agregados también están permitidas. Estas consultas pueden, en teoría, devolver más de una respuesta; si lo hacen, se produce un error en tiempo de ejecución.

Las subconsultas de la cláusula **select** de la consulta exterior pueden tener acceso a los atributos de las relaciones de la cláusula **from** de la consulta exterior, como *cliente.nombre_cliente* en este ejemplo.

Las subconsultas de la cláusula **from** (que ya se ha estudiado en el Apartado 3.8.1), sin embargo, normalmente no pueden tener acceso a los atributos de otras relaciones de la cláusula **from**; SQL:2003 soporta la cláusula **lateral**, que permite que las subconsultas de la cláusula **from** tengan acceso a los atributos de las subconsultas anteriores de esa misma cláusula **from**. Por tanto, la consulta anterior puede escribirse también del modo siguiente

```
select nombre_cliente, número_cuentas
    from cliente,
        lateral(select count(*)
                from cuenta
                where cuenta.nombre_cliente = cliente.nombre_cliente)
            as este_cliente(número_cuentas)
```

4.8.3 Constructores avanzados para la actualización de las bases de datos

Supóngase que se tiene la relación *fondos_recibidos*(*número_cuenta*, *importe*) que almacena los fondos recibidos (digamos, mediante transferencias electrónicas) para cada cuenta de un grupo. Supóngase ahora

www.detodoprogramacion.com

que se desea añadir los importes a los saldos de las cuentas correspondientes. Para utilizar la actualización **update** de SQL para llevar a cabo esta tarea hay que examinar la tabla *fondos_recibidos* para cada tupla de la tabla *cuenta*. Se pueden utilizar subconsultas en la cláusula update para llevar a cabo esta tarea, como se muestra a continuación. En aras de la sencillez se supone que la relación *fondos_recibidos* contiene, como máximo, una tupla por cada cuenta.

```
update cuenta set saldo = saldo +
                (select importe
                 from fondos_recibidos
                 where fondos_recibidos.número_cuenta = cuenta.número_cuenta)
    where exists(
                select *
                 from fondos_recibidos
                 where fondos_recibidos.número_cuenta = cuenta.número _cuenta)
```

Téngase en cuenta que la condición de la cláusula **where** de la actualización garantiza que sólo se actualicen las cuentas con sus tuplas correspondientes en *fondos_recibidos*, mientras que la subconsulta de la cláusula **set** calcula el montante que hay que añadir a cada una de esas cuentas.

Hay muchas aplicaciones que necesitan actualizaciones como la que se acaba de ilustrar. Generalmente hay una tabla, que se puede llamar **tabla maestra**, y las actualizaciones de la tabla maestra se reciben por lotes. Luego hay que actualizar a su vez la tabla maestra. SQL:2003 proporciona un constructor especial, denominado **merge**, para simplificar la labor de mezcla de toda esa información. Por ejemplo, la actualización anterior puede expresarse empleando **merge** de la manera siguiente:

```
merge into cuenta as C
using     (select *
           from fondos_recibidos) as F
      on (C.número_cuenta = F.número_cuenta)
when matched then
      update set saldo = saldo+F.importe
```

Cuando un registro de la subconsulta de la cláusula **using** coincide con un registro de la relación *cuenta*, se ejecuta la cláusula **when matched**, que puede ejecutar una actualización de la relación; en este caso, el registro coincidente de la relación *cuenta* se actualiza de la manera mostrada.

La instrucción **merge** también puede tener una cláusula **when not matched then**, que permite la inserción de registros nuevos en la relación. En el ejemplo anterior, cuando no hay cuenta coincidente para alguna tupla de *fondos_recibidos*, la acción de inserción puede crear un nuevo registro de *cuenta* (con un valor nulo para *nombre_sucursal*) utilizando la cláusula siguiente.

```
when not matched then
      insert values (F.número_cuenta, null, F.importe)
```

Aunque no sea muy significativo en este ejemplo[2], la cláusula **when not matched clause** puede resultar bastante útil en otros casos. Por ejemplo, supóngase que la relación local es copia de una relación maestra y se reciben de la relación maestra registros actualizados y otros recién insertados. La instrucción **merge** puede actualizar los registros coincidentes (serán registros antiguos actualizados) e insertar los registros que no coincidan (serán registros nuevos).

No todas las implementaciones de SQL soportan actualmente la instrucción **merge**; consúltese el manual del sistema correspondiente para obtener más detalles.

2. Una solución mejor en este caso habría sido insertar estos registros en una relación de error, pero eso no se puede hacer con la instrucción **merge**.

www.detodoprogramacion.com

CAPÍTULO 11

Almacenamiento y estructura de archivos

En los capítulos anteriores se han estudiado los modelos de bases de datos de alto nivel. Por ejemplo, en el nivel *conceptual* o *lógico* se ha presentado una bases de datos del modelo relacional como un conjunto de tablas. En realidad, el modelo lógico de las bases de datos es el mejor nivel para que se centren los *usuarios*. Esto se debe a que el objetivo de los sistemas de bases de datos es simplificar y facilitar el acceso a los datos; no se debe agobiar innecesariamente a quienes utilizan el sistema con los detalles físicos de su implementación.

En este capítulo, no obstante, así como en los Capítulos 12, 13 y 14, se analizan niveles inferiores y se describen diferentes métodos de implementación de los modelos de datos y de los lenguajes presentados en capítulos anteriores. Se comienza con las características de los medios de almacenamiento subyacentes, como sistemas de disco y de cinta. Más adelante se definen varias estructuras de datos que permiten un acceso rápido a los datos. Se consideran varias arquitecturas alternativas, idóneas para diferentes tipos de acceso a los datos. La elección final de la estructura de datos hay que hacerla en función del uso que se espera dar al sistema y de las características de cada máquina concreta.

11.1 Visión general de los medios físicos de almacenamiento

La mayoría de sistemas informáticos presentan varios tipos de almacenamientos de datos. Estos medios se clasifican según la velocidad con la que se puede tener acceso a los datos, su coste de adquisición por unidad de datos y su fiabilidad. Entre los medios disponibles habitualmente figuran:

- **Caché**. La caché es el mecanismo de almacenamiento más rápido y costoso. La memoria caché es pequeña; su uso lo gestiona el hardware del sistema informático. Los sistemas de bases de datos no la gestionan.
- **Memoria principal**. El medio de almacenamiento utilizado para los datos con los que se opera es la memoria principal. Las instrucciones máquina operan en la memoria principal. Aunque la memoria principal puede contener muchos megabytes de datos (un PC normal viene con 512 megabytes, como mínimo), o incluso cientos de gigabytes de datos en grandes sistemas servidores suele ser demasiado pequeña (o demasiado cara) para guardar toda la base de datos. El contenido de la memoria principal suele perderse en caso de fallo del suministro eléctrico o de caída del sistema.
- **Memoria flash**. La memoria flash se diferencia de la memoria principal en que los datos no se pierden en caso de que se produzca un corte de la alimentación. La lectura de los datos de la memoria flash emplea menos de cien nanosegundos (un nanosegundo es una milésima de microsegundo), más o menos igual de rápida que la lectura de los datos de la memoria principal. Sin embargo, la escritura de los datos en la memoria flash resulta más complicada (los datos pueden escribirse una vez, lo que tarda de cuatro a diez microsegundos, pero no se pueden sobrescribir

www.detodoprogramacion.com

de manera directa). Para sobrescribir la memoria es necesario borrar simultáneamente todo un banco de memoria, el cual queda preparado para volver a escribir en él. Un inconveniente de la memoria flash es que sólo permite un número limitado de ciclos de borrado, que varía entre diez mil y un millón. Es un tipo de *memoria sólo de lectura programable y borrable eléctricamente* (*electrically erasable programmable read-only memory*, *EEPROM*); otras variantes de EEPROM permiten el borrado y reescritura de posiciones individuales de la memoria, pero su empleo no está tan difundido.

La memoria flash se ha hecho popular como sustituta de los discos magnéticos para el almacenamiento de pequeños volúmenes de datos (en el año 2005, normalmente menos de un gigabyte, aunque ya hay memorias flash de mayor capacidad que pueden almacenar varios gigabytes) en los sistemas informáticos de coste reducido que se empotran en otros dispositivos, en computadoras de mano y en otros dispositivos electrónicos digitales como las cámaras digitales (a principios del año 2005, 256 megabytes de memoria flash para cámaras tenían un coste menor de 25 €, mientras que un gigabyte tenía un coste menor de 100 €). La memoria flash se utiliza también en las "llaves USB", que se pueden conectar a los puertos USB (**Universal Serial Bus**, bus serie universal) de los dispositivos informáticos. Estas llaves USB son muy populares para el transporte datos entre sistemas informáticos (los "disquetes" desempeñaron el mismo papel en su momento, pero han quedado obsoletos debido a su capacidad limitada).

- **Almacenamiento en discos magnéticos**. El principal medio de almacenamiento persistente en conexión es el disco magnético. Generalmente se guarda en ellos toda la base de datos. Para acceder a los datos es necesario trasladarlos desde el disco a la memoria principal. Después de realizar la operación deseada se deben escribir en el disco los datos que se hayan modificado.

  El tamaño de los discos magnéticos actuales varía entre unos pocos gigabytes y 400 gigabytes. Un disco de 250 gigabytes tiene un coste en 2005 alrededor de 160 €. Los dos extremos de este rango han crecido a un ritmo cercano al cincuenta por ciento anual, y cada año se pueden esperar discos de mucha mayor capacidad. El almacenamiento en disco resiste los fallos del suministro eléctrico y las caídas del sistema. Los propios dispositivos de almacenamiento en disco pueden fallar a veces y, en consecuencia, destruir los datos, pero estos fallos se producen con mucha menos frecuencia que las caídas del sistema.

- **Almacenamiento óptico**. La forma más popular de almacenamiento óptico es el disco compacto (*Compact Disk*, CD), que puede almacenar alrededor de 700 megabytes de datos y un tiempo de reproducción de 80 minutos, y el *disco de vídeo digital* (*Digital Video Disk*, DVD), que puede almacenar 4,7 u 8,5 gigabytes de datos en cada cara del disco (o hasta 17 gigabytes en un disco de doble cara). También se utiliza el término **disco digital versátil** en lugar de **disco de vídeo digital**, ya que los DVD pueden almacenar cualquier tipo de dato digital, no sólo datos de vídeo. Los datos se almacenan ópticamente en el disco y se leen mediante un láser.

  No se puede escribir en los discos ópticos empleados como discos compactos de sólo lectura (CD-ROM) o como discos de vídeo digital de sólo lectura (DVD-ROM), pero se suministran con datos pregrabados. Existen también versiones "para una sola grabación" de los discos compactos (denominados CD-R) y de los discos de vídeo digital (DVD-R y DVD+R) en los que sólo se puede escribir una vez; estos discos también se denominan **de escritura única y lectura múltiple** (**write-once, read-many**, WORM). Existen también versiones "para escribir varias veces" de los discos compactos (llamada CD-RW) y de los discos de vídeo digital (DVD-RW, DVD+RW y DVD-RAM), en los que se puede escribir varias veces.

  Los **cambiadores automáticos** (**jukebox**) de discos ópticos contienen varias unidades y numerosos discos que pueden cargarse de manera automática en las diferentes unidades (mediante un brazo robotizado) a petición del usuario.

- **Almacenamiento en cinta**. El almacenamiento en cinta se utiliza principalmente para copias de seguridad y datos archivados. Aunque la cinta magnética es más barata que los discos, el acceso a los datos resulta mucho más lento, ya que hay que acceder secuencialmente desde el comienzo de la cinta. Por este motivo, se dice que el almacenamiento en cinta es de **acceso secuencial**,

www.detodoprogramacion.com

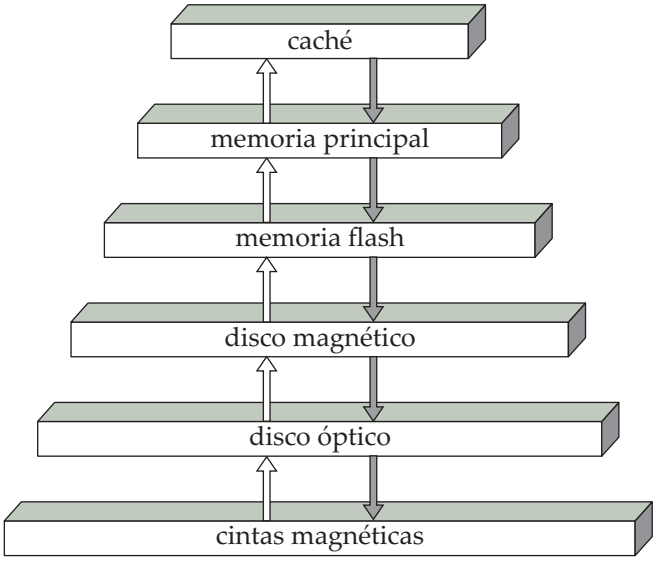


Figura 11.1 Jerarquía de los dispositivos de almacenamiento.

mientras que el almacenamiento en disco es de **acceso directo**, ya que se pueden leer datos de cualquier parte del disco.

Las cintas poseen una capacidad elevada (actualmente se dispone de cintas de 40 a 300 gigabytes) y pueden retirarse de la unidad de lectura, por lo que resultan idóneas para el almacenamiento de archivos con coste reducido. Los cambiadores automáticos de cinta (jukeboxes) se utilizan para guardar conjuntos de datos excepcionalmente grandes, como los datos obtenidos mediante satélite, que pueden ocupar centenares de terabytes (10^{12} bytes), o incluso petabytes (10^{15} bytes) en algunos casos.

Los diferentes medios de almacenamiento se pueden organizar de forma jerárquica (Figura 11.1) de acuerdo con su velocidad y su coste. Los niveles superiores resultan caros, pero son rápidos. A medida que se desciende por la jerarquía disminuye el coste por bit, pero aumenta el tiempo de acceso. Este compromiso es razonable; si un sistema de almacenamiento dado fuera a la vez más rápido y menos costoso que otro (en igualdad del resto de condiciones) no habría ninguna razón para utilizar la memoria más lenta y más cara. De hecho, muchos dispositivos de almacenamiento primitivos, como la cinta de papel y las memorias de núcleos de ferrita, se hallan relegados a los museos ahora que la cinta magnética y la memoria de semiconductores son más rápidas y baratas. Las propias cintas magnéticas se utilizaban para guardar los datos activos cuando los discos resultaban costosos y tenían una capacidad de almacenamiento reducida. Hoy en día casi todos los datos activos se almacenan en disco, excepto en los casos excepcionales en que se guardan en cambiadores automáticos de cinta u ópticos.

Los medios de almacenamiento más rápidos (por ejemplo, la caché y la memoria principal) se denominan **almacenamiento primario**. Los medios del siguiente nivel de la jerarquía (por ejemplo, los discos magnéticos) se conocen como **almacenamiento secundario** o **almacenamiento en conexión**. Los medios del nivel inferior de la jerarquía—por ejemplo, la cinta magnética y los cambiadores automáticos de discos ópticos—se denominan **almacenamiento terciario** o **almacenamiento sin conexión**.

Además de la velocidad y del coste de los diferentes sistemas de almacenamiento hay que tener en cuenta la volatilidad del almacenamiento. El **almacenamiento volátil** pierde su contenido cuando se suprime el suministro eléctrico del dispositivo. En la jerarquía mostrada en la Figura 11.1 los sistemas de almacenamiento desde la memoria principal hacia arriba son volátiles, mientras que los sistemas de almacenamiento por debajo de la memoria principal son no volátiles. Los datos se deben escribir en **almacenamiento no volátil** por motivos de seguridad. Este asunto se volverá a tratar en el Capítulo 17.

www.detodoprogramacion.com

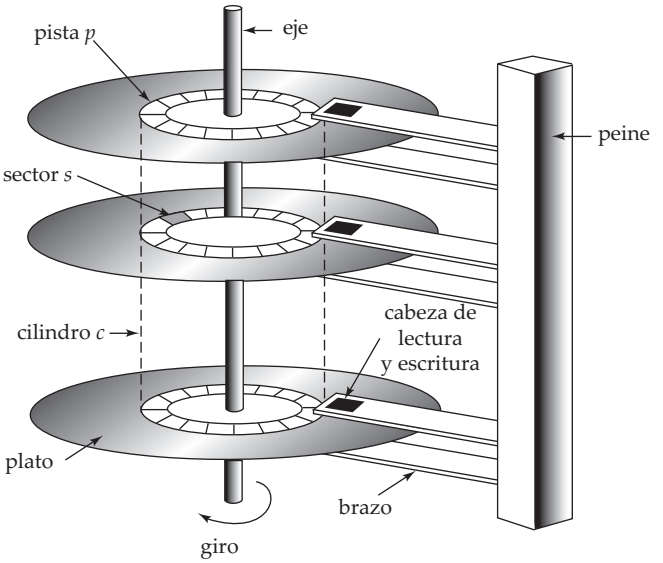


Figura 11.2 Mecanismo de disco de cabezas móviles.

11.2 Discos magnéticos

Los discos magnéticos proporcionan la mayor parte del almacenamiento secundario de los sistemas informáticos modernos. Aunque la capacidad de los discos ha crecido año tras año, los requisitos de almacenamiento de las grandes aplicaciones también han crecido muy rápido; en algunos casos, más que la capacidad de los discos. Una base de datos de gran tamaño puede necesitar centenares de discos.

11.2.1 Características físicas de los discos

Físicamente, los discos son relativamente sencillos (Figura 11.2). Cada **plato** del disco es de forma circular plana. Sus dos superficies están cubiertas por un material magnético en el que se graba la información. Los platos están hechos de metal rígido o de vidrio.

Mientras se utiliza el disco, un motor lo hace girar a una velocidad constante elevada (generalmente sesenta, noventa o ciento veinte revoluciones por segundo, aunque existen discos que giran a doscientas cincuenta revoluciones por segundo). Una cabeza de lectura y escritura está colocada justo encima de la superficie del plato. La superficie del disco se divide a efectos lógicos en **pistas**, que se subdividen en **sectores**. Un **sector** es la unidad mínima de información que se puede leer o escribir en el disco. En los discos actuales, el tamaño de los sectores suele ser de quinientos doce bytes; hay entre cincuenta mil y cien mil pistas en cada plato y de uno a cinco platos por disco. Las pistas internas (las más cercanas al eje) son más cortas, y en los discos actuales, las pistas exteriores contienen más sectores que las internas; suele haber unos quinientos sectores por pista en las pistas internas y alrededor de mil en las externas. Estos números pueden variar de un modelo a otro; los discos de mayor capacidad tienen más sectores por pista y más pistas en cada plato.

La **cabeza de lectura y escritura** guarda la información en los sectores en forma de inversiones de la dirección de magnetización del material magnético.

Cada cara de un plato del disco posee una cabeza de lectura y escritura que se desplaza por el plato para tener acceso a las diferentes pistas. El disco suele contener muchos platos y las cabezas de lectura y escritura de todas las pistas están montadas en un único dispositivo denominado **brazo del disco** y se mueven conjuntamente. El conjunto de los platos del disco montados sobre un mismo eje y de las cabezas montadas en el brazo del disco se denomina **dispositivo cabeza–disco**. Dado que las cabezas de todos los platos se desplazan conjuntamente, cuando la cabeza se halle en la pista i-ésima de un plato, las restantes también se encontrarán en la pista i-ésima de sus platos respectivos. Por tanto, el conjunto de las pistas i-ésimas de todos los platos se denomina **cilindro** i-ésimo.

www.detodoprogramacion.com

Actualmente dominan el mercado los discos con un diámetro de plato de tres pulgadas y media. Tienen un menor coste y tiempos de búsqueda más cortos (debido al menor tamaño) que los discos de mayor tamaño (de hasta catorce pulgadas) que eran habituales en el pasado y, aun así, ofrecen gran capacidad de almacenamiento. Se usan discos de diámetro incluso menor en dispositivos móviles como las computadoras portátiles, las de mano y los reproductores de música de bolsillo.

Las cabezas de lectura y escritura se mantienen tan próximas como resulta posible a la superficie de los discos para aumentar la densidad de grabación. Las cabezas suelen flotar o volar tan sólo a micras de la superficie de cada disco; el giro del disco crea una pequeña corriente de aire y el dispositivo de las cabezas se fabrica de manera que ese flujo de aire mantenga las cabezas flotando rasantes sobre la superficie de los discos. Como las cabezas flotan tan cercanas a la superficie, los platos se deben elaborar con esmero para que sean lisos.

Los choques de las cabezas con la superficie de los platos pueden suponer un problema. Si la cabeza entra en contacto con la superficie del disco, puede arrancar el medio de grabación, lo que destruye los datos que allí hubiera. En los modelos antiguos, el contacto de la cabeza con la superficie hacía que el material arrancado flotase en el aire y se interpusiera entre las cabezas y los platos, lo que causaba más choques; por tanto, la caída de las cabezas podía dar lugar a un fallo de todo el disco. Los dispositivos actuales emplean como medio de grabación una fina capa de metal magnético. Son mucho menos susceptibles de fallar a causa del choque de las cabezas con las superficies de los platos que los antiguos discos recubiertos de óxido.

El **controlador de disco** actúa como interfaz entre el sistema informático y el hardware concreto de la unidad de disco; en los sistemas de disco modernos el controlador se implementa dentro de la unidad de disco. El controlador de disco acepta los comandos de alto nivel de lectura o escritura en un sector dado e inicia las acciones correspondientes, como el desplazamiento del brazo del disco a la pista adecuada y la lectura o escritura real de los datos. Los controladores de disco también añaden **sumas de comprobación** a cada sector en el que se escribe; las sumas de comprobación se calculan a partir de los datos escritos en ese sector. Cuando se vuelve a realizar allí una operación de lectura, se vuelve a calcular esa suma a partir de los datos recuperados y se compara con el valor guardado; si los datos se han deteriorado, resulta muy probable que la suma de comprobación recién calculada no coincida con la guardada. Si se produce un error de este tipo, el controlador volverá a intentar la lectura varias veces; si el error persiste, el controlador indica un fallo de lectura.

Otra labor interesante llevada a cabo por los controladores de disco es la **reasignación de los sectores dañados**. Si el controlador detecta que un sector está dañado cuando se da formato al disco por primera vez, o cuando se realiza un intento de escribir allí, puede reasignar lógicamente el sector a una ubicación física diferente (escogida de entre un grupo de sectores adicionales preparados con esta finalidad). La reasignación se anota en disco o en memoria no volátil y la escritura se realiza en la nueva ubicación.

Los discos se conectan a los sistemas informáticos mediante conexiones de alta velocidad. Las interfaces más comunes para la conexión de los discos a las computadoras personales y a las estaciones de trabajo son: (1) la interfaz **ATA** (**AT Attachment**) (que es una versión más rápida que la **interfaz electrónica de dispositivos integrados** (IDE, **Integrated Drive Electronics**) usada antiguamente en los PCs de IBM); (2) la nueva versión de ATA: **SATA** (**ATA serie**, **Serial ATA**; la versión original de ATA se denomina **PATA**, o Parallel ATA- ATA paralela - para distinguirla de SATA); y (3) la **interfaz de conexión para sistemas informáticos pequeños** (SCSI, **Small Computer System Interconnect**, pronunciado "escasi"). Los grandes sistemas (mainframes) y los sistemas servidores suelen disponer de interfaces más rápidas y caras, como las versiones de alta capacidad de la interfaz SCSI y la interfaz Fibre Channel. Los sistemas de discos externos portátiles utilizan generalmente las interfaces USB o FireWire.

Aunque los discos se suelen conectar directamente a la interfaz de disco de la computadora mediante cables, también pueden hallarse en una ubicación remota y conectarse mediante una red de alta velocidad al controlador de disco. En la arquitectura de **red de área de almacenamiento** (**Storage Area Network**, **SAN**) se conecta un gran número de discos a varias computadoras servidoras mediante una red de alta velocidad. Los discos suelen disponerse localmente mediante una técnica de organización de almacenamiento denominada **disposición redundante de discos independientes** (**redundant arrays of independent disks**, RAID) (que se describe en el Apartado 11.3) para proporcionar a los servidores la vista lógica de un solo disco de gran tamaño y muy fiable. La computadora y el subsistema de disco siguen usando las interfaces SCSI y Fibre Channel para comunicarse entre sí, aunque puedan estar sepa-

www.detodoprogramacion.com

rados por una red. El acceso remoto a los discos también implica que pueden compartirse entre varias computadoras que pueden ejecutar en paralelo diferentes partes de una misma aplicación. El acceso remoto también supone que los discos que contienen datos importantes se pueden guardar en una sala donde los administradores del sistema pueden supervisarlos y mantenerlos, en lugar de estar dispersos por diferentes locales de la organización.

El **almacenamiento conectado en red** (**Network Attached Storage, NAS**) es una alternativa a SAN. NAS es muy parecido a SAN, salvo que, en lugar de que el almacenamiento en red aparezca como un gran disco, proporciona una interfaz al sistema de archivos que utiliza protocolos de sistemas de archivos en red como NFS o CIFS.

11.2.2 Medidas del rendimiento de los discos

Las principales medidas de la calidad de los discos son su capacidad, su tiempo de acceso, su velocidad de transferencia de datos y su fiabilidad.

El **tiempo de acceso** es el tiempo transcurrido desde que se formula una solicitud de lectura o de escritura hasta que comienza la transferencia de datos. Para tener acceso (es decir, para leer o escribir) a los datos de un sector dado del disco, primero se debe desplazar el brazo para que se ubique sobre la pista correcta y luego hay que esperar a que el sector aparezca bajo él debido a la rotación del disco. El tiempo para volver a ubicar el brazo se denomina **tiempo de búsqueda** y aumenta con la distancia que deba recorrer. Los tiempos de búsqueda típicos varían de dos a treinta milisegundos, en función de la distancia de la pista a la posición inicial del brazo. Los discos de menor tamaño tienden a tener tiempos de búsqueda menores, dado que la cabeza tiene que recorrer una distancia menor.

El **tiempo medio de búsqueda** es el promedio de los tiempos de búsqueda medido en una sucesión de solicitudes aleatorias (uniformemente distribuidas). Si todas las pistas tienen el mismo número de sectores y se desprecia el tiempo necesario para que la cabeza comience a moverse y se detenga, se puede demostrar que el tiempo medio de búsqueda es un tercio del peor de los tiempos de búsqueda posibles. Teniendo en cuenta estos factores, el tiempo medio de búsqueda es aproximadamente la mitad del tiempo máximo de búsqueda. Los tiempos medios de búsqueda varían actualmente entre cuatro y diez milisegundos, dependiendo del modelo de disco.

Una vez que la cabeza ha alcanzado la pista deseada, el tiempo de espera hasta que el sector al que hay que acceder aparece bajo la cabeza se denomina **tiempo de latencia rotacional**. Las velocidades rotacionales de los discos varían actualmente entre cinco mil cuatrocientas rotaciones por minuto (noventa rotaciones por segundo) y quince mil rotaciones por minuto (doscientas cincuenta rotaciones por segundo) o, lo que es lo mismo, de cuatro a 11,1 milisegundos por rotación. En promedio, hace falta media rotación del disco para que aparezca bajo la cabeza el comienzo del sector deseado. Por tanto, el **tiempo de latencia medio** del disco es la mitad del tiempo empleado en una rotación completa del disco.

El tiempo de acceso es la suma del tiempo de búsqueda y del tiempo de latencia y varía de ocho a veinte milisegundos. Una vez situado bajo la cabeza el primer sector de los datos, comienza la transferencia. La **velocidad de transferencia de datos** es la velocidad a la que se pueden recuperar datos del disco o guardarlos en él. Los sistemas de disco actuales soportan velocidades máximas de transferencia de veinticinco a cien megabytes por segundo; la velocidad de transferencia medias son significativamente menores que la velocidad de transferencia máxima para las pistas interiores del disco, dado que éstas tienen menos sectores. Por ejemplo, un disco con una velocidad de transferencia máxima de cien megabytes por segundo puede tener una velocidad de transferencia continuada de alrededor de treinta megabytes por segundo en sus pistas internas.

La última medida que se estudiará de entre las utilizadas habitualmente es el **tiempo medio entre fallos**, que indica una medida de la fiabilidad del disco. El tiempo medio entre fallos de un disco (o de cualquier otro sistema) es la cantidad de tiempo que, en promedio, se puede esperar que el sistema funcione de manera continua sin tener ningún fallo. De acuerdo con las afirmaciones de los fabricantes, el tiempo medio entre fallos de los discos actuales varía entre quinientas mil y un millón doscientas mil horas (de cincuenta y siete a ciento treinta y seis años). En la práctica, el tiempo medio entre fallos anunciado se calcula en términos de la probabilidad de fallo cuando el disco es nuevo (este dato significa que, dados mil discos nuevos, si el tiempo medio entre fallos es de un millón doscientas mil horas, en promedio, uno de ellos fallará cada mil doscientas horas). Un tiempo medio entre fallos de un

www.detodoprogramacion.com

millón doscientas mil horas no implica que se pueda esperar que el disco funcione ciento treinta y seis años. La mayoría de los discos tienen una esperanza de vida de unos cinco años, y tienen tasas de fallo significativamente más altas en cuanto alcanzan cierta edad.

Las unidades de disco para las máquinas de sobremesa incorporan normalmente las interfaces PATA (Parallel ATA), las cuales proporcionan velocidades de transferencia de ciento treinta y tres megabytes por segundo, y SATA (Serial ATA), capaces de soportar ciento cincuenta megabytes por segundo. Las antiguas interfaces ATA-4 y ATA-5 soportaban velocidades de transferencia de treinta y tres y sesenta y seis megabytes por segundo, respectivamente. Las unidades de disco diseñadas para los sistemas servidores suelen soportar las interfaces Ultra320 SCSI, que permite hasta trescientos veinte megabytes por segundo, y Fibre Channel FC 2Gb, que proporciona tasas de hasta doscientos cincuenta y seis megabytes por segundo. La velocidad de transferencia de cada interfaz se comparte entre todos los discos conectados a la misma (SATA sólo permite que se conecte un disco a cada interfaz).

11.2.3 Optimización del acceso a los bloques del disco

Las solicitudes de E/S al disco las generan tanto el sistema de archivos como el gestor de la memoria virtual presente en la mayor parte de los sistemas operativos. Cada solicitud especifica la dirección del disco a la que hay que hacer referencia; esa dirección se expresa como un *número de bloque*. Un **bloque** es una unidad lógica que consiste en un número fijo de sectores contiguos. El tamaño de los bloques varía de quinientos doce bytes a varios kilobytes. Entre el disco y la memoria principal los datos se transfieren por bloques.

El acceso a los datos del disco es varios órdenes de magnitud más lento que el acceso a los datos de la memoria principal. En consecuencia, se han desarrollado diversas técnicas para mejorar la velocidad de acceso a los bloques del disco. El almacenamiento de bloques en la memoria intermedia para satisfacer las solicitudes futuras es una de estas técnicas, la cual se estudia en el Apartado 11.5; aquí se examinan otras.

- **Planificación**. Si hay que transferir varios bloques de un mismo cilindro desde el disco a la memoria principal es posible disminuir el tiempo de acceso solicitando los bloques en el orden en el que pasarán por debajo de las cabezas. Si los bloques deseados se hallan en cilindros diferentes resulta ventajoso solicitar los bloques en un orden que minimice el movimiento del brazo del disco. Los algoritmos de **planificación del brazo del disco** intentan ordenar el acceso a las pistas de manera que se aumente el número de accesos que se puedan procesar. Un algoritmo utilizado con frecuencia es el **algoritmo del ascensor**, que funciona de manera parecida a muchos ascensores. Supóngase que, inicialmente, el brazo se desplaza desde la pista más interna hacia el exterior del disco. Bajo el control del algoritmo del ascensor, el brazo se detiene en cada pista para la que haya una solicitud de acceso, atiende las peticiones para esa pista y continúa desplazándose hacia el exterior hasta que no queden solicitudes pendientes para pistas más externas. En ese punto, el brazo cambia de dirección, se desplaza hacia el interior y vuelve a detenerse en cada pista solicitada hasta que alcanza una pista en la que no haya solicitudes para pistas más cercanas al centro del disco. Entonces cambia de dirección e inicia un nuevo ciclo. Los controladores de disco suelen realizar la labor de reordenar las solicitudes de lectura para mejorar el rendimiento, dado que conocen perfectamente la organización de los bloques del disco, la posición rotacional de los platos y la posición del brazo.
- **Organización de archivos**. Para reducir el tiempo de acceso a los bloques se pueden organizar los bloques del disco de una manera que se corresponda fielmente con la forma en que se espera tener acceso a los datos. Por ejemplo, si se espera tener acceso secuencial a un archivo, se deben guardar secuencialmente en cilindros adyacentes todos los bloques del archivo. Los sistemas operativos más antiguos, como los de IBM para grandes sistemas, ofrecían a los programadores un control detallado de la ubicación de los archivos, lo que permitía reservar un conjunto de cilindros para guardar un archivo. Sin embargo, este control supone una carga para el programador o el administrador del sistema porque que tienen que decidir, por ejemplo, los cilindros que debe asignar a cada archivo. Esto puede exigir una costosa reorganización si se insertan datos en el archivo o se borran de él.

www.detodoprogramacion.com

Los sistemas operativos posteriores, como Unix y Windows, ocultan a los usuarios la organización del disco y gestionan la asignación de manera interna. Sin embargo, en el transcurso del tiempo, un archivo secuencial puede quedar **fragmentado**; es decir, sus bloques pueden quedar dispersos por el disco. Para reducir la fragmentación, el sistema puede hacer una copia de seguridad de los datos del disco y restaurarlo completamente. La operación de restauración vuelve a escribir de manera contigua (o casi) los bloques de cada archivo. Algunos sistemas (como las diferentes versiones del sistema operativo Windows) disponen de utilidades que examinan el disco y desplazan los bloques para reducir la fragmentación. Las mejoras de rendimiento obtenidas con estas técnicas son elevadas en algunos casos.

- **Memoria intermedia de escritura no volátil**. Dado que el contenido de la memoria se pierde durante los fallos de suministro eléctrico, hay que guardar en disco la información sobre las actualizaciones de las bases de datos para que superen las posibles caídas del sistema. Debido a esto, el rendimiento de las aplicaciones de bases de datos con gran cantidad de actualizaciones, como los sistemas de procesamiento de transacciones, depende mucho de la velocidad de escritura en el disco.

  Se puede utilizar **memoria no volátil de acceso aleatorio** (RAM no volátil, NV-RAM, **Nonvolatile Random-Access Memory**) para acelerar drásticamente la escritura en el disco. El contenido de la NV-RAM no se pierde durante los fallos del suministro eléctrico. Una manera habitual de implementar la NV-RAM es utilizar RAM alimentada por baterías. La idea es que, cuando el sistema de bases de datos (o el sistema operativo) solicita que se escriba un bloque en el disco, el controlador del disco escriba el bloque en una memoria intermedia de NV-RAM y comunique de manera inmediata al sistema operativo que la escritura se completó con éxito. El controlador escribirá los datos en el disco cuando no haya otras solicitudes o cuando se llene la memoria intermedia de NV-RAM. Cuando el sistema de bases de datos solicita la escritura de un bloque, sólo percibe un retraso si la memoria intermedia de NV-RAM se encuentra llena. Durante la recuperación de una caída del sistema se vuelven a escribir en el disco todas las operaciones de escritura pendientes en la memoria intermedia de NV-RAM.

  La memoria intermedia NV-RAM se encuentra en algunos discos de altas prestaciones, pero más frecuentemente en los "controladores RAID"; RAID se estudia en el Apartado 11.3.

- **Disco de registro histórico**. Otra manera de reducir las latencias de escritura es utilizar un disco de registro histórico (es decir, un disco destinado a escribir un registro secuencial) de manera muy parecida a la memoria intermedia RAM no volátil. Todos los accesos al disco de registro histórico son secuenciales, lo que básicamente elimina el tiempo de búsqueda y, así, se pueden escribir simultáneamente varios bloques consecutivos, lo que hace que los procesos de escritura en el disco de registro sean varias veces más rápidos que los procesos de escritura aleatorios. Igual que ocurría anteriormente, también hay que escribir los datos en su ubicación verdadera en el disco, pero el disco de registro puede realizar este proceso de escritura más adelante, sin que el sistema de bases de datos tenga que esperar a que se complete. Además, el disco de registro histórico puede reordenar las operaciones de escritura para reducir el movimiento del brazo. Si el sistema se cae antes de que se haya completado alguna operación de escritura en la ubicación real del disco, cuando el sistema se recupera, lee el disco de registro histórico para averiguar las operaciones de escritura que no se han completado y las lleva a cabo.

  Los sistemas de archivo que soportan los discos de registro histórico mencionados se denominan **sistemas de archivos de diario**. Los sistemas de archivos de diario se pueden implementar incluso sin un disco de registro histórico independiente, guardando los datos y el registro histórico en el mismo disco. Al hacerlo así se reduce el coste económico, a expensas de un menor rendimiento.

  La mayoría de los sistemas de archivos modernos implementan un diario y usan el disco de registro histórico al escribir información interna del sistema de archivos, como la relativa a la asignación de archivos. Los primeros sistemas de archivos permitían reordenar las operaciones de escritura sin emplear disco de registro, y corrían el riesgo de que las estructuras de datos del sistema de archivos del disco se corrompiesen si se producía una caída del sistema. Supóngase, por ejemplo, que un sistema de archivos utiliza una lista enlazada y se inserta un nuevo nodo

www.detodoprogramacion.com

al final escribiendo primero los datos del nuevo nodo y actualizando posteriormente el puntero del nodo anterior. Supóngase también que las operaciones de escritura se reordenasen, de forma que se actualizase primero el puntero y que el sistema se cayese antes de escribir el nuevo nodo. El contenido del nodo sería la basura que hubiese anteriormente en el disco, lo que daría lugar a una estructura de datos corrupta.

Para evitar esta posibilidad los primeros sistemas de archivo tenían que realizar una comprobación de consistencia del sistema al reiniciar el sistema para asegurarse de que las estructuras de datos eran consistentes. Si no lo eran, había que tomar medidas adicionales para volver a hacerlas consistentes. Estas comprobaciones daban lugar a grandes retardos en el inicio del sistema después de las caídas, que se agravaron al aumentar la capacidad de las unidades de disco. Los sistemas de archivo de diario permiten un reinicio rápido sin necesidad de esas comprobaciones.

No obstante, las operaciones de escritura realizadas por las aplicaciones no se suelen escribir en el disco del registro histórico. Los sistemas de bases de datos implementan sus propias variantes de registro, que se estudian posteriormente en el Capítulo 17.

11.3 RAID

Los requisitos de almacenamiento de datos de algunas aplicaciones (en particular las aplicaciones Web, las de bases de datos y las multimedia) han crecido tan rápidamente que hace falta un gran número de discos para almacenar sus datos, pese a que la capacidad de los discos también ha aumentado mucho.

Tener gran número de discos en el sistema proporciona oportunidades para mejorar la velocidad a la que se pueden leer o escribir los datos si los discos funcionan en paralelo. También se pueden realizar varias operaciones de lectura o escritura independientes simultáneamente. Además, esta configuración ofrece la posibilidad de mejorar la fiabilidad del almacenamiento de datos, ya que se puede guardar información repetida en varios discos. Por tanto, el fallo de uno de los discos no provoca pérdidas de datos.

Para conseguir mayor rendimiento y fiabilidad se han propuesto varias técnicas de organización de los discos, denominadas colectivamente **disposición redundante de discos independientes** (**RAID**, **Redundant Array of Independent Disks**).

En el pasado, los diseñadores de sistemas consideraron los sistemas de almacenamiento compuestos de varios discos pequeños de bajo coste como alternativa económicamente efectiva al empleo de unidades grandes y caras; el coste por megabyte de los discos más pequeños era menor que el de los de gran tamaño. De hecho, la I de RAID, que ahora significa *independientes*, originalmente representaba *económicos* (*inexpensive*). Hoy en día, sin embargo, todos los discos son de pequeño tamaño y los discos de mayor capacidad tienen un menor coste por megabyte. Los sistemas RAID se utilizan por su mayor fiabilidad y por su mejor rendimiento, más que por motivos económicos. Otro motivo fundamental del empleo de RAID es su mayor facilidad de administración y de operación.

11.3.1 Mejora de la fiabilidad mediante la redundancia

Considérese en primer lugar la fiabilidad. La probabilidad de que falle, al menos, un disco de un conjunto de N discos es mucho más elevada que la posibilidad de que falle un único disco concreto. Supóngase que el tiempo medio entre fallos de un disco dado es de cien mil horas o, aproximadamente, once años. Por tanto, el tiempo medio entre fallos de algún disco de una disposición de cien discos será de $100.000/100 = 1.000$ horas, o cuarenta y dos días, ¡lo que no es mucho! Si sólo se guarda una copia de los datos, cada fallo de disco dará lugar a la pérdida de una cantidad significativa de datos (como ya se estudió en el Apartado 11.2.1). Una frecuencia de pérdida de datos tan elevada resulta inaceptable.

La solución al problema de la fiabilidad es introducir la **redundancia**; es decir, se guarda información adicional que normalmente no se necesita pero que se puede utilizar en caso de fallo de un disco para reconstruir la información perdida. Por tanto, aunque falle un disco, no se pierden datos, por lo que el tiempo medio efectivo entre fallos aumenta, siempre que se cuenten sólo los fallos que dan lugar a pérdida de datos o a su no disponibilidad.

El enfoque más sencillo (pero el más costoso) para la introducción de la redundancia es duplicar todos los discos. Esta técnica se denomina **creación de imágenes** (o, a veces, *creación de sombras*). Cada

www.detodoprogramacion.com

unidad lógica consta, por tanto, de dos unidades físicas y cada proceso de escritura se lleva a cabo por duplicado. Si uno de los discos falla se pueden leer los datos del otro. Los datos sólo se perderán si falla el segundo disco antes de que se repare el primero que falló.

El tiempo medio entre fallos (entendiendo fallo como pérdida de datos) de un disco con imagen depende del tiempo medio entre fallos de cada disco y del **tiempo medio de reparación**, que es el tiempo que se tarda (en promedio) en sustituir un disco averiado y en restaurar sus datos. Supóngase que los fallos de los dos discos son *independientes*; es decir, no hay conexión entre el fallo de uno y el del otro. Por tanto, si el tiempo medio entre fallos de un solo disco es de cien mil horas y el tiempo medio de reparación es de diez horas, el **tiempo medio entre pérdidas de datos** de un sistema de discos con imagen es $100.000^2/(2 * 10) = 500 * 10^6$ horas, o ¡57.000 años! (aquí no se entra en detalle en los cálculos; se proporcionan en las referencias de las notas bibliográficas).

Hay que tener en cuenta que la suposición de independencia de los fallos de los discos no resulta válida. Los fallos en el suministro eléctrico y los desastres naturales como los terremotos, los incendios y las inundaciones pueden dar lugar a daños simultáneos de ambos discos. A medida que los discos envejecen, la probabilidad de fallo aumenta, lo que incrementa la posibilidad de que falle el segundo disco mientras se repara el primero. A pesar de todas estas consideraciones, sin embargo, los sistemas de discos con imagen ofrecen una fiabilidad mucho más elevada que los sistemas de disco único. Hoy en día se dispone de sistemas de discos con imagen con un tiempo medio entre pérdidas de datos de entre quinientas mil y un millón de horas, de cincuenta y cinco a ciento diez años.

Los fallos en el suministro eléctrico son una causa especial de preocupación, dado que tienen lugar mucho más frecuentemente que los desastres naturales. Los fallos en el suministro eléctrico no son un problema si no se está realizando ninguna transferencia de datos al disco cuando tienen lugar. Sin embargo, incluso con la creación de imágenes de los discos, si se hallan en curso procesos de escritura en el mismo bloque en ambos discos y el suministro eléctrico falla antes de que se haya acabado de escribir en ambos bloques, los dos pueden quedar en un estado inconsistente. La solución a este problema es escribir primero una copia y luego la otra, de modo que una de las copias siempre sea consistente. Cuando se vuelve a iniciar el sistema tras un fallo en el suministro eléctrico hacen falta algunas acciones adicionales para recuperar los procesos de escritura incompletos. Este asunto se examina en el Ejercicio práctico 11.2.

11.3.2 Mejora del rendimiento mediante el paralelismo

Considérense ahora las ventajas del acceso en paralelo a varios discos. Con la creación de imágenes de los discos la velocidad a la que se pueden procesar las solicitudes de lectura se duplica, dado que pueden enviarse a cualquiera de los discos (siempre y cuando los dos integrantes de la pareja estén operativos, como es el caso habitual). La velocidad de transferencia de cada proceso de lectura es la misma que en los sistemas de disco único, pero el número de procesos de lectura por unidad de tiempo se ha duplicado.

También se puede mejorar la velocidad de transferencia con varios discos **distribuyendo los datos** entre ellos. En su forma más sencilla, la distribución de datos consiste en dividir los bits de cada byte entre varios discos; esto se denomina **distribución en el nivel de bits**. Por ejemplo, si se dispone de una disposición de ocho discos, se puede escribir el bit i de cada byte en el disco i. La disposición de ocho discos puede tratarse como un solo disco con sectores que tienen ocho veces el tamaño normal y, lo que es más importante, que tiene ocho veces la velocidad de acceso habitual. En una organización así cada disco toma parte en todos los accesos (de lectura o de escritura) por lo que el número de accesos que pueden procesarse por segundo es aproximadamente el mismo que con un solo disco, pero cada acceso puede octuplicar el número de datos leídos por unidad de tiempo respecto a un solo disco. La distribución en el nivel de bits puede generalizarse a cualquier número de discos que sea múltiplo o divisor de ocho. Por ejemplo, si se utiliza una disposición de cuatro discos, los bits i y $4 + i$ de cada byte irán al disco i.

La **distribución en el nivel de bloques** reparte cada bloque entre varios discos. Trata la disposición de discos como un único disco de gran capacidad y asigna números lógicos a los bloques; se da por supuesto que los números de bloque comienzan en cero. Con una disposición de n discos, la distribución en el nivel de bloques asigna el bloque lógico i de la disposición de discos al disco $(i \bmod n) + 1$; utiliza

www.detodoprogramacion.com

el bloque físico $\lfloor i/n \rfloor$-ésimo del disco para almacenar el bloque lógico i. Por ejemplo, con ocho discos, el bloque lógico 0 se almacena el bloque físico 0 del disco 1, mientras que el bloque lógico 11 se almacena en el bloque físico 1 del disco 4. Si se lee un archivo grande, la distribución en el nivel de bloques busca simultáneamente n bloques en paralelo en los n discos, lo que permite una gran velocidad de transferencia para lecturas de gran tamaño. Cuando se lee un solo bloque, la velocidad de transferencia de datos es igual que con un único disco, pero los restantes $n-1$ discos quedan libres para realizar otras acciones.

La distribución en el nivel de bloques es la forma de distribución de datos más usada. También son posibles otros niveles de distribución, como el de los bytes de cada sector o el de los sectores de cada bloque.

En resumen, en un sistema de discos el paralelismo tiene dos objetivos principales:

1. Equilibrar la carga de varios accesos de pequeño tamaño (accesos a bloques), de manera que aumente la productividad de ese tipo de accesos.
2. Convertir en paralelos los accesos de gran tamaño para que su tiempo de respuesta se reduzca.

11.3.3 Niveles de RAID

La creación de imágenes proporciona gran fiabilidad pero resulta costosa. La distribución proporciona velocidades de transferencia de datos elevadas, pero no mejora la fiabilidad. Varios esquemas alternativos pretenden proporcionar redundancia a un coste menor combinando la distribución de los discos combinada con los bits de "paridad" (que se describen a continuación). Estos esquemas tienen diferentes compromisos entre el coste y el rendimiento y se clasifican en los denominados **niveles de RAID**, como se muestra en la Figura 11.3 (en la figura, P indica los bits para la corrección de errores y C indica una segunda copia de los datos). La figura muestra cuatro discos de datos para todos los niveles, y los discos adicionales se utilizan para guardar la información redundante para la recuperación en caso de fallo.

- **El nivel 0 de RAID** hace referencia a disposiciones de discos con distribución en el nivel de bloques, pero sin redundancia (como la creación de imágenes o los bits de paridad). La Figura 11.3a muestra una disposición de tamaño cuatro.
- **El nivel 1 de RAID** hace referencia a la creación de imágenes del disco con distribución de bloques. La Figura 11.3b muestra una organización con imagen que guarda cuatro discos de datos.

  Obsérvese que algunos fabricantes emplean el término **nivel 1+0 de RAID** o **nivel 10 de RAID** para referirse a imágenes del disco con distribución, y utilizan el término nivel 1 de RAID para las imágenes del disco sin distribución. Esto último también se puede emplear con disposiciones de discos para dar la apariencia de un disco grande y fiable: si cada disco tiene M bloques, los bloques lógicos 1 a M se almacenan en el disco 1; $M+1$ a $2M$ en el segundo, y así sucesivamente, y se crea una imagen de cada disco.[1]
- **El nivel 2 de RAID**, también conocido como organización de códigos de corrección de errores tipo memoria (memory-style-error-correcting-code organization, ECC), emplea bits de paridad. Hace tiempo que los sistemas de memoria utilizan los bits de paridad para la detección y corrección de errores. Cada byte del sistema de memoria puede tener asociado un bit de paridad que registra si el número de bits del byte que valen uno es par (paridad = 0) o impar (paridad = 1). Si alguno de los bits del byte se deteriora (un uno se transforma en cero y viceversa) la paridad del byte se modifica y, por tanto, no coincide con la paridad guardada. Análogamente, si el bit de paridad guardado se deteriora no coincide con la paridad calculada. Por tanto, el sistema de memoria detecta todos los errores que afectan a un número impar de bits de un mismo byte. Los esquemas

1. Obsérvese que algunos fabricantes usan el término RAID 0+1 para referirse a una versión de RAID que utiliza la distribución para crear una disposición RAID 0, y crea una imagen de esa disposición en otra disposición, con la diferencia respecto de RAID 1 de que, si falla un disco, la disposición RAID 0 que contiene el disco queda inutilizable. La disposición con imagen se puede seguir utilizando, así que no hay pérdida de datos. Esta organización es inferior a RAID 1 cuando falla un disco, ya que los demás discos de la disposición RAID 0 se pueden seguir usando en RAID 1, pero quedan inactivos en RAID 0+1.

www.detodoprogramacion.com

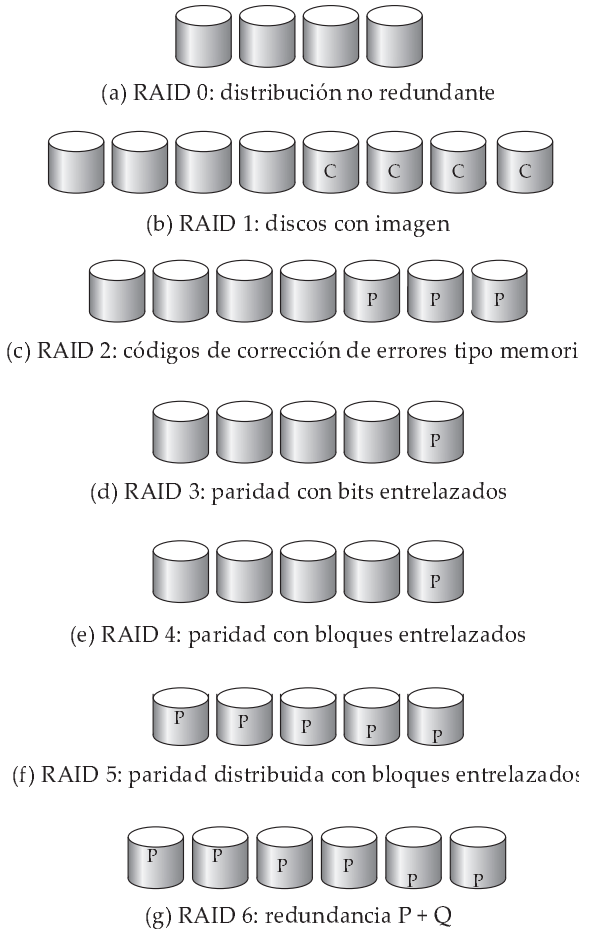


Figura 11.3 Niveles de RAID.

de corrección de errores guardan dos o más bits adicionales y pueden reconstruir los datos si se deteriora un solo bit.

La idea de los códigos para la corrección de errores se puede utilizar directamente en las disposiciones de discos mediante la distribución de los bytes entre los diferentes discos. Por ejemplo, el primer bit de cada byte puede guardarse en el disco uno, el segundo en el disco dos, etc., hasta que se guarde el octavo bit en el disco ocho; los bits para la corrección de errores se guardan en discos adicionales.

La Figura 11.3c muestra el esquema de nivel 2. Los discos marcados como *P* guardan los bits para la corrección de errores. Si uno de los discos falla, los restantes bits del byte y los de corrección de errores asociados se pueden leer en los demás discos y se pueden utilizar para reconstruir los datos deteriorados. La Figura 11.3c muestra una disposición de tamaño cuatro; obsérvese que el RAID de nivel 2 sólo necesita tres discos adicionales para los cuatro discos de datos, a diferencia del RAID de nivel 1, que necesitaba cuatro discos adicionales.

- **El nivel 3 de RAID**, organización de paridad con bits entrelazados, mejora respecto al nivel 2 al aprovechar que los controladores de disco, a diferencia de los sistemas de memoria, pueden detectar si los sectores se han leído correctamente, por lo que se puede utilizar un solo bit de paridad para la corrección y para la detección de los errores. La idea es la siguiente: si uno de los sectores se deteriora, se sabe exactamente el sector que es y, para cada uno de sus bits, se puede determinar si es un uno o un cero calculando la paridad de los bits correspondientes de los sectores de los demás discos. Si la paridad de los bits restantes es igual que la paridad guardada, el bit perdido es un cero; en caso contrario, es un uno.

www.detodoprogramacion.com

El nivel 3 de RAID es tan bueno como el nivel 2, pero resulta menos costoso en cuanto al número de discos adicionales (sólo tiene un disco adicional), por lo que el nivel 2 no se utiliza en la práctica. La Figura 11.3d muestra el esquema de nivel 3.

El nivel 3 de RAID tiene dos ventajas respecto al nivel 1. Sólo necesita un disco de paridad para varios discos normales, mientras que el nivel 1 necesita un disco imagen por cada disco, por lo que se reduce la necesidad de almacenamiento adicional. Dado que los procesos de lectura y de escritura de cada byte se distribuyen por varios discos, con la distribución de los datos en N fracciones, la velocidad de transferencia para la lectura o la escritura de un solo bloque multiplica por N la de una organización RAID de nivel 1 que emplee la distribución entre N discos. Por otro lado, el nivel 3 de RAID permite un menor número de operaciones de E/S por segundo, ya que todos los discos tienen que participar en cada solicitud de E/S.

- **El nivel 4 de RAID**, organización de paridad con bloques entrelazados, emplea la distribución del nivel de bloques, como RAID 0, y, además, guarda un bloque de paridad en un disco independiente para los bloques correspondientes de los otros N discos. Este esquema se muestra gráficamente en la Figura 11.3e. Si falla uno de los discos, se puede utilizar el bloque de paridad con los bloques correspondientes de los demás discos para restaurar los bloques del disco averiado.

  La lectura de un bloque sólo accede a un disco, lo que permite que los demás discos procesen otras solicitudes. Por tanto, la velocidad de transferencia de datos de cada acceso es menor, pero se pueden ejecutar en paralelo varios accesos de lectura, lo que produce una mayor velocidad global de E/S. Las velocidades de transferencia para los procesos de lectura de gran tamaño son elevadas, dado que se pueden leer todos los discos en paralelo; los procesos de escritura de gran tamaño también tienen velocidades de transferencia elevadas, dado que los datos y la paridad pueden escribirse en paralelo.

  Los procesos de escritura independientes de pequeño tamaño, por otro lado, no pueden realizarse en paralelo. La operación de escritura de un bloque tiene que acceder al disco en el que se guarda ese bloque, así como al disco de paridad, ya que hay que actualizar el bloque de paridad. Además, hay que leer tanto el valor anterior del bloque de paridad como el del bloque que se escribe para calcular la nueva paridad. Por tanto, un solo proceso de escritura necesita cuatro accesos a disco: dos para leer los dos bloques antiguos y otros dos para escribir los dos nuevos.

- **El nivel 5 de RAID**, paridad distribuida con bloques entrelazados, mejora respecto al nivel 4 al dividir los datos y la paridad entre todos los $N + 1$ discos, en vez de guardar los datos en N discos y la paridad en uno solo. En el nivel 5 todos los discos pueden atender a las solicitudes de lectura, a diferencia del nivel 4 de RAID, en el que el disco de paridad no puede participar, por lo que el nivel 5 aumenta el número total de solicitudes que pueden atenderse en una cantidad de tiempo dada. En cada conjunto de N bloques lógicos, uno de los discos guarda la paridad y los otros N guardan los bloques.

  La Figura 11.3f muestra esta configuración, en la que las P están distribuidas entre todos los discos. Por ejemplo, con una disposición de cinco discos, el bloque de paridad, marcado como Pk, para los bloques lógicos $4k, 4k + 1, 4k + 2, 4k + 3$ se guarda en el disco $(k \bmod 5) + 1$; los bloques correspondientes de los otros cuatro discos guardan los cuatro bloques de datos $4k$ to $4k + 3$. La siguiente tabla indica la manera en que se disponen los primeros veinte bloques, numerados de 0 to 19, y sus bloques de paridad. El patrón mostrado se repite para los siguientes bloques.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

Obsérvese que un bloque de paridad no puede guardar la paridad de los bloques del mismo disco, ya que un fallo del disco supondría la pérdida de los datos y de la paridad y, por tanto, no

www.detodoprogramacion.com

sería recuperable. El nivel 5 incluye al nivel 4, dado que ofrece mejor rendimiento de lectura y de escritura por el mismo coste, por lo que el nivel 4 no se utiliza en la práctica.

- **El nivel 6 de RAID**, también denominado esquema de redundancia P+Q, es muy parecido al nivel 5, pero guarda información redundante adicional para la protección contra los fallos de disco múltiples. En lugar de utilizar la paridad, el nivel 6 utiliza códigos para la corrección de errores como los de Reed–Solomon (véanse las notas bibliográficas). En el esquema de la Figura 11.3g se guardan dos bits de datos redundantes por cada cuatro bits de datos (a diferencia del único bit de paridad del nivel 5) y el sistema puede tolerar dos fallos del disco.

Finalmente, se debe destacar que se han propuesto varias modificaciones a los esquemas RAID básicos aquí descritos, y que los diferentes fabricantes emplean su propia terminología para las variantes.

11.3.4 Elección del nivel RAID adecuado

Los factores que se deben tener en cuenta para elegir el nivel RAID son:

- El coste económico de los requisitos adicionales de almacenamiento en disco.
- Los requisitos de rendimiento en términos de número de operaciones de E/S.
- El rendimiento cuando falla un disco.
- El rendimiento durante la reconstrucción (esto es, mientras los datos del disco averiado se reconstruyen en un disco nuevo).

El tiempo que se tarda en reconstruir los datos que contenía el disco averiado puede ser significativo, y varía con el nivel RAID utilizado. La reconstrucción resulta más sencilla para el RAID de nivel 1, ya que se pueden copiar los datos de otro disco; para los otros niveles hay que acceder a todos los demás discos de la disposición para reconstruir los datos del disco averiado. El **rendimiento de la reconstrucción** de un sistema RAID puede ser un factor importante si se necesita que los datos estén disponibles ininterrumpidamente, como ocurre en los sistemas de bases de datos de alto rendimiento. Además, dado que el tiempo de reconstrucción puede suponer una parte importante del tiempo de reparación, el rendimiento de la reconstrucción también influye en el tiempo medio entre pérdidas de datos.

El nivel 0 de RAID se utiliza en aplicaciones de alto rendimiento en las que la seguridad de los datos no es crítica. Dado que los niveles 2 y 4 de RAID se subsumen en el 3 y en el 5, respectivamente, la elección de nivel RAID se limita a los niveles restantes. La distribución de bits (nivel 3) es inferior a la distribución de bloques (nivel 5) ya que ésta permite velocidades de transferencia de datos similares para las transferencias de gran tamaño y emplea menos discos para las pequeñas. Para las transferencias de pequeño tamaño, el tiempo de acceso al disco es el factor dominante, por lo que disminuye la ventaja de las operaciones de lectura paralelas. De hecho, el nivel 3 puede presentar un peor rendimiento que el nivel 5 para transferencias de pequeño tamaño, ya que sólo se completan cuando se han encontrado los sectores correspondientes de todos los discos; la latencia media de la disposición de discos se comporta, por tanto, de forma muy parecida a la máxima latencia en el caso de un solo disco, lo que anula las ventajas de la mayor velocidad de transferencia. Muchas implementaciones RAID no soportan actualmente el nivel 6, pero ofrece mayor fiabilidad que el nivel 5 y se puede utilizar en aplicaciones en las que la seguridad de los datos sea muy importante.

La elección entre el nivel 1 y el nivel 5 de RAID es más difícil de tomar. El nivel 1 se utiliza mucho en aplicaciones como el almacenamiento de archivos de registro histórico en sistemas de bases de datos, ya que ofrece el mejor rendimiento para escritura. El nivel 5 tiene menos sobrecarga de almacenamiento que el nivel 1, pero presenta una mayor sobrecarga temporal en las operaciones de escritura. Para las aplicaciones en las que los datos se leen con frecuencia y se escriben raramente, el nivel 5 es la opción preferida.

La capacidad de almacenamiento en disco ha aumentado anualmente más del cincuenta por ciento durante muchos años, y el coste por byte ha disminuido a la misma velocidad. En consecuencia, para muchas aplicaciones existentes de bases de datos con requisitos de almacenamiento moderados, el coste económico del almacenamiento adicional en disco necesario para la creación de imágenes ha pasado a

www.detodoprogramacion.com

ser relativamente pequeño (sin embargo, el coste económico adicional, sigue siendo un aspecto significativo para las aplicaciones de almacenamiento intensivo como el almacenamiento de datos de vídeo). La velocidad de acceso ha mejorado mucho más lentamente (del orden de un factor tres en diez años), mientras que el número de operaciones E/S necesarias por segundo se ha incrementado enormemente, especialmente para los servidores de aplicaciones Web.

El nivel 5 de RAID que incrementa el número de operaciones E/S necesarias para escribir un solo bloque lógico, sufre una penalización de tiempo significativa en términos del rendimiento de escritura. El nivel 1 de RAID es, por tanto, la elección adecuada para muchas aplicaciones con requisitos moderados de almacenamiento y grandes de E/S.

Los diseñadores de sistemas RAID también tienen que tomar otras decisiones. Por ejemplo, la cantidad de discos que habrá en la disposición y los bits que debe proteger cada bit de paridad. Cuantos más discos haya en la disposición, mayores serán las velocidades de transferencia de datos, pero el sistema será más caro. Cuantos más bits proteja cada bit de paridad, menor será la necesidad adicional de espacio debida a los bits de paridad, pero habrá más posibilidades de que falle un segundo disco antes de que el primer disco en averiarse esté reparado y de que eso dé lugar a pérdidas de datos.

11.3.5 Aspectos hardware

Otro aspecto que debe tenerse en cuenta en la elección de implementaciones RAID es el hardware. RAID se puede implementar sin cambios en el nivel hardware, modificando sólo el software. Estas implementaciones se conocen como **RAID software**. Sin embargo, se pueden obtener ventajas significativas al crear hardware específico para dar soporte a RAID, que se describen a continuación; los sistemas con soporte hardware especial se denominan sistemas **RAID hardware**.

Las implementaciones RAID hardware pueden utilizar RAM no volátil para registrar las operaciones de escritura antes de llevarlas a cabo. En caso de fallo de corriente, cuando el sistema se restaura, recupera la información relativa a las operaciones de escritura incompletas de la memoria RAM no volátil y completa esas operaciones. Sin ese soporte hardware, hay que llevar a cabo trabajo adicional para detectar los bloques que se han escrito parcialmente antes del fallo de corriente (véase el Ejercicio práctico 11.2).

Algunas implementaciones RAID permiten el **intercambio en caliente**; esto es, se pueden retirar los discos averiados y sustituirlos por otros nuevos sin desconectar la corriente. El intercambio en caliente reduce el tiempo medio de reparación, ya que los cambios de disco no necesitan esperar a que se pueda apagar el sistema. De hecho, hoy en día muchos sistemas críticos trabajan con un horario 24×7; esto es, funcionan 24 horas al día, 7 días a la semana, lo que no ofrece ningún momento para apagar el sistema y cambiar los discos averiados. Además, muchas implementaciones RAID asignan un disco de repuesto para cada disposición (o para un conjunto de disposiciones de disco). Si falla algún disco, el disco de repuesto se utiliza como sustituto de manera inmediata. En consecuencia, el tiempo medio de reparación se reduce notablemente, lo que minimiza la posibilidad de pérdida de datos. El disco averiado se puede reemplazar cuando resulte más conveniente.

La fuente de alimentación, el controlador de disco o, incluso, la interconexión del sistema en un sistema RAID pueden ser el punto de fallo que detiene el funcionamiento del sistema RAID. Para evitar esta posibilidad, las buenas implementaciones RAID tienen varias fuentes de alimentación redundantes (con baterías de respaldo que les permiten seguir funcionando aunque se corte la corriente). Estos sistemas RAID tienen varias interfaces de disco y varias interconexiones para conectar el sistema RAID con el sistema informático (o con la red de sistemas informáticos). En consecuencia, el fallo de un solo componente no detiene el funcionamiento del sistema RAID.

11.3.6 Otras aplicaciones de RAID

Los conceptos de RAID se han generalizado a otros dispositivos de almacenamiento, como los conjuntos de cintas, e incluso a la transmisión de datos por radio. Cuando se aplican a los conjuntos de cintas, las estructuras RAID pueden recuperar datos aunque se haya dañado una de las cintas de la disposición. Cuando se aplican a la transmisión de datos, cada bloque de datos se divide en unidades menores y se transmite junto con una unidad de paridad; si, por algún motivo, no se recibe alguna de las unidades, se puede reconstruir a partir del resto.

www.detodoprogramacion.com

11.4 Almacenamiento terciario

Puede que en los sistemas de bases de datos de gran tamaño parte de los datos tenga que residir en un almacenamiento terciario. Los dos medios de almacenamiento terciario más habituales son los discos ópticos y las cintas magnéticas.

11.4.1 Discos ópticos

Los discos compactos han sido un medio popular de distribución de software, datos multimedia como el sonido y las imágenes, y otra información editada de manera electrónica. Tienen una capacidad de almacenamiento de 640 a 700 megabytes y resultan baratos de producir en masa. Actualmente, los discos de vídeo digital (DVD, Digital Video Disk) han reemplazado a los discos compactos en las aplicaciones que necesitan grandes cantidades de datos. Los discos de formato DVD-5 pueden almacenar 4,7 gigabytes de datos (en una capa de grabación), mientras que los discos de formato DVD-9 pueden almacenar 8,5 gigabytes de datos (en dos capas de grabación). La grabación en las dos caras del disco ofrece capacidades mayores incluso; los formatos DVD-10 y DVD-18, que son las versiones de doble cara de DVD-5 y DVD-9, pueden almacenar 9,4 y 17 gigabytes, respectivamente. Los formatos más recientes, denominados *HD-DVD* y *Blu-Ray DVD* tienen una capacidad significativamente mayor: los discos HD-DVD pueden almacenar de 12 a 30 gigabytes por disco, mientras que los discos Blu-Ray DVD de 25 a 50 gigabytes por disco. Se espera que estén disponibles entre 2005 y 2006.

Las unidades de CD y de DVD presentan tiempos de búsqueda mucho mayores (100 milisegundos es un valor habitual) que las unidades de disco magnético, debido a que el dispositivo de cabezas es más pesado. Las velocidades rotacionales suelen ser menores, aunque las unidades de CD y de DVD más rápidas alcanzan alrededor de tres mil revoluciones por minuto, que son comparables a las velocidades de los discos magnéticos de gama baja. Las velocidades rotacionales de las unidades de CD se correspondían inicialmente con las normas de los CD de sonido; y las velocidades de las unidades de DVD, con las de los DVD de vídeo, pero las unidades actuales rotan a varias veces la velocidad indicada por las normas. Las velocidades de transferencia de datos son algo menores que para los discos magnéticos. Las unidades de CD actuales leen entre 3 y 6 megabytes por segundo; y las de DVD, de 8 a 20 megabytes por segundo. Al igual que las unidades de discos magnéticos, los discos ópticos almacenan más datos en las pistas exteriores y menos en las interiores. La velocidad de transferencia de las unidades ópticas se caracteriza por $n\times$, que significa que la unidad soporta transferencias a n veces la velocidad indicada por la norma; las velocidades de $50\times$ para CD y de $16\times$ para DVD son frecuentes hoy en día.

Las versiones de los discos ópticos en las que sólo se puede escribir una vez (CD-R, DVD-R y DVD+R) son populares para la distribución de datos y, en especial, para el almacenamiento de datos con fines de archivo, debido a que tienen una gran capacidad, una vida más larga que los discos magnéticos y pueden retirarse de la unidad y almacenarse en un lugar remoto. Dado que no pueden sobrescribirse, se pueden utilizar para almacenar información que no se deba modificar, como los registros de auditoría. Las versiones en las que se puede escribir más de una vez (CD-RW, DVD-RW, DVD+RW y DVD-RAM) también se emplean para archivo.

Los **cambiadores de discos** (**jukebox**) son dispositivos que guardan gran número de discos ópticos (hasta varios cientos) y los cargan automáticamente a petición de los usuarios en unas cuantas unidades (usualmente entre una y diez). La capacidad de almacenamiento agregada de estos sistemas puede ser de muchos terabytes. Cuando se accede a un disco, un brazo mecánico lo carga en la unidad desde una estantería (antes hay que volver a colocar en la estantería el disco que se hallara en la unidad). El tiempo de carga y descarga suele ser del orden de segundos (mucho mayor que los tiempos de acceso a disco).

11.4.2 Cintas magnéticas

Aunque las cintas magnéticas son relativamente permanentes y pueden albergar grandes volúmenes de datos, resultan lentas en comparación con los discos magnéticos y ópticos. Lo que es aún más importante, la cinta magnética está limitada al acceso secuencial. Por tanto, no puede proporcionar acceso aleatorio para los requisitos de almacenamiento secundario, aunque históricamente, antes del uso de los discos magnéticos, las cintas se utilizaran como medio de almacenamiento secundario.

www.detodoprogramacion.com

Las cintas se emplean principalmente para copias de seguridad, para el almacenamiento de la información poco utilizada y como medio sin conexión para la transferencia de información de un sistema a otro. Las cintas también se utilizan para almacenar grandes volúmenes de datos, como los datos de vídeo o de imágenes, a los que no es necesario acceder rápidamente o que son tan voluminosos que su almacenamiento en disco magnético resultaría demasiado caro.

Cada cinta se guarda en una bobina y se enrolla o desenrolla por delante de una cabeza de lectura y escritura. El desplazamiento hasta el punto correcto de la cinta puede tardar segundos o, incluso, minutos, en vez de milisegundos; una vez en posición, sin embargo, las unidades de cinta pueden escribir los datos con densidades y velocidades que se aproximan a las de las unidades de disco. La capacidad varía en función de la longitud y de la anchura de la cinta y de la densidad con la que la cabeza pueda leer y escribir. El mercado ofrece actualmente una amplia variedad de formatos de cinta. La capacidad disponible actualmente varía de unos pocos gigabytes con el formato DAT (**Digital Audio Tape**, **cinta de audio digital**), de 10 a 40 gigabytes con el formato DLT (**Digital Linear Tape**, **cinta lineal digital**), de 100 gigabytes en adelante con el formato **Ultrium**, hasta 330 gigabytes con los formatos de cinta de **exploración helicoidal de Ampex**. Las velocidades de transferencia de datos llegan a las decenas de megabytes por segundo.

Las unidades de cinta son bastante fiables, y los buenos sistemas de unidades de cinta realizan una lectura de los datos recién escritos para asegurarse de que se han grabado correctamente. Sin embargo, las cintas presentan límites en cuanto al número de veces que se pueden leer o escribir con fiabilidad.

Los **cambiadores de cintas**, al igual que los cambiadores de discos ópticos, tienen gran número de cintas y unas cuantas unidades en las que se pueden montar esas cintas; se utilizan para guardar grandes volúmenes de datos, que pueden llegar a varios petabytes (10^{15} bytes) con tiempos de acceso que varían entre los segundos y unos cuantos minutos. Entre las aplicaciones que necesitan estas enormes cantidades de datos están los sistemas de imágenes que reunen los datos de los satélites de teledetección y las grandes videotecas de las emisoras de televisión.

Algunos formatos de cinta (como el formato Accelis) soportan menores tiempos de búsqueda (del orden de decenas de segundos) y están pensadas para las aplicaciones que recuperan información mediante cambiadores de cintas. La mayor parte del resto de los formatos de cinta proporcionan mayores capacidades, a cambio de un acceso más lento; estos formatos resultan idóneos para las copias de seguridad de los datos, en las que las búsquedas rápidas no son importantes.

Las unidades de cinta no han podido competir con la enorme mejora de capacidad de las unidades de disco y la correspondiente reducción del coste de almacenamiento. Aunque el coste de las cintas es bajo, el de las unidades y las bibliotecas de cintas es significativamente superior que el coste de las unidades de disco: una biblioteca de cintas capaz de almacenar algunos terabytes puede costar decenas de millares de euros. Las copias de seguridad en unidades de disco se han convertido en una alternativa rentable a las copias en cinta para gran número aplicaciones.

11.5 Acceso al almacenamiento

Cada base de datos se corresponde con varios archivos diferentes que el sistema operativo subyacente mantiene. Esos archivos residen permanentemente en los discos, con copias de seguridad en cinta. Cada archivo está dividido en unidades de almacenamiento de longitud constante denominadas **bloques**, que son las unidades de asignación de almacenamiento y de transferencia de datos. En el Apartado 11.6 se estudian varias maneras de organizar lógicamente los datos en archivos.

Cada bloque puede contener varios elementos de datos. El conjunto exacto de elementos de datos que contiene cada bloque viene determinado por la forma de organización física de los datos que se utilice (véase el Apartado 11.6). En nuestro caso se supone que ningún elemento de datos ocupa dos o más bloques. Esta suposición es realista para la mayor parte de las aplicaciones de procesamiento de datos, como el ejemplo bancario propuesto.

Uno de los principales objetivos del sistema de bases de datos es minimizar el número de transferencias de bloques entre el disco y la memoria. Una manera de reducir el número de accesos al disco es mantener en la memoria principal tantos bloques como sea posible. El objetivo es maximizar la posibilidad de que, cuando se acceda a un bloque, ya se encuentre en la memoria principal y, por tanto, no se necesite acceder al disco.

www.detodoprogramacion.com

Dado que no resulta posible mantener en la memoria principal todos los bloques, hay que gestionar la asignación del espacio allí disponible para su almacenamiento. La **memoria intermedia** (buffer) es la parte de la memoria principal disponible para el almacenamiento de las copias de los bloques del disco. Siempre se guarda en el disco una copia de cada bloque, pero esta copia puede ser una versión del bloque más antigua que la de la memoria intermedia. El subsistema responsable de la asignación del espacio de la memoria intermedia se denomina **gestor de la memoria intermedia**.

11.5.1 Gestor de la memoria intermedia

Los programas de los sistemas de bases de datos formulan solicitudes (es decir, llamadas) al gestor de la memoria intermedia cuando necesitan bloques del disco. Si el bloque ya se encuentra en la memoria intermedia, el gestor pasa al solicitante la dirección del bloque en la memoria principal. Si el bloque no se halla en la memoria intermedia, asigna en primer lugar espacio al bloque en la memoria intermedia, descartando algún otro, si hace falta, para hacer sitio al nuevo. El bloque descartado sólo se vuelve a escribir en el disco si se ha modificado desde la última vez que se escribió. A continuación, el gestor de la memoria intermedia lee el bloque solicitado en el disco, lo escribe en la memoria intermedia y pasa la dirección del bloque en la memoria principal al solicitante. Las acciones internas del gestor de la memoria intermedia resultan transparentes para los programas que formulan solicitudes de bloques de disco.

Si se está familiarizado con los conceptos de los sistemas operativos, se observará que el gestor de la memoria intermedia no parece ser más que un gestor de memoria virtual, como los que se hallan en la mayor parte de los sistemas operativos. Una diferencia radica en que el tamaño de las bases de datos puede ser mucho mayor que el espacio de direcciones de hardware de la máquina, por lo que las direcciones de memoria no resultan suficientes para direccionar todos los bloques del disco. Además, para dar un buen servicio al sistema de bases de datos, el gestor de la memoria intermedia debe utilizar técnicas más complejas que los esquemas habituales de gestión de la memoria virtual:

- **Estrategia de sustitución**. Cuando no queda espacio libre en la memoria intermedia hay que eliminar un bloque de ésta antes de que se pueda escribir otro nuevo. La mayor parte de los sistemas operativos utilizan un esquema de **menos recientemente utilizado** (**Least Recently Used**, **LRU**), en el que se vuelve a escribir en el disco y se elimina de la memoria intermedia el bloque al que se ha hecho referencia menos recientemente. Este sencillo enfoque se puede mejorar para las aplicaciones de bases de datos.
- **Bloques clavados**. Para que el sistema de bases de datos pueda recuperarse de las caídas del sistema (Capítulo 17) hay que restringir las ocasiones en que los bloques se pueden volver a escribir en el disco. Por ejemplo, la mayor parte de los sistemas de recuperación exigen que no se escriban en disco los bloques mientras se esté procediendo a su actualización. Se dice que los bloques que no se permite que se vuelvan a escribir en el disco están **clavados**. Aunque muchos sistemas operativos no permiten trabajar con bloques clavados, esta característica resulta fundamental para los sistemas de bases de datos resistentes a las caídas.
- **Salida forzada de los bloques**. Hay situaciones en las que hace falta volver a escribir los bloques en el disco, aunque no se necesite el espacio de memoria intermedia que ocupan. Este proceso de escritura se denomina **salida forzada** del bloque. Se verá el motivo de las salidas forzadas en el Capítulo 17; en resumen, el contenido de la memoria principal y, por tanto, el de la memoria intermedia se pierden en las caídas, mientras que los datos del disco suelen sobrevivir a ellas.

11.5.2 Políticas para la sustitución de la memoria intermedia

El objetivo de las estrategias de sustitución de los bloques de la memoria intermedia es minimizar los accesos al disco. En los programas de propósito general no resulta posible predecir con precisión a qué bloques se hará referencia. Por tanto, el sistema operativo utiliza la pauta anterior de referencias a bloques para predecir las futuras. La suposición que suele hacerse es que es probable que se vuelva a hacer referencia a los bloques a los que se ha hecho referencia recientemente. Por tanto, si hay que sustituir

www.detodoprogramacion.com

```
for each tupla b de prestatario do
    for each tupla c de cliente do
        if b[nombre_cliente] = c[nombre_cliente]
        then begin
                sea x una tupla definida de la manera siguiente:
                x[nombre_cliente] := p[nombre_cliente]
                x[número_préstamo] := p[número_préstamo]
                x[calle_cliente] := c[calle_cliente]
                x[ciudad_cliente] := c[ciudad_cliente]
                incluir la tupla x como parte del resultado de prestatario ⋈ cliente
             end
    end
end
```

Figura 11.4 Procedimiento para calcular la reunión.

un bloque, se sustituye el bloque al que se ha hecho referencia menos recientemente. Este enfoque se denomina **esquema de sustitución de bloques utilizados menos recientemente** (Least Recently Used, LRU).

LRU es un esquema de sustitución aceptable para los sistemas operativos. Sin embargo, los sistemas de bases de datos pueden predecir la pauta de referencias futuras con más precisión que los sistemas operativos. Las peticiones de los usuarios al sistema de bases de datos comprenden varias etapas. El sistema de bases de datos suele poder determinar con antelación los bloques que se necesitarán examinando cada una de las etapas necesarias para llevar a cabo la operación solicitada por el usuario. Por tanto, a diferencia de los sistemas operativos, que deben confiar en el pasado para predecir el futuro, los sistemas de bases de datos pueden tener información relativa, al menos, al futuro a corto plazo.

Para ilustrar la manera en que la información sobre el futuro acceso a los bloques permite mejorar la estrategia LRU considérese el procesamiento de la expresión del álgebra relacional

$$prestatario \bowtie cliente$$

Supóngase que la estrategia escogida para procesar esta solicitud viene dada por el programa en seudocódigo mostrado en la Figura 11.4 (se estudiarán otras estrategias más eficientes en el Capítulo 13).

Supóngase que las dos relaciones de este ejemplo se guardan en archivos diferentes. En este ejemplo se puede ver que, una vez que se haya procesado la tupla de *prestatario*, no vuelve a ser necesaria. Por tanto, una vez completado el procesamiento de un bloque completo de tuplas de *prestatario*, ese bloque ya no se necesita en la memoria principal, aunque se haya utilizado recientemente. Deben darse instrucciones al gestor de la memoria intermedia para que libere el espacio ocupado por el bloque de *prestatario* en cuanto se haya procesado la última tupla. Esta estrategia de gestión de la memoria intermedia se denomina estrategia **de extracción inmediata**.

Considérense ahora los bloques que contienen las tuplas de *cliente*. Hay que examinar cada bloque de las tuplas *cliente* una vez por cada tupla de la relación *prestatario*. Cuando se completa el procesamiento del bloque *cliente*, se sabe que no se accederá nuevamente a él hasta que no se hayan procesado todos los demás bloques de *cliente*. Por tanto, el bloque de *cliente* al que se haya hecho referencia más recientemente será el último bloque al que se vuelva a hacer referencia, y el bloque de *cliente* al que se haya hecho referencia menos recientemente será el bloque al que se vuelva a hacer referencia a continuación. Este conjunto de suposiciones es justo el contrario del que forma la base de la estrategia LRU. En realidad, la estrategia óptima de sustitución de bloques para el procedimiento anterior es la **estrategia más recientemente utilizado** (**Most Recently Used**, **MRU**). Si hay que eliminar de la memoria intermedia un bloque de *cliente*, la estrategia MRU escoge el bloque utilizado más recientemente (los bloques en uso no se pueden eliminar).

Para que la estrategia MRU funcione correctamente en el ejemplo propuesto, el sistema debe clavar el bloque de *cliente* que se esté procesando. Después de que se haya procesado la última tupla de *cliente* el bloque se desclava y pasa a ser el bloque utilizado más recientemente.

www.detodoprogramacion.com

Además de utilizar la información que pueda tener el sistema respecto de la solicitud que se esté procesando, el gestor de la memoria intermedia puede utilizar información estadística relativa a la probabilidad de que una solicitud haga referencia a una relación concreta. Por ejemplo, el diccionario de datos (como se verá con detalle en el Apartado 11.8) que realiza un seguimiento del esquema lógico de las relaciones y de la información de su almacenamiento físico es una de las partes de la base de datos a la que se tiene acceso con mayor frecuencia. Por tanto, el gestor de la memoria intermedia debe intentar no eliminar de la memoria principal los bloques del diccionario de datos, a menos que se vea obligado a hacerlo. En el Capítulo 12 se estudian los índices de los archivos. Dado que puede que se acceda más frecuentemente al índice del archivo que al propio archivo, el gestor de la memoria intermedia no deberá, en general, eliminar de la memoria principal los bloques del índice si se dispone de alternativas.

La estrategia ideal para la sustitución de bloques necesita información sobre las operaciones de la bases de datos (las que se estén realizando y las que se vayan a realizar en el futuro). No se conoce una sola estrategia que sea adecuada para todas las situaciones posibles. En realidad, un número sorprendentemente grande de bases de datos utiliza LRU, a pesar de sus defectos. Las preguntas prácticas y los ejercicios exploran estrategias alternativas.

La estrategia utilizada por el gestor de la memoria intermedia para la sustitución de los bloques se ve influida por factores distintos del momento en que se volverá a hacer referencia a cada bloque. Si el sistema está procesando de manera concurrente las solicitudes de varios usuarios, puede que el subsistema para el control de concurrencia (Capítulo 16) tenga que posponer ciertas solicitudes para asegurar la conservación de la consistencia de la base de datos. Si se proporciona al gestor de la memoria intermedia información del subsistema de control de concurrencia que indique las solicitudes que se posponen, puede utilizar esa información para modificar su estrategia de sustitución de los bloques. Concretamente, los bloques que necesiten las solicitudes activas (no pospuestas) pueden conservarse en la memoria intermedia a expensas de los que necesiten las solicitudes pospuestas.

El subsistema para la recuperación de caídas (Capítulo 17) impone severas restricciones a la sustitución de bloques. Si se ha modificado un bloque, no se permite que el gestor de la memoria intermedia vuelva a copiar al disco la versión nueva del bloque existente en la memoria intermedia, dado que eso destruiría la versión anterior. Por el contrario, el gestor de bloques debe solicitar permiso del subsistema para la recuperación de caídas antes de escribir cada bloque. Puede que el subsistema para la recuperación de caídas exija que se fuerce la salida de otros bloques antes de conceder autorización al gestor de la memoria intermedia para que escriba el bloque solicitado. En el Capítulo 17 se define con precisión la interacción entre el gestor de la memoria intermedia y el subsistema para la recuperación de caídas.

11.6 Organización de los archivos

Los **archivos** se organizan lógicamente como secuencias de registros. Esos registros se corresponden con los bloques del disco. Los archivos constituyen un elemento fundamental de los sistemas operativos, por lo que se supone la existencia de un *sistema de archivos* subyacente. Hay que tomar en consideración diversas maneras de representar los modelos lógicos de datos en términos de los archivos.

Aunque los bloques son de un tamaño fijo determinado por las propiedades físicas del disco y por el sistema operativo, el tamaño de los registros varía. En las bases de datos relacionales, las tuplas de las diferentes relaciones suelen ser de tamaño diferente.

Un enfoque de la correspondencia entre base de datos y archivos es utilizar varios archivos y guardar los registros de la misma longitud en un mismo archivo. Una alternativa es estructurar los archivos de modo que puedan aceptar registros de longitudes diferentes; no obstante, los archivos con registros de longitud fija son más sencillos de implementar que los que tienen registros de longitud variable. Muchas de las técnicas empleadas para los primeros pueden aplicarse a los de longitud variable. Por tanto, se comienza por tomar en consideración los archivos con registros de longitud fija.

11.6.1 Registros de longitud fija

A manera de ejemplo, considérese un archivo con registros de *cuentas* de la base de datos bancaria. Cada registro de este archivo se define (en pseudocódigo) de la manera siguiente:

www.detodoprogramacion.com

```
type depósito = record
                    número_cuenta char(10);
                    nombre_sucursal char(22);
                    saldo numeric(12,2);
                end
```

Si se supone que cada carácter ocupa un byte y que los valores de tipo numeric(12,2) ocupan ocho bytes, el registro de *cuenta* tiene cuarenta bytes de longitud. Un enfoque sencillo es utilizar los primeros cuarenta bytes para el primer registro, los cuarenta bytes siguientes para el segundo, y así sucesivamente (Figura 11.5). Sin embargo, hay dos problemas con este sencillo enfoque:

1. Resulta difícil borrar registros de esta estructura. Hay que rellenar el espacio ocupado por el registro que se va a borrar con algún otro registro del archivo, o tener alguna manera de marcar los registros borrados para poder pasarlos por alto.
2. A menos que el tamaño de los bloques sea múltiplo de cuarenta (lo que resulta improbable), algún registro se saltará los límites de los bloques. Es decir, parte del registro se guardará en un bloque y parte en otro. Harán falta, por tanto, dos accesos a bloques para leer o escribir esos registros.

Cuando se borra un registro, se puede desplazar el situado a continuación al espacio ocupado que ocupaba el registro borrado y hacer lo mismo con los demás, hasta que todos los registros situados a continuación del borrado se hayan desplazado hacia delante (Figura 11.6). Este tipo de enfoque necesita desplazar gran número de registros. Puede que fuera más sencillo desplazar simplemente el último registro del archivo al espacio ocupado por el registro borrado (Figura 11.7).

No resulta deseable desplazar los registros para que ocupen el espacio liberado por los registros borrados, ya que para hacerlo se necesitan accesos adicionales a los bloques. Dado que las operaciones de inserción tienden a ser más frecuentes que las de borrado, resulta aceptable dejar libre el espacio

registro 0	C-102	Navacerrada	400
registro 1	C-305	Collado Mediano	350
registro 2	C-215	Becerril	700
registro 3	C-101	Centro	500
registro 4	C-222	Moralzarzal	700
registro 5	C-201	Navacerrada	900
registro 6	C-217	Galapagar	750
registro 7	C-110	Centro	600
registro 8	C-218	Navacerrada	700

Figura 11.5 Archivo que contiene los registros de *cuenta*.

registro 0	C-102	Navacerrada	400
registro 1	C-305	Collado Mediano	350
registro 3	C-101	Centro	500
registro 4	C-222	Moralzarzal	700
registro 5	C-201	Navacerrada	900
registro 6	C-217	Galapagar	750
registro 7	C-110	Centro	600
registro 8	C-218	Navacerrada	700

Figura 11.6 El archivo de la Figura 11.5 con el registro 2 borrado y todos los registros desplazados.

www.detodoprogramacion.com

registro 0	C-102	Navacerrada	400
registro 1	C-305	Collado Mediano	350
registro 8	C-218	Navacerrada	700
registro 3	C-101	Centro	500
registro 4	C-222	Moralzarzal	700
registro 5	C-201	Navacerrada	900
registro 6	C-217	Galapagar	750
registro 7	C-110	Centro	600

Figura 11.7 El archivo de la Figura 11.5 con el registro 2 borrado y el último registro desplazado.

ocupado por los registros borrados y esperar a una inserción posterior antes de volver a utilizar ese espacio. No basta con una simple marca en el registro borrado, ya que resulta difícil hallar el espacio disponible mientras se realiza una inserción. Por tanto, hay que introducir una estructura adicional.

Al comienzo del archivo se asigna cierto número de bytes como **cabecera del archivo**. La cabecera contiene gran variedad de información sobre el archivo. Por ahora, todo lo que hace falta guardar es la dirección del primer registro cuyo contenido se haya borrado. Se utiliza este primer registro para guardar la dirección del segundo registro disponible, y así sucesivamente. De manera intuitiva se pueden considerar estas direcciones guardadas como *punteros*, dado que indican la posición de un registro. Los registros borrados, por tanto, forman una lista enlazada a la que se suele denominar **lista libre**. La Figura 11.8 muestra el archivo de la Figura 11.5, con la lista libre, después de haberse borrado los registros 1, 4 y 6.

Al insertar un registro nuevo se utiliza el registro al que apunta la cabecera. Se modifica el puntero de la cabecera para que señale al siguiente registro disponible. Si no hay espacio disponible, se añade el nuevo registro al final del archivo.

La inserción y el borrado de archivos con registros de longitud fija son sencillas de implementar, dado que el espacio que deja libre cada registro borrado es exactamente el mismo que se necesita para insertar otro registro. Si se permiten en un archivo registros de longitud variable, esta coincidencia no se mantiene. Puede que el registro insertado no quepa en el espacio liberado por el registro borrado, o que sólo llene una parte.

11.6.2 Registros de longitud variable

Los registros de longitud variable surgen de varias maneras en los sistemas de bases de datos:

- Almacenamiento de varios tipos de registros en un mismo archivo.

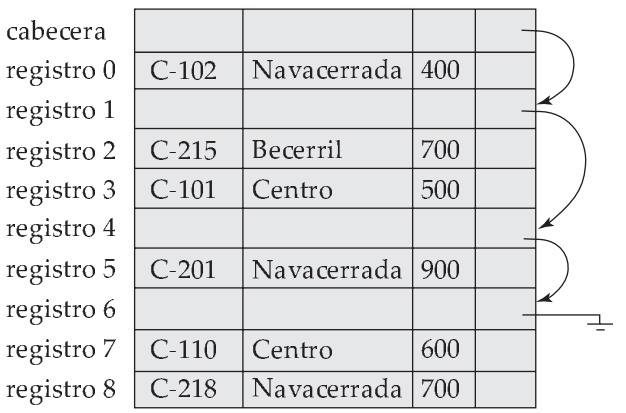


Figura 11.8 El archivo de la Figura 11.5 con la lista libre después del borrado de los registros 1, 4 y 6.

www.detodoprogramacion.com

- Tipos de registro que permiten longitudes variables para uno o varios de los campos.
- Tipos de registro que permiten campos repetidos, como los arrays o los multiconjuntos.

Existen diferentes técnicas para implementar los registros de longitud variable.

La **estructura de páginas con ranuras** se utiliza habitualmente para organizar los registros en bloques, y puede verse en la Figura 11.9. Hay una cabecera al principio de cada bloque, que contiene la información siguiente:

1. El número de elementos del registro de la cabecera.
2. El final del espacio vacío del bloque.
3. Un *array* cuyas entradas contienen la ubicación y el tamaño de cada registro.

Los registros reales se ubican en el bloque *de manera contigua*, empezando por el final. El espacio libre dentro del bloque es contiguo, entre la última entrada del array de la cabecera y el primer registro. Si se inserta un registro, se le asigna espacio al final del espacio libre y se añade a la cabecera una entrada que contiene su tamaño y su ubicación.

Si se borra un registro, se libera el espacio que ocupa y se da el valor de borrada a su entrada (por ejemplo, se le da a su tamaño el valor de -1). Además, se desplazan los registros del bloque situados antes del registro borrado, de modo que se ocupe el espacio libre creado por el borrado y todo el espacio libre vuelve a hallarse entre la última entrada del array de la cabecera y el primer registro. También se actualiza de manera adecuada el puntero de final del espacio libre de la cabecera. Se puede aumentar o disminuir el tamaño de los registros mediante técnicas parecidas, siempre y cuando quede espacio en el bloque. El coste de trasladar los registros no es demasiado elevado, ya que el tamaño del bloque es limitado: un valor habitual es cuatro kilobytes.

La estructura de páginas con ranuras necesita que no haya punteros que señalen directamente a los registros. Por el contrario, los punteros deben apuntar a la entrada de la cabecera que contiene la ubicación verdadera del registro. Este nivel de tratamiento por la dirección permite que se desplacen los registros para evitar la fragmentación del espacio del bloque al tiempo que permite los punteros indirectos al registro.

Las bases de datos almacenan a menudo datos que pueden ser mucho más grandes que los bloques del disco. Por ejemplo, las imágenes o las grabaciones de sonido pueden tener un tamaño de varios megabytes, mientras que los objetos de vídeo pueden llegar a los gigabytes. Recuérdese que SQL soporta los tipos **blob** y **clob**, que almacenan objetos de gran tamaño de los tipos binario y carácter, respectivamente.

La mayor parte de las bases de datos relacionales limitan el tamaño de los registros para que no superen el tamaño de los bloques, con objeto de simplificar la gestión de la memoria intermedia y del espacio libre. Los objetos de gran tamaño y los campos largos suelen guardarse en archivos especiales (o conjuntos de archivos) en lugar de almacenarse con los otros atributos de pequeño tamaño de los registros en los que aparecen. Los objetos de gran tamaño se suelen representar en organizaciones de archivos de árboles B^+, que se estudian en el Apartado 12.3.4. Estas organizaciones permiten leer el objeto completo o rangos concretos de bytes del mismo, así como insertar y borrar partes del objeto.

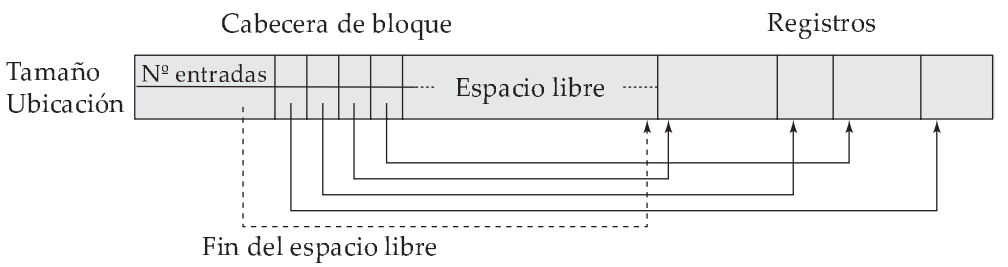


Figura 11.9 Estructura de páginas con ranuras.

www.detodoprogramacion.com

11.7 Organización de los registros en archivos

Hasta ahora se ha estudiado la manera en que se representan los registros en la estructura de archivo. Las relaciones son conjuntos de registros. Dado un conjunto de registros, la pregunta siguiente es cómo organizarlos en archivos. A continuación se indican varias maneras de organizar los registros en archivos:

- **Organización de los archivos en montículos**. Se puede colocar cualquier registro en cualquier parte del archivo en que haya espacio suficiente. Los registros no se ordenan. Generalmente sólo hay un archivo para cada relación.
- **Organización secuencial de los archivos**. Los registros se guardan en orden secuencial, según el valor de la "clave de búsqueda" de cada uno. El Apartado 11.7.1 describe esta organización.
- **Organización asociativa (hash) de los archivos**. Se calcula una función de asociación (*hash*) para algún atributo de cada registro. El resultado de la función de asociación especifica el bloque del archivo en que se debe colocar cada registro. El Capítulo 12 describe esta organización; está estrechamente relacionada con las estructuras para la creación de índices que se describen en ese capítulo.

Generalmente se emplea un archivo separado para almacenar los registros de cada relación. No obstante, en cada **organización de archivos en agrupaciones de varias tablas** se pueden guardar en el mismo archivo registros de relaciones diferentes; además, los registros relacionados de las diferentes relaciones se guardan en el mismo bloque, por lo que cada operación de E/S afecta a registros relacionados de todas esas relaciones. Por ejemplo, los registros de dos relaciones se pueden considerar relacionados si casan en una reunión de las dos relaciones. El Apartado 11.7.2 describe esta organización.

11.7.1 Organización de archivos secuenciales

Los **archivos secuenciales** están diseñados para el procesamiento eficiente de los registros de acuerdo con un orden basado en alguna clave de búsqueda. La **clave de búsqueda** es cualquier atributo o conjunto de atributos; no tiene por qué ser la clave primaria, ni siquiera una superclave. Para permitir la recuperación rápida de los registros según el orden de la clave de búsqueda, éstos se vinculan mediante punteros. El puntero de cada registro señala al siguiente registro según el orden indicado por la clave de búsqueda. Además, para minimizar el número de accesos a los bloques en el procesamiento de los archivos secuenciales, los registros se guardan físicamente en el orden indicado por la clave de búsqueda, o lo más cercano posible.

La Figura 11.10 muestra un archivo secuencial de registros de *cuenta* tomado del ejemplo bancario propuesto. En ese ejemplo, los registros se guardan de acuerdo con el orden de la clave de búsqueda; en este caso, *nombre_sucursal*.

La organización secuencial de archivos permite que los registros se lean de forma ordenada, lo que puede resultar útil para la visualización, así como para ciertos algoritmos de procesamiento de consultas que se estudian en el Capítulo 13.

Sin embargo, resulta difícil mantener el orden físico secuencial a medida que se insertan y se borran registros, dado que resulta costoso desplazar muchos registros como consecuencia de una sola operación de inserción o de borrado. Se puede gestionar el borrado mediante cadenas de punteros, como ya se ha visto. Para la inserción se aplican las reglas siguientes:

1. Localizar el registro del archivo que precede al registro que se va a insertar según el orden de la clave de búsqueda.
2. Si existe algún registro vacío (es decir, un espacio que haya quedado libre después de una operación de borrado) dentro del mismo bloque que ese registro, el registro nuevo se insertará ahí. En caso contrario, el nuevo registro se insertará en un *bloque de desbordamiento*. En cualquier caso, hay que ajustar los punteros para vincular los registros según el orden de la clave de búsqueda.

www.detodoprogramacion.com

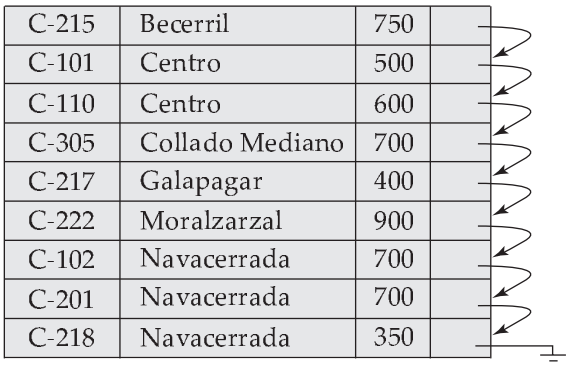


Figura 11.10 Archivo secuencial para los registros de *cuenta*.

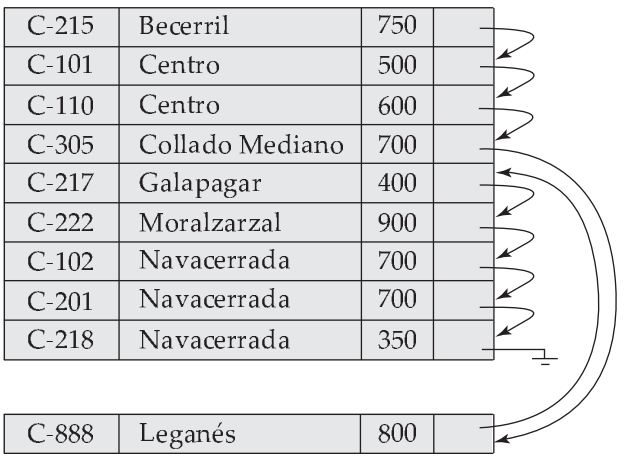


Figura 11.11 El archivo secuencial después de una inserción.

La Figura 11.11 muestra el archivo de la Figura 11.10 después de la inserción del registro (C-888, Leganés, 800). La estructura de la Figura 11.11 permite la inserción rápida de registros nuevos, pero obliga a las aplicaciones de procesamiento de archivos secuenciales a procesar los registros en un orden que no coincide con el físico.

Si hay que guardar un número relativamente pequeño de registros en los bloques de desbordamiento, este enfoque funciona bien. Finalmente, no obstante, la correspondencia entre el orden de la clave de búsqueda y el físico puede perderse totalmente, en cuyo caso el procesamiento secuencial acaba siendo mucho menos eficiente. Llegados a este punto, se debe **reorganizar** el archivo de modo que vuelva a estar físicamente en orden secuencial. Estas reorganizaciones resultan costosas y deben realizarse en momentos en los que la carga del sistema sea baja. La frecuencia con la que las reorganizaciones son necesarias depende de la frecuencia de inserción de registros nuevos. En el caso extremo en que rara vez se produzcan inserciones, es posible mantener siempre el archivo en el orden físico correcto. En ese caso, el campo puntero de la Figura 11.10 no es necesario.

11.7.2 Organización de archivos en agrupaciones de varias tablas

Muchos sistemas de bases de datos relacionales guardan cada relación en un archivo diferente, de modo que puedan aprovechar completamente el sistema de archivos proporcionado por el sistema operativo. Generalmente las tuplas de cada relación se pueden representar como registros de longitud fija. Por tanto, se puede hacer que las relaciones se correspondan con una estructura de archivos sencilla. Esta implementación sencilla de los sistemas de bases de datos relacionales resulta adecuada para las implementaciones de bajo coste de las bases de datos como, por ejemplo, los sistemas empotrados o los dispositivos portátiles. En estos sistemas el tamaño de la base de datos es pequeño, por lo que se obtiene

www.detodoprogramacion.com

nombre_cliente	*número_cuenta*
López	C-102
López	C-220
López	C-503
Abril	C-305

Figura 11.12 La relación *impositor*.

poco provecho de una estructura de archivos avanzada. Además, en esos entornos, es fundamental que el tamaño total del código objeto del sistema de bases de datos sea pequeño. Una estructura de archivos sencilla reduce la cantidad de código necesaria para implementar el sistema.

Este enfoque sencillo de la implementación de las bases de datos relacionales resulta menos satisfactorio a medida que aumenta el tamaño de la base de datos. Ya se ha visto que se pueden obtener mejoras en el rendimiento mediante la asignación esmerada de los registros a los bloques y la cuidadosa organización de los propios bloques. Por tanto, resulta evidente que una estructura de archivos más compleja puede resultar beneficiosa, aunque se mantenga la estrategia de guardar cada relación en un archivo diferente.

Sin embargo, muchos sistemas de bases de datos de gran tamaño no utilizan directamente el sistema operativo subyacente para la gestión de los archivos. Por el contrario, se asigna al sistema de bases de datos un archivo de gran tamaño del sistema operativo. En este archivo, el sistema de bases de datos, que también lo administra, guarda todas las relaciones. Para comprender la ventaja de guardar muchas relaciones en un solo archivo considérese la siguiente consulta SQL de la base de datos bancaria:

select *número_cuenta, nombre_cliente, calle_cliente, ciudad_cliente*
from *impositor, cliente*
where *impositor.nombre_cliente = cliente.nombre_cliente*

Esta consulta calcula una reunión de las relaciones *impositor* y *cliente*. Por tanto, por cada tupla *impositor* el sistema debe encontrar las tuplas de *cliente* con el mismo valor de *nombre_cliente*. Lo ideal sería poder encontrar estos registros con la ayuda de *índices*, que se estudiarán en el Capítulo 12. Independientemente de la manera en que se encuentren esos registros, hay que transferirlos desde el disco a la memoria principal. En el peor de los casos, cada registro se hallará en un bloque diferente, lo que obligará a efectuar un proceso de lectura de bloque por cada registro necesario para la consulta.

A modo de ejemplo, considérense las relaciones *impositor* y *cliente* de las Figuras 11.12 y 11.13, respectivamente. En la Figura 11.14 se muestra una estructura de archivo diseñada para la ejecución eficiente de las consultas que implican $impositor \bowtie cliente$. Las tuplas *impositor* para cada *nombre_cliente* se guardan cerca de la tupla *cliente* para el *nombre_cliente* correspondiente. Esta estructura mezcla las tuplas de dos relaciones, pero permite el procesamiento eficaz de la reunión. Cuando se lee una tupla de la relación *cliente*, se copia del disco a la memoria principal todo el bloque que contiene esa tupla. Dado que las tuplas correspondientes de *impositor* se guardan en el disco cerca de la tupla *cliente*, el bloque que contiene la tupla *cliente* también contiene tuplas de la relación *impositor* necesarias para procesar la consulta. Si un cliente tiene tantas cuentas que los registros de *impositor* no caben en un solo bloque, los registros restantes aparecerán en bloques cercanos.

Una **organización de archivos en agrupaciones de varias tablas** es una organización de archivos, como la mostrada en la Figura 11.14, que almacena registros relacionados de dos o más relaciones en cada bloque. Este tipo de organización de archivos permite leer registros que satisfacen la condición de

nombre_cliente	*calle_cliente*	*ciudad_cliente*
López	Mayor	Arganzuela
Abril	Preciados	Valsaín

Figura 11.13 La relación *cliente*.

www.detodoprogramacion.com

López	Mayor	Arganzuela
López	C-102	
López	C-220	
López	C-503	
Abril	Preciados	Valsaín
Abril	C-305	

Figura 11.14 Estructura de archivo en agrupaciones de varias tablas.

reunión en un solo proceso de lectura de bloques. Por tanto, esta consulta concreta se puede procesar de manera más eficiente.

El empleo de la agrupación de varias tablas en un único archivo ha mejorado el procesamiento de una reunión concreta (*impositor* ⋈ *cliente*) pero retarda el procesamiento de otros tipos de consulta. Por ejemplo:

select *
from *cliente*

necesita más accesos a los bloques que con el esquema en el que cada relación se guardaba en un archivo diferente. En lugar de aparecer varios registros de *cliente* en un mismo bloque, cada registro se halla en un bloque diferente. En realidad, lograr hallar todos los registros de *cliente* resulta imposible sin alguna estructura adicional. Para encontrar todas las tuplas de la relación *cliente* en la estructura de la Figura 11.14 se pueden enlazar todos los registros de esa relación mediante punteros, tal y como se muestra en la Figura 11.15.

El empleo de la agrupación de varias tablas depende de los tipos de consulta que el diseñador de la base de datos considere más frecuentes. El uso cuidadoso de la agrupación de varias tablas puede producir mejoras de rendimiento significativas en el procesamiento de las consultas.

11.8 Almacenamiento con diccionarios de datos

Hasta ahora sólo se ha considerado la representación de las propias relaciones. Un sistema de bases de datos relacionales necesita tener datos *sobre* las relaciones, como puede ser su esquema. Esta información se denomina **diccionario de datos** o **catálogo del sistema**. Entre los tipos de información que debe guardar el sistema figuran los siguientes:

- El nombre de las relaciones.
- El nombre de los atributos de cada relación.
- El dominio y la longitud de los atributos.
- El nombre de las vistas definidas en la base de datos, y la definición de esas vistas.
- Las restricciones de integridad (por ejemplo, las restricciones de las claves).

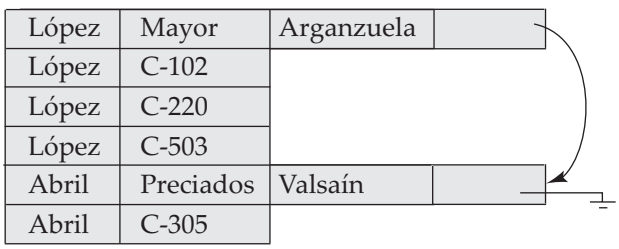


Figura 11.15 Estructura de archivo con agrupaciones de varias tablas y cadenas de punteros.

www.detodoprogramacion.com

Metadatos_relación (*nombre_relación*, *número_de_atributos*, *organización_almacenamiento*, *ubicación*)
Metadatos_atributos (*nombre_atributo*, *nombre_relación*, *tipo_dominio*, *posición*, *longitud*)
Metadatos_usuarios (*nombre_usuario*, *contraseña_cifrada*, *grupo*)
Metadatos_índices (*nombre_índice*, *nombre_relación*, *tipo_índice*, *atributos_índice*)
Metadatos_vistas (*nombre_vista*, *definición*)

Figura 11.16 Base de datos relacional que representa datos del sistema.

Además, muchos sistemas guardan los siguientes datos de los usuarios del sistema:

- El nombre de los usuarios autorizados.
- La autorización y la información sobre las cuentas de los usuarios.
- Las contraseñas u otra información utilizada para autentificar a los usuarios.

Además, la base de datos puede guardar información estadística y descriptiva sobre las relaciones, como:

- El número de tuplas de cada relación.
- El método de almacenamiento utilizado para cada relación (por ejemplo, con agrupaciones o sin agrupaciones).

El diccionario de datos puede también tener en cuenta la organización del almacenamiento (secuencial, asociativa o en montículos) de las relaciones y la ubicación donde se guarda cada relación:

- Si las relaciones se almacenan en archivos del sistema operativo, el diccionario tendrá en cuenta el nombre del archivo (o archivos) que guarda cada relación.
- Si la base de datos almacena todas las relaciones en un solo archivo, puede que el diccionario tenga en cuenta los bloques que almacenan los registros de cada relación en una estructura de datos como, por ejemplo, una lista enlazada.

En el Capítulo 12, en el que se estudian los índices, se verá que hace falta guardar información sobre cada índice de cada una de las relaciones:

- El nombre del índice.
- El nombre de la relación para la que se crea.
- Los atributos sobre los que se define.
- El tipo de índice formado.

Toda esta información constituye, en efecto, una base de datos en miniatura. Algunos sistemas de bases de datos guardan esta información utilizando código y estructuras de datos especiales. Suele resultar preferible guardar los datos sobre la base de datos en la misma base de datos. Al utilizar la base de datos para guardar los datos del sistema se simplifica la estructura global del sistema y se dedica toda la potencia de la base de datos a obtener un acceso rápido a los datos del sistema.

La elección exacta de la manera de representar los datos del sistema mediante las relaciones debe tomarla el diseñador del sistema. La Figura 11.16 ofrece una representación posible, con las claves primarias subrayadas. En esta representación se da por supuesto que el atributo *atributos_índice* de la relación *Metadatos_índices* contiene una lista de uno o varios atributos, que se pueden representar mediante una cadena de caracteres como "*nombre_sucursal, ciudad_sucursal*". Por tanto, la relacion *Metadatos_índices* no está en la primera forma normal; se puede normalizar, pero es probable que la representación anterior sea más eficiente en el acceso. El diccionario de datos se suele almacenar de forma no normalizada para conseguir un acceso rápido.

www.detodoprogramacion.com

La organización del almacenamiento y de la ubicación de la propia *Metadatos_relación* se debe registrar en otro lugar (por ejemplo, en el propio código de la base de datos) ya que esa información es necesaria para encontrar el contenido de *Metadatos_relación*.

11.9 Resumen

- En la mayor parte de los sistemas informáticos hay varios tipos de almacenamiento de datos. Se clasifican según la velocidad con la que se puede acceder a los datos, el coste de adquisición de la memoria por unidad de datos y su fiabilidad. Entre los medios disponibles figuran la memoria caché, la principal, la flash, los discos magnéticos, los discos ópticos y las cintas magnéticas.
- La fiabilidad de los medios de almacenamiento la determinan dos factores: si un corte en el suministro eléctrico o una caída del sistema hace que los datos se pierdan y la probabilidad de fallo físico del dispositivo de almacenamiento.
- Se puede reducir la probabilidad del fallo físico conservando varias copias de los datos. Para los discos se pueden crear imágenes. También se pueden usar métodos más sofisticados como las disposiciones redundantes de discos independientes (RAID). Mediante la distribución de los datos entre los discos estos métodos ofrecen elevados niveles de productividad en los accesos de gran tamaño; la introducción de la redundancia entre los discos se mejora mucho la fiabilidad. Se han propuesto varias organizaciones RAID diferentes, con características de coste, rendimiento y fiabilidad diferentes. Las organizaciones RAID de nivel 1 (creación de imágenes) y de nivel 5 son las más utilizadas.
- Una manera de reducir el número de accesos al disco es conservar todos los bloques posibles en la memoria principal. Dado que allí no se pueden guardar todos, hay que gestionar la asignación del espacio disponible en la memoria principal para el almacenamiento de los bloques. La *memoria intermedia* (*buffer*) es la parte de la memoria principal disponible para el almacenamiento de las copias de los bloques del disco. El subsistema responsable de la asignación del espacio de la memoria intermedia se denomina *gestor de la memoria intermedia*.
- Los archivos se pueden organizar lógicamente como secuencias de registros asignados a bloques de disco. Un enfoque de la asociación de la base de datos con los archivos es utilizar varios archivos y guardar registros de una única longitud fija en cualquier archivo dado. Una alternativa es estructurar los archivos de modo que puedan aceptar registros de longitud variable. El método de la página con ranuras se usa mucho para manejar los registros de longitud variable en los bloques de disco.
- Dado que los datos se transfieren entre el almacenamiento en disco y la memoria principal en unidades de bloques merece la pena asignar los registros de los archivos a los bloques de modo que cada bloque contenga registros relacionados entre sí. Si se puede tener acceso a varios de los registros deseados utilizando sólo un acceso a bloques, se evitan accesos al disco. Dado que los accesos al disco suelen ser el cuello de botella del rendimiento de los sistemas de bases de datos, la esmerada asignación de registros a los bloques puede ofrecer mejoras significativas del rendimiento.
- El diccionario de datos, también denominado catálogo del sistema, guarda información de los metadatos, que son datos relativos a los datos, como el nombres de las relaciones, el nombre de los atributos y su tipo, la información de almacenamiento, las restricciones de integridad y la información de usuario.

Términos de repaso

- Medios de almacenamiento físico:
  - ☐ Caché.
  - ☐ Memoria principal.
  - ☐ Memoria flash.
  - ☐ Disco magnético.
  - ☐ Almacenamiento óptico.
- Disco magnético

www.detodoprogramacion.com

CAPÍTULO 12

Indexación y asociación

Muchas consultas hacen referencia sólo a una pequeña parte de los registros de un archivo. Por ejemplo, la consulta "Buscar todas las cuentas de la sucursal Pamplona" o "Buscar el saldo del número de cuenta C-101" hace referencia solamente a una fracción de los registros de la relación cuenta. No es eficiente que el sistema tenga que leer cada registro y comprobar que el campo *nombre_sucursal* contiene el nombre "Pamplona" o el valor C-101 del campo *número_cuenta*. Lo más adecuado sería que el sistema fuese capaz de localizar directamente esos registros. Para facilitar estas formas de acceso se diseñan estructuras adicionales que se asocian con los archivos.

12.1 Conceptos básicos

Un índice para un archivo del sistema funciona como el índice de este libro. Si se va a buscar un tema (especificado por una palabra o una frase) se puede buscar en el índice al final del libro, encontrar las páginas en las que aparece y después leerlas para encontrar la información buscada. Las palabras del índice están ordenadas alfabéticamente, lo cual facilita la búsqueda. Además, el índice es mucho más pequeño que el libro, con lo que se reduce aún más el esfuerzo necesario para encontrar las palabras en cuestión.

Los índices de los sistemas de bases de datos juegan el mismo papel que los índices de los libros en las bibliotecas. Por ejemplo, para recuperar un registro *cuenta* dado su número de cuenta, el sistema de bases de datos buscaría en un índice para encontrar el bloque de disco en que se localice el registro correspondiente, y entonces extraería ese bloque de disco para obtener el registro *cuenta*.

Almacenar una lista ordenada de números de cuenta no funcionaría bien en bases de datos muy grandes con millones de cuentas, ya que el propio índice sería muy grande; más aún, incluso el mantener ordenado el índice reduce el tiempo de búsqueda, por lo que encontrar una cuenta podría consumir mucho tiempo. En su lugar se usan técnicas más sofisticadas de indexación. Algunas de estas técnicas se estudiarán más adelante.

Hay dos tipos básicos de índices:

- **Índices ordenados**. Estos índices están basados en una disposición ordenada de los valores.
- **Índices asociativos**. Estos índices están basados en una distribución uniforme de los valores a través de una serie de cajones (*buckets*). El valor asignado a cada cajón está determinado por una función, llamada *función de asociación* (*hash function)*.

Se considerarán varias técnicas de indexación y asociación. Ninguna de ellas es la mejor. Sin embargo, para cada aplicación específica de bases de datos existe una técnica más apropiada. Cada una de ellas debe ser valorada según los siguientes criterios:

www.detodoprogramacion.com

- **Tipos de acceso**. Los tipos de acceso que se soportan eficazmente. Estos tipos podrían incluir la búsqueda de registros con un valor concreto en un atributo, o la búsqueda de los registros cuyos atributos contengan valores en un rango especificado.
- **Tiempo de acceso**. El tiempo que se tarda en hallar un determinado elemento de datos, o conjunto de elementos, usando la técnica en cuestión.
- **Tiempo de inserción**. El tiempo empleado en insertar un nuevo elemento de datos. Este valor incluye el tiempo utilizado en hallar el lugar apropiado donde insertar el nuevo elemento de datos, así como el tiempo empleado en actualizar la estructura del índice.
- **Tiempo de borrado**. El tiempo empleado en borrar un elemento de datos. Este valor incluye el tiempo utilizado en hallar el elemento a borrar, así como el tiempo empleado en actualizar la estructura del índice.
- **Espacio adicional requerido**. El espacio adicional ocupado por la estructura del índice. Como normalmente la cantidad necesaria de espacio adicional suele ser moderada, es razonable sacrificar el espacio para alcanzar un rendimiento mejor.

A menudo se desea tener más de un índice por archivo. Por ejemplo se puede buscar un libro según el autor, la materia o el título.

Los atributos o conjunto de atributos usados para buscar en un archivo se llaman **claves de búsqueda**. Hay que observar que esta definición de *clave* difiere de la usada en *clave primaria*, *clave candidata* y *superclave*. Este doble significado de *clave* está (desafortunadamente) muy extendido en la práctica. Usando el concepto de clave de búsqueda se determina que, si existen varios índices para un archivo, existirán varias claves de búsqueda.

12.2 Índices ordenados

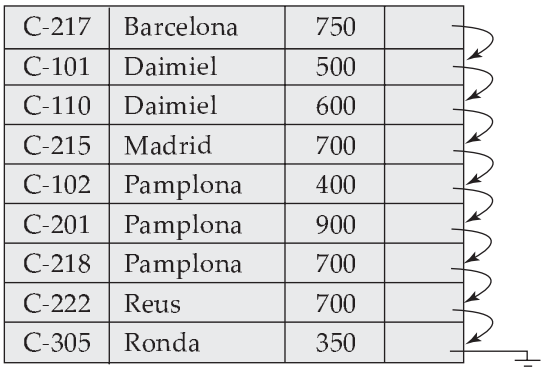
Para permitir un acceso directo rápido a los registros de un archivo se puede usar una estructura de índice. Cada estructura de índice está asociada con una clave de búsqueda concreta. Al igual que en el catálogo de una biblioteca, un índice almacena de manera ordenada los valores de las claves de búsqueda, y asocia a cada clave los registros que contienen esa clave de búsqueda.

Los registros en el archivo indexado pueden estar a su vez almacenados siguiendo un orden, semejante a como los libros están ordenados en una biblioteca por algún atributo como el número decimal Dewey. Un archivo puede tener varios índices según diferentes claves de búsqueda. Si el archivo que contiene los registros está ordenado secuencialmente, el índice cuya clave de búsqueda especifica el orden secuencial del archivo es el **índice con agrupación** (clustering index). Los índices con agrupación también se llaman **índices primarios**; el término índice primario se usa algunas veces para hacer alusión a un índice según una clave primaria pero en realidad se puede usar sobre cualquier clave de búsqueda. La clave de búsqueda de un índice con agrupación es normalmente la clave primaria, aunque no es así necesariamente. Los índices cuyas claves de búsqueda especifican un orden diferente del orden secuencial del archivo se llaman **índices sin agrupación** (non clustering indices) o **secundarios**. Se suelen usar los términos "agrupado" y "no agrupado" en lugar de "con agrupación" y "sin agrupación".

Desde el Apartado 12.2.1 hasta el 12.2.3 se asume que todos los archivos están ordenados secuencialmente según alguna clave de búsqueda. Estos archivos con un índice con agrupación según la clave de búsqueda se llaman **archivos secuenciales indexados**. Representan uno de los esquemas de índices más antiguos usados por los sistemas de bases de datos. Se usan en aquellas aplicaciones que demandan un procesamiento secuencial del archivo completo, así como un acceso directo a sus registros. En el Apartado 12.2.4 se tratan los índices secundarios.

En la Figura 12.1 se muestra un archivo secuencial de los registros *cuenta* tomados del ejemplo bancario. En esta figura, los registros están almacenados según el orden de la clave de búsqueda: *nombre_sucursal*.

www.detodoprogramacion.com



C-217	Barcelona	750	
C-101	Daimiel	500	
C-110	Daimiel	600	
C-215	Madrid	700	
C-102	Pamplona	400	
C-201	Pamplona	900	
C-218	Pamplona	700	
C-222	Reus	700	
C-305	Ronda	350	

Figura 12.1 Archivo secuencial para los archivos de *cuenta*.

12.2.1 Índices densos y dispersos

Un **registro índice** o **entrada del índice** consiste en un valor de la clave de búsqueda y punteros a uno o más registros con ese valor de la clave de búsqueda. El puntero a un registro consiste en el identificador de un bloque de disco y un desplazamiento en el bloque de disco para identificar el registro dentro del bloque.

Existen dos clases de índices ordenados que se pueden usar:

- **Índice denso**. Aparece un registro índice por cada valor de la clave de búsqueda en el archivo. En un índice denso con agrupación el registro índice contiene el valor de la clave y un puntero al primer registro con ese valor de la clave de búsqueda. El resto de registros con el mismo valor de la clave de búsqueda se almacenan consecutivamente después del primer registro, dado que, ya que el índice es con agrupación, los registros se ordenan sobre la misma clave de búsqueda.

  Las implementaciones de índices densos pueden almacenar una lista de punteros a todos los registros con el mismo valor de la clave de búsqueda; esto no es esencial para los índices con agrupación.

- **Índice disperso**. Sólo se crea un registro índice para algunos de los valores. Al igual que en los índices densos, cada registro índice contiene un valor de la clave de búsqueda y un puntero al primer registro con ese valor de la clave. Para localizar un registro se busca la entrada del índice con el valor más grande que sea menor o igual que el valor que se está buscando. Se empieza por el registro apuntado por esa entrada del índice y se continúa con los punteros del archivo hasta encontrar el registro deseado.

Las Figuras 12.2 y 12.3 son ejemplos de índices densos y dispersos, respectivamente, para el archivo *cuenta*. Supóngase que se desea buscar los registros de la sucursal Pamplona. Mediante el índice denso de la Figura 12.2, se sigue el puntero que va directo al primer registro de Pamplona. Se procesa el registro y se sigue el puntero en ese registro hasta localizar el siguiente registro según el orden de la clave de búsqueda (*nombre_sucursal*). Se continuaría procesando registros hasta encontrar uno cuyo nombre de sucursal fuese distinto de Pamplona. Si se usa un índice disperso (Figura 12.3), no se encontraría entrada del índice para "Pamplona". Como la última entrada (en orden alfabético) antes de "Pamplona" es "Madrid", se sigue ese puntero. Entonces se lee el archivo *cuenta* en orden secuencial hasta encontrar el primer registro Pamplona, y se continúa procesando desde este punto.

Como se ha visto, generalmente es más rápido localizar un registro si se usa un índice denso en vez de un índice disperso. Sin embargo, los índices dispersos tienen algunas ventajas sobre los índices densos, como el utilizar un espacio más reducido y un mantenimiento adicional menor para las inserciones y borrados.

Existe un compromiso que el diseñador del sistema debe mantener entre el tiempo de acceso y el espacio adicional requerido. Aunque la decisión sobre este compromiso depende la aplicación en particular, un buen compromiso es tener un índice disperso con una entrada del índice por cada bloque. La razón por la cual este diseño alcanza un buen compromiso reside en que el mayor coste de procesar

www.detodoprogramacion.com

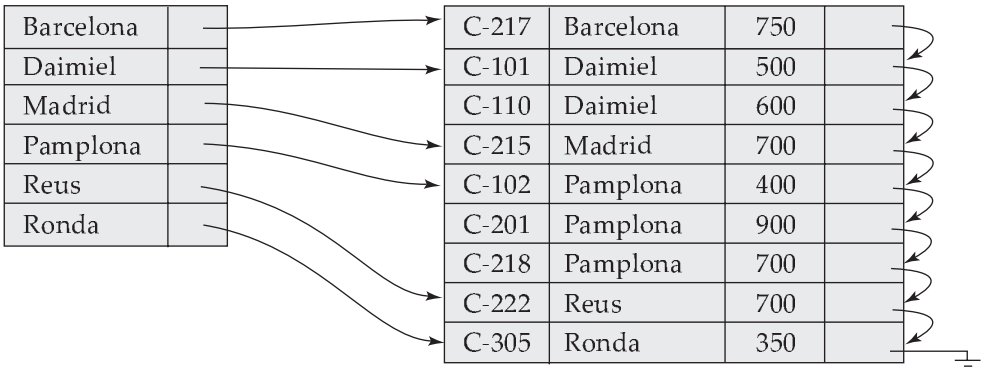


Figura 12.2 Índice denso.

una petición en la base de datos es el empleado en traer un bloque de disco a la memoria. Una vez el bloque en memoria, el tiempo en examinarlo es prácticamente inapreciable. Usando este índice disperso se localiza el bloque que contiene el registro solicitado. De este manera, a menos que el registro esté en un bloque de desbordamiento (véase el Apartado 12.7.1) se minimizan los accesos a bloques mientras el tamaño del índice se mantiene (y así, el espacio adicional requerido) tan pequeño como sea posible.

Para generalizar la técnica anterior hay que tener en cuenta si los registros de una clave de búsqueda ocupan varios bloques. Es fácil modificar el esquema para acomodarse a esta situación.

12.2.2 Índices multinivel

Aunque se use un índice disperso, puede que el propio índice sea demasiado grande para un procesamiento eficiente. En la práctica no es excesivo tener un archivo con 100.000 registros, con 10 registros almacenados en cada bloque. Si se dispone de un registro índice por cada bloque, el índice tendrá 10.000 registros. Como los registros índices son más pequeños que los registros de datos, se puede suponer que caben 100 registros índices en un bloque. Por tanto el índice ocuparía 100 bloques. Estos índices de mayor tamaño se almacenan como archivos secuenciales en disco.

Si un índice es lo bastante pequeño como para guardarlo en la memoria principal, el tiempo de búsqueda para encontrar una entrada será breve. Sin embargo, si el índice es tan grande que se debe guardar en disco, buscar una entrada implicará leer varios bloques de disco. Para localizar una entrada en el archivo índice se puede realizar una búsqueda binaria, pero aun así ésta conlleva un gran coste. Si el índice ocupa b bloques, la búsqueda binaria tendrá que leer a lo sumo $\lceil \log_2(b) \rceil$ bloques. ($\lceil x \rceil$ denota el menor entero que es mayor o igual que x; es decir, se redondea hacia abajo). Para el índice de 100 bloques, la búsqueda binaria necesitará leer siete bloques. En un disco en el que la lectura de un bloque tarda 30 milisegundos, la búsqueda empleará 210 milisegundos, que es bastante. Obsérvese que si se están usando bloques de desbordamiento, la búsqueda binaria no será posible. En ese caso, lo normal es una

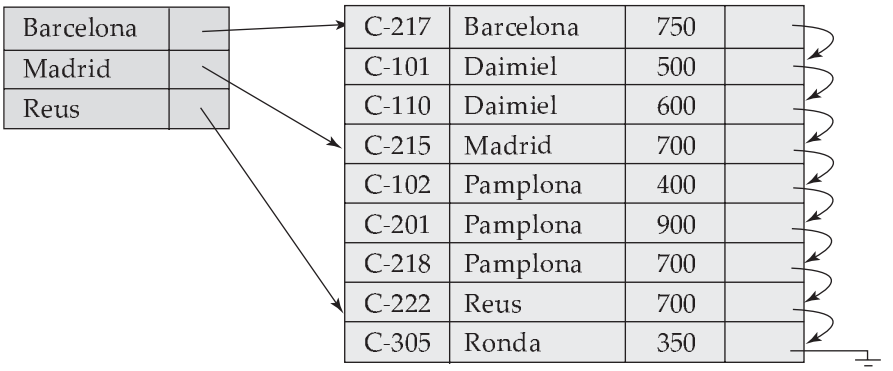


Figura 12.3 Índice disperso.

www.detodoprogramacion.com

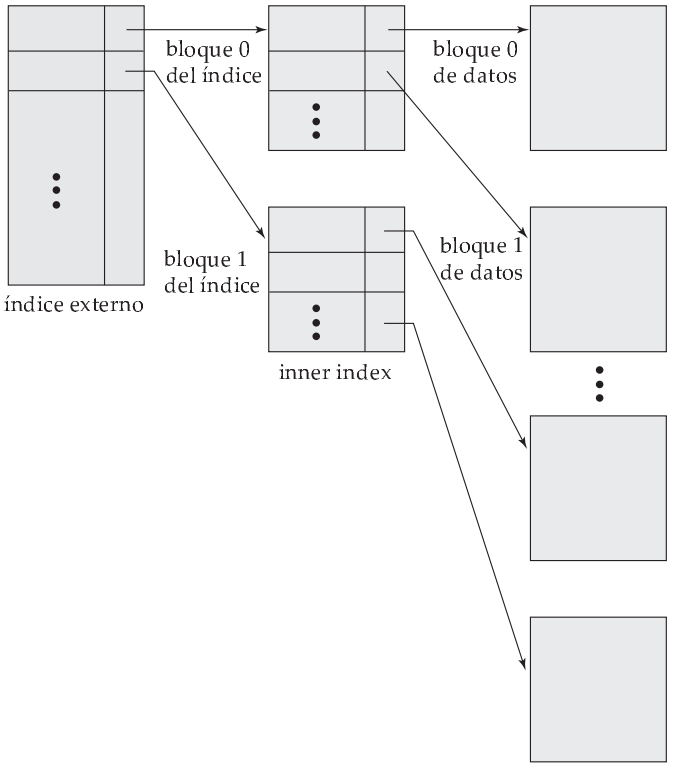


Figura 12.4 Índice disperso de dos niveles.

búsqueda secuencial, y eso requiere leer *b* bloques, lo que podría consumir incluso más tiempo. Así, el proceso de buscar en un índice grande puede ser muy costoso.

Para resolver este problema el índice se trata como si fuese un archivo secuencial y se construye un índice disperso sobre el índice con agrupación, como se muestra en la Figura 12.4. Para localizar un registro se usa en primer lugar una búsqueda binaria sobre el índice más externo para buscar el registro con el mayor valor de la clave de búsqueda que sea menor o igual al valor deseado. El puntero apunta a un bloque en el índice más interno. Hay que examinar este bloque hasta encontrar el registro con el mayor valor de la clave que sea menor o igual que el valor deseado. El puntero de este registro apunta al bloque del archivo que contiene el registro buscado.

Usando los dos niveles de indexación y con el índice más externo en memoria principal hay que leer un único bloque índice, en vez de los siete que se leían con la búsqueda binaria. Si al archivo es extremadamente grande, incluso el índice exterior podría crecer demasiado para caber en la memoria principal. En este caso se podría crear todavía otro nivel más de indexación. De hecho, se podría repetir este proceso tantas veces como fuese necesario. Los índices con dos o más niveles se llaman índices **multinivel**. La búsqueda de registros usando un índice multinivel necesita claramente menos operaciones de E/S que las que se emplean en la búsqueda de registros con la búsqueda binaria. Cada nivel de índice se podría corresponder con una unidad del almacenamiento físico. Así, es posible disponer de índices para pistas, cilindros y discos.

Un diccionario normal es un ejemplo de un índice multinivel en el mundo ajeno a las bases de datos. La cabecera de cada página lista la primera palabra en el orden alfabético en esa página. Este índice es multinivel: las palabras en la parte superior de la página del índice del libro forman un índice disperso sobre los contenidos de las páginas del diccionario.

Los índices multinivel están estrechamente relacionados con la estructura de árbol, tales como los árboles binarios usados para la indexación en memoria. Esta relación se examinará posteriormente en el Apartado 12.3.

www.detodoprogramacion.com

12.2.3 Actualización del índice

Sin importar el tipo de índice que se esté usando, los índices se deben actualizar siempre que se inserte o borre un registro del archivo. A continuación se describirán los algoritmos para actualizar índices de un solo nivel.

- **Inserción**. Primero se realiza una búsqueda usando el valor de la clave de búsqueda del registro a insertar. Las acciones que emprende el sistema a continuación dependen de si el índice es denso o disperso.
  - □ Índices densos:
    1. Si el valor de la clave de búsqueda no aparece en el índice, el sistema inserta en éste un registro índice con el valor de la clave de búsqueda en la posición adecuada.
    2. En caso contrario se emprenden las siguientes acciones:
       a. Si el registro índice almacena punteros a todos los registros con el mismo valor de la clave de búsqueda, el sistema añade un puntero al nuevo registro en el registro índice.
       b. En caso contrario, el registro índice almacena un puntero sólo hacia el primer registro con el valor de la clave de búsqueda. El sistema sitúa el registro insertado después de los otros con los mismos valores de la clave de búsqueda.
  - □ Índices dispersos: se asume que el índice almacena una entrada por cada bloque. Si el sistema crea un bloque nuevo, inserta el primer valor de la clave de búsqueda (en el orden de la clave de búsqueda) que aparezca en el nuevo bloque del índice. Por otra parte, si el nuevo registro tiene el menor valor de la clave de búsqueda en su bloque, el sistema actualiza la entrada del índice que apunta al bloque; si no, el sistema no realiza ningún cambio sobre el índice.

- **Borrado**. Para borrar un registro, primero se busca el índice a borrar. Las acciones que emprende el sistema a continuación dependen de si el índice es denso o disperso.
  - □ Índices densos:
    1. Si el registro borrado era el único registro con ese valor de la clave de búsqueda, el sistema borra el registro índice correspondiente del índice.
    2. En caso contrario se emprenden las siguientes acciones:
       a. Si el registro índice almacena punteros a todos los registros con el mismo valor de la clave de búsqueda, el sistema borra del registro índice el puntero al registro borrado.
       b. En caso contrario, el registro índice almacena un puntero sólo al primer registro con el valor de la clave de búsqueda. En este caso, si el registro borrado era el primer registro con el valor de la clave de búsqueda, el sistema actualiza el registro índice para apuntar al siguiente registro.
  - □ Índices dispersos:
    1. Si el índice no contiene un registro índice con el valor de la clave de búsqueda del registro borrado, no hay que hacer nada.
    2. En caso contrario se emprenden las siguientes acciones:
       a. Si el registro borrado era el único registro con la clave de búsqueda, el sistema reemplaza el registro índice correspondiente con un registro índice para el siguiente valor de la clave de búsqueda (en el orden de la clave de búsqueda). Si el siguiente valor de la clave de búsqueda ya tiene una entrada en el índice, se borra en lugar de reemplazarla.

www.detodoprogramacion.com

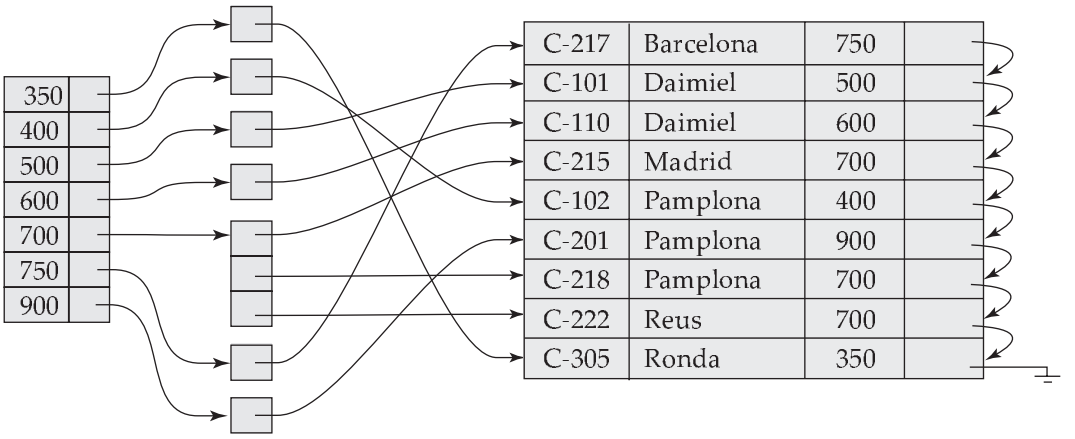


Figura 12.5 Índice secundario del archivo *cuenta*, con la clave no candidata *saldo*.

b. En caso contrario, si el registro índice para el valor de la clave de búsqueda apunta al registro a borrar, el sistema actualiza el registro índice para que apunte al siguiente registro con el mismo valor de la clave de búsqueda.

Los algoritmos de inserción y borrado para los índices multinivel se extienden de manera sencilla a partir del esquema descrito anteriormente. Al borrar o al insertar se actualiza el índice de nivel más bajo como se describió anteriormente. Por lo que respecta al índice del segundo nivel, el índice de nivel más bajo es simplemente un archivo de registros—así, si hay algún cambio en el índice de nivel más bajo, se tendrá que actualizar el índice del segundo nivel como ya se describió. La misma técnica se aplica al resto de niveles del índice, si los hubiera.

12.2.4 Índices secundarios

Los índices secundarios deben ser densos, con una entrada en el índice por cada valor de la clave de búsqueda, y un puntero a cada registro del archivo. Un índice con agrupación puede ser disperso, almacenando sólo algunos de los valores de la clave de búsqueda, ya que siempre es posible encontrar registros con valores de la clave de búsqueda intermedios mediante un acceso secuencial a parte del archivo, como se describió antes. Si un índice secundario almacena sólo algunos de los valores de la clave de búsqueda, los registros con los valores de la clave de búsqueda intermedios pueden estar en cualquier lugar del archivo y, en general, no se pueden encontrar sin explorar el archivo completo.

Un índice secundario sobre una clave candidata es como un índice denso con agrupación, excepto en que los registros apuntados por los sucesivos valores del índice no están almacenados secuencialmente. Generalmente los índices secundarios están estructurados de manera diferente a como lo están los índices con agrupación. Si la clave de búsqueda de un índice con agrupación no es una clave candidata, es suficiente si el valor de cada entrada en el índice apunta al primer registro con ese valor en la clave de búsqueda, ya que los otros registros podrían ser alcanzados por una búsqueda secuencial del archivo.

En cambio, si la clave de búsqueda de un índice secundario no es una clave candidata, no sería suficiente apuntar sólo al primer registro de cada valor de la clave. El resto de registros con el mismo valor de la clave de búsqueda podría estar en cualquier otro sitio del archivo, ya que los registros están ordenados según la clave de búsqueda del índice con agrupación, en vez de la clave de búsqueda del índice secundario. Por tanto, un índice secundario debe contener punteros a todos los registros.

Se puede usar un nivel adicional de referencia para implementar los índices secundarios sobre claves de búsqueda que no sean claves candidatas. Los punteros en estos índices secundarios no apuntan directamente al archivo. En vez de eso, cada puntero apunta a un cajón que contiene punteros al archivo. En la Figura 12.5 se muestra la estructura del archivo *cuenta*, con un índice secundario que usa un nivel de referencia adicional, y teniendo como clave de búsqueda el *saldo.*

Siguiendo el orden de un índice primario, una búsqueda secuencial es eficiente porque los registros del archivo están guardados físicamente de la misma manera a como está ordenado el índice. Sin embar-

www.detodoprogramacion.com

go, no se puede (salvo en casos excepcionales) almacenar el archivo ordenado físicamente por el orden de la clave de búsqueda del índice con agrupación y la clave de búsqueda del índice secundario. Ya que el orden de la clave secundaria y el orden físico difieren, si se intenta examinar el archivo secuencialmente por el orden de la clave secundaria, es muy probable que la lectura de cada bloque suponga la lectura de un nuevo bloque del disco, lo cual es muy lento.

El procedimiento ya descrito para borrar e insertar se puede aplicar también a los índices secundarios; las acciones a emprender son las descritas para los índices densos que almacenan un puntero a cada registro del archivo. Si un archivo tiene varios índices, siempre que se modifique el archivo, se debe actualizar *cada* uno de ellos.

Los índices secundarios mejoran el rendimiento de las consultas que usan claves que no son la de búsqueda del índice con agrupación. Sin embargo, implican un tiempo adicional importante al modificar la base de datos. El diseñador de la base de datos debe decidir qué índices secundarios son deseables, según una estimación sobre las frecuencias de las consultas y de las modificaciones.

12.3 Archivos de índices de árbol B^+

El inconveniente principal de la organización de un archivo secuencial indexado reside en que el rendimiento, tanto para buscar en el índice como para buscar secuencialmente a través de los datos, se degrada según crece el archivo. Aunque esta degradación se puede remediar reorganizando el archivo, no es deseable hacerlo frecuentemente.

La estructura de índice de **árbol B^+** es la más extendida de las estructuras de índices que mantienen su eficiencia a pesar de la inserción y borrado de datos. Un índice de árbol B^+ toma la forma de un **árbol equilibrado** donde los caminos de la raíz a cada hoja del árbol son de la misma longitud. Cada nodo que no sea hoja tiene entre $\lceil n/2 \rceil$ y n hijos, donde n es fijo para cada árbol concreto.

Se verá que la estructura de árbol B^+ implica una degradación del rendimiento al insertar y al borrar, además de un espacio extra. Este tiempo adicional es aceptable incluso en archivos con altas frecuencias de modificación, ya que se evita el coste de reorganizar el archivo. Además, puesto que los nodos podrían estar a lo sumo medio llenos (si tienen el mínimo número de hijos) se desperdicia algo de espacio. Este gasto de espacio adicional también es aceptable dados los beneficios en el rendimiento aportados por las estructura de árbol B^+.

12.3.1 Estructura de árbol B^+

Un índice de árbol B^+ es un índice multinivel pero con una estructura que difiere del índice multinivel de un archivo secuencial. En la Figura 12.6 se muestra un nodo típico de un árbol B^+. Puede contener hasta $n - 1$ claves de búsqueda $K_1, K_2, \ldots, K_{n-1}$ y n punteros $P_1, P_2, \ldots, P_n$. Los valores de la clave de búsqueda de un nodo se mantienen ordenados; así, si $i < j$, entonces $K_i < K_j$.

Considérese primero la estructura de los nodos hoja. Para $i = 1, 2, \ldots, n-1$, el puntero P_i apunta, o bien a un registro del archivo con valor de la clave de búsqueda K_i, o bien a un cajón de punteros, cada uno de los cuales apunta a un registro del archivo con valor de la clave de búsqueda K_i. La estructura cajón se usa solamente si la clave de búsqueda no es una clave candidata y si el archivo no está ordenado según la clave de búsqueda (en el Apartado 12.5.3 se estudia la forma de evitar la creación de cajones haciendo que parezca que la clave de búsqueda sea única). El puntero P_n tiene un propósito especial que se estudiará más adelante.

En la Figura 12.7 se muestra un nodo hoja en el árbol B^+ del archivo *cuenta*, donde n vale tres y la clave de búsqueda es *nombre_sucursal*. Obsérvese que, como el archivo cuenta está ordenado por *nombre_sucursal*, los punteros en el nodo hoja apuntan directamente al archivo.

Ahora que se ha visto la estructura de un nodo hoja, se mostrará cómo los valores de la clave de búsqueda se asignan a nodos concretos. Cada hoja puede guardar hasta $n - 1$ valores. Está permitido

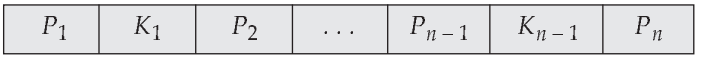


Figura 12.6 Nodo típico de un árbol B^+.

www.detodoprogramacion.com

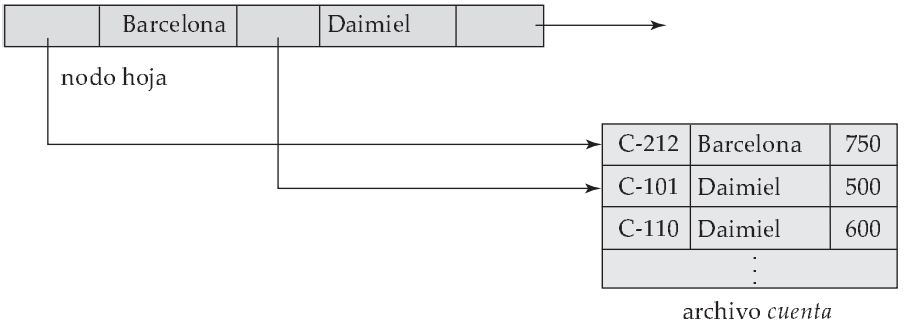


Figura 12.7 Nodo hoja para el índice del árbol B+ de *cuenta* ($n = 3$).

que los nodos hojas contengan al menos $\lceil (n-1)/2 \rceil$ valores. Los rangos de los valores en cada hoja no se solapan. Así, si L_i y L_j son nodos hoja e $i < j$, entonces cada valor de la clave de búsqueda en L_i es menor que cada valor de la clave en L_j. Si el índice de árbol B+ es un índice denso, cada valor de la clave de búsqueda debe aparecer en algún nodo hoja.

Ahora se puede explicar el uso del puntero P_n. Dado que existe un orden lineal en las hojas basado en los valores de la clave de búsqueda que contienen, se usa P_n para encadenar juntos los nodos hojas en el orden de la clave de búsqueda. Esta ordenación permite un procesamiento secuencial eficaz del archivo.

Los nodos internos del árbol B+ forman un índice multinivel (disperso) sobre los nodos hoja. La estructura de los nodos internos es la misma que la de los nodos hoja, excepto que todos los punteros son punteros a nodos del árbol. Un nodo interno podría guardar hasta *n* punteros y *debe* guardar al menos $\lceil n/2 \rceil$ punteros. El número de punteros de un nodo se llama *grado de salida* del nodo.

Considérese un nodo que contiene *m* punteros. Para $i = 2, 3, \ldots, m-1$, el puntero P_i apunta al subárbol que contiene los valores de la clave de búsqueda menores que K_i y mayor o igual que K_{i-1}. El puntero P_m apunta a la parte del subárbol que contiene los valores de la clave mayores o iguales que K_{m-1}, y el puntero P_1 apunta a la parte del subárbol que contiene los valores de la clave menores que K_1.

A diferencia de otros nodos internos, el nodo raíz puede tener menos de $\lceil n/2 \rceil$; sin embargo, debe tener al menos dos punteros, salvo que el árbol tenga un solo nodo. Siempre es posible construir un árbol B+, para cualquier *n*, que satisfaga los requisitos anteriores. En la Figura 12.8 se muestra un árbol B+ completo para el archivo *cuenta* ($n = 3$). Por simplicidad se omiten los punteros al propio archivo y los punteros nulos. Como ejemplo de un árbol B+ en el cual la raíz debe tener menos de $\lceil n/2 \rceil$ valores, en la Figura 12.9 se muestra un árbol B+ para el archivo *cuenta* con $n = 5$.

En todos los ejemplos mostrados de árboles B+, éstos están equilibrados. Es decir, la longitud de cada camino desde la raíz a cada nodo hoja es la misma. Esta propiedad es un requisito de los árboles B+. De hecho, la "B" de árbol B+ proviene del inglés "*balanced*" (equilibrado). Es esta propiedad de equilibrio de los árboles B+ la que asegura un buen rendimiento para las búsquedas, inserciones y borrados.

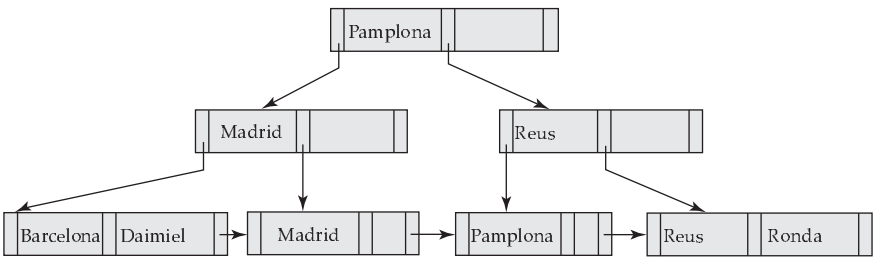


Figura 12.8 Árbol B+ para el archivo *cuenta* ($n = 3$).

www.detodoprogramacion.com

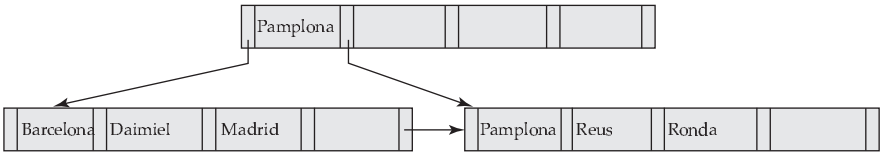


Figura 12.9 Árbol B$^+$ para el archivo *cuenta* ($n = 5$).

12.3.2 Consultas con árboles B$^+$

Considérese ahora cómo procesar consultas usando árboles B$^+$. Supóngase que se desea encontrar todos los registros cuyo valor de la clave de búsqueda sea V. La Figura 12.10 muestra el pseudocódigo para hacerlo. Primero se examina el nodo raíz para buscar el menor valor de la clave de búsqueda mayor que V. Supóngase que este valor de la clave de búsqueda es K_i. Después se sigue el puntero P_i hasta otro nodo. Si no se encuentra ese valor, entonces $k \geq K_{m-1}$, donde m es el número de punteros del nodo. Es este caso se sigue P_m hasta otro nodo. En el nodo alcanzado anteriormente se busca de nuevo el menor valor de la clave de búsqueda que es mayor que V para seguir el puntero correspondiente. Finalmente se alcanza un nodo hoja. En este nodo hoja, si se encuentra que el valor K_i es igual a V, entonces el puntero P_i nos ha conducido al registro o cajón deseado. Si no se encuentra el valor V en el nodo hoja, no existe ningún registro con el valor clave V.

De esta manera, para procesar una consulta, se tiene que recorrer un camino en el árbol desde la raíz hasta algún nodo hoja. Si hay K valores de la clave de búsqueda en el archivo, este camino no será más largo que $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.

En la práctica sólo se accede a unos cuantos nodos. Generalmente un nodo se construye para tener el mismo tamaño que un bloque de disco, el cual ocupa normalmente 4 KB . Con una clave de búsqueda del tamaño de 12 bytes y un tamaño del puntero a disco de 8 bytes, n está alrededor de 200. Incluso con una estimación más conservadora de 32 bytes para el tamaño de la clave de búsqueda, n está entorno a 100. Con $n = 100$, si se tienen un millón de valores de la clave de búsqueda en el archivo, una búsqueda necesita solamente $\lceil \log_{50}(1.000.000) \rceil = 4$ accesos a nodos. Por tanto, se necesitan leer a lo sumo cuatro bloques del disco para realizar la búsqueda. Normalmente, se accede mucho al nodo raíz del árbol y, por ello, suele guardarse en una memoria intermedia; por tanto, solamente hace falta leer, como máximo, tres bloques del disco.

Una diferencia importante entre las estructuras de árbol B$^+$ y los árboles en memoria, tales como los árboles binarios, está en el tamaño de los nodos y, por tanto, la altura del árbol. En los árboles binarios, todos los nodos son pequeños y tienen, a lo sumo, dos punteros. En los árboles B$^+$, cada nodo es grande —normalmente un bloque del disco—y puede tener gran número de punteros. Así, los árboles B$^+$ tienden a ser bajos y anchos, en lugar de los altos y estrechos árboles binarios. En un árbol equilibrado, el camino de una búsqueda puede tener una longitud de $\lceil \log_2(K) \rceil$, donde K es el número de valores

```
procedure buscar(valor V)
    C = nodo raíz
    while C no sea un nodo raíz begin
        K_i = mínimo valor de la clave de búsqueda, si lo hay, mayor que V
        if no hay tal valor then begin
            m = número de punteros en el nodo
            C = nodo al que apunta P_m
        end
            else C = el nodo al que apunta P_i
    end
    if hay un valor de la clave K_i en C tal que K_i = V
        then el puntero P_i conduce al registro o cajón deseado
        else no existe ningún registro con el valor de la clave k
```

Figura 12.10 Consulta con un árbol B$^+$.

www.detodoprogramacion.com

de la clave de búsqueda. Con $K = 1.000.000$, como en el ejemplo anterior, un árbol binario equilibrado necesita alrededor de 20 accesos a nodos. Si cada nodo estuviera en un bloque del disco distinto, serían necesarias 20 lecturas a bloques para procesar la búsqueda, en contraste con las cuatro lecturas del árbol B$^+$.

12.3.3 Actualizaciones en árboles B$^+$

El borrado y la inserción son operaciones más complicadas que las búsquedas, ya que podría ser necesario **dividir** un nodo que fuese demasiado grande como resultado de una inserción, o **fusionar** nodos si un nodo se volviera demasiado pequeño (menor que $\lceil n/2 \rceil$ punteros). Además, cuando se divide un nodo o se fusionan un par de ellos, se debe asegurar que el equilibrio del árbol se mantenga. Para presentar la idea que hay detrás del borrado y la inserción en un árbol B$^+$, se asumirá que los nodos nunca serán demasiado grandes ni demasiado pequeños. Bajo esta suposición, el borrado y la inserción se realizan como se indica a continuación.

- **Inserción**. Usando la misma técnica que para buscar, se busca un nodo hoja donde tendría que aparecer el valor de la clave de búsqueda. Si el valor de la clave de búsqueda ya aparece en el nodo hoja, se inserta un nuevo registro en el archivo y, si es necesario, un puntero al cajón. Si el valor de la clave de búsqueda no aparece, se inserta el valor en el nodo hoja de tal manera que las claves de búsqueda permanezcan ordenadas. Luego se inserta el nuevo registro en el archivo y, si es necesario, se crea un nuevo cajón con el puntero apropiado.
- **Borrado**. Usando la misma técnica que para buscar, se busca el registro a borrar y se elimina del archivo. Si no existe un cajón asociado con el valor de la clave de búsqueda o si el cajón se queda vacío como resultado del borrado, se borra el valor de la clave de búsqueda del nodo hoja.

A continuación se considera un ejemplo en el que se tiene que dividir un nodo. Por ejemplo, insertar un registro en el árbol B$^+$ de la Figura 12.8, cuyo valor de *nombre_sucursal* es "Cádiz". Usando el algoritmo de búsqueda, "Cádiz" debería aparecer en el nodo que incluye "Barcelona" y "Daimiel". No hay sitio para insertar el valor de la clave de búsqueda "Cádiz". Por tanto, se *divide* el nodo en otros dos nodos. En la Figura 12.11 se muestran los nodos hoja que resultan de insertar "Cádiz" y de dividir el nodo que incluía "Barcelona" y "Daimiel". En general, si existen n valores de la clave de búsqueda (los $n-1$ valores del nodo hoja más el valor que se inserta), se asignará $\lceil n/2 \rceil$ al nodo existente y el resto de valores en el nuevo nodo.

Después de dividir el nodo hoja hay que insertar el nuevo nodo hoja en el árbol B$^+$. En el nuevo nodo del ejemplo, "Daimiel" es el valor más pequeño de la clave de búsqueda. Después hay que insertar este valor de la clave de búsqueda en el padre del nodo hoja dividido. En el árbol B$^+$ de la Figura 12.12 se muestra el resultado de la inserción. El valor "Daimiel" de la clave de búsqueda se ha insertado en el padre. Ha sido posible llevar a cabo esta inserción porque había sitio para añadir un valor de la clave de búsqueda. En caso de no haber sitio se dividiría el padre. En el peor de los casos, todos los nodos en el camino hacia la raíz se tendrían que dividir. Si la propia raíz se tuviera que dividir, el árbol hubiera sido más profundo.

La técnica general para la inserción en un árbol B$^+$ es determinar el nodo hoja *h* en el cual realizar la inserción. Si es necesario dividir, se inserta el nuevo nodo dentro del padre del nodo *h*. Si esta inserción produce otra división, se procedería recursivamente o bien hasta que una inserción no produzca otra división o bien hasta crear una nueva raíz.

En la Figura 12.13 se bosqueja el algoritmo de inserción en pseudocódigo. El procedimiento inserta un par valor de la clave y puntero en el índice usando los procedimientos insertar_en_hoja e inser-

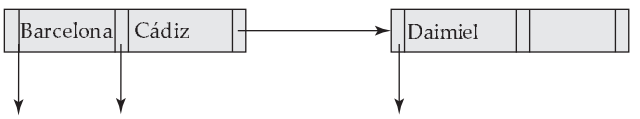


Figura 12.11 División del nodo hoja tras la inserción de "Cádiz".

www.detodoprogramacion.com

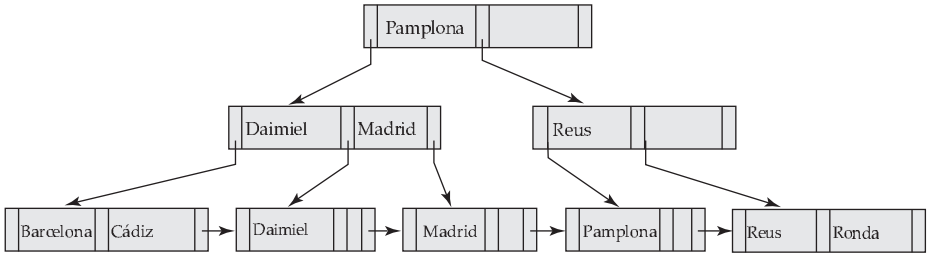


Figura 12.12 Inserción de "Cádiz" en el árbol B$^+$ de la Figura 12.8.

tar_en_padre. En el pseudocódigo L, N, P y T denotan punteros a nodos y L se usa para denotar un nodo hoja. $L.K_i$ y $L.P_i$ denotan el i-ésimo valor y el i-ésimo puntero en el nodo L, respectivamente. $T.K_i$ y $T.P_i$ se usan de forma análoga. El pseudocódigo también emplea la función $padre(N)$ para encontrar el padre del nodo N. Se puede obtener una lista de los nodos en el camino de la raíz a la hoja al buscar inicialmente el nodo hoja, y se puede usar después para encontrar eficazmente el padre de cualquier nodo del camino.

El procedimiento insertar_en_padre tiene como parámetros N, K' y N', donde el nodo N se ha dividido en N y N', siendo K' el valor mínimo en N'. El procedimiento modifica el padre de N para registrar la división. Los procedimientos insertar_en_hoja e insertar_en_padre usan el área temporal de memoria T para almacenar los contenidos del nodo que se está dividiendo. Los procedimientos se pueden modificar para que copien directamente los datos del nodo que se divide en el nodo que se crea, reduciendo el tiempo necesario para la copia. Sin embargo, el uso del espacio temporal T simplifica los procedimientos.

A continuación se consideran los borrados que provocan que el árbol se quede con muy pocos punteros. Primero se borra "Daimiel" del árbol B$^+$ de la Figura 12.12. Para ello se localiza la entrada "Daimiel" usando el algoritmo de búsqueda. Cuando se borra la entrada "Daimiel" de su nodo hoja, la hoja se queda vacía. Ya que en el ejemplo $n = 3$ y $0 < \lceil (n-1)/2 \rceil$, este nodo se debe borrar del árbol B$^+$. Para borrar un nodo hoja se tiene que borrar el puntero que le llega desde su padre. En el ejemplo, este borrado deja al nodo padre, el cual contenía tres punteros, con sólo dos punteros. Ya que $2 \geq \lceil n/2 \rceil$, el nodo es todavía lo suficientemente grande y la operación de borrado se completa. El árbol B$^+$ resultante se muestra en la Figura 12.14.

Cuando un borrado se hace sobre el padre de un nodo hoja, el propio nodo padre podría quedar demasiado pequeño. Esto es exactamente los que ocurre si se borra "Pamplona" del árbol B$^+$ de la Figura 12.14. El borrado de la entrada Pamplona provoca que un nodo hoja se quede vacío. Cuando se borra el puntero a este nodo en su padre, éste sólo se queda con un puntero. Así, $n = 3$, $\lceil n/2 \rceil = 2$ y queda tan sólo un puntero, que es demasiado poco. Sin embargo, ya que el nodo padre contiene información útil, no basta simplemente con borrarlo. En vez de eso, se busca el nodo hermano (el nodo interno que contiene al menos una clave de búsqueda, Madrid). Este nodo hermano dispone de sitio para colocar la información contenida en el nodo que quedó demasiado pequeño, así que se fusionan estos nodos, de tal manera que el nodo hermano ahora contiene las claves "Madrid" y "Reus". El otro nodo (el nodo que contenía solamente la clave de búsqueda "Reus") ahora contiene información redundante y se puede borrar desde su padre (el cual resulta ser la raíz del ejemplo). En la Figura 12.15 se muestra el resultado. Hay que observar que la raíz tiene solamente un puntero hijo como resultado del borrado, así que éste se borra y el hijo solitario se convierte en la nueva raíz. De esta manera la profundidad del árbol ha disminuido en una unidad.

No siempre es posible fusionar nodos. Como ejemplo se borrará "Pamplona" del árbol B$^+$ de la Figura 12.11. En este ejemplo, la entrada "Daimiel" es todavía parte del árbol. Una vez más, el nodo hoja que contiene "Pamplona" se queda vacío. El padre del nodo hoja se queda también demasiado pequeño (únicamente con un puntero). De cualquier modo, en este ejemplo, el nodo hermano contiene ya el máximo número de punteros: tres. Así, no puede acomodar a un puntero adicional. La solución en este caso es **redistribuir** los punteros de tal manera que cada hermano tenga dos punteros. El resultado se muestra en la Figura 12.16. Obsérvese que la redistribución de los valores necesita de un cambio en el valor de la clave de búsqueda en el padre de los dos hermanos.

www.detodoprogramacion.com

```
procedure insertar(valor K, puntero P)
    hallar el nodo hoja L que debe contener el valor de la clave K
    if (L tiene menos de n − 1 valores de la clave)
        then insertar_en_hoja (L, K, P)
        else begin /* L ya tiene n − 1 valores de la clave, dividirlo */
            Crear nodo L'
            Copiar L.P1 . . . L.Kn−1 a un bloque de memoria T que pueda
                almacenar n pares (puntero, valor de la clave)
            insertar_en_hoja (T, K, P)
            L'.Pn = L.Pn; L.Pn = L'
            Borrar desde L.P1 hasta L.Kn−1 de L
            Copiar desde T.P1 hasta T.K⌈n/2⌉ de T a L comenzando en L.P1
            Copiar desde T.P⌈n/2⌉+1 hasta T.Kn de T a L' comenzando en L'.P1
            K' el valor mínimo de la clave en L'
            insertar_en_padre(L, K', L')
        end

procedure insertar_en_hoja (nodo L, valor K, puntero P)
    if K es menor que L.K1
        then insertar P, K en L justo antes de L.P1
        else begin
            Ki el mayor valor de L que sea menor que K
            insertar P, K en L justo después de T.Ki
        end

procedure insertar_en_padre(nodo N, valor K', nodo N')
    if N es la raíz del árbol
        then begin
            crear un nuevo nodo R que contenga N, K', N'    /* N y N' son punteros */
            hacer de R la raíz del árbol
            return
        end
    P = padre (N)
    if (P tiene menos de n punteros)
        then insertar (K', N') en P justo después de N
        else begin /* Dividir P */
            Copiar P a un bloque de memoria T que pueda almacenar P y (K', N')
            Insertar (K', N') en T justo después de N
            Borrar todas las entradas de P; Crear el nodo P'
            Copiar T.P1 . . . T.P⌈n/2⌉ en P
            K'' = T.K⌈n/2⌉
            Copiar T.P⌈n/2⌉+1 . . . T.Pn+1 en P'
            insertar_en_padre(P, K'', P')
        end
```

Figura 12.13 Inserción de una entrada en un árbol B$^+$.

En general, para borrar un valor en un árbol B$^+$ se realiza una búsqueda según el valor y se borra. Si el nodo es demasiado pequeño, se borra desde su padre. Este borrado se realiza como una aplicación recursiva del algoritmo de borrado hasta que se alcanza la raíz, un nodo padre queda lleno de manera adecuada después de borrar, o hasta aplicar una redistribución.

En la Figura 12.17 se describe el pseudocódigo para el borrado en un árbol B$^+$. El procedimiento intercambiar_variables(N, N') simplemente cambia de lugar los valores (punteros) de las variables N y N'; este cambio no afecta al árbol en sí mismo. El pseudocódigo utiliza la condición "muy pocos va-

www.detodoprogramacion.com

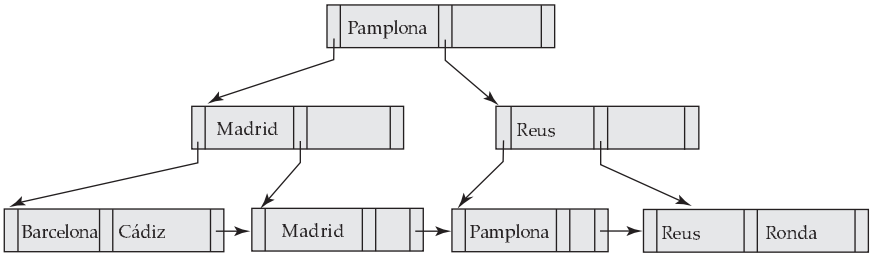


Figura 12.14 Borrado de "Daimiel" del árbol B$^+$ de la Figura 12.12.

lores/punteros". Para nodos internos, este criterio quiere decir: menos que $\lceil n/2 \rceil$ punteros; para nodos hoja, quiere decir: menos que $\lceil (n-1)/2 \rceil$ valores. El pseudocódigo realiza la redistribución tomando prestada una sola entrada desde un nodo adyacente. También se puede redistribuir mediante la distribución equitativa de entradas entre dos nodos. El pseudocódigo hace referencia al borrado de una entrada (K, P) desde un nodo. En el caso de los nodos hoja, el puntero a una entrada realmente precede al valor de la clave; así, el puntero P precede al valor de la clave K. Para nodos internos, P sigue al valor de la clave K.

Es interesante señalar que, como resultado de un borrado, puede que haya valores de la clave de nodos internos de un árbol B$^+$ que no estén en ninguna hoja del árbol.

Aunque las operaciones inserción y borrado en árboles B$^+$ son complicadas, requieren relativamente pocas operaciones E/S, lo que es un beneficio importante dado su coste. Se puede demostrar que el número de operaciones E/S necesarias para una inserción o borrado es, en el peor de los casos, proporcional a $\log_{\lceil n/2 \rceil}(K)$, donde n es el número máximo de punteros en un nodo y K es el número de valores de la clave de búsqueda. En otras palabras, el coste de las operaciones inserción y borrado es proporcional a la altura del árbol B$^+$ y es, por tanto, bajo. Debido a la velocidad de las operaciones en los árboles B$^+$, estas estructuras de índice se usan frecuentemente al implementar las bases de datos.

12.3.4 Organización de archivos con árboles B$^+$

Como se mencionó en el Apartado 12.3, el inconveniente de la organización de archivos secuenciales de índices es la degradación del rendimiento según crece el archivo: con el crecimiento, un porcentaje mayor de registros índice y registros reales se desaprovechan y se almacenan en bloques de desbordamiento. La degradación de las búsquedas en el índice se resuelve mediante el uso de índices de árbol B$^+$ en el archivo. También se soluciona el problema de la degradación al almacenar los registros reales

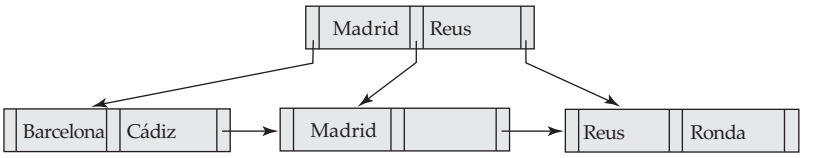


Figura 12.15 Borrado de "Pamplona" del árbol B$^+$ de la Figura 12.14.

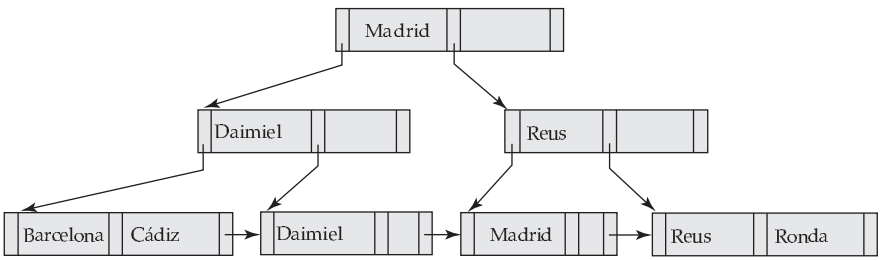


Figura 12.16 Borrado de "Pamplona" del árbol B$^+$ de la Figura 12.12.

www.detodoprogramacion.com

```
procedure borrar(valor K, puntero P)
    hallar el nodo hoja L que contiene (K, P)
    borrar_entrada(L, K, P)

procedure borrar_entrada(nodo N, valor K, puntero P)
    borrar (K, P) de N
    if (N es la raíz and a N sólo le queda un hijo)
    then hacer del hijo de N la nueva raíz del árbol y borrar N
    else if (N tiene muy pocos valores/punteros) then begin
        sea N' el hijo anterior o siguiente de padre(N)
        sea K' el valor entre los punteros N y N' en padre(N)
        if (las entradas en N y N' caben en un solo nodo)
            then begin /* Fusionar los nodos */
                if (N es predecesor de N') then intercambiar_variables(N, N')
                if (N no es una hoja)
                    then concatenar K' y todos los punteros y valores en N a N'
                    else  concatenar todos los pares (K_i, P_i) en N a N'; sea N'.P_n = N.P_n
                borrar_entrada(padre(N), K', N); borrar el nodo N
            end
        else begin /* Redistribución: tomar prestada una entrada de N' */
            if (N' es predecesor de N) then begin
                if (N es un nodo interno) then begin
                    sea m tal que N'.P_m es el último puntero en N'
                    suprimir (N'.K_{m-1}, N'.P_m) de N'
                    insertar (N'.P_m, K') como primer puntero y valor en N,
                        desplazando otros punteros y valores a la derecha
                    sustituir K' en padre(N) por N'.K_{m-1}
                end
                else begin
                    sea m tal que (N'.P_m, N'.K_m) es el último par puntero/valor
                        en N'
                    suprimir (N'.P_m, N'.K_m) de N'
                    insertar (N'.P_m, N'.K_m) como primer puntero y valor en N,
                        desplazando otros punteros y valores a la derecha
                    sustituir K' en padre(N) por N'.K_m
                end
            end
            else  ... simétrico al caso then ...
        end
    end
```

Figura 12.17 Borrado de una entrada de un árbol B$^+$.

utilizando el nivel de hoja del árbol B$^+$ para almacenar los registros reales en los bloques. En estas estructuras, la estructura del árbol B$^+$ se usa no sólo como un índice, sino también como un organizador de los registros dentro del archivo. En la **organización de archivo con árboles B$^+$**, los nodos hoja del árbol almacenan registros, en lugar de almacenar punteros a registros. En la Figura 12.18 se muestra un ejemplo de la organización de un archivo con un árbol B$^+$. Ya que los registros son normalmente más grandes que los punteros, el número máximo de registros que se pueden almacenar en un nodo hoja es menor que el número de punteros en un nodo interno. Sin embargo, todavía se requiere que los nodos hoja estén llenos al menos hasta la mitad.

La inserción y el borrado de registros en las organizaciones de archivos con árboles B$^+$ se tratan del mismo modo que la inserción y el borrado de entradas en los índices de árboles B$^+$. Cuando se inserta un registro con un valor de clave v, se localiza el bloque que debería contener ese registro mediante la

www.detodoprogramacion.com

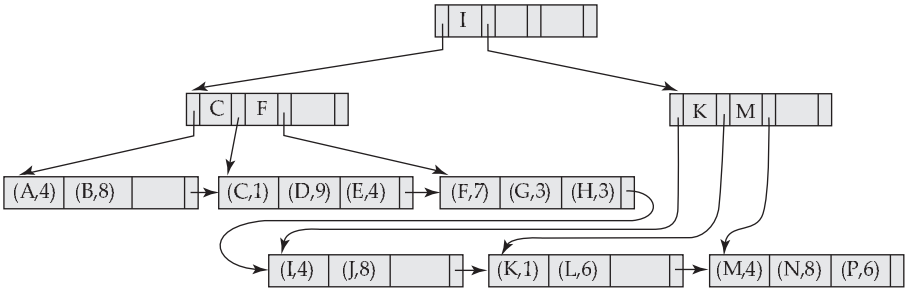


Figura 12.18 Organizacion de archivos con árboles B^+.

búsqueda en el árbol B^+ de la mayor clave que sea $\leq v$. Si el bloque localizado tiene bastante espacio libre para el registro, se almacena el registro en el bloque. De no ser así, como en una inserción en un árbol B^+, se divide el bloque en dos y se redistribuyen sus registros (en el orden de la clave del árbol B^+) creando espacio para el nuevo registro. Esta división se propaga hacia arriba en el árbol B+ de la manera usual. Cuando se borra un registro, primero se elimina del bloque que lo contiene. Si como resultado un bloque B llega a ocupar menos que la mitad, los registros en B se redistribuyen con los registros en un bloque B' adyacente. Si se asume que los registros son de tamaño fijo, cada bloque contendrá por lo menos la mitad de los registros que pueda contener como máximo. Los nodos internos del árbol B^+ se actualizan por tanto de la manera habitual.

Cuando un árbol B^+ se utiliza para la organización de un archivo, la utilización del espacio es particularmente importante, ya que el espacio ocupado por los registros es mucho mayor que el espacio ocupado por las claves y punteros. Se puede mejorar la utilización del espacio en un árbol B^+ implicando a más nodos hermanos en la redistribución durante las divisiones y fusiones. La técnica es aplicable a los nodos hoja y nodos internos y funciona como sigue.

Durante la inserción, si un nodo está lleno se intenta redistribuir algunas de sus entradas en uno de los nodos adyacentes para hacer sitio a una nueva entrada. Si este intento falla porque los nodos adyacentes están llenos, se divide el nodo y las entradas entre uno de los nodos adyacentes y los dos nodos que se obtienen al dividir el nodo original. Puesto que los tres nodos juntos contienen un registro más que puede encajar en dos nodos, cada nodo estará lleno aproximadamente hasta sus dos terceras partes. Para ser más precisos, cada nodo tendrá por lo menos $\lfloor 2n/3 \rfloor$ entradas, donde n es el número máximo de entradas que puede tener un nodo ($\lfloor x \rfloor$ denota el mayor entero menor o igual que x; es decir, se elimina la parte fraccionaria si la hay).

Durante el borrado de un registro, si la ocupación de un nodo está por debajo de $\lfloor 2n/3 \rfloor$, se intentará tomar prestada una entrada desde uno de sus nodos hermanos. Si ambos nodos hermanos tienen $\lfloor 2n/3 \rfloor$ registros, en lugar de tomar prestada una entrada, se redistribuyen las entradas en el nodo y en los dos nodos hermanos uniformemente entre dos de los nodos y se borra el tercer nodo. Se puede usar este enfoque porque el número total de entradas es $3\lfloor 2n/3 \rfloor - 1$, lo cual es menor que $2n$. Utilizando tres nodos adyacentes para la redistribución se puede garantizar que cada nodo tenga $\lfloor 3n/4 \rfloor$ entradas. En general, si hay m nodos ($m - 1$ hermanos) implicados en la redistribución se puede garantizar que cada nodo contenga, al menos, $\lfloor (m-1)n/m \rfloor$ entradas. Sin embargo, el coste de la actualización se vuelve mayor según haya más nodos hermanos involucrados en la redistribución.

Obsérvese que en un índice de árbol B^++ los nodos hoja adyacentes en el árbol se puede ubicar en diferentes lugares del disco. Cuando se crea un índice sobre un conjunto de registros es posible asignar bloques contiguos en disco para nodos hoja que también lo sean en el árbol. Así, una exploración secuencial de los nodos hoja correspondería a una exploración secuencial en el disco. Como se dan inserciones y borrados en el árbol, se pierde la secuencialidad, y las esperas a disco en el acceso secuencial se incrementan. Sería necesario una reorganización del índice para mantener la secuencialidad.

Las organizaciones de archivo del tipo árbol B^+ se pueden usar para almacenar grandes objetos, como los tipos clob y blob de SQL, que pueden ser más grandes que un bloque de disco y de tamaño de varios gigabytes. Estos objetos se pueden almacenar dividiéndolos en secuencias de registros más pequeños que se organizan en un archivo B^+. Los registros se pueden numerar secuencialmente o por

www.detodoprogramacion.com

el desplazamiento en bytes del registro dentro del objeto, y este número se puede usar como clave de búsqueda.

12.3.5 Índices sobre cadenas de caracteres

La creación de índices de árboles B^+ sobre atributos de tipo cadena de caracteres plantea dos problemas. El primero es que las cadenas pueden ser de longitud variable. El segundo es que pueden ser largas, lo que produce un grado de salida bajo y a una altura del árbol incrementada de manera acorde.

Con las claves de búsqueda de longitud variable varios nodos pueden tener diferentes grados de salida incluso estando llenos. Un nodo se debe dividir si está lleno, es decir, si no hay espacio para añadir una nueva entrada, independientemente de cuántas entradas contenga. Análogamente, los nodos se pueden fusionar o las entradas se pueden redistribuir dependiendo de la fracción de espacio que se use en los nodos, en lugar de basarse en el número máximo de entradas que el nodo pueda contener.

Se puede incrementar el grado de salida usando la técnica denominada **compresión del prefijo**. Con esta compresión no se almacena la clave de búsqueda completa en los nodos internos. Sólo se almacena el prefijo de la clave de búsqueda que sea suficiente para distinguir las claves de los subárboles bajo ella. Por ejemplo, si se indexa el nombre, el valor de la clave en un nodo interno podría ser un prefijo del nombre; sería suficiente almacenar "Silb" en un nodo interno en lugar del nombre completo "Silberschatz" si los valores más próximos en los dos subárboles bajo esta clave sean, por ejemplo, "Silas" y "Silver" respectivamente.

12.4 Archivos de índices de árbol B

Los *índices de árbol B* son similares a los índices de árbol B^+. La diferencia principal entre los dos enfoques es que un árbol B elimina el almacenamiento redundante de los valores de la clave búsqueda. En el árbol B^+ de la Figura 12.12, las claves de búsqueda "Daimiel", "Madrid", "Reus" y "Pamplona" aparecen dos veces. Cada valor de clave de búsqueda aparece en algún nodo hoja; algunos se repiten en nodos internos.

Los árboles B permiten que los valores de la clave de búsqueda aparezcan solamente una vez. En la Figura 12.19 se muestra un árbol B que representa las mismas claves de búsqueda que el árbol B^+ de la Figura 12.12. Ya que las claves de búsqueda no están repetidas en el árbol B, sería posible almacenar el índice usando menos nodos del árbol que con el correspondiente índice de árbol B^+. Sin embargo, puesto que las claves de búsqueda que aparecen en los nodos internos no aparecen en ninguna otra parte del árbol B, es necesario incluir un campo adicional para un puntero por cada clave de búsqueda de un nodo interno. Estos punteros adicionales apuntan a registros del archivo o a los cajones de la clave de búsqueda asociada.

En la Figura 12.20a aparece un nodo hoja generalizado de un árbol B; en la Figura 12.20b aparece un nodo interno. Los nodos hoja son como en los árboles B^+. En los nodos internos los punteros P_i son los punteros del árbol que se utilizan también para los árboles B^+, mientras que los punteros B_i en los nodos internos son punteros a cajones o registros del archivo. En la figura del árbol B generalizado hay

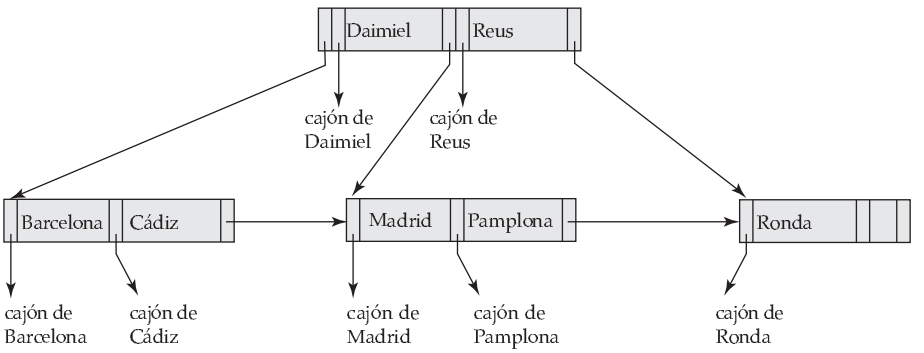


Figura 12.19 Árbol B equivalente al árbol B^+ de la Figura 12.12.

www.detodoprogramacion.com

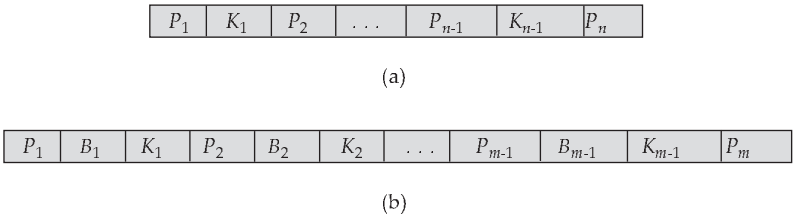


Figura 12.20 Nodos típicos de un árbol B. (a) Nodo hoja. (b) Nodo interno.

$n - 1$ claves en el nodo hoja, mientras que hay $m - 1$ claves en el nodo interno. Esta discrepancia ocurre porque los nodos internos deben incluir los punteros B_i, y de esta manera se reduce el número de claves de búsqueda que pueden contener estos nodos. Claramente, $m < n$, pero la relación exacta entre m y n depende del tamaño relativo de las claves de búsqueda y de los punteros.

El número de nodos a los que se accede en una búsqueda en un árbol B depende de dónde esté situada la clave de búsqueda. Una búsqueda en un árbol B^+ requiere atravesar un camino desde la raíz del árbol hasta algún nodo hoja. En cambio, algunas veces es posible encontrar en un árbol B el valor deseado antes de alcanzar el nodo hoja. Sin embargo, hay que realizar aproximadamente n accesos según cuántas claves haya almacenadas tanto en el nivel de hoja de un árbol B como en los niveles internos de hoja y, dado que n es normalmente grande, la probabilidad de encontrar ciertos valores pronto es relativamente pequeña. Por otra parte, el hecho de que aparezcan menos claves de búsqueda en los nodos internos del árbol B, comparado con los árboles B^+, implica que un árbol B tiene un grado de salida menor y, por tanto, puede que tenga una profundidad mayor que la correspondiente al árbol B^+. Así, la búsqueda en un árbol B es más rápida para algunas claves de búsqueda pero más lenta para otras, aunque en general, el tiempo de la búsqueda es todavía proporcional al logaritmo del número de claves de búsqueda.

El borrado en un árbol B es más complicado. En un árbol B^+ la entrada borrada siempre aparece en una hoja. En un árbol B, la entrada borrada podría aparecer en un nodo interno. El valor apropiado a colocar en su lugar se debe elegir del subárbol del nodo que contiene la entrada borrada. Concretamente, si se borra la clave de búsqueda K_i, la clave de búsqueda más pequeña que aparezca en el subárbol del puntero P_{i+1} se debe trasladar al campo ocupado anteriormente por K_i. Será necesario tomar otras medidas si ahora el nodo hoja tuviera pocas entradas. Por el contrario, la inserción en un árbol B es sólo un poco más complicada que la inserción en un árbol B^+.

Las ventajas de espacio que tienen los árboles B son escasas para índices grandes y normalmente no son de mayor importancia los inconvenientes advertidos. De esta manera, muchos implementadores de sistemas de bases de datos aprovechan la sencillez estructural de un árbol B^+. Los detalles de los algoritmos de inserción y borrado para árboles B se estudian en los ejercicios.

12.5 Accesos bajo varias claves

Hasta ahora se ha asumido implícitamente que se utiliza solamente un índice (o tabla asociativa) para procesar una consulta en una relación. Sin embargo, para ciertos tipos de consultas es ventajoso el uso de varios índices si existen o usar un índice construido sobre una clave de búsqueda de varios atributos.

12.5.1 Uso de varios índices de clave única

Considérese que el archivo *cuenta* tiene dos índices: uno para el *nombre_sucursal* y otro para *saldo*. Dada la consulta: "Encontrar el número de todas las cuentas de la sucursal de Pamplona con saldo igual a 1.000 €", se escribe

select *número_préstamo*
from *cuenta*
where *nombre_sucursal* = "Pamplona" **and** *saldo* = 1000

Hay tres estrategias posibles para procesar esta consulta:

www.detodoprogramacion.com

1. Usar el índice de *nombre_sucursal* para encontrar todos los registros correspondientes a la sucursal de Pamplona. Examinar luego esos registros para ver si *saldo* = 1.000.
2. Usar el índice de *saldo* para encontrar todos los registros pertenecientes a cuentas con un saldo de 1.000 €. Examinar luego esos registros para ver si *nombre_sucursal* = "Pamplona".
3. Usar el índice de *nombre_sucursal* para encontrar *punteros* a registros correspondientes a la sucursal de Pamplona. Usar también el índice de *saldo* para encontrar los punteros a todos los registros correspondientes a cuentas con un saldo de 1.000 €. Realizar la intersección de esos dos conjuntos de punteros. Los punteros que están en la intersección apuntan a los registros correspondientes a la vez a Pamplona y a las cuentas con un saldo de 1.000 €.

La tercera estrategia es la única de las tres que aprovecha la ventaja de tener varios índices. Sin embargo, incluso esta estrategia podría ser una mala elección si hubiese:

- Muchos registros correspondientes a la sucursal Pamplona.
- Muchos registros correspondientes a cuentas con un saldo de 1.000 €.
- Sólo unos cuantos registros pertenecientes *tanto a* la sucursal de Pamplona como a las cuentas con un saldo de 1.000 €.

Si ocurrieran estas condiciones se tendrían que examinar un gran número de punteros para producir un resultado pequeño. La estructura de índices denominada "índice de mapas de bits" acelera significativamente la operación de inserción usada en la tercera estrategia. Los índices de mapas de bits se describen en el Apartado 12.9.

12.5.2 Índices sobre varias claves

Una estrategia más eficiente para este caso es crear y utilizar un índice con una clave de búsqueda (*nombre_sucursal*, *saldo*)—esto es, la clave de búsqueda consistente en el nombre de la sucursal concatenado con el saldo de la cuenta. Esa clave de búsqueda, que contiene más de un atributo, se denomina a veces **clave de búsqueda compuesta**. La estructura del índice es la misma que para cualquier otro índice, con la única diferencia de que la clave de búsqueda no es un simple atributo sino una lista de atributos. La clave de búsqueda se puede representar como una tupla de valores, de la forma $(a_1, \ldots, a_n)$, donde los atributos indexados son $A_1, \ldots, A_n$. El orden de los valores de la clave de búsqueda es el *orden lexicográfico*. Por ejemplo, para el caso de dos atributos en la clave de búsqueda, $(a_1, a_2) < (b_1, b_2)$ si $a_1 < b_1$ o bien $a_1 = b_1$ y $a_2 < b_2$. El orden lexicográfico es básicamente el mismo que el alfabético.

Se puede usar un índice ordenado (árbol B^+) para responder eficientemente consultas de la forma:

select *número_préstamo*
from *cuenta*
where *nombre_sucursal* = 'Pamplona' **and** *saldo* = 1000

Las consultas como la siguiente, que especifica una condición de igualdad sobre el primer atributo de la clave de búsqueda (*nombre_sucursal*) y un rango sobre el segundo (*saldo*) se pueden también tratar eficientemente ya que corresponden a una consulta por rangos sobre el atributo de búsqueda.

select *número_préstamo*
from *cuenta*
where *nombre_sucursal* = 'Pamplona' **and** *saldo* < 1000

Incluso se puede utilizar un índice ordenado sobre la clave de búsqueda (*nombre_sucursal*, *saldo*) para responder de manera eficiente a la siguiente consulta sobre un solo atributo:

select *número_préstamo*
from *cuenta*
where *nombre_sucursal* = 'Pamplona'

www.detodoprogramacion.com

La condición de igualdad *nombre_sucursal* = "Pamplona" es equivalente a una consulta por rangos sobre el rango con extremo inferior (Pamplona, $-\infty$) y superior (Pamplona, $+\infty$). Las consultas por rangos sobre sólo el atributo *nombre_sucursal* se pueden tratar de forma similar.

Sin embargo, el uso de una estructura de índice ordenado con múltiples atributos presenta algunas deficiencias. Como ilustración considérese la consulta:

select *número_préstamo*
from *cuenta*
where *nombre_sucursal* $<$ "Pamplona" **and** *saldo* $= 1000$

Se puede responder a esta consulta usando un índice ordenado con la clave de búsqueda (*nombre_sucursal*, *saldo*) de la manera siguiente: para cada valor de *nombre_sucursal* que está alfabéticamente por delante de "Pamplona", hay que localizar los registros con un *saldo* de 1.000. Sin embargo, debido a la ordenación de los registros en el archivo, es probable que cada registro esté en un bloque de disco diferente, lo que genera muchas operaciones de E/S.

La diferencia entre esta consulta y la anterior es que la condición sobre *nombre_sucursal* es de comparación y no de igualdad. La condición no se corresponde con una consulta por rangos sobre la clave de búsqueda.

Para acelerar el procesamiento en general de consultas con claves de búsqueda compuestas (que pueden implicar una o más operaciones de comparación) se pueden emplear varias estructuras especiales. En el Apartado 12.9.3 se considerará la estructura de *índices de mapas de bits*. Existe otra estructura, denominada *árbol R*, que también se puede usar para este propósito. Los árboles R son una extensión de los árboles B^+ para el tratamiento de índices en varias dimensiones. Dado que se emplean fundamentalmente con datos de tipo geográfico, su estructura se describe en el Capítulo 23.

12.5.3 Claves de búsqueda duplicadas

La creación de cajones de punteros para el tratamiento de las claves de búsqueda duplicadas (es decir, claves sobre archivos que puedan tener más de un registro con el mismo valor de la clave) plantea varias complicaciones cuando se implementan árboles B^+. Si los cajones se guardan en un nodo hoja se necesita código extra para tratar cajones de tamaño variable y para tratar el caso en que crecen por encima del tamaño del nodo hoja. Si los cajones se almacenan en páginas separadas puede ser necesario una operación E/S adicional para acceder a los registros.

Una solución sencilla a este problema, empleada por la mayoría de los sistemas de bases de datos, es hacer que las claves de búsqueda sean únicas añadiendo un atributo único adicional a la clave. El valor de este atributo podría ser un identificador de registro (si el sistema de bases de datos les da soporte) o simplemente un número único para todos los registros con el mismo valor de la clave de búsqueda. Por ejemplo, si se tuviese un índice sobre el atributo *nombre_cliente* de la tabla *impositor*, las entradas correspondientes a un cliente en particular tendrían diferentes valores para el atributo adicional. Así se garantizaría que la clave de búsqueda extendida fuese única.

Las búsquedas con el atributo original de la clave de búsqueda se convierten en búsquedas por rangos en la clave de búsqueda extendida, como se vio en el Apartado 12.5.2; al realizar la búsqueda, se ignora el valor del atributo adicional.

12.5.4 Índices de cobertura

Los **índices de cobertura** almacenan los valores de algunos atributos (distintos de los atributos de la clave de búsqueda) junto con los punteros a los registros. El almacenamiento de valores de atributo adicionales es útil en los índices secundarios, ya que permite responder algunas consultas utilizando únicamente el índice, sin ni siquiera buscar en los registros de datos.

Por ejemplo, supóngase un índice sin agrupación sobre el atributo *número_cuenta* de la relación *cuenta*. Si se almacena el valor del atributo *saldo* junto con el puntero a registro, se pueden responder consultas que soliciten el saldo (sin el otro atributo, *nombre_sucursal*) sin acceder al registro de *cuenta*.

Se obtendría el mismo efecto creando un índice sobre la clave de búsqueda (*número_cuenta*, *saldo*), pero los índices de cobertura reducen el tamaño de las claves de búsqueda, lo que permite un mayor grado de salida en los nodos internos y puede reducir la altura del índice.

www.detodoprogramacion.com

12.5.5 Índices secundarios y reubicación de registros

Algunas organizaciones de archivos, como los árboles B^+, pueden cambiar la ubicación de los registros aunque éstos no se hayan modificado. A modo de ejemplo, considérese que cuando en una organización de archivo de árbol B^+ se divide una página hoja, varios registros se trasladan a una nueva página. En esos casos hay que actualizar todos los índices secundarios que almacenan punteros a los registros reubicados, aunque sus contenidos no hayan cambiado. Cada página hoja puede contener gran número de registros, y cada uno de ellos puede estar en diferentes ubicaciones de cada índice secundario. Así, la división de una página hoja puede exigir decenas o incluso centenas de operaciones de E/S para actualizar todos los índices secundarios afectados, lo que puede convertirla en una operación muy costosa.

A continuación se explica una técnica para resolver este problema. En los índices secundarios, en lugar de punteros a los registros indexados, se almacenan los valores de los atributos de la clave de búsqueda del índice primario. Por ejemplo, supóngase que se tiene un índice primario sobre el atributo *número_cuenta* de la relación *cuenta*; un índice secundario sobre *nombre_sucursal* almacenaría con cada nombre de sucursal una lista de valores de *número_cuenta* de los registros correspondientes, en lugar de almacenar punteros a los registros.

Por tanto, la reubicación de los registros debido a la división de las páginas hoja no exige ninguna actualización de los índices secundarios. Sin embargo, la ubicación de un registro mediante el índice secundario exige ahora dos pasos: primero se utiliza el índice secundario para buscar los valores de la clave de búsqueda del índice primario y luego se utiliza el índice primario para buscar los registros correspondientes.

Este enfoque reduce en gran medida el coste de actualización de los índices debido a la reorganización de los archivos, aunque incrementa el coste de acceso a los datos mediante un índice secundario.

12.6 Asociación estática

Un inconveniente de la organización de archivos secuenciales es que hay que acceder a una estructura de índices para localizar los datos o utilizar una búsqueda binaria y, como resultado, más operaciones de E/S. La organización de archivos basada en la técnica de **asociación** (hashing) permite evitar el acceso a la estructura de índice. La asociación también proporciona una forma de construir índices. En los apartados siguientes se estudian las organizaciones de archivos y los índices basados en asociación.

En esta descripción de la asociación, se usará el término **cajón** (bucket) para indicar una unidad de almacenamiento que puede guardar uno o más registros. Un cajón es normalmente un bloque de disco, aunque también se podría elegir de tamaño mayor o menor que un bloque de disco.

Formalmente, sea K el conjunto de todos los valores de clave de búsqueda y sea B el conjunto de todas las direcciones de cajón. Una **función de asociación** h es una función de K a B. Sea h una función asociación.

Para insertar un registro con clave de búsqueda K_i, se calcula $h(K_i)$, lo que proporciona la dirección del cajón para ese registro. De momento se supone que hay espacio en el cajón para almacenar el registro. Luego, el registro se almacena en ese cajón.

Para realizar una búsqueda con el valor K_i de la clave de búsqueda, basta con calcular $h(K_i)$ y buscar luego el cajón con esa dirección. Supóngase que dos claves de búsqueda, K_5 y K_7, tienen el mismo valor de asociación; es decir, $h(K_5) = h(K_7)$. Si se realiza una búsqueda en K_5, el cajón $h(K_5)$ contendrá registros con valores de la clave de búsqueda K_5 y registros con valores de la clave de búsqueda K_7. Por tanto, hay que comprobar el valor de clave de búsqueda de todos los registros del cajón para asegurarse de que el registro es el deseado.

El borrado es igual de sencillo. Si el valor de la clave de búsqueda del registro que se debe borrar es K_i, se calcula $h(K_i)$, después se busca el cajón correspondiente a ese registro y se borra el registro del cajón.

La asociación se puede usar para dos propósitos diferentes. En la **organización de archivo asociativo** se obtiene directamente la dirección del bloque de disco que contiene el registro deseado calculando una función del valor de la clave de búsqueda del registro. En la **organización de índice asociativo** se organizan las claves de búsqueda con sus punteros asociados en una estructura de archivo asociativo.

www.detodoprogramacion.com

12.6.1 Funciones de asociación

La peor función de asociación posible asigna todos los valores de la clave de búsqueda al mismo cajón. Una función así no es deseable, ya que hay que guardar todos los registros en el mismo cajón. Durante una búsqueda hay que examinar todos esos registros hasta encontrar el deseado. Una función de asociación ideal distribuye las claves almacenadas uniformemente entre todos los cajones, de modo que cada uno de ellos tenga el mismo número de registros.

Puesto que durante la etapa de diseño no se conocen con precisión los valores de la clave de búsqueda que se almacenarán en el archivo, se pretende elegir una función de asociación que asigne los valores de la clave de búsqueda a los cajones de manera que la distribución tenga las propiedades siguientes:

- Distribución *uniforme*. Esto es, la función de asociación asigna a cada cajón el mismo número de valores de la clave de búsqueda dentro del conjunto de *todos* los valores posibles de la misma.
- Distribución *aleatoria*. Esto es, en el caso promedio, cada cajón tendrá asignado casi el mismo número de valores, independientemente de la distribución real de los valores de la clave de búsqueda. Para ser más exactos, el valor de asociación no estará correlacionado con ningún ordenamiento de los valores de la clave de búsqueda visible exteriormente como, por ejemplo, el orden alfabético o el orden determinado por la longitud de las claves de búsqueda; parecerá que la función de asociación es aleatoria.

Como ilustración de estos principios, se escogerá una función de asociación para el archivo *cuenta* que utilice la clave búsqueda *nombre_sucursal*. La función de asociación que se escoja debe tener las propiedades deseadas no sólo para el ejemplo del archivo *cuenta* que se ha estado utilizando, sino también para el archivo *cuenta* de tamaño real de un gran banco con muchas sucursales.

Supóngase que se decide tener 26 cajones y se define una función de asociación que asigna a los nombres que empiezan con la letra *i*-ésima del alfabeto el cajón *i*-ésimo. Esta función de asociación tiene la virtud de la simplicidad, pero no logra proporcionar una distribución uniforme, ya que se espera que haya más nombres de sucursales que comiencen con letras como B y R que con Q y con X, por ejemplo.

Supóngase ahora que se desea una función de asociación en la clave de búsqueda *saldo*. Supóngase que el saldo mínimo es 1, el saldo máximo es 100.000 y se utiliza una función de asociación que divide el valor en 10 rangos, 1—10.000, 10.001—20.000, y así sucesivamente. La distribución de los valores de la clave de búsqueda es uniforme (ya que cada cajón tiene el mismo número de valores del *saldo* diferentes), pero no es aleatoria. Los registros con saldos entre 1 y 10.000 son más frecuentes que los registros con saldos entre 90.001 y 100.000. En consecuencia, la distribución de los registros no es uniforme—algunos cajones reciben más registros que otros. Si la función presenta una distribución aleatoria, aunque se dieran esas correlaciones entre las claves de búsqueda, la aleatoriedad de la distribución haría, muy probablemente, que todos los cajones tuvieran más o menos el mismo número de registros, siempre y cuando cada clave de búsqueda apareciera sólo en una pequeña parte de los registros (si una sola clave de búsqueda aparece en gran parte de registros, es probable que el cajón que la contiene contenga más registros que otros cajones, independientemente de la función de asociación empleada).

Las funciones de asociación habituales realizan cálculos sobre la representación binaria interna de la máquina de los caracteres de la clave de búsqueda. Una función de asociación sencilla de este tipo calcula en primer lugar la suma de las representaciones binarias de los caracteres de la clave y, luego, devuelve el resto de la división entre la suma y el número de cajones. En la Figura 12.21 se muestra la aplicación de este esquema, con 10 cajones, al archivo *cuenta*, con la suposición de que la letra *i*-ésima del alfabeto está representada por el número entero *i*.

La siguiente función de asociación se puede usar para asociar una cadena en una implementación Java.

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \cdots + s[n-1]$$

La función se puede implementar eficientemente estableciendo el valor asociativo inicialmente como 0 e iterando desde el primero hasta el último carácter de la cadena, multiplicando en cada paso el valor asociativo por 31 y añadiendo el siguiente carácter (tratado como un entero). El resto de la división entre el resultado de esta función y el número de cajones se puede usar para la indexación.

www.detodoprogramacion.com

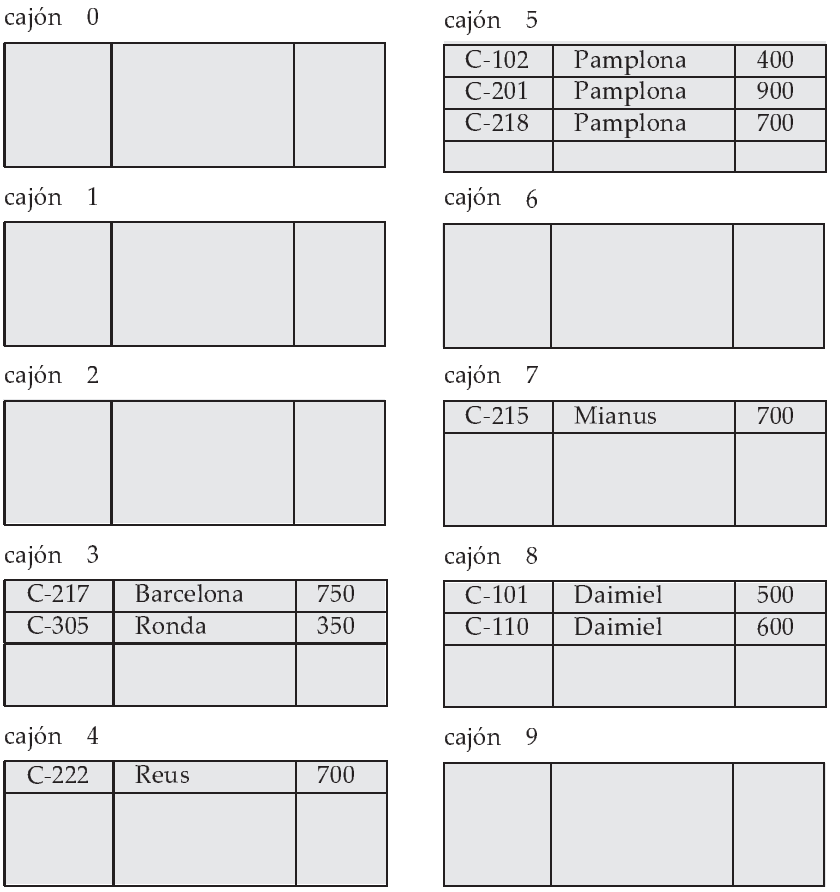


Figura 12.21 Organización asociativa del archivo *cuenta* utilizando *nombre_sucursal* como clave.

Las funciones de asociación requieren un diseño cuidadoso. Una mala función de asociación puede hacer que la búsqueda tarde un tiempo proporcional al número de claves de búsqueda del archivo. Una función bien diseñada da un tiempo de búsqueda para casos promedio que es una constante (pequeña), independiente del número de claves de búsqueda del archivo.

12.6.2 Gestión de desbordamientos de cajones

Hasta ahora se ha asumido que cuando se inserta un registro, el cajón al que se asigna tiene espacio para almacenarlo. Si el cajón no tiene suficiente espacio, sucede lo que se denomina **desbordamiento de cajones**. Los desbordamientos de cajones se pueden producir por varias razones:

- **Cajones insuficientes**. Se debe elegir un número de cajones, que se denota con n_B, tal que $n_B > n_r/f_r$, donde n_r denota el número total de registros que se van a almacenar y f_r denota el número de registros que caben en cada cajón. Esta designación, por supuesto, supone que se conoce el número total de registros en el momento de definir la función de asociación.
- **Atasco**. Algunos cajones tienen asignados más registros que otros, por lo que algún cajón se puede desbordar, aunque otros cajones tengan todavía espacio libre. Esta situación se denomina **atasco** de cajones. El atasco puede producirse por dos motivos:
  1. Puede que varios registros tengan la misma clave de búsqueda.
  2. Puede que la función de asociación elegida genere una distribución no uniforme de las claves de búsqueda.

www.detodoprogramacion.com

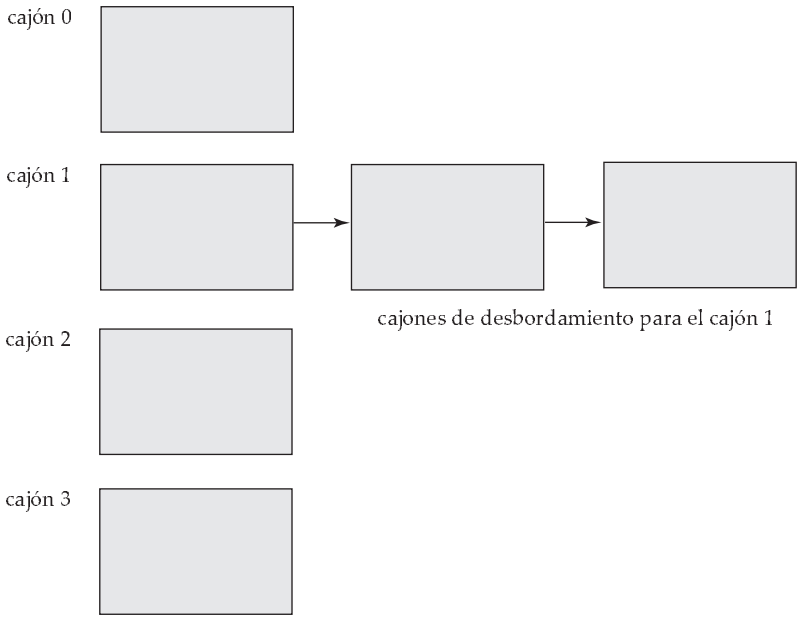


Figura 12.22 Cadena de desbordamiento en una estructura asociativa.

Para que la probabilidad de desbordamiento de cajones se reduzca, se escoge un número de cajones igual a $(n_r/f_r) * (1 + d)$, donde d es un factor de corrección, normalmente cercano a 0.2. Se pierde algo de espacio: alrededor del veinte por ciento del espacio en los cajones queda vacío. Pero la ventaja es que la probabilidad de desbordamiento se reduce.

Pese a la asignación de unos pocos cajones más de los necesarios, todavía se puede producir el desbordamiento de los cajones. El desbordamiento de los cajones se trata mediante **cajones de desbordamiento**. Si hay que insertar un registro en un cajón c y c está ya lleno, el sistema proporcionará un cajón de desbordamiento para c y el registro se insertará en ese cajón de desbordamiento. Si el cajón de desbordamiento también se encuentra lleno, el sistema proporcionará otro cajón de desbordamiento, y así sucesivamente. Todos los cajones de desbordamiento de un cajón determinado se encadenan en una lista enlazada, como se muestra en la Figura 12.22. El tratamiento del desbordamiento mediante una lista enlazada, se denomina **cadena de desbordamiento**.

Para tratar la cadena de desbordamiento es necesario modificar ligeramente el algoritmo de búsqueda. Como se dijo antes, el sistema utiliza la función de asociación sobre la clave de búsqueda para identificar un cajón c. Luego debe examinar todos los registros del cajón c para ver si coinciden con la clave de búsqueda, como antes. Además, si el cajón c tiene cajones de desbordamiento, también hay que examinar los registros de todos los cajones de desbordamiento.

La forma de la estructura asociativa que se acaba de describir se denomina a veces **asociación cerrada**. En una aproximación alternativa, conocida como **asociación abierta**, se fija el conjunto de cajones y no hay cadenas de desbordamiento. En su lugar, si un cajón está lleno, el sistema inserta los registros en algún otro cajón del conjunto inicial C de cajones. Un criterio es utilizar el siguiente cajón (en orden cíclico) que tenga espacio; esta política se llama *ensayo lineal*. Se utilizan también otros criterios, como el cálculo de funciones de asociación adicionales. La asociación abierta se emplea en la construcción de tablas de símbolos para compiladores y ensambladores, pero la asociación cerrada es preferible para los sistemas de bases de datos. El motivo es que el borrado con asociación abierta es problemático. Normalmente, los compiladores y los ensambladores sólo realizan operaciones de búsqueda e inserción en sus tablas de símbolos. Sin embargo, en los sistemas de bases de datos es importante poder tratar tanto el borrado como la inserción. Por tanto, la asociatividad abierta sólo tiene una importancia menor en la implementación de bases de datos.

Un inconveniente importante de la forma de asociación que se ha descrito, es que hay que elegir la función de asociación cuando se implementa el sistema y no se puede cambiar fácilmente después si el archivo que se está indexando aumenta o disminuye de tamaño. Como la función h asigna valores

www.detodoprogramacion.com

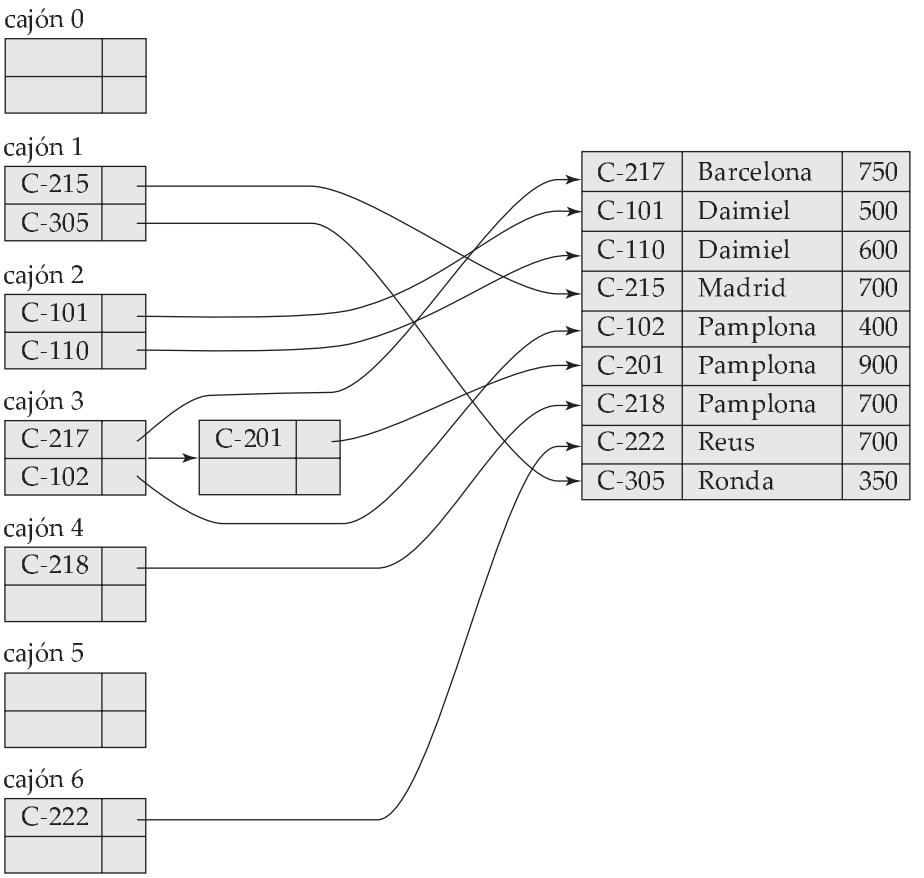


Figura 12.23 Índice asociativo de la clave de búsqueda *número_cuenta* del archivo *cuenta*.

de la clave búsqueda a un conjunto fijo *C* de direcciones de cajón, se desperdicia espacio si se hace *C* de gran tamaño para manejar el futuro crecimiento del archivo. Si *C* es demasiado pequeño, cada cajón contendrá registros de muchos valores de la clave de búsqueda y se pueden provocar desbordamientos de cajones. A medida que el archivo aumenta de tamaño, el rendimiento se degrada. Más adelante, en el Apartado 12.7, se estudiará la manera de cambiar dinámicamente el número de cajones y la función de asociación.

12.6.3 Índices asociativos

La asociatividad se puede utilizar no solamente para la organización de archivos, sino también para la creación de estructuras de índice. Cada **índice asociativo** (hash index) organiza las claves de búsqueda, con sus punteros asociados, en una estructura de archivo asociativo. Los índices asociativos se construyen como se indica a continuación. Primero se aplica una función de asociación sobre la clave de búsqueda para identificar un cajón, luego se almacenan la clave y los punteros asociados en el cajón (o en los cajones de desbordamiento). En la Figura 12.23 se muestra un índice asociativo secundario del archivo *cuenta* para la clave de búsqueda *número_cuenta*. La función de asociación utilizada calcula la suma de las cifras del número de cuenta módulo siete. El índice asociativo posee siete cajones, cada uno de tamaño dos (un índice realista tendría, por supuesto, cajones de mayor tamaño). Uno de los cajones tiene tres claves asignadas, por lo que tiene un cajón de desbordamiento. En este ejemplo, *número_cuenta* es clave primaria de *cuenta*, por lo que cada clave de búsqueda sólo tiene asociado un puntero. En general, se pueden asociar varios punteros con cada clave.

Se usa el término *índice asociativo* para denotar las estructuras de archivo asociativo, así como los índices secundarios asociativos. Estrictamente hablando, los índices asociativos son sólo estructuras de índices secundarios. Los índices asociativos no se necesitan nunca como estructuras de índices con agru-

www.detodoprogramacion.com

pación ya que, si un archivo está organizado mediante asociatividad, no hay necesidad de ninguna estructura de índice asociativo adicional. Sin embargo, como la organización en archivos asociativos proporciona el mismo acceso directo a los registros que el indexado, se finge que la organización de un archivo mediante asociación también tiene en él un índice con agrupación asociativo.

12.7 Asociación dinámica

Como se ha visto, la necesidad de fijar el conjunto *C* de direcciones de los cajones presenta un problema serio con la técnica de asociación estática vista en el apartado anterior. La mayor parte de las bases de datos aumenta de tamaño con el tiempo. Si se va a utilizar la asociación estática para esas bases de datos, hay tres opciones:

1. Elegir una función de asociación de acuerdo con el tamaño actual del archivo. Esta opción provocará una degradación del rendimiento a medida que la base de datos aumente de tamaño.
2. Elegir una función de asociación de acuerdo con el tamaño previsto del archivo en un momento determinado del futuro. Aunque se evite la degradación del rendimiento, puede que inicialmente se pierda una cantidad de espacio significativa.
3. Reorganizar periódicamente la estructura asociativa en respuesta al crecimiento del archivo. Esta reorganización supone elegir una nueva función de asociación, volver a calcular la función de asociación de cada registro del archivo y generar nuevas asignaciones de cajones. Esta reorganización es una operación masiva que consume mucho tiempo. Además, hay que prohibir el acceso al archivo durante la reorganización.

Algunas técnicas de **asociación dinámica** permiten modificar dinámicamente la función de asociación para adaptarse al aumento o disminución de tamaño de la base de datos. En esta apartado se describe una forma de asociación dinámica, denominada **asociación extensible**. Las notas bibliográficas proporcionan referencias a otras formas de asociación dinámica.

12.7.1 Estructura de datos

La asociación extensible hace frente a los cambios del tamaño de la base de datos dividiendo y fusionando los cajones a medida que la base de datos aumenta o disminuye. En consecuencia, se conserva la eficiencia espacial. Además, puesto que la reorganización sólo se lleva a cabo en un cajón simultáneamente, la degradación del rendimiento resultante es aceptablemente baja.

Con la asociación extensible se elige una función de asociación h con las propiedades deseadas de uniformidad y aleatoriedad. Sin embargo, esa función de asociación genera valores dentro de un rango relativamente amplio—por ejemplo, los enteros binarios de b bits. Un valor normal de b es 32.

No se crea un cajón para cada valor de la función de asociación. De hecho, 2^{32} es más de cuatro mil millones, y no son razonables tantos cajones salvo para las mayores bases de datos. En vez de eso, se crean cajones bajo demanda, a medida que se insertan registros en el archivo. Inicialmente no se utilizan todos los b bits del valor de la función de asociación. En cualquier momento se utilizan i bits, donde $0 \leq i \leq b$. Esos i bits son utilizados como reserva en una tabla adicional de direcciones de los cajones. El valor de i aumenta o disminuye con el tamaño de la base de datos.

En la Figura 12.24 se muestra una estructura general de asociación extensible. La i que aparece en la figura encima de la tabla de direcciones de los cajones indica que se requieren i bits del valor de la función de asociación $h(K)$ para determinar el cajón apropiado para K. Obviamente, ese número cambia a medida que el archivo aumenta de tamaño. Aunque se requieren i bits para encontrar la entrada correcta en la tabla de direcciones de los cajones, varias entradas consecutivas de la tabla pueden apuntar al mismo cajón. Todas esas entradas tendrán un prefijo de asociación común, pero la longitud de ese prefijo puede ser menor que i. Por lo tanto, se asocia con cada cajón un número entero que proporciona la longitud del prefijo de asociación común. En la Figura 12.24, el entero asociado con el cajón j aparece como i_j. El número de entradas de la tabla de direcciones de cajones que apuntan al cajón j es

$$2^{(i - i_j)}$$

www.detodoprogramacion.com

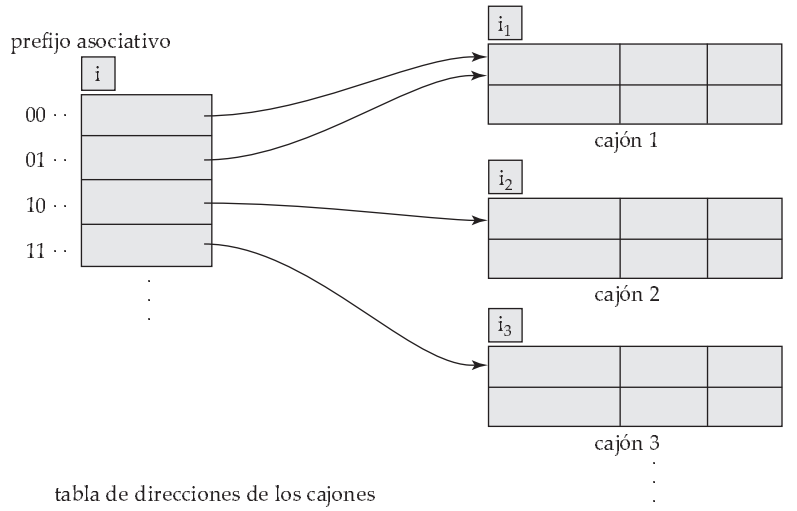


Figura 12.24 Estructura asociativa general extensible.

12.7.2 Consultas y actualizaciones

A continuación se estudiará la forma de realizar la búsqueda, la inserción y el borrado en una estructura asociativa extensible.

Para localizar el cajón que contiene el valor de la clave de búsqueda K_l, el sistema toma los primeros *i* bits más significativos de $h(K_l)$, busca la entrada de la tabla correspondiente a esa cadena de bits, y sigue el puntero del cajón de esa entrada de la tabla.

Para insertar un registro con un valor de la clave de búsqueda K_l se sigue el mismo procedimiento de búsqueda que antes, y se llega a un cajón—por ejemplo, *j*. Si hay sitio en ese cajón, se inserta en él el registro. Si, por el contrario, el cajón está lleno, hay que dividir el cajón y redistribuir los registros actuales, junto con el nuevo. Para dividir el cajón, primero hay que determinar, a partir del valor de la función de asociación, si hace falta incrementar el número de bits que hay que utilizar.

- Si $i = i_j$, entonces solamente apunta al cajón *j* una entrada de la tabla de direcciones de los cajones. Por tanto, es necesario incrementar el tamaño de la tabla de direcciones de los cajones para incluir los punteros a los dos cajones que resultan de la división del cajón *j*. Esto se consigue considerando otro bit más del valor de asociación. Se incrementa en uno el valor de *i*, lo que duplica el tamaño de la tabla de direcciones de los cajones. Cada entrada se sustituye por dos entradas, ambas con el mismo puntero que la entrada original. Ahora, dos entradas de la tabla de direcciones de cajones apuntan al cajón *j*. Se asigna un nuevo cajón (el cajón *z*) y se hace que la segunda entrada apunte al nuevo cajón. Se definen i_j e i_z como *i*. A continuación se vuelve a calcular la función de asociación para todos los registros del cajón *j* y, en función de los primeros *i* bits (recuérdese que se ha añadido uno a *i*), se mantienen en el cajón *j* o se colocan en el cajón recién creado.

  Ahora se vuelve a intentar la inserción del nuevo registro. Normalmente el intento tiene éxito. Sin embargo, si todos los registros del cajón *j*, así como el registro nuevo, tienen el mismo prefijo de asociación, será necesario volver a dividir el cajón, ya que tanto los registros del cajón *j* como el registro nuevo tienen asignado el mismo cajón. Si la función de asociación se eligió cuidadosamente, es poco probable que una simple inserción provoque que un cajón se divida más de una vez, a menos que haya un gran número de registros con la misma clave de búsqueda. Si todos los registros del cajón *j* tienen el mismo valor de la clave de búsqueda, la división no servirá de nada. En esos casos se utilizan cajones de desbordamiento para almacenar los registros, como en la asociación estática.

- Si $i > i_j$, entonces más de una entrada en la tabla de direcciones de los cajones apunta al cajón *j*. Por tanto, se puede dividir el cajón *j* sin aumentar el tamaño de la tabla de direcciones

www.detodoprogramacion.com

C-217	Barcelona	750
C-101	Daimiel	500
C-110	Daimiel	600
C-215	Madrid	700
C-102	Pamplona	400
C-201	Pamplona	900
C-218	Pamplona	700
C-222	Reus	700
C-305	Ronda	350

Figura 12.25 Archivo *cuenta* de ejemplo.

nombre_sucursal	*h(nombre_sucursal)*
Barcelona	0010 1101 1111 1011 0010 1100 0011 0000
Daimiel	1010 0011 1010 0000 1100 0110 1001 1111
Madrid	1100 0111 1110 1101 1011 1111 0011 1010
Pamplona	1111 0001 0010 0100 1001 0011 0110 1101
Reus	0011 0101 1010 0110 1100 1001 1110 1011
Ronda	1101 1000 0011 1111 1001 1100 0000 0001

Figura 12.26 Función de asociación para *nombre_sucursal*.

de los cajones. Obsérvese que todas las entradas que apuntan al cajón *j* corresponden a prefijos de asociación que tienen el mismo valor en los i_j bits situados más a la izquierda. Se asigna un nuevo cajón (el cajón *z*) y se definen i_j e i_z con el valor que resulta de añadir uno al valor original de i_j. A continuación hay que ajustar las entradas de la tabla de direcciones de los cajones que anteriormente apuntaban al cajón *j* (obsérvese que, con el nuevo valor de i_j, no todas las entradas corresponden a prefijos de asociación que tienen el mismo valor en los i_j bits situados más a la izquierda). La primera mitad de las entradas se deja como estaba (apuntando al cajón *j*) y se hace que el resto de las entradas apunte al cajón recién creado (el cajón *z*). Luego, como en el caso anterior, se vuelve a calcular la función de asociación para todos los registros del cajón *j* y se asignan o bien al cajón *j* o bien al cajón *z* recién creado.

Luego se vuelve a intentar la inserción. En el improbable caso de que vuelva a fallar, se aplica uno de los dos casos, $i = i_j$ o $i > i_j$, según corresponda.

Obsérvese que en ambos casos solamente se necesita volver a calcular la función de asociación de los registros del cajón *j*.

Para borrar un registro con valor de la clave de búsqueda K_l se sigue el mismo procedimiento de búsqueda, que finaliza en un cajón—por ejemplo, *j*. Se borran tanto el registro del archivo como la clave de búsqueda del cajón. También se elimina el cajón si se queda vacío. Obsérvese que, en este momento, se pueden fusionar varios cajones y se puede reducir el tamaño de la tabla de direcciones de los cajones a la mitad. El procedimiento para decidir los cajones que se deben fusionar y el momento de hacerlo se deja al lector como ejercicio. Las condiciones bajo las que la tabla de direcciones de los cajones se puede reducir de tamaño también se dejan como ejercicio. A diferencia de la fusión de los cajones, el cambio de tamaño de la tabla de direcciones de los cajones es una operación bastante costosa si la tabla es de gran tamaño. Por tanto, sólo merece la pena reducir el tamaño de la tabla de direcciones de los cajones si el número de cajones se reduce considerablemente.

El ejemplo del archivo *cuenta* de la Figura 12.25 ilustra la operación de inserción. Los valores de asociación de 32 bits de *nombre_sucursal* se muestran en la Figura 12.26. Supóngase que, inicialmente, el archivo está vacío, como se muestra en la Figura 12.27. Los registros se insertan de uno en uno. Para mostrar todas las características de la asociación extensible en una estructura pequeña, se hará la suposición no realista de que cada cajón sólo puede contener dos registros.

www.detodoprogramacion.com

Se va a insertar el registro (C-217, Barcelona, 750). La tabla de direcciones de los cajones contiene un puntero al único cajón existente y el sistema inserta el registro. A continuación se inserta el registro (C-101, Daimiel, 500). Este registro también se inserta en el único cajón de la estructura.

Cuando se intenta insertar el siguiente registro (C-110, Daimiel, 600), el cajón está lleno. Ya que $i = i_0$, es necesario incrementar el número de bits del valor de asociación que se utilizan. Ahora se utiliza un bit, lo que permite $2^1 = 2$ cajones. Este incremento en el número de bits necesarios necesita doblar el tamaño de la tabla de direcciones de cajones a dos entradas. El sistema divide el cajón, coloca en el nuevo aquellos registros cuya clave de búsqueda tiene un valor de asociación que comienza por 1 y deja el resto de los registros en el cajón original. En la Figura 12.28 se muestra el estado de la estructura después de la división.

A continuación se inserta (C-215, Madrid, 700). Como el primer bit de *h*(Madrid) es 1, hay que insertar ese registro en el cajón al que apunta la entrada "1" de la tabla de direcciones de los cajones. Una vez más, el cajón se encuentra lleno e $i = i_1$. Se incrementa a dos el número de bits del valor de asociación que se usan. Este incremento en el número de bits necesarios hace que se doble el tamaño de la tabla de direcciones de los cajones a cuatro entradas, como se muestra en la Figura 12.29. Como el cajón de la Figura 12.28 con el prefijo 0 del valor de asociación no se dividió, las dos entradas, 00 y 01, de la tabla de direcciones de los cajones apuntan a ese cajón.

Para cada registro del cajón de la Figura 12.28 con prefijo de asociación 1 (el cajón que se va a dividir) se examinan los dos primeros bits del valor de asociación para determinar el cajón de la nueva estructura que le corresponde.

A continuación se inserta el registro (C-102, Pamplona, 400), que se aloja en el mismo cajón que Madrid. La siguiente inserción, la de (C-201, Pamplona, 900), provoca un desbordamiento en un cajón, lo que provoca el incremento del número de bits y la duplicación del tamaño de la tabla de direcciones de los cajones. La inserción del tercer registro de Pamplona, (C-218, Pamplona, 700), produce otro desbordamiento. Sin embargo, este desbordamiento no se puede resolver incrementando el número de bits, ya que hay tres registros con el mismo valor de asociación exactamente. Por tanto se utiliza un cajón de desbordamiento, como se muestra en la Figura 12.30.

Se continúa de esta manera hasta que se hayan insertado todos los registros del archivo *cuenta* de la Figura 12.25. La estructura resultante se muestra en la Figura 12.31.

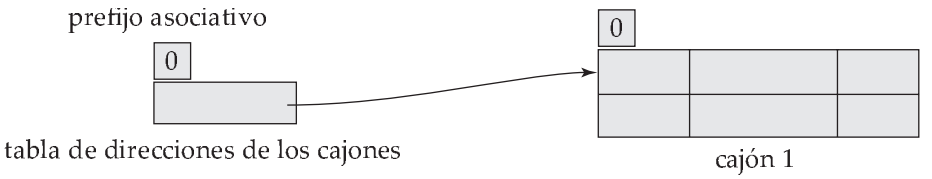


Figura 12.27 Estructura asociativa extensible inicial.

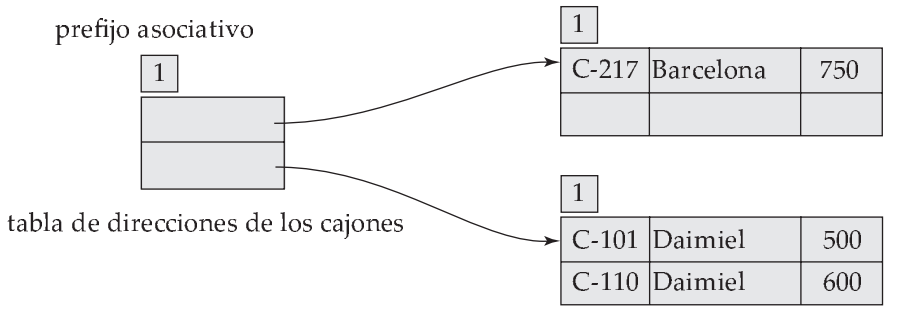


Figura 12.28 Estructura asociativa después de tres inserciones.

www.detodoprogramacion.com

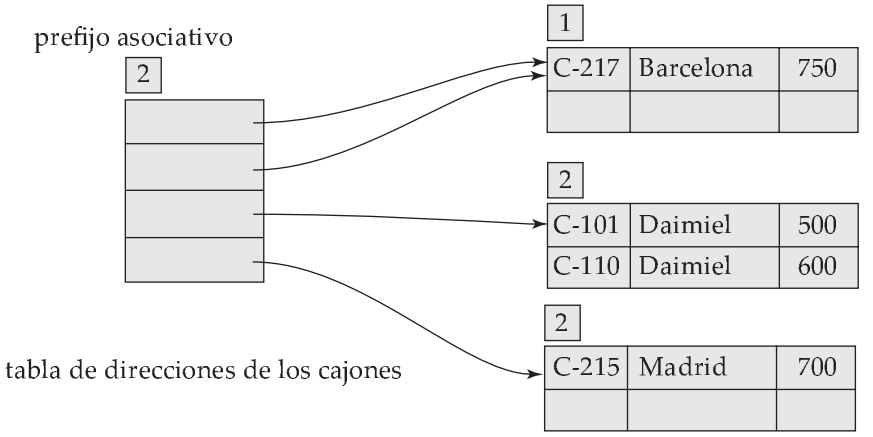


Figura 12.29 Estructura asociativa después de cuatro inserciones.

12.7.3 Comparación entre la asociación estática y la dinámica

A continuación se examinan las ventajas e inconvenientes de la asociación extensible respecto de la asociación dinámica. La ventaja principal de la asociación extensible es que el rendimiento no se degrada a medida que el archivo aumenta de tamaño. Además, el espacio adicional necesario es mínimo. Aunque la tabla de direcciones de los cajones provoca un gasto adicional, sólo contiene un puntero por cada valor de asociación para la longitud actual del prefijo. Por tanto, el tamaño de la tabla es pequeño. El principal ahorro de espacio de la asociación extensible respecto de otras formas de asociación es que no es necesario reservar cajones para un futuro aumento de tamaño; en vez de eso, los cajones se pueden asignar de manera dinámica.

Un inconveniente de la asociación extensible es que la búsqueda implica un nivel adicional de referencia, ya que se debe acceder a la tabla de direcciones de los cajones antes que a los propios cajones. Esta referencia adicional sólo tiene una mínima repercusión en el rendimiento. Aunque las estructuras asociativas que se estudiaron en el Apartado 12.6 no tienen este nivel adicional de referencia, pierden esa mínima ventaja de rendimiento cuando se llenan.

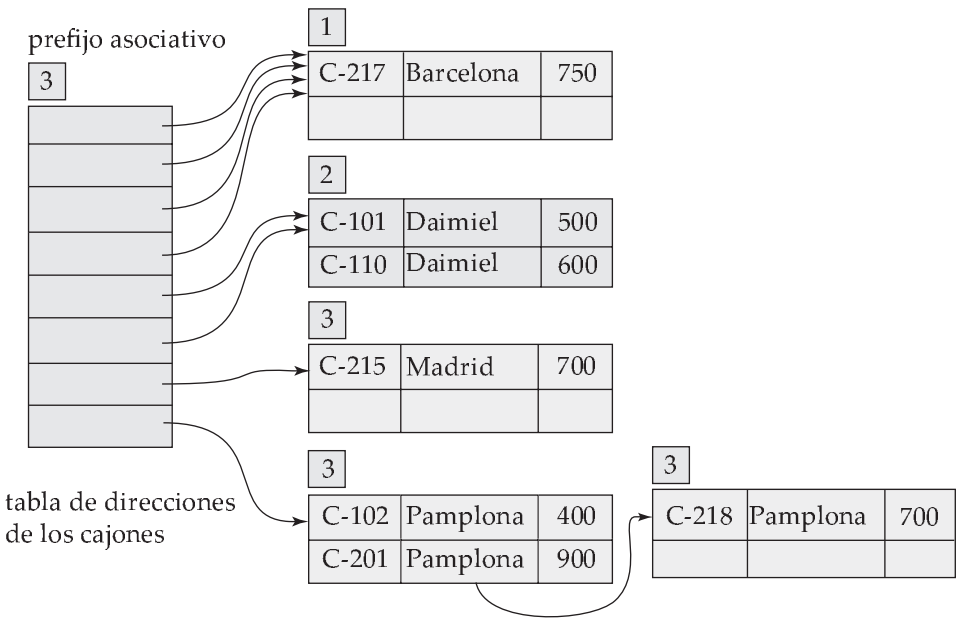


Figura 12.30 Estructura asociativa después de siete inserciones.

www.detodoprogramacion.com

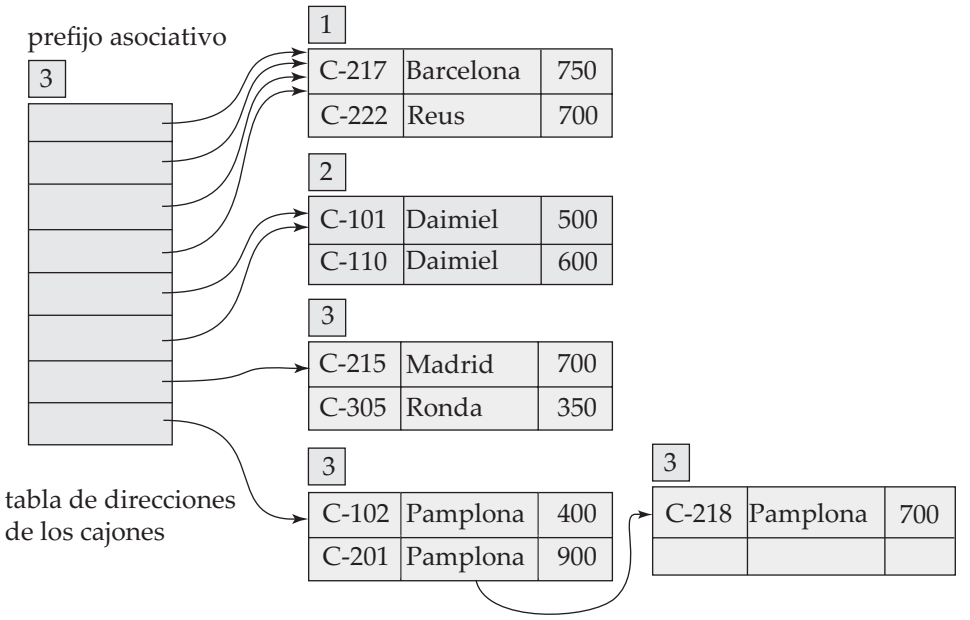


Figura 12.31 Estructura asociativa extensible para el archivo *cuenta*.

Por tanto, la asociación extensible se muestra como una técnica muy atractiva, siempre que se acepte la complejidad añadida de su implementación. En las notas bibliográficas se proporcionan descripciones más detalladas de la implementación de la asociación extensible.

Las notas bibliográficas también ofrecen referencias a otra forma de asociación dinámica denominada **asociación lineal**, que evita el nivel adicional de referencia asociado con la asociación extensible al posible coste de más cajones de desbordamiento.

12.8 Comparación de la indexación ordenada y la asociación

Se han examinado varios esquemas de indexación ordenada y varios esquemas de asociación. Se pueden organizar los archivos de registros como archivos ordenados, mediante una organización de índice secuencial u organizaciones de árbol B^+. Alternativamente, se pueden organizar los archivos mediante la asociación. Finalmente, los archivos se pueden organizar como montículos, en los que los registros no están ordenados de ninguna manera en especial.

Cada esquema tiene sus ventajas, dependiendo de la situación. Un fabricante de un sistema de bases de datos puede proporcionar muchos esquemas y dejar al diseñador de la base de datos la decisión final sobre los esquemas que se utilizarán. Sin embargo, este enfoque exige que el fabricante escriba más código, lo que aumenta tanto el coste del sistema como el espacio que ocupa. La mayor parte de los sistemas de bases de datos soportan árboles B^+ y pueden dar soporte también a alguna forma de organización de archivos asociativos o de índices asociativos.

Para hacer una buena elección de organización de archivos y de técnica de indexado, el fabricante o el diseñador de la base de datos debe tener en consideración los siguientes aspectos:

- ¿Es aceptable el coste de una reorganización periódica del índice o de la estructura asociativa?
- ¿Cuál es la frecuencia relativa de las inserciones y de los borrados?
- ¿Es deseable optimizar el tiempo medio de acceso a expensas de incrementar el peor tiempo de acceso posible?
- ¿Qué tipos de consultas se supone que van a formular los usuarios?

De estos puntos ya se han examinado los tres primeros, comenzando con la revisión de las ventajas relativas de las distintas técnicas de indexado, y nuevamente en la discusión de las técnicas de asociación.

www.detodoprogramacion.com

El cuarto punto, el tipo de consultas esperado, resulta fundamental para la elección entre la indexación ordenada y la asociación.

Si la mayoría de las consultas son de la forma:

select $A_1, A_2, \ldots, A_n$
from *r*
where $A_i = c$

entonces, para procesarla, el sistema realiza una búsqueda en un índice ordenado o en una estructura asociativa del atributo A_i con el valor *c*. Para este tipo de consultas es preferible un esquema asociativo. Las búsquedas en índices ordenados requieren un tiempo proporcional al logaritmo del número de valores de A_i en *r*. Sin embargo, en las estructuras asociativas, el tiempo medio de búsqueda es una constante independiente del tamaño de la base de datos. La única ventaja de los índices respecto de las estructuras asociativas en este tipo de consultas es que el tiempo de búsqueda en el peor de los casos es proporcional al logaritmo del número de valores de A_i en *r*. Por el contrario, si se utiliza una estructura asociativa, el tiempo de búsqueda en el peor de los casos es proporcional al número de valores de A_i en *r*. Sin embargo, es poco probable en el caso de la asociación que se dé el peor caso de búsqueda posible (máximo tiempo de búsqueda) y, en este caso, es preferible emplear una estructura asociativa.

Las técnicas de índices ordenados son preferibles a las estructuras asociativas en los casos en los que la consulta especifica un rango de valores. Estas consultas tienen el siguiente aspecto:

select $A_1, A_2, ..., A_n$
from *r*
where $A_i \leq c_2$ **and** $A_i \geq c_1$

En otras palabras, la consulta anterior busca todos los registros con A_i valores comprendidos entre c_1 y c_2.

Considérese la manera de procesar esta consulta empleando un índice ordenado. En primer lugar se realiza una búsqueda del valor c_1. Una vez que se ha encontrado el cajón que contiene el valor c_1, se sigue el orden de la cadena de punteros del índice para leer el siguiente cajón y se continúa de esta manera hasta que se llega a c_2.

Si, en vez de un índice ordenado, se tiene una estructura asociativa, se puede llevar a cabo una búsqueda de c_1 y localizar el cajón correspondiente—pero, en general, no resulta fácil determinar el cajón que hay que examinar a continuación. La dificultad surge porque una buena función de asociación asigna valores a los cajones de manera aleatoria. Por tanto, no hay un concepto sencillo de "siguiente cajón según el orden establecido". La razón por la que no se puede encadenar un cajón detrás de otro según un cierto orden de A_i es que cada cajón tiene asignado muchos valores de la clave de búsqueda. Como los valores están diseminados aleatoriamente según la función de asociación, es probable que los valores del rango especificado estén esparcidos por muchos cajones o, tal vez, por todos. Por esa razón, hay que leer todos los cajones para encontrar las claves de búsqueda necesarias.

Normalmente el diseñador elige la indexación ordenada, a menos que se sepa de antemano que las consultas de rango van a ser poco frecuentes, en cuyo caso se escoge la asociación. Las organizaciones asociativas resultan especialmente útiles para los archivos temporales creados durante el procesamiento de las consultas, siempre que se realicen búsquedas basadas en valores de la clave pero no se vayan a realizar consultas de rango.

12.9 Índices de mapas de bits

Los índices de mapas de bits son un tipo de índices especializado diseñado para la consulta sencilla sobre varias claves, aunque cada índice de mapas de bits se construya para una única clave.

Para que se utilicen los índices de mapas de bits, los registros de la relación deben estar numerados secuencialmente comenzando, por ejemplo, por 0. Dado un número n debe ser fácil recuperar el registro con número n. Esto resulta especialmente fácil de conseguir si los registros son de tamaño fijo y están asignados a bloques consecutivos de un archivo. El número de registro se puede traducir fácilmente en un número de bloque y en un número que identifica el registro dentro de ese bloque.

Considérese una relación r con un atributo A que sólo puede tomar como valor un número pequeño (por ejemplo, entre 2 y 20). Por ejemplo, la relación *info_cliente* puede tener el atributo *sexo*, que sólo

www.detodoprogramacion.com

puede tomar los valores m (masculino) o f (femenino). Otro ejemplo puede ser el atributo *nivel_ingresos*, en el que los ingresos se han dividido en cinco niveles: $L1$: \$0 − 9999, $L2$: \$10.000 − 19.999, $L3$: 20.000 − 39.999, $L4$: 40.000 − 74.999 y $L5$: $75,000 - \infty$. Aquí, los datos originales pueden tomar muchos valores, pero un analista de datos los ha dividido en un número pequeño de rangos para simplificar su análisis.

12.9.1 Estructura de los índices de mapas de bits

Un **mapa de bits** es simplemente un array de bits. En su forma más sencilla, un **índice de mapas de bits** sobre el atributo A de la relación r consiste en un mapa de bits para cada valor que pueda tomar A. Cada mapa de bits tiene tantos bits como el número de registros de la relación. El i-ésimo bit del mapa de bits para el valor v_j se define como 1 si el registro con número i tiene el valor v_j para el atributo A. El resto de los bits del mapa de bits se define como 0.

En este ejemplo hay un mapa de bits para el valor m y otro para f. El i-ésimo bit del mapa de bits para m se define como 1 si el valor *sexo* del registro con número i es m. El resto de bits del mapa de bits de m se definen como 0. Análogamente, el mapa de bits de f tiene el valor 1 para los bits correspondientes a los registros con el valor f para el atributo *sexo*; el resto de bits tienen el valor 0. La Figura 12.32 muestra un ejemplo de índices de mapa de bits para la relación *info_cliente*.

Ahora se considerará cuándo resultan útiles los mapas de bits. La manera más sencilla de recuperar todos los registros con el valor m (o f) sería simplemente leer todos los registros de la relación y seleccionar los de valor m (o f, respectivamente). El índice de mapas de bits no ayuda realmente a acelerar esa selección.

De hecho, los índices de mapas de bits resultan útiles para las selecciones sobre todo cuando hay selecciones bajo varias claves. Supóngase que se crea un índice de mapas de bits sobre el atributo *nivel_ingresos*, que ya se ha descrito antes, además del índice de mapas de bits para *sexo*.

Considérese ahora una consulta que seleccione mujeres con ingresos en el rango 10.000 − 19.999 €. Esta consulta se puede expresar como $\sigma_{sexo=\text{f} \wedge nivel_ingresos=L2}(r)$. Para evaluar esta selección se busca el valor f en el mapa de bits de *sexo* y el valor $L2$ en el de *nivel_ingresos* y se realiza la **intersección** (conjunción lógica) de los dos mapas de bits. En otras palabras, se calcula un nuevo mapa de bits en el que el bit i tenga el valor 1 si el i-ésimo bit de los dos mapas de bits es 1, y el valor 0 en caso contrario. En el ejemplo de la Figura 12.32, la intersección del mapa de bits de $sexo = \text{f}$ (01101) y el de $nivel_ingresos = L2$ (01000) da como resultado el mapa de bits 01000.

Como el primer atributo puede tomar dos valores y el segundo cinco, se puede esperar, en promedio, que sólo de 1 a 10 registros satisfagan la condición combinada de los dos atributos. Si hay más condiciones, es probable que la proporción de los registros que satisfacen todas las condiciones sea bastante pequeña. El sistema puede calcular así el resultado de la consulta buscando todos los bits con valor 1 del mapa de bits resultado de la intersección y recuperando los registros correspondientes. Si la proporción es grande, la exploración de la relación completa seguirá siendo la alternativa menos costosa.

Otro uso importante de los mapas de bits es el recuento del número de tuplas que satisfacen una selección dada. Esas consultas son importantes para el análisis de datos. Por ejemplo, si se desea de-

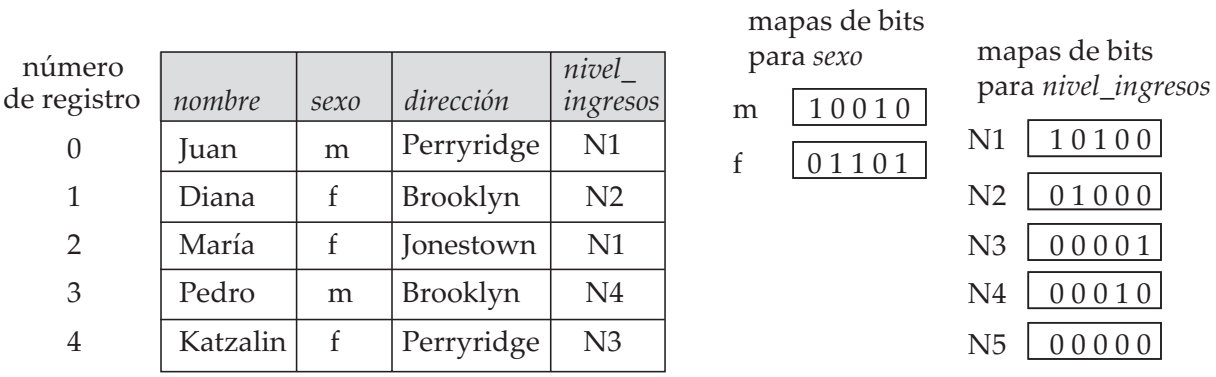


Figura 12.32 Índices de mapas de bits para la relación *info_cliente*.

www.detodoprogramacion.com

terminar el número de mujeres que tienen un nivel de ingresos $L2$, se calcula la intersección de los dos mapas de bits y luego se cuenta el número de bits de valor 1 en el mapa de bits resultado de la intersección. Así se puede obtener el resultado deseado del índice de mapa de bits sin ni siquiera acceder a la relación.

Los índices de mapas de bits son generalmente bastante pequeños en comparación con el tamaño real de la relación. Los registros suelen tener entre decenas y centenares de bytes de longitud, mientras que un solo bit representa a un registro en el mapa de bits. Por tanto, el espacio ocupado por cada mapa de bits suele ser menos del uno por ciento del espacio ocupado por la relación. Por ejemplo, si el tamaño del registro de una relación dada es de 100 bytes, el espacio ocupado por cada mapa de bits sería la octava parte del uno por ciento del espacio ocupado por la relación. Si el atributo A de la relación sólo puede tomar uno valor de entre ocho, el índice de mapas de bits de ese atributo consiste en ocho mapas de bits que, juntos, sólo ocupan el uno por ciento del tamaño de la relación.

El borrado de registros crea huecos en la secuencia de registros, ya que el desplazamiento de registros (o de los números de registro) para rellenar los huecos resultaría excesivamente costoso. Para reconocer los registros borrados se puede almacenar un **mapa de bits de existencia** en el que el bit i sea 0 si el registro i no existe, y 1 en caso contrario. Se verá la necesidad de la existencia de los mapas de bits en el Apartado 12.9.2. La inserción de registros no debe afectar a la secuencia de numeración de los demás registros. Por tanto, se puede insertar tanto añadiendo registros al final del archivo como reemplazando los registros borrados.

12.9.2 Implementación eficiente de las operaciones de mapas de bits

Se puede calcular fácilmente la intersección de dos mapas de bits usando un bucle **for**: la iteración i-ésima del bucle calcula la conjunción (**and**) de los bits i-ésimos de los dos mapas de bits. Se puede acelerar considerablemente el cálculo de la intersección usando las instrucciones de bits **and** soportadas por la mayoría de los conjuntos de instrucciones de las computadoras. Cada *palabra* suele constar de 32 o 64 bits, en función de la arquitectura de la computadora. La instrucción de bits **and** toma dos palabras como entrada y devuelve una palabra en que cada bit es la conjunción lógica de los bits de igual posición de las palabras de entrada. Lo que es importante observar es que una sola instrucción de bits **and** puede calcular la intersección de 32 o de 64 bits *a la vez*.

Si una relación tuviese un millón de registros, cada mapa de bits contendría un millón de bits, o, lo que es equivalente, 128 kilobytes. Sólo se necesitan 31.250 instrucciones para calcular la intersección de dos mapas de bits de la relación, suponiendo un tamaño de palabra de 32 bits. Por tanto, el cálculo de intersecciones de mapas de bits es una operación extremadamente rápida.

Al igual que la intersección de mapas de bits resulta útil para calcular la conjunción de dos condiciones, la unión de mapas de bits resulta útil para calcular la disyunción de dos condiciones. El procedimiento para la unión de mapas de bits es exactamente igual que el de la intersección, salvo que se utiliza la instrucción de bits **or** en lugar de **and**.

La operación complemento se puede usar para calcular un predicado que incluya la negación de una condición, como **not** (*nivel-ingresos* = $L1$). El complemento de un mapa de bits se genera complementando cada uno de sus bits (el complemento de 1 es 0 y el complemento de 0 is 1). Puede parecer que **not** (*nivel_ingresos* = $L1$) se puede implementar simplemente calculando el complemento del mapa de bits del nivel de ingresos $L1$. Sin embargo, si se ha borrado algún registro, el mero cálculo del complemento del mapa de bits no resulta suficiente. Los bits correspondientes a esos registros serán 0 en el mapa de bits original, pero pasarán a ser 1 en el complemento, aunque el registro no exista. También surge un problema similar cuando el valor de un atributo es *nulo*. Por ejemplo, si el valor de *nivel_ingresos* es nulo, el bit será 0 en el mapa de bits original para el valor $L1$ y 1 en el complementado.

Para asegurarse de que los bits correspondientes a registros borrados se definen como 0 en el resultado, hay que intersecar el mapa de bits complementado con el mapa de bits de existencia para desactivar los bits de los registros borrados. Análogamente, para manejar los valores nulos, también se debe intersecar el mapa de bits complementado con el complemento del mapa de bits para el valor *nulo*.[1]

1. El tratamiento de predicados como **is unknown** puede causar aún más complicaciones, que requerirían en general el uso de un mapa de bits adicional para determinar los resultados de las operaciones que son desconocidos.

www.detodoprogramacion.com

Se puede contar rápidamente el número de bits que valen 1 en el mapa de bits si se emplea una técnica inteligente. Se puede mantener un array de 256 entradas, donde la entrada i-ésima almacene el número de bits que valen 1 en la representación binaria de i. Hay que definir el recuento inicial como 0. Se toma cada byte del mapa de bits, se usa para indexar en el array y se añade el recuento almacenado al recuento total. El número de operaciones de suma sería la octava parte del número de tuplas y, por tanto el proceso de recuento es muy eficiente. Un gran array (que emplee $2^{16} = 65.536$ entradas), indexado por pares de bytes, dará incluso aceleraciones mayores, pero con un coste de almacenamiento mayor.

12.9.3 Mapas de bits y árboles B^+

Los mapas de bits se pueden combinar con los índices normales de árboles B^+ para las relaciones donde unos pocos valores de los atributos sean extremadamente frecuentes y también aparezcan otros valores, pero con mucha menor frecuencia. En las hojas de los índices de los árboles B^+, para cada valor se suele mantener una lista de todos los registros con ese valor para el atributo indexado. Cada elemento de la lista sería un identificador de registro que consta, al menos, de 32 bits, y normalmente más. Para cada valor que aparece en muchos registros se almacena un mapa de bits en lugar de una lista de registros.

Supónganse que un valor dado v_i aparece en la dieciseisava parte de los registros de una relación. Sea N el número de registros de la relación y supóngase que cada registro tiene un número de 64 bits que lo identifica. El mapa de bits sólo necesita sólo un bit por registro, o N en total. En cambio, la representación de lista necesita sesenta y cuatro bits por registro en el que aparezca el valor, o $64 * N/16 = 4N$ bits. Por tanto, es preferible el mapa de bits para representar la lista de registros del valor v_i. En el ejemplo (con un identificador de registros de 64 bits), si menos de uno de cada sesenta y cuatro registros tiene un valor dado, es preferible la representación de lista de registros para la identificación de los registros con ese valor, ya que emplea menos bits que la representación con mapas de bits. Si más de uno de cada sesenta y cuatro registros tiene ese valor dado, es preferible la representación de mapas de bits.

Por tanto, los mapas de bits se pueden emplear como mecanismo de almacenamiento comprimido en los nodos hoja de los árboles B^+ de los valores que aparecen muy frecuentemente.

12.10 Definición de índices en SQL

La norma SQL no proporciona al usuario o administrador de la base de datos ninguna manera de controlar qué índices se crean y se mantienen por el sistema de base de datos. Los índices no se necesitan para la corrección, ya que son estructuras de datos redundantes. Sin embargo, los índices son importantes para el procesamiento eficiente de las transacciones, incluyendo las transacciones de actualización y consulta. Los índices son también importantes para un cumplimiento eficiente de las ligaduras de integridad. Por ejemplo, las implementaciones típicas obligan a declarar una clave (Capítulo 4) mediante la creación de un índice con la clave declarada como la clave de búsqueda del índice.

En principio, un sistema de base de datos puede decidir automáticamente qué índices crear. Sin embargo, debido al coste en espacio de los índices, así como el efecto de los índices en el procesamiento de actualizaciones, no es fácil hacer una elección apropiada automáticamente sobre qué índices mantener. Por este motivo, la mayoría de las implementaciones de SQL proporcionan al programador control sobre la creación y eliminación de índices mediante comandos del lenguaje de definición de datos.

A continuación se ilustrará las sintaxis de estos comandos. Aunque la sintaxis que se muestra se usa ampliamente y está soportada en muchos sistemas de bases de datos, no es parte de la norma SQL:1999. Las normas SQL (hasta SQL:1999, al menos) no dan soporte al control del esquema físico de la base de datos y este libro se limita al esquema lógico de la base de datos.

Un índice se crea mediante el comando **create index**, que tiene la forma

create index <nombre-índice> **on** <nombre-relación> (<lista-atributos>)

lista-atributos es la lista de atributos de la relación que constituye la clave de búsqueda del índice

Para definir un índice llamado *índice_sucursal* de la relación *sucursal* con la clave de búsqueda *nombre_sucursal*, se escribe

create index *índice_sucursal* **on** *sucursal* (*nombre_sucursal*)

www.detodoprogramacion.com

Si se desea declarar que la clave de búsqueda es una clave candidata hay que añadir el atributo **unique** a la definición del índice. Con esto, el comando

create unique index *índice_sucursal* **on** *sucursal* (*nombre_sucursal*)

declara *nombre_sucursal* como una clave candidata de *sucursal*. Si cuando se introduce el comando **create unique index**, *nombre_sucursal* no es una clave candidata, se mostrará un mensaje de error y el intento de crear un índice fallará. Por otro lado, si el intento de crear el índice ha tenido éxito, cualquier intento de insertar una tupla que viole la declaración de clave fallará. Hay que observar que el carácter **unique** es redundante si el sistema de bases de datos soporta la declaración **unique** de SQL estándar.

Muchos sistemas de bases de datos ofrecen un modo de especificar el tipo de índice que se va a utilizar (como los árboles B^+ o la asociación). Algunos sistemas de bases de datos permiten también que se declare uno de los índices de una relación como agrupado; el sistema almacena entonces la relación ordenada de acuerdo con la clave de búsqueda del índice agrupado.

Para hacer posible la eliminación (drop) de un índice es necesario especificar su nombre. El comando **drop index** tiene la forma:

drop index <*nombre_índice*>

12.11 Resumen

- Muchas consultas solamente hacen referencia a una pequeña proporción de los registros de un archivo. Para reducir el gasto adicional en la búsqueda de estos registros se pueden construir *índices* para los archivos almacenados en la base de datos.
- Los archivos secuenciales indexados son unos de los esquemas de índice más antiguos usados en los sistemas de bases de datos. Para permitir una rápida recuperación de los registros según el orden de la clave de búsqueda, los registros se almacenan consecutivamente y los que no siguen el orden se encadenan entre sí. Para permitir un acceso aleatorio, se usan estructuras índice.
- Existen dos tipos de índices que se pueden utilizar: los índices densos y los índices dispersos. Los índices densos contienen una entrada por cada valor de la clave de búsqueda, mientras que los índices dispersos contienen entradas sólo para algunos de esos valores.
- Si el orden de una clave de búsqueda se corresponde con el orden secuencial del archivo, un índice sobre la clave de búsqueda se conoce como *índice con agrupación*. Los otros índices son los *índices sin agrupación* o *secundarios*. Los índices secundarios mejoran el rendimiento de las consultas que utilizan otras claves de búsqueda distinta de la del índice con agrupación. Sin embargo, éstas implican un gasto adicional en la modificación de la base de datos.
- El inconveniente principal de la organización del archivo secuencial indexado es que el rendimiento disminuye según crece el archivo. Para superar esta deficiencia se puede usar un *índice de árbol* B^+.
- Un índice de árbol B^+ tiene la forma de un árbol *equilibrado*, en el cual cada camino de la raíz a las hojas del árbol tiene la misma longitud. La altura de un árbol B^+ es proporcional al logaritmo en base N del número de registros de la relación, donde cada nodo interno almacena N punteros; el valor de N está usualmente entre 50 y 100. Los árboles B^+ son más cortos que otras estructuras de árboles binarios equilibrados como los árboles AVL y, por tanto, necesitan menos accesos a disco para localizar los registros.
- Las búsquedas en un índice de árbol B^+ son directas y eficientes. Sin embargo, la inserción y el borrado son algo más complicados pero eficientes. El número de operaciones que se necesitan para la inserción y borrado en un árbol B^+ es proporcional al logaritmo en base N del número de registros de la relación, donde cada nodo interno almacena N punteros.
- Se pueden utilizar los árboles B^+ tanto para indexar un archivo con registros, como para organizar los registros de un archivo.

www.detodoprogramacion.com

CAPÍTULO 13

Procesamiento de consultas

El **procesamiento de consultas** hace referencia a una serie de actividades implicadas en la extracción de datos de una base de datos. Estas actividades incluyen la traducción de consultas expresadas en lenguajes de bases de datos de alto nivel en expresiones implementadas en el nivel físico del sistema, así como transformaciones de optimización de consultas y la evaluación real de las mismas.

13.1 Visión general

En la Figura 13.1 se ilustran los pasos involucrados en el procesamiento de una consulta. Los pasos básicos son:

1. Análisis y traducción.
2. Optimización.
3. Evaluación.

Antes de empezar el procesamiento de una consulta, el sistema debe traducirla a una forma utilizable. Un lenguaje como SQL es adecuado para el uso humano, pero es poco apropiado para una representación interna en el sistema de la consulta. Una representación interna más útil estaría basada en el álgebra relacional extendido.

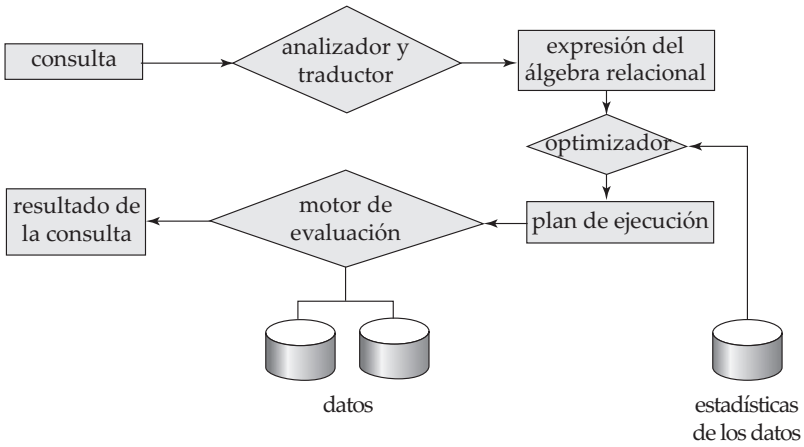


Figura 13.1 Pasos en el procesamiento de una consulta.

www.detodoprogramacion.com

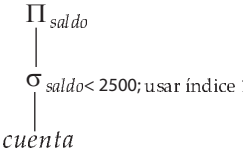


Figura 13.2 Plan de ejecución de una consulta.

Así, la primera acción que el sistema tiene que emprender para procesar una consulta es su traducción a su formato interno. Este proceso de traducción es similar al trabajo que realiza el analizador de un compilador. Durante la generación del formato interno de una consulta, el analizador comprueba la sintaxis de la consulta del usuario, verifica que los nombres de las relaciones que aparecen en ella sean nombres de relaciones en la base de datos, etc. Posteriormente se construye un árbol para el análisis de la consulta, que se transformará en una expresión del álgebra relacional. Si la consulta estuviera expresada en términos de una vista, la fase de traducción también sustituye todas las referencias a vistas por las expresiones del álgebra relacional que las definen[1]. El análisis de lenguajes se describe en la mayoría de los libros sobre compiladores (véanse las notas bibliográficas).

Dada una consulta hay generalmente distintos métodos para obtener la respuesta. Por ejemplo, ya se ha visto que en SQL se puede expresar una consulta de diferentes maneras. Cada consulta SQL puede traducirse a diferentes expresiones del álgebra relacional. Además de esto, la representación de una consulta en el álgebra relacional especifica de manera parcial cómo evaluar la consulta; existen normalmente varias maneras de evaluar expresiones del álgebra relacional. Como ejemplo, considérese la consulta:

select *saldo*
from *cuenta*
where *saldo* < 2500

Esta consulta se puede traducir en alguna de las siguientes expresiones del álgebra relacional:

- $\sigma_{saldo<2500}\ (\Pi_{saldo}\ (cuenta))$
- $\Pi_{saldo}\ (\sigma_{saldo<2500}\ (cuenta))$

Además, se puede ejecutar cada operación del álgebra relacional utilizando alguno de los diferentes algoritmos. Por ejemplo, para implementar la selección anterior se puede examinar cada tupla de *cuenta* para encontrar las tuplas cuyo saldo sea menor que 2500. Por otro lado, si se dispone de un índice de árbol B^+ en el atributo *saldo*, se puede utilizar este índice para localizar las tuplas.

Para especificar completamente cómo evaluar una consulta, no basta con proporcionar la expresión del álgebra relacional, además hay que anotar en ella las instrucciones que especifiquen cómo evaluar cada operación. Estas anotaciones podrían ser el algoritmo a usar para una operación específica o el índice o índices concretos a utilizar. Las operaciones del álgebra relacional anotadas con instrucciones sobre su evaluación reciben el nombre de **primitivas de evaluación**. La secuencia de operaciones primitivas que se pueden utilizar en la evaluación de una consulta establece un **plan de ejecución de la consulta** o un **plan de evaluación de la consulta**. En la Figura 13.2 se ilustra un plan de evaluación para nuestro ejemplo de consulta, en el que se especifica un índice concreto (denotado en la figura como "índice 1") para la operación selección. El **motor de ejecución de consultas** escoge un plan de evaluación, lo ejecuta y devuelve su respuesta a la consulta.

Los diferentes planes de evaluación de una misma consulta dada pueden tener costes distintos. No se puede esperar que los usuarios escriban las consultas de manera que sugieran el plan de evaluación más eficiente. En su lugar, es responsabilidad del sistema construir un plan de evaluación de la consulta

1. Para vistas materializadas, la expresión que define la vista ha sido ya evaluada y almacenada. Por tanto, se puede usar la relación almacenada en lugar de reemplazar los usos de la vista por la expresión que define la vista. Las vistas recursivas se tratan de manera diferente mediante un procedimiento de búsqueda de punto fijo, según se vio en los Apartados 4.7 y 5.4.6.

www.detodoprogramacion.com

que minimice el coste de la evaluación de la consulta; esta tarea se denomina *optimización de consultas*. El Capítulo 14 describe en detalle la optimización de consultas.

Una vez elegido el plan de la consulta, ésta se evalúa con este plan y se muestra su resultado.

La secuencia de pasos que se han descrito para procesar una consulta son representativos; no todas las bases de datos los siguen exactamente. Por ejemplo, en lugar de utilizar la representación del álgebra relacional, varias bases de datos usan una representación del árbol de análisis con anotaciones basada en la estructura de la consulta SQL. Sin embargo, los conceptos que se describen aquí constituyen la base del procesamiento de consultas en las bases de datos.

Para optimizar una consulta, el optimizador de consultas debe conocer el coste de cada operación. Aunque el coste exacto es difícil de calcular, dado que depende de muchos parámetros como la memoria real disponible, es posible obtener una estimación aproximada del coste de ejecución para cada operación.

En este capítulo se estudia la forma de evaluar operaciones individuales en un plan de consulta y cómo estimar su coste; se vuelve a la optimización de consultas en el Capítulo 14. En el Apartado 13.2 se describe cómo se mide el coste de una consulta. Desde el Apartado 13.3 hasta el 13.6 se estudia la evaluación de operaciones individuales del álgebra relacional. Varias operaciones se pueden agrupar en un **cauce**, en el que cada una de las ellas empieza trabajando sobre sus tuplas de entrada del mismo modo que si fuesen generadas por otra operación. En el Apartado 13.7 se examina cómo coordinar la ejecución de varias operaciones de un plan de evaluación de consultas, en particular, cómo usar las operaciones encauzadas para evitar escribir resultados intermedios en disco.

13.2 Medidas del coste de una consulta

El coste de la evaluación de una consulta se puede expresar en términos de diferentes recursos, incluyendo los accesos a disco, el tiempo de CPU en ejecutar una consulta y, en sistemas de bases de datos distribuidos o paralelos, el coste de la comunicación (que se estudiará más tarde en los Capítulos 21 y 22). El tiempo de respuesta para un plan de evaluación de una consulta (esto es, el tiempo de reloj que se necesita para ejecutar el plan), si se supone que no hay otra actividad ejecutándose en el sistema, podría tener en cuenta todos estos costes y utilizarlos como una buena medida del coste del plan.

Sin embargo, en grandes sistemas de bases de datos, el coste de acceso a los datos en disco es normalmente el más importante, ya que los accesos a disco son más lentos comparados con las operaciones en memoria. Además, la velocidad de la CPU aumenta mucho más rápidamente que las velocidades de los discos. Así, lo más probable es que el tiempo empleado en operaciones del disco siga influyendo en el tiempo total de ejecución de una consulta. Las estimaciones del tiempo de CPU son más difíciles de hacer porque dependen de detalles de bajo nivel del código de ejecución. Aunque los optimizadores reales de consultas consideran los costes de CPU, aquí se ignoran y sólo se considera el coste de los accesos a disco para medir el coste del plan de evaluación de una consulta.

Se usarán el *número de transferencias de bloques* de disco y el *número de búsquedas de disco* para medir el coste del acceso a datos en disco. Si al subsistema de disco le lleva una media de t_T segundos para transferir un bloque de datos, con un tiempo medio de acceso a bloque (tiempo de búsqueda en el disco más latencia rotacional) de t_S segundos, entoces una operación que transfiera b bloques y ejecute S búsquedas tardaría $b * t_T + S * t_S$ segundos. Los valores de t_T y de t_S se deben ajustar al disco usado, pero los valores normales de los discos de altas prestaciones actuales serían $t_S = 4$ milisegundos y $t_T = 0.1$ milisegundos, asumiendo un tamaño de bloque de 4 kilobytes y una velocidad de transferencia de 40 megabytes por segundo[2].

Se puede refinar aún más la estimación de coste distinguiendo entre las lecturas y escrituras de bloques, dado que normalmente se tarda el doble de tiempo en escribir un bloque que en leerlo de disco (debido a que los sistemas de disco leen los sectores después de escribirlos para comprobar que la escritura fue correcta). Para simplificar se ignorará este detalle y se propone al lector que desarrolle estimaciones de coste más precisas para distintas operaciones.

Las estimaciones de coste que se proporcionan no incluyen el coste de escribir el resultado final de una operación en disco. Esto se tendrá en cuenta cuando sea preciso. Los costes de todos los algoritmos

2. Algunos sistemas de bases de datos ejecutan búsquedas y transferencias de bloque de prueba para estimar los costes medios de búsqueda y transferencia de bloque, como parte del proceso de instalación del software.

www.detodoprogramacion.com

que se consideran aquí dependen del tamaño de la memoria intermedia en la memoria principal. En el mejor caso, todos los datos se pueden leer en las memorias intermedias y no es necesario acceder de nuevo a disco. En el peor caso se asume que la memoria intermedia puede contener sólo unos pocos bloques de datos (aproximadamente un bloque por relación). Al presentar las estimaciones de coste se asume generalmente el peor caso.

Además, aunque se asuma que los datos se deben leer inicialmente de disco, es posible que el bloque al que se acceda pueda estar en la memoria intermedia. De nuevo y para simplificar se ignorará este efecto; como resultado el coste del acceso a disco real durante la ejecución de un plan puede ser menor que el coste estimado.

13.3 Operación selección

En el procesamiento de consultas, el **explorador de archivo** es el operador de nivel más bajo para acceder a los datos. Los exploradores de archivo son algoritmos de búsqueda que localizan y recuperan los registros que cumplen una condición de selección. En los sistemas relacionales, el explorador de archivo permite leer una relación completa en los casos en que la relación se almacena en un único archivo dedicado.

13.3.1 Algoritmos básicos

Considérese una operación selección en una relación cuyas tuplas se almacenan juntas en un archivo. A continuación se muestran dos algoritmos exploradores que implementan la operación selección:

- **A1** (**búsqueda lineal**). En una búsqueda lineal se explora cada bloque del archivo y se comprueban todos los registros para determinar si satisfacen o no la condición de selección. Para una selección sobre un atributo clave, el sistema puede terminar la exploración si encuentra el registro requerido sin necesidad de examinar los otros registros de la relación.

  El coste de la búsqueda lineal, en términos de operaciones E/S, es una búsqueda más b_r transferencias de bloque, donde b_r denota el número de bloques del archivo o, de forma equivalente, el coste temporal es $t_S + b_r * t_T$

  En una selección sobre un atributo clave el sistema puede terminar la exploración si se encuentra el registro requerido, sin examinar los otros registros de la relación. Las selecciones sobre atributos clave presentan un coste medio de $b_r/2$, pero en el peor caso aún tiene un coste de b_r transferencias de bloque más una búsqueda.

  Aunque podría ser más lento que otros algoritmos para la implementación de la selección, el algoritmo de búsqueda lineal se puede aplicar a cualquier archivo, sin importar su ordenación, la presencia de índices o la naturaleza de la operación selección. El resto de algoritmos que se estudiarán no son aplicables en todos los casos, pero cuando lo son, son más rápidos que la búsqueda lineal.

- **A2** (**búsqueda binaria**). Si el archivo está ordenado según un atributo y la condición de la selección es una comparación de igualdad en ese atributo, se puede utilizar una búsqueda binaria para localizar los registros que satisfacen la selección. El sistema realiza la búsqueda binaria de los bloques del archivo.

  En el peor caso, el número de bloques que deben ser examinados para encontrar los registros requeridos es $\lceil \log_2(b_r) \rceil$, donde b_r denota el número de bloques del archivo. Cada uno de estos accesos a bloque requiere una búsqueda además de una transferencia de bloque, y por tanto el coste temporal es $\lceil \log_2(b_r) \rceil * (t_T + t_S)$.

  Si la selección es sobre un atributo que no sea clave, es posible que más de un bloque contenga los registros requeridos, y el coste de la lectura de los bloques extra se debe añadir a la estimación del coste. Se puede calcular este número mediante la estimación del tamaño del resultado de la selección (que se explica en el Apartado 14.3) y dividiéndolo entre el número de registros que se almacenan por bloque de la relación. Se asume que estos bloques se almacenan contiguos por lo que sólo se añade un coste de transferencia t_T por cada bloque adicional.

www.detodoprogramacion.com

13.3.2 Selecciones con índices

Las estructuras índice se denominan **caminos de acceso**, ya que proporcionan un camino a través del cual se pueden localizar y acceder a los datos. En el Capítulo 12 se señaló la eficiencia de leer los registros del archivo en un orden próximo al orden físico. Recuérdese que un *índice primario* (también conocido como *índice con agrupación*) es un índice que permite leer los registros de un archivo en el mismo orden que el orden físico del archivo. Un índice que no es primario se llama *índice secundario*.

Los algoritmos de búsqueda que utilizan un índice reciben el nombre de **exploraciones del índice**. Los índices ordenados, como los árboles B^+, también permiten acceder a las tuplas según cierto orden que es útil para la implementación de las consultas de rangos. Aunque los índices pueden proporcionar un acceso rápido, directo y ordenado, su uso implica un gasto adicional en los accesos a los bloques que contienen el índice. Utilizaremos el predicado de selección como guía en la elección del índice a usar en el procesamiento de la consulta. Los algoritmos de búsqueda que usan un índice son:

- **A3** (**índice primario, igualdad basada en la clave**). Para una condición de igualdad en un atributo clave con un índice primario se puede utilizar el índice para recuperar el único registro que satisface la correspondiente condición de igualdad.

  Si se usa un árbol B^+, el coste de la operación en términos de operaciones E/S es igual a la altura del árbol más una operación E/S para recuperar el registro[3]; cada una de las cuales requiere una búsqueda y una transferencia de bloque. Por tanto, el coste es $(h_i + 1) * (t_T + t_S)$, donde h_i denota la altura del índice[4].

- **A4** (**índice primario, igualdad basada en un atributo no clave**). Se pueden recuperar varios registros mediante el uso de un índice primario cuando la condición de selección especifica una comparación de igualdad en un atributo A que no sea clave. La única diferencia del caso anterior es que puede ser necesario recuperar varios registros. Sin embargo, estos registros estarían almacenados consecutivamente en el archivo ya que el archivo se ordena según la clave de búsqueda.

  El coste de la operación depende de la altura del árbol más el número de bloques que contengan registros con la clave de búsqueda especificada. Se necesita una búsqueda para cada nivel del árbol. Además se necesita una búsqueda para obtener el primer bloque que contenga el registro deseado; el resto de bloques se almacenan consecutivamente y no requieren más búsquedas. En concreto el coste es $h_i * (t_T + t_S) + t_S + b * t_T$, donde h_i denota la altura del índice y b denota el número de bloques que contienen bloques con la clave de búsqueda especificada.

- **A5** (**índice secundario, igualdad**). Las selecciones con una condición de igualdad pueden utilizar un índice secundario. Esta estrategia puede recuperar un único registro si la condición de igualdad es sobre una clave; puede que se recuperen varios registros si el campo índice no es clave.

  En el primer caso sólo se obtiene un registro, y el coste es igual a la altura del árbol más una operación E/S para recuperar el registro. Cada operación E/S requiere una búsqueda y una transferencia de bloque. El coste temporal en este caso es el mismo que el del índice primario (caso A3).

  En el segundo caso, cada registro puede residir en un bloque diferente, que puede provocar una operación E/S por cada registro recuperado, y cada operación E/S requiere una búsqueda y una transferencia de bloque. El coste podría llegar a ser incluso peor que el de la búsqueda lineal si se obtiene un gran número de registros. El coste temporal es en este caso $(h_i + n) * (t_S + t_T)$, donde n es el número de registros recuperados[5].

3. Si se usa un árbol B^+, no se requiere E/S adicional porque los registros se almacenan en las hojas del árbol. Se deberían hacer ajustes similares a algunos de los algoritmos descritos a continuación en este apartado cuando se usen árboles B^+.
4. Los optimizadores reales asumen generalmente que la raíz del árbol está en la memoria intermedia porque se accede frecuentemente a ella. Algunos optimizadores incluso asumen que todos menos las hojas se encuentran en memoria porque se accede a ellos con relativa frecuencia, y normalmente el uno por ciento de los nodos no son hojas. La fórmula del coste se debe modificar adecuadamente.
5. Si la memoria intermedia es grande, el bloque que contenga el registro puede estar ya en memoria principal. Es posible construir una estimación del coste *medio* o *esperado* de la selección teniendo en cuenta la probabilidad de que el bloque se encuentre ya en memoria. Para grandes memorias intermedias la estimación será mucho menor que la estimación del peor caso.

www.detodoprogramacion.com

Como se describió en el Apartado 12.5.5, cuando los registros se almacenan en una organización de árbol B^+ u otras organizaciones que puedan requerir la reubicación de los registros, los índices secundarios generalmente no almacenan punteros a los registros[6]. En su lugar, estos índices almacenan los valores de los atributos usados como la clave de búsqueda en una organización de árbol B^+. El acceso a un registro mediante un índice secundario es por tanto más caro: primero se busca en el índice secundario para encontrar los valores de la clave de búsqueda del índice primario y después se busca en el índice primario para encontrar los registros. La fórmula del coste descrita para índices secundarios se tendrá que modificar adecuadamente si se usan estos índices.

13.3.3 Selecciones con condiciones de comparación

Considérese una selección de la forma $\sigma_{A \leq v}(r)$. Se pueden implementar utilizando búsqueda lineal o binaria, o con índices de alguna de las siguientes maneras:

- **A6 (índice primario, comparación)**. Se puede utilizar un índice ordenado primario (por ejemplo, un índice primario de árbol B^+) cuando la condición de selección sea una comparación. Para condiciones con comparaciones de la forma $A > v$ o $A \geq v$ se puede usar el índice primario sobre A para guiar la recuperación de las tuplas de la manera siguiente. Para el caso de $A \geq v$ se busca el valor de v en el índice para encontrar la primera tupla del archivo que tenga un valor de $A = v$. Un explorador de archivo comenzando en esa tupla y hasta el final del archivo devuelve todas las tuplas que satisfacen la condición. Para $A > v$, el explorador de archivo comienza con la primera tupla tal que $A > v$. La estimación de coste en este caso es idéntica a la del caso A4.

  Para comparaciones de la forma $A < v$ o $A \leq v$ no es necesario buscar en el índice. Para el caso de $A < v$ se utiliza un simple explorador partiendo del inicio del archivo y continuando hasta (pero sin incluirlo) la primera tupla con el atributo $A = v$. El caso $A \leq v$ es similar, excepto que el explorador continúa hasta (pero sin incluir) la primera tupla con el atributo $A > v$. En ninguno de los dos casos el índice es de utilidad alguna.

- **A7 (índice secundario, comparación)**. Se puede utilizar un índice secundario ordenado para guiar la recuperación bajo condiciones de comparación que contengan $<, \leq, \geq$, o $>$. Los bloques del índice del nivel más bajo se exploran o bien desde el valor más pequeño hasta v (para $<$ y $\leq$) o bien desde v hasta el valor mayor (para $>$ y $\geq$).

  El índice secundario proporciona punteros a los registros, pero para obtener los reales es necesario extraerlos usando los punteros. Este paso puede requerir una operación E/S por cada registro extraído, dado que los consecutivos pueden estar en diferentes bloques de disco; como antes, cada operación E/S requiere una búsqueda y una transferencia de bloque. Si el número de registros extraídos es grande, el uso del índice secundario puede ser incluso más caro que la búsqueda lineal. Por tanto, el índice secundario sólo se debería usar si se seleccionan muy pocos registros.

13.3.4 Implementación de selecciones complejas

Hasta ahora sólo se han considerado condiciones de selección simples de la forma $AopB$, donde op es una operación igualdad o de comparación. Se revisarán a continuación predicados de selección más complejos.

- **Conjunción**. Una *selección conjuntiva* es una selección de la forma

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n}(r)$$

- **Disyunción**. Una *selección disyuntiva* es una selección de la forma

$$\sigma_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n}(r)$$

6. Recuérdese que si se usan organizaciones de árboles B^+ para almacenar relaciones, los registros se pueden trasladar entre bloques cuando los nodos hoja se dividen o fusionan, y cuando se redistribuyen los registros.

www.detodoprogramacion.com

Una condición disyuntiva se cumple mediante la unión de todos los registros que cumplen alguna de las condiciones θ_i.

- **Negación**. El resultado de una selección $\sigma_{\neg\theta}(r)$ es el conjunto de tuplas de r para las que la condición θ es falsa. A falta de valores nulos, este conjunto es simplemente el conjunto de tuplas que no están en $\sigma_\theta(r)$.

Se puede implementar una operación selección con una conjunción o una disyunción de condiciones sencillas utilizando alguno de los siguientes algoritmos:

- **A8** (**selección conjuntiva utilizando un indice**). Inicialmente hay que determinar si para un atributo hay disponible algún camino de acceso en alguna de las condiciones simples. Si lo hay, cualquiera de los algoritmos de selección A2 hasta A7 puede recuperar los registros que cumplan esa condición. Se completa la operación mediante la comprobación en la memoria intermedia de que cada registro recuperado cumpla o no el resto de condiciones simples.

  Para reducir el coste, se elige un θ_i y uno de los algoritmos entre A1 y A7 para el que la combinación resulte en el menor coste de $\sigma_{\theta_i}(r)$. El coste del algoritmo A8 está determinado por el coste del algoritmo elegido.

- **A9** (**selección conjuntiva utilizando un índice compuesto**). Puede que se disponga de un *índice compuesto* (es decir, un índice sobre varios atributos) apropiado para algunas selecciones conjuntivas. Si la selección especifica una condición de igualdad en dos o más atributos y existe un índice compuesto en estos campos con atributos combinados, entonces se podría buscar en el índice directamente. El tipo de índice determina cuál de los algoritmos A3, A4 o A5 se utilizará.

- **A10** (**selección conjuntiva mediante la intersección de identificadores**). Otra alternativa para implementar la operación selección conjuntiva implica la utilización de punteros a registros o identificadores de registros. Este algoritmo necesita índices con punteros a registros en los campos involucrados por cada condición individual. De este modo se explora cada índice en busca de punteros cuyas tuplas cumplan alguna condición individual. La intersección de todos los punteros recuperados forma el conjunto de punteros a tuplas que satisfacen la condición conjuntiva. Luego se usa el conjunto de punteros para recuperar los registros reales. Si no hubiera índices disponibles para algunas condiciones concretas entonces habría que comprobar el resto de condiciones de los registros recuperados.

  El coste del algoritmo **A10** es la suma de los costes de las cada una de las exploraciones del índice más el coste de recuperar los registros en la intersección de las listas recuperadas de punteros. Este coste se puede reducir ordenando la lista de punteros y recuperando registros en orden. Por tanto, (1) todos los punteros a los registros de un bloque van juntos, y así todos los registros seleccionados en el bloque se pueden recuperar usando una única operación E/S, y (2) los bloques se leen ordenados, minimizando el movimiento del brazo del disco. El Apartado 13.4 describe algunos algoritmos de ordenación.

- **A11** (**selección disyuntiva mediante la unión de identificadores**). Si se dispone de caminos de acceso en todas las condiciones de la selección disyuntiva, se explora cada índice en busca de punteros cuyas tuplas cumplan una condición individual. La unión de todos los punteros recuperados proporciona el conjunto de punteros a todas las tuplas que cumplen la condición disyuntiva. Después se utilizan estos punteros para recuperar los registros reales.

  Sin embargo, aunque sólo una de las condiciones no tenga un camino de acceso, se tiene que realizar una búsqueda lineal en la relación para encontrar todas las tuplas que cumplan la condición. Por tanto, aunque sólo exista una de estas condiciones en la disyunción, el método de acceso más eficiente es una exploración lineal, que comprueba durante la exploración la condición disyuntiva en cada tupla.

La implementación de selecciones con condiciones negativas se deja como ejercicio (Ejercicio práctico 13.6).

www.detodoprogramacion.com

13.4 Ordenación

La ordenación de los datos juega un papel importante en los sistemas de bases de datos por dos razones. Primero, las consultas SQL pueden solicitar que los resultados sean ordenados. Segundo, e igualmente importante para el procesamiento de consultas, varias de las operaciones relacionales, como las reuniones, se pueden implementar eficientemente si las relaciones de entrada están ordenadas. Por este motivo se revisa la ordenación antes que la operación reunión en el Apartado 13.5.

La ordenación se puede conseguir mediante la construcción de un índice en la clave de ordenación y utilizando luego ese índice para leer la relación de manera ordenada. No obstante, este proceso ordena la relación sólo *lógicamente* a través de un índice, en lugar de *físicamente*. Por tanto, la lectura de las tuplas de manera ordenada podría implicar un acceso a disco (búsqueda más transferencia de bloque) para cada tupla, lo que puede ser muy costoso dado que el número de registros puede ser mucho mayor que el número de bloques. Por esta razón sería deseable ordenar las tuplas físicamente.

El problema de la ordenación se ha estudiado ampliamente para los casos en que la relación cabe completamente en memoria principal, y para el caso en el que la relación es mayor que la memoria. En el primer caso se utilizan técnicas de ordenación clásicas como la ordenación rápida (*quick-sort*). Aquí se estudia cómo tratar el segundo caso.

La ordenación de relaciones que no caben en memoria se denomina **ordenación externa**. La técnica más utilizada para la ordenación externa es normalmente el algoritmo de **ordenación–mezcla externa**. A continuación se describe el algoritmo de ordenación–mezcla externa. Sea M el número de marcos de página en la memoria intermedia de la memoria principal (el número de bloques de disco cuyos contenidos se pueden alojar en la memoria intermedia de la memoria principal).

1. En la primera etapa, se crean varias **secuencias** ordenadas; cada **secuencia** está ordenada, pero sólo contiene parte de los registros de la relación.

```
i = 0;
repeat
    leer M bloques, o bien de la relación, o bien del resto de la relación,
        según el que tenga menor número de bloques;
    ordenar la parte de la relación que está en la memoria;
    escribir los datos ordenados al archivo de secuencias S_i;
    i = i + 1;
until el final de la relación
```

2. En la segunda etapa, las secuencias se *mezclan*. Supóngase que, por ahora, el número total de secuencias N es menor que M, así que se puede asignar un marco de página para cada secuencia y reservar espacio para guardar una página con el resultado. La etapa de mezcla se lleva a cabo de la siguiente manera:

```
leer un bloque de cada uno de los N archivos S_i y
    guardarlos en una página de la memoria intermedia en memoria;
repeat
    elegir la primera tupla (según el orden) de entre todas las páginas
        de la memoria intermedia;
    escribir la tupla y suprimirla de la página de la memoria intermedia;
    if la página de la memoria intermedia de alguna secuencia S_i está vacía
      and not fin-de-archivo(S_i)
        then leer el siguiente bloque de S_i y guardarlo en la página de la memoria intermedia;
until todas las páginas de la memoria intermedia estén vacías
```

El resultado de la etapa de mezcla es la relación ya ordenada. El archivo de salida se almacena en una memoria intermedia para reducir el número de operaciones de escritura en el disco. La operación anterior de mezcla es una generalización de la mezcla de dos vías utilizado por el algoritmo normal de ordenación–mezcla en memoria; éste mezcla N secuencias, por eso se llama **mezcla de N vías**.

www.detodoprogramacion.com

En general, si la relación es mucho mayor que la memoria, se podrían generar M o más secuencias en la primera etapa y no sería posible asignar un marco de página para cada secuencia durante la etapa de mezcla. En este caso, se realiza la operación de mezcla en varios ciclos. Como hay suficiente memoria para $M - 1$ páginas de la memoria intermedia de entrada, cada mezcla puede tomar $M - 1$ secuencias como entrada.

El *ciclo* inicial se realiza como sigue. Se mezclan las $M - 1$ secuencias primeras (según se ha descrito ya en el punto 2) para obtener un única secuencia en el siguiente ciclo. Luego se mezclan de manera similar las siguientes $M-1$ secuencias, continuando así hasta que todas las secuencias iniciales se hayan procesado. En este punto, el número de secuencias se ha reducido por un factor de $M - 1$. Si el número reducido de secuencias es todavía mayor o igual que M, se realiza otro ciclo con las secuencias creadas en el primer ciclo como entrada. Cada ciclo reduce el número de secuencias por un factor de $M - 1$. Estos ciclos se repiten tantas veces como sea necesario, hasta que el número de secuencias sea menor que M; momento en el que un último ciclo genera el resultado ordenado.

En la Figura 13.3 se ilustran los pasos de la ordenación–mezcla externa en una relación ficticia. Por motivos didácticos supóngase que solamente cabe una tupla en cada bloque ($f_r = 1$) y que la memoria puede contener como mucho tres marcos de página. Durante la etapa de mezcla se utilizan dos marcos de página como entrada y uno para la salida.

El coste de acceso a disco de la ordenación—mezcla externa se calcula de la siguiente manera: sea b_r el número de bloques que contienen registros de la relación r. En la primera etapa, se lee y se copia de nuevo cada bloque de la relación, lo que resulta en un total de $2b_r$ transferencias de bloques. El número inicial de secuencias es $\lceil b_r/M \rceil$. Puesto que el número de secuencias decrece en un factor de $M - 1$ en cada ciclo de la mezcla, el número total de ciclos requeridos viene dado por la expresión $\lceil \log_{M-1}(b_r/M) \rceil$. Cada uno de estos ciclos lee y copia todos los bloques de la relación una vez, con dos excepciones. En primer lugar, el ciclo final puede producir la ordenación como resultado sin escribir el resultado en disco. En segundo lugar, puede que haya secuencias que ni lean ni copien durante un ciclo (por ejemplo, si hay M secuencias que mezclar en un ciclo, se leen y se mezclan $M - 1$, y no se accede a una de las secuencias durante ese ciclo). Si se ignora el ahorro (relativamente pequeño) debido a este

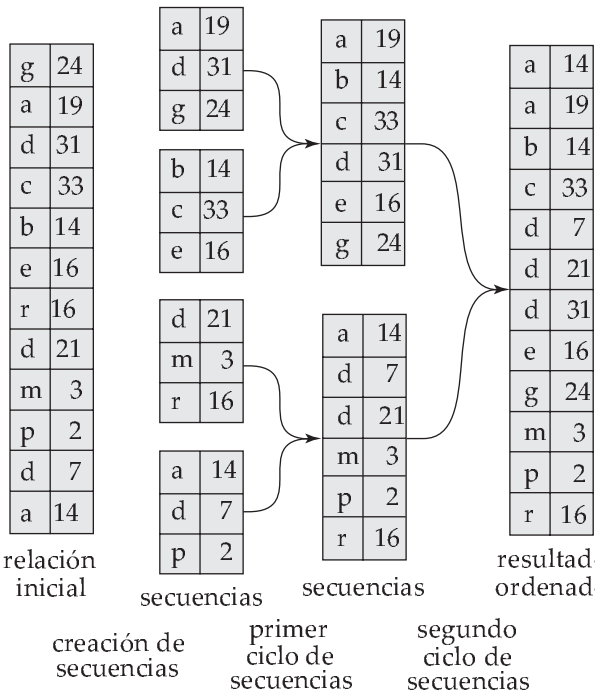


Figura 13.3 Ordenación externa utilizando ordenación–mezcla.

www.detodoprogramacion.com

último efecto, el número total de transferencias de bloques para la ordenación externa de la relación es

$$b_r(2\lceil \log_{M-1}(b_r/M)\rceil + 1)$$

Aplicando esta ecuación al ejemplo de la Figura 13.3 se obtiene un total de $12*(4+1) = 60$ transferencias de bloques, como se puede comprobar en la figura. Obsérvese que este número no incluye el coste de escribir el resultado final.

También es necesario añadir los costes de la búsqueda en disco. La generación de ciclos requiere búsquedas para la lectura de datos para cada una de las secuencias, así como para escribir las secuencias. Durante la fase de mezcla, si se leen simultáneamente b_b bloques de cada secuencia (es decir, se asignan b_b bloques de memoria intermedia a cada secuencia), entonces cada paso de mezcla requeriría cerca de $\lceil b_r/b_b \rceil$ búsquedas para la lectura de los datos[7]. Aunque la salida se escriba secuencialmente, si está en el mismo disco que las secuencias de entrada, la cabeza se puede haber desplazado entre las escrituras de los bloques consecutivos. Así que habría que añadir un total de $2\lceil b_r/b_b \rceil$ búsquedas por cada paso de mezcla, excepto el paso final (ya que se asume que el resultado final no se escribe a disco). El número total de búsquedas es, por tanto:

$$2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil (2\lceil \log_{M-1}(b_r/M)\rceil - 1)$$

Aplicando esta ecuación al ejemplo de la Figura 13.3 se obtiene un total de $8 + 12 * (2 * 2 - 1) = 44$ búsquedas si se establece el número de bloques por secuencia b_b como 1.

13.5 Operación reunión

En este apartado se estudian varios algoritmos para calcular la reunión de relaciones y para analizar sus costes asociados.

Se utiliza la palabra **equirreunión** para hacer referencia a las reuniones de la forma $r \bowtie_{r.A=s.B} s$, donde A y B son atributos o conjuntos de atributos de las relaciones r y s, respectivamente.

Utilizaremos como ejemplo la expresión

$$\textit{impositor} \bowtie \textit{cliente}$$

la cual utiliza los mismos esquemas de relación de los Capítulos 2 y 3. Se da por supuesta la siguiente información de catálogo acerca de las dos relaciones:

- Número de registros de *cliente*: $n_{cliente} = 10.000$.
- Número de bloques de *cliente*: $b_{cliente} = 400$.
- Número de registros de *impositor*: $n_{impositor} = 5.000$.
- Número de bloques de *impositor*: $b_{impositor} = 100$.

13.5.1 Reunión en bucle anidado

En la Figura 13.4 se muestra un algoritmo sencillo para calcular la reunión zeta, $r \bowtie_\theta s$, de dos relaciones r y s. Este algoritmo se llama de **reunión en bucle anidado**, ya que básicamente consiste en un par de bucles **for** anidados. La relación r se denomina la **relación externa** y s la **relación interna** de la reunión, puesto que el bucle de r incluye al bucle de s. El algoritmo utiliza la notación $t_r \cdot t_s$, donde t_r y t_s son tuplas; $t_r \cdot t_s$ denota la tupla construida mediante la concatenación de los valores de los atributos de las tuplas t_r y t_s.

Al igual que el algoritmo de búsqueda lineal en archivos de la selección, el algoritmo de reunión en bucle anidado tampoco necesita índices y se puede utilizar sin importar la condición de la reunión. La manera de extender el algoritmo para calcular la reunión natural es directa, puesto que la reunión natural se puede expresar como una reunión zeta seguida de la eliminación de los atributos repetidos mediante una proyección. El único cambio que se necesita es un paso adicional para borrar los atributos repetidos de la tupla $t_r \cdot t_s$ antes de añadirla al resultado.

7. Para ser más precisos, dado que cada secuencia se lee por separado y se pueden obtener menos de b_b bloques al leer el final de una secuencia, es posible que se necesite una búsqueda adicional por cada secuencia. Para simplificar se ignorará este detalle.

www.detodoprogramacion.com

```
for each tupla t_r in r do begin
    for each tupla t_s in s do begin
        comprobar que el par (t_r, t_s) satisface la condición θ de la reunión
        si la cumple, añadir t_r · t_s al resultado.
    end
end
```

Figura 13.4 Reunión en bucle anidado.

```
for each bloque B_r of r do begin
    for each bloque B_s of s do begin
        for each tupla t_r in B_r do begin
            for each tupla t_s in B_s do begin
                comprobar que el par (t_r, t_s) satisface la condición de la reunión
                si la cumple, añadir t_r · t_s al resultado.
            end
        end
    end
end
```

Figura 13.5 Reunión en bucle anidado por bloques.

El algoritmo de reunión en bucle anidado es costoso, ya que examina cada pareja de tuplas en las dos relaciones. Considérese el coste del algoritmo de reunión en bucle anidado. El número de pares de tuplas a considerar es $n_r * n_s$, donde n_r denota el número de tuplas en r, y n_s denota el número de tuplas en s. Para registro de r se tiene que realizar una exploración completa de s. En el peor caso, la memoria intermedia solamente puede contener un bloque de cada relación, necesitándose un total de $n_r * b_s + b_r$ transferencias de bloques, donde b_r y b_s denotan el número de bloques que contienen tuplas de r y de s, respectivamente. Sólo se necesita una búsqueda por cada exploración de la relación interna s, dado que se lee secuencialmente, y un total de b_r búsquedas para leer r, por lo que resulta un total de $n_r + b_r$ búsquedas. En el mejor caso hay suficiente espacio para que las dos relaciones quepan en memoria, así que cada bloque se tendrá que leer solamente una vez; en consecuencia, sólo se necesitarán $b_r + b_s$ transferencias de bloques, además de 2 búsquedas.

Si una de las relaciones cabe en memoria por completo, es útil usar esa relación como la relación más interna, ya que solamente será necesario leer una vez la relación del bucle más interno. Por lo tanto, si s es lo suficientemente pequeña para caber en la memoria principal, esta estrategia necesita solamente un total de $b_r + b_s$ transferencias de bloques y dos búsquedas (el mismo coste que en el caso en que las dos relaciones quepan en memoria).

Considérese ahora el caso de la reunión natural de *impositor* y *cliente*. Supóngase que, por el momento, no hay ningún índice en cualquiera de las relaciones y que no se desea crear ninguno. Se pueden utilizar los bucles anidados para calcular la reunión; supóngase que *impositor* es la relación externa y que *cliente* es la relación interna de la reunión. Se tendrán que examinar $5.000 * 10.000 = 50 * 10^6$ pares de tuplas. En el peor de los casos el número de transferencias de bloques será de $5.000 * 400 + 100 = 2.000.100$, más $5.000 + 100 = 5.100$ búsquedas. En la mejor de las situaciones, sin embargo, se tienen que leer ambas relaciones solamente una vez y realizar el cálculo. Este cálculo necesita a lo sumo $100 + 400 = 500$ transferencias de bloques más dos búsquedas (una mejora significativa respecto de la peor situación posible). Si se hubiera utilizado *cliente* como la relación del bucle externo e *impositor* para el bucle interno, el coste en el peor de los casos de la última estrategia habría sido menor: $10.000 * 100 + 400 = 1.000.400$ transferencias de bloques más 10.400 búsquedas. El número de transferencias de bloques es significativamente menor y, aunque el número de búsquedas es superior, el coste total se reduce, suponiendo que $t_S = 4$ milisegundos y que $t_T = 0,1$ milisegundos.

www.detodoprogramacion.com

13.5.2 Reunión en bucle anidado por bloques

Si la memoria intermedia es demasiado pequeña para contener las dos relaciones por completo en memoria, todavía se puede lograr un mayor ahorro en los accesos a los bloques si se procesan las relaciones por bloques en lugar de por tuplas. En la Figura 13.5 se muestra la **reunión en bucle anidado por bloques**, que es una variante de la reunión en bucle anidado donde se empareja cada bloque de la relación interna con cada bloque de la relación externa. En cada par de bloques se empareja cada tupla de un bloque con cada tupla del otro bloque para generar todos los pares de tuplas. Al igual que antes se añaden al resultado todas las parejas de tuplas que satisfacen la condición de la reunión.

La diferencia principal en coste entre la reunión en bucle anidado por bloques y la reunión en bucle anidado básica es que, en el peor de los casos, cada bloque de la relación interna s se lee solamente una vez por cada *bloque* de la relación externa, en lugar de una vez por cada *tupla* de la relación externa. De este modo, en el peor de los casos, habrá un total de $b_r * b_s + b_r$ transferencias de bloques, donde b_r y b_s denotan respectivamente el número de bloques que contienen registros de r y de s, respectivamente. Cada exploración de la relación interna requiere una búsqueda y la exploración de la relación externa una búsqueda por bloque, dando un total de $2 * b_r$ búsquedas. Evidentemente, será más eficiente utilizar la relación más pequeña como la relación externa, en caso de que ninguna de ellas quepa en memoria. En el mejor de los casos, cuando la relación interna quepa en memoria, habrá que realizar $b_r + b_s$ transferencias de bloques y sólo dos búsquedas (en este caso se escogería la relación de menor tamaño como relación interna).

En el ejemplo del cálculo de *impositor* $\bowtie$ *cliente* se usará ahora el algoritmo de reunión en bucle anidado por bloques. En el peor de los casos hay que leer una vez cada bloque de *cliente* por cada bloque de *impositor*. Así, en el peor caso, son necesarias un total de $100 * 400 + 100 = 40.100$ transferencias de bloques más $2 * 100 = 200$ búsquedas. Este coste supone una mejora importante frente a las $5.000 * 400 + 100 = 2.000.100$ transferencias de bloques más 5.100 búsquedas necesarias en el peor de los casos para la reunión en bucle anidado básica. El coste en el mejor caso sigue siendo el mismo (es decir, $100 + 400 = 500$ transferencias de bloques y dos búsquedas).

El rendimiento de los procedimientos de bucle anidado y bucle anidado por bloques se puede mejorar aún más:

- Si los atributos de la reunión en una reunión natural o en una equirreunión forman una clave de la relación interna, entonces el bucle interno puede finalizar tan pronto como se encuentre la primera correspondencia.
- En el algoritmo en bucle anidado por bloques, en lugar de utilizar bloques de disco como la unidad de bloqueo de la relación externa, se puede utilizar el mayor tamaño que quepa en memoria, mientras quede suficiente espacio para las memorias intermedias de la relación interna y la salida. En otras palabras, si la memoria tiene M bloques, se leen $M - 2$ bloques de la relación externa de una vez, y cuando se lee cada bloque de la relación interna se reúne con los $M - 2$ bloques de la relación externa. Este cambio reduce el número de exploraciones de la relación interna de b_r a $\lceil b_r/(M-2) \rceil$, donde b_r es el número de bloques de la relación externa. El coste total es $\lceil b_r/(M-2) \rceil * b_s + b_r$ transferencias de bloques y $2\lceil b_r/(M-2) \rceil$ búsquedas.
- Se puede explorar el bucle interno alternativamente hacia adelante y hacia atrás. Este método de búsqueda ordena las peticiones de bloques de disco de tal manera que los datos restantes en la memoria intermedia de la búsqueda anterior se reutilizan, reduciendo de este modo el número de accesos a disco necesarios.
- Si se dispone de un índice en un atributo de la reunión del bucle interno se pueden sustituir las búsquedas en archivos por búsquedas más eficientes en el índice. Esta optimización se describe en el Apartado 13.5.3.

13.5.3 Reunión en bucle anidado indexada

En una reunión en bucle anidado (Figura 13.4), si se dispone de un índice sobre el atributo de la reunión del bucle interno, se pueden sustituir las exploraciones de archivo por búsquedas en el índice. Para cada

www.detodoprogramacion.com

tupla t_r de la relación externa *r*, se utiliza el índice para buscar tuplas en s que cumplan la condición de reunión con la tupla t_r.

Este método de reunión se denomina **reunión en bucle anidado indexada**; se puede utilizar cuando existen índices y cuando se crean índices temporales con el único propósito de evaluar la reunión.

La búsqueda de tuplas en s que cumplan las condiciones de la reunión con una tupla dada t_r es esencialmente una selección en s. Por ejemplo, considérese *impositor* $\bowtie$ *cliente*. Supóngase que se tiene una tupla de *impositor* con *nombre_cliente* "Martín". Entonces, las tuplas relevantes de s son aquellas que satisfacen la selección "*nombre_cliente* $=$ Martín."

El coste de la reunión en bucle anidado indexada se puede calcular como se indica a continuación. Para cada tupla en la relación externa r se realiza una búsqueda en el índice para s recuperando las tuplas apropiadas. En el peor de los casos sólo hay espacio en la memoria intermedia para una página de r y una página del índice. Por tanto, son necesarios b_r operaciones de E/S para leer la relación r, donde b_r denota el número de bloques que contienen registros de r; cada E/S requiere una búsqueda y una transferencia de bloque ya que la cabeza del disco se puede haber trasladado entre cada E/S. Para cada tupla de r, se realiza una búsqueda en el índice para s. Por tanto, el coste temporal de la reunión se puede calcular como $b_r(t_T + t_S) + n_r * c$, donde n_r es el número de registros de la relación r y c es el coste de una única selección en s utilizando la condición de la reunión. Ya se vio en el Apartado 13.3 cómo estimar el coste del algoritmo de una única selección (posiblemente utilizando índices) cuyo cálculo proporciona el valor de c.

La fórmula del coste indica que, si hay índices disponibles en ambas relaciones r y s, normalmente es más eficiente usar como relación más externa aquélla que tenga menos tuplas.

Por ejemplo, considérese una reunión en bucle anidado indexada de *impositor* $\bowtie$ *cliente*, con *impositor* como relación externa. Supóngase también que *cliente* tiene un índice de árbol B^+ primario en el atributo de la reunión *nombre_cliente*, que contiene 20 entradas en promedio por cada nodo del índice. Dado que *cliente* tiene 10.000 tuplas, la altura del árbol es 4, y será necesario un acceso más para encontrar el dato real. Como $n_{impositor}$ es 5.000, el coste total es $100 + 5000 * 5 = 25.100$ accesos a disco, cada uno de los cuales requiere una búsqueda y una transferencia de bloque. En cambio, como se indicó anteriormente, se necesitan 40.100 transferencias de bloque más 200 búsquedas para una reunión en bucle anidado por bloques. Aunque se ha reducido el número de transferencias de bloques, el coste de búsqueda se ha incrementado, aumentando el coste total dado que una búsqueda es considerablemente más costosa que una transferencia de bloque. Sin embargo, si se tuviese una selección sobre la relación *impositor* que redujera significativamente el número de filas, la reunión en bucle anidado sería significativamente más rápida que una reunión en bucle anidado por bloques.

13.5.4 Reunión por mezcla

El algoritmo de **reunión por mezcla** (también llamado algoritmo de **reunión por ordenación-mezcla**) se puede utilizar para calcular reuniones naturales y equirreuniones. Sean $r(R)$ y $s(S)$ las relaciones que vamos a utilizar para realizar la reunión natural, y sean $R \cap S$ sus atributos en común. Supóngase que ambas relaciones están ordenadas según los atributos de $R \cap S$. Por tanto, su reunión se puede calcular mediante un proceso muy parecido a la etapa de mezcla del algoritmo de ordenación–mezcla.

El algoritmo de reunión por mezcla se muestra en la Figura 13.6. En este algoritmo, *AtribsReunión* se refiere a los atributos de $R \cap S$ y, a su vez, $t_r \bowtie t_s$, donde t_r y t_s son las tuplas que tienen los mismos valores en *AtribsReunión*, y denota la concatenación de los atributos de las tuplas, seguido de una proyección para no incluir los atributos repetidos. El algoritmo de reunión por mezcla asocia un puntero con cada relación. Al comienzo, estos punteros apuntan a la primera tupla de sus respectivas relaciones. Según avanza el algoritmo, el puntero se mueve a través de la relación. De este modo se leen en S_s un grupo de tuplas de una relación con el mismo valor en los atributos de la reunión. El algoritmo de la Figura 13.6 *necesita* que cada conjunto de tuplas S_s quepa en memoria principal; más adelante, en este apartado, se examinarán extensiones del algoritmo que evitan este supuesto. Después, las tuplas correspondientes de la otra relación (si las hay) se leen y se procesan según se están leyendo.

En la Figura 13.7 se muestran dos relaciones que están ordenadas en su atributo de la reunión $a1$. Es instructivo examinar los pasos del algoritmo de reunión por mezcla con las relaciones que se muestran en la figura.

www.detodoprogramacion.com

Dado que las relaciones están ordenadas, las tuplas con el mismo valor en los atributos de la reunión aparecerán consecutivamente. De este modo solamente es necesario leer ordenadamente cada tupla una vez y, como resultado, leer también cada bloque solamente una vez. Puesto que sólo se recorre un ciclo en ambos archivos, el método de reunión por mezcla resulta eficiente; el número de transferencias de bloques es igual a la suma de los bloques en los dos archivos, $b_r + b_s$.

Asumiendo que se asignen b_b bloques de memoria intermedia para cada relación, el número de búsquedas requeridas sería $\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil$. Puesto que las búsquedas son mucho más costosas que las transferencias de bloques, tiene sentido asignar varios bloques de la memoria intermedia para cada relación, siempre que haya memoria disponible. Por ejemplo con $t_T = 0.1$ milisegundos por bloque de 4 megabytes y $t_S = 4$ milisegundos, el tamaño de la memoria intermedia es de 400 bloques (o $1,6$ megabytes), el tiempo de búsqueda sería 4 milisegundos por cada 40 milisegundos de tiempo de transferencia, en otras palabras, el tiempo de búsqueda sería sólo el 10 por ciento del tiempo de transferencia.

Si alguna de las relaciones de entrada r o s no está ordenada según los atributos de la reunión, se pueden ordenar primero y luego utilizar el algoritmo de reunión por mezcla. El algoritmo de reunión por mezcla también se puede extender fácilmente desde las reuniones naturales al caso más general de las equirreuniones.

```
pr := dirección de la primera tupla de r;
ps := dirección de la primera tupla de s;
while (ps ≠ null and pr ≠ null) do
    begin
        t_s := tupla a la que apunta ps;
        S_s := {t_s};
        hacer que ps apunte a la siguiente tupla de s;
        hecho := falso;
        while (not hecho and ps ≠ null) do
            begin
                t_s' := tupla a la que apunta ps;
                if (t_s'[AtribsReunión] = t_s[AtribsReunión])
                    then begin
                            S_s := S_s ∪ {t_s'};
                            hacer que ps apunte a la siguiente tupla de s;
                         end
                    else hecho := cierto;
            end
        t_r := tupla a la que apunta pr;
        while (pr ≠ null and t_r[AtribsReunión] < t_s[AtribsReunión]) do
            begin
                hacer que pr apunte a la siguiente tupla de r;
                t_r := tupla a la que apunta pr;
            end
        while (pr ≠ null and t_r[AtribsReunión] = t_s[AtribsReunión]) do
            begin
                for each t_s in S_s do
                    begin
                        añadir t_s ⋈ t_r al resultado;
                    end
                hacer que pr apunte a la siguiente tupla de r;
                t_r := tupla a la que apunta pr;
            end
    end.
```

Figura 13.6 Reunión por mezcla.

www.detodoprogramacion.com

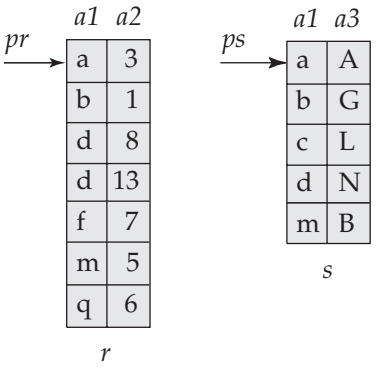


Figura 13.7 Relaciones ordenadas para la reunión por mezcla.

Como se mencionó anteriormente, el algoritmo de reunión por mezcla de la Figura 13.6 requiere que el conjunto S_s de todas las tuplas con el mismo valor de los atributos de la reunión deben caber en memoria principal. Este requisito se suele cumplir, incluso cuando la relación s es grande. Si no se puede cumplir, se debería realizar una reunión en bucle anidado por bloques entre S_s y las tuplas de r con los mismos valores de los atributos de reunión. Por tanto, se incrementa el coste total de la reunión por mezcla.

También es posible realizar una variación de la operación reunión por mezcla en tuplas desordenadas si existen índices secundarios en los dos atributos de la reunión. Así, se examinan los registros a través de los índices recuperándolos de manera ordenada. Sin embargo, esta variación presenta un importante inconveniente, puesto que los registros podrían estar diseminados a través de los bloques del archivo. Por tanto, cada acceso a una tupla podría implicar acceder a un bloque del disco, y esto es muy costoso.

Para evitar este coste se podría utilizar una técnica de reunión por mezcla híbrida que combinase índices con reunión por mezcla. Supóngase que una de las relaciones se encuentra ordenada y la otra desordenada, pero con un índice secundario de árbol B^+ en los atributos de la reunión. El **algoritmo híbrido de reunión por mezcla** fusiona la relación ordenada con las entradas hoja del índice secundario de árbol B^+. El archivo resultante contiene tuplas de la relación ordenada y direcciones de las tuplas de la relación desordenada. Después se ordena el archivo resultante según las direcciones de las tuplas de la relación desordenada, permitiendo así una recuperación eficiente de las correspondientes tuplas, según el orden físico de almacenamiento, para completar la reunión. Se deja como ejercicio la extensión de esta técnica para tratar el caso de dos relaciones desordenadas.

Supóngase que se aplica el esquema de reunión por mezcla al ejemplo de *impositor* $\bowtie$ *cliente*. En este caso, el atributo de la reunión es *nombre_cliente*. Supóngase además que las dos relaciones se encuentren ya ordenadas según el atributo de la reunión *nombre_cliente*. En este caso, la reunión por mezcla emplea un total de $400 + 100 = 500$ transferencias de bloques. Si se asume que en el peor caso sólo se asigna un bloque de memoria intermedia para cada relación de entrada (es decir, $b_b = 1$), se necesitarían un total de $400 + 100 = 500$ búsquedas; en realidad se puede hacer que b_b sea mucho mayor ya que se necesitan bloques de la memoria intermedia sólo para dos relaciones, y el coste de búsqueda sería significativamente menor.

Supóngase que las relaciones no están ordenadas y que el tamaño de memoria en el peor caso es de sólo tres bloques. El coste en este caso se calcularía:

1. Usando la fórmula desarrollada en el Apartado 13.4, la ordenación de la relación *cliente* hacen falta $\lceil \log_{3-1}(400/3) \rceil = 8$ pasos de mezcla. La relación de esta relación necesitaría $400 * (2\lceil \log_{3-1}(400/3) \rceil + 1)$, o 6.800 transferencias de bloques, con otras 400 transferencias para escribir el resultado. El número de búsquedas necesarias es $2*\lceil 400/3 \rceil + 400*(2*8-1)$, 6.268 búsquedas para ordenar y 400 búsquedas para escribir el resultado, con un total de 6.668 búsquedas dado que sólo hay disponible un bloque de memoria intermedia para cada ciclo.

2. Análogamente, la ordenación de *impositor* necesita $\lceil \log_{3-1}(100/3) \rceil = 6$ pasos de mezcla y $100 * (2\lceil \log_{3-1}(100/3) \rceil + 1)$, 1.300 transferencias de bloques, con otras 100 transferencias para escribir

www.detodoprogramacion.com

el resultado. El número de búsquedas necesarias es $2*\lceil 100/3\rceil + 100*(2*6-1) = 1.164$ búsquedas para ordenar, y 100 búsquedas para escribir el resultado, con un total de 1.264 búsquedas.

3. Finalmente, la mezcla de ambas relaciones necesita $400 + 100 = 500$ transferencias de bloques y 500 búsquedas.

Por tanto, el coste total es de 9.100 transferencias de bloques más 8.932 búsquedas si las relaciones no están ordenadas y el tamaño de memoria es sólo de 3 bloques.

Con un tamaño de memoria de 25 bloques y sin tener ordenadas las relaciones, el coste de la ordenación seguida de la reunión por mezcla sería:

1. La ordenación de *cliente* se puede hacer en un único paso de mezcla y requiere un total de $400 * (2\lceil \log_{24}(400/25)\rceil + 1)$ = 1.200 transferencias de bloques. De forma similar, la ordenación de *impositor* emplea 300 transferencias de bloques. Escribir la salida ordenada a disco requiere $400 + 100 = 500$ transferencias de bloques, y el paso de mezcla necesita 500 transferencias de bloques para volver a leer los datos. Al sumar estos costes se calcula un total de 2.500 transferencias de bloques.

2. Si se asume que sólo se asigna un bloque de memoria intermedia para cada secuencia, el número de búsquedas requeridas en este caso es $2 * \lceil 400/25\rceil + 400 + 400 = 832$ búsquedas para ordenar *cliente* y escribir la salida ordenada a disco y, análogamente, $2 * \lceil 100/25\rceil + 100 + 100 = 208$ para *impositor*, más $400 + 100$ búsquedas para la lectura de los datos ordenados en el paso de reunión por mezcla. Al sumar estos costes se calcula un total de 1.640 búsquedas.

   Se puede reducir significativamente el número de búsquedas asignando más bloques de memoria intermedia en cada ciclo. Por ejemplo, si se asignan cinco bloques y para la salida de la de la mezcla de las cuatro secuencias de *impositor*, el coste se reduce de 208 a $2*\lceil 100/25\rceil + \lceil 100/5\rceil + \lceil 100/5\rceil = 48$ búsquedas. Si en el paso de reunión por mezcla se asignan 12 bloques para ambas relaciones, el número de búsquedas para este paso se reduce de 500 a $\lceil 400/12\rceil + \lceil 100/12\rceil = 43$ búsquedas. Por tanto, el número total de búsquedas es 251.

Por tanto, el coste total es de 2.500 transferencias de bloques más 251 búsquedas si las relaciones no están ordenadas y el tamaño de memoria es de 25 bloques.

13.5.5 Reunión por asociación

Al igual que el algoritmo de reunión por mezcla, el algoritmo de reunión por asociación se puede utilizar para implementar reuniones naturales y equirreuniones. En el algoritmo de reunión por asociación se utiliza una función de asociación *h* para dividir las tuplas de ambas relaciones. La idea fundamental es dividir las tuplas de cada relación en conjuntos con el mismo valor de la función de asociación en los atributos de la reunión.

Se da por supuesto que

- *h* es una función de asociación que asigna a los *AtribsReunión* los valores $\{0, 1, \ldots, n_h\}$, donde los *AtribsReunión* denotan los atributos comunes de r y s utilizados en la reunión natural.
- H_{r_0}, $H_{r_1}, \ldots, H_{r_{n_h}}$ denotan las particiones de las tuplas de *r*, inicialmente todas vacías. Cada tupla $t_r \in r$ se pone en la partición H_{r_i}, donde $i = h(t_r[\textit{AtribsReunión}])$.
- H_{s_0}, H_{s_1}, ..., $H_{s_{n_h}}$ denotan las particiones de las tuplas de *s*, inicialmente todas vacías. Cada tupla $t_s \in s$ se pone en la partición H_{s_i}, donde $i = h(t_s[\textit{AtribsReunión}])$.

La función de asociación *h* debería de tener las "buenas" propiedades de aleatoriedad y uniformidad que se estudiaron en el Capítulo 12. La división de las relaciones se muestra de manera gráfica en la Figura 13.8.

www.detodoprogramacion.com

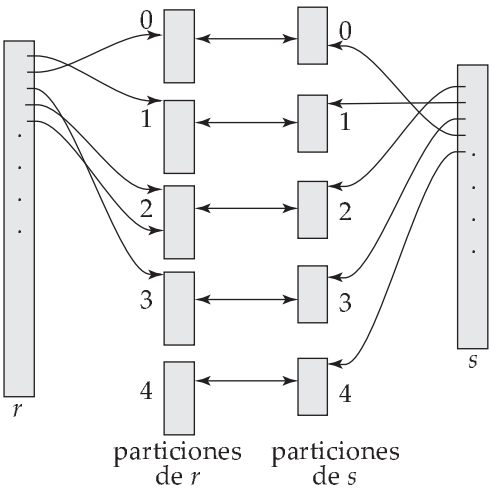


Figura 13.8 División por asociación de relaciones.

13.5.5.1 Fundamentos

La idea en que se basa el algoritmo de reunión por asociación es la siguiente. Supóngase que una tupla de r y una tupla de s satisfacen la condición de la reunión; por tanto, tendrán el mismo valor en los atributos de la reunión. Si el valor se asocia con algún valor i, la tupla de r tiene que estar en H_{r_i} y la tupla de s en H_{s_i}. De este modo solamente es necesario comparar las tuplas de r en H_{r_i} con las tuplas de s en H_{s_i}; no es necesario compararlas con las tuplas de s de ninguna otra partición.

Por ejemplo, si *i* es una tupla de *impositor*, *c* una tupla de *cliente* y h una función de asociación en los atributos *nombre_cliente* de las tuplas, solamente hay que comprobar *i* y *c* si $h(c) = h(i)$. Si $h(c) \neq h(d)$, entonces *c* e *i* poseen valores distintos de *nombre_cliente*. Sin embargo, si $h(c) = h(i)$ hay que comprobar que *c* e *i* tengan los mismos valores en los atributos de la reunión, ya que es posible que *c* e *i* tengan valores diferentes de *nombre_cliente*, con el mismo valor de la función de asociación.

En la Figura 13.9 se muestran los detalles del algoritmo de **reunión por asociación** para calcular la reunión natural de las relaciones r y s. Como en el algoritmo de reunión por mezcla, $t_r \bowtie t_s$ denota la concatenación de los atributos de las tuplas de t_r y t_s, seguido de la proyección para eliminar los atributos repetidos. Después de la división de las relaciones, el resto del código de reunión por asociación realiza una reunión en bucle anidado indexada separada en cada una de los partición pares i, con $i = 0, \ldots, n_h$. Para lograr esto, primero se **construye** un índice asociativo en cada H_{s_i} y luego se **prueba** (es decir, se busca en H_{s_i}) con las tuplas de H_{r_i}. La relación s es la **entrada de construcción** y r es la **entrada de prueba**.

El índice asociativo en H_{s_i} se crea en la memoria, así que no es necesario acceder al disco para recuperar las tuplas. La función de asociación utilizada para construir este índice asociativo debe ser distinta de la función de asociación h utilizada anteriormente, pero aún se aplica exclusivamente a los atributos de la reunión. A lo largo de la reunión en bucle anidado indexada, el sistema utiliza este índice asociativo para recuperar los registros que concuerden con los registro de la entrada de prueba.

Las etapas de construcción y prueba necesitan solamente un único ciclo a través de las entradas de construcción y prueba. La extensión del algoritmo de reunión por asociación para calcular equirreuniones más generales es directa.

Se tiene que elegir el valor de n_h lo bastante grande como para que, para cada i, las tuplas de la partición H_{s_i} de la relación de construcción, junto con el índice asociativo de la partición, quepan en memoria. Pero no tienen por qué caber en memoria las particiones de la relación que se prueban. Sería obviamente mejor utilizar la relación de entrada más pequeña como la relación de construcción. Si el tamaño de la relación de construcción es de b_s bloques, entonces para que cada una de las n_h particiones tengan un tamaño menor o igual que M, n_h debe ser al menos $\lceil b_s/M \rceil$. Con más exactitud, hay que tener en cuenta también el espacio adicional ocupado por el índice asociativo de la partición, incrementando

www.detodoprogramacion.com

```
/* División de s */
for each tupla t_s in s do begin
   i := h(t_s[AtribsReunión]);
   H_{s_i} := H_{s_i} ∪ {t_s};
end
/* División de r */
for each tupla t_r in r do begin
   i := h(t_r[AtribsReunión]);
   H_{r_i} := H_{r_i} ∪ {t_r};
end
/* Realizar la reunión de cada partición */
for i := 0 to n_h do begin
   leer H_{s_i} y construir un índice asociativo en memoria en él
   for each tupla t_r in H_{r_i} do begin
      explorar el índice asociativo en H_{s_i} para localizar todas las tuplas t_s
         tales que t_s[AtribsReunión] = t_r[AtribsReunión]
      for each tupla t_s que concuerde in H_{s_i} do begin
         añadir t_r ⋈ t_s al resultado
      end
   end
end
```

Figura 13.9 Reunión por asociación.

n_h según corresponda. Por simplificar, en estos análisis se ignoran muchas veces los requisitos de espacio del índice asociativo.

13.5.5.2 División recursiva

Si el valor de n_h es mayor o igual que el número de marcos de página de la memoria, la división de las relaciones no se puede hacer en un solo ciclo, puesto que no habría suficientes páginas para memorias intermedias. En lugar de eso, la división se tiene que hacer mediante la repetición de ciclos. En un ciclo se puede dividir la entrada en tantas particiones como marcos de página haya disponibles para utilizarlos como memorias intermedias de salida. Cada cajón generado por un ciclo se lee de manera separada y se divide de nuevo en el siguiente ciclo para crear particiones más pequeñas. La función de asociación utilizada en un ciclo es, por supuesto, diferente de la que se ha utilizado en el ciclo anterior. Se repite esta división de la entrada hasta que cada partición de la entrada de construcción quepa en memoria. Esta división se denomina **división recursiva**.

Una relación no necesita de la división recursiva si $M > n_h + 1$ o, lo que es equivalente, $M > (b_s/M) + 1$, lo cual se simplifica (de manera aproximada) a $M > \sqrt{b_s}$. Por ejemplo, considerando un tamaño de la memoria de 12 megabytes, dividido en bloques de 4 megabytes, la relación contendría un total de 3K (3.072) bloques. Se puede utilizar una memoria de este tamaño para dividir relaciones de 3K $*$ 3K bloques, que son 36 gigabytes. Del mismo modo, una relación del tamaño de 1 gigabyte necesita $\sqrt{256\text{K}}$ bloques, o 2 megabytes, para evitar la división recursiva.

13.5.5.3 Gestión de desbordamientos

Se produce el **desbordamiento de una tabla de asociación** en la partición i de la relación de construcción s, si el índice asociativo de H_{s_i} es mayor que la memoria principal. El desbordamiento de la tabla de asociación puede ocurrir si existen muchas tuplas en la relación de construcción con los mismos valores en los atributos de la reunión, o si la función de asociación no tiene las propiedades de aleatoriedad y uniformidad. En cualquier caso, algunas de las particiones tendrán más tuplas que la media, mientras que otras tendrán menos; se dice entonces que la división está **sesgada**.

Se puede controlar parcialmente el sesgo mediante el incremento del número de particiones, de tal manera que el tamaño esperado de cada partición (incluído el índice asociativo en la partición) sea algo

www.detodoprogramacion.com

menor que el tamaño de la memoria. Por consiguiente, el número de particiones se incrementa en un pequeño valor llamado **factor de escape**, que normalmente es del orden del 20 por ciento del número de particiones calculadas, como se describe en el Apartado 13.5.5.

Aunque se sea conservador con los tamaños de las particiones utilizando un factor de escape, todavía pueden ocurrir desbordamientos. Los desbordamientos de la tabla de asociación se pueden tratar mediante *resolución del desbordamiento* o *evitación del desbordamiento*. La **resolución del desbordamiento** se realiza como sigue. Si H_{s_i}, para cualquier i, resulta ser demasiado grande, se divide de nuevo en particiones más pequeñas utilizando una función de asociación distinta. Del mismo modo, también se divide H_{r_i} utilizando la nueva función de asociación, y solamente es necesario reunir las tuplas en las particiones concordantes.

En cambio, la **evitación del desbordamiento** realiza la división más cuidadosamente, de tal manera que el desbordamiento nunca se produce en la fase de construcción. Para evitar el desbordamiento se implementa la división de la relación de construcción s en muchas particiones pequeñas inicialmente, para luego combinar algunas de estas particiones de tal manera que cada partición combinada quepa en memoria. Además, la relación de prueba r se tiene que combinar de la misma manera que se combinan las particiones de s, sin importar los tamaños de H_{r_i}.

Las técnicas de resolución y evitación podrían fallar en algunas particiones, si un gran número de las tuplas en s tuvieran el mismo valor en los atributos de la reunión. En ese caso, en lugar de crear un índice asociativo en memoria y utilizar una reunión en bucle anidado para reunir las particiones, se pueden utilizar otras técnicas de reunión, tales como la reunión en bucle anidado por bloques, en esas particiones.

13.5.5.4 Coste de la reunión por asociación

Se considera ahora el coste de una reunión por asociación. El análisis supone que no hay desbordamiento de la tabla de asociación. Primero se considera el caso en el que no es necesaria una división recursiva.

- La división de las dos relaciones r y s reclama una lectura completa de ambas así como su posterior escritura. Este operación necesita $2(b_r + b_s)$ transferencias de bloques, donde b_r y b_s denotan respectivamente el número de bloques que contienen registros de las relaciones r y s. Las fases de construcción y prueba leen cada una de las particiones una vez, empleando $b_r + b_s$ transferencias adicionales. El número de bloques ocupados por las particiones podría ser ligeramente mayor que $b_r + b_s$ debido a que los bloques están parcialmente ocupados. El acceso a estos bloques llenos en parte puede añadir un gasto adicional de $2n_h$ a lo sumo, ya que cada una de las n_h particiones podría tener un bloque lleno parcialmente que haya que escribir y leer de nuevo. Así, el coste estimado para una reunión por asociación es

  $$3(b_r + b_s) + 4n_h$$

  transferencias de bloques. La sobrecarga $4n_h$ es muy pequeña comparada con $b_r + b_s$ y se puede ignorar.

- Si se da por supuesto que se asignan b_b bloques para las memorias intermedias de entrada y salida, la división necesita $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)$ búsquedas. Las fases de construcción y prueba requieren sólo una búsqueda para cada una de las n_h particiones de cada relación, ya que cada partición se puede leer secuencialmente. Por tanto, la reunión por asociación necesita $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil) + 2n_h$ búsquedas.

Considérese ahora el caso en el que es necesario realizar la división recursiva. Cada ciclo reduce el tamaño de cada una de las particiones por un factor esperado de $M - 1$; y los ciclos se repiten hasta que cada partición tenga como mucho un tamaño de M bloques. Por tanto, el número esperado de ciclos necesarios para dividir s es $\lceil \log_{M-1}(b_s) - 1 \rceil$.

- Como todos los bloques de s se leen y se escriben en cada ciclo, el número total de transferencias de bloques para dividir s es $2b_s\lceil \log_{M-1}(b_s) - 1 \rceil$. Como el número de ciclos para dividir r es el

www.detodoprogramacion.com

mismo que el número de ciclos para dividir s, por tanto la estimación del coste de la reunión es

$$2(b_r + b_s)\lceil \log_{M-1}(b_s) - 1 \rceil + b_r + b_s$$

transferencias de bloques.

- De nuevo, dando por supuesto que se asignen b_b bloques en memoria intermedia para cada partición, e ignorando el número relativamente pequeño de búsquedas durante las fases de construcción y prueba, la reunión por asociación con división recursiva necesita

$$2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)\lceil \log_{M-1}(b_s) - 1 \rceil$$

búsquedas.

Considérese, por ejemplo, la reunión *cliente* $\bowtie$ *impositor*. Con un tamaño de memoria de 20 bloques, se puede dividir *impositor* en cinco partes, cada una de 20 bloques, con un tamaño tal que caben en memoria. Así, sólo es necesario un ciclo para la división. De la misma manera la relación *cliente* se divide en cinco particiones, cada una de 80 bloques. Si se ignora el coste de escribir los bloque parcialmente llenos, el coste es $3(100 + 400) = 1.500$ transferencias de bloques. Hay suficiente memoria para asignar tres bloques a la entrada y cada uno de los cinco bloques de salida durante la división, lo que resulta en $2(\lceil 100/3 \rceil + \lceil 400/3 \rceil) = 336$ búsquedas.

Es posible mejorar la reunión por asociación si el tamaño de la memoria principal es grande. Cuando la entrada de construcción se puede guardar por completo en memoria principal hay que establecer n_h a 0; de este modo el algoritmo de reunión por asociación se ejecuta rápidamente, sin dividir las relaciones en archivos temporales y sin importar el tamaño de la entrada de prueba. El coste estimado desciende a $b_r + b_s$ transferencias de bloques y dos búsquedas.

13.5.5.5 Reunión por asociación híbrida

El algoritmo de **reunión por asociación híbrida** realiza otra optimización; es útil cuando los tamaños de la memoria son relativamente grandes pero no cabe toda la relación de construcción en memoria. La fase de división del algoritmo de reunión por asociación necesita un bloque de memoria como memoria intermedia para cada partición que se cree, más un bloque de memoria como memoria intermedia de entrada. Por tanto, se necesitan $n_h + 1$ bloques de memoria para dividir las dos relaciones. Si la memoria es mayor que $n_h + 1$, se puede utilizar el resto de la memoria ($M - n_h - 1$ bloques) para guardar en memorias intermedias la primera partición de la entrada de construcción (esto es, H_{s_0}), así que no es necesario escribirla ni leerla de nuevo. Más aún, la función de asociación se diseña de tal manera que el índice asociativo en H_{s_0} quepa en $M - n_h - 1$ bloques, así que, al final de la división de s, H_{s_0} está en memoria por completo y se puede construir un índice asociativo en H_{s_0}.

Cuando r se divide de nuevo, las tuplas en H_{r_0} no se escriben en disco; en su lugar, según se van generando, el sistema las utiliza para examinar el índice asociativo residente en memoria de H_{s_0} y para generar las tuplas de salida de la reunión. Después de utilizarlas para la prueba, se descartan las tuplas, así que la partición H_{r_0} no ocupa ningún espacio en memoria. De este modo se ahorra un acceso de lectura y otro de escritura para cada bloque de H_{r_0} y H_{s_0}. Las tuplas en otras particiones se escriben de la manera usual para reunirlas más tarde. El ahorro de la reunión por asociación híbrida puede ser importante si la entrada de construcción es ligeramente mayor que la memoria.

Si el tamaño de la relación de construcción es b_s, n_h es aproximadamente igual a b_s/M. Así, la reunión por asociación híbrida es más útil si $M >> b_s/M$ o si $M >> \sqrt{b_s}$, donde la notación $>>$ significa *mucho mayor que*. Por ejemplo, supóngase que el tamaño del bloque es de 4 kilobytes y que el tamaño de la relación de construcción es de 1 gigabyte. Entonces, el algoritmo híbrido de reunión por asociación es útil si el tamaño de la memoria es claramente mayor que 2 megabytes; las memorias de 100 megabytes o más son comunes en las computadoras de hoy en día.

Considérese de nuevo la reunión *cliente* $\bowtie$ *impositor*. Con una memoria de 25 bloques de tamaño, se puede dividir *impositor* en cinco particiones, cada una de 20 bloques, y con la primera de las particiones de la relación de construcción almacenada en memoria. Ocupa 20 bloques de memoria; un bloque se utiliza para la entrada y cuatro bloques más para guardar en memorias intermedias cada partición. De manera similar se divide la relación *cliente* en cinco particiones cada una de tamaño 80, la primera de

www.detodoprogramacion.com

las cuales de utiliza precisamente para comprobar, en lugar de escribirse y leerse de nuevo. Ignorando el coste de escribir los bloques parcialmente llenos, el coste es de $3(80 + 320) + 20 + 80 = 1.300$ bloques transferidos, en lugar de las 1.500 transferencias de bloques sin la optimización por asociación híbrida. Sin embargo, en este caso el tamaño de la memoria intermedia para la entrada y para cada partición escrita a disco desciende a un bloque, aumentando el número de búsquedas a alrededor del número de accesos a bloques, y aumentando también el coste total. Por tanto, en este caso es mejor usar una reunión por asociación con una memoria intermedia mayor de b_b en lugar de usar la reunión por asociación híbrida. Sin embargo, con tamaños mucho mayores de memoria, el aumento de b_b más allá de un punto determinado disminuye el rendimiento, y la memoria restante se puede usar para implementar la reunión por asociación híbrida.

13.5.6 Reuniones complejas

Las reuniones en bucle anidado y en bucle anidado por bloques son útiles sean cuales sean las condiciones de la reunión. Las otras técnicas de reunión son más eficientes que las reuniones en bucle anidado y sus variantes, aunque sólo se pueden utilizar con condiciones simples, tales como las reuniones naturales o las equirreuniones. Se pueden implementar reuniones con condiciones de la reunión más complejas, tales como conjunciones y disyunciones, utilizando las técnicas eficientes de la reunión mediante la aplicación de las técnicas desarrolladas en el Apartado 13.3.4 para el manejo de selecciones complejas.

Considérese la siguiente reunión con una condición conjuntiva:

$$r \bowtie_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n} s$$

Se pueden aplicar una o más de las técnicas de reunión descritas anteriormente en cada condición individual $r \bowtie_{\theta_1} s$, $r \bowtie_{\theta_2} s$, $r \bowtie_{\theta_3} s$, y así sucesivamente. El resultado total de la reunión se determina calculando primero el resultado de una de estas reuniones simples $r \bowtie_{\theta_i} s$; cada par de tuplas del resultado intermedio se compone de una tupla de r y otra de s. El resultado total de la reunión consiste en las tuplas del resultado intermedio que satisfacen el resto de condiciones

$$\theta_1 \wedge \cdots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \cdots \wedge \theta_n$$

Estas condiciones se pueden ir comprobando según se generan las tuplas de $r \bowtie_{\theta_i} s$.

Una reunión cuya condición es una disyunción se puede calcular como se indica a continuación. Considérese

$$r \bowtie_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n} s$$

La reunión se puede calcular como la unión de los registros de las reuniones $r \bowtie_{\theta_i} s$:

$$(r \bowtie_{\theta_1} s) \cup (r \bowtie_{\theta_2} s) \cup \cdots \cup (r \bowtie_{\theta_n} s)$$

Los algoritmos para calcular la unión de relaciones se describen en el Apartado 13.6.

13.6 Otras operaciones

Otras operaciones relacionales y operaciones relacionales extendidas (como eliminación de duplicados, proyección, operaciones sobre conjuntos, reunión externa y agregación) se pueden implementar según se describe en los Apartados 13.6.1 al 13.6.5.

13.6.1 Eliminación de duplicados

Se puede implementar fácilmente la eliminación de duplicados utilizando la ordenación. Las tuplas idénticas aparecerán consecutivas durante la ordenación, pudiéndose eliminar todas las copias menos una. Con la ordenación–mezcla externa, se pueden eliminar los duplicados mientras se crea una secuencia antes de que ésta se escriba en el disco, reduciendo así el número de bloques transferidos. El resto de duplicados se pueden suprimir durante la etapa de reunión/mezcla, así que el resultado final está libre de repeticiones. El coste estimado en el peor de los casos para la eliminación de duplicados es el mismo que el coste estimado en el peor caso para la ordenación de una relación.

Se puede implementar también la eliminación de duplicados utilizando la asociación de una manera similar al algoritmo de reunión por asociación. Primero, se divide la relación basándose en una función

www.detodoprogramacion.com

de asociación en la tupla entera. Luego, se lee cada partición, y se construye un índice asociativo en memoria. Mientras se construye el índice asociativo se inserta una tupla solamente si no estaba ya presente. En otro caso se descarta. Después de que todas las tuplas de la relación se hayan procesado, las tuplas en el índice asociativo se escriben al resultado. El coste estimado es el mismo que el coste de procesar (división y lectura de cada partición) la relación de construcción en una reunión por asociación.

Debido al coste relativamente alto de la eliminación de duplicados, SQL exige una petición explícita del usuario para suprimir los duplicados; en caso contrario, se conservan.

13.6.2 Proyección

La proyección se puede implementar fácilmente realizando la proyección de cada tupla, pudiendo originar una relación con registros repetidos y suprimiendo después los registros duplicados. La eliminación de duplicados se puede llevar a cabo según se ha descrito en el Apartado 13.6.1. Si los atributos de la lista de proyección incluyen una clave de la relación, no se producirán duplicados; por tanto no será necesario eliminarlos. La proyección generalizada (tratada en el Apartado 2.4.1) se puede implementar de la misma manera que las proyecciones.

13.6.3 Operaciones sobre conjuntos

Las operaciones *unión*, *intersección* y *diferencia de conjuntos* se pueden implementar ordenando primero ambas relaciones y examinando después cada relación para producir el resultado. En $r \cup s$, cuando una exploración concurrente en ambas relaciones descubre la misma tupla en los dos archivos, solamente se conserva una de las tuplas. Por otra parte, el resultado de $r \cap s$ contendrá únicamente aquellas tuplas que aparezcan en ambas relaciones. De la misma manera se implementa la *diferencia de conjuntos*, $r - s$, guardando aquellas tuplas de r que estén ausentes de s.

Para todas estas operaciones solamente se necesita una exploración en cada relación de entrada, así el coste es $b_r + b_s$ transferencias de bloques si las relaciones están ordenadas de igual forma. Suponiendo que en el peor caso sólo hay un bloque de memoria intermedia para cada relación, se necesitarían un total de $b_r + b_s$ búsquedas además de las $b_r + b_s$ transferencias de bloques. El número de búsquedas se puede reducir asignando bloques adicionales de memoria intermedia.

Si las relaciones no están ordenadas inicialmente, hay que incluir el coste de la ordenación. Se puede utilizar cualquier otro orden en la evaluación de la operación de conjuntos, siempre que las dos entradas tengan la misma ordenación.

La asociación proporciona otra manera de implementar estas operaciones sobre conjuntos. El primer paso en cada caso es dividir las dos relaciones utilizando la misma función de asociación y de este modo crear las particiones $H_{r_0}, H_{r_1}, \ldots, H_{r_{n_h}}$ y $H_{s_0}, H_{s_1}, \ldots, H_{s_{n_h}}$. En función de cada operación, el sistema da a continuación estos pasos en cada partición $i = 0, 1, \ldots, n_h$:

- $r \cup s$
  1. Construir un índice asociativo en memoria sobre H_{r_i}.
  2. Añadir las tuplas de H_{s_i} al índice asociativo solamente si no estaban ya presentes.
  3. Añadir las tuplas del índice asociativo al resultado.

- $r \cap s$
  1. Construir un índice asociativo en memoria en H_{r_i}.
  2. Para cada tupla en H_{s_i} probar el índice asociativo y pasar la tupla al resultado únicamente si ya estaba presente en el índice.

- $r - s$
  1. Construir un índice asociativo en memoria en H_{r_i}.
  2. Para cada tupla de H_{s_i} probar el índice asociativo y, si la tupla está presente en el índice, suprimirla del índice asociativo.
  3. Añadir las tuplas restantes del índice asociativo al resultado.

www.detodoprogramacion.com

13.6.4 Reunión externa

Recordemos las *operaciones de reunión externa* que se describieron en el Apartado 2.4.3. Por ejemplo, la reunión externa por la izquierda *cliente* $\supset\!\bowtie$ *impositor* contiene la reunión de *cliente* y de *impositor* y además, para cada tupla t de *cliente* que no concuerde con alguna tupla en *impositor* (es decir, donde no esté *nombre_cliente* en *impositor*), se añade la siguiente tupla t_1 al resultado. Para todos los atributos del esquema de *cliente* la tupla t_1 tiene los mismos valores que la tupla t. El resto de los atributos (del esquema de *impositor*) de la tupla t_1 contienen el valor nulo.

Se pueden implementar las operaciones de reunión externa empleando una de las dos estrategias siguientes:

1. Calcular la reunión correspondiente, y luego añadir más tuplas al resultado de la reunión hasta obtener la reunión externa resultado. Considérese la operación de reunión externa por la izquierda y dos relaciones: $r(R)$ y $s(S)$. Para evaluar $r \supset\!\bowtie_\theta s$, se calcula primero $r \bowtie_\theta s$ y se guarda este resultado como relación temporal q_1. A continuación se calcula $r - \Pi_R(q_1)$, que produce las tuplas de r que no participaron en la reunión. Se puede utilizar cualquier algoritmo para calcular las reuniones. Luego se rellenan cada una de estas tuplas con valores nulos en los atributos de s y se añaden a q_1 para obtener el resultado de la reunión externa.

   La operación reunión externa por la derecha $r \bowtie\!\subset_\theta s$ es equivalente a $s \supset\!\bowtie_\theta r$ y, por tanto, se puede implementar de manera simétrica a la reunión externa por la izquierda. También se puede implementar la operación reunión externa completa $r \supset\!\bowtie\!\subset_\theta s$ calculando la reunión $r \bowtie s$ y añadiendo luego las tuplas adicionales de las operaciones reunión externa por la izquierda y por la derecha, al igual que antes.

2. Modificar los algoritmos de la reunión. Así, es fácil extender los algoritmos de reunión en bucle anidado para calcular la reunión externa por la izquierda. Las tuplas de la relación externa que no concuerdan con ninguna tupla de la relación interna se escriben en la salida después de haber sido completadas con valores nulos. Sin embargo, es difícil extender la reunión en bucle anidado para calcular la reunión externa completa.

   Las reuniones externas naturales y las reuniones externas con una condición de equirreunión se pueden calcular mediante extensiones de los algoritmos de reunión por mezcla y reunión por asociación. La reunión por mezcla se puede extender para realizar la reunión externa completa como se muestra a continuación. Cuando se está produciendo la mezcla de las dos relaciones, las tuplas de la relación que no encajan con ninguna tupla de la otra relación se pueden completar con valores nulos y escribirse en la salida. Del mismo modo se puede extender la reunión por mezcla para calcular las reuniones externas por la izquierda y por la derecha mediante la copia de las tuplas que no concuerden (rellenadas con valores nulos) desde solamente una de las relaciones. Puesto que las relaciones están ordenadas, es fácil detectar si una tupla coincide o no con alguna de las tuplas de la otra relación. Por ejemplo, cuando se hace una reunión por mezcla de *cliente* e *impositor*, las tuplas se leen según el orden de *nombre_cliente* y es fácil comprobar para cada tupla si hay alguna tupla coincidente.

   El coste estimado para implementar reuniones externas utilizando el algoritmo de reunión por mezcla es el mismo que para la correspondiente reunión. La única diferencia está en el tamaño del resultado y, por tanto, en los bloques transferidos para copiarlos que no se han tenido en cuenta en las estimaciones anteriores del coste.

   La extensión del algoritmo de reunión por asociación para calcular reuniones externas se deja como ejercicio (Ejercicio 13.13).

13.6.5 Agregación

Considérese el operador de agregación $\mathcal{G}$ estudiado en el Apartado 2.4.2. Por ejemplo, la operación:

$$_{nombre_sucursal}\mathcal{G}_{\mathbf{sum}(saldo)}(cuenta)$$

agrupa tuplas de *cuenta* por sucursal y calcula el saldo total de todas las cuentas de cada sucursal.

La operación agregación se puede implementar de una manera parecida a la eliminación de duplicados. Se utiliza la ordenación o la asociación, al igual que se hizo para la supresión de duplicados, pero

www.detodoprogramacion.com

basándose ahora en la agrupación de atributos (*nombre_sucursal* en el ejemplo anterior). Sin embargo, en lugar de eliminar las tuplas con el mismo valor en los atributos de la agrupación, se reúnen en grupos y se aplican las operaciones agregación en cada grupo para obtener el resultado.

El coste estimado para la implementación de la operación agregación es el mismo coste de la eliminación de duplicados para las funciones de agregación como **min**, **max**, **sum**, **count** y **avg**.

En lugar de reunir todas las tuplas en grupos y aplicar entonces las funciones de agregación, se pueden implementar las funciones de agregación **sum**, **min**, **max**, **count** y **avg** sobre la marcha según se construyen los grupos. Para el caso de **sum**, **min** y **max**, cuando se encuentran dos tuplas del mismo grupo el sistema las sustituye por una sola tupla que contenga **sum**, **min** o **max**, respectivamente, de las columnas que se están agregando. Para la operación **count**, se mantiene una cuenta incremental para cada grupo con tuplas descubiertas. Por último, la operación **avg** se implementa calculando la suma y la cuenta de valores sobre la marcha, para dividir finalmente la suma entre la cuenta para obtener la media.

Si todas las tuplas del resultado caben en memoria, las implementaciones basadas en ordenación y las basadas en asociación no necesitan escribir ninguna tupla en disco. Según se leen las tuplas se pueden insertar en un estructura ordenada de árbol o en un índice asociativo. Así, cuando se utilizan las técnicas de agregación sobre la marcha, solamente es necesario almacenar una tupla para cada uno de los grupos. Por tanto, la estructura ordenada de árbol o el índice asociativo caben en memoria y se puede procesar la agregación con sólo b_r transferencias de bloques (y una búsqueda), en lugar de las $3b_r$ transferencias (y en el peor caso de hasta $2b_r$ búsquedas) que se necesitarían en el otro caso.

13.7 Evaluación de expresiones

Hasta aquí se ha estudiado cómo llevar a cabo operaciones relacionales individuales. Ahora se considera cómo evaluar una expresión que contiene varias operaciones. La manera evidente de evaluar una expresión es simplemente evaluar una operación a la vez en un orden apropiado. El resultado de cada evaluación se **materializa** en una relación temporal para su inmediata utilización. Un inconveniente de esta aproximación es la necesidad de construir relaciones temporales, que (a menos que sean pequeñas) se tienen que escribir en disco. Un enfoque alternativo es evaluar varias operaciones de manera simultánea en un **cauce**, con los resultados de una operación pasados a la siguiente, sin la necesidad de almacenar relaciones temporales.

En los Apartados 13.7.1 y 13.7.2 se consideran ambos enfoques, *materialización* y *encauzamiento*. Se verá que el coste de estos enfoques puede diferir substancialmente, pero también que existen casos donde sólo la materialización es posible.

13.7.1 Materialización

Intuitivamente es fácil de entender cómo evaluar una expresión observando una representación gráfica de la expresión en un **árbol de operadores**. Considérese la expresión

$$\Pi_{nombre_cliente}\ (\sigma_{saldo<2500}\ (cuenta) \bowtie\ cliente)$$

que se muestra en la Figura 13.10.

Si se aplica el enfoque de la materialización, se comienza por las operaciones de la expresión de nivel más bajo (al fondo del árbol). En el ejemplo solamente hay una de estas operaciones: la operación selección en *cuenta*. Las entradas de las operaciones de nivel más bajo son las relaciones de la base de datos. Se ejecutan estas operaciones utilizando los algoritmos ya estudiados, almacenando sus resultados en relaciones temporales. Luego se utilizan estas relaciones temporales para ejecutar las operaciones del siguiente nivel en el árbol, cuyas entradas son ahora o bien relaciones temporales o bien relaciones almacenadas en la base de datos. En este ejemplo, las entradas de la reunión son la relación *cliente* y la relación temporal producida por la selección en *cuenta.* Ahora se puede evaluar la reunión creando otra relación temporal.

Repitiendo este proceso se calcularía finalmente la operación en la raíz del árbol, obteniendo el resultado final de la expresión. En el ejemplo se consigue el resultado final mediante la ejecución de la operación proyección de la raíz utilizando como entrada la relación temporal creada por la reunión.

www.detodoprogramacion.com

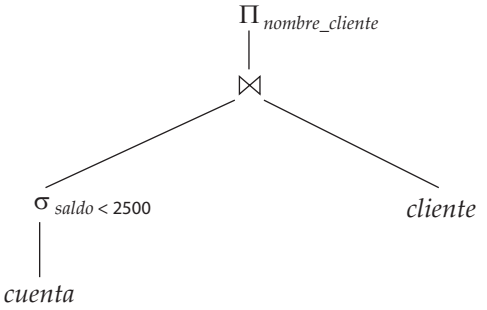


Figura 13.10 Representación gráfica de una expresión.

Una evaluación como la descrita se llama **evaluación materializada**, puesto que los resultados de cada operación intermedia se crean (materializan) para utilizarse a continuación en la evaluación de las operaciones del siguiente nivel.

El coste de una evaluación materializada no es simplemente la suma de los costes de las operaciones involucradas. Cuando se calcularon los costes estimados de los algoritmos se ignoró el coste de escribir el resultado de la operación en disco. Para calcular el coste de evaluar una expresión como la que se ha hecho hay que añadir los costes de todas las operaciones, incluyendo el coste de escribir los resultados intermedios en disco. Supóngase que los registros del resultado se acumulan en una memoria intermedia y que cuando ésta se llena, los registros se escriben en el disco. El número de bloques escritos, b_r, se puede estimar en n_r/f_r, donde n_r es el número aproximado de tuplas de la relación resultado r y f_r es el *factor de bloqueo* de la relación resultado, es decir, el número de registros de r que caben en un bloque. Además del tiempo de transferencia es posible que se necesiten algunas búsquedas, ya que la cabeza del disco se puede haber desplazado entre escrituras sucesivas. Se puede estimar el número de búsquedas como $\lceil b_r/b_b \rceil$, donde b_b es el tamaño en bloques de la memoria intermedia para la salida.

La **memoria intermedia doble** (usando dos memorias intermedias, una donde progresa la ejecución del algoritmo mientras que la otra se está copiando) permite que el algoritmo se ejecute más rápidamente mediante la ejecución en paralelo de acciones en la CPU con acciones de E/S. El número de búsquedas se puede reducir asignando bloques adicionales en la memoria intermedia de salida y escribiendo varios bloques a la vez.

13.7.2 Encauzamiento

Se puede mejorar la eficiencia de la evaluación de consultas mediante la reducción del número de archivos temporales que se producen. Se lleva a cabo esta reducción con la combinación de varias operaciones relacionales en un *encauzamiento* de operaciones, en el que se pasan los resultados de una operación a la siguiente operación del encauzamiento. Esta evaluación, como se ha descrito, se denomina **evaluación encauzada**. La combinación de operaciones en un encauzamiento elimina el coste de leer y escribir relaciones temporales.

Por ejemplo, considérese la reunión de un par de relaciones seguida de una proyección ($\Pi_{a1,a2}(r \bowtie s)$). Si se aplicara la materialización, la evaluación implicaría la creación de una relación temporal para guardar el resultado de la reunión y la posterior lectura del resultado para realizar la proyección. Estas operaciones se pueden combinar como sigue. Cuando la operación reunión genera una tupla del resultado, se pasa inmediatamente esa tupla al operador de proyección para su procesamiento. Mediante la combinación de la reunión y de la proyección, se evita la creación de resultados intermedios, creando en su lugar el resultado final directamente.

13.7.2.1 Implementación del encauzamiento

Se puede implementar el encauzamiento construyendo una única y compleja operación que combine las operaciones que constituyen el encauzamiento. Aunque este enfoque podría ser factible en muchas situaciones, es deseable en general reusar el código en operaciones individuales en la construcción del encauzamiento. Por lo tanto, cada operación del encauzamiento se modela como un proceso aislado o

www.detodoprogramacion.com

una hebra en el sistema, que toma un flujo de tuplas de sus entradas encauzadas y produce un flujo de tuplas como salida. Para cada pareja de operaciones adyacentes en el encauzamiento se crea una memoria intermedia para guardar las tuplas que se envían de una operación a la siguiente.

En el ejemplo de la Figura 13.10, las tres operaciones se pueden situar en un encauzamiento, en el que los resultados de la selección se pasan a la reunión según se generan. Por su parte, los resultados de la reunión se envían a la proyección según se van generando. Así, los requisitos de memoria son bajos, ya que los resultados de una operación no se almacenan por mucho tiempo. Sin embargo, como resultado del encauzamiento, las entradas de las operaciones no están disponibles todas a la vez para su procesamiento.

Los encauzamientos se pueden ejecutar de alguno de los siguientes modos:

1. Bajo demanda.
2. Desde los productores.

En un **encauzamiento bajo demanda**, el sistema reitera peticiones de tuplas desde la operación de la cima del encauzamiento. Cada vez que una operación recibe una petición de tuplas, calcula la siguiente tupla (o tuplas) a devolver y la envía. Si las entradas de la operación no están encauzadas, la(s) siguiente(s) tupla(s) a devolver se calcula(n) de las relaciones de entrada mientras se lleva cuenta de lo que se ha remitido hasta el momento. Si alguna de sus entradas está encauzada, la operación también hace peticiones de tuplas desde sus entradas encauzadas. Así, utilizando las tuplas recibidas en sus entradas encauzadas, la operación calcula sus tuplas de salida y las envía hasta su padre.

En un **encauzamiento por los productores**, los operadores no esperan a que se produzcan peticiones para producir tuplas, en su lugar generan las tuplas **impacientemente**. Cada operación del fondo del encauzamiento genera continuamente tuplas de salida y las ponen en su memoria intermedia de salida hasta que se llena. Una operación en cualquier otro nivel del encauzamiento obtiene sus tuplas de entrada de un nivel inferior del encauzamiento hasta llenar su memoria intermedia de salida. Una vez que la operación ha utilizado una tupla de su entrada encauzada, la elimina de ella. En cualquier caso, una vez que la memoria intermedia de salida esté llena, la operación esperará hasta que su operación padre elimine las tuplas de la memoria intermedia para hacer más espacio a nuevas tuplas. En este momento, la operación genera más tuplas hasta que se llene la memoria intermedia de nuevo. Este proceso se repite por una operación hasta que se hayan generado todas las tuplas de salida.

El sistema necesita cambiar de una operación a otra solamente cuando se llena una memoria intermedia de salida o cuando una memoria intermedia de entrada está vacía y se necesitan más tuplas para generar las tuplas de salida. En un sistema de procesamiento paralelo, las operaciones del encauzamiento se pueden ejecutar concurrentemente en distintos procesadores (véase el Capítulo 21).

Se puede imaginar el encauzamiento por los productores como una **inserción** de datos de abajo hacia arriba en el árbol de operaciones, mientras que el encauzamiento bajo demanda se puede pensar como una **extracción** de datos desde la cima del árbol de operaciones. Mientras que las tuplas se generan *impacientemente* en el encauzamiento por los productores, se generan de forma **perezosa**, bajo demanda, en el encauzamiento bajo demanda.

Cada operación en un encauzamiento bajo demanda se puede implementar como un **iterador**, que proporciona las siguientes funciones: *abrir*(), *siguiente*() y *cerrar*(). Después de una llamada a *abrir*(), cada llamada a *siguiente*() devuelve la siguiente tupla de salida de la operación. La implementación de la operación realiza por turno llamadas a *abrir*() y a *siguiente*() desde sus entradas, cuando necesita tuplas de entrada. La función *cerrar*() comunica al iterador que no necesita más tuplas. De este modo, el iterador mantiene el **estado** de su ejecución entre las llamadas, de tal manera que las sucesivas peticiones *siguiente*() reciban sucesivas tuplas resultado.

Por ejemplo, en un iterador que implemente la operación selección usando la búsqueda lineal, la operación *abrir*() inicia una exploración de archivo y el estado del iterador registra el punto en el que el archivo se ha explorado. Cuando se llama a la función *siguiente*() la exploración del archivo continúa a continuación del punto anterior; cuando se encuentre la siguiente tupla que cumpla la selección al explorar el archivo, se devuelve la tupla después de almacenar el punto donde se encontró en el estado del iterador. Una operación *abrir*() del iterador de reunión por mezcla abriría sus entradas y, si aún no están ordenadas, también las ordenaría. En las llamadas a *siguiente*() se devolvería el siguiente par

www.detodoprogramacion.com

de tuplas coincidentes. La información de estado consistiría en el grado en que se ha explorado cada entrada.

Los detalles de la implementación de iteradores se dejan para completar en el Ejercicio práctico 13.7. El encauzamiento bajo demanda se utiliza normalmente más que el encauzamiento por los productores, ya que es más fácil de implementar.

13.7.2.2 Algoritmos de evaluación para el encauzamiento

Considérese una operación reunión cuya entrada del lado izquierdo está encauzada. Puesto que está encauzada, no toda la entrada está disponible al mismo tiempo para el procesamiento de la operación reunión. Al no estar disponible, se limita la elección del algoritmo de reunión a emplear. Por ejemplo, no se puede usar la reunión por mezcla si las entradas no están ordenadas, puesto que no es posible ordenar la relación hasta que todas las tuplas estén disponibles (así, en efecto, se convierte el encauzamiento en materialización). Sin embargo, se puede utilizar la reunión en bucle anidado indexada: según se reciben las tuplas por el lado izquierdo de la reunión se pueden utilizar para indexar el lado derecho de la relación y para generar las tuplas resultado de la reunión. Este ejemplo ilustra que las elecciones respecto del algoritmo a utilizar para una operación y las elecciones respecto del encauzamiento no son independientes.

Las restricciones en los algoritmos de evaluación que se pueden utilizar es un factor determinante del encauzamiento. Como resultado, a pesar de las aparentes ventajas del encauzamiento, hay casos donde la materialización alcanza un coste total menor. Supóngase que se desea realizar la reunión de r y de s, y que la entrada r está encauzada. Si se utiliza una reunión en bucle anidado indexada para llevar a cabo el encauzamiento, podría ser necesario un acceso a disco para cada tupla de la relación de entrada encauzada. El coste de esta técnica es $n_r * \mathrm{AA}_i * (t_S + t_T)$, donde AA_i es la altura del índice de s. Con la materialización, el coste de copiar r sería $b_r * t_T$. Con una técnica de reunión, como la reunión por asociación, podría ser posible realizar la reunión con un coste aproximado de $3(b_r + b_s)$ transferencias de bloques más $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)$ búsquedas (asumiendo suficiente memoria para evitar la división recursiva). Para n_r grandes, la materialización sería más económica.

El uso eficiente del encauzamiento necesita la utilización de algoritmos de evaluación que puedan generar tuplas de salida según se están recibiendo tuplas por las entradas de la operación. Se pueden distinguir dos casos:

1. Solamente una de las entradas de la reunión está encauzada.
2. Las dos entradas de la reunión están encauzadas.

Si únicamente una de las entradas de la reunión está encauzada, la reunión en bucle anidado indexada es la elección más natural. Si las tuplas de entrada encauzadas están ordenadas según los atributos de la reunión y la condición de la reunión es una equirreunión, también se puede emplear la reunión por mezcla. La reunión por asociación híbrida se puede utilizar con la entrada encauzada como la relación de prueba. Sin embargo, las tuplas que no están en la primera partición se enviarán a la salida solamente después de que la relación de entrada encauzada se reciba por completo. La reunión por asociación híbrida es útil si las entradas no encauzadas caben completamente en memoria, o si al menos la mayoría de las entradas caben en memoria.

Si ambas entradas están encauzadas, la elección de los algoritmos de reunión está más limitada. Si ambas entradas están ordenadas en los atributos de la reunión y la condición de la reunión es una equirreunión, entonces se puede usar la reunión por mezcla. Otra técnica alternativa es la **reunión encauzada**, que se muestra en la Figura 13.11. El algoritmo supone que las tuplas de entrada de ambas relaciones, r y s, están encauzadas. Las tuplas disponibles de ambas relaciones se dejan listas para su procesamiento en una cola simple. Así mismo, se generan unas entradas de la cola especiales, llamadas Fin_r y Fin_s, que sirven como marcas de fin de archivo y que se insertan en la cola después de que se hayan generado todas las tuplas de r y de s (respectivamente). Para una evaluación eficaz se deberían construir los índices apropiados en las relaciones r y s. Según se añaden tuplas a r y a s se deben mantener los índices actualizados.

www.detodoprogramacion.com

```
hecho_r := falso;
hecho_s := falso;
r := ∅;
s := ∅;
resultado := ∅;
while not hecho_r or not hecho_s do
    begin
        if la cola está vacía, then esperar hasta que la cola no esté vacía;
        t := entrada de la cima de la cola;
        if t = Fin_r then hecho_r := cierto
          else if t = Fin_s then hecho_s := cierto
              else if t es de la entrada r
                  then
                      begin
                          r := r ∪ {t};
                          resultado := resultado ∪ ({t} ⋈ s);
                      end
                  else /* t es de la entrada s */
                      begin
                          s := s ∪ {t};
                          resultado := resultado ∪ (r ⋈ {t});
                      end
    end
```

Figura 13.11 Algoritmo de reunión encauzada.

13.8 Resumen

- La primera acción que el sistema debe realizar en una consulta es traducirla en su formato interno, que (para sistemas de bases de datos relacionales) está basado normalmente en el álgebra relacional. En el proceso de generación del formato interno de la consulta, el analizador comprueba la sintaxis, verifica que los nombres de relación que figuran en la consulta son nombres de relaciones de la base de datos, etc. Si la consulta se expresó en términos de una vista, el analizador sustituye todas las referencias al nombre de la vista con su expresión del álgebra relacional para calcularla.

- Dada una consulta, generalmente hay diversos métodos para calcular la respuesta. Es responsabilidad del sistema transformar la consulta proporcionada por el usuario en otra consulta equivalente que se pueda ejecutar de un modo más eficiente. El Capítulo 14 estudia la optimización de consultas.

- Se pueden procesar consultas que impliquen selecciones sencillas mediante una búsqueda lineal, una búsqueda binaria o utilizando índices. Así mismo, se pueden manejar selecciones más complejas mediante uniones e intersecciones de los resultados de selecciones simples.

- Se pueden ordenar relaciones que sean más grandes que la memoria utilizando el algoritmo de ordenación–mezcla externa.

- Las consultas que impliquen una reunión natural se pueden procesar de varias maneras, dependiendo de la disponibilidad de índices y del tipo de almacenamiento físico utilizado para las relaciones.
  - Si la reunión resultante es casi tan grande como el producto cartesiano de las dos relaciones, una estrategia de *reunión en bucle anidado por bloques* podría ser ventajosa.
  - Si hay índices disponibles, se puede utilizar *la reunión en bucle anidado indexada*.

www.detodoprogramacion.com

CAPÍTULO 14

Optimización de consultas

La **optimización de consultas** es el proceso de selección del plan de evaluación de las consultas más eficiente de entre las muchas estrategias generalmente disponibles para el procesamiento de una consulta dada, especialmente si la consulta es compleja. No se espera que los usuarios escriban las consultas de modo que puedan procesarse de manera eficiente. Por el contrario, se espera que el sistema cree un plan de evaluación que minimice el coste de la evaluación de las consultas. Ahí es donde entra en acción la optimización de consultas.

Un aspecto de la optimización de las consultas tiene lugar en el nivel del álgebra relacional, donde el sistema intenta hallar una expresión que sea equivalente a la expresión dada, pero de ejecución más eficiente. Otro aspecto es la elección de una estrategia detallada para el procesamiento de la consulta, como puede ser la selección del algoritmo que se usará para ejecutar una operación, la selección de los índices concretos que se van a emplear, etc.

La diferencia en coste (en términos de tiempo de evaluación) entre una estrategia buena y una mala suele ser sustancial, y puede resultar de varios órdenes de magnitud. Por tanto, merece la pena que el sistema invierta una cantidad importante de tiempo en la selección de una buena estrategia para el procesamiento de la consulta, aunque esa consulta sólo se ejecute una vez.

14.1 Visión general

Considérese la expresión del álgebra relacional para la consulta "Hallar los nombres de todos los clientes que tengan una cuenta en cualquier sucursal ubicada en Arganzuela".

$$\Pi_{nombre_cliente}\ (\sigma_{ciudad_sucursal = \text{"Arganzuela"}}\ (sucursal \bowtie\ (cuenta \bowtie\ impositor)))$$

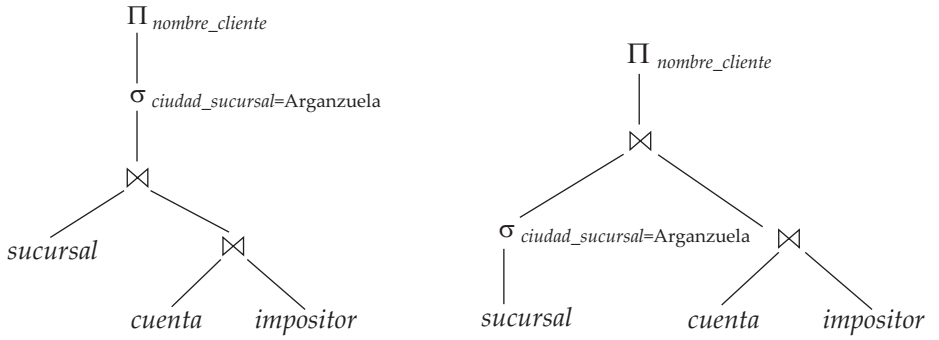
Esta expresión crea una relación intermedia de gran tamaño, *sucursal* $\bowtie$ *cuenta* $\bowtie$ *impositor*. Sin embargo, sólo resultan de interés unas pocas tuplas de esta relación (las correspondientes a las sucursales ubicadas en Arganzuela), y sólo en uno de los seis atributos de la relación. Dado que sólo son relevantes las tuplas de la relación *sucursal* que corresponden a las sucursales ubicadas en Arganzuela, no hace falta considerar las tuplas que no cumplen *ciudad_sucursal* = "Arganzuela." Al reducir el número de tuplas de la relación *sucursal* a las que es necesario tener acceso, se reduce el tamaño del resultado intermedio. La consulta queda ahora representada por la siguiente expresión del álgebra relacional:

$$\Pi_{nombre_cliente}\ (\ (\sigma_{ciudad_sucursal = \text{"Arganzuela"}}\ (sucursal))\ \bowtie\ (cuenta \bowtie\ impositor))$$

la cual es equivalente a la expresión algebraica original, pero genera relaciones intermedias de menor tamaño. La Figura 14.1 muestra la expresión inicial y la transformada.

Dada una expresión del álgebra relacional, es labor del optimizador de consultas diseñar un plan de evaluación para ellas que calcule el mismo resultado que la expresión dada, y de la forma menos costosa (o, como mínimo, no mucho más que la menos costosa).

www.detodoprogramacion.com



(a) Árbol inicial de la expresión (b) Árbol transformado de la expresión

Figura 14.1 Expresiones equivalentes.

Para hallar el plan de evaluación de consultas menos costoso el optimizador necesita generar planes alternativos que produzcan el mismo resultado que la expresión dada y escoger el de menor coste. La generación de planes de evaluación de consultas implica tres etapas: (1) la generación de expresiones que sean equivalentes lógicamente a la expresión dada, (2) la estimación del coste de cada plan de evaluación y (3) la anotación de las expresiones alternativas resultantes para generar planes distintos de ejecución. Las etapas (1) y (3) se encuentran entrelazadas en el optimizador de consultas; algunas expresiones se generan y se anotan, luego se generan y se anotan otras expresiones, etc. La etapa (2) se realiza en segundo plano recopilando información estadística sobre las relaciones, tales como sus tamaños y la profundidad de los índices, para hacer una buena estimación del coste de un plan.

Para implementar la primera etapa el optimizador de consultas debe generar expresiones equivalentes a la expresión dada. Esto se lleva a cabo mediante las *reglas de equivalencia*, que especifican el modo de transformar una expresión en otra lógicamente equivalente. Estas reglas se describen en el Apartado 14.2.

En el Apartado 14.3 se describe el modo de estimar las estadísticas de los resultados de cada operación en los planes de consultas. El empleo de estas estadísticas con las fórmulas de costes del Capítulo 13 permite estimar los costes de cada operación. Los costes individuales se combinan para determinar el coste estimado de la evaluación de la expresión de álgebra relacional dada, como se indicó previamente en el Apartado 13.7.

En el Apartado 14.4 se describe el modo de escoger un plan de evaluación de consultas. Se puede escoger uno basado en el coste estimado de los planes. Dado que el coste es una estimación, el plan seleccionado no es necesariamente el menos costoso; no obstante, siempre y cuando las estimaciones sean buenas, es probable que el plan sea el menos costoso, o no mucho más costoso. Esta optimización, denominada **optimización basada en costes**, se describe en el Apartado 14.4.2.

Finalmente, las vistas materializadas ayudan a acelerar el procesamiento de ciertas consultas. En el Apartado 14.5 se estudia el modo de "mantener" las vistas materializadas (es decir, mantenerlas actualizadas) y la manera de llevar a cabo la optimización de consultas con las vistas materializadas.

14.2 Transformación de expresiones relacionales

Las consultas se pueden expresar de varias maneras diferentes, con costes de evaluación diferentes. En este apartado, en lugar de tomar la expresión relacional original, se consideran expresiones alternativas equivalentes.

Se dice que dos expresiones del álgebra relacional son **equivalentes** si, en cada ejemplar legal de la base de datos, las dos expresiones generan el mismo conjunto de tuplas (recuérdese que un ejemplar legal de la base de datos es la que satisface todas las restricciones de integridad especificadas en el esquema de la base de datos). Obsérvese que el orden de las tuplas resulta irrelevante; puede que las dos expresiones generen las tuplas en órdenes diferentes, pero se considerarán equivalentes siempre que el conjunto de tuplas sea el mismo.

www.detodoprogramacion.com

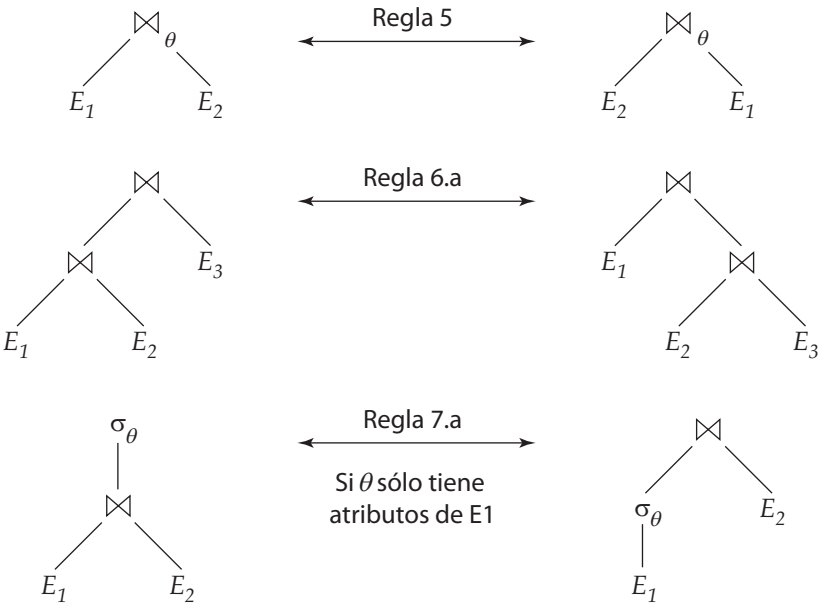


Figura 14.2 Representación gráfica de las equivalencias.

En SQL las entradas y las salidas son multiconjuntos de tuplas, y se usa la versión para multiconjuntos del álgebra relacional para evaluar las consultas de SQL. Se dice que dos expresiones de la versión *para multiconjuntos* del álgebra relacional son equivalentes si en cada base de datos legal las dos expresiones generan el mismo multiconjunto de tuplas. El estudio de este capítulo se basa en el álgebra relacional. Las extensiones a la versión para multiconjuntos del álgebra relacional se dejan al lector como ejercicios.

14.2.1 Reglas de equivalencia

Una **regla de equivalencia** establece que dos expresiones son equivalentes. Se puede sustituir la primera por la segunda forma, o viceversa, ya que las dos expresiones generan el mismo resultado en cualquier base de datos válida. El optimizador usa las reglas de equivalencia para transformar las expresiones en otras lógicamente equivalentes.

A continuación se enumeran varias reglas generales de equivalencia para las expresiones del álgebra relacional. Algunas de las equivalencias relacionadas aparecen en la Figura 14.2. Se usan $\theta, \theta_1, \theta_2$, etc., para denotar los predicados, L_1, L_2, L_3, etc., para denotar las listas de atributos y E, E_1, E_2, etc. para denotar las expresiones del álgebra relacional. El nombre de relación r no es más que un caso especial de expresión del álgebra relacional y puede usarse siempre que aparezca E.

1. Las operaciones de selección conjuntivas pueden dividirse en una secuencia de selecciones individuales. Esta transformación se denomina cascada de σ.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Las operaciones de selección son **conmutativas**.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Sólo son necesarias las últimas operaciones de una secuencia de operaciones de proyección, las demás pueden omitirse. Esta transformación también puede denominarse cascada de Π.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{L_n}(E))\ldots)) = \Pi_{L_1}(E)$$

4. Las selecciones pueden combinarse con los productos cartesianos y con las reuniones zeta.
   a. $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$. Esta expresión es precisamente la definición de la reunión zeta.
   b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

www.detodoprogramacion.com

5. Las operaciones de reunión zeta son conmutativas.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

Realmente, el orden de los atributos es diferente en el término de la derecha y en el de la izquierda, por lo que la equivalencia no se cumple si se tiene en cuenta el orden de los atributos. Se puede añadir una operación proyección a uno de los lados de la equivalencia para reordenar los atributos de la manera adecuada, pero para mayor sencillez se omite la proyección y se ignora el orden de los atributos en la mayor parte de los ejemplos.

Recuérdese que el operador de reunión natural es simplemente un caso especial del operador de reunión zeta; por tanto, las reuniones naturales también son conmutativas.

6. **a.** Las operaciones de reunión natural son **asociativas**.

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

b. Las reuniones zeta son asociativas en el sentido siguiente:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

donde θ_2 implica solamente atributos de E_2 y de E_3. Cualquiera de estas condiciones puede estar vacía; por tanto, se deduce que la operación producto cartesiano ($\times$) también es asociativa. La conmutatividad y la asociatividad de las operaciones de reunión son importantes para la reordenación de las reuniones en la optimización de las consultas.

7. La operación selección se distribuye por la operación reunión zeta bajo las dos condiciones siguientes:

a. Cuando todos los atributos de la condición de selección θ_0 implican únicamente los atributos de una de las expresiones (por ejemplo, E_1) que se están reuniendo.

$$\sigma_{\theta_0}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_\theta E_2$$

b. Cuando la condición de selección θ_1 implica únicamente los atributos de E_1 y θ_2 implica únicamente los atributos de E_2.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_\theta (\sigma_{\theta_2}(E_2))$$

8. La operación proyección se distribuye entre la operación reunión zeta bajo las condiciones siguientes.

a. Sean L_1 y L_2 atributos de E_1 y de E_2, respectivamente. Supóngase que la condición de reunión θ implica únicamente los atributos de $L_1 \cup L_2$. Entonces,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

b. Considérese una reunión $E_1 \bowtie_\theta E_2$. Sean L_1 y L_2 conjuntos de atributos de E_1 y de E_2, respectivamente. Sean L_3 los atributos de E_1 que están implicados en la condición de reunión θ, pero que no están en $L_1 \cup L_2$, y sean L_4 los atributos de E_2 que están implicados en la condición de reunión θ, pero que no están en $L_1 \cup L_2$. Entonces,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

9. Las operaciones de conjuntos unión e intersección son conmutativas.

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

La diferencia de conjuntos no es conmutativa.

10. La unión y la intersección de conjuntos son asociativas.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

www.detodoprogramacion.com

11. La operación selección se distribuye por las operaciones de unión, intersección y diferencia de conjuntos.

$$\sigma_P(E_1 - E_2) = \sigma_P(E_1) - \sigma_P(E_2)$$

De manera parecida, la equivalencia anterior, con $-$ sustituido por $\cup$ o por $\cap$, también es válida. Además,

$$\sigma_P(E_1 - E_2) = \sigma_P(E_1) - E_2$$

La equivalencia anterior, con $-$ sustituido por $\cap$, también es válida, pero no se cumple si $-$ se sustituye por $\cup$.

12. La operación proyección es distributiva con respecto a la operación unión.

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Ésta es sólo una lista parcial de las equivalencias. En los ejercicios se estudian más equivalencias que implican a los operadores relacionales extendidos, como la reunión externa y la agregación.

14.2.2 Ejemplos de transformaciones

Ahora se mostrará el empleo de las reglas de equivalencia. Se usará el ejemplo del banco con los esquemas de relaciones:

Esquema_sucursal = (*nombre_sucursal*, *ciudad_sucursal*, *activos*)
Esquema_cuenta = (*número_cuenta*, *nombre_sucursal*, *saldo*)
Esquema_impositor = (*nombre_cliente*, *número_cuenta*)

Las relaciones *sucursal*, *cuenta* e *impositor* son ejemplos de estos esquemas.

En el ejemplo del Apartado 14.1 la expresión

$$\Pi_{nombre_cliente}(\sigma_{ciudad_sucursal = \text{“Arganzuela”}}(sucursal \bowtie (cuenta \bowtie impositor)))$$

se transformaba en la expresión siguiente,

$$\Pi_{nombre_cliente}((\sigma_{ciudad_sucursal = \text{“Arganzuela”}}(sucursal)) \bowtie (cuenta \bowtie impositor))$$

la cual es equivalente a la expresión algebraica original, pero generando relaciones intermedias de menor tamaño. Esta transformación se puede llevar a cabo empleando la regla 7.a. Recuérdese que la regla sólo dice que las dos expresiones son equivalentes; no dice que una sea mejor que la otra.

Se pueden usar varias reglas de equivalencia, una tras otra, sobre una consulta o sobre partes de una consulta. Como ejemplo, supóngase que se modifica la consulta original para restringir la atención a los clientes que tienen un saldo superior a 1000 €. La nueva consulta del álgebra relacional es

$$\Pi_{nombre_cliente}(\sigma_{ciudad_sucursal = \text{“Arganzuela”} \wedge saldo > 1000}(sucursal \bowtie (cuenta \bowtie impositor)))$$

No se puede aplicar el predicado de la selección directamente a la relación *sucursal*, ya que éste implica atributos tanto de la relación *sucursal* como de la relación *cuenta*. No obstante, se puede aplicar antes la regla 6.a (asociatividad de la reunión natural) para transformar la reunión $sucursal \bowtie (cuenta \bowtie impositor)$ en $(sucursal \bowtie cuenta) \bowtie impositor$:

$$\Pi_{nombre_cliente}(\sigma_{ciudad_sucursal = \text{“Arganzuela”} \wedge saldo > 1000}((sucursal \bowtie cuenta) \bowtie impositor))$$

Luego, empleando la regla 7.a, se puede reescribir la consulta como

$$\Pi_{nombre_cliente}((\sigma_{ciudad_sucursal = \text{“Arganzuela”} \wedge saldo > 1000}(sucursal \bowtie cuenta)) \bowtie impositor)$$

Considérese ahora la subexpresión de selección de esta expresión. Empleando la regla 1 se puede dividir la selección en dos, para obtener la subexpresión siguiente:

$$\sigma_{ciudad_sucursal = \text{“Arganzuela”}}(\sigma_{saldo > 1000}(sucursal \bowtie cuenta))$$

www.detodoprogramacion.com

Las dos expresiones anteriores seleccionan tuplas con *ciudad_sucursal* = "Arganzuela" y *saldo* > 1000. Sin embargo, la última forma de la expresión ofrece una nueva oportunidad de aplicar la regla 7.a ("llevar a cabo primero las selecciones"), que da lugar a la subexpresión

$$\sigma_{ciudad_sucursal = \text{"Arganzuela"}} (sucursal) \bowtie \sigma_{saldo > 1000} (cuenta)$$

La Figura 14.3 muestra la expresión inicial y la expresión final después de todas estas transformaciones. También se podría haber usado la regla 7.b para obtener directamente la expresión final, sin usar la regla 1 para dividir la selección en dos selecciones. De hecho, la regla 7.b puede obtenerse de las reglas 1 y 7.a.

Se dice que un conjunto de reglas de equivalencia es **mínimo** si no se puede obtener ninguna regla a partir de una reunión de las demás. El ejemplo anterior muestra que el conjunto de reglas de equivalencia del Apartado 14.2.1 no es mínimo. Se puede generar una expresión equivalente a la original de diferentes maneras; el número de maneras diferentes de generar una expresión aumenta cuando se usa un conjunto de reglas de equivalencia que no es mínimo. Los optimizadores de consultas, por tanto, usan conjuntos mínimos de reglas de equivalencia.

Considérese ahora la siguiente forma de la consulta de ejemplo:

$$\Pi_{nombre_cliente} ((\sigma_{ciudad_sucursal = \text{"Arganzuela"}} (sucursal) \bowtie cuenta) \bowtie impositor)$$

Cuando se calcula la subexpresión

$$(\sigma_{ciudad_sucursal = \text{"Arganzuela"}} (sucursal) \bowtie cuenta)$$

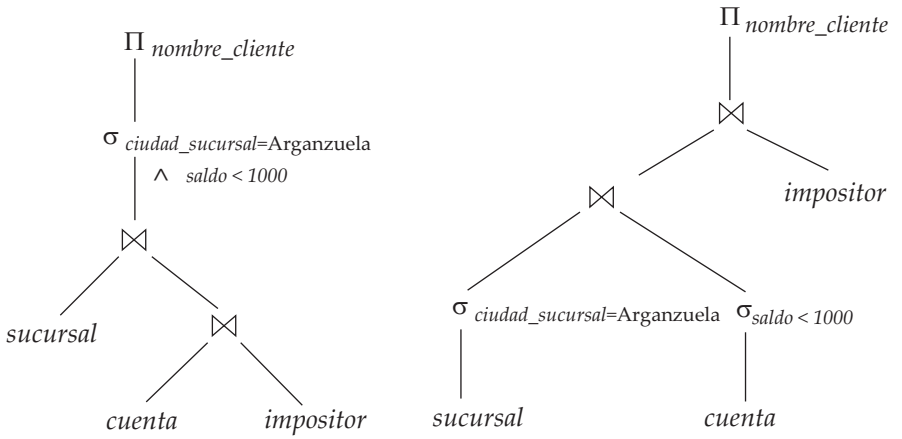
se obtiene una relación cuyo esquema es:

(*nombre_sucursal*, *ciudad_sucursal*, *activos*, *número_cuenta*, *saldo*)

Es posible eliminar varios atributos del esquema, forzando las proyecciones de acuerdo con las reglas de equivalencia 8.a y 8.b. Los únicos atributos que se deben conservar son los que aparecen en el resultado de la consulta y los que se necesitan para procesar las operaciones subsiguientes. Al eliminar los atributos innecesarios se reduce el número de columnas del resultado intermedio. Por tanto, se reduce el tamaño del resultado intermedio. En el ejemplo el único atributo que se necesita de la reunión de *sucursal* y de *cuenta* es *número_cuenta*. Por tanto, se puede modificar la expresión hasta

$$\Pi_{nombre_cliente} ((\Pi_{número_cuenta} ((\sigma_{ciudad_sucursal = \text{"Arganzuela"}} (sucursal)) \bowtie cuenta)) \bowtie impositor)$$

La proyección $\Pi_{número_cuenta}$ reduce el tamaño de los resultados de las reuniones intermedias.



(a) Árbol inicial de la expresión (b) Árbol tras varias transformaciones

Figura 14.3 Varias transformaciones.

www.detodoprogramacion.com

14.2.3 Ordenación de las reuniones

Es importante conseguir una buena ordenación de las operaciones reunión para reducir el tamaño de los resultados temporales; por tanto, la mayor parte de los optimizadores de consultas prestan mucha atención al orden de las reuniones. Como se mencionó en el Capítulo 3 y en la regla de equivalencia 6.a, la operación reunión natural es asociativa. Por tanto, para todas las relaciones r_1, r_2 y r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

Aunque estas expresiones sean equivalentes, los costes de calcular cada una de ellas pueden ser diferentes. Considérese una vez más la expresión

$$\Pi_{nombre_cliente} ((\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal)) \bowtie cuenta \bowtie impositor)$$

Se podría escoger calcular primero *cuenta* $\bowtie$ *impositor* y luego combinar el resultado con

$$\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal)$$

Sin embargo, es probable que *cuenta* $\bowtie$ *impositor* sea una relación de gran tamaño, ya que contiene una tupla por cada cuenta. En cambio,

$$\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal) \bowtie cuenta$$

es, probablemente, una relación de pequeño tamaño. Para comprobar que es así, se observa que, dado que el banco tiene un gran número de sucursales ampliamente distribuidas, es probable que sólo una pequeña parte de los clientes del banco tenga cuenta en las sucursales ubicadas en Arganzuela. Por tanto, la expresión anterior da lugar a una tupla por cada cuenta abierta por un residente de Arganzuela. Así, la relación temporal que se debe almacenar es menor que si se hubiera calculado primero *cuenta* $\bowtie$ *impositor*.

Hay otras opciones a considerar a la hora de evaluar la consulta. No hay que preocuparse del orden en que aparecen los atributos en las reuniones, ya que resulta sencillo cambiarlo antes de mostrar el resultado. Por tanto, para todas las relaciones r_1 y r_2,

$$r_1 \bowtie r_2 = r_2 \bowtie r_1$$

Es decir, la reunión natural es conmutativa (regla de equivalencia 5).

Mediante la asociatividad y la conmutatividad de la reunión natural (reglas 5 y 6) se puede considerar reescribir la expresión del álgebra relacional como

$$\Pi_{nombre_cliente} (((\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal)) \bowtie impositor) \bowtie cuenta)$$

Es decir, se puede calcular primero

$$(\sigma_{ciudad_sucursal = \text{“Arganzuela”}} (sucursal)) \bowtie impositor$$

y, luego, reunir el resultado con *cuenta*. Obsérvese, no obstante, que no existen atributos en común entre *Esquema_sucursal* y *Esquema_impositor*, por lo que la reunión no es más que un producto cartesiano. Si hay s sucursales en Arganzuela e i tuplas en la relación *impositor*, este producto cartesiano genera $s * i$ tuplas, una por cada par posible de tuplas de impositor y de sucursal (independientemente de si la cuenta de *impositor* está abierta en la sucursal). Por tanto, parece que este producto cartesiano producirá una relación temporal de gran tamaño. En consecuencia, esta estrategia se rechaza. No obstante, si el usuario ha introducido la expresión anterior, se pueden usar la asociatividad y la conmutatividad de la reunión natural para transformarla en la expresión más eficiente que se usó anteriormente.

14.2.4 Enumeración de expresiones equivalentes

Los optimizadores de consultas usan las reglas de equivalencia para generar de manera sistemática expresiones equivalentes a la expresión de consulta dada. Conceptualmente el proceso se desarrolla de la manera siguiente. Dada una expresión, si alguna subexpresión coincide con el lado izquierdo o derecho de una regla de equivalencia, el optimizador genera una nueva expresión con la subexpresión transformada de modo que coincida con el otro lado de la regla. Este proceso continúa hasta que no se puedan generar expresiones nuevas.

www.detodoprogramacion.com

El proceso anterior resulta costoso tanto en espacio como en tiempo. Éste es el modo en que se puede reducir los requisitos de espacio: si se genera una expresión E_1 a partir de una expresión E_2 empleando una regla de equivalencia, E_1 y E_2 son parecidas en estructura y tienen subexpresiones que son idénticas. Las técnicas de representación de expresiones que permiten que las dos expresiones apunten a las subexpresiones compartidas pueden reducir de manera significativa los requisitos de espacio, y muchos optimizadores de consultas las usan.

Además, no siempre resulta necesario generar todas las posibles expresiones con las reglas de equivalencia. Si un optimizador tiene en cuenta las estimaciones de costes, puede que logre evitar el examen de algunas de las expresiones, como se verá en el Apartado 14.4. Se puede reducir el tiempo necesario para la optimización empleando técnicas como éstas.

14.3 Estimación de las estadísticas de los resultados de las expresiones

El coste de cada operación depende del tamaño y de otras estadísticas de sus valores de entrada. Dada una expresión como $a \bowtie (b \bowtie c)$, para estimar el coste de combinar a con $(b \bowtie c)$ hay que hacer estimaciones de estadísticas como el tamaño de $b \bowtie c$.

En este apartado se relacionarán en primer lugar algunas estadísticas de las relaciones de bases de datos que se almacenan en los catálogos de los sistemas de bases de datos y luego se mostrará el modo de usar las estadísticas para estimar estadísticas de los resultados de varias operaciones relacionales.

Algo que quedará claro más adelante en este apartado es que las estimaciones no son muy precisas, ya que se basan en suposiciones que pueden no cumplirse exactamente. El plan de evaluación de consultas que tenga el coste estimado de ejecución más reducido puede, por tanto, no tener el coste real de ejecución más bajo. Sin embargo, la experiencia real ha mostrado que, aunque las estimaciones no sean muy precisas, los planes con los costes estimados más reducidos tienen costes de ejecución reales que, o bien son los más reducidos o bien se hallan cercanos al menor coste real de ejecución.

14.3.1 Información de catálogo

Los catálogos de los SGBD almacenan la siguiente información estadística sobre las relaciones de las bases de datos:

- n_r, el número de tuplas de la relación *r*.
- b_r, el número de bloques que contienen tuplas de la relación r.
- t_r, el tamaño de cada tupla de la relación *r* en bytes.
- f_r, el factor de bloqueo de la relación r—es decir, el número de tuplas de la relación r que caben en un bloque.
- $V(A, r)$, el número de valores distintos que aparecen en la relación *r* para el atributo A. Este valor es igual que el tamaño de $\Pi_A(r)$. Si A es una clave de la relación r, $V(A, r)$ es n_r.

La última estadística, $V(A, r)$, también puede calcularse para conjuntos de atributos, si se desea, en vez de sólo para atributos aislados. Por tanto, dado un conjunto de atributos, $\mathcal{A}$, $V(\mathcal{A}, r)$ es el tamaño de $\Pi_{\mathcal{A}}(r)$.

Si se da por supuesto que las tuplas de la relación r se almacenan físicamente juntas en un archivo, se cumple la ecuación siguiente:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

Las estadísticas sobre los índices, como las alturas de los árboles B^+ y el número de páginas hojas de los índices, también se conservan en el catálogo.

Por tanto, si se desean conservar estadísticas precisas, cada vez que se modifica una relación también hay que actualizar las estadísticas. Esta actualización supone una sobrecarga sustancial. Por tanto, la

www.detodoprogramacion.com

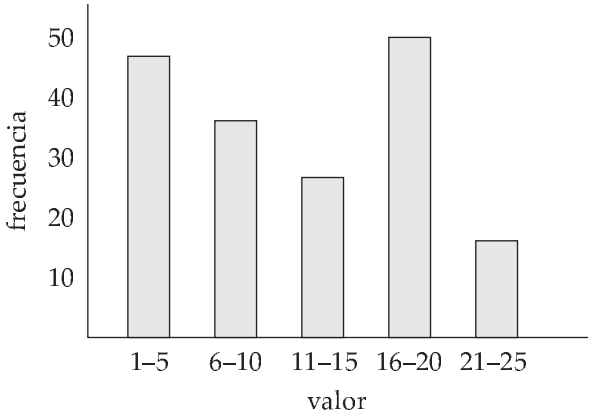


Figura 14.4 Ejemplo de histograma.

mayor parte de los sistemas no actualizan las estadísticas con cada modificación. En lugar de eso, actualizan las estadísticas durante los periodos de poca carga del sistema. En consecuencia, puede que las estadísticas usadas para escoger una estrategia de procesamiento de consultas no sean completamente exactas. Sin embargo, si no se producen demasiadas actualizaciones en los intervalos entre las actualizaciones de las estadísticas, éstas serán lo bastante precisas como para proporcionar una buena estimación de los costes relativos de los diferentes planes.

La información estadística indicada aquí está simplificada. Los optimizadores reales suelen conservar información estadística adicional para mejorar la precisión de sus estimaciones de costes de los planes de evaluación. Por ejemplo, la mayoría de las bases de datos almacenan la distribución de los valores de cada atributo en forma de **histograma**: en los histogramas los valores del atributo se dividen en una serie de rangos, y con cada rango el histograma asocia el número de tuplas cuyo valor del atributo se halla en ese rango. En la Figura 14.4 se muestra un ejemplo de histograma para un atributo entero que toma valores en el rango de 1 a 25.

Los histogramas usados en los sistemas de bases de datos registran normalmente el número de los valores distintos en cada rango, además del número de tuplas con en ese rango.

Como ejemplo de histograma, el rango de valores del atributo *edad* de la relación *persona* puede dividirse en $0–9$, $10–19$, ..., $90–99$ (suponiendo una edad máxima de 99). Con cada rango se almacena un recuento del número de tuplas *persona* cuyos valores de *edad* se hallan en ese rango, junto con el número de valores distintos de edad que se encuentran en ese rango. Sin la información del histograma un optimizador tendría que suponer que la distribución de los valores es uniforme; es decir, que cada rango tiene el mismo recuento.

Un histograma ocupa muy poco espacio, por lo que los histogramas sobre varios atributos diferentes se pueden almacenar en el catálogo del sistema. En los sistemas de bases de datos se usan varios tipos de histogramas. Por ejemplo, el **histograma de equianchura** divide el rango de valores en rangos de igual tamaño, mientras que el **histograma de equiprofundidad** ajusta las fronteras de los rangos de forma que cada rango tenga el mismo número de valores.

14.3.2 Estimación del tamaño de la selección

La estimación del tamaño del resultado de una operación selección depende del predicado de la selección. En primer lugar se considerará un solo predicado de igualdad, luego un solo predicado de comparación y, finalmente, combinaciones de predicados.

- $\sigma_{A=a}(r)$: Si se supone una distribución uniforme de los valores (es decir, que cada valor aparece con igual probabilidad), se puede estimar que el resultado de la selección tiene $n_r/V(A,r)$ tuplas, suponiendo que el valor a aparece en el atributo A de algún registro de r. La suposición de que el valor a de la selección aparece en algún registro suele ser cierta, y las estimaciones de costes suelen hacerla de manera implícita. No obstante, no suele ser realista suponer que cada valor aparece con igual probabilidad. El atributo *nombre_sucursal* de la relación *cuenta* es un ejemplo en

www.detodoprogramacion.com

el que esta suposición no es válida. Hay una tupla de la relación *cuenta* para cada cuenta. Resulta razonable esperar que las sucursales grandes tengan más cuentas que las pequeñas. Por tanto, algunos valores *nombre_sucursal* aparecen con mayor probabilidad que otros. Pese al hecho de que la suposición de distribución uniforme no suele ser correcta, resulta una aproximación razonable de la realidad en muchos casos, y ayuda a que la representación siga siendo relativamente sencilla.

Si hay disponible un histograma sobre el atributo A, se puede localizar el rango que contiene el valor a y modificar la estimación anterior $n_r/V(A,r)$ usando el contador de frecuencia para el rango en lugar de n_r, y el número de valores distintos que aparecen en el rango en lugar de $V(A,r)$.

- $\sigma_{A \leq v}(r)$: Considérese una selección de la forma $\sigma_{A \leq v}(r)$. Si el valor real usado en la comparación (v) está disponible en el momento de la estimación del coste, puede hacerse una estimación más precisa. Los valores mínimo y máximo ($\min(A,r)$ y $\max(A,r)$) del atributo pueden almacenarse en el catálogo. Suponiendo que los valores están distribuidos de manera uniforme, se puede estimar el número de registros que cumplirán la condición $A \leq v$ como 0 si $v < \min(A,r)$, como n_r si $v \geq \max(A,r)$ y como

$$n_r \cdot \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$$

en el resto de los casos.

Si hay disponible un histograma sobre el atributo A, se puede obtener una estimación más precisa; se dejan los detalles al lector.

En algunos casos, como cuando la consulta forma parte de un procedimiento almacenado, puede que el valor v no esté disponible cuando se optimice la consulta. En esos casos, se supondrá que aproximadamente la mitad de los registros cumplen la condición de comparación. Es decir, se supone que el resultado tiene $n_r/2$ tuplas; la estimación puede resultar muy imprecisa, pero es lo mejor que se puede hacer sin más información.

- Selecciones complejas:
  - **Conjunción**. Una *selección conjuntiva* es una selección de la forma

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n}(r)$$

Se puede estimar el tamaño del resultado de esta selección: para cada θ_i, se estima el tamaño de la selección $\sigma_{\theta_i}(r)$, denotada por s_i, como se ha descrito anteriormente. Por tanto, la probabilidad de que una tupla de la relación satisfaga la condición de selección θ_i es s_i/n_r.

La probabilidad anterior se denomina **selectividad** de la selección $\sigma_{\theta_i}(r)$. Suponiendo que las condiciones sean *independientes* entre sí, la probabilidad de que una tupla satisfaga todas las condiciones es simplemente el producto de todas estas probabilidades. Por tanto, se estima el número de tuplas de la selección completa como

$$n_r * \frac{s_1 * s_2 * \cdots * s_n}{n_r^n}$$

  - **Disyunción**. Una *selección disyuntiva* es una selección de la forma

$$\sigma_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n}(r)$$

Una condición disyuntiva se satisface por la unión de todos los registros que satisfacen las condiciones simples θ_i.

Como anteriormente, admitamos que s_i/n_r denota la probabilidad de que una tupla satisfaga la condición θ_i. La probabilidad de que la tupla satisfaga la disyunción es, pues, 1 menos la probabilidad de que no satisfaga *ninguna* de las condiciones:

$$1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \cdots * (1 - \frac{s_n}{n_r})$$

Multiplicando este valor por n_r se obtiene el número estimado de tuplas que satisfacen la selección.

www.detodoprogramacion.com

□ **Negación**. A falta de valores nulos el resultado de una selección $\sigma_{\neg\theta}(r)$ es simplemente las tuplas de r que no están en $\sigma_\theta(r)$. Ya se sabe el modo de estimar el número de tuplas de $\sigma_\theta(r)$. El número de tuplas de $\sigma_{\neg\theta}(r)$ se estima, por tanto, que es $n(r)$ menos el número estimado de tuplas de $\sigma_\theta(r)$.

Se pueden tener en cuenta los valores nulos estimando el número de tuplas para las que la condición θ se evalúa como *desconocida*, y restar ese número de la estimación anterior que ignora los valores nulos. La estimación de ese número exige conservar estadísticas adicionales en el catálogo.

14.3.3 Estimación del tamaño de las reuniones

En este apartado se verá el modo de estimar el tamaño del resultado de una reunión.

El producto cartesiano $r \times s$ contiene $n_r * n_s$ tuplas. Cada tupla de $r \times s$ ocupa $t_r + t_s$ bytes, de donde se puede calcular el tamaño del producto cartesiano.

La estimación del tamaño de una reunión natural resulta algo más complicada que la estimación del tamaño de una selección del producto cartesiano. Sean $r(R)$ y $s(S)$ dos relaciones.

- Si $R \cap S = \emptyset$, es decir, las relaciones no tienen ningún atributo en común, entonces $r \bowtie s$ es igual que $r \times s$, y se puede usar la técnica de estimación anterior para los productos cartesianos.
- Si $R \cap S$ es clave de R, entonces se sabe que cada tupla de s se combinará como máximo con una tupla de r. Por tanto, el número de tuplas de $r \bowtie s$ no es mayor que el número de tuplas de s. El caso de que $R \cap S$ sea clave de S es simétrico al caso que se acaba de describir. Si $R \cap S$ forma una clave externa de S, que hace referencia a R, el número de tuplas de $r \bowtie s$ es exactamente el mismo que el número de tuplas de s.
- El caso más difícil es que $R \cap S$ no sea clave de R ni de S. En ese caso se supone, como se hizo para las selecciones, que todos los valores aparecen con igual probabilidad. Considérese una tupla t de r y supóngase que $R \cap S = \{A\}$. Se estima que la tupla t produce

$$\frac{n_s}{V(A,s)}$$

tuplas en $r \bowtie s$, ya que este número es el número promedio de tuplas de s con un valor dado para los atributos A. Considerando todas las tuplas de r se estima que hay

$$\frac{n_r * n_s}{V(A,s)}$$

tuplas in $r \bowtie s$. Obsérvese que, si se invierten los papeles de r y de s en la estimación anterior, se obtiene una estimación de

$$\frac{n_r * n_s}{V(A,r)}$$

tuplas en $r \bowtie s$. Estas dos estimaciones son diferentes si $V(A,r) \neq V(A,s)$. Si esta situación se produce, probablemente haya tuplas pendientes que no participen en la reunión. Por tanto, probablemente la menor de las dos estimaciones sea la más precisa.

La estimación anterior del tamaño de la reunión puede ser demasiado elevada si los valores de $V(A,r)$ para el atributo A de r tienen pocas tuplas en común con los valores de $V(A,s)$ para el atributo A de s. No obstante, es improbable que se dé esta situación en la realidad, ya que las tuplas pendientes o bien no existen o sólo constituyen una pequeña fracción de las tuplas, en la mayor parte de las relaciones reales.

Lo más importante es que la estimación anterior depende de la suposición de que todos los valores aparecen con igual probabilidad. Hay que usar técnicas más sofisticadas para la estimación del tamaño si esta suposición no resulta válida. Por ejemplo, si se tuviesen histogramas sobre los atributos de la reunión de ambas relaciones, y ambos histogramas tuviesen los mismos rangos, se podría usar la técnica de la estimación anterior en los dos rangos usando el número de filas con valores en el rango en lugar de n_r o de n_s, y el número de valores distintos en el rango en lugar de $V(A,r)$ o de $V(A,s)$. Después se sumarían las estimaciones de tamaño obtenidas para cada rango para estimar el tamaño total. Como ejercicio, se deja al lector el caso en que ambas

www.detodoprogramacion.com

relaciones tengan histogramas sobre el atributo de la reunión, pero con rangos diferentes en los histogramas.

Se puede estimar el tamaño de una reunión zeta $r \bowtie_\theta s$ reescribiendo la reunión como $\sigma_\theta(r \times s)$ y empleando las estimaciones de tamaño de los productos cartesianos junto con las estimaciones de tamaño de las selecciones, que se vieron en el Apartado 14.3.2.

Para ilustrar todas estas maneras de estimar el tamaño de las reuniones, considérese la expresión:

$$impositor \bowtie cliente$$

Supóngase que se dispone de la siguiente información de catálogo sobre las dos relaciones:

- $n_{cliente} = 10.000$.
- $f_{cliente} = 25$, lo que implica que $b_{cliente} = 10.000/25 = 400$.
- $n_{impositor} = 5.000$.
- $f_{impositor} = 50$, lo que implica que $b_{impositor} = 5.000/50 = 100$.
- $V(nombre_cliente, impositor) = 2.500$, lo que implica que, en promedio, cada cliente tiene dos cuentas.

Supóngase también que *nombre_cliente* de *impositor* es clave externa de *cliente*.

En el ejemplo de *impositor* $\bowtie$ *cliente*, *nombre_cliente* de *impositor* es una clave externa que hace referencia a *cliente*; por tanto, el tamaño del resultado es exactamente $n_{impositor}$, que es 5.000.

Calculemos ahora las estimaciones de tamaño de *impositor* $\bowtie$ *cliente* sin usar la información sobre las claves externas. Como $V(nombre_cliente, impositor) = 2.500$ y $V(nombre_cliente, cliente) = 10.000$, las dos estimaciones que se consiguen son $5.000 * 10.000/2.500 = 20.000$ y $5.000 * 10.000/10.000 = 5.000$, y se escoge la menor. En este caso, la menor de las estimaciones es igual que la que se calculó anteriormente a partir de la información sobre las claves externas.

14.3.4 Estimación del tamaño de otras operaciones

A continuación se esbozará el modo de estimar el tamaño de los resultados de otras operaciones del álgebra relacional.

- **Proyección:** El tamaño estimado (número de registros de las tuplas) de una proyección de la forma $\Pi_A(r)$ es $V(A, r)$, ya que la proyección elimina los duplicados.
- **Agregación:** El tamaño de ${}_A\mathcal{G}_F(r)$ es simplemente $V(A, r)$, ya que hay una tupla de ${}_A\mathcal{G}_F(r)$ por cada valor distinto de A.
- **Operaciones de conjuntos:** Si las dos entradas de una operación de conjuntos son selecciones de la misma relación se puede reescribir la operación de conjuntos como disyunciones, conjunciones o negaciones. Por ejemplo, $\sigma_{\theta_1}(r) \cup \sigma_{\theta_2}(r)$ puede reescribirse como $\sigma_{\theta_1 \vee \theta_2}(r)$. De manera parecida, las intersecciones se pueden reescribir como conjunciones y la diferencia de conjuntos empleando la negación, siempre que las dos relaciones que participan en la operación de conjuntos sean selecciones de la misma relación. Luego se pueden usar las estimaciones de las selecciones que impliquen conjunciones, disyunciones y negaciones del Apartado 14.3.2.

  Si las entradas no son selecciones de la misma relación se estiman los tamaños de esta manera: el tamaño estimado de $r \cup s$ es la suma de los tamaños de r y de s. El tamaño estimado de $r \cap s$ es el mínimo de los tamaños de r y de s. El tamaño estimado de $r - s$ es el mismo tamaño de r. Las tres estimaciones pueden ser imprecisas, pero proporcionan cotas superiores para los tamaños.
- **Reunión externa:** El tamaño estimado de $r \supset\bowtie s$ es el tamaño de $r \bowtie s$ más el tamaño de r; el de $r \bowtie\subset s$ es simétrico, mientras que el de $r \supset\bowtie\subset s$ es el tamaño de $r \bowtie s$ más los tamaños de r y s. Las tres estimaciones pueden ser imprecisas, pero proporcionan cotas superiores para los tamaños.

www.detodoprogramacion.com

14.3.5 Estimación del número de valores distintos

Para las selecciones el número de valores distintos de un atributo (o de un conjunto de atributos) A en el resultado de una selección, $V(A, \sigma_\theta(r))$, puede estimarse de la manera siguiente:

- Si la condición de selección θ obliga a que A adopte un valor especificado (por ejemplo, $A = 3$), $V(A, \sigma_\theta(r)) = 1$.
- Si θ obliga a que A adopte un valor de entre un conjunto especificado de valores (por ejemplo, $(A = 1 \vee A = 3 \vee A = 4)$), entonces $V(A, \sigma_\theta(r))$ se define como el número de valores especificados.
- Si la condición de selección θ es de la forma $Aopv$, donde op es un operador de comparación, $V(A, \sigma_\theta(r))$ se estima que es $V(A, r) * s$, donde s es la selectividad de la selección.
- En todos los demás casos de selecciones se da por supuesto que la distribución de los valores de A es independiente de la distribución de los valores para los que se especifican las condiciones de selección y se usa una estimación aproximada de min $\min(V(A, r), n_{\sigma_\theta(r)})$. Se puede obtener una estimación más precisa para este caso usando la teoría de la probabilidad, pero la aproximación anterior funciona bastante bien.

Para las reuniones el número de valores distintos de un atributo (o de un conjunto de atributos) A en el resultado de una reunión, $V(A, r \bowtie s)$, puede estimarse de la manera siguiente:

- Si todos los atributos de A proceden de r, $V(A, r \bowtie s)$ se estima como $\min(V(A, r), n_{r \bowtie s})$, y de manera parecida si todos los atributos de A proceden de s, $V(A, r \bowtie s)$ se estima que es $\min(V(A, s), n_{r \bowtie s})$.
- Si A contiene atributos $A1$ de r y $A2$ de s, entonces $V(A, r \bowtie s)$ se estima como

$$\min(V(A1, r) * V(A2 - A1, s), V(A1 - A2, r) * V(A2, s), n_{r \bowtie s})$$

  Obsérvese que algunos atributos pueden estar en $A1$ y en $A2$, y que $A1 - A2$ y $A2 - A1$ denotan, respectivamente, a los atributos de A que sólo proceden de r y a los atributos de A que sólo proceden de s. Nuevamente, se pueden obtener estimaciones más precisas usando la teoría de la probabilidad, pero las aproximaciones anteriores funcionan bastante bien.

Las estimaciones de los distintos valores son directas para las proyecciones: son iguales en $\Pi_A(r)$ que en r. Lo mismo resulta válido para los atributos de agrupación de las agregaciones. Para los resultados de **suma**, **cuenta** y **promedio**, se puede suponer, por simplificar, que todos los valores agregados son distintos. Para **min**(A) y **max**(A), el número de valores distintos puede estimarse como $\min(V(A, r), V(G, r))$, donde G denota los atributos de agrupamiento. Se omiten los detalles de la estimación de los valores distintos para otras operaciones.

14.4 Elección de los planes de evaluación

La generación de expresiones sólo es una parte del proceso de optimización de consultas, ya que cada operación de la expresión puede implementarse con algoritmos diferentes. Por tanto, se necesita un plan de evaluación para definir exactamente el algoritmo que se usará para cada operación y el modo en que se coordinará la ejecución de las operaciones. La Figura 14.5 muestra un plan de evaluación posible para la expresión de la Figura 14.3. Como ya se ha visto, se pueden emplear varios algoritmos diferentes para cada operación relacional, lo que da lugar a planes de evaluación alternativos. Además, hay que tomar decisiones sobre el encauzamiento. En esta figura, los trazos de las operaciones de selección hasta la operación reunión mezcla están marcados como encauzados; el encauzamiento es factible si las operaciones de selección generan sus resultados ordenados según los atributos de reunión. Lo harán si los índices de *sucursal* y *cuenta* almacenan los registros con valores iguales de los atributos de índice ordenados por *nombre_sucursal*.

www.detodoprogramacion.com

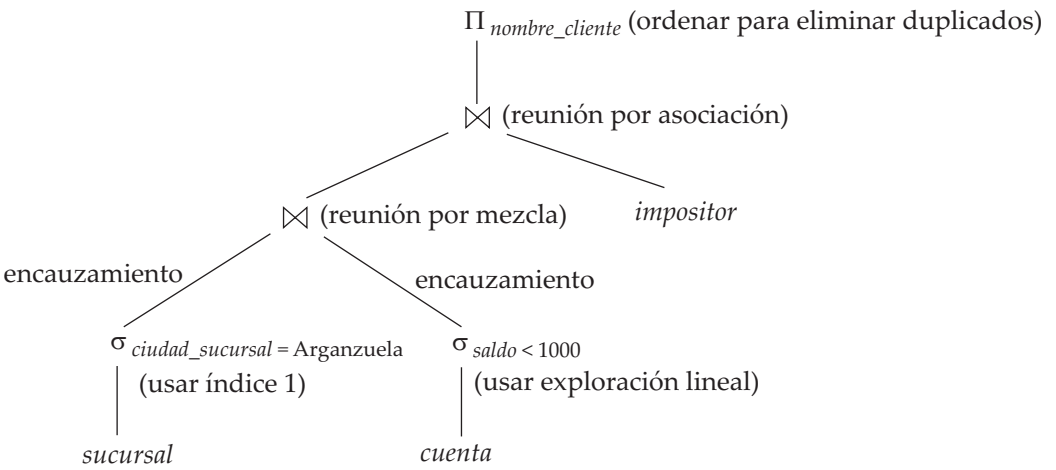


Figura 14.5 Un plan de evaluación.

14.4.1 Interacción de las técnicas de evaluación

Una manera de escoger un plan de evaluación para una expresión de consulta es sencillamente escoger el algoritmo más económico para evaluar cada operación. Se puede escoger cualquier ordenación de las operaciones que asegure que las operaciones ubicadas por debajo en el árbol se ejecuten antes que las operaciones situadas más arriba.

Sin embargo, la selección del algoritmo más económico para cada operación no es necesariamente una buena idea. Aunque puede que una reunión por mezcla en un nivel dado resulte más costosa que una reunión por asociación, puede que proporcione un resultado ordenado que haga más económica la evaluación de operaciones posteriores (como la eliminación de duplicados, la intersección u otra reunión por asociación). De manera parecida, puede que una reunión en bucle anidado indexada proporcione oportunidades para el encauzamiento de los resultados de la operación siguiente y, por tanto, puede que resulte útil aunque no sea la manera más económica de llevar a cabo la reunión. Para escoger el mejor algoritmo global hay que considerar incluso los algoritmos no óptimos para cada una de las operaciones.

Por tanto, además de considerar las expresiones alternativas de cada consulta, también hay que considerar los algoritmos alternativos para cada operación de cada expresión. Se pueden usar reglas muy parecidas a las reglas de equivalencia para definir los algoritmos que pueden usarse para cada operación, y si su resultado puede encauzarse o se debe materializar. Se pueden usar estas reglas para generar todos los planes de evaluación de consultas para una expresión dada.

Dado un plan de evaluación, se puede estimar su coste empleando las estadísticas estimadas mediante las técnicas del Apartado 14.3 junto con las estimaciones de costes de varios algoritmos y métodos de evaluación descritos en el Capítulo 13. Dependiendo de los índices disponibles, ciertas operaciones selección se pueden evaluar sólo con un índice y sin acceder a la relación en sí. Eso sigue dejando el problema de la selección del mejor plan de evaluación de la consulta. Hay dos enfoques generales: el primero busca todos los planes y escoge el mejor de una manera basada en los costes. El segundo usa la heurística para escoger el plan. A continuación se estudiarán los dos enfoques. Los optimizadores de consultas prácticos incorporan elementos de ambos enfoques.

14.4.2 Optimización basada en el coste

Los **optimizadores basados en el coste** generan una gama de planes de evaluación a partir de la consulta dada empleando las reglas de equivalencia y escogen el de coste mínimo. Para las consultas complejas el número de planes de consulta diferentes que son equivalentes a un plan dado puede ser grande. A modo de ejemplo, considérese la expresión

$$r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

www.detodoprogramacion.com

en la que las reuniones se expresan sin ninguna ordenación. Con $n = 3$, hay 12 ordenaciones diferentes de la mezcla:

$$\begin{array}{llll} r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\ r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\ r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3 \end{array}$$

En general, con n relaciones, hay $(2(n-1))!/(n-1)!$ órdenes de reunión diferentes (se deja el cálculo de esta expresión al lector en el Ejercicio 14.10). Para las reuniones que implican números pequeños de relaciones, este número resulta aceptable; por ejemplo, con $n = 5$, el número es 1.680. Sin embargo, a medida que n se incrementa, este número crece rápidamente. Con $n = 7$, el número es 665.280; con $n = 10$, ¡el número es mayor de 17.600 millones!

Afortunadamente, no es necesario generar todas las expresiones equivalentes a la expresión dada. Por ejemplo, supóngase que se desea hallar el mejor orden de reunión de la forma

$$(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$$

que representa todos los órdenes de reunión en que r_1, r_2 y r_3 se reúnen primero (en algún orden), y el resultado se reúne (en algún orden) con r_4 and r_5. Hay doce órdenes de reunión diferentes para calcular $r_1 \bowtie r_2 \bowtie r_3$, y otros doce órdenes para calcular la reunión de este resultado con r_4 y r_5. Por tanto, parece que hay 144 órdenes de reunión que examinar. Sin embargo, una vez hallado el mejor orden de reunión para el subconjunto de relaciones $\{r_1, r_2, r_3\}$, se puede usar ese orden para las reuniones posteriores con r_4 and r_5, y se pueden ignorar todos los órdenes de reunión más costosos de $r_1 \bowtie r_2 \bowtie r_3$. Por lo que, en lugar de examinar 144 opciones, sólo hace falta examinar 12 + 12 opciones.

Usando esta idea se puede desarrollar un *algoritmo de programación dinámica* para hallar los órdenes de reunión óptimos. Los algoritmos de programación dinámica almacenan los resultados de los cálculos y los vuelven a usar, un procedimiento que puede reducir enormemente el tiempo de ejecución. En la Figura 14.6 aparece un procedimiento recursivo que implementa el algoritmo de programación dinámica.

El procedimiento almacena los planes de evaluación que calcula en el array asociado $mejorplan$, que está indexado por conjuntos de relaciones. Cada elemento del array asociativo contiene dos componentes: el coste del mejor plan de S y el propio plan. El valor de $mejorplan[S].coste$ se supone que se inicializa como ∞ si $mejorplan[S]$ no se ha calculado todavía.

El procedimiento comprueba en primer lugar si el mejor plan para calcular la reunión del conjunto de relaciones dado S se ha calculado ya (y se ha almacenado en el array asociativa $mejorplan$); si es así, devuelve el plan ya calculado.

Si S contiene sólo una relación, se registra en $mejorplan$ la mejor forma de acceder a S (teniendo en cuenta las selecciones sobre S, si las hay). Esto puede implicar el uso de un índice para identificar las

```
procedure HallarMejorPlan(S)
    if (mejorplan[S].coste ≠ ∞) /* mejorplan[S] ya se ha calculado */
        return mejorplan[S]
    if (S contiene sólo una relación)
        establecer mejorplan[S].plan y mejorplan[S].coste en función de la mejor manera de acceder a S
    else for each subconjunto no vacío S1 de S tal que S1 ≠ S
        P1 = HallarMejorPlan(S1)
        P2 = HallarMejorPlan(S − S1)
        A = mejor algoritmo para reunir los resultados de P1 y de P2
        coste = P1.coste + P2.coste + coste de A
        if coste < mejorplan[S].coste
            mejorplan[S].coste = coste
            mejorplan[S].plan = "ejecutar P1.plan; ejecutar P2.plan;
                                 reunir resultados de P1 y P2 usando A"
    return mejorplan[S]
```

Figura 14.6 Algoritmo de programación dinámica para la optimización del orden de la reunión.

www.detodoprogramacion.com

tuplas, y después buscar las tuplas (conocido frecuentemente como *exploración del índice*), o explorar la relación completa (conocido frecuentemente como *exploración de la relación*)[1].

En caso contrario, el procedimiento prueba todas las maneras posibles de dividir S en dos subconjuntos disjuntos. Para cada división el procedimiento halla de manera recursiva los mejores planes para cada uno de los dos subconjuntos y luego calcula el coste del plan global usando esa división. El procedimiento escoge el plan más económico de entre todas las alternativas para dividir S en dos conjuntos. El procedimiento almacena el plan más económico y su coste en el array $mejorplan$ y los devuelve. La complejidad temporal del procedimiento puede demostrarse que es $O(3^n)$ (véase el Ejercicio práctico 14.8).

En realidad, el orden en que la reunión de un conjunto de relaciones genera las tuplas también es importante para hallar el mejor orden global de reunión, ya que puede afectar al coste de las reuniones posteriores (por ejemplo, si se usa una mezcla). Se dice que un orden determinado de las tuplas es un **orden interesante** si puede resultar útil para alguna operación posterior. Por ejemplo, la generación del resultado de $r_1 \bowtie r_2 \bowtie r_3$ ordenado según los atributos comunes con r_4 o r_5 puede resultar útil, pero generarlo ordenado según los atributos comunes solamente con r_1 y r_2 no resulta útil. El uso de la reunión por mezcla para calcular $r_1 \bowtie r_2 \bowtie r_3$ puede resultar más costoso que emplear algún otro tipo de técnica de reunión, pero puede que proporcione un resultado ordenado según un orden interesante.

Por tanto, no basta con hallar el mejor orden de reunión para cada subconjunto del conjunto de n relaciones dadas. Por el contrario, hay que hallar el mejor orden de reunión para cada subconjunto para cada orden interesante de la reunión resultante para ese subconjunto. El número de subconjuntos de n relaciones es 2^n. El número de criterios de ordenación interesantes no suele ser grande. Así, hay que almacenar alrededor de 2^n expresiones de reunión. El algoritmo de programación dinámica para hallar el mejor orden de reunión puede extenderse de manera sencilla para que trabaje con los criterios de ordenación. El coste del algoritmo extendido depende del número de órdenes interesantes para cada subconjunto de relaciones; dado que se ha hallado que este número en la práctica es pequeño, el coste se queda en $O(3^n)$. Con $n = 10$ este número es de alrededor de 59.000, que es mucho mejor que los 17.600 millones de órdenes de reunión diferentes. Y lo que es más importante, el almacenamiento necesario es mucho menor que antes, ya que sólo hace falta almacenar un orden de reunión por cada orden interesante de cada uno de los 1024 subconjuntos de $r_1, \ldots, r_{10}$. Aunque los dos números siguen creciendo rápidamente con n, las reuniones que se producen con frecuencia suelen tener menos de diez relaciones y pueden manejarse con facilidad.

Se pueden usar varias técnicas para reducir aún más el coste de la búsqueda entre un gran número de planes. Por ejemplo, al examinar los planes para una expresión se puede concluir tras examinar sólo una parte de la expresión si se determina que el plan más económico para esa parte ya resulta más costoso que el plan de evaluación más económico para una expresión completa examinada anteriormente. De manera parecida, supóngase que se determina que la manera más económica de evaluar una subexpresión es más costosa que el plan de evaluación más económico para una expresión completa examinada anteriormente. Entonces, no hace falta examinar ninguna expresión completa que incluya esa subexpresión. Se puede reducir aún más el número de planes de evaluación que hay que considerar completamente llevando a cabo antes una selección heurística de un buen plan y estimando el coste de ese plan. Luego, sólo unos pocos planes competidores necesitarán un análisis completo de los costes. Estas optimizaciones pueden reducir de manera significativa la sobrecarga de la optimización de consultas.

Las complejidades de SQL introducen complicaciones para los optimizadores de consultas. El tratamiento de las consultas anidadas se describe brevemente en el Apartado 14.4.4.

La optimización descrita anteriormente se concentra en la optimización del orden de la reunión. En cambio, los optimizadores usados en algunos otros sistemas, principalmente en SQL Server de Microsoft, se basan en reglas de equivalencia. La ventaja de usar reglas de equivalencia es que se puede ampliar fácilmente el optimizador con nuevas reglas. Por ejemplo, las consultas anidadas se pueden representar usando constructoras del álgebra relacional extendida, y las transformaciones de las consultas anidadas se pueden expresar como reglas de equivalencia. Para que este enfoque sea eficaz se requieren técnicas

1. Si un índice contiene todos los atributos de la relación que se usan en la consulta, es posible realizar una *exploración sólo del índice* que recupera los valores de atributo requeridos del índice sin capturar las tuplas reales.

www.detodoprogramacion.com

eficientes para detectar las derivaciones de duplicados, y una forma de programación dinámica para evitar optimizar de nuevo las mismas subexpresiones. Este enfoque se propuso en el proyecto de investigación Volcano. Véanse las notas bibliográficas para consultar referencias con más información.

14.4.3 Heurísticas de optimización

Un inconveniente de la optimización basada en el coste es el coste de la propia optimización. Aunque el coste de la optimización de las consultas puede reducirse mediante algoritmos inteligentes, el número de planes de evaluación distintos para una consulta puede ser muy grande y buscar el plan óptimo a partir de este conjunto requiere un gran esfuerzo de cómputo. Por ello, muchos sistemas usan la **heurística** para reducir el coste de la optimización.

Un ejemplo de regla heurística es la siguiente regla para la transformación de consultas del álgebra relacional:

- Realizar las operaciones de selección tan pronto como sea posible.

Los optimizadores heurísticos usan esta regla sin averiguar si se reduce el coste mediante esta transformación. En el primer ejemplo de transformación del Apartado 14.2 se forzó la operación selección en una reunión.

Se dice que la regla anterior es heurística porque suele ayudar a reducir el coste, aunque no lo haga siempre. Como ejemplo de dónde puede dar lugar a un incremento del coste, considérese una expresión $\sigma_\theta(r \bowtie s)$, donde la condición θ sólo hace referencia a atributos de s. Ciertamente, la selección puede llevarse a cabo antes de la reunión. Sin embargo, si r es tremendamente pequeña comparada con s, y si hay un índice basado en los atributos de reunión de s pero no hay ningún índice basado en los atributos usados por θ, probablemente resulte una mala idea llevar a cabo la selección pronto. Llevar a cabo pronto la selección (es decir, directamente sobre s) exigiría hacer una exploración de todas las tuplas de s. Es posible que resulte más económico calcular la reunión usando el índice y luego rechazar las tuplas que no superen la selección.

La operación proyección, como la operación selección, reduce el tamaño de las relaciones. Por tanto, siempre que haya que generar una relación temporal, resulta ventajoso aplicar inmediatamente cuantas proyecciones sea posible. Esta ventaja sugiere un acompañante a la heurística "realizar las selecciones tan pronto como sea posible":

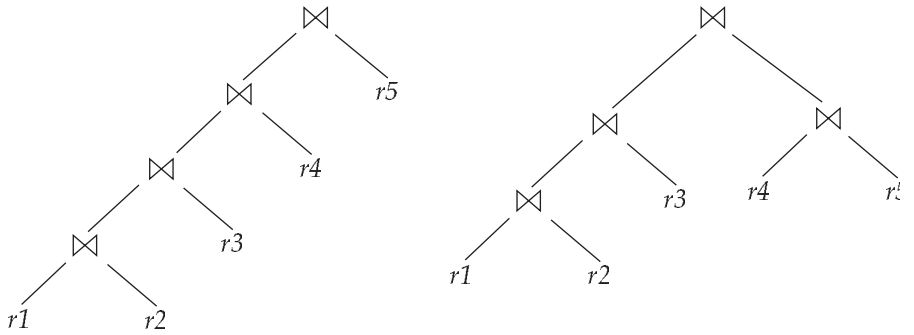
- Realizar las proyecciones tan pronto como sea posible.

Suele resultar mejor llevar a cabo las selecciones antes que las proyecciones, ya que las selecciones tienen la posibilidad de reducir mucho el tamaño de las relaciones y permiten el empleo de índices para tener acceso a las tuplas. Un ejemplo parecido al usado para la heurística de selección debería convencer al lector de que esta heurística no siempre reduce el coste.

La mayoría de optimizadores de consultas tienen más heurísticas para reducir el coste de la optimización. Por ejemplo, muchos optimizadores de consultas, como el optimizador de System R, no toman en consideración todos los órdenes de reunión, sino que restringen la búsqueda a tipos concretos de órdenes de reunión. El optimizador de System R sólo toma en consideración los órdenes de reunión en que el operando de la derecha de cada reunión es una de las relaciones iniciales $r_1, \ldots, r_n$. Estos órdenes de reunión se denominan **órdenes de reunión en profundidad por la izquierda**. Los órdenes de reunión en profundidad por la izquierda resultan especialmente convenientes para la evaluación encauzada, ya que el operando de la derecha es una relación almacenada y, así, sólo se encauza una entrada por cada reunión.

La Figura 14.7 muestra la diferencia entre un árbol de reunión en profundidad por la izquierda y otro que no lo es. El tiempo que se tarda en tomar en consideración todos los órdenes de reunión en profundidad por la izquierda es $O(n!)$, que es mucho menor que el tiempo necesario para tomar en consideración todos los órdenes de reunión. Con el empleo de las optimizaciones de programación dinámica, el optimizador de System R puede hallar el mejor orden de reunión en un tiempo de $O(n2^n)$. Compárese este coste con el tiempo de $O(3^n)$ necesario para hallar el mejor orden de reunión global.

www.detodoprogramacion.com



(a) Árbol de reunión en profundidad por la izquierda

(b) Árbol de reunión que no es en profundidad por la izquierda

Figura 14.7 Árboles de reunión en profundidad por la izquierda.

El optimizador System R usa la heurística para forzar el desplazamiento de las selecciones y de las proyecciones hacia la parte inferior del árbol de consultas.

El enfoque heurístico para reducir el coste de la selección del orden de reunión, que se usó originalmente en algunas versiones de Oracle, funciona básicamente de esta manera: para cada reunión de grado n toma en consideración n planes de evaluación. Cada plan usa un orden de reunión en profundidad por la izquierda, comenzando con una relación diferente de las n existentes. La heurística crea el orden de reunión para cada uno de los n planes de evaluación seleccionando de manera repetida la "mejor" relación que reunir a continuación, con base en la clasificación de los caminos de acceso disponibles. Se escoge la reunión en bucle anidado o mezcla-ordenación para cada una de las reuniones, en función de los caminos de acceso disponibles. Finalmente, la heurística escoge uno de los n planes de evaluación de manera heurística, basada en la minimización del número de reuniones de bucle anidado que no tienen disponible un índice para la relación interna y en el número de reuniones por mezcla-ordenación.

Los enfoques de la optimización de consultas que integran la selección heurística y la generación de planes de acceso alternativos se han adoptado en varios sistemas. El enfoque usado en System R y en su sucesor, el proyecto Starburst, es un procedimiento jerárquico basado en el concepto de bloques anidados de SQL. Las técnicas de optimización basadas en costes aquí descritas se usan por separado para cada bloque de la consulta. Los optimizadores de varios productos de bases de datos, como DB2 de IBM y Oracle, se basan en este enfoque, con extensiones para tratar otras operaciones como la agregación. Para las consultas compuestas de SQL (que usan la operación $\cup$, $\cap$ o $-$), el optimizador procesa cada componente por separado y combina los planes de evaluación para formar el plan global de evaluación.

Muchas aplicaciones ejecutan la misma consulta repetidamente pero con diferentes valores de las constantes. Por ejemplo, una aplicación bancaria puede ejecutar repetidamente una consulta para hallar las transacciones recientes sobre una cuenta, pero con diferentes valores del número de cuenta. Como heurística, muchos optimizadores optimizan una vez la consulta con los valores de la primera vez que se emitió, y almacenan en caché el plan de la consulta. Cada vez que se ejecuta de nuevo la consulta, quizás con nuevos valores de las constantes, el plan de la consulta en caché se reutiliza (obviamente usando los nuevos valores de las constantes). El plan óptimo para las nuevas constantes puede diferir del plan óptimo para los valores iniciales, pero como heurística se reutiliza el plan guardado en la caché.

Incluso con el uso de la heurística la optimización de consultas basada en los costes impone una sobrecarga sustancial al procesamiento de las consultas. No obstante, el coste añadido de la optimización de las consultas basada en los costes suele compensarse con creces por el ahorro en tiempo de ejecución de la consulta, que queda dominado por los accesos lentos a los discos. La diferencia en el tiempo de ejecución entre un buen plan y uno malo puede ser enorme, lo cual hace que la optimización de las consultas sea esencial. El ahorro conseguido se multiplica en las aplicaciones que se ejecutan de manera regular, en las que se puede optimizar la consulta una sola vez y usarse el plan de consultas seleccionado cada vez que se ejecute la consulta. Por tanto, la mayor parte de los sistemas comerciales incluyen

www.detodoprogramacion.com

optimizadores relativamente sofisticados. Las notas bibliográficas dan referencias de las descripciones de los optimizadores de consultas de los sistemas de bases de datos reales.

14.4.4 Optimización de subconsultas anidadas**

SQL trata conceptualmente a las subconsultas anidadas de la cláusula **where** como funciones que toman parámetros y devuelven un solo valor o un conjunto de valores (quizás, un conjunto vacío). Los parámetros son las variables de la consulta del nivel externo que se usan en la subconsulta anidada (estas variables se denominan **variables de correlación**). Por ejemplo, supóngase que se tiene la consulta siguiente:

```
select nombre_cliente
from prestatario
where exists (select *
              from impositor
              where impositor.nombre_cliente = prestatario.nombre_cliente)
```

Conceptualmente, la subconsulta puede considerarse como una función que toma un parámetro (aquí, *prestatario.nombre_cliente*) y devuelve el conjunto de todos los impositores con el mismo nombre.

SQL evalúa la consulta global (conceptualmente) calculando el producto cartesiano de las relaciones de la cláusula **from** externa y comprobando luego los predicados de la cláusula **where** para cada tupla del producto. En el ejemplo anterior, el predicado comprueba si el resultado de la evaluación de la subconsulta está vacío.

Esta técnica para evaluar una consulta con una subconsulta anidada se denomina **evaluación correlacionada**. La evaluación correlacionada no resulta muy eficiente, ya que la subconsulta se evalúa por separado para cada tupla de la consulta del nivel externo. Puede dar lugar a gran número de operaciones aleatorias de E/S a disco.

Por tanto, los optimizadores de SQL intentan transformar las subconsultas anidadas en reuniones, siempre que resulta posible. Los algoritmos de reunión eficientes evitan las costosas operaciones aleatorias de E/S. Cuando la transformación no resulta posible el optimizador conserva las subconsultas como expresiones independientes, las optimiza por separado y luego las evalúa mediante la evaluación correlacionada.

Como ejemplo de transformación de una subconsulta anidada en una reunión, la consulta del ejemplo anterior puede reescribirse como

```
select nombre_cliente
from prestatario, impositor
where impositor.nombre_cliente = prestatario.nombre_cliente
```

Para reflejar correctamente la semántica de SQL no debe cambiar el número de derivaciones duplicadas debido a la reescritura; la consulta reescrita puede modificarse para asegurarse de que se cumple esta propiedad, como se verá en breve.

En el ejemplo la subconsulta anidada era muy sencilla. En general, puede que no resulte posible desplazar las relaciones de la subconsulta anidada a la cláusula **from** de la consulta externa. En lugar de eso, se crea una relación temporal que contiene el resultado de la consulta anidada *sin* las selecciones empleando las variables de correlación de la consulta externa y se reúne la tabla temporal con la consulta del nivel exterior. Por ejemplo, una consulta de la forma

```
select ...
from L1
where P1 and exists (select *
                     from L2
                     where P2)
```

donde P_2 es una conjunción de predicados más sencillos, puede reescribirse como

www.detodoprogramacion.com

create table t_1 **as**
    select distinct V
        from L_2
        where P_2^1
select ...
from L_1, t_1
where P_1 **and** P_2^2

donde P_2^1 contiene los predicados de P_2 sin las selecciones que implican las variables de correlación, y P_2^2 reintroduce las selecciones que implican las variables de correlación (con las relaciones a que se hace referencia en el predicado renombradas adecuadamente). Aquí V contiene todos los atributos que se usan en las selecciones con las variables de correlación en la subconsulta anidada.

En el ejemplo la consulta original se habría transformado en:

create table t_1 **as**
    select distinct *nombre_cliente*
    from *impositor*
select *nombre_cliente*
from *prestatario*, t_1
where t_1.*nombre_cliente* = *prestatario.nombre_cliente*

La consulta que se reescribió para mostrar la creación de relaciones temporales puede obtenerse simplificando la consulta transformada más arriba, suponiendo que el número de duplicados de cada tupla no importa.

El proceso de sustituir una consulta anidada por una consulta con una reunión (acaso con una relación temporal) se denomina **descorrelación**.

La descorrelación resulta más complicada cuando la subconsulta anidada usa la agregación o cuando el resultado de la subconsulta anidada se usa para comprobar la igualdad, o cuando la condición que enlaza la subconsulta anidada con la consulta exterior es **not exists**, etc. No se intentará dar algoritmos para el caso general y, en vez de eso, se remitirá al lector a los elementos de importancia en las notas bibliográficas.

La optimización de las subconsultas anidadas complejas es una labor difícil, como puede deducirse del estudio anterior, y muchos optimizadores sólo llevan a cabo una cantidad limitada de descorrelación. Resulta más conveniente evitar el empleo de subconsultas anidadas complejas, siempre que sea posible, ya que no se puede asegurar que el optimizador de consultas tenga éxito en su conversión a una forma que pueda evaluarse de manera eficiente.

14.5 Vistas materializadas**

Cuando se define una vista, normalmente la base de datos sólo almacena la consulta que define la vista. Por el contrario, una **vista materializada** es una vista cuyo contenido se calcula y se almacena. Las vistas materializadas constituyen datos redundantes, en el sentido de que su contenido puede deducirse de la definición de la vista y del resto del contenido de la base de datos. No obstante, resulta mucho más económico en muchos casos leer el contenido de una vista materializada que calcular el contenido de la vista ejecutando la consulta que la define.

Las vistas materializadas resultan importantes para la mejora del rendimiento de algunas aplicaciones. Considérese esta vista, que da el importe total de los préstamos de cada sucursal:

create view *total_préstamos_sucursal*(*nombre_sucursal*, *total_préstamos*) **as**
select *nombre_sucursal*, **sum**(*importe*)
from *préstamos*
group by *nombre_sucursal*

www.detodoprogramacion.com

Supóngase que el importe total de los préstamos de la sucursal se solicita con frecuencia (antes de conceder un nuevo préstamo, por ejemplo). El cálculo de la vista exige la lectura de cada tupla de *préstamos* correspondiente a la sucursal y sumar los importes de los préstamos, lo que puede llevar mucho tiempo.

En cambio, si la definición de la vista del importe total de los préstamos estuviera materializada, el importe total de los préstamos podría hallarse buscando una sola tupla de la vista materializada.

14.5.1 Mantenimiento de las vistas

Un problema con las vistas materializadas es que hay que mantenerlas actualizadas cuando se modifican los datos empleados en la definición de la vista. Por ejemplo, si se actualiza el valor *importe* de un préstamo, la vista materializada se vuelve inconsistente con los datos subyacentes y hay que actualizarla. La tarea de mantener actualizada una vista materializada con los datos subyacentes se denomina **mantenimiento de la vista**.

Las vistas pueden mantenerse mediante código escrito a mano: es decir, cada fragmento del código que actualiza el valor *importe* del préstamo puede modificarse para actualizar el importe total de los préstamos de la sucursal correspondiente.

Otra opción para el mantenimiento de las vistas materializadas es la definición de desencadenadores para la inserción, el borrado y la actualización de cada relación de la definición de la vista. Los desencadenadores deben modificar el contenido de la vista materializada para tener en cuenta el cambio que ha provocado que se active el desencadenador. Una manera sencilla de hacerlo es volver a calcular completamente la vista materializada con cada actualización.

Una opción mejor consiste en modificar sólo las partes afectadas de la vista materializada, lo que se conoce como **mantenimiento incremental de la vista**. En el Apartado 14.5.2 se describe la manera de llevar a cabo el mantenimiento incremental de la vista.

Los sistemas modernos de bases de datos proporcionan más soporte directo para el mantenimiento incremental de las vistas. Los programadores de bases de datos ya no necesitan definir disparadores para el mantenimiento de las vistas. Por el contrario, una vez que se ha declarado materializada una vista, el sistema de bases de datos calcula su contenido y actualiza de manera incremental el contenido cuando se modifican los datos subyacentes.

La mayoría de sistemas de bases de datos realizan un **mantenimiento inmediato de las vistas**; es decir, se realiza un mantenimiento incremental tan pronto como sucede una actualización como parte de una transacción de actualización. Algunos sistemas de bases de datos también permiten el **mantenimiento diferido de las vistas**, donde se difiere el mantenimiento a un momento posterior; por ejemplo, las actualizaciones se pueden recopilar a lo largo del día y las vistas materializadas se pueden actualizar durante la noche. Este enfoque reduce la sobrecarga en las transacciones de actualización. Sin embargo, es posible que las vistas materializadas con mantenimiento diferido no sean consistentes con las relaciones sobre las que están definidas.

14.5.2 Mantenimiento incremental de las vistas

Para comprender el modo de mantener de manera incremental las vistas materializadas se comenzará por considerar las operaciones individuales y luego se verá la manera de manejar una expresión completa.

Los cambios de cada relación que puedan hacer que se quede desactualizada una vista materializada son las inserciones, los borrados y las actualizaciones. Para simplificar la descripción se sustituyen las actualizaciones a cada tupla por el borrado de esa tupla seguida de la inserción de la tupla actualizada. Por tanto, sólo hay que considerar las inserciones y los borrados. Los cambios (inserciones y borrados) en la relación o en la expresión se denominan su **diferencial**.

14.5.2.1 La operación reunión

Considérese la vista materializada $v = r \bowtie s$. Supóngase que se modifica r insertando un conjunto de tuplas denotado por i_r. Si el valor antiguo de r se denota por r^{vieja} y el valor nuevo de r por r^{nueva}, $r^{nueva} = r^{vieja} \cup i_r$. Ahora bien, el valor antiguo de la vista, v^{vieja} viene dado por $r^{vieja} \bowtie s$, y el valor nuevo v^{nueva} viene dado por $r^{nueva} \bowtie s$. Se puede reescribir $r^{nueva} \bowtie s$ como $(r^{vieja} \cup i_r) \bowtie s$, lo que se

www.detodoprogramacion.com

puede reescribir una vez más como $(r^{vieja} \bowtie s) \cup (i_r \bowtie s)$. En otras palabras,

$$v^{nueva} = v^{vieja} \cup (i_r \bowtie s)$$

Por tanto, para actualizar la vista materializada v, sólo hace falta añadir las tuplas $i_r \bowtie s$ al contenido antiguo de la vista materializada. Las inserciones en s se manejan de una manera completamente simétrica.

Supóngase ahora que se modifica r borrando un conjunto de tuplas denotado por d_r. Usando el mismo razonamiento que anteriormente se obtiene

$$v^{nueva} = v^{vieja} - (d_r \bowtie s)$$

Las operaciones de borrado en s se manejan de una manera completamente simétrica.

14.5.2.2 Las operaciones selección y proyección

Considérese una vista $v = \sigma_\theta(r)$. Si se modifica r insertando un conjunto de tuplas i_r, el nuevo valor de v puede calcularse como

$$v^{nueva} = v^{vieja} \cup \sigma_\theta(i_r)$$

De manera parecida, si se modifica r borrando un conjunto de tuplas e_r, el valor nuevo de v puede calcularse como

$$v^{nueva} = v^{vieja} - \sigma_\theta(e_r)$$

La proyección es una operación más difícil de tratar. Hay que considerar una vista materializada $v = \Pi_A(r)$. Supóngase que la relación r está en el esquema $R = (A, B)$ y que r contiene dos tuplas, $(a, 2)$ y $(a, 3)$. Entonces, $\Pi_A(r)$ tiene una sola tupla, (a). Si se borra la tupla $(a, 2)$ de r, no se puede borrar la tupla (a) de $\Pi_A(r)$: si se hiciera, el resultado sería una relación vacía, mientras que en realidad $\Pi_A(r)$ sigue teniendo una tupla ((a)). El motivo es que la misma tupla ((a)) se obtiene de dos maneras, y que el borrado de una tupla de r sólo borra una de las formas de obtener (a); la otra sigue presente.

Este motivo también ofrece una pista de la solución: para cada tupla de una proyección como $\Pi_A(r)$, se lleva la cuenta del número de veces que se ha obtenido.

Cuando se borra un conjunto de tuplas d_r de r, para cada tupla t de d_r hay que hacer lo siguiente. $t.A$ denota la proyección de t sobre el atributo A. Se busca $(t.A)$ en la vista materializada y se disminuye la cuenta almacenada con ella en 1. Si la cuenta llega a 0, se borra $(t.A)$ de la vista materializada.

El manejo de las inserciones resulta relativamente directo. Cuando un conjunto de tuplas i_r se inserta en r, para cada tupla t de i_r se hace lo siguiente. Si $(t.A)$ ya está presente en la vista materializada, se incrementa la cuenta almacenada con ella en 1. En caso contrario, se añade $(t.A)$ a la vista materializada con la cuenta puesta a 1.

14.5.2.3 Las operaciones de agregación

Las operaciones de agregación se comportan aproximadamente como las proyecciones. Las operaciones de agregación en SQL son **count, sum, avg, min** y **max**:

- **count**. Considérese la vista materializada $v =_A \mathcal{G}_{count(B)}(r)$, que calcula la cuenta del atributo B, después de agrupar r según el atributo A.

  Cuando se inserta un conjunto de tuplas i_r en r, para cada tupla t de i_r se realiza lo siguiente. Se busca el grupo $t.A$ en la vista materializada. Si no se halla presente, se añade $(t.A, 1)$ a la vista materializada. Si el grupo $t.A$ se halla presente, se añade 1 a su cuenta.

  Cuando un conjunto e_r se borra de r, para cada tupla t de e_r se realiza lo siguiente. Se busca el grupo $t.A$ en la vista materializada y se resta 1 de la cuenta del grupo. Si la cuenta llega a 0, se borra la tupla para el grupo $t.A$ de la vista materializada.

- **sum**. Considérese la vista materializada $v =_A \mathcal{G}_{sum(B)}(r)$.

  Cuando un conjunto de tuplas i_r se inserta en r, para cada tupla t de i_r se realiza lo siguiente. Se busca el grupo $t.A$ en la vista materializada. Si no se halla presente se añade $(t.A, t.B)$ a la vista materializada; además, se almacena una cuenta de 1 asociada con $(t.A, t.B)$, igual que se

www.detodoprogramacion.com

hizo para las proyecciones. Si el grupo $t.A$ se halla presente, se añade el valor de $t.B$ al valor agregado para el grupo y se añade 1 a su cuenta.

Cuando se borra un conjunto de tuplas e_r de r, para cada tupla t de e_r se realiza lo siguiente. Se busca el grupo $t.A$ en la vista materializada y se resta $t.B$ del valor agregado para el grupo. También se resta 1 de la cuenta del grupo y, si la cuenta llega a 0, se borra la tupla para el grupo $t.A$ de la vista materializada.

Si no se guardara el valor de cuenta adicional no se podría distinguir el caso de que la suma para el grupo sea 0 del caso en que se ha borrado la última tupla de un grupo.

- **avg**. Considérese la vista materializada $v =_A \mathcal{G}_{avg(B)}(r)$.

  La actualización directa del promedio de una inserción o de un borrado no resulta posible, ya que no sólo depende del promedio antiguo y de la tupla que se inserta o borra, sin también del número de tuplas del grupo.

  En lugar de eso, para tratar el caso de **avg**, se conservan los valores de agregación **sum** y **count** como se describieron anteriormente y se calcula el promedio como la suma dividida por la cuenta.

- **min, max**. Considérese la vista materializada $v =_A \mathcal{G}_{min(B)}(r)$ (el caso de **max** es completamente equivalente).

  El tratamiento de las inserciones en r es inmediato. La conservación de los valores de agregación **min** and **max** para los borrados puede resultar más costoso. Por ejemplo, si la tupla correspondiente al valor mínimo para un grupo se borra de r, hay que examinar las demás tuplas de r que están en el mismo grupo para hallar el nuevo valor mínimo.

14.5.2.4 Otras operaciones

La operación de conjuntos *intersección* se conserva de la manera siguiente. Dada la vista materializada $v = r \cap s$, cuando una tupla se inserta en r se comprueba si está presente en s y, en caso afirmativo, se añade a v. Si se borra una tupla de r, se borra de la intersección si se halla presente. Las otras operaciones con conjuntos, *unión* y *diferencia de conjuntos*, se tratan de manera parecida; los detalles se dejan al lector.

Las reuniones externas se tratan de manera muy parecida a las reuniones, pero con algún trabajo adicional. En el caso del borrado de r hay que manejar las tuplas de s que ya no coinciden con ninguna tupla de r. En el caso de una inserción en r, hay que manejar las tuplas de s que no coincidían con ninguna tupla de r. De nuevo se dejan los detalles al lector.

14.5.2.5 Tratamiento de expresiones

Hasta ahora se ha visto el modo de actualizar de manera incremental el resultado de una sola operación. Para tratar una expresión entera se pueden obtener expresiones para el cálculo del cambio incremental en el resultado de cada subexpresión, comenzando por las de menor tamaño.

Por ejemplo, supóngase que se desea actualizar de manera incremental la vista materializada $E_1 \bowtie E_2$ cuando se inserta un conjunto de tuplas i_r en la relación r. Supóngase que r se usa sólo en E_1. Supóngase que el conjunto de tuplas que se va a insertar en E_1 viene dado por la expresión D_1. Entonces, la expresión $D_1 \bowtie E_2$ da el conjunto de tuplas que hay que insertar en $E_1 \bowtie E_2$.

Véanse las notas bibliográficas para obtener más detalles sobre la conservación incremental de las vistas con expresiones.

14.5.3 Optimización de consultas y vistas materializadas

La optimización de consultas puede llevarse a cabo tratando las vistas materializadas igual que a las relaciones normales. No obstante, las vistas materializadas ofrecen más oportunidades para la optimización:

- Reescritura de las consultas para el empleo de vistas materializadas:

  Supóngase que está disponible la vista materializada $v = r \bowtie s$ y que un usuario emite la consulta $r \bowtie s \bowtie t$. Puede que la reescritura de la consulta como $v \bowtie t$ proporcione un plan de

www.detodoprogramacion.com

consulta más eficiente que la optimización de la consulta tal y como se ha emitido. Por tanto, es función del optimizador de consultas reconocer si se puede usar una vista materializada para acelerar una consulta.

- Sustitución del uso de una vista materializada por la definición de la vista:
  Supóngase que está disponible la vista materializada $v = r \bowtie s$, pero sin ningún índice definido sobre ella, y que un usuario emite la consulta $\sigma_{A=10}(v)$. Supóngase también que s tiene un índice sobre el atributo común B, y que r tiene un índice sobre el atributo A. Puede que el mejor plan para esta consulta sea sustituir v por $r \bowtie s$, lo que puede llevar al plan de consulta $\sigma_{A=10}(r) \bowtie s$; la selección y la reunión pueden llevarse a cabo de manera eficiente empleando los índices sobre $r.A$ y sobre $s.B$, respectivamente. Por el contrario, puede que la evaluación de la selección directamente sobre v necesite una exploración completa de v, lo que puede resultar más costoso.

Las notas bibliográficas dan indicaciones para investigar el modo de llevar a cabo de manera eficiente la optimización de las consultas con vistas materializadas.

Otro problema de optimización relacionado es el de la **selección de las vistas materializadas**, es decir, "la identificación del mejor conjunto de vistas para su materialización". Esta decisión debe tomarse con base en la **carga de trabajo** del sistema, que es una secuencia de consultas y de actualizaciones que refleja la carga típica del sistema. Un criterio sencillo sería la selección de un conjunto de vistas materializadas que minimice el tiempo global de ejecución de la carga de trabajo de consultas y de actualizaciones, incluido el tiempo empleado para conservar las vistas materializadas. Los administradores de bases de datos suelen modificar este criterio para tener en cuenta la importancia de las diferentes consultas y actualizaciones: puede ser necesaria una respuesta rápida para algunas consultas y actualizaciones, mientras que puede resultar aceptable una respuesta lenta para otras.

Los índices son como las vistas materializadas, en el sentido de que también son datos obtenidos, pueden acelerar las consultas y pueden desacelerar las actualizaciones. Por tanto, el problema de la **selección de índices** se halla íntimamente relacionado con el de la selección de las vistas materializadas, aunque resulta más sencillo.

Estos problemas se examinan con más detalle en los Apartados 23.1.5 y 23.1.6.

La mayoría de sistemas de bases de datos proporcionan herramientas para ayudar a los administradores de bases de datos en la selección de los índices y de las vistas materializadas. Estas herramientas examinan el historial de consultas y de actualizaciones y sugieren los índices y las vistas que hay que materializar. Database Tuning Assistant de SQL Server de Microsoft, Design Advisor de DB2 de IBM y Tuning Wizard de SQL de Oracle son ejemplos de estas herramientas.

14.6 Resumen

- Dada una consulta, suele haber gran variedad de métodos para calcular la respuesta. Es responsabilidad del sistema transformar la consulta tal y como la introdujo el usuario en una consulta equivalente que pueda calcularse de manera más eficiente. El proceso de búsqueda de una buena estrategia para el procesamiento de la consulta se denomina *optimización de consultas*.
- La evaluación de las consultas complejas implica muchos accesos a disco. Dado que la transferencia de los datos desde el disco resulta lenta en comparación con la velocidad de la memoria principal y de la CPU del sistema informático, merece la pena asignar una cantidad considerable de procesamiento a la elección de un método que minimice los accesos al disco.
- Hay varias reglas de equivalencia que se pueden emplear para transformar una expresión en otra equivalente. Estas reglas se emplean para generar de manera sistemática todas las expresiones equivalentes a la consulta dada.
- Cada expresión del álgebra relacional representa una secuencia concreta de operaciones. El primer paso para la selección de una estrategia de procesamiento de consultas es la búsqueda de una expresión del álgebra relacional que sea equivalente a la expresión dada y que se estime menos costosa de ejecutar.

www.detodoprogramacion.com

PARTE 5

Gestión de transacciones

El término *transacción* hace referencia a un conjunto de operaciones que forman una única unidad lógica de trabajo. Por ejemplo, la transferencia de dinero de una cuenta a otra es una transacción que consta de dos actualizaciones, una para cada cuenta.

Resulta importante que, o bien se ejecuten completamente todas las acciones de una transacción, o bien, en caso de fallo, se deshagan los efectos parciales de cada transacción incompleta. Esta propiedad se denomina *atomicidad*. Además, una vez ejecutada con éxito una transacción, sus efectos deben persistir en la base de datos—un fallo en el sistema no debe tener como consecuencia que la base de datos descarte una transacción que se haya completado con éxito. Esta propiedad se denomina *durabilidad*.

En los sistemas de bases de datos en los que se ejecutan de manera concurrente varias transacciones, si no se controlan las actualizaciones de los datos compartidos, existe la posibilidad de que las transacciones operen sobre estados intermedios inconsistentes creados por las actualizaciones de otras transacciones. Esta situación puede dar lugar a actualizaciones erróneas de los datos almacenados en la base de datos. Por tanto, los sistemas de bases de datos deben proporcionar los mecanismos para aislar las transacciones de otras transacciones que se ejecuten de manera concurrente. Esta propiedad se denomina *aislamiento*.

El Capítulo 15 describe con detalle el concepto de transacción, incluidas las propiedades de atomicidad, durabilidad, aislamiento y otras propiedades proporcionadas por la abstracción de las transacciones. En concreto, el capítulo precisa el concepto de aislamiento por medio de un concepto denominado secuencialidad.

El Capítulo 16 describe varias técnicas de control de concurrencia que ayudan a implementar la propiedad del aislamiento. El Capítulo 17 describe el componente de las bases de datos para la administración de las recuperaciones, que implementa las propiedades de atomicidad y de durabilidad.

Entendido como un todo, el componente de gestión de transacciones de un sistema de bases de datos permite a los desarrolladores de bases de datos centrarse en la implementación de las transacciones individualmente, ignorando los aspectos de concurrencia y tolerancia a fallos.

www.detodoprogramacion.com

CAPÍTULO 15

Transacciones

A menudo, desde el punto de vista del usuario de una base de datos, se considera a un conjunto de varias operaciones sobre una base de datos como una única operación. Por ejemplo, una transferencia de fondos desde una cuenta corriente a una cuenta de ahorros es una operación simple desde el punto de vista del cliente; sin embargo, en el sistema de base de datos, está compuesta internamente por varias operaciones. Evidentemente es esencial que tengan lugar todas las operaciones o que, en caso de fallo, ninguna de ellas se produzca. Sería inaceptable efectuar el cargo de la transferencia en la cuenta corriente y que no se abonase en la cuenta de ahorros.

Se llama **transacción** a una colección de operaciones que forman una única unidad lógica de trabajo. Un sistema de base de datos debe asegurar que la ejecución de las transacciones se realice adecuadamente a pesar de la existencia de fallos—o se ejecuta la transacción completa o no se ejecuta en absoluto. Además debe gestionar la ejecución concurrente de las transacciones evitando introducir inconsistencias. Volviendo al ejemplo de la transferencia de fondos, una transacción que calcule el saldo total del cliente podría ver el saldo de la cuenta corriente antes de que sea cargado por la transacción de la transferencia de fondos, y el saldo de la cuenta de ahorros después del abono. Como resultado, se obtendría un resultado incorrecto.

Este capítulo es una introducción a los conceptos básicos en el procesamiento de transacciones. En los Capítulos 16 y 17 se incluyen más detalles sobre el procesamiento concurrente de transacciones y la recuperación de fallos. En el Capítulo 24 se tratan temas adicionales acerca del procesamiento de transacciones.

15.1 Concepto de transacción

Una **transacción** es una **unidad** de la ejecución de un programa que accede y posiblemente actualiza varios elementos de datos. Una transacción se inicia por la ejecución de un programa de usuario escrito en un lenguaje de manipulación de datos de alto nivel o en un lenguaje de programación (por ejemplo SQL, C++ o Java), y está delimitado por instrucciones (o llamadas a función) de la forma **begin transaction** (inicio transacción) y **end transaction** (fin transacción). La transacción consiste en todas las operaciones que se ejecutan entre **begin transaction** y **end transaction**.

Para asegurar la integridad de los datos se necesita que el sistema de base de datos mantenga las siguientes propiedades de las transacciones:

- **Atomicidad**. O bien todas las operaciones de la transacción se realizan adecuadamente en la base de datos o ninguna de ellas.
- **Consistencia**. La ejecución aislada de la transacción (es decir, sin otra transacción que se ejecute concurrentemente) conserva la consistencia de la base de datos.

www.detodoprogramacion.com

- **Aislamiento**. Aunque se ejecuten varias transacciones concurrentemente, el sistema garantiza que para cada par de transacciones T_i y T_j, se cumple que para los efectos de T_i, o bien T_j ha terminado su ejecución antes de que comience T_i, o bien que T_j ha comenzado su ejecución después de que T_i termine. De este modo, cada transacción ignora al resto de las transacciones que se ejecuten concurrentemente en el sistema.
- **Durabilidad**. Tras la finalización con éxito de una transacción, los cambios realizados en la base de datos permanecen, incluso si hay fallos en el sistema.

Estas propiedades a menudo reciben el nombre de **propiedades ACID**; el acrónimo se obtiene de la primera letra de cada una de las cuatro propiedades en inglés (*Atomicity, Consistency, Isolation* y *Durability*, respectivamente).

Para comprender mejor las propiedades ACID y la necesidad de dichas propiedades, considérese un sistema bancario simplificado constituido por varias cuentas y un conjunto de transacciones que acceden y actualizan dichas cuentas. Por ahora se asume que la base de datos reside permanentemente en disco, pero una porción de la misma reside temporalmente en la memoria principal.

El acceso a la base de datos se lleva a cabo mediante las dos operaciones siguientes:

- leer(X), que transfiere el dato X de la base de datos a una memoria intermedia local perteneciente a la transacción que ejecuta la operación leer.
- escribir(X), que transfiere el dato X desde la memoria intermedia local de la transacción que ejecuta la operación escribir a la base de datos.

En un sistema de base de datos real, la operación escribir no tiene por qué producir necesariamente una actualización de los datos en disco; la operación escribir puede almacenarse temporalmente en memoria y llevarse a disco más tarde. Sin embargo, por el momento se supondrá que la operación escribir actualiza inmediatamente la base de datos. Se volverá a este tema en el Capítulo 17.

Sea T_i una transacción para transferir 50 € de la cuenta A a la cuenta B. Se puede definir dicha transacción como

T_i: leer(A);
$A := A - 50$;
escribir(A);
leer(B);
$B := B + 50$;
escribir(B).

Considérense ahora cada una de las propiedades ACID (para una presentación más cómoda, se consideran en distinto orden al indicado por A-C-I-D).

- **Consistencia**. En este caso el requisito de consistencia es que la suma de A y B no sea alterada al ejecutar la transacción. Sin el requisito de consistencia, ¡la transacción podría crear o destruir dinero! Se puede comprobar fácilmente que si una base de datos es consistente antes de ejecutar una transacción, sigue siéndolo después de ejecutar dicha transacción.

  La responsabilidad de asegurar la consistencia de una transacción es del programador de la aplicación que codifica dicha transacción. La comprobación automática de las restricciones de integridad puede facilitar esta tarea, como se vio en el Apartado 4.2.
- **Atomicidad**. Supóngase que justo antes de ejecutar la transacción T_i los valores de las cuentas A y B son de 1.000 € y de 2.000 €, respectivamente. Supóngase ahora que durante la ejecución de la transacción T_i se produce un fallo que impide que dicha transacción finalice con éxito su ejecución. Ejemplos de este tipo de fallos pueden ser los fallos en la alimentación, los fallos del hardware y los errores software. Además, supóngase que el fallo tiene lugar después de ejecutarse la operación escribir(A), pero antes de ejecutarse la operación escribir(B). Es ese caso, los valores de las cuentas A y B que se ven reflejados en la base de datos son 950 € y 2.000 €. Se han

www.detodoprogramacion.com

perdido 50 € de la cuenta A como resultado de este fallo. En particular se puede ver que ya no se conserva la suma $A + B$.

Así, como resultado del fallo, el estado del sistema deja de reflejar el estado real del mundo que se supone que modela la base de datos. Un estado así se denomina **estado inconsistente**. Hay que asegurarse de que estas inconsistencias no sean visibles en un sistema de base de datos. Obsérvese, sin embargo, que un sistema puede en algún momento alcanzar un estado inconsistente. Incluso si la transacción T_i se ejecuta por completo, existe un punto en el que el valor de la cuenta A es de 950 € y el de la cuenta B es de 2.000 €, lo cual constituye claramente un estado inconsistente. Este estado, sin embargo, se sustituye eventualmente por otro estado consistente en el que el valor de la cuenta A es de 950 € y el de la cuenta B es de 2.050 €. De este modo, si la transacción no empieza nunca o se garantiza que se complete, un estado inconsistente así no será visible excepto durante la ejecución de la transacción. Ésta es la razón de que aparezca el requisito de atomicidad. Si se proporciona la propiedad de atomicidad, o todas las acciones de la transacción se ven reflejadas en la base de datos, o ninguna de ellas.

La idea básica para asegurar la atomicidad es la siguiente. El sistema de base de datos mantiene los valores antiguos (en disco) de aquellos datos sobre los que una transacción realiza una escritura y, si la transacción no completa su ejecución, los valores antiguos se recuperan para que parezca que la transacción no se ha ejecutado. Estas ideas se muestran más adelante en el Apartado 15.2. La responsabilidad de asegurar la atomicidad es del sistema de base de datos; en concreto, lo maneja un componente llamado **componente de gestión de transacciones**, que se describe en detalle en el Capítulo 17.

- **Durabilidad**. Una vez que se completa con éxito la ejecución de una transacción, y después de comunicar al usuario que inició la transacción que se ha realizado la transferencia de fondos, un fallo en el sistema no debe producir la pérdida de datos correspondientes a dicha transferencia.

  La propiedad de durabilidad asegura que, una vez que se completa con éxito una transacción, persisten todas las modificaciones realizadas en la base de datos, incluso si hay un fallo en el sistema después de completarse la ejecución de dicha transacción.

  A partir de ahora se asume que un fallo en la computadora del sistema produce una pérdida de datos de la memoria principal, pero los datos almacenados en disco nunca se pierden. Se puede garantizar la durabilidad si se asegura que

  1. Las modificaciones realizadas por la transacción se guardan en disco antes de que finalice la transacción.
  2. La información de las modificaciones realizadas por la transacción guardada en disco es suficiente para permitir a la base de datos reconstruir dichas modificaciones cuando el sistema se reinicie después del fallo.

  La responsabilidad de asegurar la durabilidad pertenece a un componente software del sistema de base de datos llamado **componente de gestión de recuperaciones**. El componente de gestión de transacciones y el componente de gestión de recuperaciones están estrechamente relacionados, y su implementación se describe en el Capítulo 17.

- **Aislamiento**. Incluso si se aseguran las propiedades de consistencia y de atomicidad para cada transacción, si varias transacciones se ejecutan concurrentemente, se pueden entrelazar sus operaciones de un modo no deseado, produciendo un estado inconsistente.

  Por ejemplo, como se ha visto antes, la base de datos es inconsistente temporalmente durante la ejecución de la transacción para transferir fondos de la cuenta A a la cuenta B, con el total deducido escrito ya en A y el total incrementado todavía sin escribir en B. Si una segunda transacción que se ejecuta concurrente lee A y B en este punto intermedio y calcula $A + B$, observará un valor inconsistente. Además, si esta segunda transacción realiza después modificaciones en A y B basándose en los valores leídos, la base de datos puede permanecer en un estado inconsistente aunque ambas transacciones terminen.

  Una solución para el problema de ejecutar transacciones concurrentemente es ejecutarlas secuencialmente—es decir, una tras otra. Sin embargo, la ejecución concurrente de transacciones

www.detodoprogramacion.com

produce notables beneficios en el rendimiento, como se verá en el Apartado 15.4. Se han desarrollado otras soluciones que permiten la ejecución concurrente de varias transacciones.

Los problemas que causa la ejecución concurrente de transacciones se muestra en el Apartado 15.4. La propiedad de aislamiento asegura que el resultado obtenido al ejecutar concurrentemente las transacciones es un estado del sistema equivalente a uno obtenido al ejecutar una tras otra en algún orden. Los principios de aislamiento se presentarán más adelante en el Apartado 15.5. La responsabilidad de asegurar la propiedad de aislamiento es de un componente del sistema de base de datos llamado **componente de control de concurrencia**, que se presenta en el Capítulo 16.

15.2 Estados de una transacción

En ausencia de fallos, todas las transacciones se completan con éxito. Sin embargo, como se ha visto antes, una transacción puede que no siempre termine su ejecución con éxito. Una transacción de este tipo se denomina **abortada**. Si se pretende asegurar la propiedad de atomicidad, una transacción abortada no debe tener efecto sobre el estado de la base de datos. Así, cualquier cambio que haya hecho la transacción abortada sobre la base de datos debe deshacerse. Una vez que se han deshecho los cambios efectuados por la transacción abortada, se dice que la transacción está **retrocedida**. Parte de la responsabilidad del esquema de recuperaciones es gestionar las transacciones abortadas.

Una transacción que termina con éxito se dice que está **comprometida**. Una transacción comprometida que haya hecho modificaciones transforma la base de datos llevándola a un nuevo estado consistente, que permanece incluso si hay un fallo en el sistema.

Cuando una transacción se ha comprometido no se pueden deshacer sus efectos abortándola. La única forma de deshacer los cambios de una transacción comprometida es ejecutando una **transacción compensadora**. Por ejemplo, si una transacción añade 20 € a una cuenta, la transacción compensadora debería restar 20 € de la cuenta. Sin embargo, no siempre se puede crear dicha transacción compensadora. Por tanto, se deja al usuario la responsabilidad de crear y ejecutar transacciones compensadoras, y no la gestiona el sistema de base de datos. En el Capítulo 25 se incluye un estudio de las transacciones compensadoras.

Es necesario precisar qué se entiende por *terminación con éxito* de una transacción. Se establece por tanto un simple modelo abstracto de transacción. Una transacción debe estar en uno de los estados siguientes:

- **Activa**, el estado inicial; la transacción permanece en este estado durante su ejecución.
- **Parcialmente comprometida**, después de ejecutarse la última instrucción.
- **Fallida**, tras descubrir que no puede continuar la ejecución normal.
- **Abortada**, después del retroceso de la transacción y de haber restablecido la base de datos la base de datos a su estado anterior al comienzo de la transacción.
- **Comprometida**, tras completarse con éxito.

El diagrama de estados correspondiente a una transacción se muestra en la Figura 15.1. Se dice que una transacción se ha comprometido sólo si ha llegado al estado comprometida. Análogamente, se dice que una transacción ha abortado sólo si ha llegado al estado abortada. Una transacción se dice que ha **terminado** si se ha comprometido o bien se ha abortado.

Una transacción comienza en el estado activa. Cuando acaba su última instrucción pasa al estado de parcialmente comprometida. En este punto la transacción ha terminado su ejecución, pero es posible que aún tenga que ser abortada, puesto que los datos actuales pueden estar todavía en la memoria principal y puede producirse un fallo en el hardware antes de que se complete con éxito.

El sistema de base de datos escribe en disco la información suficiente para que, incluso al producirse un fallo, puedan reproducirse los cambios hechos por la transacción al reiniciar el sistema tras el fallo. Cuando se termina de escribir esta información, la transacción pasa al estado comprometida.

www.detodoprogramacion.com

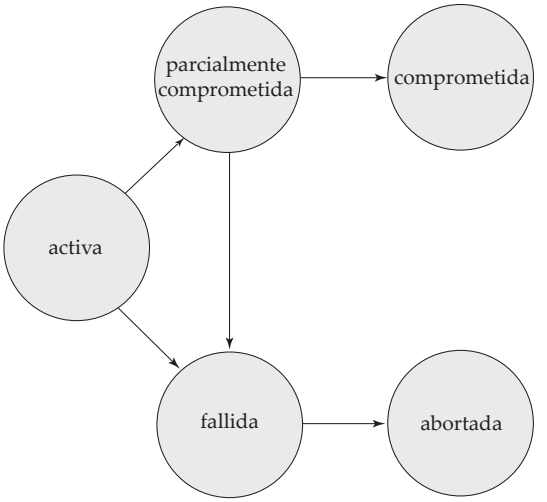


Figura 15.1 Diagrama de transición de estado de una transacción.

Como se ha mencionado antes, se asume que los fallos no provocan pérdidas de datos en disco. Las técnicas para tratar las pérdidas de datos en disco se muestran en el Capítulo 17.

Una transacción llega al estado fallida después de que el sistema determine que dicha transacción no puede continuar su ejecución normal (por ejemplo, a causa de errores de hardware o lógicos). Una transacción de este tipo se debe retroceder. Después pasa al estado abortada. En este punto, el sistema tiene dos opciones:

- **Reiniciar** la transacción, pero sólo si la transacción se ha abortado a causa de algún error hardware o software que no lo haya provocado la lógica interna de la transacción. Una transacción reiniciada se considera una nueva transacción.
- **Cancelar** la transacción. Normalmente se hace esto si hay algún error interno lógico que sólo se puede corregir escribiendo de nuevo el programa de aplicación, o debido a una entrada incorrecta o a que no se han encontrado los datos deseados en la base de datos.

Hay que tener cuidado cuando se trabaja con **escrituras externas observables**, como en un terminal o en una impresora. Cuando una escritura así tiene lugar, no puede borrarse puesto que puede haber sido vista fuera del sistema de base de datos. Muchos sistemas permiten que tales escrituras tengan lugar sólo después de que la transacción llegue al estado comprometida. Una manera de implementar dicho esquema es hacer que el sistema de base de datos almacene temporalmente cualquier valor asociado con estas escrituras externas en memoria no volátil, y realice las escrituras actuales sólo si la transacción llega al estado comprometida. Si el sistema falla después de que la transacción llegue al estado comprometida, pero antes de que finalicen las escrituras externas, el sistema de base de datos puede llevar a cabo dichas escrituras externas (usando los datos de la memoria no volátil) cuando el sistema se reinicie.

La gestión de las escrituras externas puede ser más complicada en ciertas situaciones. Por ejemplo, supóngase que la acción externa es dispensar dinero en un cajero automático y el sistema falla justo antes de que se dispense el dinero (se asume que el dinero se dispensa atómicamente). No tiene sentido dispensar el dinero cuando se reinicie el sistema, ya que el usuario probablemente no esté en el cajero. En tal caso es necesario una transacción compensadora, como devolver el dinero a la cuenta del usuario.

Para algunas aplicaciones puede ser deseable permitir a las transacciones activas que muestren datos a los usuarios, particularmente para transacciones de larga duración que se ejecutan durante minutos u horas. Desafortunadamente no se puede permitir dicha salida de datos observables a no ser que se quiera arriesgar la atomicidad de la transacción. Muchos sistemas de bases de datos actuales aseguran la atomicidad y, por tanto, prohíben esta forma de interacción con el usuario. En el Capítulo 25 se describen modelos alternativos que proporcionan transacciones interactivas de larga duración.

www.detodoprogramacion.com

15.3 Implementación de la atomicidad y la durabilidad

El componente de gestión de recuperaciones de un sistema de base de datos implementa el soporte para la atomicidad y durabilidad. En primer lugar consideramos un esquema simple pero extremadamente ineficiente, denominado **copia en la sombra**. Este esquema, que se basa en hacer copias de la base de datos, denominadas copias *sombra*, asume que sólo una transacción está activa en cada momento. El esquema asume que la base de datos es simplemente un archivo en disco. En disco se mantiene un puntero llamado puntero_bd que apunta a la copia actual de la base de datos.

En el esquema de copia en la sombra, una transacción que quiera actualizar una base de datos crea primero una copia completa de dicha base de datos. Todos los cambios se hacen en la nueva copia de la base de datos dejando la copia original, la **copia en la sombra**, inalterada. Si en cualquier punto hay que abortar la transacción, la copia nueva simplemente se borra. La copia antigua de la base de datos no se ve afectada.

Si la transacción se completa, se compromete como sigue. En primer lugar se consulta al sistema operativo para asegurarse de que todas las páginas de la nueva copia de la base de datos se han escrito en disco (en los sistemas Unix se usa el comando `fsync` para este propósito). Después de terminar este comando se actualiza el puntero puntero_bd para que apunte a la nueva copia de la base de datos; la nueva copia se convierte entonces en la copia de la base de datos actual. La copia antigua de la base de datos se borra después. El esquema se describe en la Figura 15.2, en la cual se muestra el estado de la base de datos antes y después de la actualización. Se dice que la transacción está *comprometida* en el momento en que puntero_bd actualizado se escribe en disco.

Considérese ahora la manera en que esta técnica trata los fallos de las transacciones y del sistema. En primer lugar, considérese un fallo en la transacción. Si la transacción falla en algún momento antes de actualizar puntero_bd, el contenido de la base de datos anterior no se ve afectado. Se puede abortar la transacción simplemente borrando la nueva copia de la base de datos. Una vez que se ha comprometido la transacción, puntero_bd apunta a todas las modificaciones que ésta ha realizado en la base de datos. De este modo, o todas las modificaciones de la transacción se ven reflejadas o ninguna de ellas, independientemente del fallo de la transacción.

Considérese ahora el resultado de un fallo en el sistema. Supóngase que el sistema falla en algún momento antes de escribir en disco el puntero_bd actualizado. Entonces cuando se reinicie el sistema, leerá puntero_bd y verá el contenido original de la base de datos, y ninguno de los efectos de la transacción será visible en la base de datos. A continuación supóngase que el sistema falla después de actualizar en disco puntero_bd. Antes de que el puntero se actualice, todas las páginas actualizadas de la nueva copia de la base de datos se escriben en disco. De nuevo, se asume que una vez que un archivo se escribe en disco, su contenido no se daña incluso si hay un fallo del sistema. Por tanto, cuando el sistema se reinicie, leerá puntero_bd y verá el contenido de la base de datos *después* de la transacción haya realizado todas las modificaciones.

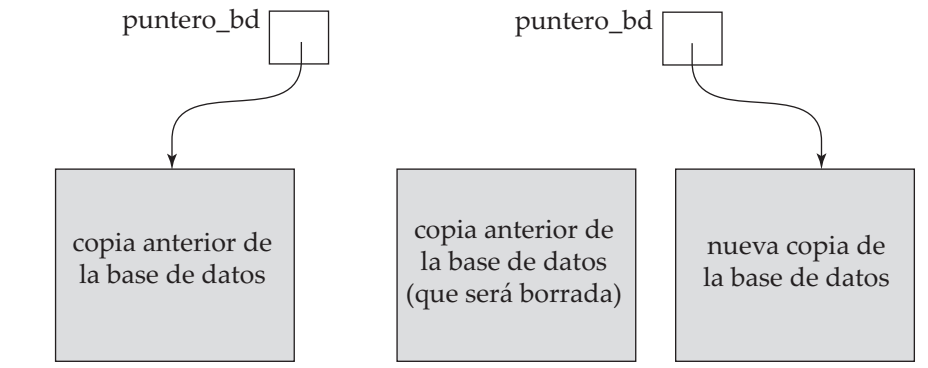


Figura 15.2 Técnica de copia en la sombra para la atomicidad y durabilidad.

www.detodoprogramacion.com

La implementación depende realmente de que escribir puntero_bd sea una operación atómica; es decir, o se escriben todos sus bytes o ninguno de ellos. Si se escribieran algunos de los bytes del puntero y otros no, el puntero no tendría sentido y al reiniciarse el sistema no se podrían encontrar ni la versión anterior ni la nueva de la base de datos. Afortunadamente los sistemas de disco proporcionan actualizaciones atómicas de bloques enteros, o al menos de un sector del disco. En otras palabras, el sistema de disco garantiza la actualización automática de puntero_bd siempre que se cumpla que puntero_bd quepa en un único sector, lo que se puede asegurar almacenándolo al comienzo de un bloque.

De este modo, la implementación de la copia en la sombra del componente de gestión de recuperaciones asegura las propiedades de atomicidad y durabilidad de las transacciones.

Como ejemplo simple fuera del dominio de las bases de datos, considérese una sesión de edición de texto. Una sesión completa de edición de texto puede modelar una transacción. Las acciones que ejecuta la transacción son leer y actualizar el archivo. Guardar el archivo cuando se termina de editar significa completar la transacción de edición; terminar la sesión de edición sin guardar el archivo significa abortar la transacción de edición.

Muchos editores de texto usan fundamentalmente la implementación que se acaba de describir para asegurar que una sesión de edición es como una transacción. Se usa un nuevo archivo para almacenar las modificaciones. Al finalizar la sesión de edición, si el archivo modificado se va a guardar, se usa un comando para *renombrar* el nuevo archivo para que tenga el nombre del archivo actual. Se asume que el renombramiento está implementado como una operación atómica en el sistema de archivos, y que al mismo tiempo borra el archivo antiguo.

Desafortunadamente, esta implementación es extremadamente ineficiente en el contexto de grandes bases de datos, ya que la ejecución de una simple transacción requiere copiar la base de datos *completa*. Además, la implementación no permite a las transacciones ejecutarse concurrentemente unas con otras. Existen formas prácticas de implementar la atomicidad y durabilidad que son mucho menos costosas y más potentes. Estas técnicas se estudian en el Capítulo 17.

15.4 Ejecuciones concurrentes

Los sistemas de procesamiento de transacciones permiten normalmente la ejecución de varias transacciones concurrentemente. Permitir varias transacciones que actualizan concurrentemente los datos provoca complicaciones en la consistencia de los mismos, como se ha visto antes. Asegurar la consistencia a pesar de la ejecución concurrente de las transacciones requiere un trabajo extra; es mucho más sencillo exigir que las transacciones se ejecuten **secuencialmente**—es decir, una a una, comenzado cada una sólo después de que la anterior se haya completado. Sin embargo, existen dos buenas razones para permitir la concurrencia:

- **Productividad y utilización de recursos mejoradas**. Una transacción consiste en varios pasos. Algunos implican operaciones de E/S; otros implican operaciones de CPU. La CPU y los discos pueden trabajar en paralelo en una computadora. Por tanto, las operaciones de E/S se pueden realizar en paralelo con el procesamiento de la CPU. Se puede entonces explotar el paralelismo de la CPU y del sistema de E/S para ejecutar varias transacciones en paralelo. Mientras una transacción ejecuta una lectura o una escritura en un disco, otra puede ejecutarse en la CPU mientras una tercera transacción ejecuta una lectura o una escritura en otro disco. Todo esto incrementa la **productividad** (*throughput*) del sistema—es decir, en el número de transacciones que puede ejecutar en un tiempo dado. Análogamente, la **utilización** del procesador y del disco aumenta también; en otras palabras, el procesador y el disco están menos tiempo desocupados o sin hacer ningún trabajo útil.

- **Tiempo de espera reducido**. Debe haber una mezcla de transacciones que se ejecutan en el sistema, algunas cortas y otras largas. Si las transacciones se ejecutan secuencialmente, la transacción corta debe esperar a que la transacción larga anterior se complete, lo cual puede llevar a un retardo impredecible en la ejecución de la transacción. Si las transacciones operan en partes diferentes de la base de datos es mejor hacer que se ejecuten concurrentemente, compartiendo los ciclos de la CPU y los accesos a disco entre ambas. La ejecución concurrente reduce los retardos im-

www.detodoprogramacion.com

predecibles en la ejecución de las transacciones. Además se reduce también el **tiempo medio de respuesta**: el tiempo medio desde que una transacción comienza hasta que se completa.

La razón para usar la ejecución concurrente en una base de datos es esencialmente la misma que para usar **multiprogramación** en un sistema operativo.

Cuando se ejecutan varias transacciones concurrentemente, la consistencia de la base de datos se puede destruir a pesar de que cada transacción individual sea correcta. En este apartado se presenta el concepto de planificaciones que ayudan a identificar aquellas ejecuciones que garantizan que se asegura la consistencia.

El sistema de base de datos debe controlar la interacción entre las transacciones concurrentes para evitar que se destruya la consistencia de la base de datos. Esto se lleva a cabo a través de una serie de mecanismos denominados **esquemas de control de concurrencia**. En el Capítulo 16 se estudian los esquemas de control de concurrencia; por ahora nos centraremos en el concepto de ejecución concurrente correcta.

Considérese de nuevo el sistema bancario simplificado del Apartado 15.1, el cual tiene varias cuentas, y un conjunto de transacciones que acceden y modifican dichas cuentas. Sean T_1 y T_2 dos transacciones para transferir fondos de una cuenta a otra. La transacción T_1 transfiere 50 € de la cuenta A a la cuenta B y se define como sigue

T_1: leer(A);
$A := A - 50$;
escribir(A);
leer(B);
$B := B + 50$;
escribir(B).

La transacción T_2 transfiere el 10 por ciento del saldo de la cuenta A a la cuenta B, y se define

T_2: leer(A);
$temp := A * 0.1$;
$A := A - temp$;
escribir(A);
leer(B);
$B := B + temp$;
escribir(B).

Supóngase que los valores actuales de las cuentas A y B son 1.000 € y 2.000 € respectivamente. Supóngase que las dos transacciones se ejecutan de una en una en el orden T_1 seguida de T_2. Esta secuencia de ejecución se representa en la Figura 15.3. En esta figura la secuencia de pasos o instrucciones aparece en orden cronológico de arriba abajo, con las instrucciones de T_1 en la columna izquierda y las de T_2 en la derecha. Los valores finales de las cuentas A y B, después de que tenga lugar la ejecución de la Figura 15.3, son de 855 € y de 2.145 € respectivamente. De este modo, la suma total de saldo de las cuentas A y B—es decir, la suma $A + B$—se conserva tras la ejecución de ambas transacciones.

Análogamente, si las transacciones se ejecutan de una en una en el orden T_2 seguida de T_1, entonces la secuencia de ejecución es la de la Figura 15.4. De nuevo, como se esperaba, se conserva la suma $A + B$ y los valores finales de las cuentas A y B son de 850 € y de 2.150 € respectivamente.

Las secuencias de ejecución que se acaban de describir se denominan **planificaciones**. Representan el orden cronológico en el que se ejecutan las instrucciones en el sistema. Obviamente una planificación para un conjunto de transacciones debe consistir en todas las instrucciones de dichas transacciones, y debe conservar el orden en que aparecen las instrucciones en cada transacción individual. Por ejemplo, en la transacción T_1 la instrucción escribir(A) debe aparecer antes de la instrucción leer(B) en cualquier planificación válida. En los párrafos siguientes, planificación 1 se referirá a la primera secuencia de ejecución (T_1 seguida de T_2) y planificación 2 a la segunda secuencia de ejecución (T_2 seguida de T_1).

Estas planificaciones son **secuenciales**. Cada planificación secuencial consiste en una secuencia de instrucciones de varias transacciones, en la cual las instrucciones pertenecientes a una única transac-

www.detodoprogramacion.com

T_1	T_2
leer(A)	
A := A - 50	
escribir(A)	
leer(B)	
B := B + 50	
escribir(B)	
	leer(A)
	$temp := A * 0,1$
	A := A - $temp$
	escribir(A)
	leer(B)
	B := B + $temp$
	escribir(B)

Figura 15.3 Planificación 1—una planificación secuencial en la que T_2 sigue a T_1.

ción están juntas en dicha planificación. De este modo, para un conjunto de *n* transacciones existen *n*! planificaciones secuenciales válidas distintas.

Cuando el sistema de bases de datos ejecuta concurrentemente varias transacciones, la planificación correspondiente no tiene por qué ser secuencial. Si dos transacciones se ejecutan concurrentemente, el sistema operativo puede ejecutar una transacción durante un tiempo pequeño, luego realizar un cambio de contexto, ejecutar la segunda transacción durante un tiempo, cambiar de nuevo a la primera transacción durante un tiempo y así sucesivamente. Si hay muchas transacciones, todas ellas comparten el tiempo de la CPU.

Son posibles muchas secuencias de ejecución, puesto que varias instrucciones de ambas transacciones se pueden intercalar. En general no es posible predecir exactamente cuántas instrucciones se ejecutarán antes de que la CPU cambie a otra transacción. Así, el número de planificaciones posibles para un conjunto de *n* transacciones es mucho mayor que *n*!.

Volviendo al ejemplo anterior supóngase que las dos transacciones se ejecutan concurrentemente. Una posible planificación se muestra en la Figura 15.5. Una vez que la ejecución tiene lugar, se llega al mismo estado que cuando las transacciones se ejecutan secuencialmente en el orden T_1 seguida de T_2. La suma A + B se conserva igualmente.

No todas las ejecuciones concurrentes producen un estado correcto. Como ejemplo de esto considérese la planificación de la Figura 15.6. Después de ejecutarse esta planificación se llega a un estado cuyos valores finales de las cuentas A y B son de 950 € y de 2.100 € respectivamente. Este estado final es

T_1	T_2
	leer(A)
	$temp := A * 0,1$
	A := A - $temp$
	escribir(A)
	leer(B)
	B := B + $temp$
	escribir(B)
leer(A)	
A := A - 50	
escribir(A)	
leer(B)	
B := B + 50	
escribir(B)	

Figura 15.4 Planificación 2—una planificación secuencial en la cual T_1 sigue a T_2.

www.detodoprogramacion.com

T_1	T_2
leer(A)	
$A := A$ - 50	
escribir(A)	
	leer(A)
	$temp := A * 0,1$
	$A := A$ - $temp$
	escribir(A)
leer(B)	
$B := B$ + 50	
escribir(B)	
	leer(B)
	$B := B + temp$
	escribir(B)

Figura 15.5 Planificación 3—una planificación concurrente equivalente a la planificación 1.

un *estado inconsistente*, ya que se han ganado 50 € al procesar la ejecución concurrente. Realmente la ejecución de las dos transacciones no conserva la suma $A + B$.

Si se deja el control de la ejecución concurrente completamente al sistema operativo son posibles muchas planificaciones, incluyendo las que dejan a la base de datos en un estado inconsistente, como la que se acaba de describir. Es una tarea del sistema de base de datos asegurar que cualquier planificación que se ejecute lleva a la base de datos a un estado consistente. El componente del sistema de base de datos que realiza esta tarea se denomina **componente de control de concurrencia**.

Se puede asegurar la consistencia de la base de datos en una ejecución concurrente si se está seguro de que cualquier planificación que se ejecute tiene el mismo efecto que otra que se hubiese ejecutado sin concurrencia. Es decir, la planificación debe ser, en cierto modo, equivalente a una planificación secuencial. Esta idea se examina en el Apartado 15.5.

15.5 Secuencialidad

El sistema de base de datos debe controlar la ejecución concurrente de las transacciones para asegurar que el estado de la base sigue siendo consistente. Antes de examinar cómo debe realizar esta tarea el sistema de base de datos hay que entender primero las planificaciones que aseguran la consistencia y las que no.

T_1	T_2
leer(A)	
$A := A$ - 50	
	leer(A)
	$temp := A * 0,1$
	$A := A$ - $temp$
	escribir(A)
	leer(B)
escribir(A)	
leer(B)	
$B := B$ + 50	
escribir(B)	
	$B := B + temp$
	escribir(B)

Figura 15.6 Planificación 4—una planificación concurrente.

www.detodoprogramacion.com

Puesto que las transacciones son programas, es difícil calcular cuáles son las operaciones exactas que realiza una transacción y cómo interaccionan las operaciones de varias transacciones. Por este motivo no se van a interpretar los tipos de operaciones que puede realizar una transacción sobre un elemento de datos. En lugar de esto se consideran sólo dos operaciones: leer y escribir. Se asume así que entre una instrucción leer(Q) y otra escribir(Q) sobre un elemento de datos Q, una transacción puede realizar una secuencia arbitraria de operaciones con la copia Q que reside en la memoria intermedia local de dicha transacción. De este modo las únicas operaciones significativas de la transacción son, desde el punto de vista de la planificación, las instrucciones leer y escribir. Por tanto, normalmente sólo se mostrarán las instrucciones leer y escribir en las planificaciones, como se muestra en la planificación 3 de la Figura 15.7.

En este apartado se estudian diferentes formas de equivalencia de planificación; esto lleva a los conceptos de **secuencialidad en cuanto a conflictos** y **secuencialidad en cuanto a vistas**.

15.5.1 Secuencialidad en cuanto a conflictos

Considérese una planificación P en la cual hay dos instrucciones consecutivas I_i e I_j, pertenecientes a las transacciones T_i y T_j respectivamente ($i \neq j$). Si I_i e I_j se refieren a distintos elementos de datos se pueden intercambiar I_i e I_j sin afectar al resultado de cualquier instrucción de la planificación. Sin embargo, si I_i e I_j se refieren al mismo elemento Q entonces el orden de los dos pasos puede ser importante. Puesto que sólo se tienen en cuenta las instrucciones leer y escribir se deben considerar cuatro casos:

1. I_i = leer(Q), I_j = leer(Q). El orden de I_i e I_j no importa, puesto que T_i y T_j leen el mismo valor de Q, independientemente del orden.
2. I_i = leer(Q), I_j = escribir(Q). Si I_i está antes que I_j, entonces T_i no lee el valor de Q que escribe la instrucción I_j de T_j. Si I_j está antes que I_i, entonces T_i lee el valor de Q escrito por T_j. Por tanto, orden de I_i e I_j es importante.
3. I_i = escribir(Q), I_j = leer(Q). El orden de I_i e I_j es importante por razones similares a las del caso anterior.
4. I_i = escribir(Q), I_j = escribir(Q). Puesto que ambas instrucciones son operaciones escribir el orden de dichas instrucciones no afecta ni a T_i ni a T_j. Sin embargo, el valor que obtendrá la siguiente instrucción leer(Q) de P sí se ve afectado, ya que únicamente se conserva en la base de datos la última de las dos instrucciones escribir. Si no hay ninguna otra instrucción escribir(Q) después de I_i e I_j en P, entonces el orden de I_i e I_j afecta directamente al valor final de Q en el estado de la base de datos que se obtiene con la planificación P.

De esta manera sólo en el caso en el cual I_i e Ij son instrucciones leer no tiene importancia el orden de ejecución de las mismas.

Se dice que I_i e I_j están en **conflicto** si existen operaciones de diferentes transacciones sobre el mismo elemento de datos, y al menos una de esas instrucciones es una operación escribir.

T_1	T_2
leer(A)	
escribir(A)	
	leer(A)
	escribir(A)
leer(B)	
escribir(B)	
	leer(B)
	escribir(B)

El Mundo de la Programacion en tus Manos...
DETODOPROGRAMACION.COM

Figura 15.7 Planificación 3—sólo se muestran las instrucciones leer y escribir.

T_1	T_2
leer(A)	
escribir(A)	
	leer(A)
leer(B)	
	escribir(A)
escribir(B)	
	leer(B)
	escribir(B)

Figura 15.8 Planificación 5—planificación 3 después de intercambiar un par de instrucciones.

Para ilustrar el concepto de instrucciones conflictivas considérese la planificación 3 mostrada en la Figura 15.7. La instrucción escribir(A) de T_1 está en conflicto con la instrucción leer(A) de T_2. Sin embargo, la instrucción escribir(A) de T_2 no está en conflicto con la instrucción leer(B) de T_1, ya que las dos instrucciones acceden a diferentes elementos de datos.

Sean I_i e I_j instrucciones consecutivas de una planificación P. Si I_i e I_j son instrucciones de transacciones diferentes y además I_i e I_j no están en conflicto, entonces se puede cambiar el orden de I_i e I_j para obtener una nueva planificación P'. Lo esperado es que P sea equivalente a P', ya que todas las instrucciones aparecen en el mismo orden en ambas planificaciones salvo I_i e I_j, cuyo orden no es importante.

Puesto que la instrucción escribir(A) de T_2 en la planificación 3 de la Figura 15.7 no está en conflicto con la instrucción leer(B) de T_1, se pueden intercambiar dichas instrucciones para generar una planificación equivalente, la planificación 5, que se muestra en la Figura 15.8. Independientemente de cuál sea el estado inicial del sistema, las planificaciones 3 y 5 producen el mismo estado final del sistema.

Se puede continuar intercambiando instrucciones no conflictivas como sigue:

- Intercambiar la instrucción leer(B) de T_1 con la instrucción leer(A) de T_2.
- Intercambiar la instrucción escribir(B) de T_1 con la instrucción escribir(A) de T_2.
- Intercambiar la instrucción escribir(B) de T_1 con la instrucción leer(A) de T_2.

El resultado final de estos intercambios, como se muestra en la Figura 15.9, es una planificación secuencial. Así se muestra que la planificación 3 es equivalente a una planificación secuencial. Esta equivalencia implica que independientemente del estado inicial, la planificación 3 produce el mismo estado final que una planificación secuencial.

Si una planificación P se puede transformar en otra P' por medio de una serie de intercambios de instrucciones no conflictivas, se dice que P y P' son **equivalentes en cuanto a conflictos**.

Volviendo a los ejemplos anteriores, se observa que la planificación 1 no es equivalente en cuanto a conflictos a la planificación 2. Sin embargo, la planificación 1 es equivalente en cuanto a conflictos a la

T_1	T_2
leer(A)	
escribir(A)	
leer(B)	
escribir(B)	
	leer(A)
	escribir(A)
	leer(B)
	escribir(B)

Figura 15.9 Planificación 6—una planificación secuencial equivalente a la planificación 3.

www.detodoprogramacion.com

T_3	T_4
leer(Q)	
	escribir(Q)
escribir(Q)	

Figura 15.10 Planificación 7.

planificación 3, puesto que las instrucciones leer(B) y escribir(B) de T_1 se pueden intercambiar con las instrucciones leer(A) y escribir(A) de T_2.

El concepto de equivalencia en cuanto a conflictos lleva al concepto de secuencialidad en cuanto a conflictos. Se dice que una planificación P es **secuenciable en cuanto a conflictos** si es equivalente en cuanto a conflictos a una planificación secuencial. Así, la planificación 3 es secuenciable en cuanto a conflictos, ya que es equivalente en cuanto a conflictos a la planificación secuencial 1.

Finalmente considérese la planificación 7 de la Figura 15.10; sólo consta de las operaciones significativas de las transacciones T_3 y T_4 (es decir, leer y escribir). Esta planificación no es secuenciable en cuanto a conflictos, ya que no es equivalente ni a la planificación secuencial $<T_3,T_4>$ ni a $<T_4,T_3>$.

Es posible encontrar dos planificaciones que produzcan el mismo resultado y que no sean equivalentes en cuanto a conflictos. Por ejemplo, considérese la transacción T_5, la cual transfiere 10 € de la cuenta B a la A. Sea la planificación 8 la que se define en la Figura 15.11. Se puede afirmar que la planificación 8 no es equivalente en cuanto a conflictos a la planificación secuencial $<T_1,T_5>$, ya que en la planificación 8 la instrucción escribir(B) de T_5 está en conflicto con la instrucción leer(B) de T_1. Por este motivo no se pueden colocar todas las instrucciones de T_1 antes de las de T_5 intercambiando instrucciones no conflictivas consecutivas. Sin embargo, los valores finales de las cuentas A y B son los mismos después de ejecutar tanto la planificación 8 como la planificación secuencial $<T_1,T_5>$—esto es, 960 € y 2.040 € respectivamente.

Con este ejemplo se puede observar que existen definiciones de equivalencia de planificaciones que son menos rigurosas que la de equivalencia en cuanto a conflictos. Para que el sistema pueda determinar que el resultado de la planificación sea el mismo que el de la planificación secuencial $<T_1,T_5>$, debe analizar los cálculos realizados por T_1 y T_5, en lugar de tener sólo en cuenta las operaciones leer y escribir. En general es difícil de implementar tal análisis y es costoso en términos de cómputo. Sin embargo, existen otras definiciones de equivalencia de planificación que se basan únicamente en las operaciones leer y escribir. En el siguiente apartado se considera una de estas definiciones.

15.5.2 Secuencialidad en cuanto a vistas**

En este apartado se va a considerar una forma de equivalencia que es menos rigurosa que la equivalencia en cuanto a conflictos pero que, al igual que ésta, se basa únicamente en las operaciones leer y escribir de las transacciones.

T_1	T_5
leer(A)	
$A := A$ - 50	
escribir(A)	
	leer(B)
	$B := B$ - 10
	escribir(B)
leer(B)	
$B := B$ + 50	
escribir(B)	
	leer(A)
	$A := A$ + 10
	escribir(A)

Figura 15.11 Planificación 8.

www.detodoprogramacion.com

Considérense dos planificaciones, P y P', en las cuales participa el mismo conjunto de transacciones. Se dice que las planificaciones P y P' son **equivalentes en cuanto a vistas** si se cumplen las tres condiciones siguientes:

1. Para todo elemento de datos Q, si la transacción T_i lee el valor inicial de Q en la planificación P, entonces T_i debe leer también el valor inicial de Q en la planificación P'.
2. Para todo elemento de datos Q, si la transacción T_i ejecuta leer(Q) en la planificación P y el valor lo ha producido la transacción T_j (si existe dicha transacción) entonces, en la planificación P', la transacción T_i debe leer también el valor de Q que haya producido la transacción T_j.
3. Para todo elemento de datos Q, la transacción (si existe) que realice la última operación escribir(Q) en la planificación P, debe realizar la última operación escribir(Q) en la planificación P'.

Las condiciones 1 y 2 aseguran que cada transacción lee los mismos valores en ambas planificaciones y por tanto realizan los mismos cálculos. La condición 3, junto con las condiciones 1 y 2, asegura que ambas planificaciones producen como resultado el mismo estado final del sistema.

Volviendo a los ejemplos anteriores, obsérvese que la planificación 1 no es equivalente en cuanto a vistas a la planificación 2, ya que en la planificación 1 el valor de la cuenta A que lee la transacción T_2 lo produce T_1, mientras que esto no ocurre en la planificación 2. Sin embargo, la planificación 1 es equivalente en cuanto a vistas a la planificación 3, ya que los valores de las cuentas A y B que lee la transacción T_2 los produce T_1 en ambas planificaciones.

El concepto de equivalencia en cuanto a vistas lleva al concepto de secuencialidad en cuanto a vistas. Se dice que la planificación P es **secuenciable en cuanto a vistas** si es equivalente en cuanto a vistas a una planificación secuencial.

Como ejemplo supóngase que se aumenta la planificación 7 con la transacción T_6 y se obtiene la planificación 9, tal y como se muestra en la Figura 15.12. La planificación 9 es secuenciable en cuanto a vistas. Realmente es equivalente en cuanto a vistas a la planificación secuencial $<T_3, T_4, T_6>$, ya que la instrucción leer(Q) lee el valor inicial de Q en ambas planificaciones, y T_6 realiza la escritura final de Q en ambas planificaciones.

Toda planificación secuenciable en cuanto a conflictos es secuenciable en cuanto a vistas, pero existen planificaciones secuenciables en cuanto a vistas que no son secuenciables en cuanto a conflictos. Realmente, la planificación 9 no es secuenciable en cuanto a conflictos puesto que para todo par de instrucciones, éstas están en conflicto y por tanto no es posible ningún intercambio de instrucciones.

Obsérvese que en la planificación 9, las transacciones T_4 y T_6 realizan operaciones escribir(Q) sin haber realizado ninguna operación leer(Q). Este tipo de escrituras se denominan **escrituras a ciegas**. Las escrituras a ciegas aparecen en toda planificación secuenciable en cuanto a vistas que no sea secuenciable en cuanto a conflictos.

15.6 Recuperabilidad

Antes se han estudiado las planificaciones que son aceptables desde el punto de vista de la consistencia de la base de datos asumiendo implícitamente que no había fallos en las transacciones. Ahora se va a estudiar el efecto de los fallos en una transacción durante una ejecución concurrente.

Si la transacción T_i falla, por la razón que sea, es necesario deshacer el efecto de dicha transacción para asegurar la propiedad de atomicidad de la misma. En un sistema que permita la concurrencia es necesario asegurar también que toda transacción T_j que dependa de T_i (es decir, T_j lee datos que ha

T_3	T_4	T_6
leer(Q)		
	escribir(Q)	
escribir(Q)		
		escribir(Q)

Figura 15.12 Planificación 9—una planificación secuenciable en cuanto a vistas.

www.detodoprogramacion.com

T_8	T_9
leer(A)	
escribir(A)	
	leer(A)
leer(B)	

Figura 15.13 Planificación 10.

escrito T_i) se aborta también. Para alcanzar esta garantía, es necesario poner restricciones al tipo de planificaciones permitidas en el sistema.

En los dos subapartados siguientes se estudian las planificaciones que son aceptables desde el punto de vista que se ha descrito. Como ya se dijo antes, en el Capítulo 16 se describe la manera de asegurar que sólo se generan dichas planificaciones aceptables.

15.6.1 Planificaciones recuperables

Considérese la planificación 10 que se muestra en la Figura 15.13, en la cual la transacción T_9 realiza sólo una instrucción: leer(A). Supóngase que el sistema permite que T_9 se complete inmediatamente después de ejecutar la instrucción leer(A). Así se completa T_9 antes de que lo haga T_8. Supóngase ahora que T_8 falla antes de completarse. Puesto que T_9 ha leído el valor del elemento de datos A escrito por T_8, se debe abortar T_9 para asegurar la atomicidad de la transacción. Sin embargo, T_9 ya se ha comprometido y no puede abortarse. De este modo se llega a una situación en la cual es imposible recuperarse correctamente del fallo de T_8.

La planificación 10, cuyo compromiso tiene lugar inmediatamente después de ejecutar la instrucción leer(A), es un ejemplo de planificación *no recuperable*, la cual no debe permitirse. La mayoría de los sistemas de bases de datos requieren que todas las planificaciones sean *recuperables*. Una **planificación recuperable** es aquélla en la que para todo par de transacciones T_i y T_j tales que T_j lee elementos de datos que ha escrito previamente T_i, la operación comprometer de T_i aparece antes que la de T_j.

15.6.2 Planificaciones sin cascada

Incluso si una planificación es recuperable, hay que retroceder varias transacciones para recuperar correctamente el estado previo a un fallo en una transacción T_i. Tales situaciones ocurren si las transacciones leen datos que ha escrito T_i. Como ejemplo considérese la planificación parcial de la Figura 15.14. La transacción T_{10} escribe un valor de A que lee la transacción T_{11}. La transacción T_{11} escribe un valor de A que lee la transacción T_{12}. Supóngase que en ese momento falla T_{10}. Se debe retroceder T_{10}. Puesto que T_{11} depende de T_{10}, se debe retroceder T_{11}. Puesto que T_{12} depende de T_{11}, se debe retroceder T_{12}. Este fenómeno en el cual un fallo en una única transacción provoca una serie de retrocesos de transacciones se denomina **retroceso en cascada**.

No es deseable el retroceso en cascada, ya que provoca un aumento significativo del trabajo necesario para deshacer cálculos. Es deseable restringir las planificaciones a aquéllas en las que no puedan ocurrir retrocesos en cascada. Tales planificaciones se denominan planificaciones *sin cascada*. Una **planificación sin cascada** es aquélla para la que todo par de transacciones T_i y T_j tales que T_j lee un elemento de

T_{10}	T_{11}	T_{12}
leer(A)		
leer(B)		
escribir(A)		
	leer(A)	
	escribir(A)	
		leer(A)

Figura 15.14 Planificación 11.

www.detodoprogramacion.com

datos que ha escrito previamente T_i, la operación comprometer de T_i aparece antes que la operación de lectura de T_j. Es sencillo comprobar que toda planificación sin cascada es también recuperable.

15.7 Implementación del aislamiento

Antes se han visto las propiedades que debe tener una planificación para dejar a la base de datos en un estado consistente y para permitir fallos en la transacción que se puedan manejar de una manera segura. En concreto, las planificaciones que son secuenciables en cuanto a conflictos o secuenciables en cuanto a vistas y sin cascada cumplen estos requisitos.

Existen varios **esquemas de control de concurrencia** que se pueden utilizar para asegurar que, incluso si se ejecutan concurrentemente muchas transacciones, sólo se generen planificaciones aceptables sin tener en cuenta la forma en que el sistema operativo comparte en el tiempo los recursos (tales como el tiempo de CPU) entre las transacciones.

Como ejemplo trivial de esquema de control de concurrencia considérese éste: una transacción realiza un **bloqueo** en la base de datos completa antes de comenzar y lo libera después de haberse comprometido. Mientras una transacción mantiene el bloqueo, no se permite que ninguna otra lo obtenga, y todas ellas deben esperar hasta que se libere el bloqueo. Como resultado de esta política de bloqueo, sólo se puede ejecutar una transacción cada vez. Por tanto, sólo se generan planificaciones secuenciales. Éstas son obviamente secuenciables y es sencillo probar que también son sin cascada.

Un esquema de control de concurrencia como éste produce un rendimiento pobre, ya que fuerza a que las transacciones esperen a que finalicen las precedentes para poder comenzar. En otras palabras, produce un grado de concurrencia pobre. Como se ha explicado en el Apartado 15.4, la ejecución concurrente presenta varios beneficios para el rendimiento.

El objetivo de los esquemas de control de concurrencia es proporcionar un elevado grado de concurrencia, al mismo tiempo que aseguran que todas las planificaciones que se generan son secuenciables en cuanto a conflictos o en cuanto a vistas y son sin cascada.

En el Capítulo 16 se estudian gran número de esquemas de control de concurrencia. Los esquemas adoptan diferentes compromisos en función del aumento de concurrencia que permiten y del aumento de coste en el que incurren. Algunos permiten que se generen planificaciones secuenciables en cuanto a conflictos; otros permiten que se generen planificaciones secuenciables en cuanto a vistas que no son secuenciables en cuanto a conflictos.

15.8 Comprobación de la secuencialidad

Cuando se diseñan esquemas de control de concurrencia es necesario que demostrar que las planificaciones que genera el esquema son secuenciables. Para lograrlo se debe entender primero la forma de determinar si, dada una planificación concreta P, es secuenciable.

A continuación se presenta un método simple y eficiente de determinar la secuencialidad en cuanto a conflictos de una planificación. Considérese una planificación P. Se construye un grafo dirigido, llamado **grafo de precedencia** para P. Este grafo consiste en un par $G = (V,A)$, siendo V un conjunto de vértices y A un conjunto de arcos. El conjunto de vértices consiste en todas las transacciones que participan en la planificación. El conjunto de arcos consiste en todos los arcos $T_i \rightarrow T_j$ para los cuales se dan una de las tres condiciones siguientes:

1. T_i ejecuta escribir(Q) antes de que T_j ejecute leer(Q).
2. T_i ejecuta leer(Q) antes de que T_j ejecute escribir(Q).
3. T_i ejecuta escribir(Q) antes de que T_j ejecute escribir(Q).

Si existe un arco $T_i \rightarrow T_j$ en el grafo de precedencia , entonces en toda planificación secuencial P' equivalente a P, T_i debe aparecer antes de T_j.

Por ejemplo, en la Figura 15.15a se muestra el grafo de precedencia de la planificación 1. Sólo contiene el arco $T_1 \rightarrow T_2$, puesto que todas las instrucciones de T_1 se ejecutan antes de que lo haga la primera de

www.detodoprogramacion.com



Figura 15.15 Grafo de precedencia para (a) la planificación 1 y (b) la planificación 2.

T_2. Análogamente, la Figura 15.15b muestra el grafo de precedencia de la planificación 2 con el único arco $T_2 \rightarrow T_1$, ya que todas las instrucciones de T_2 se ejecutan antes de que lo haga la primera de T_1.

El grafo de precedencia de la planificación 4 se representa en la Figura 15.16. Contiene el arco $T_1 \rightarrow T_2$ debido a que T_1 ejecuta leer(A) antes de que T_2 ejecute escribir(A). También contiene el arco $T_2 \rightarrow T_1$ debido a que T_2 ejecuta leer(B) antes de que T_1 ejecute escribir(B).

Si el grafo de precedencia de P tiene un ciclo, entonces la planificación P no es secuenciable en cuanto a conflictos. Si el grafo no contiene ciclos, entonces la planificación P es secuenciable en cuanto a conflictos.

El **orden de secuencialidad** se puede obtener a través de la **ordenación topológica**, la cual determina un orden lineal que consiste en el orden parcial del grafo de precedencia. En general se pueden obtener muchos órdenes lineales posibles a través de la ordenación topológica. Por ejemplo, el grafo de la Figura 15.17a tiene dos órdenes lineales aceptables, como se observa en las Figuras 15.17b y 15.17c.

Así, para probar la secuencialidad en cuanto a conflictos es necesario construir el grafo de precedencia e invocar a un algoritmo de detección de ciclos. Los algoritmos de detección de ciclos se pueden consultar en cualquier libro de texto sobre algoritmos. Los algoritmos de detección de ciclos, tales como los que se basan en la búsqueda primero en profundidad, requieren del orden de n^2 operaciones, siendo n el número de vértices del grafo (es decir, el número de transacciones). De este modo se consigue un esquema práctico para determinar la secuencialidad en cuanto a conflictos.

Volviendo a los ejemplos anteriores, obsérvese que los grafos de precedencia para las planificaciones 1 y 2 (Figura 15.15) no contienen ciclos. El grafo de precedencia para la planificación 4 (Figura 15.16) sin embargo, contiene ciclos, lo que indica que esta planificación no es secuenciable en cuanto a conflictos.

La comprobación de la secuencialidad en cuanto a vistas es más complicada. De hecho, se ha demostrado que el problema de determinar la secuencialidad en cuanto a vistas es *NP*-completo. Por tanto, seguramente no exista ningún algoritmo eficiente para comprobar la secuencialidad en cuanto a vistas.

Véanse la notas bibliográficas para consultar referencias sobre ello. Sin embargo, los esquemas de control de concurrencia aún pueden utilizar *condiciones suficientes* para la secuencialidad en cuanto a vistas. Es decir, si se cumplen las condiciones suficientes, la planificación es secuencialidad en cuanto a vistas, pero puede haber algunas planificaciones secuencialidad en cuanto a vistas que no satisfagan las condiciones suficientes.

15.9 Resumen

- Una *transacción* es una *unidad* de la ejecución de un programa que accede y posiblemente actualiza varios elementos de datos. Es fundamental comprender el concepto de transacción para entender e implementar las actualizaciones de los datos en una base de datos, de manera que las ejecuciones concurrentes y los fallos de varios tipos no den como resultado que la base de datos se vuelva inconsistente.
- Es necesario que las transacciones tengan las propiedades ACID: atomicidad, consistencia, aislamiento y durabilidad.

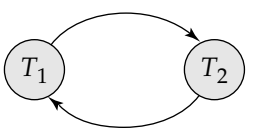


Figura 15.16 Grafo de precedencia para la planificación 4.

www.detodoprogramacion.com